

本节内容

最小生成树

知识总览

最小生成树

最小生成树的概念

Prim 算法

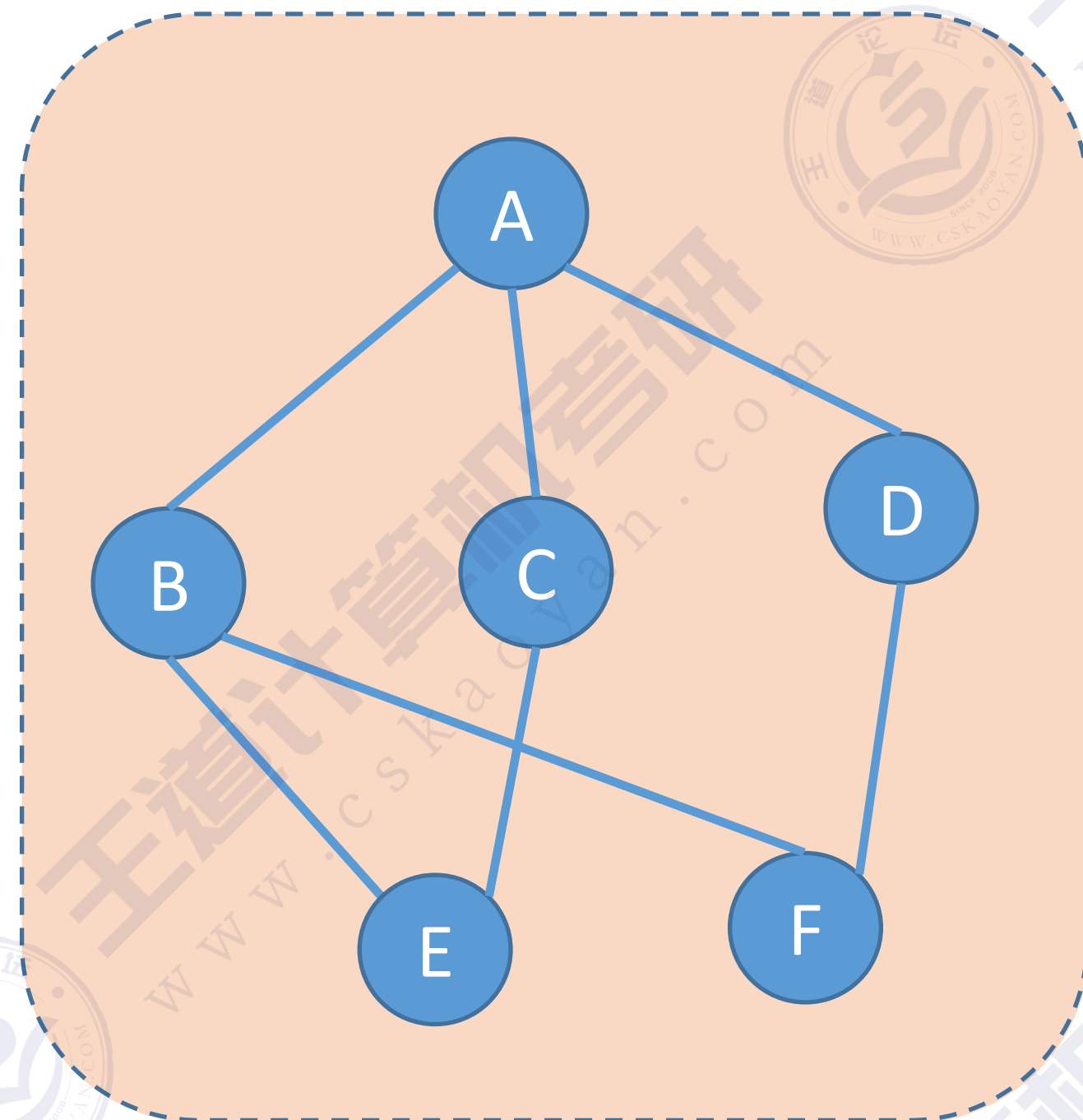
Kruskal 算法

生成树

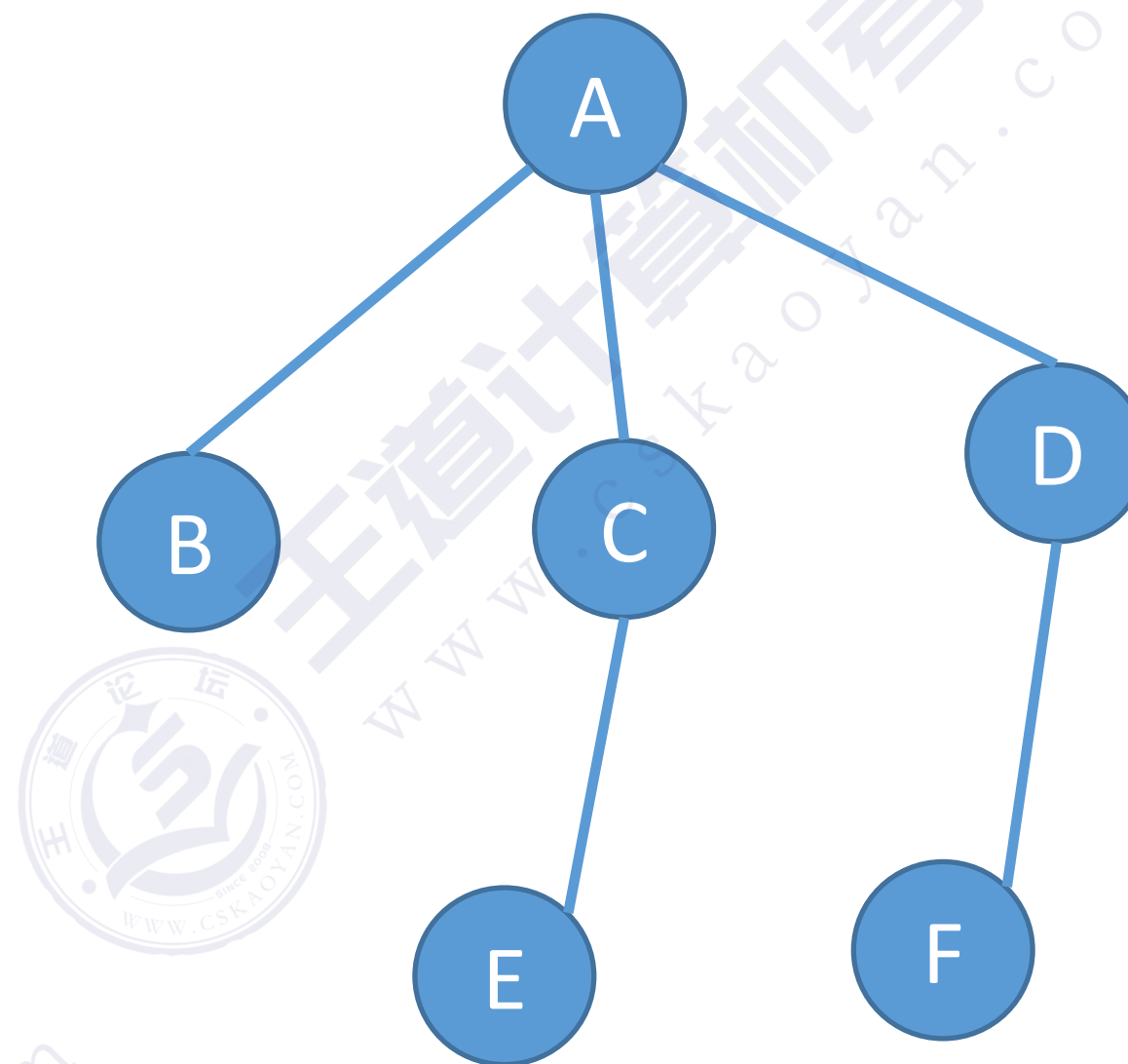
边尽可能的少，
但要保持连通

连通图的生成树是包含图中全部顶点的一个极小连通子图。

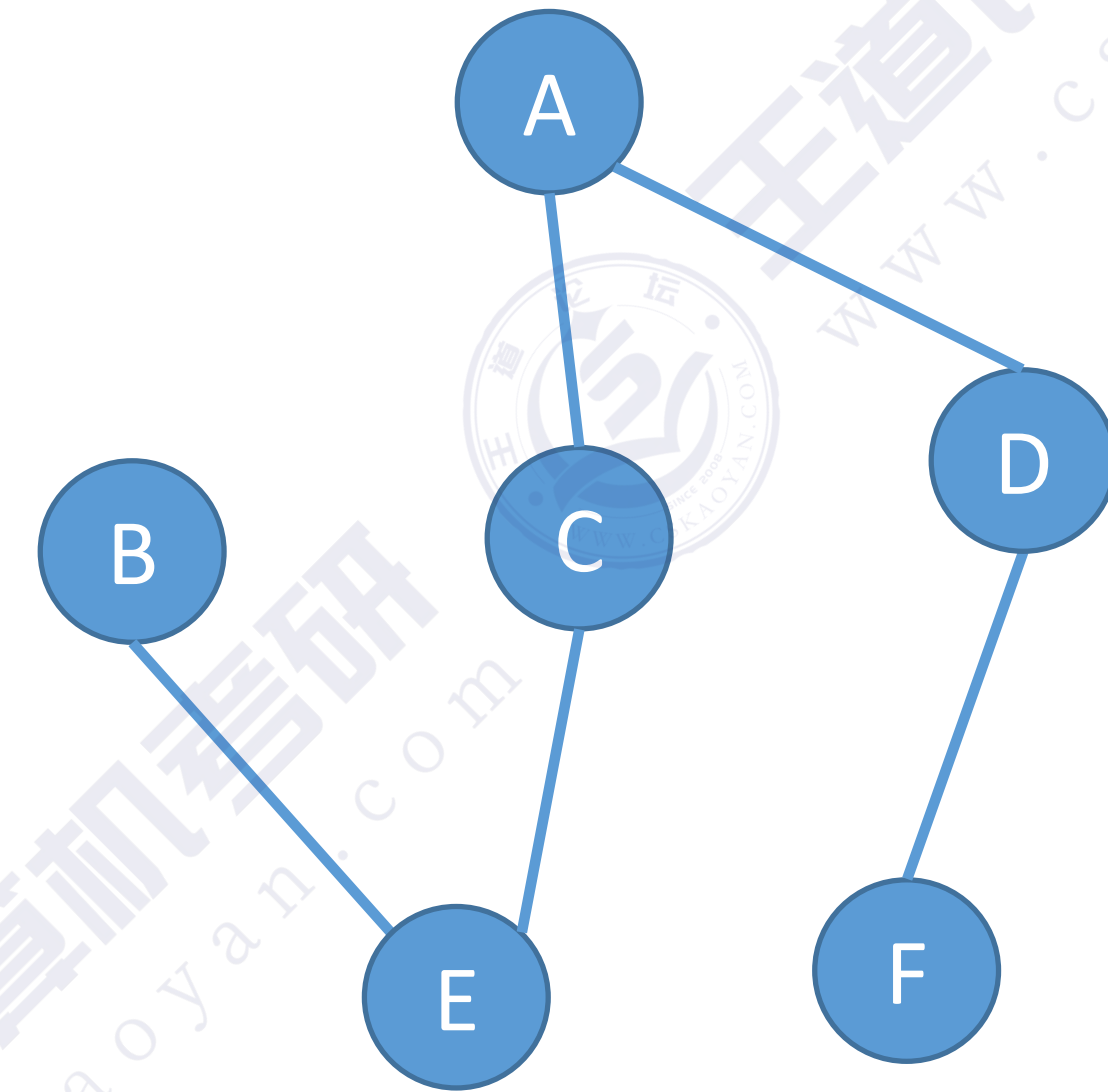
若图中顶点数为 n ，则它的生成树含有 $n-1$ 条边。对生成树而言，若砍去它的一条边，则会变成非连通图，若加上一条边则会形成一个回路。



无向图G



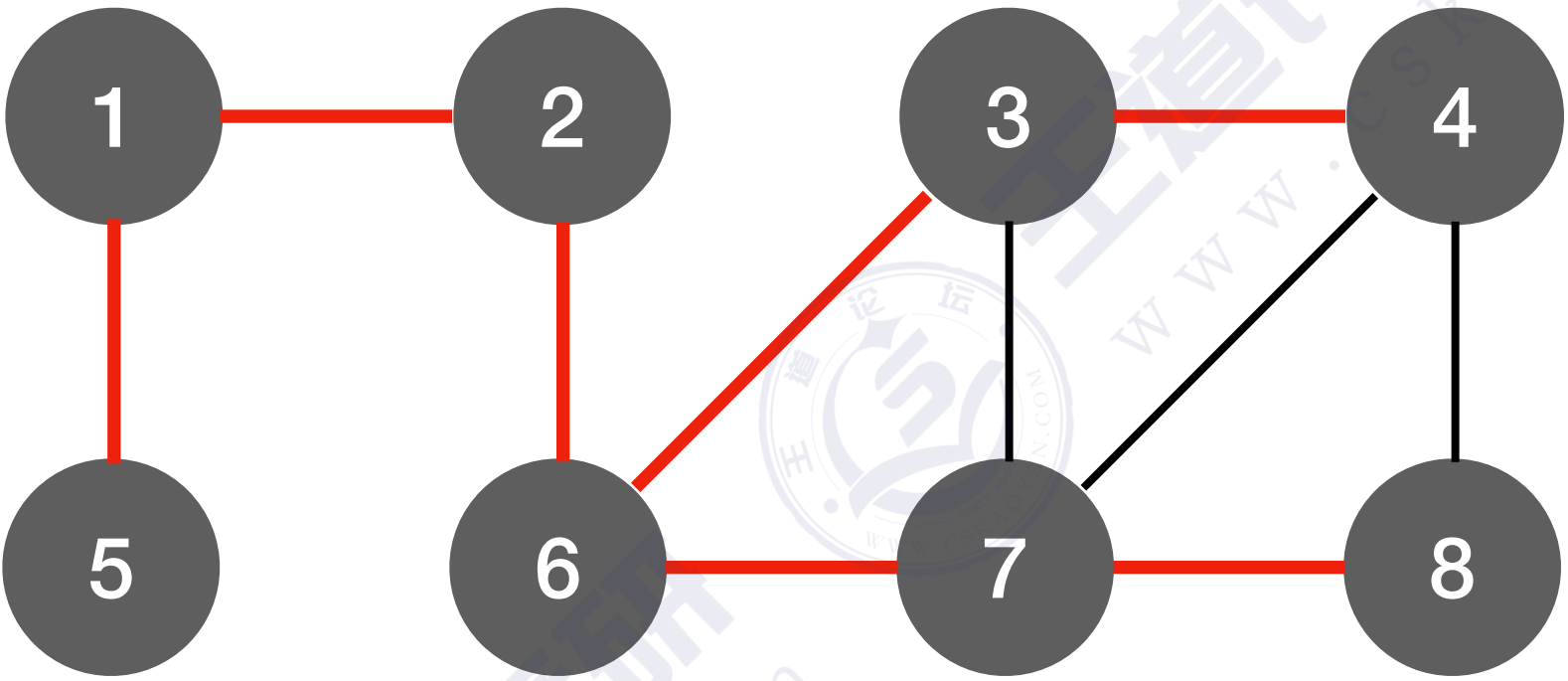
G的生成树1



G的生成树2

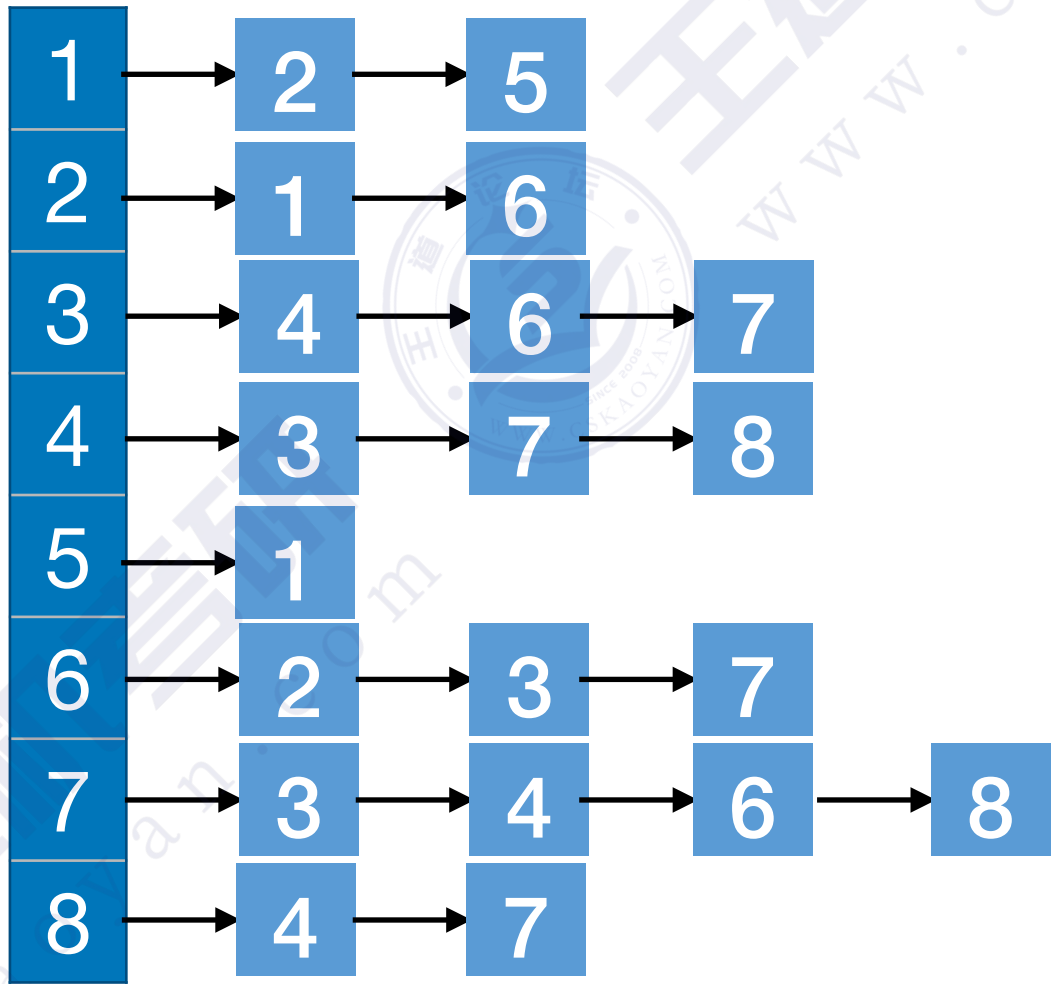
.....

广度优先生成树

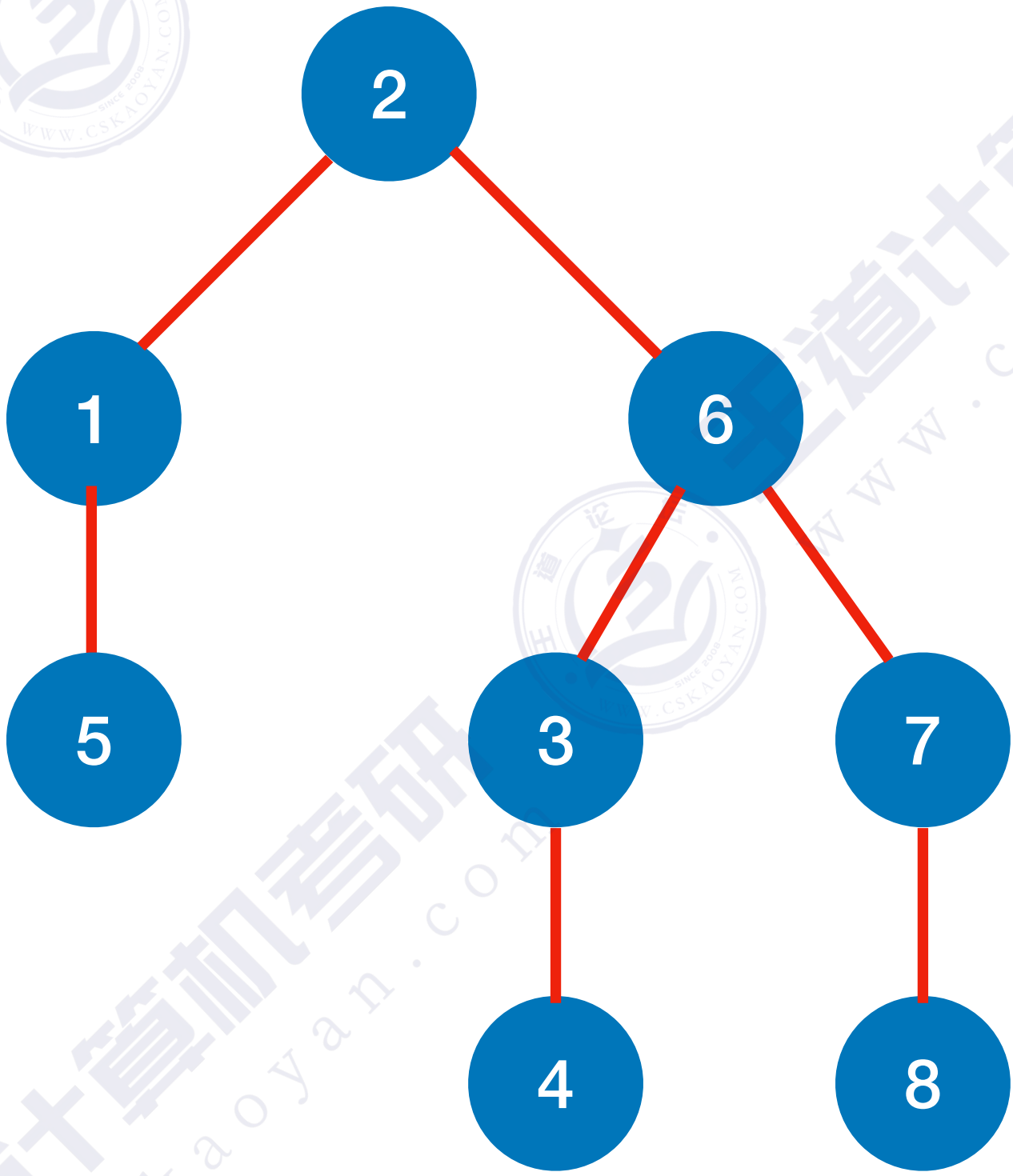


	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	0	1	0	0	1	0

邻接矩阵

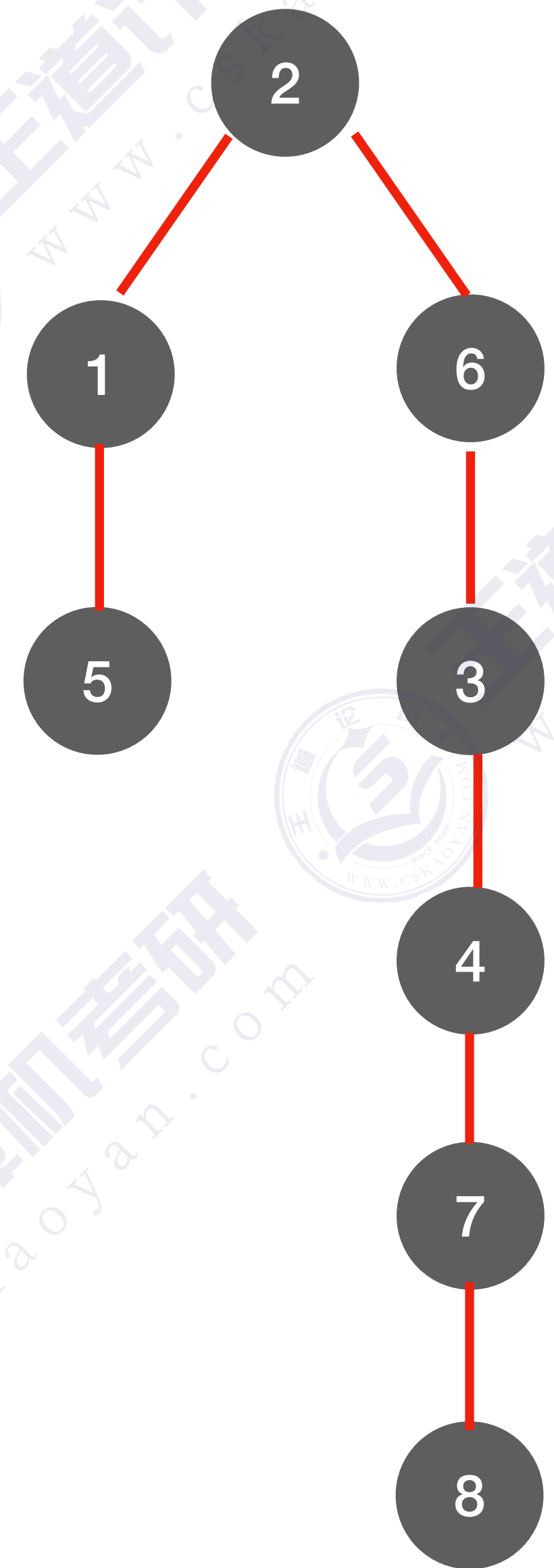
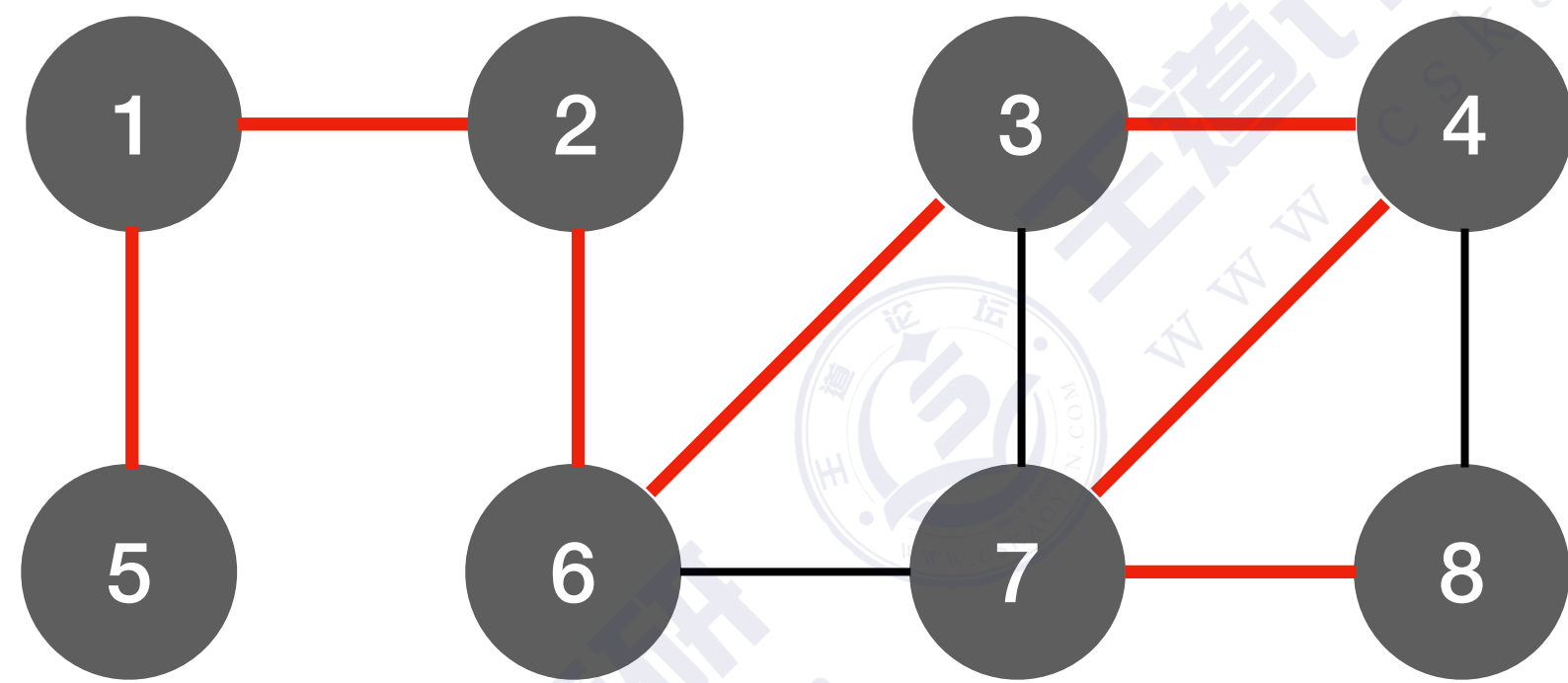


邻接表



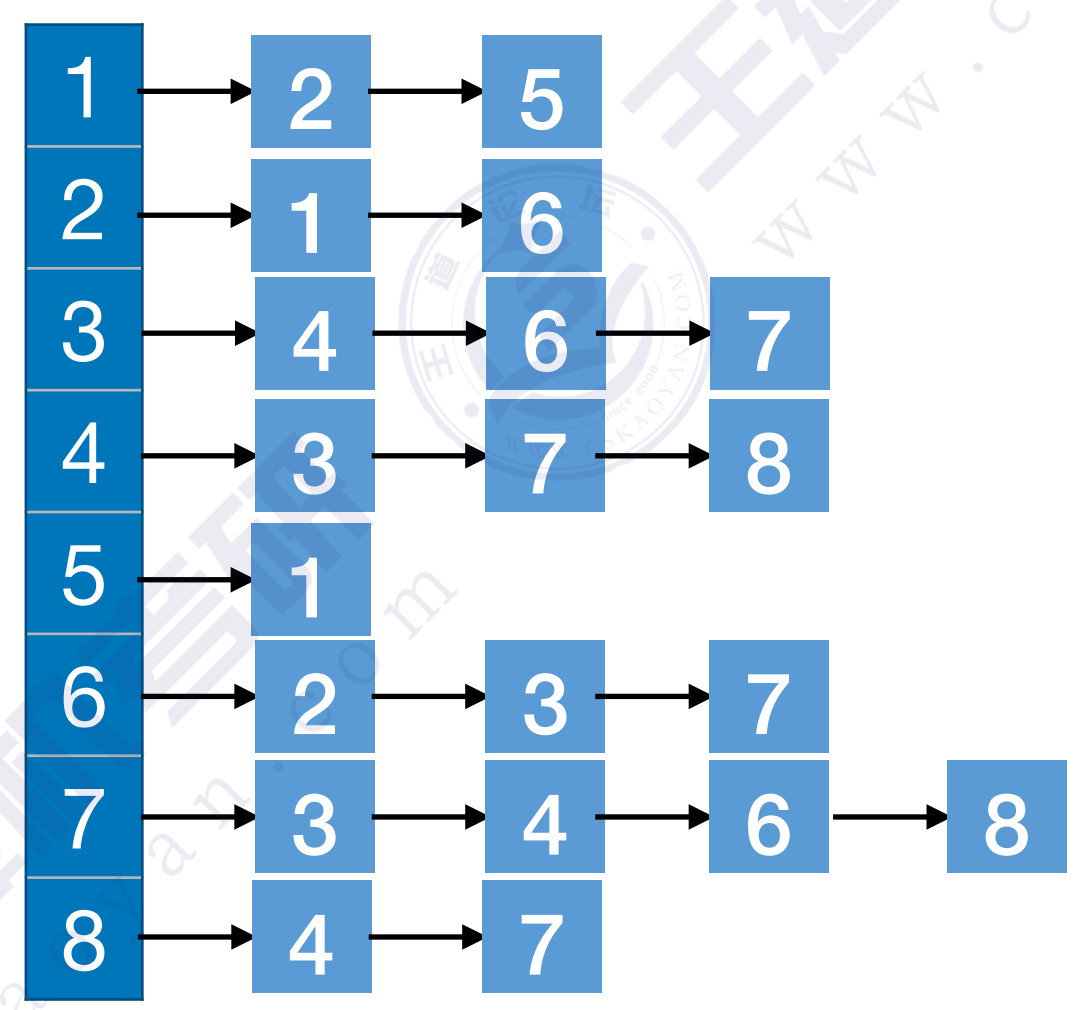
广度优先生成树

深度优先生成树



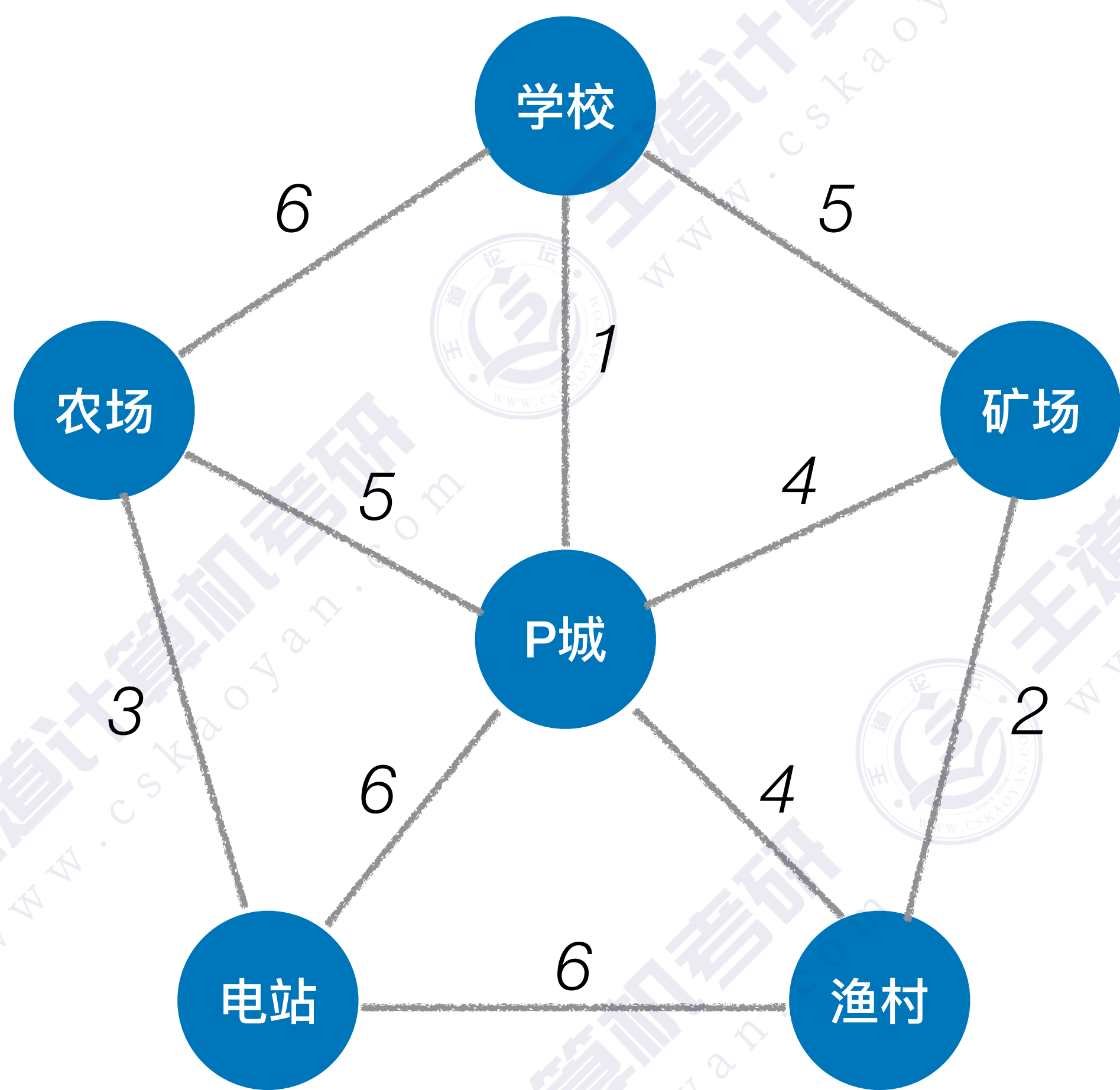
	1	2	3	4	5	6	7	8
1	0	1	0	0	1	0	0	0
2	1	0	0	0	0	1	0	0
3	0	0	0	1	0	1	1	0
4	0	0	1	0	0	0	1	1
5	1	0	0	0	0	0	0	0
6	0	1	1	0	0	0	1	0
7	0	0	1	1	0	1	0	1
8	0	0	0	1	0	0	1	0

邻接矩阵



邻接表

最小生成树（最小代价树）

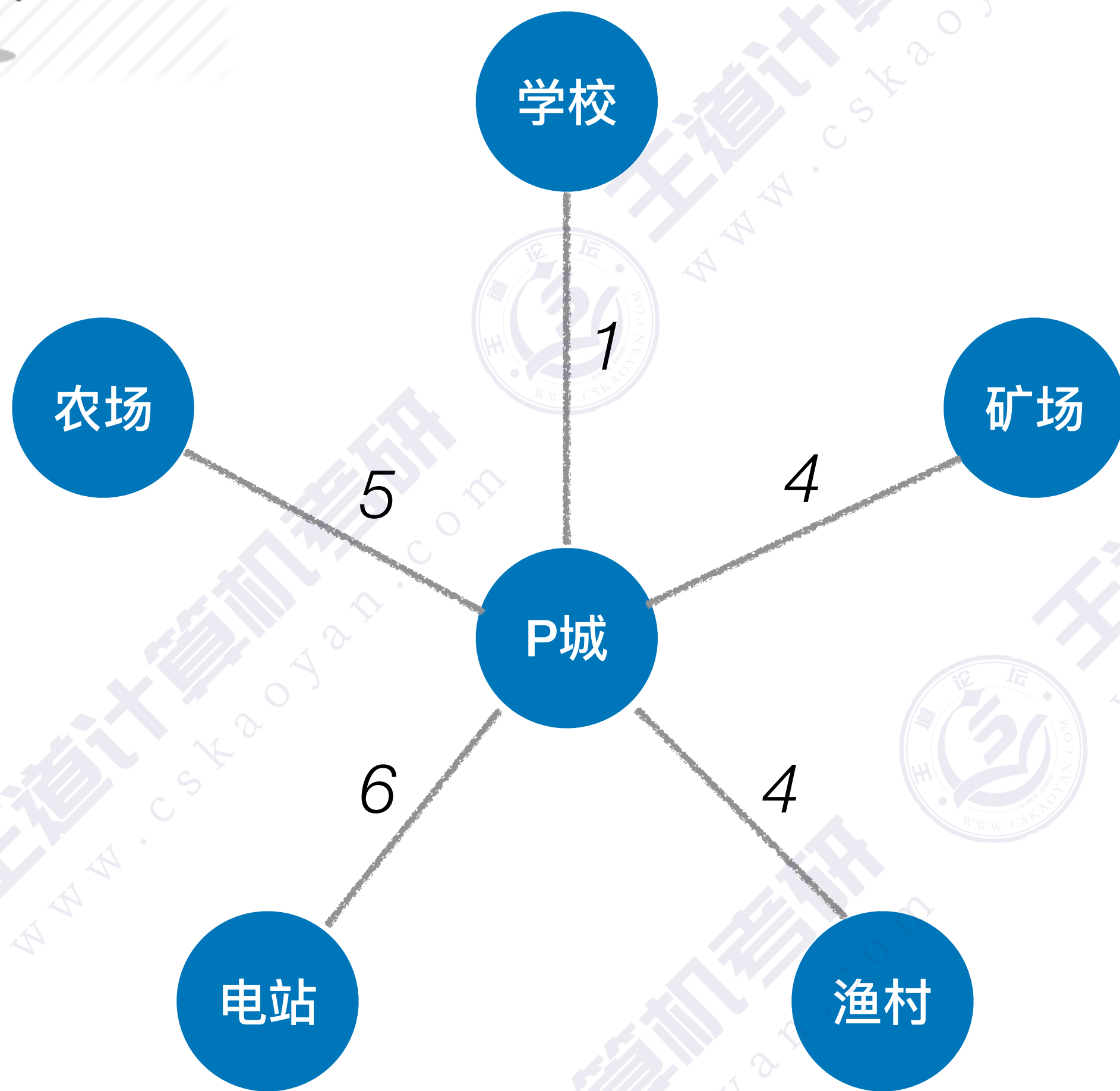


道路规划要求：所有地方都连通，且成本尽可能的低

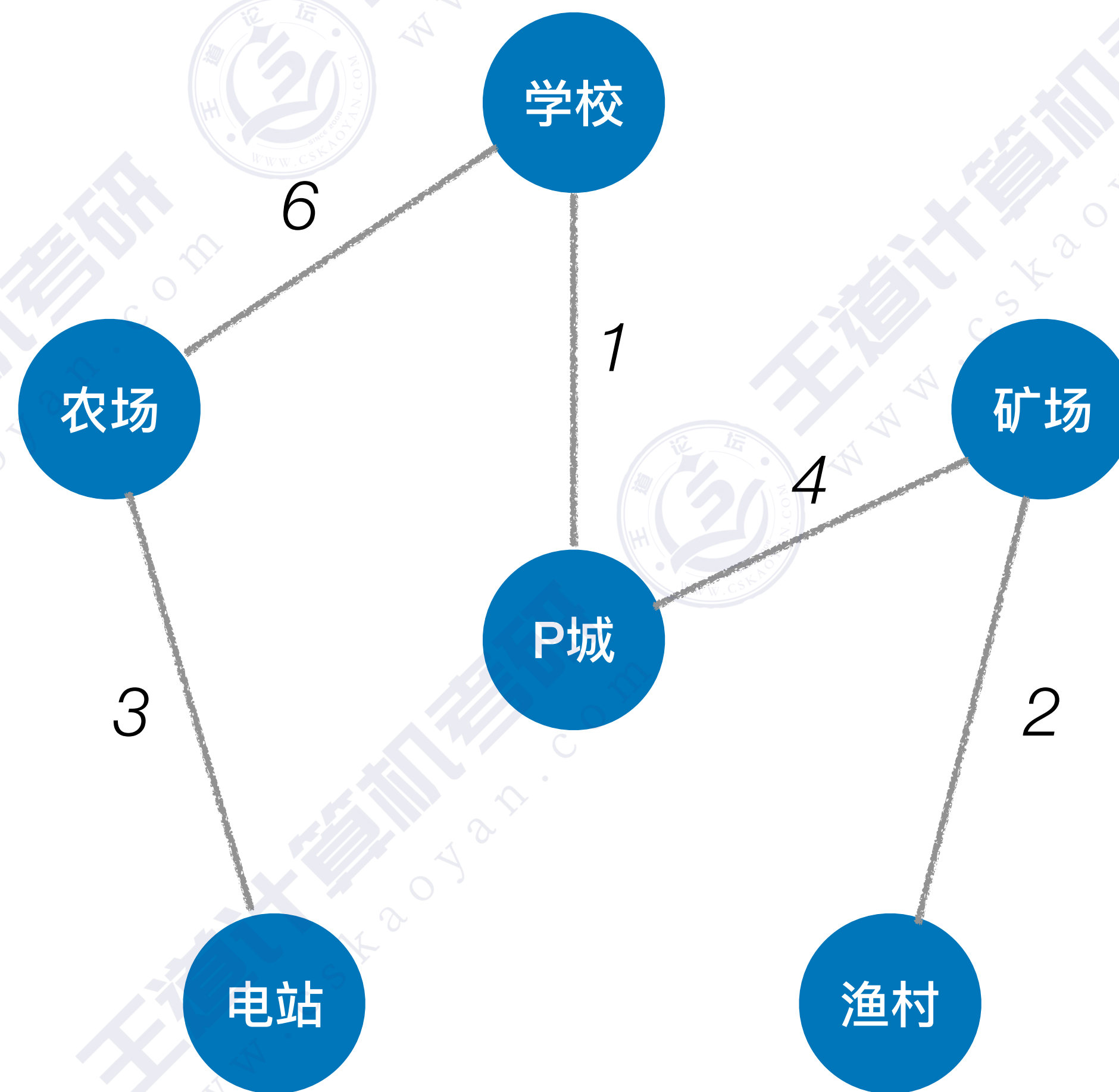


还有没有更便宜的
修路方案...?

最小生成树 (最小代价树)



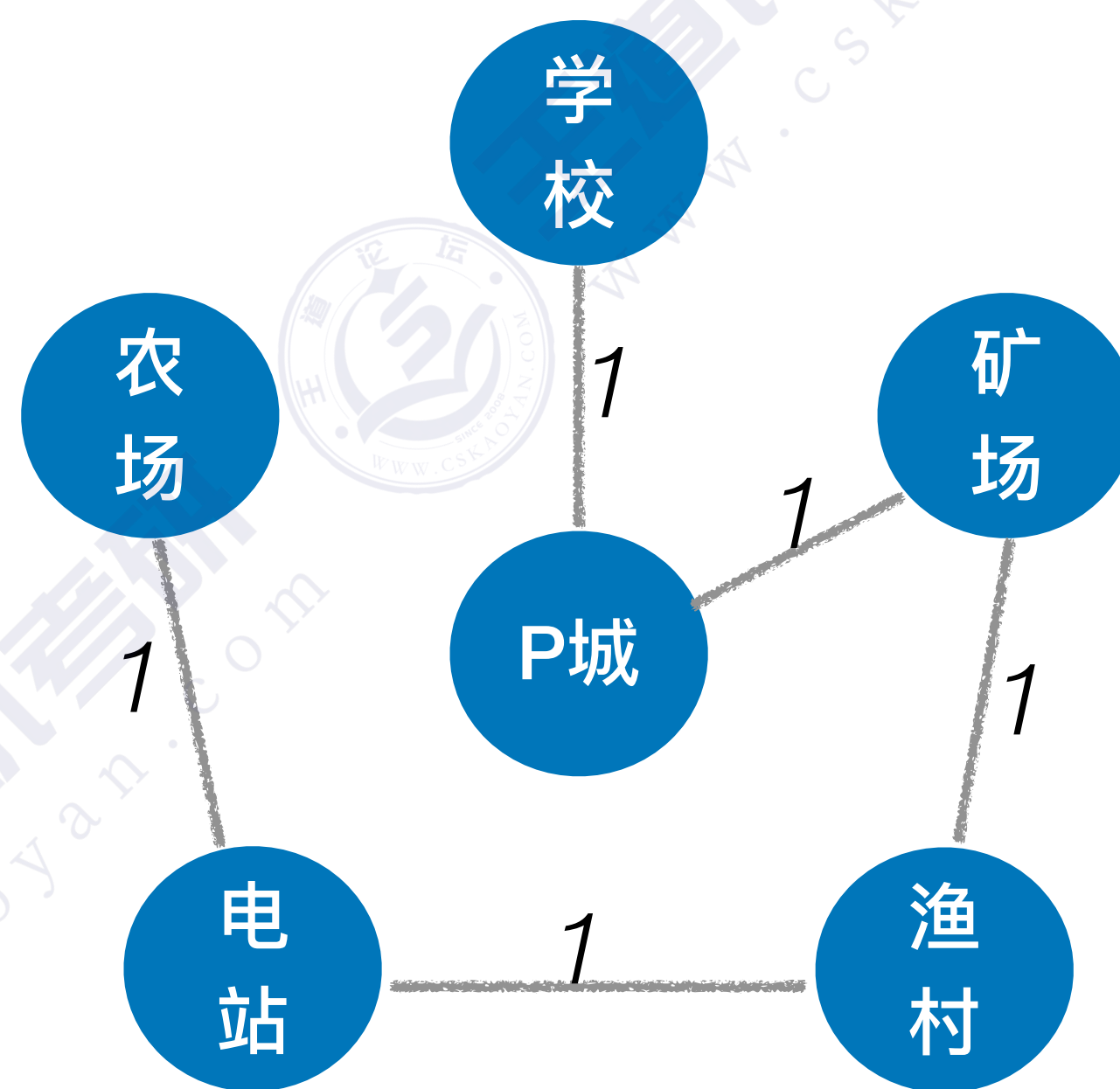
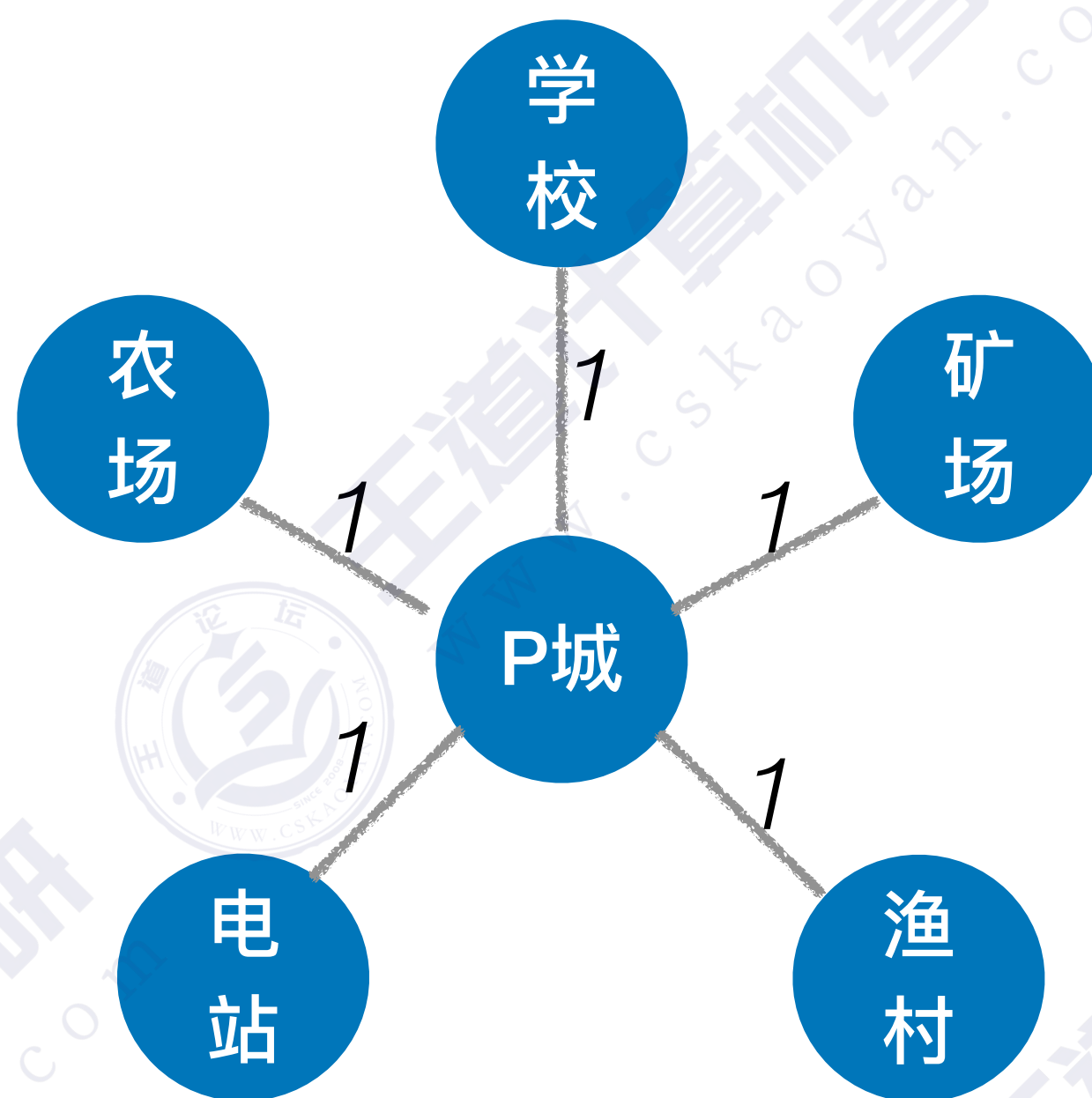
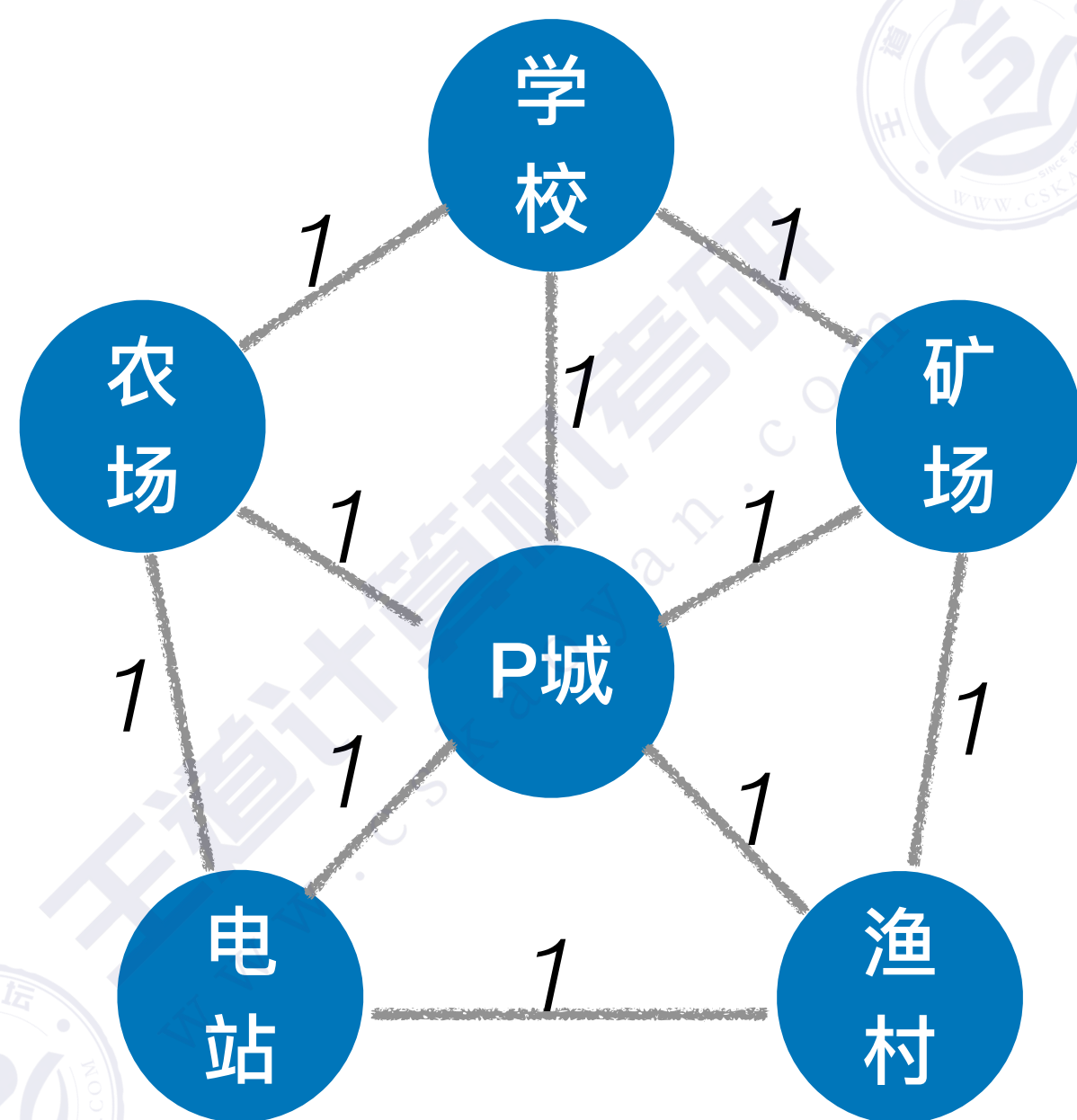
代价: 19



代价: 16

最小生成树（最小代价树）

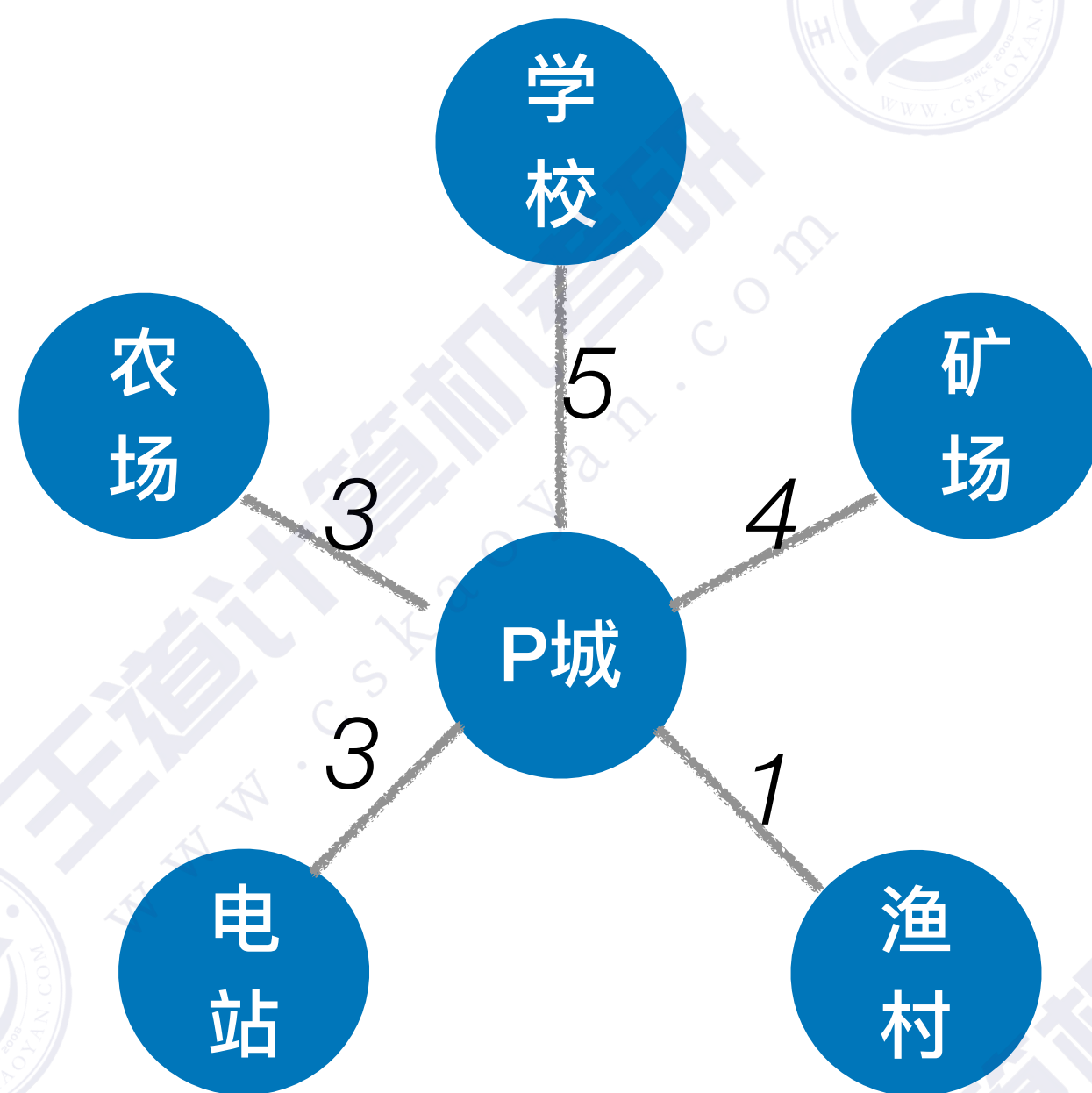
对于一个带权连通无向图 $G = (V, E)$ ，生成树不同，每棵树的权（即树中所有边上的权值之和）也可能不同。设 R 为 G 的所有生成树的集合，若 T 为 R 中边的权值之和最小的生成树，则 T 称为 G 的最小生成树（Minimum-Spanning-Tree, MST）。



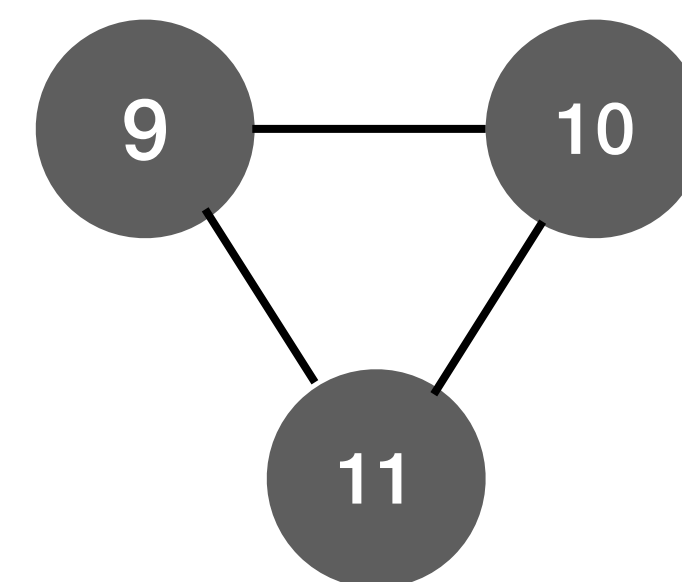
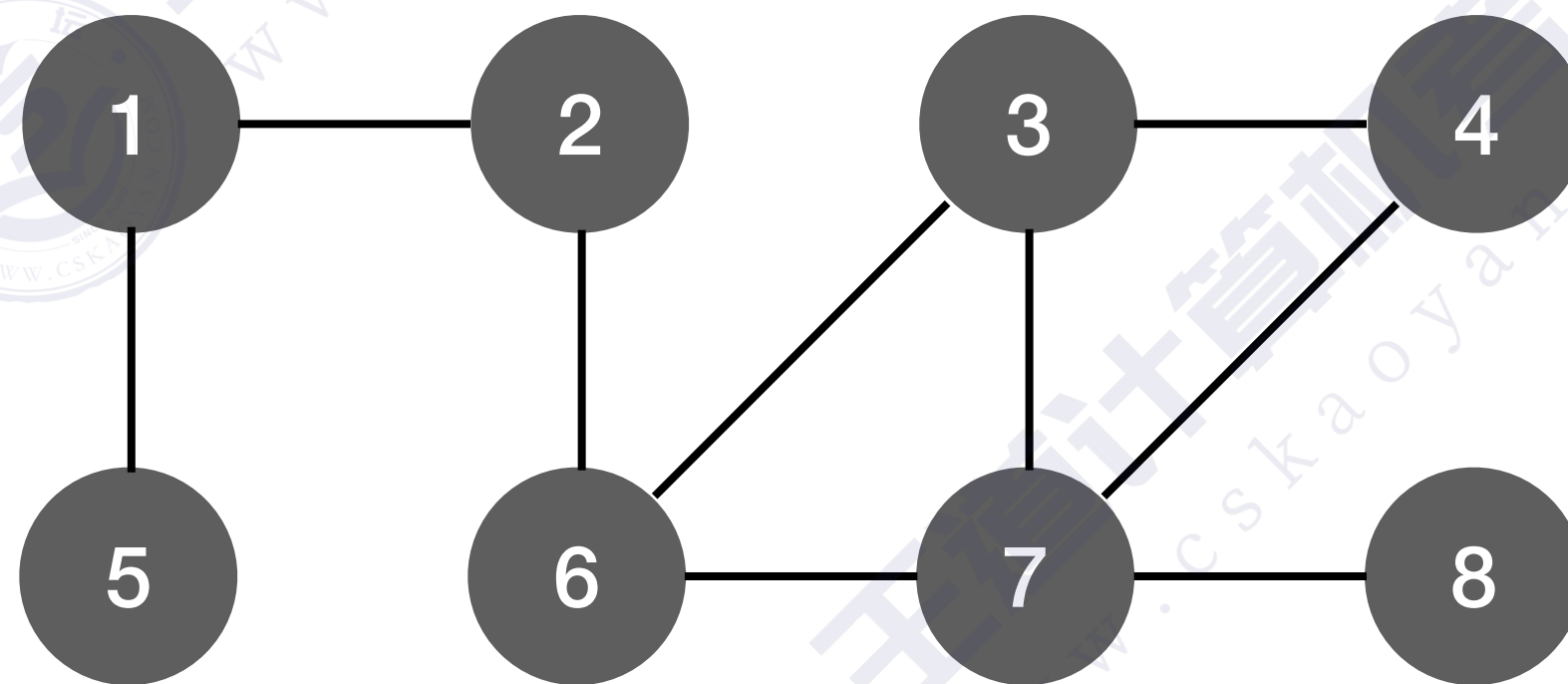
- 最小生成树可能有多个，但边的权值之和总是唯一且最小的
- 最小生成树的边数 = 顶点数 - 1。砍掉一条则不连通，增加一条边则会出现回路

最小生成树（最小代价树）

对于一个带权连通无向图 $G = (V, E)$ ，生成树不同，每棵树的权（即树中所有边上的权值之和）也可能不同。设 R 为 G 的所有生成树的集合，若 T 为 R 中边的权值之和最小的生成树，则 T 称为 G 的最小生成树（Minimum-Spanning-Tree, MST）。



- 如果一个连通图本身就是一棵树，则其最小生成树就是它本身
- 只有连通图才有生成树，非连通图只有生成森林



最小生成树（最小代价树）

对于一个带权连通无向图 $G = (V, E)$ ，生成树不同，每棵树的权（即树中所有边上的权值之和）也可能不同。设 R 为 G 的所有生成树的集合，若 T 为 R 中边的权值之和最小的生成树，则 T 称为 G 的最小生成树（Minimum-Spanning-Tree, MST）。

- 最小生成树可能有多个，但边的权值之和总是唯一且最小的
- 最小生成树的边数 = 顶点数 - 1。砍掉一条则不连通，增加一条边则会出现回路
- 如果一个连通图本身就是一棵树，则其最小生成树就是它本身
- 只有连通图才有生成树，非连通图只有生成森林

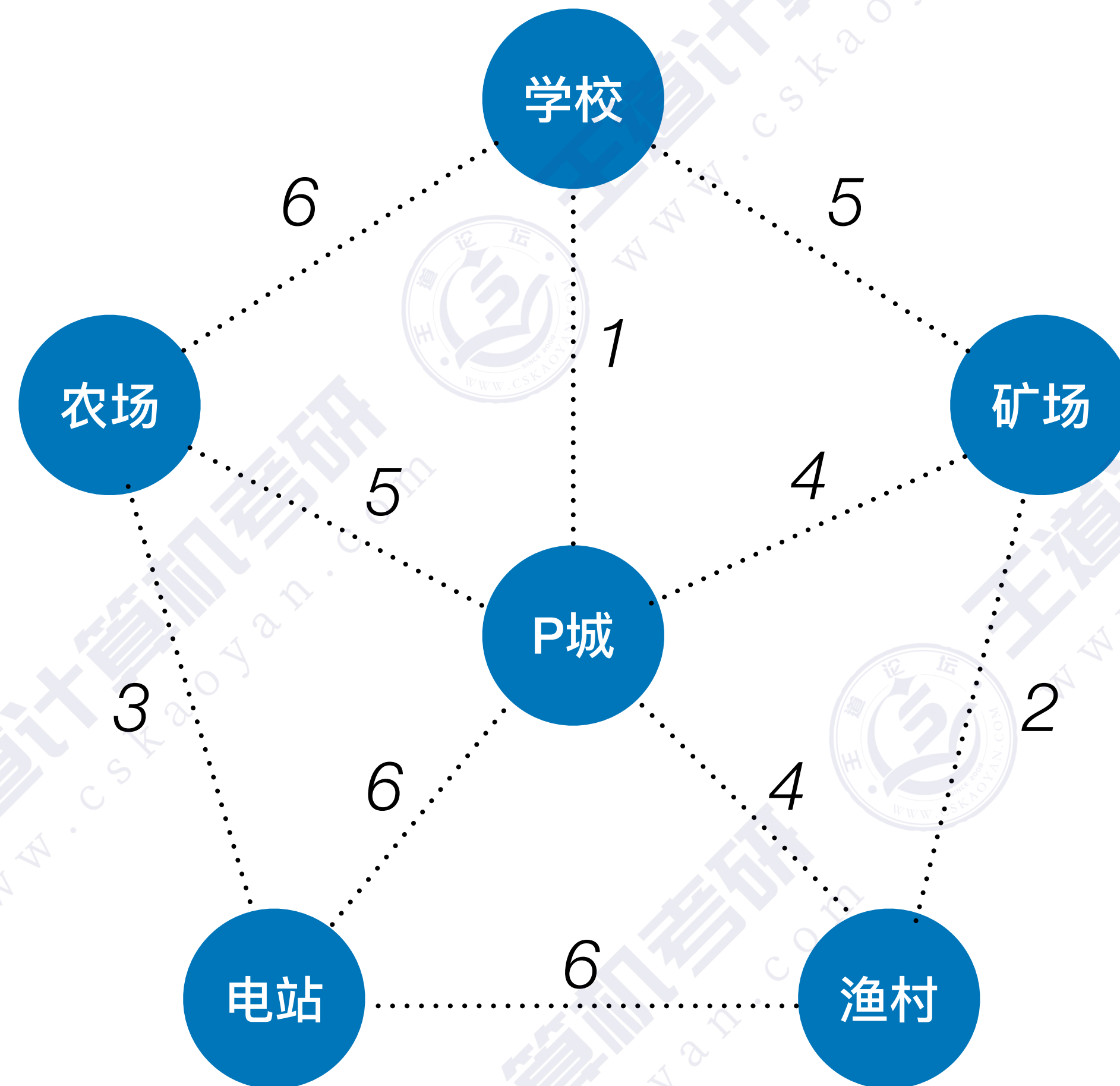


求最小生成树

Prim 算法

Kruskal 算法

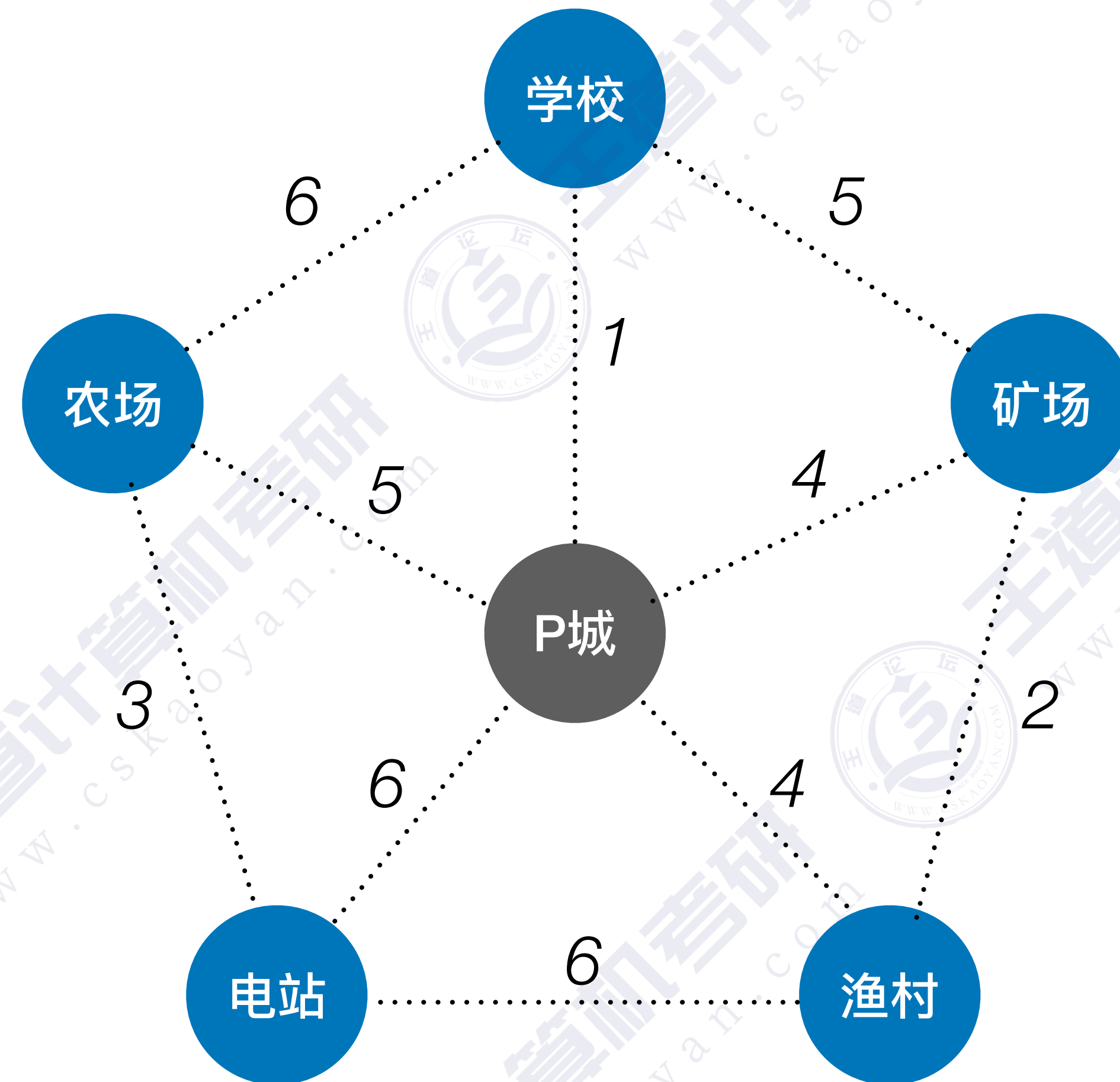
Prim 算法（普里姆）



Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

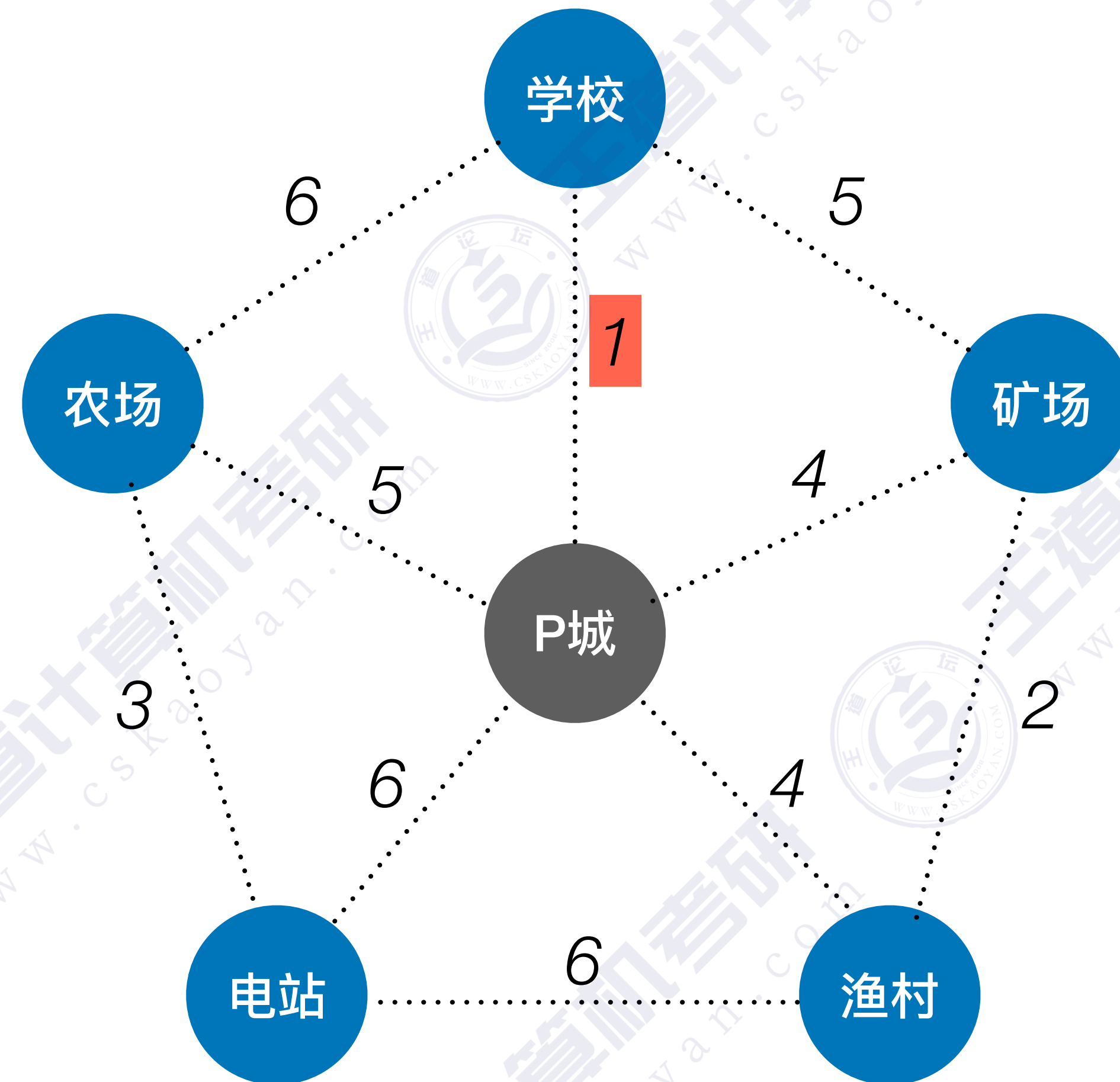
Prim 算法（普里姆）



Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

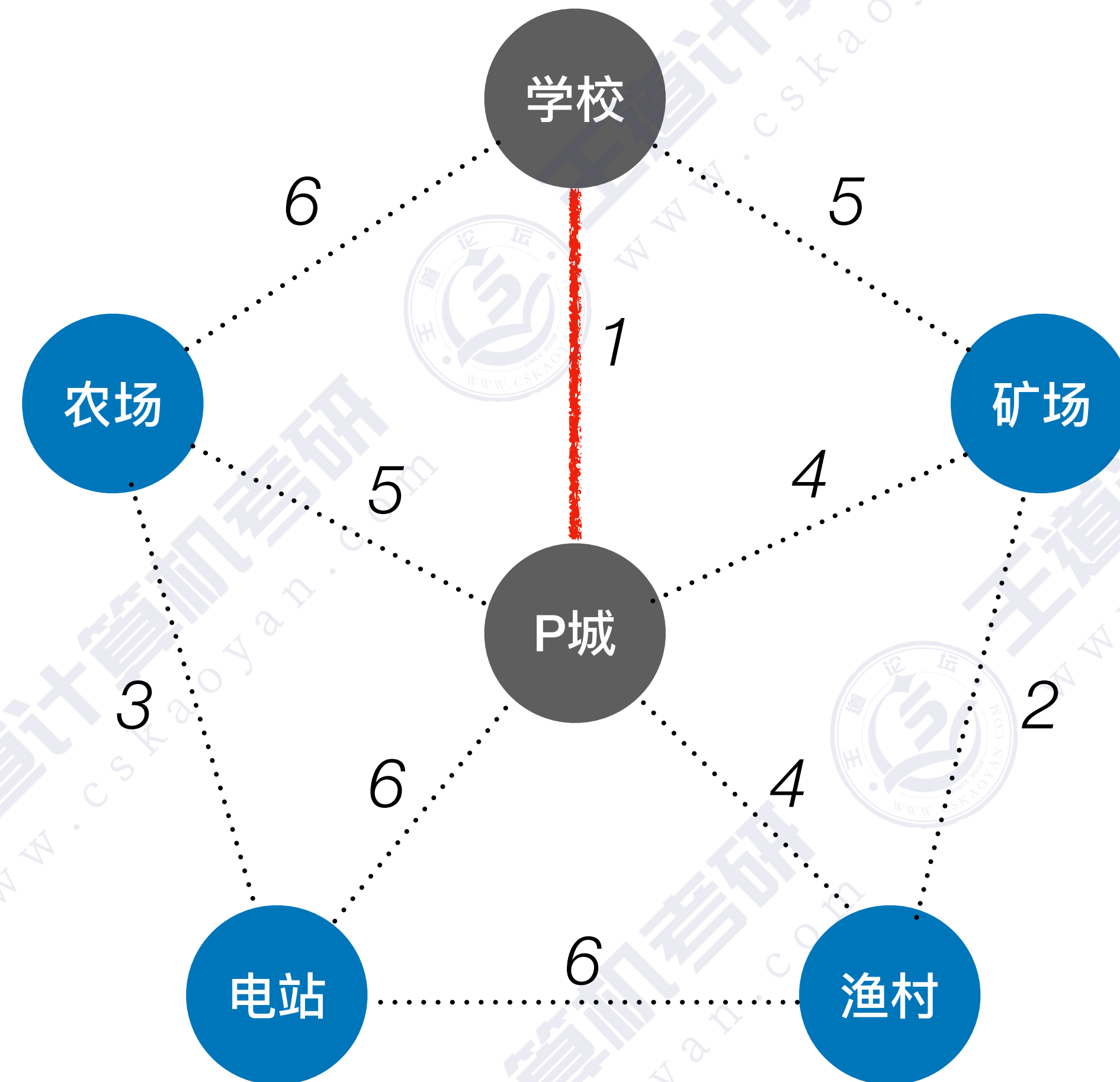
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

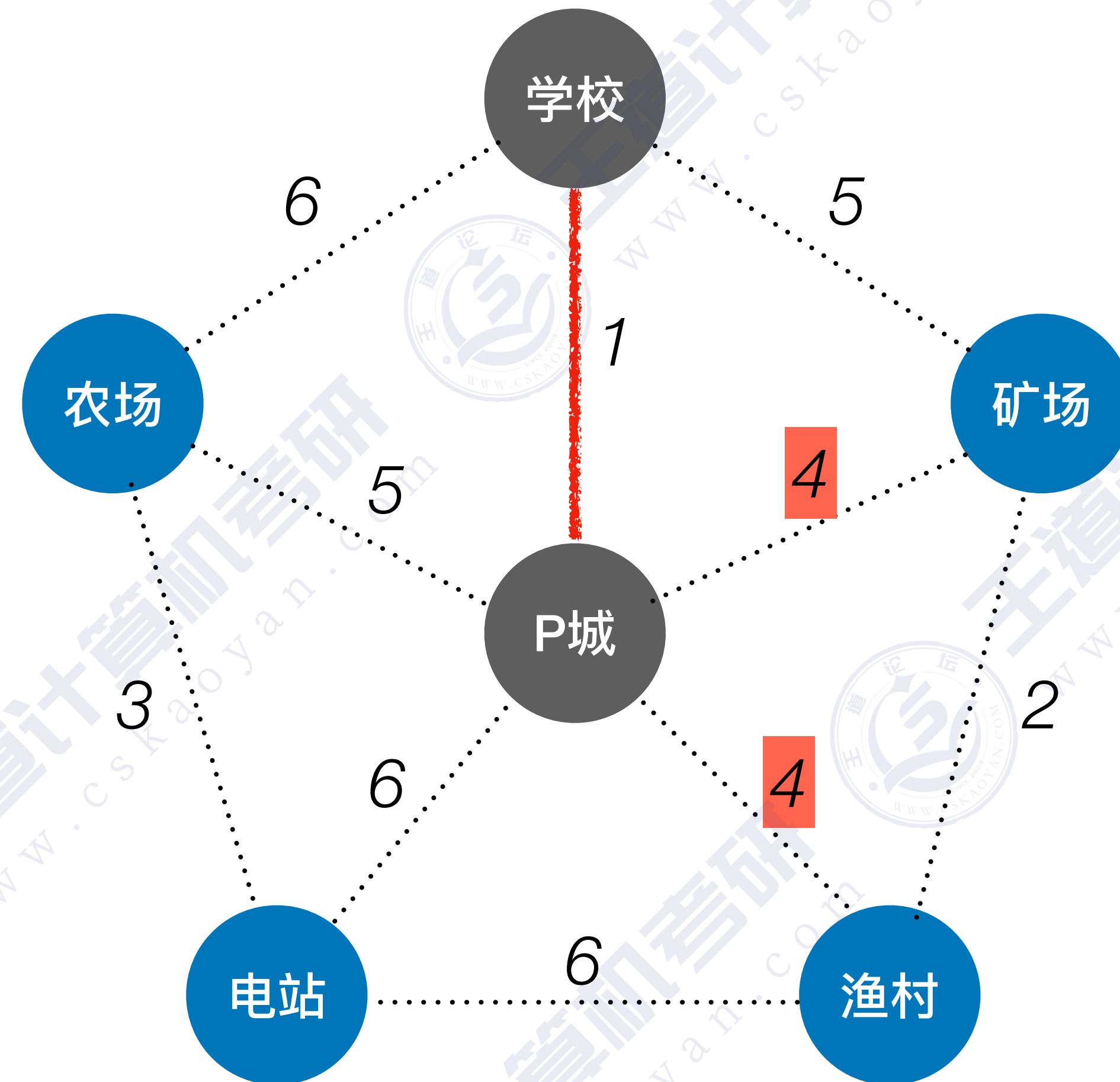
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

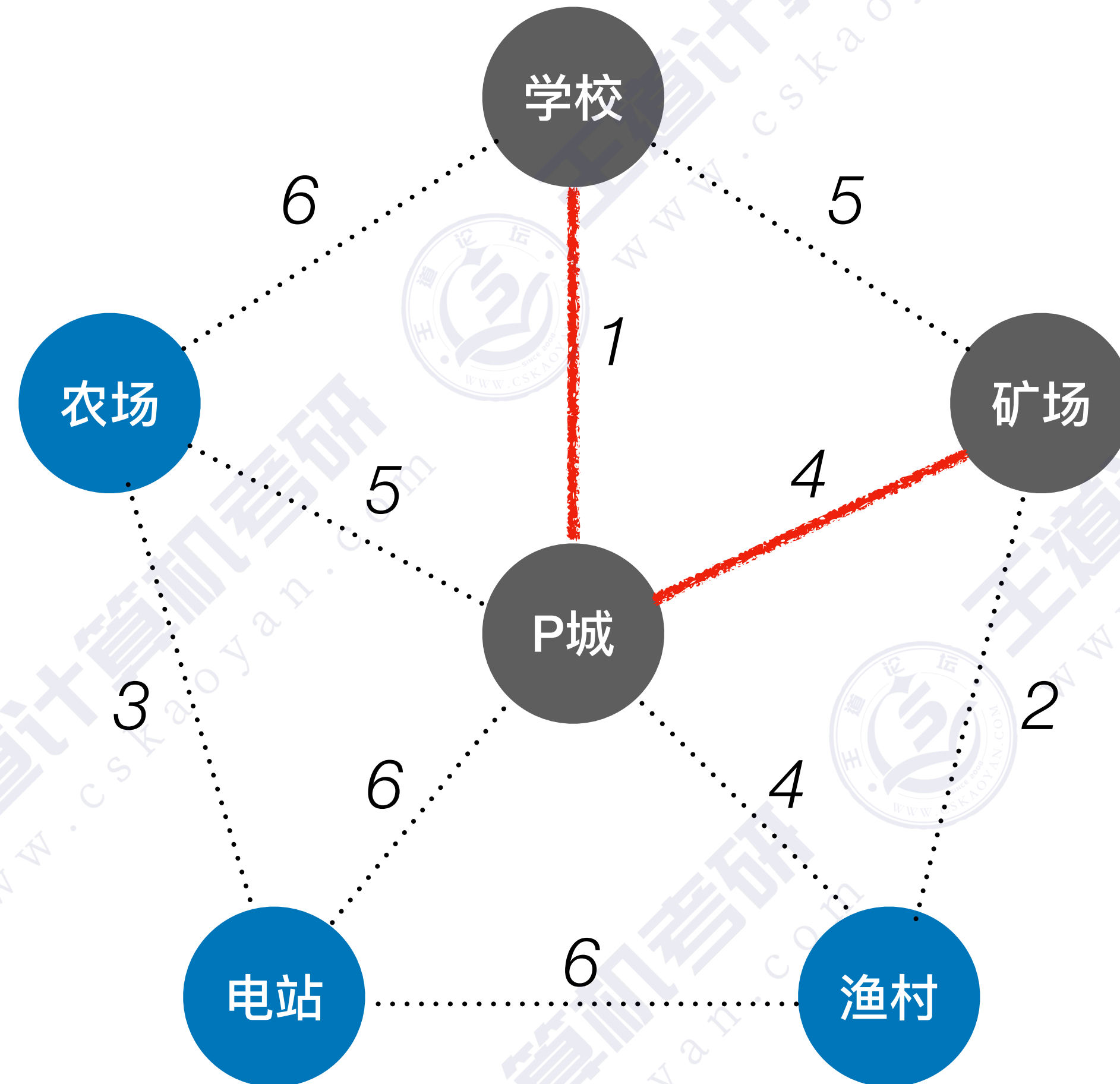
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

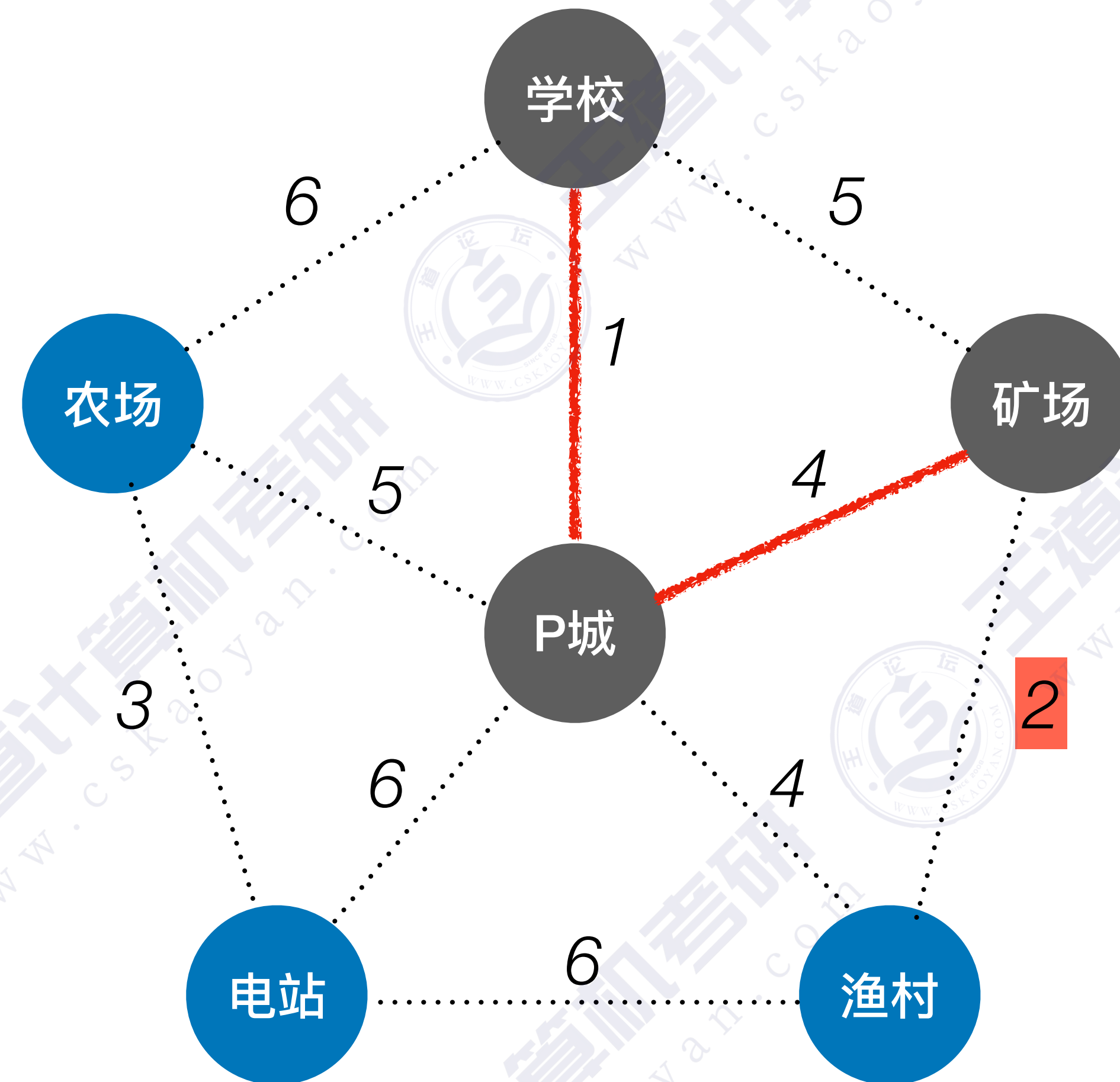
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成
树, 直到所有顶点都纳入为止。

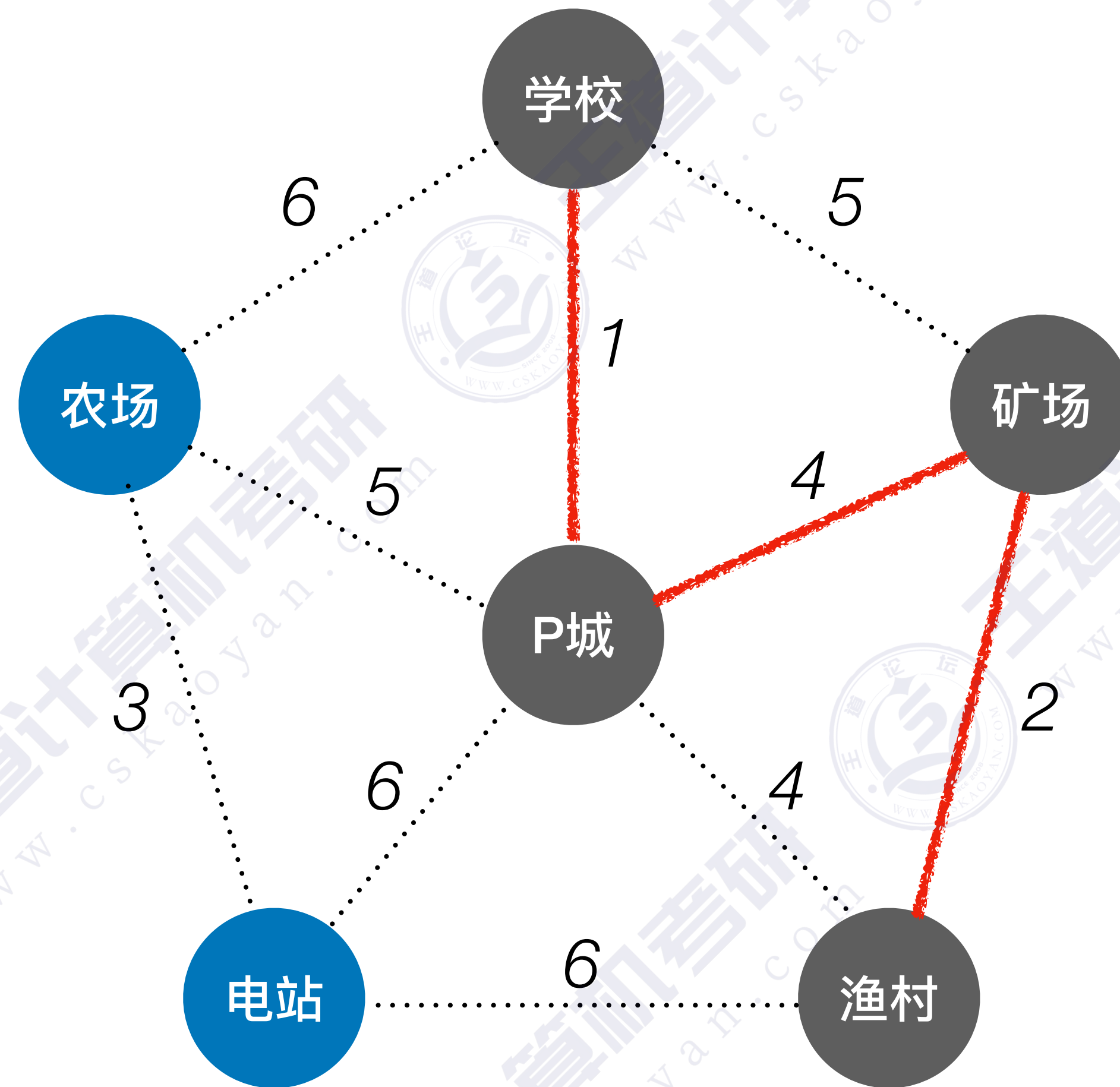
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

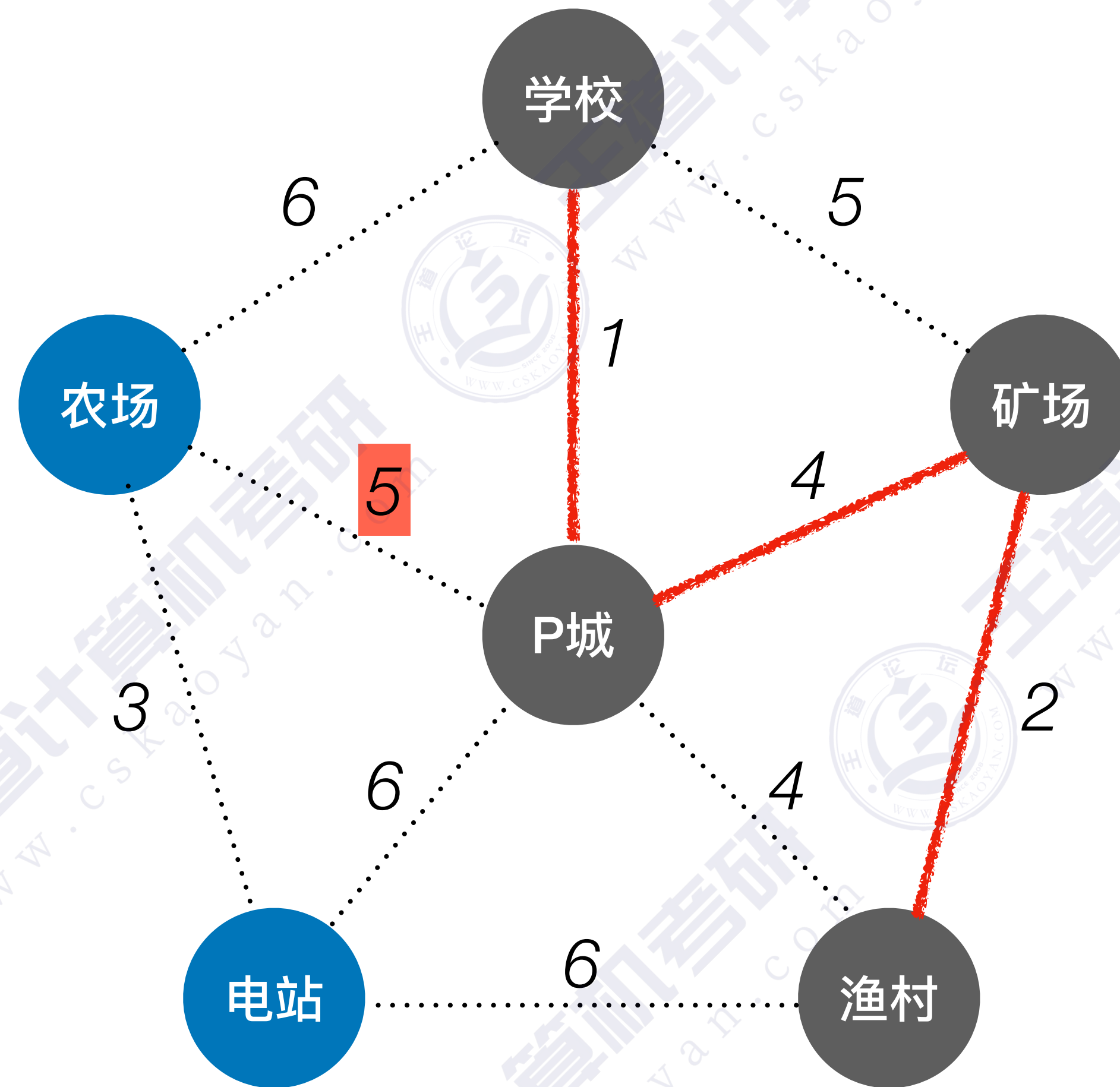
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

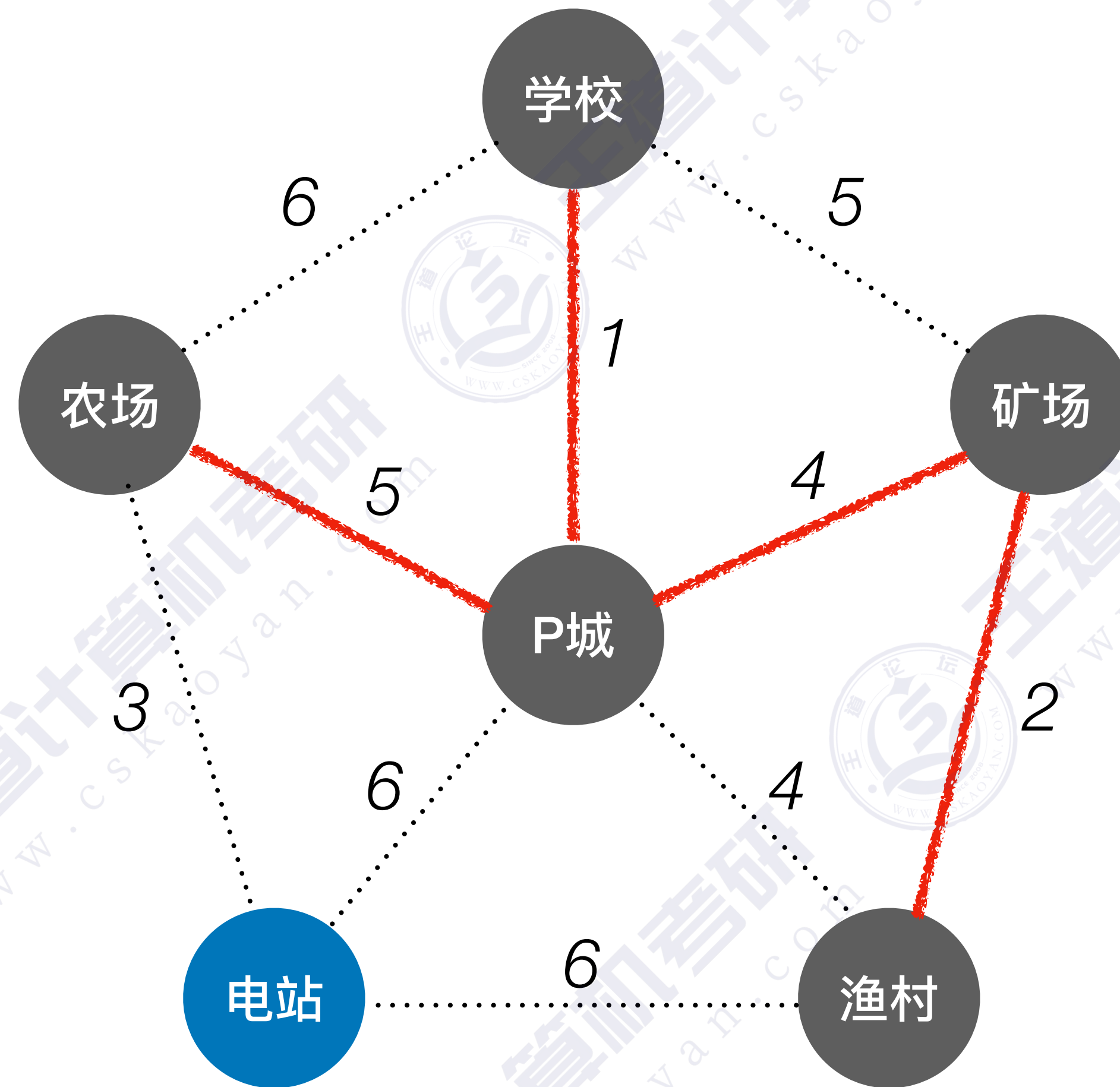
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

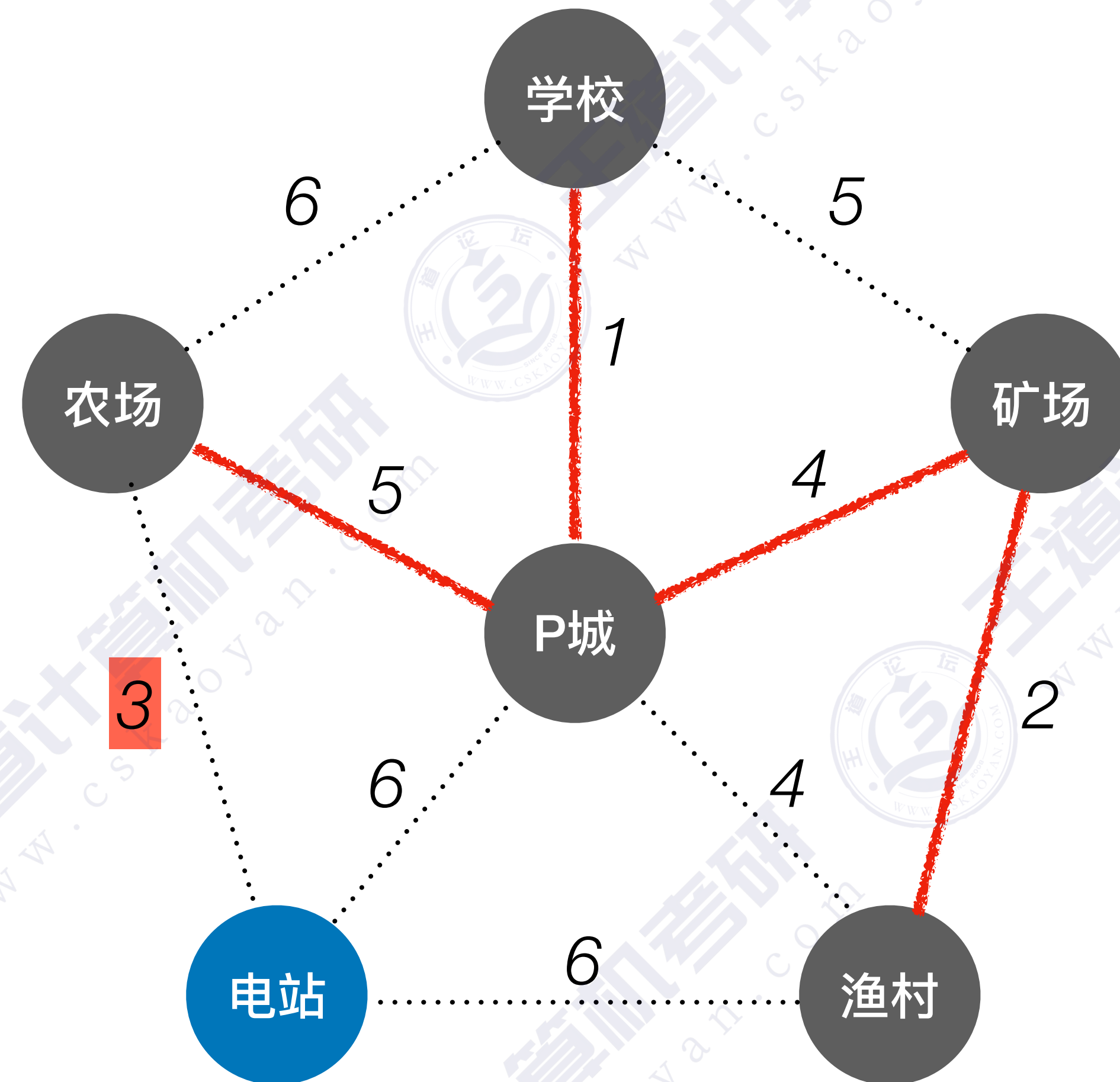
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

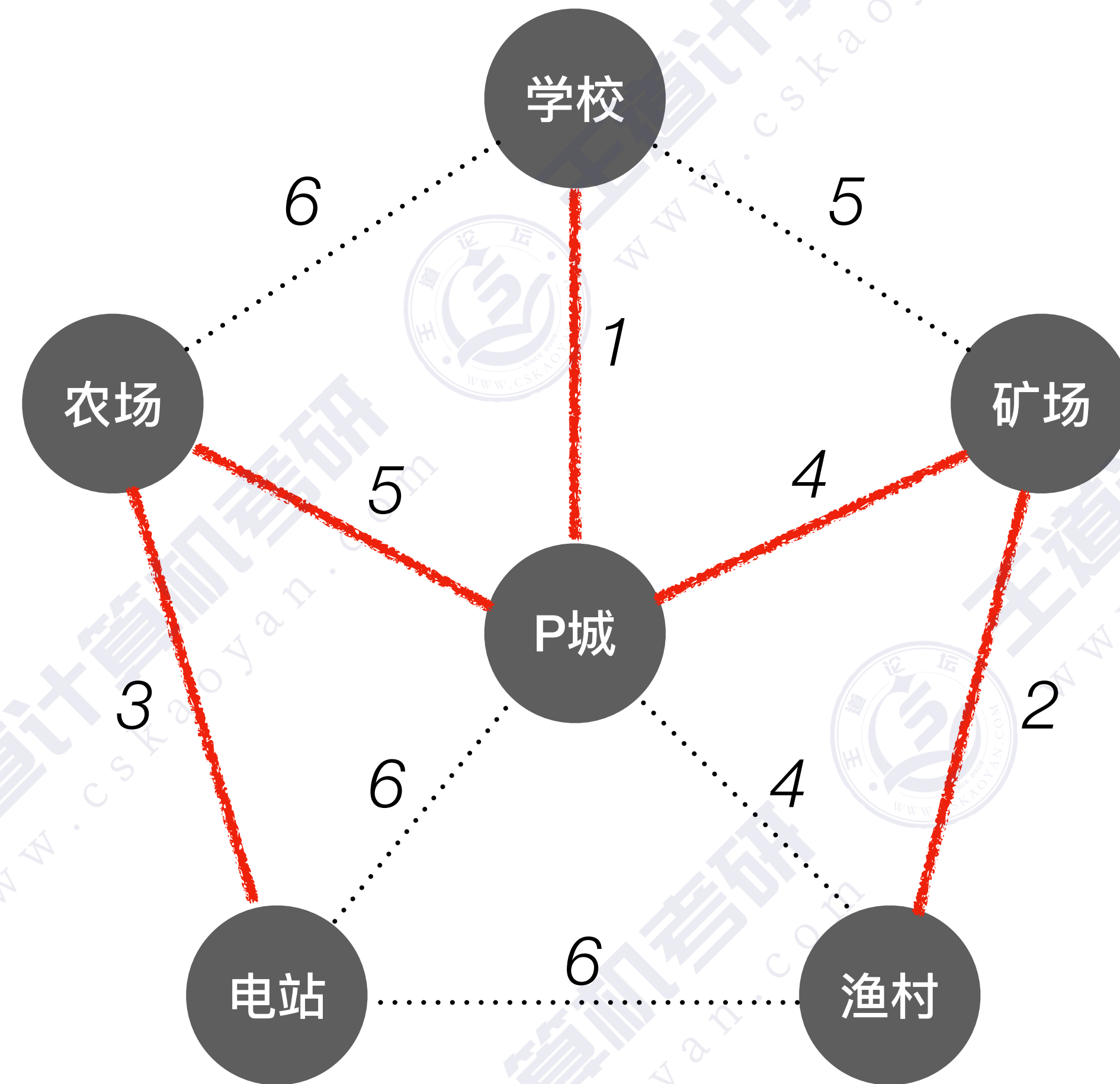
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成
树, 直到所有顶点都纳入为止。

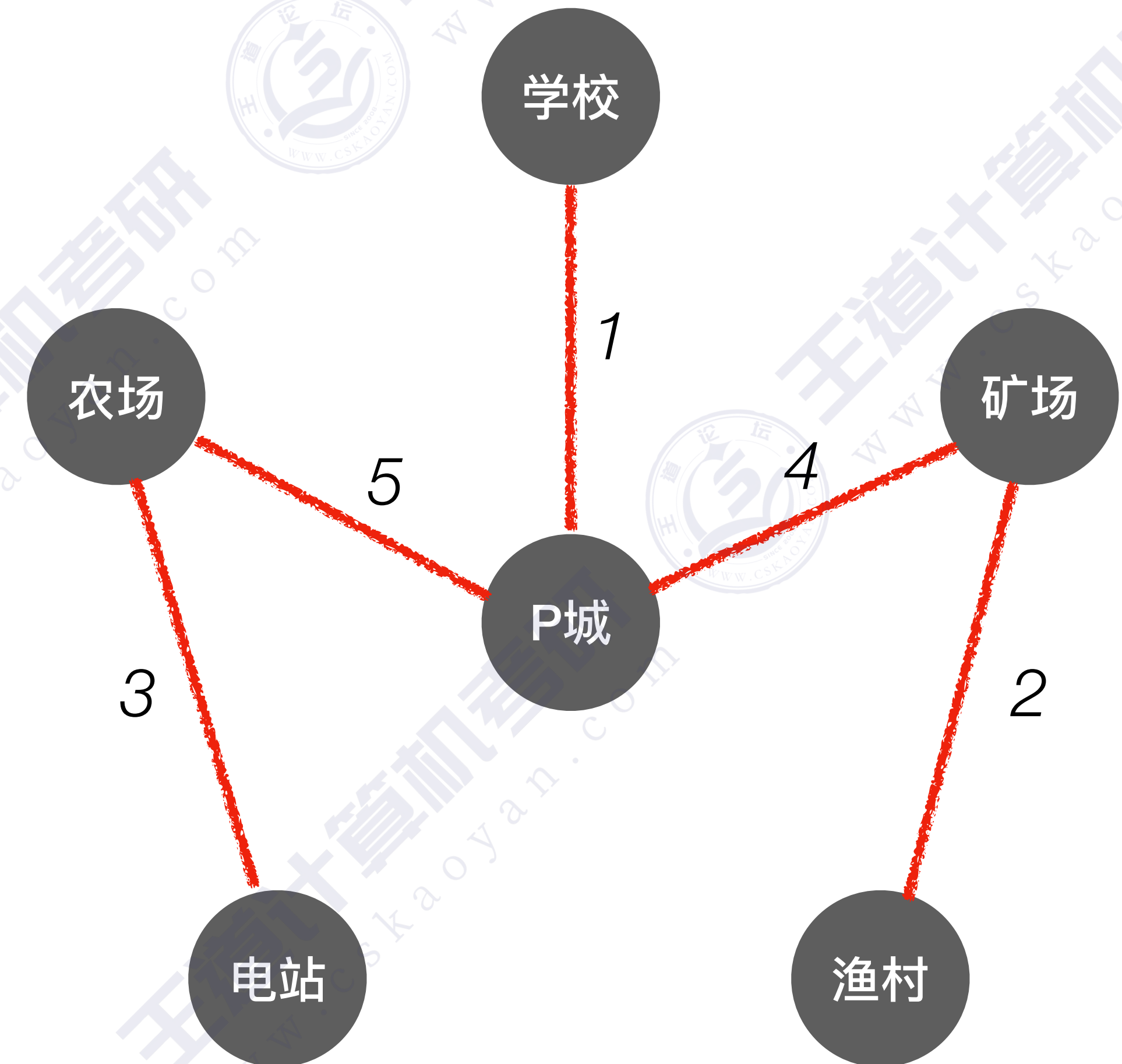
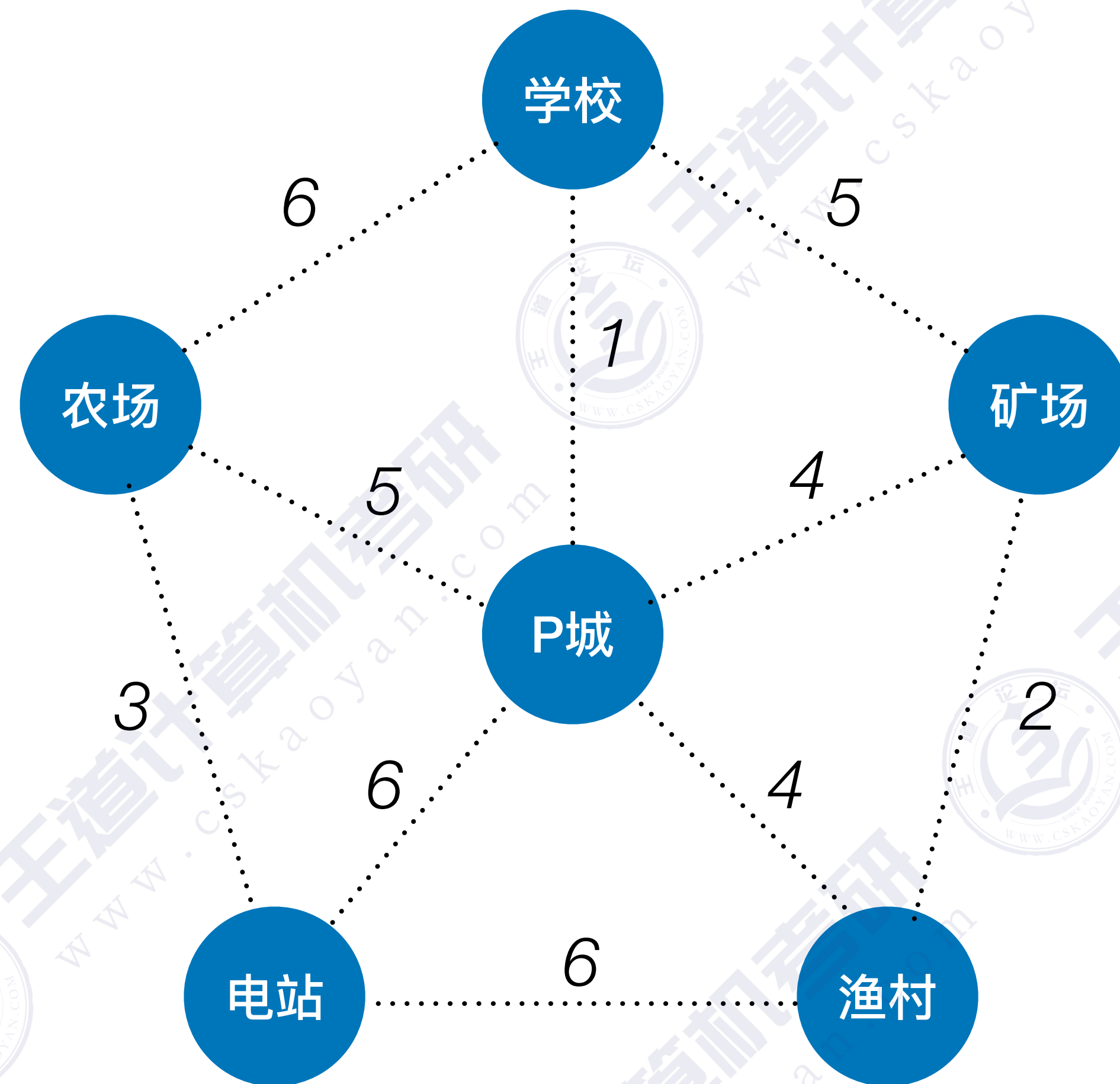
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

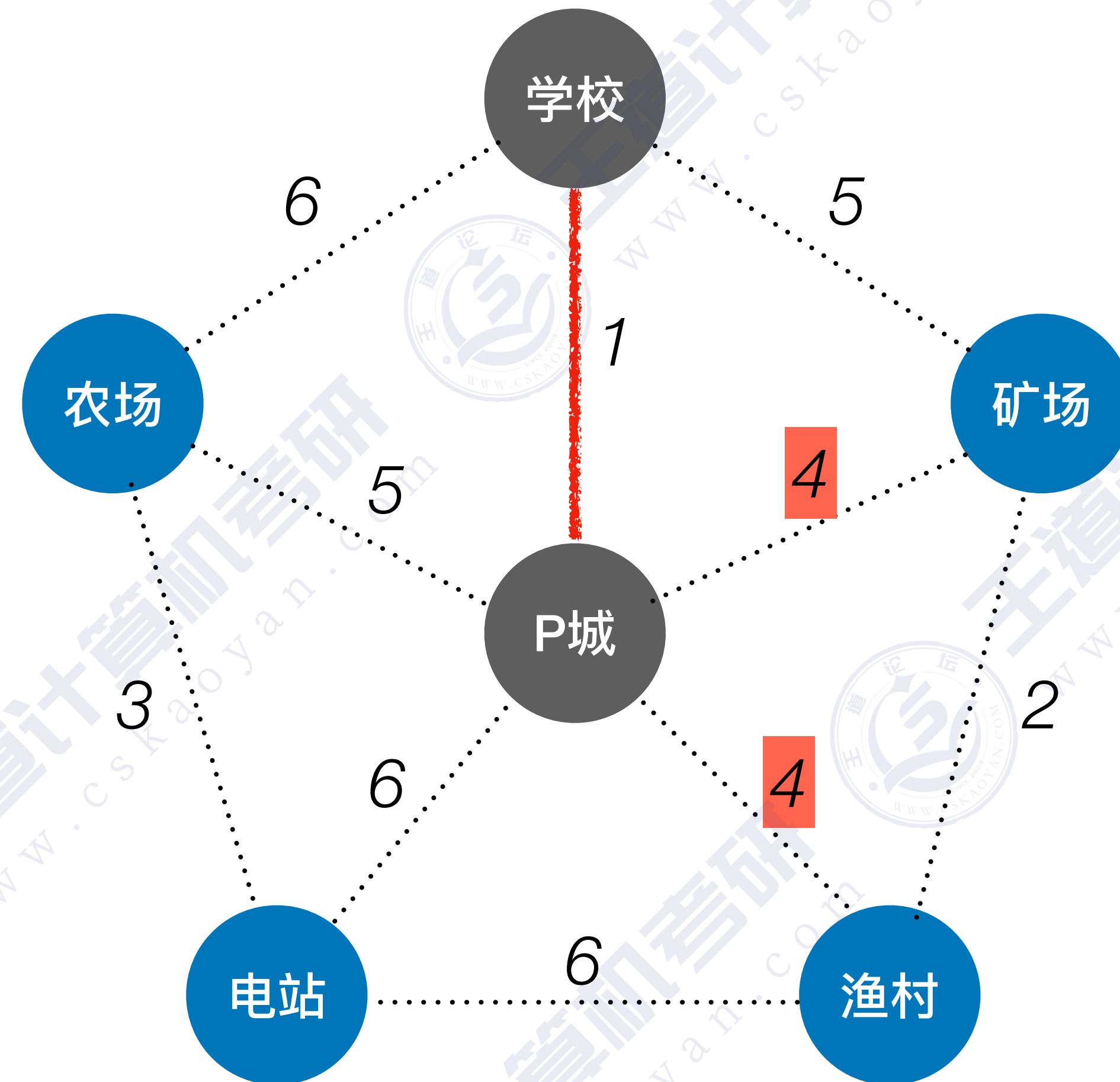
从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成
树, 直到所有顶点都纳入为止。

Prim 算法 (普里姆)



最小代价: 15

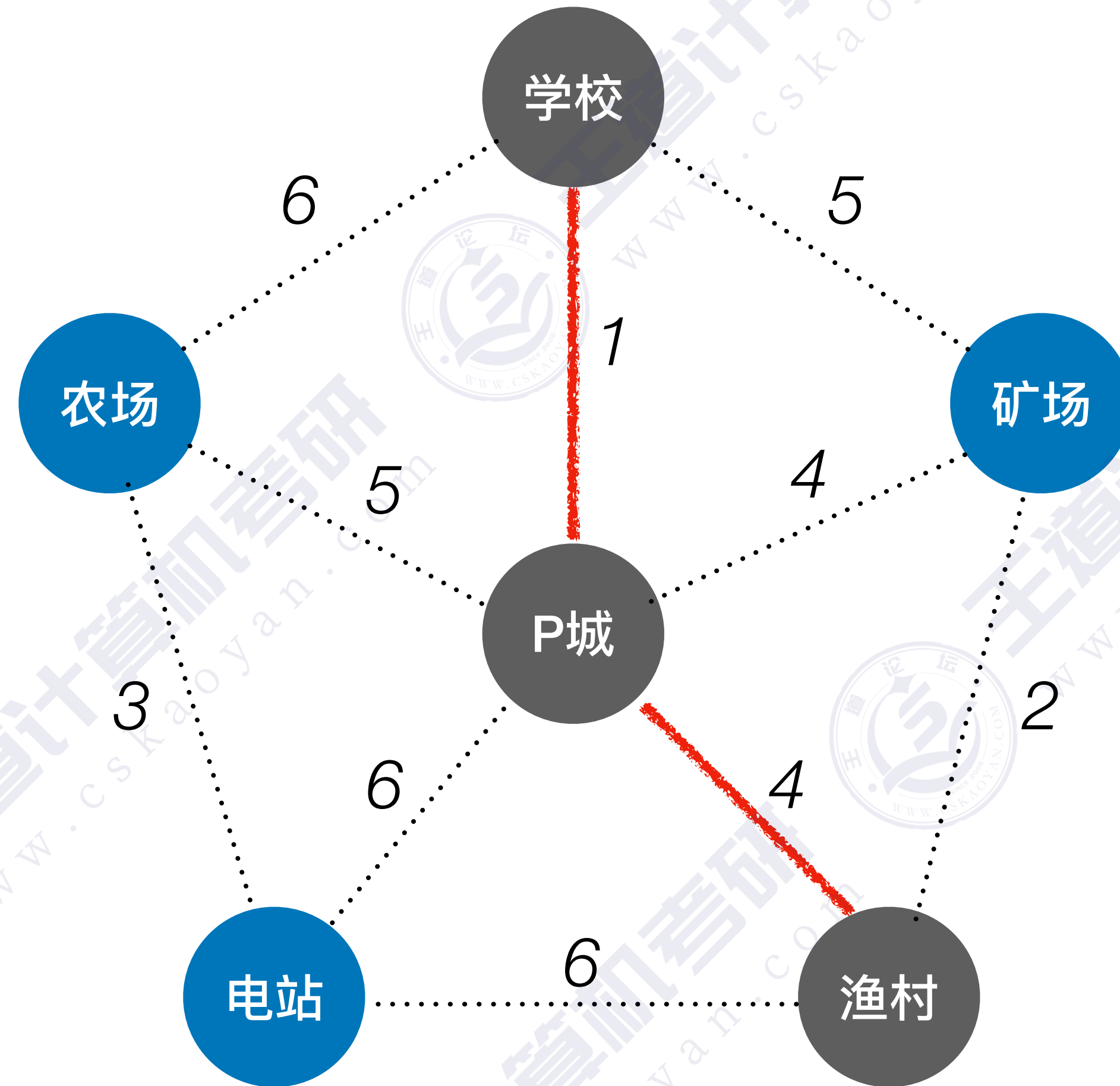
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

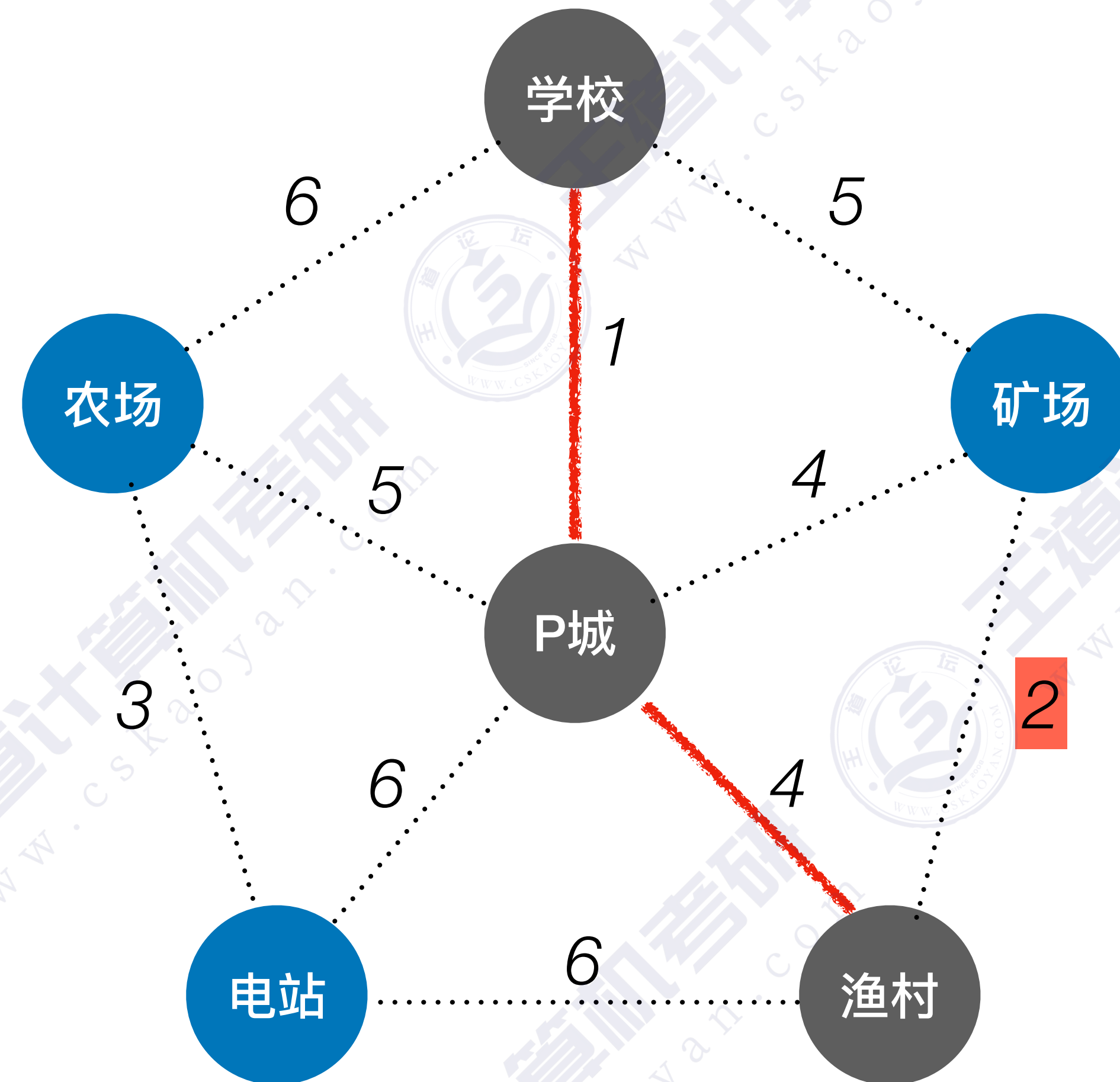
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

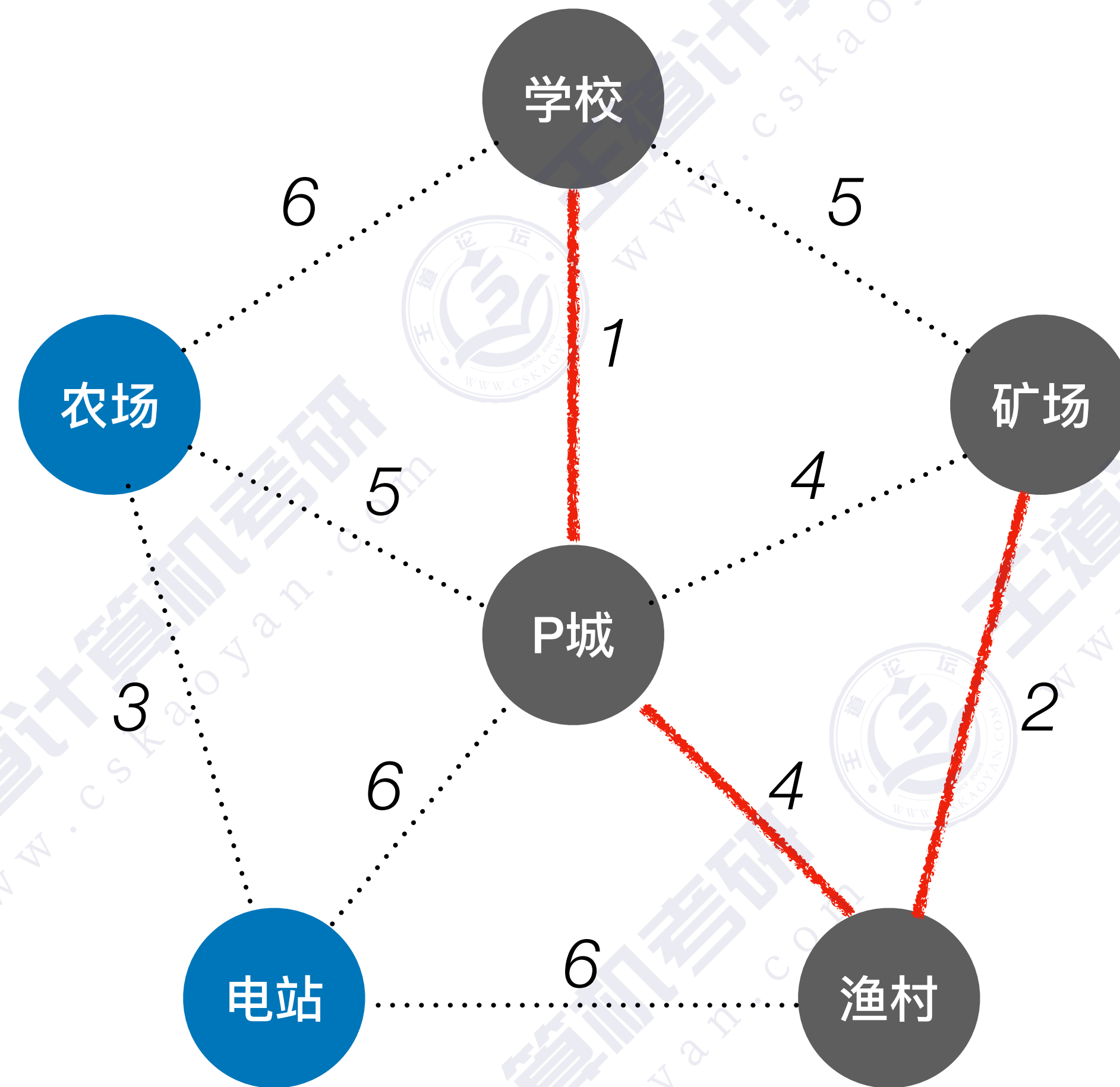
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

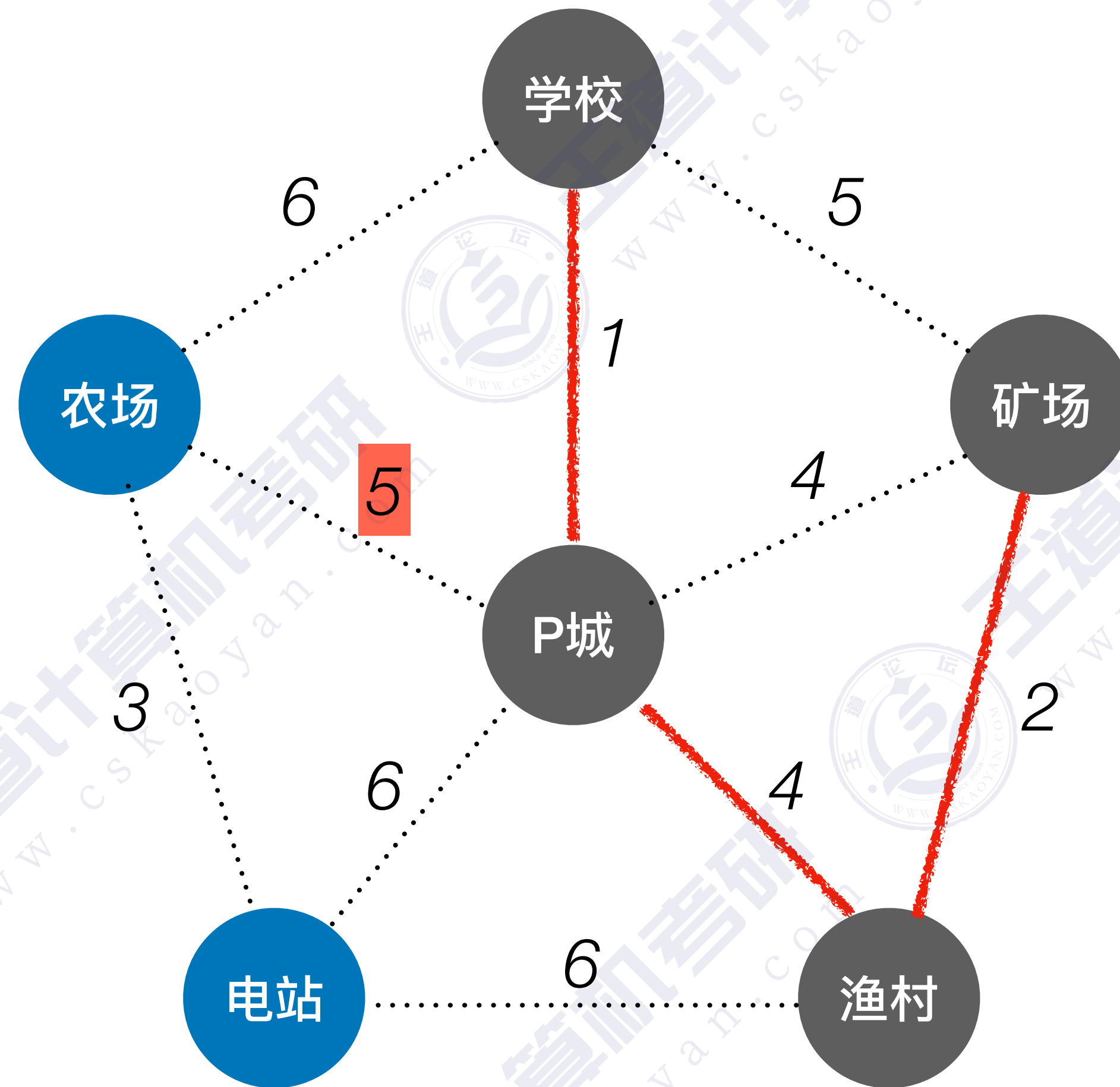
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成
树, 直到所有顶点都纳入为止。

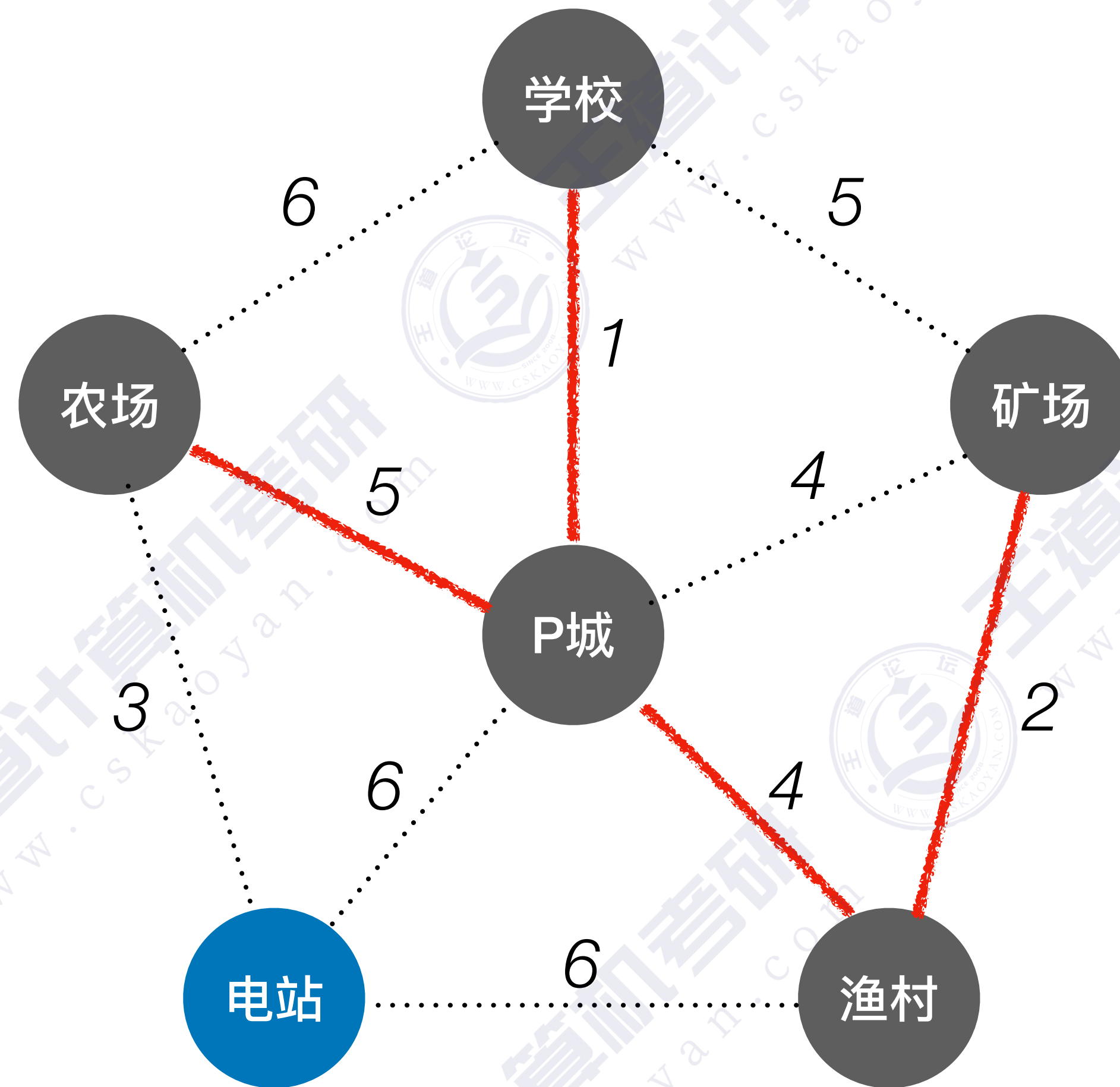
Prim 算法（普里姆）



Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

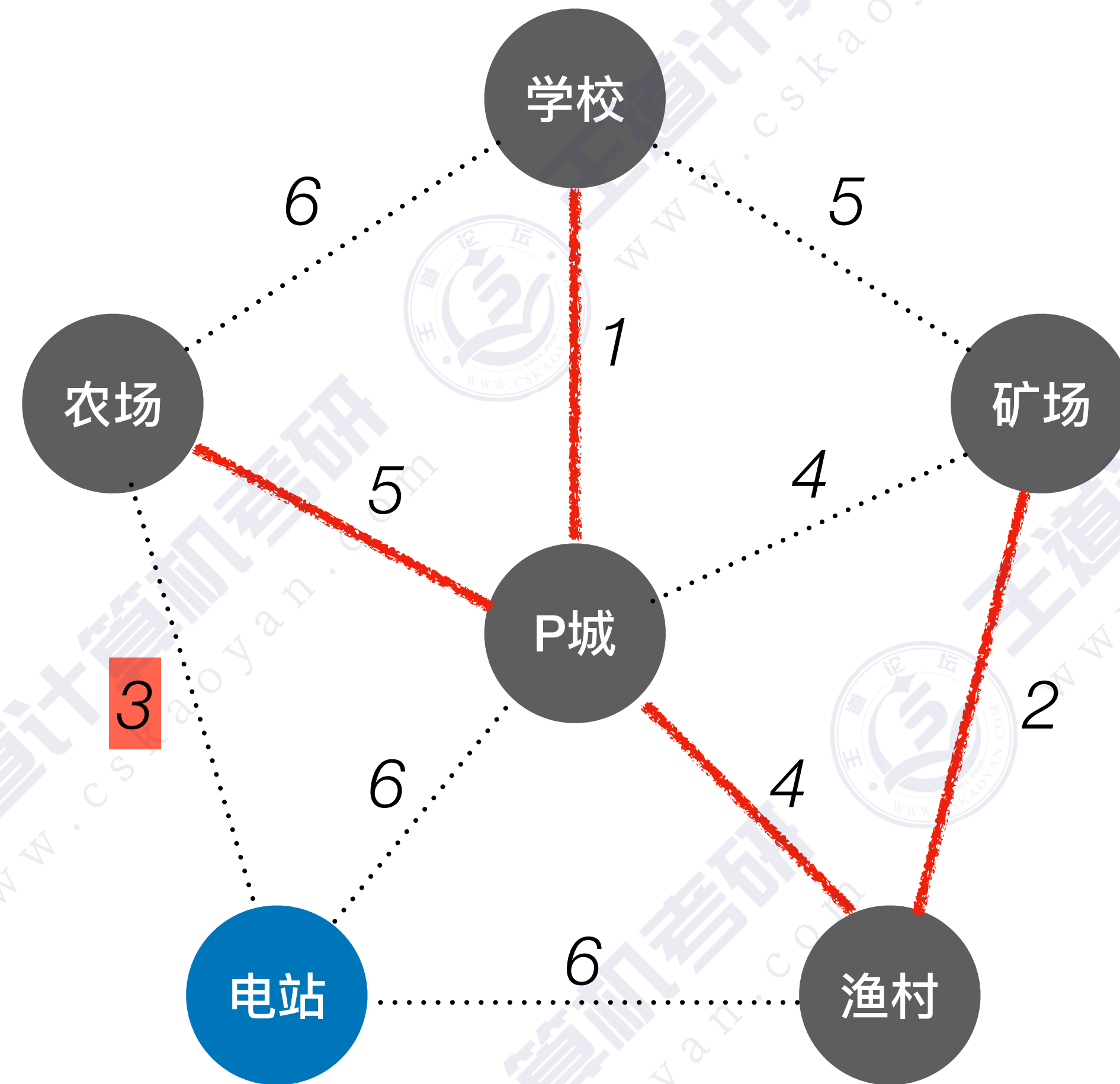
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

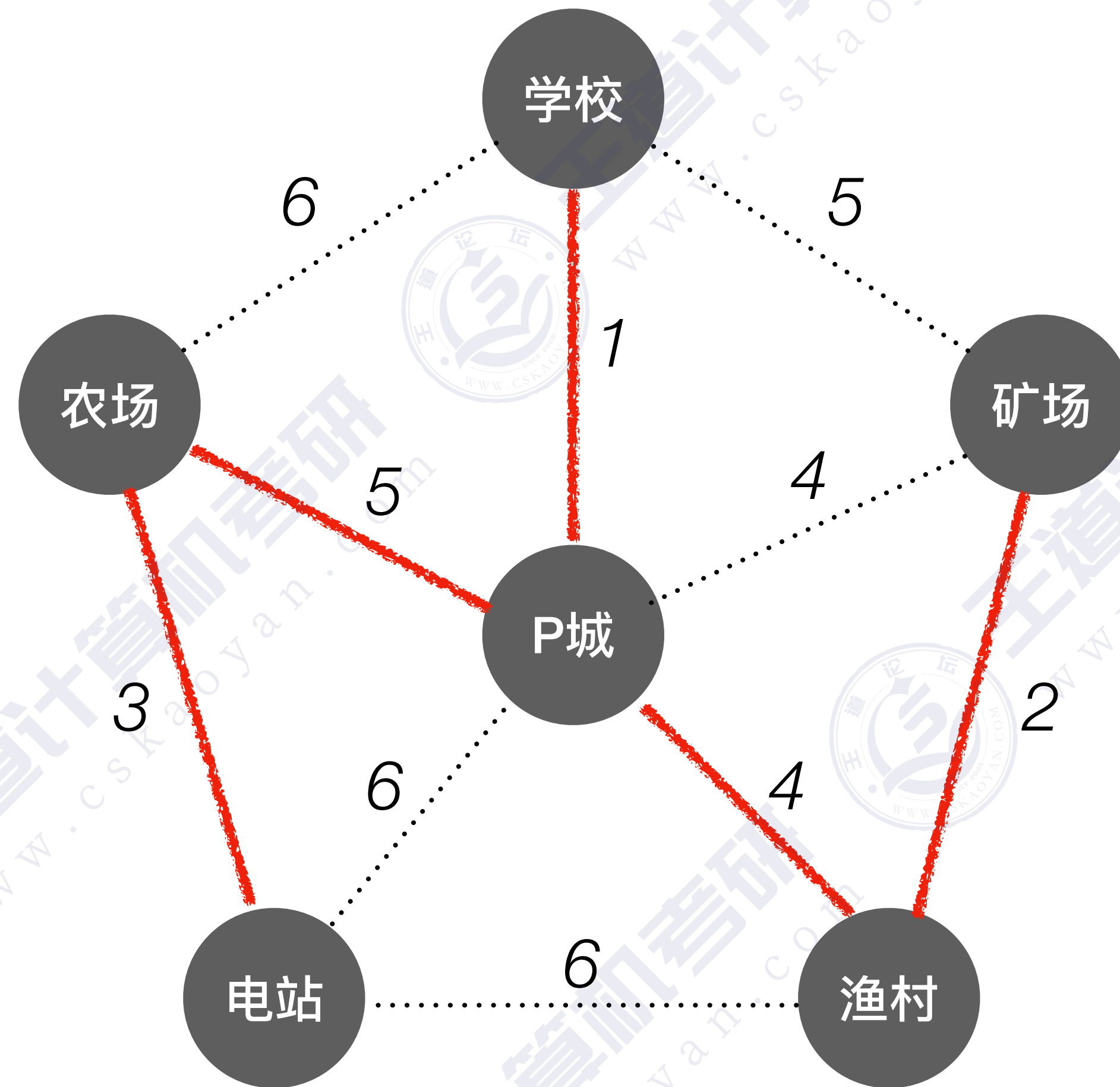
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

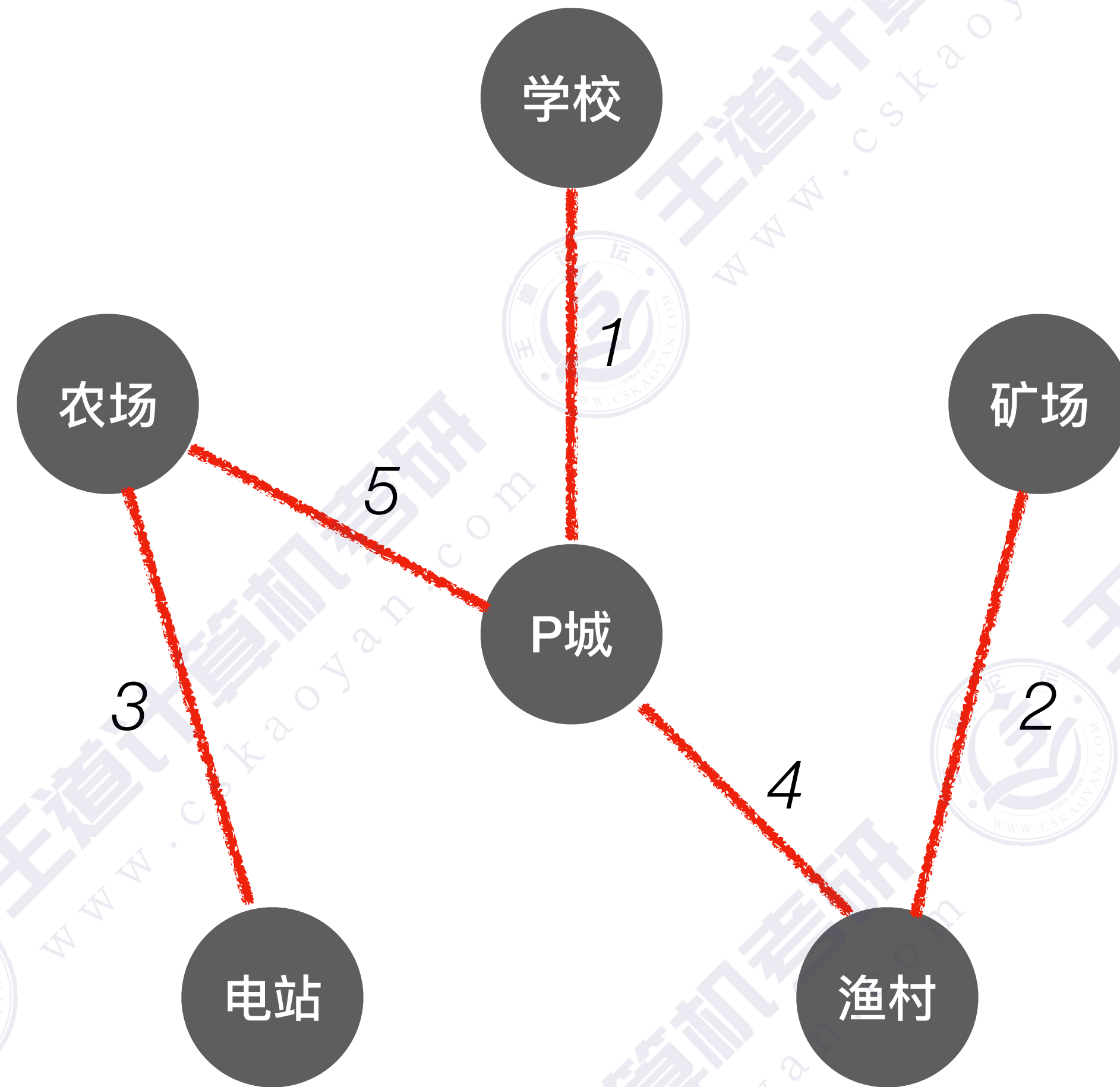
Prim 算法 (普里姆)



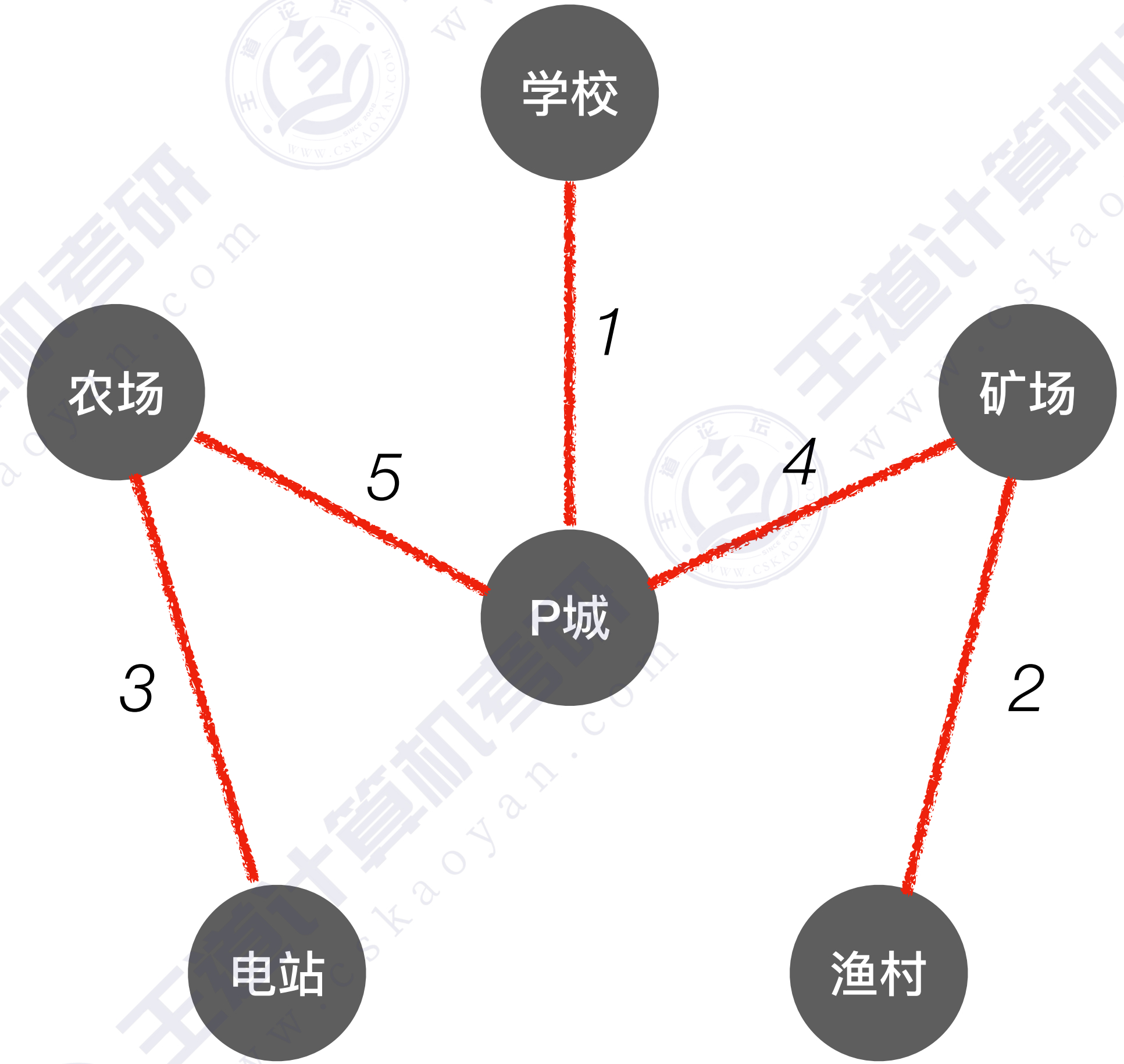
Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

Prim 算法 (普里姆)

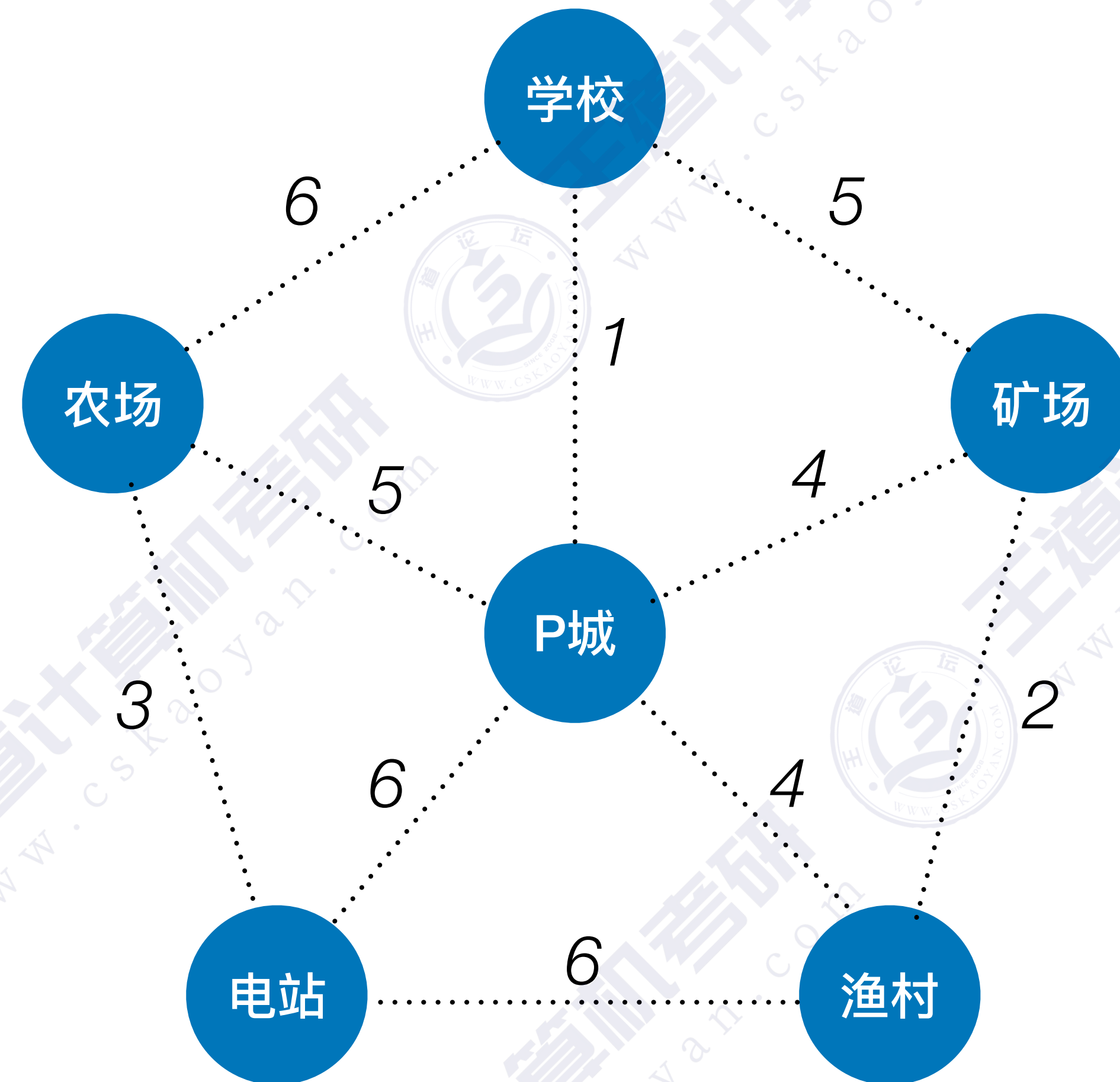


最小代价: 15



最小代价: 15

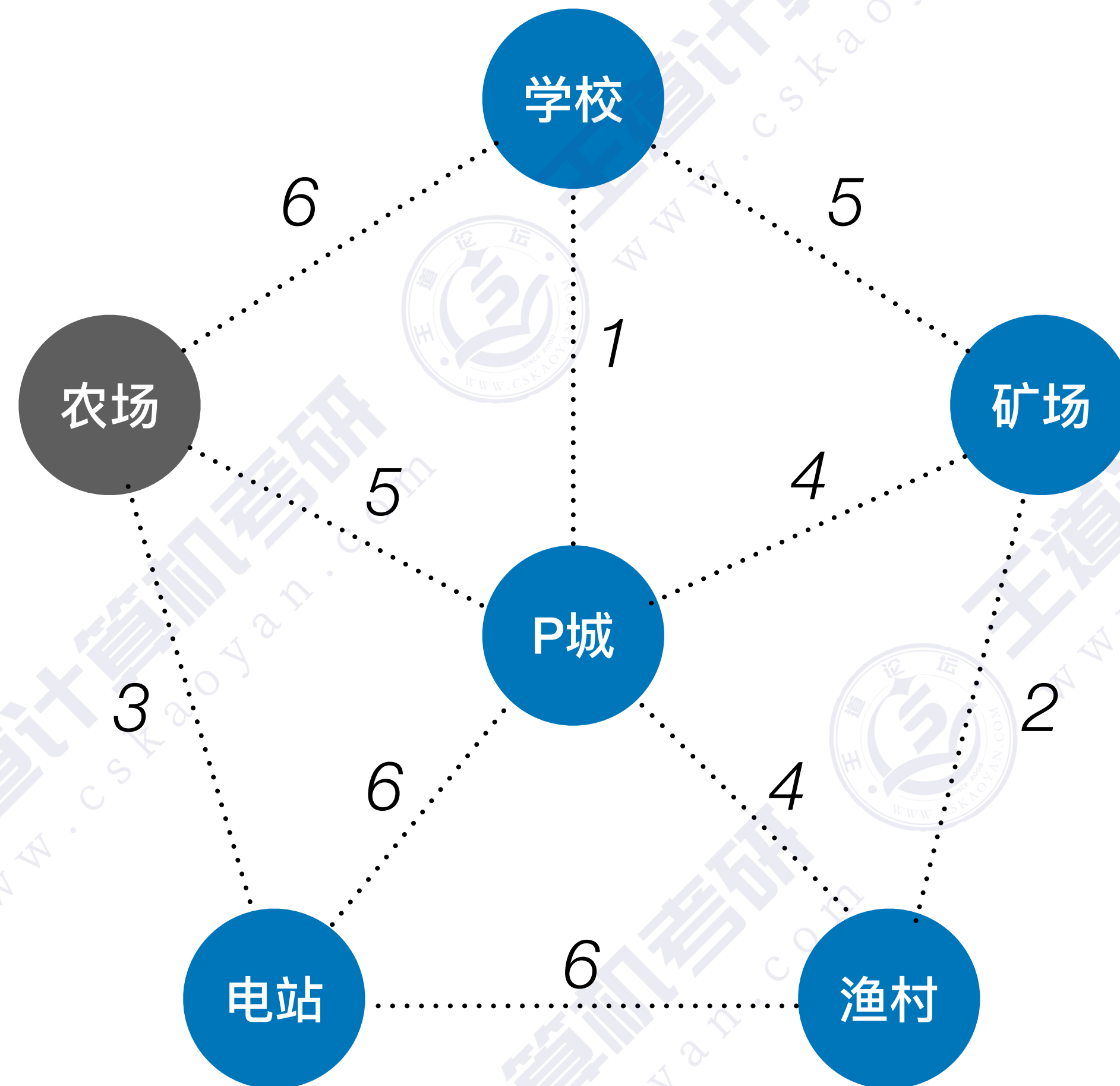
Prim 算法（普里姆）



Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

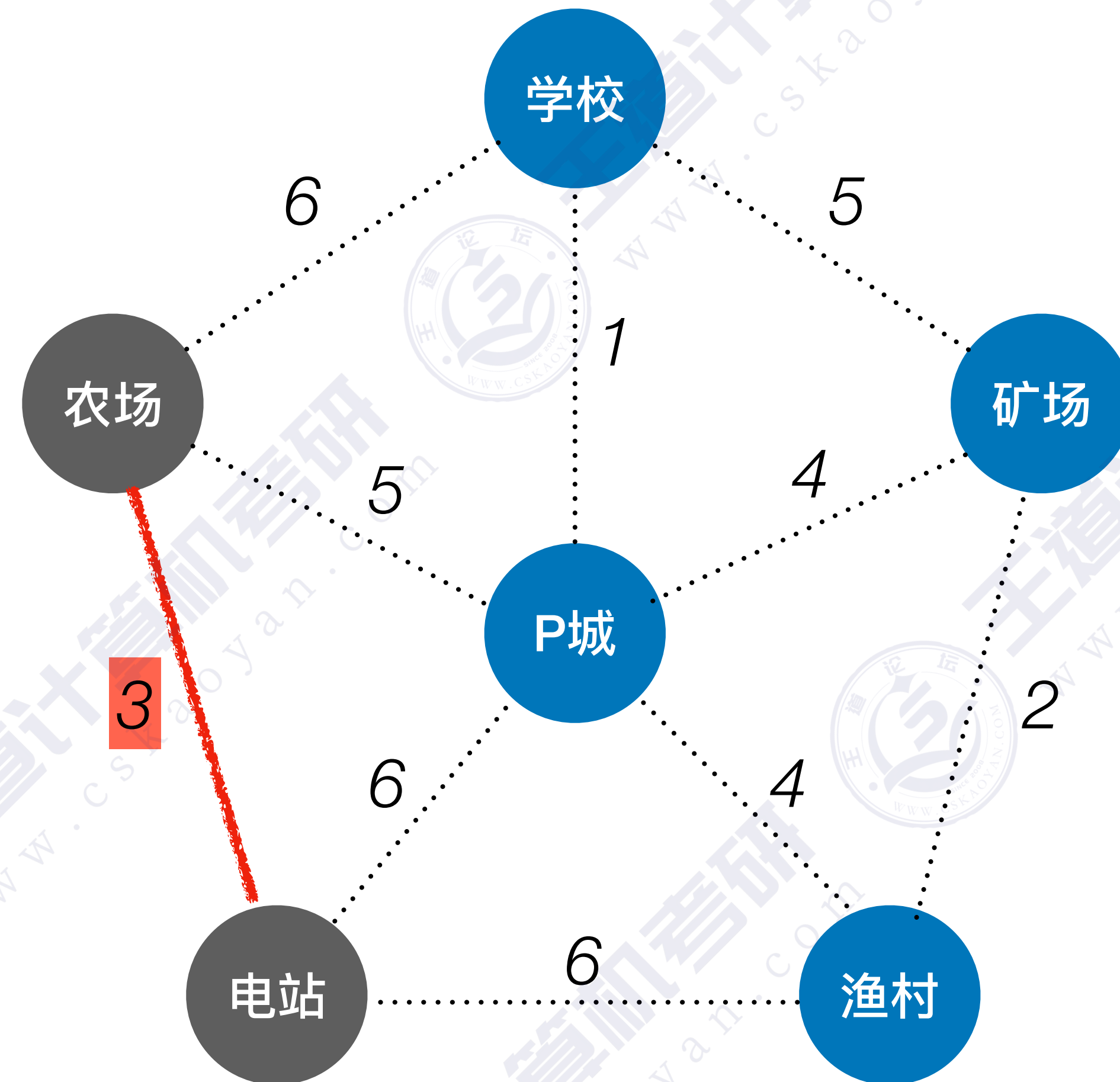
Prim 算法（普里姆）



Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

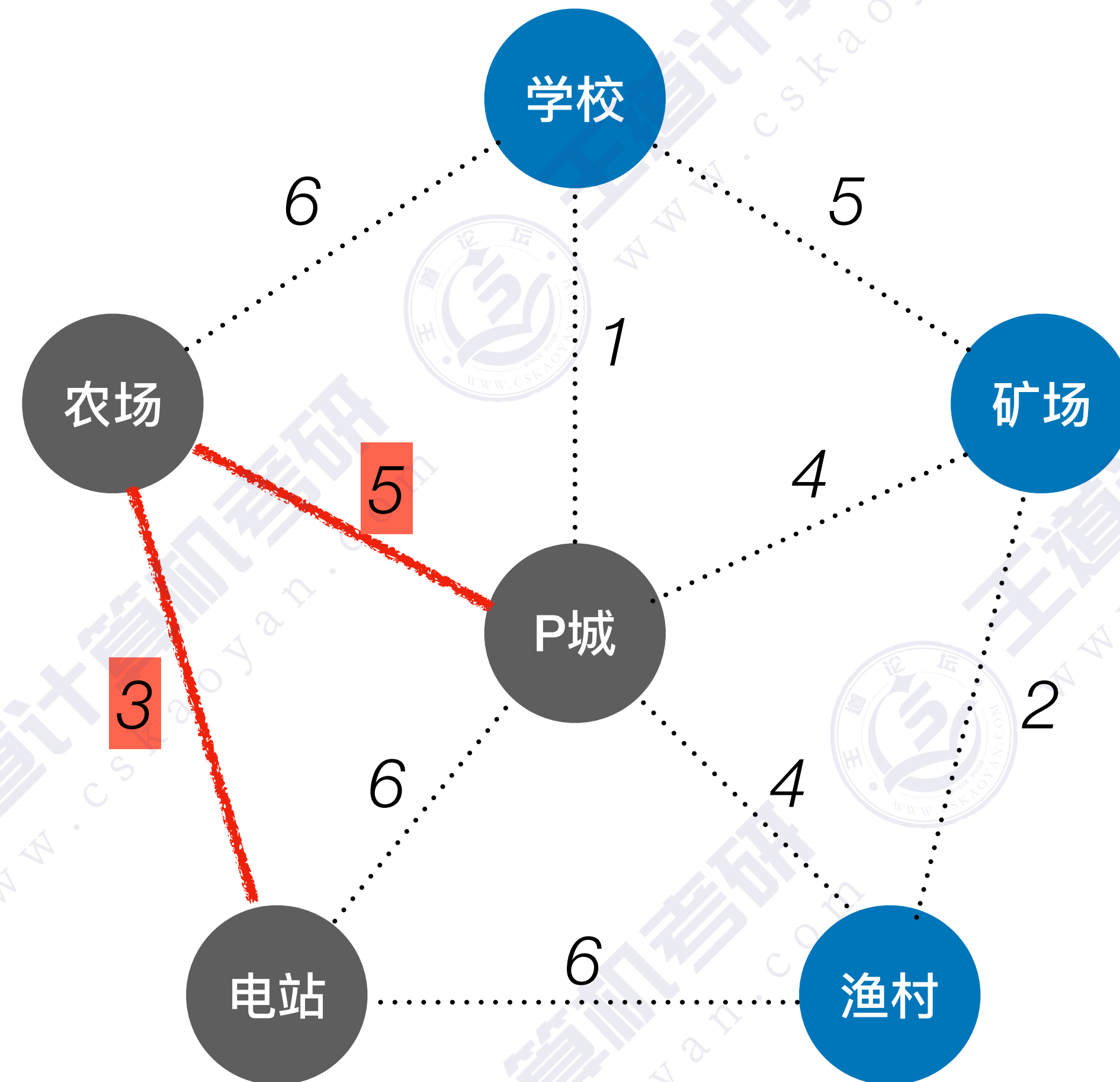
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

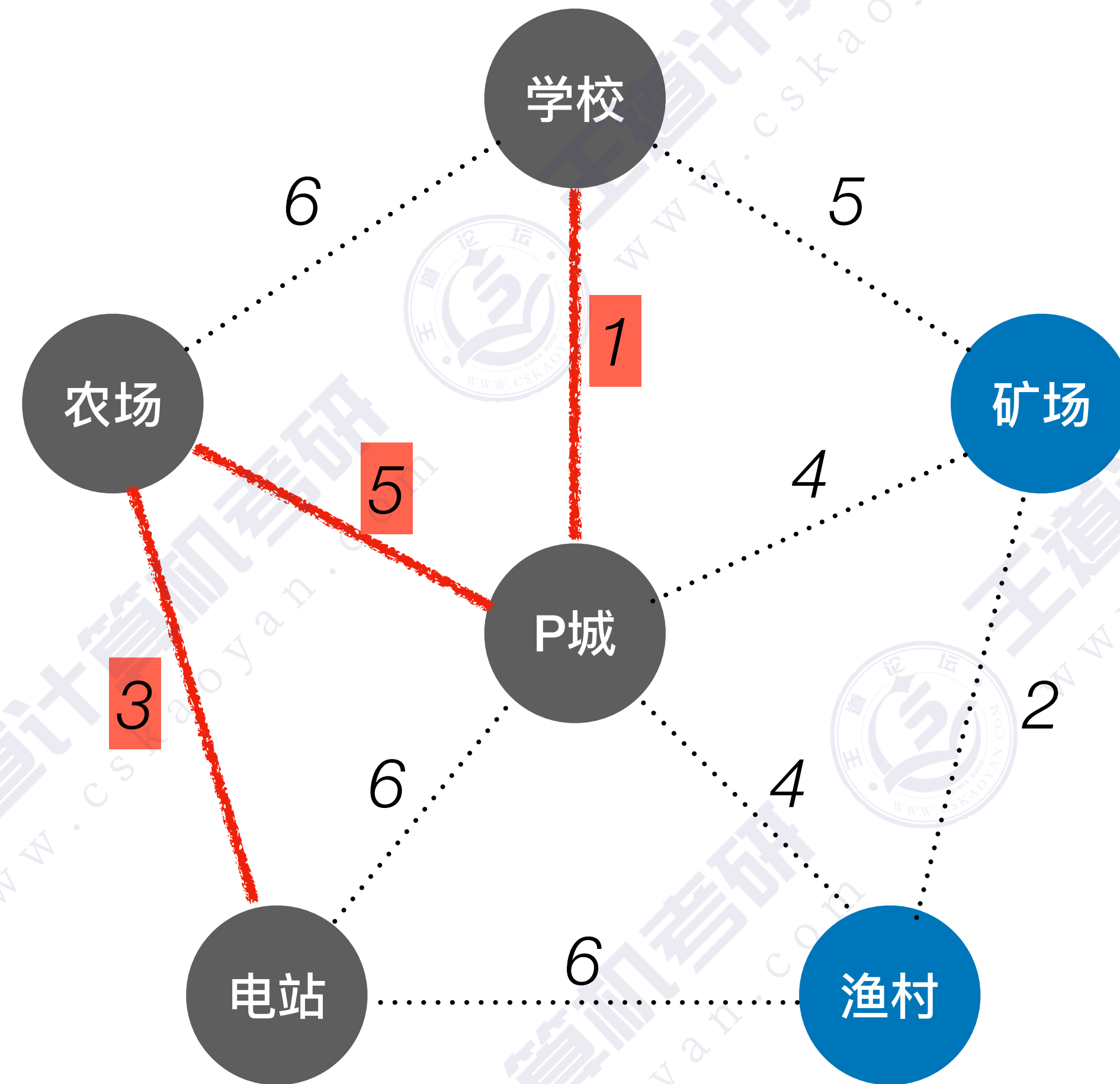
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

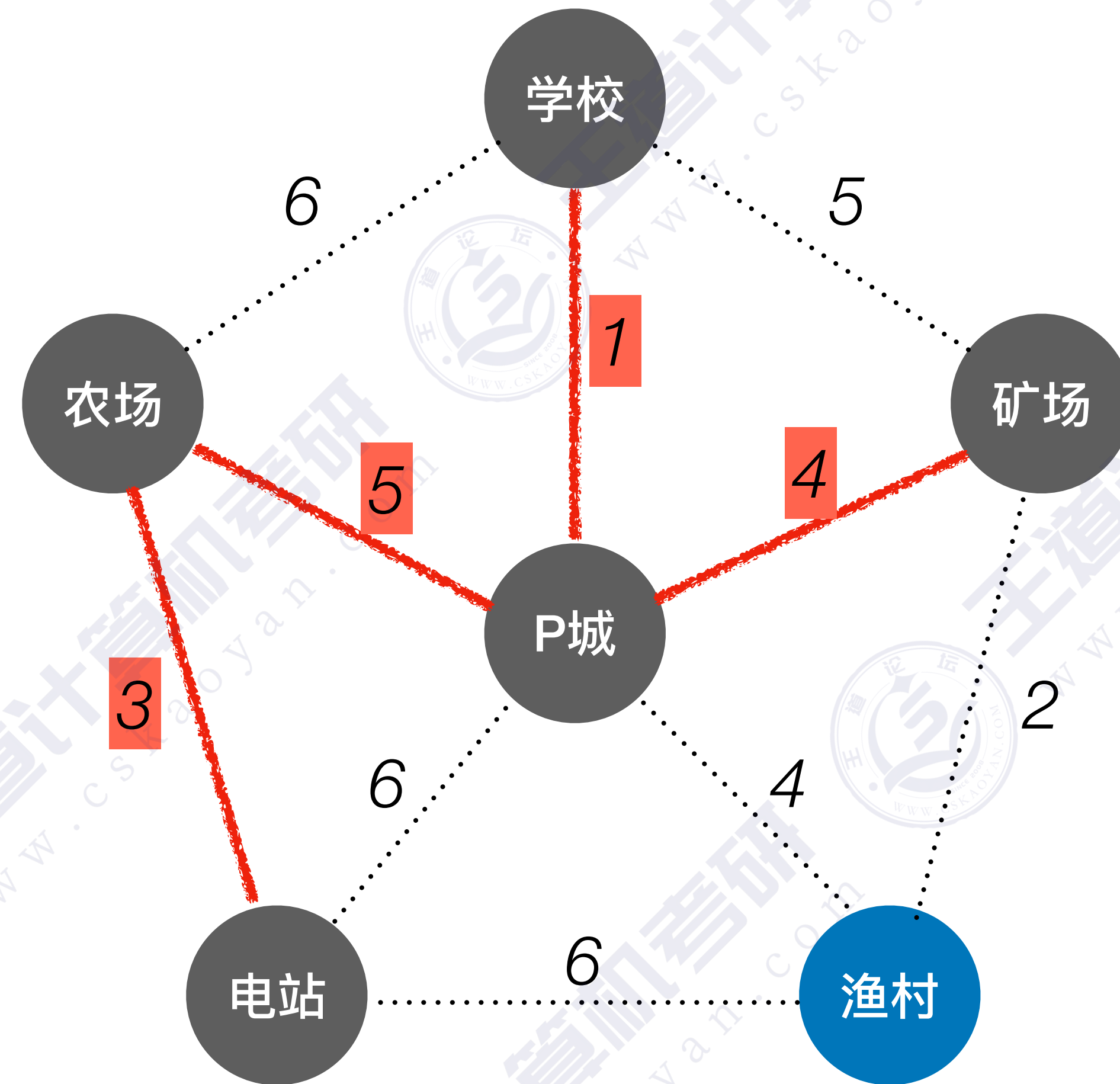
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

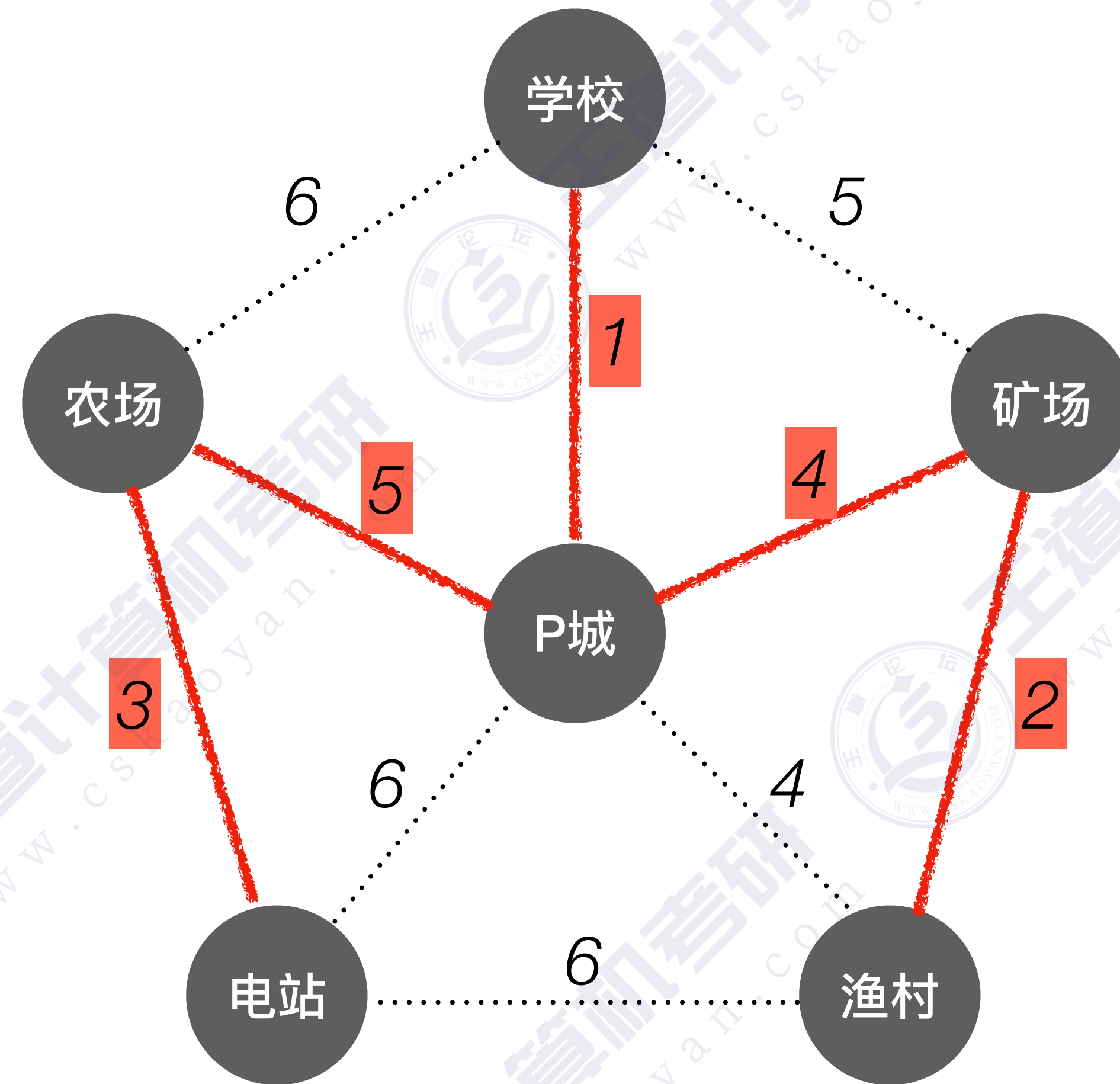
Prim 算法 (普里姆)



Prim 算法 (普里姆) :

从某一个顶点开始构建生成树;
每次将代价最小的新顶点纳入生成树, 直到所有顶点都纳入为止。

Prim 算法（普里姆）

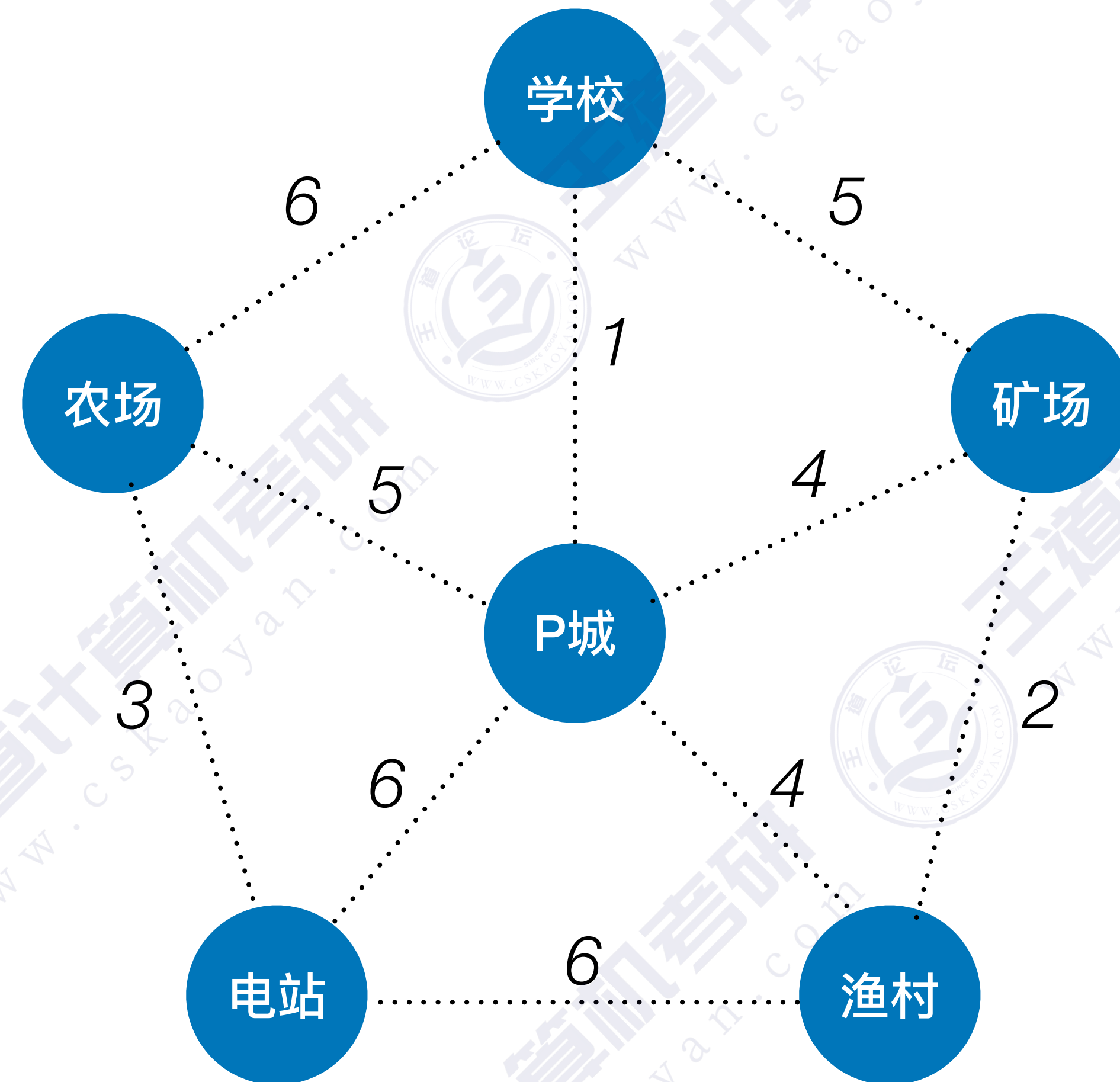


最小代价：15

Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

Kruskal 算法 (克鲁斯卡尔)



Kruskal 算法 (克鲁斯卡尔) :

每次选择一条权值最小的边, 使这条边的两头连通 (原本已经连通的就不选)

直到所有结点都连通

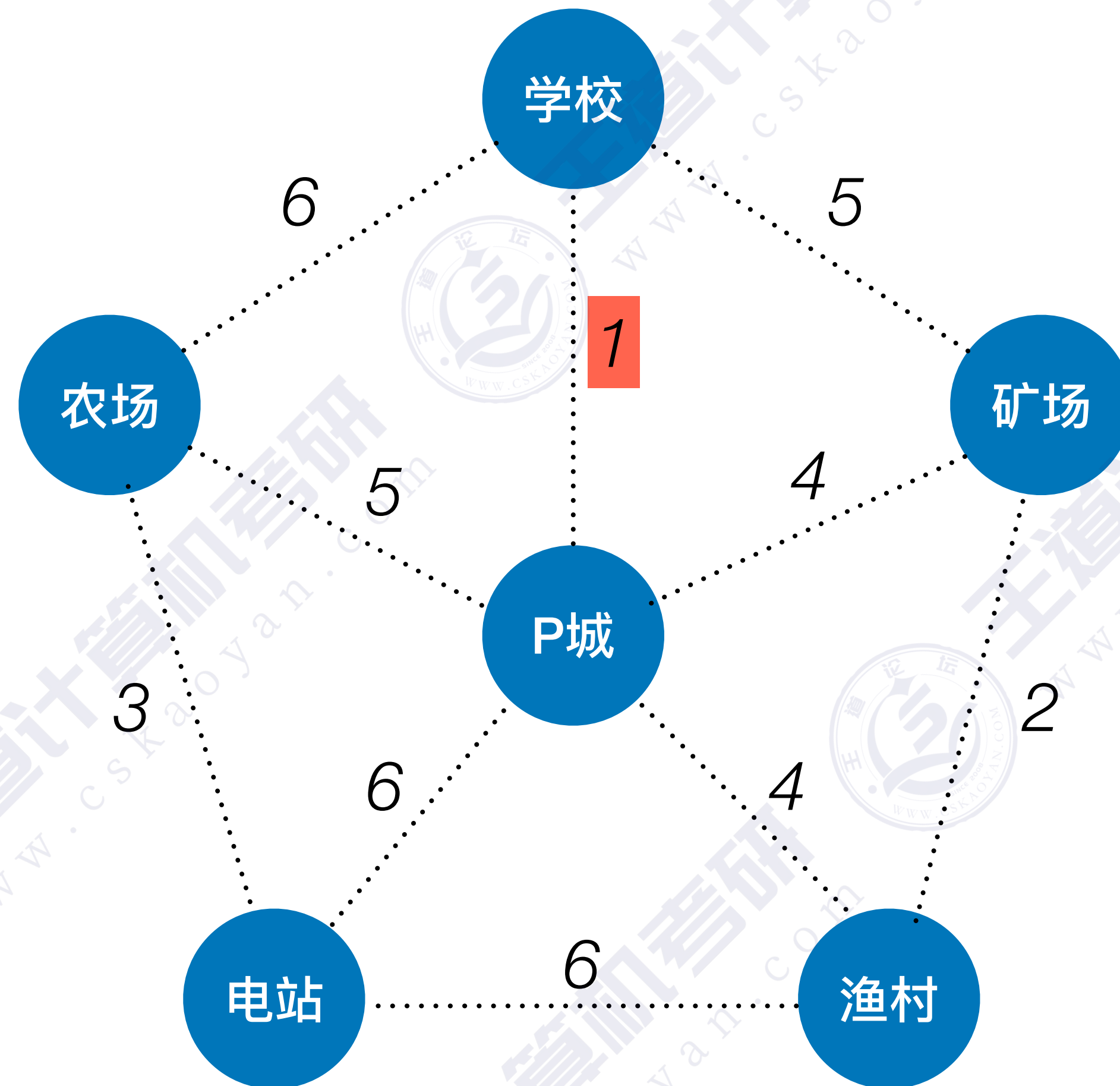
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



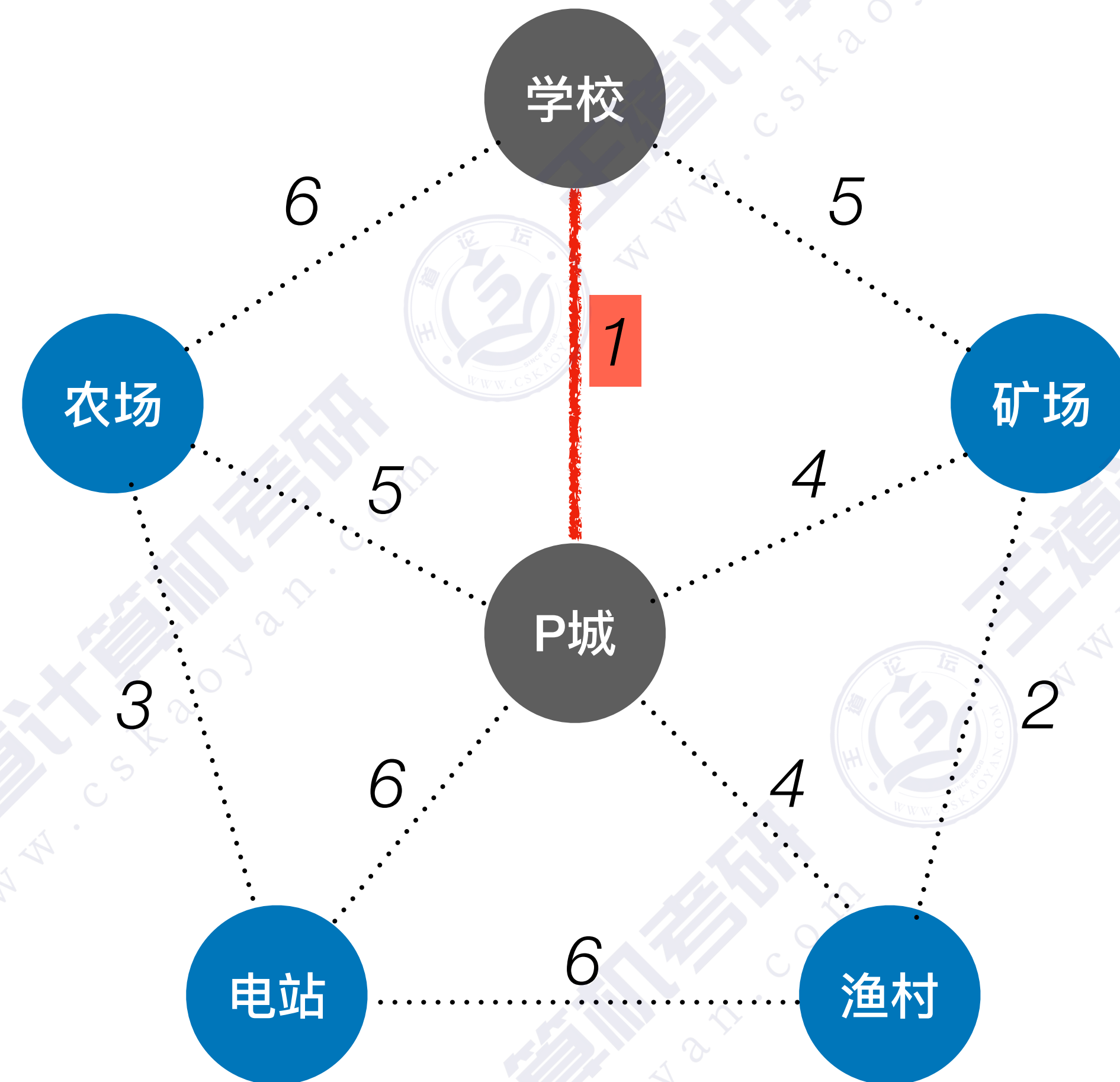
Kruskal 算法 (克鲁斯卡尔)



Kruskal 算法 (克鲁斯卡尔) :

每次选择一条权值最小的边, 使这条边的两头连通 (原本已经连通的就不选)

直到所有结点都连通



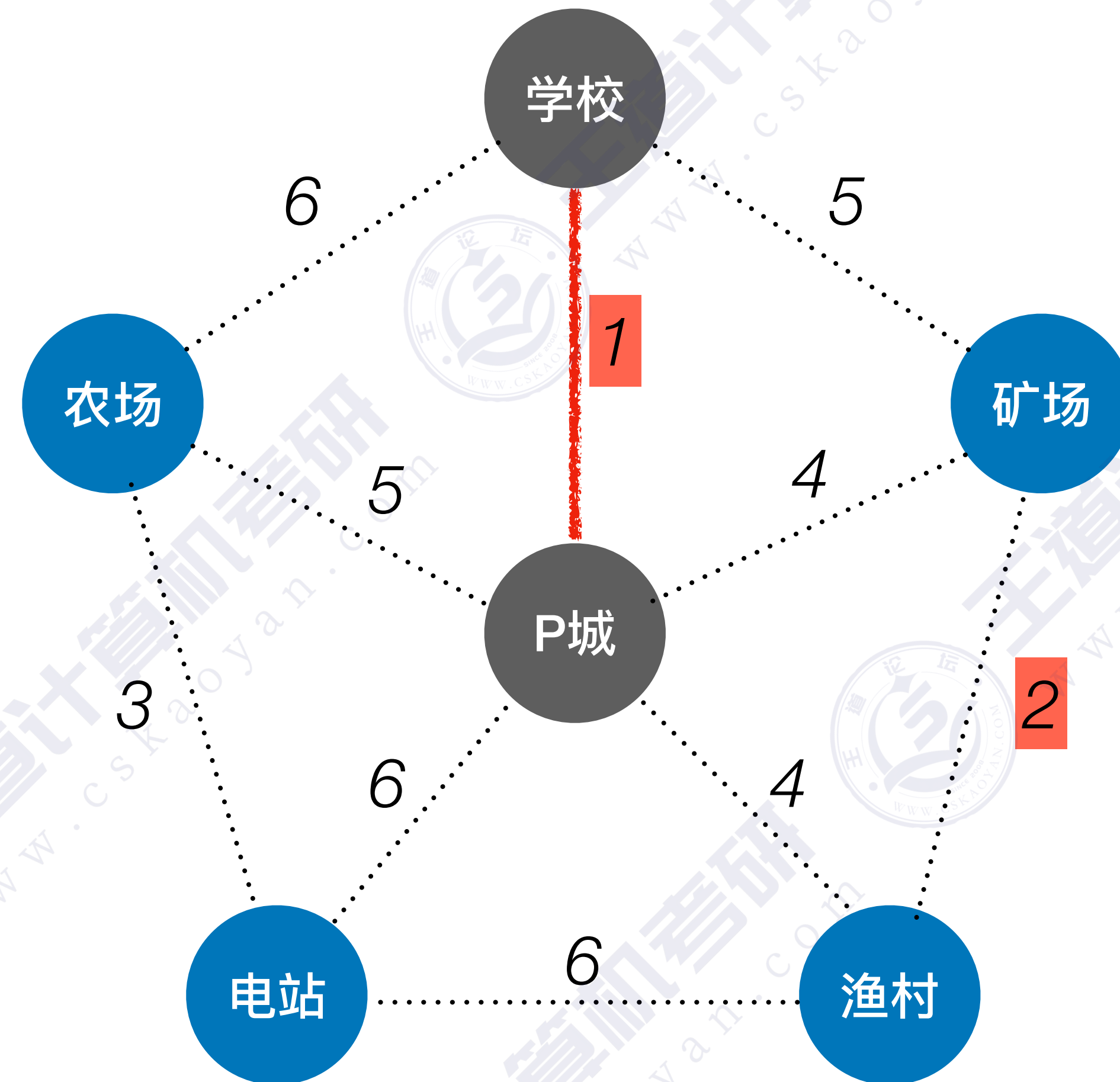
Kruskal 算法（克鲁斯卡尔）



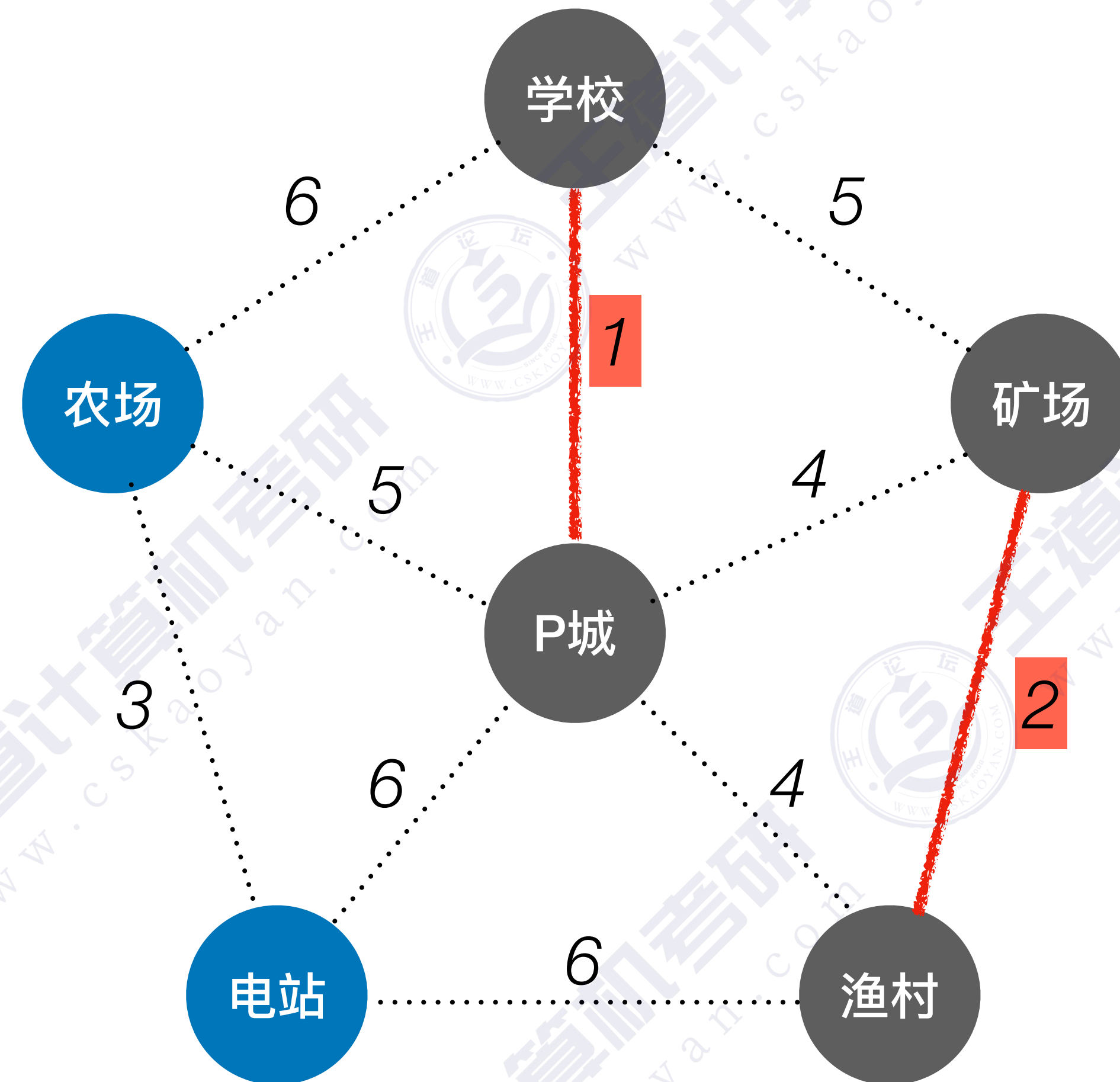
Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通

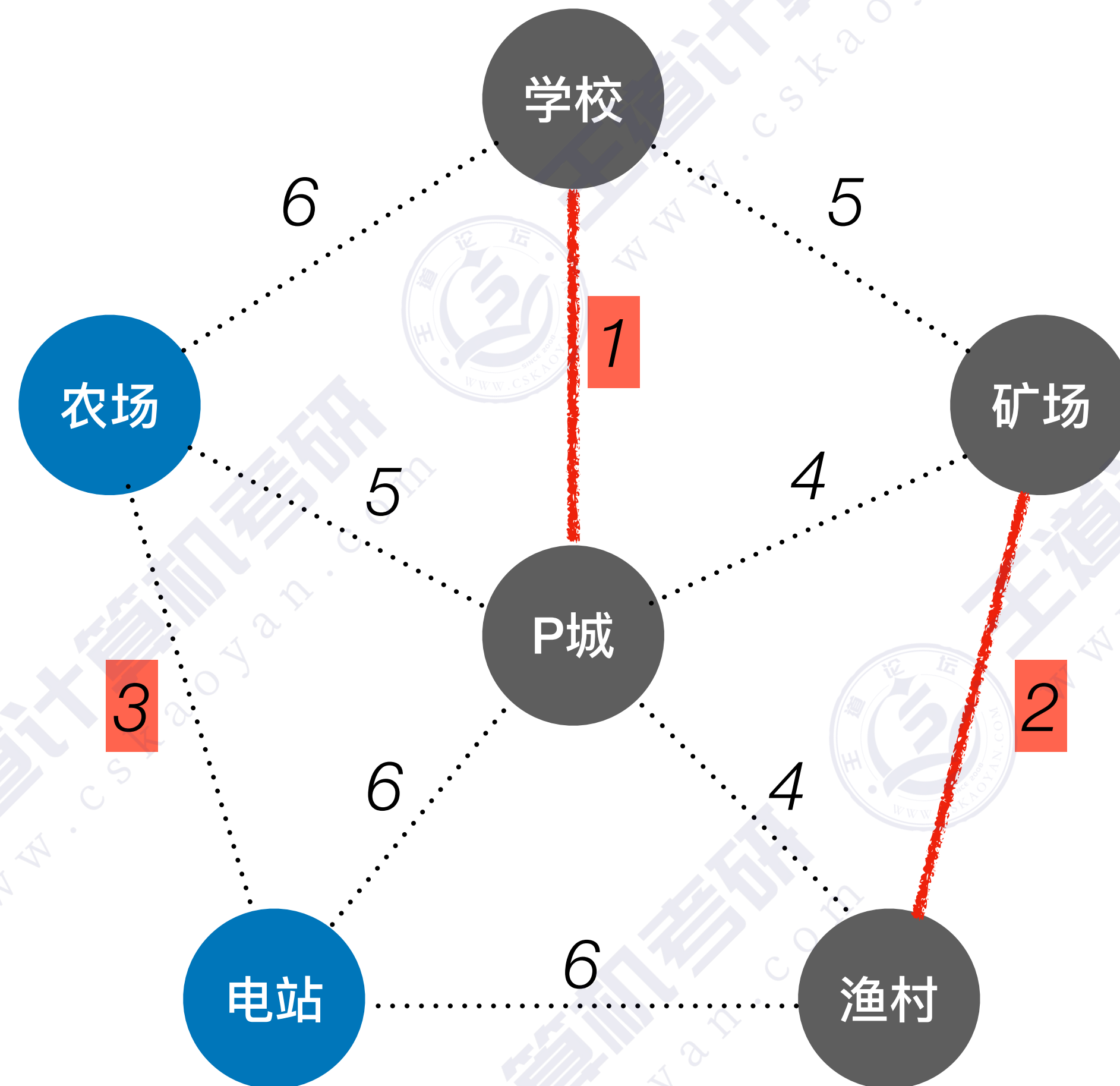
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



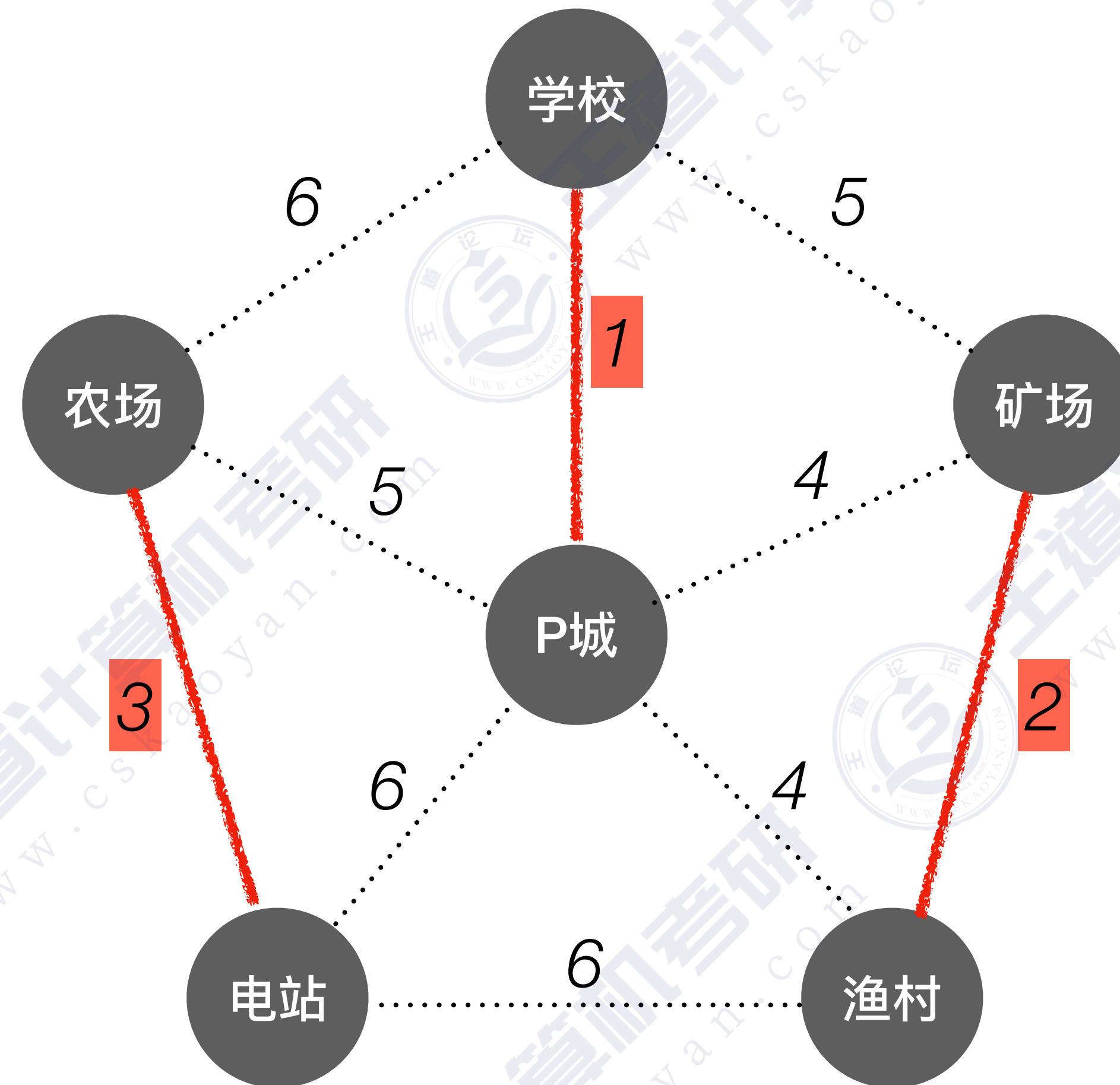
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



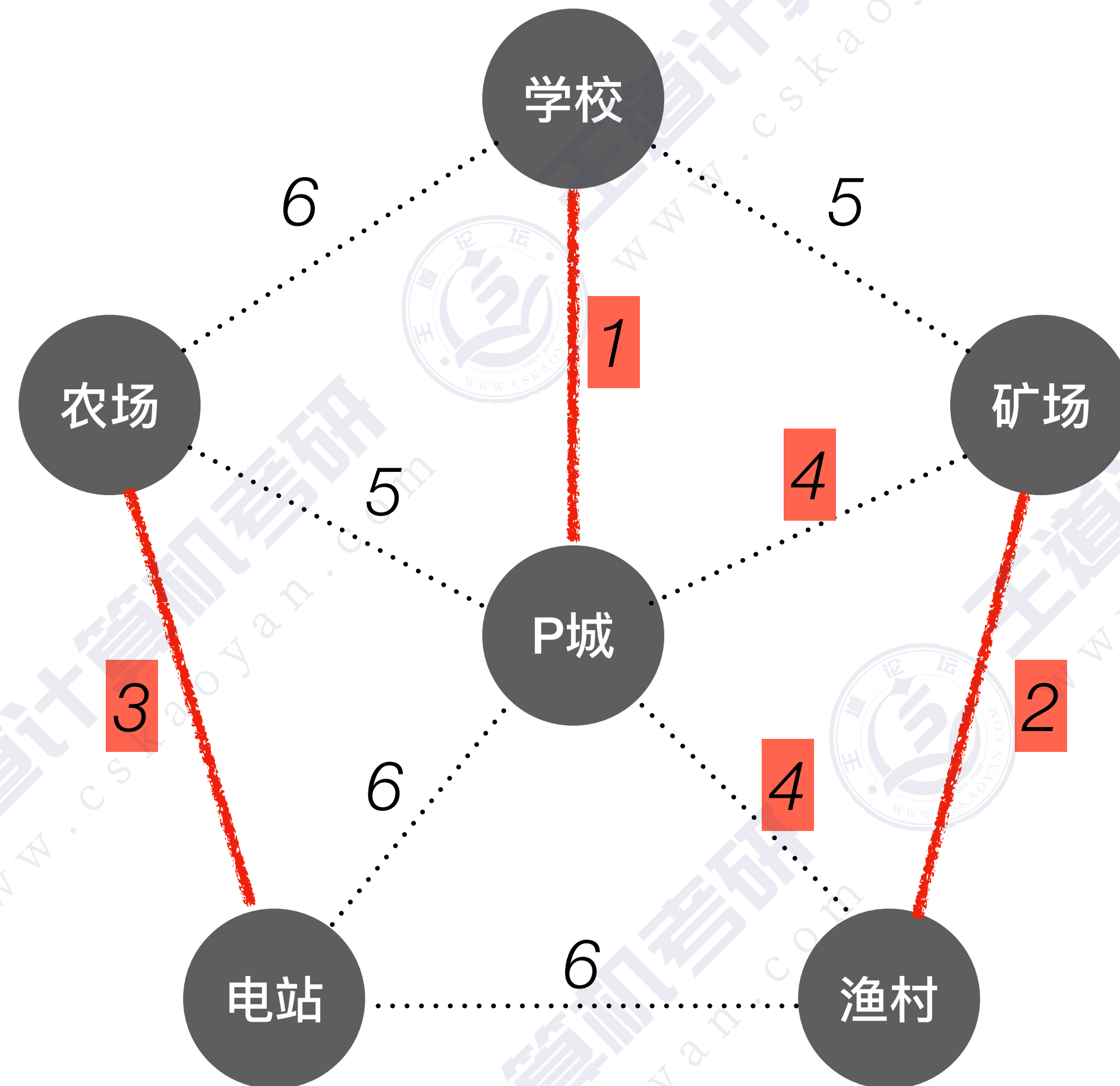
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



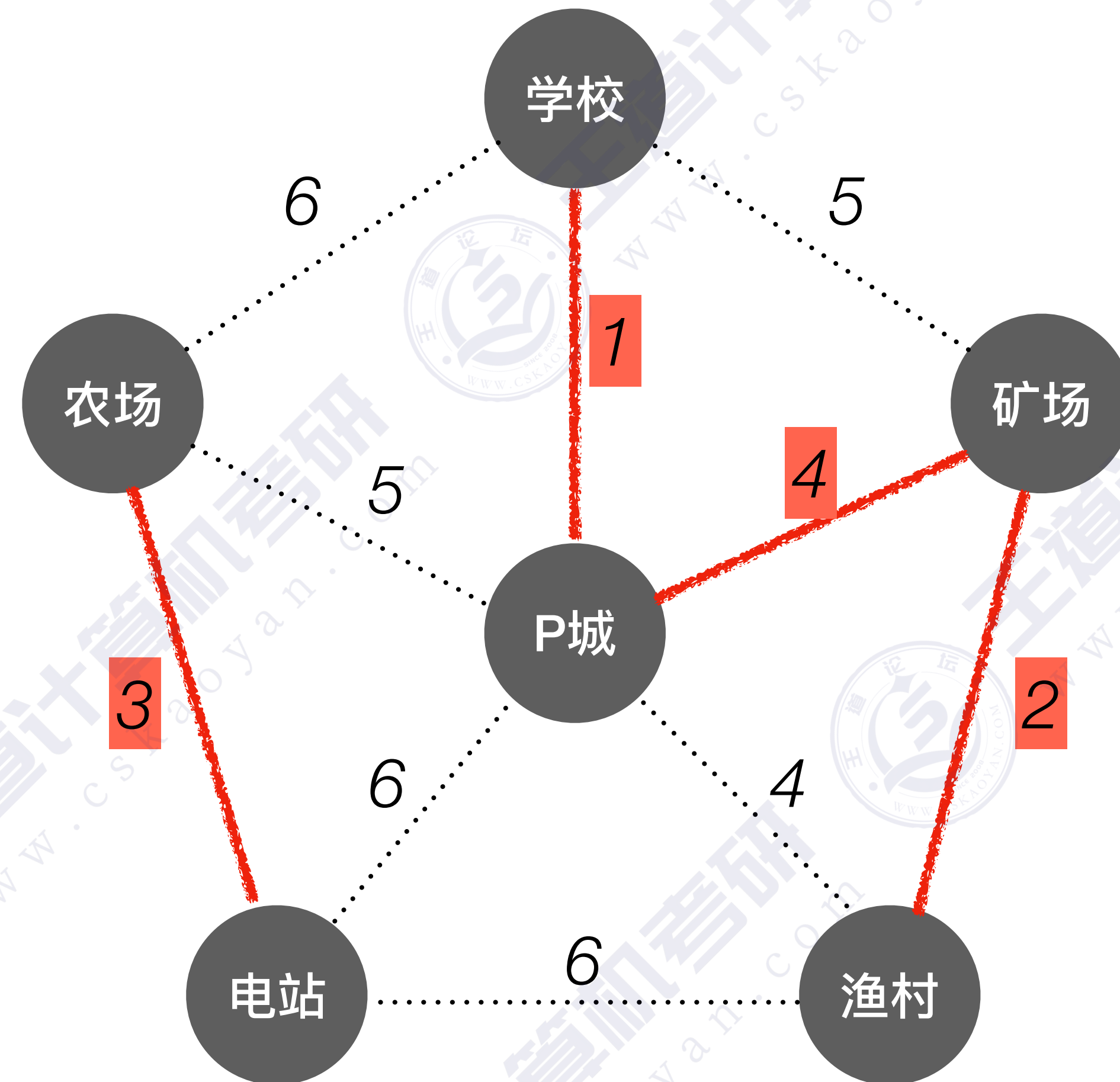
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



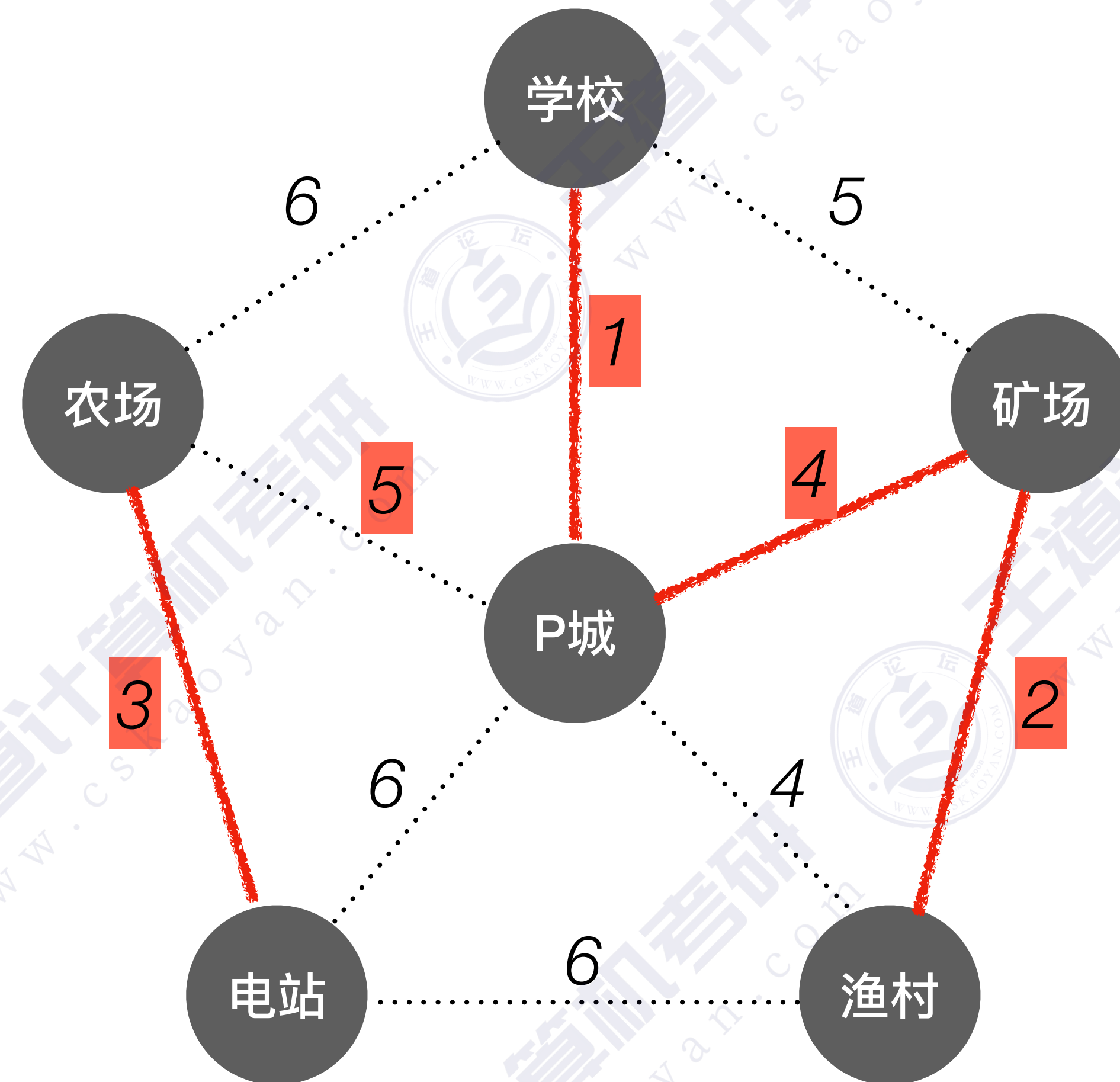
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



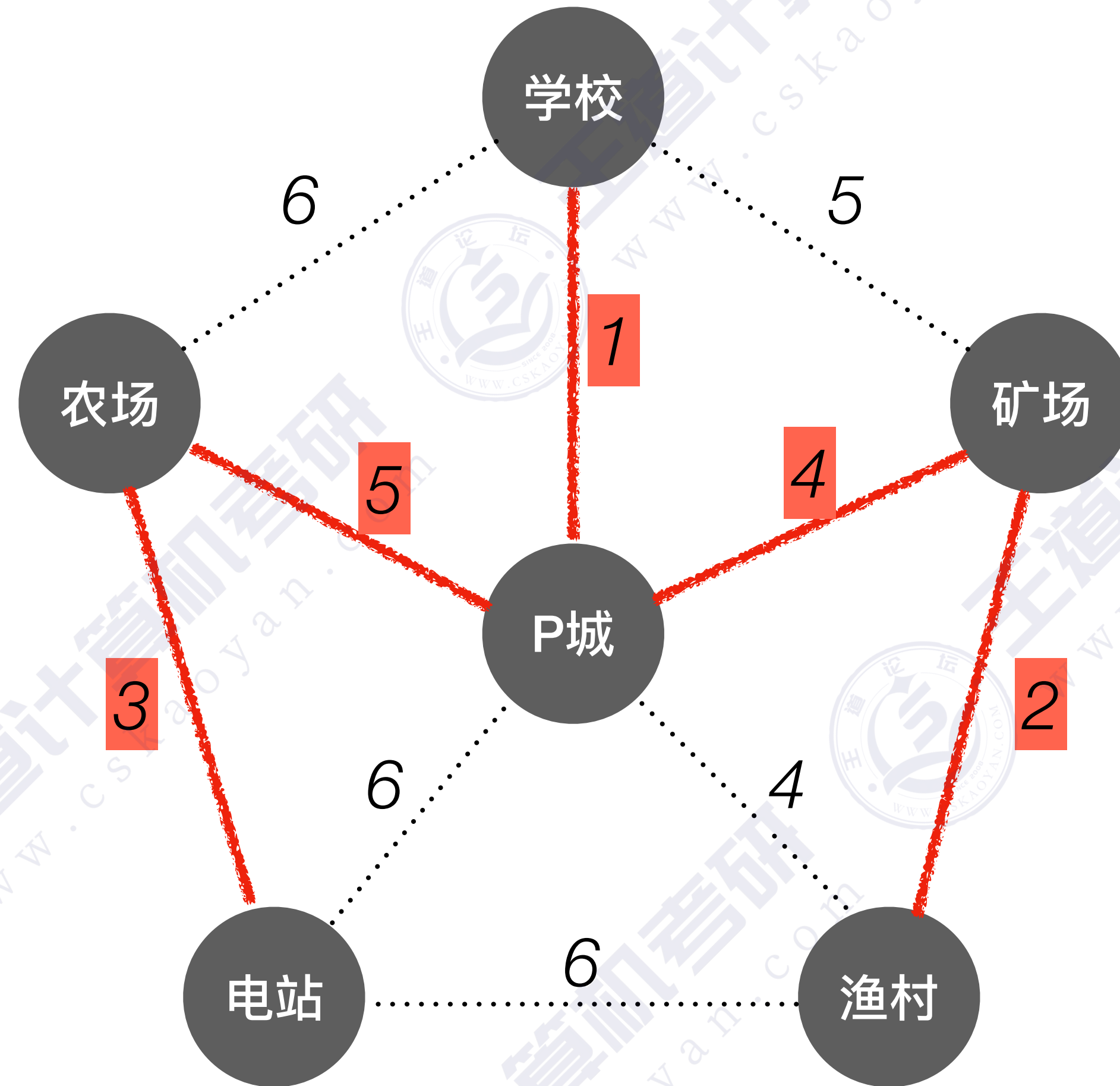
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



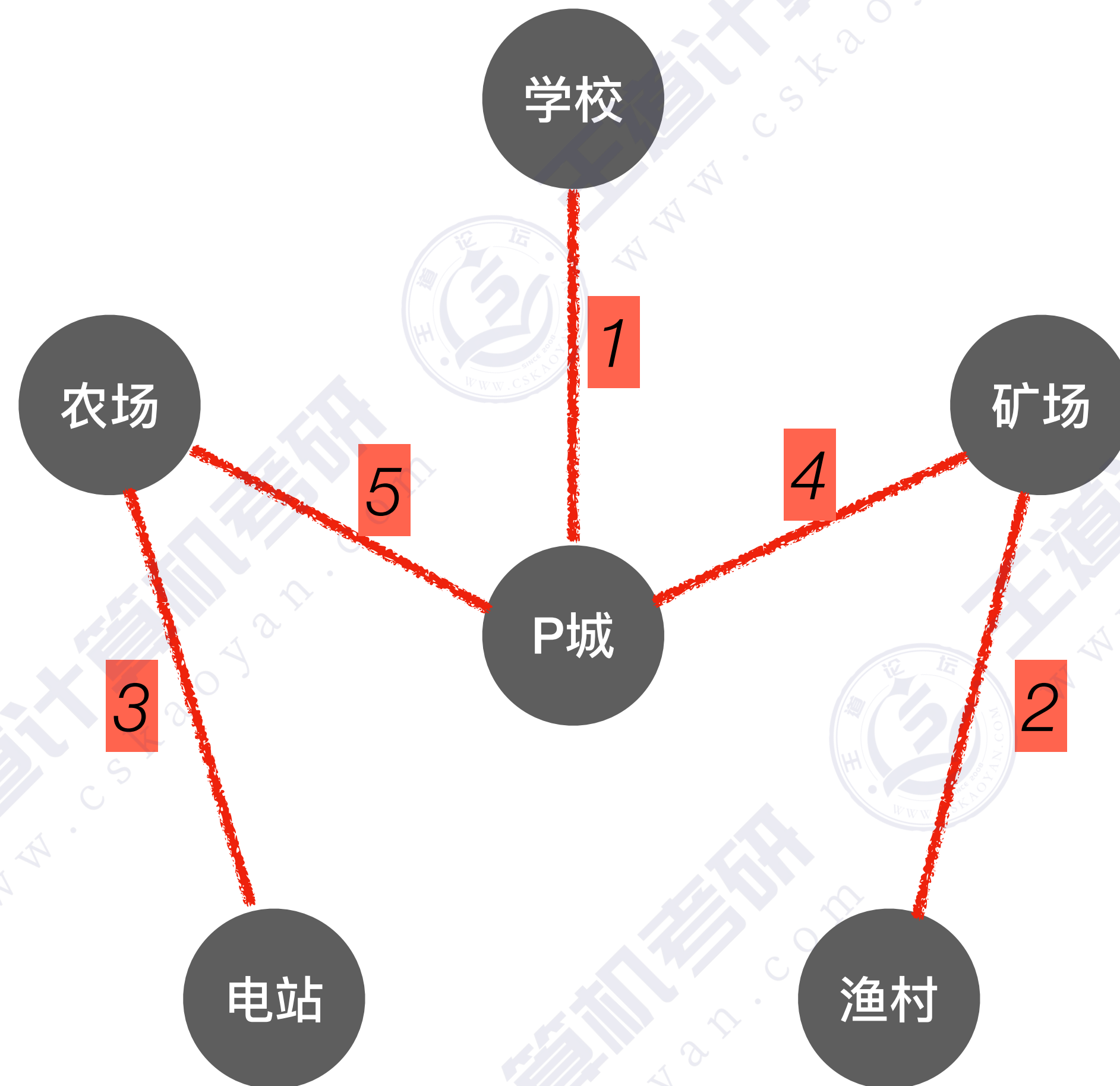
Kruskal 算法（克鲁斯卡尔）



Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）

直到所有结点都连通



最小代价：15

Prim 算法 v.s. Kruskal 算法

Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

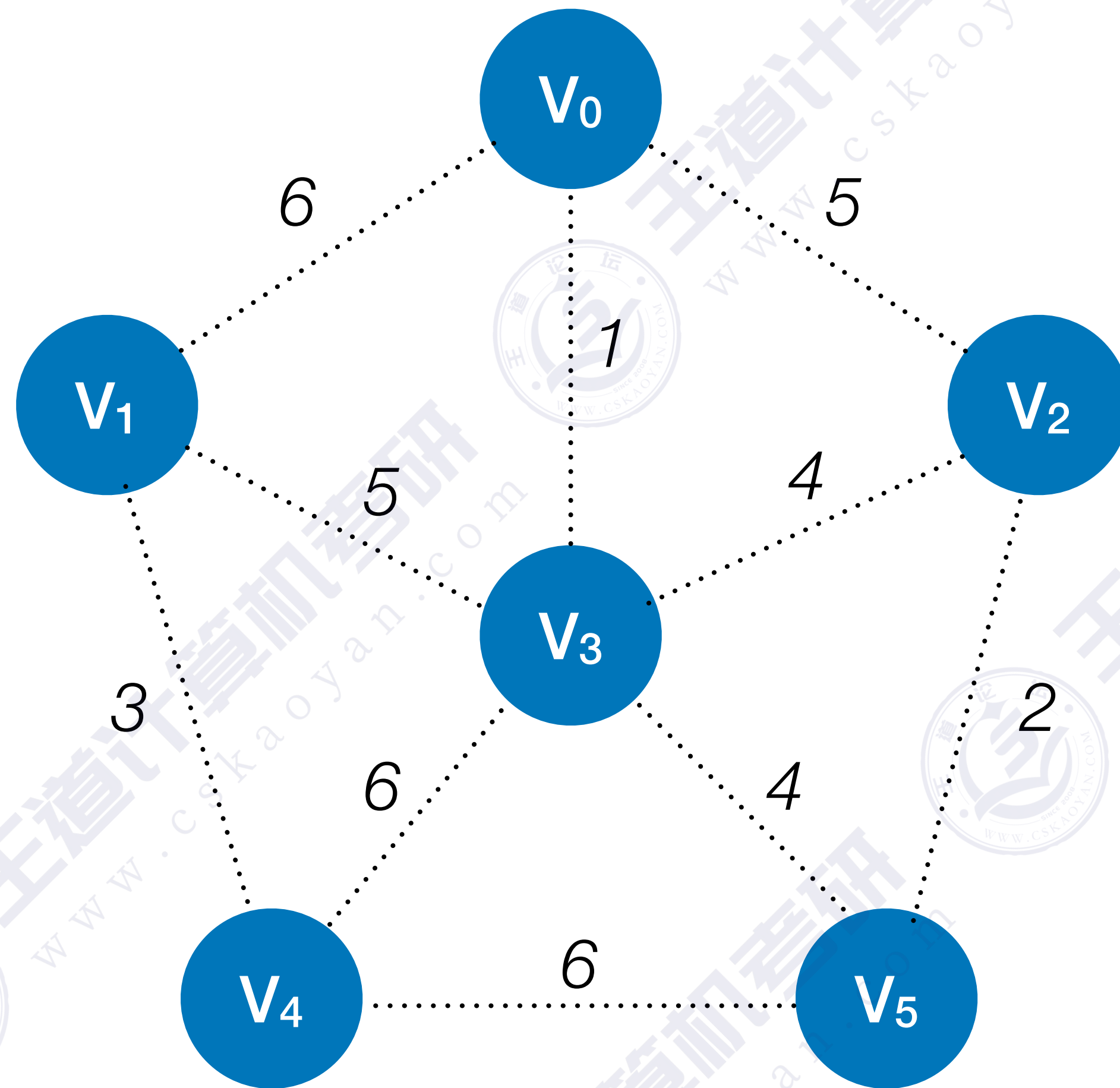
时间复杂度： $O(|V|^2)$
适合用于边稠密图

Kruskal 算法（克鲁斯卡尔）：

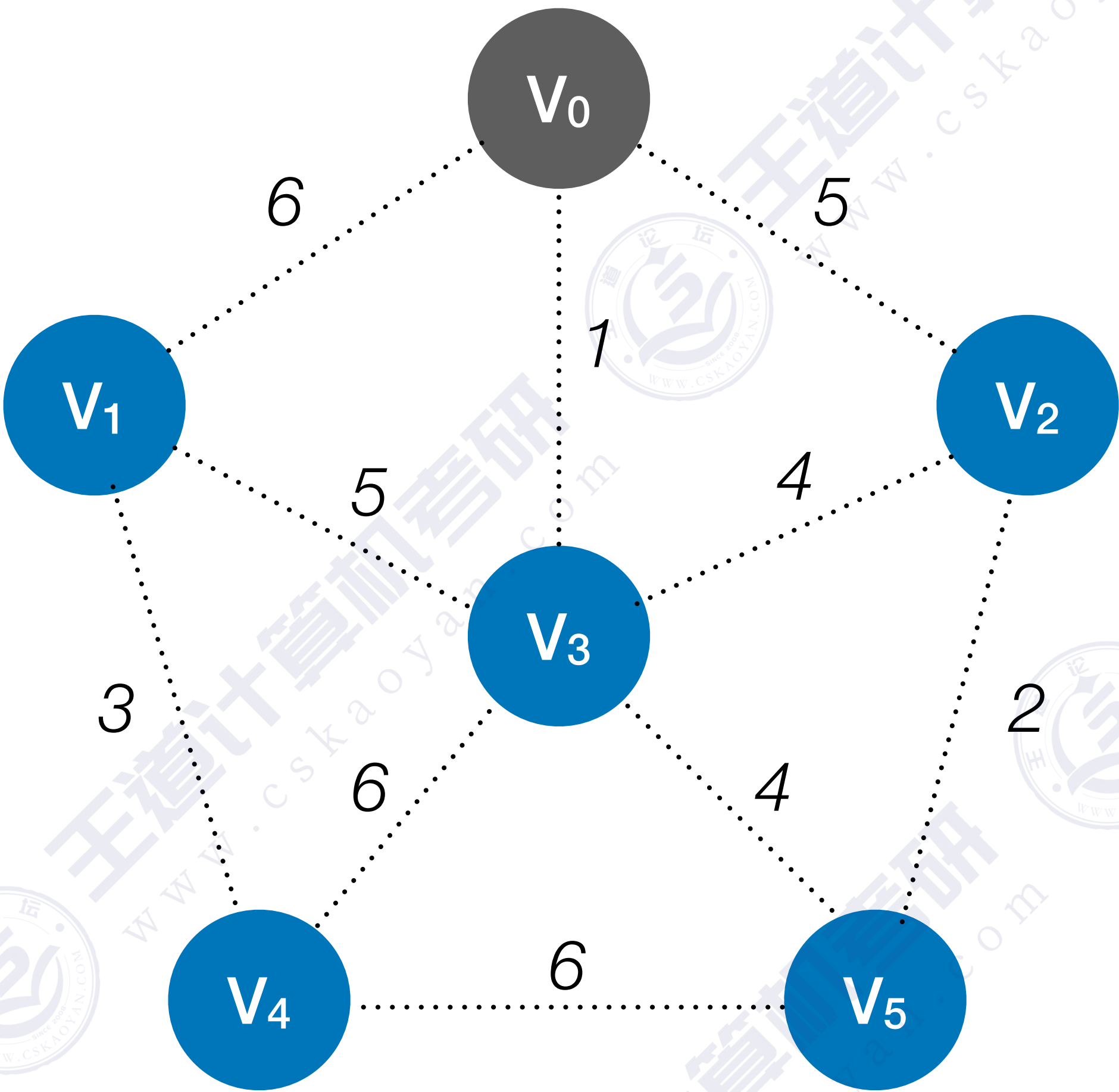
每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）
直到所有结点都连通

时间复杂度： $O(|E|\log_2|E|)$
适合用于边稀疏图

Prim 算法的实现思想



Prim 算法的实现思想



初始：从V₀开始

标记各节点是否已加入树

isJoin[6]

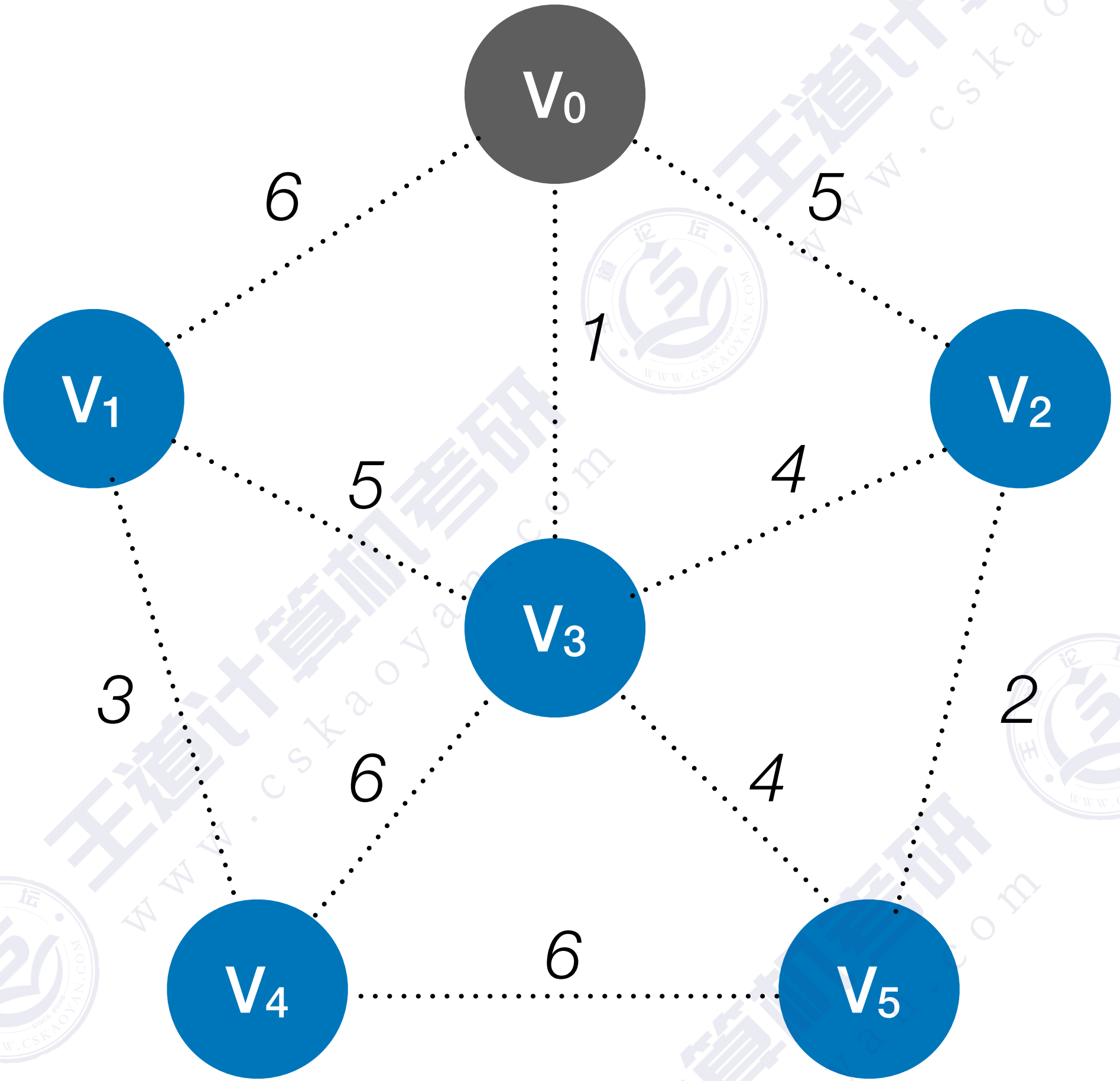
V0	V1	V2	V3	V4	V5
✓	✗	✗	✗	✗	✗

lowCost[6]

各节点加入树的最低代价

0	6	5	1	∞	∞
---	---	---	---	---	---

Prim 算法的实现思想



第1轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

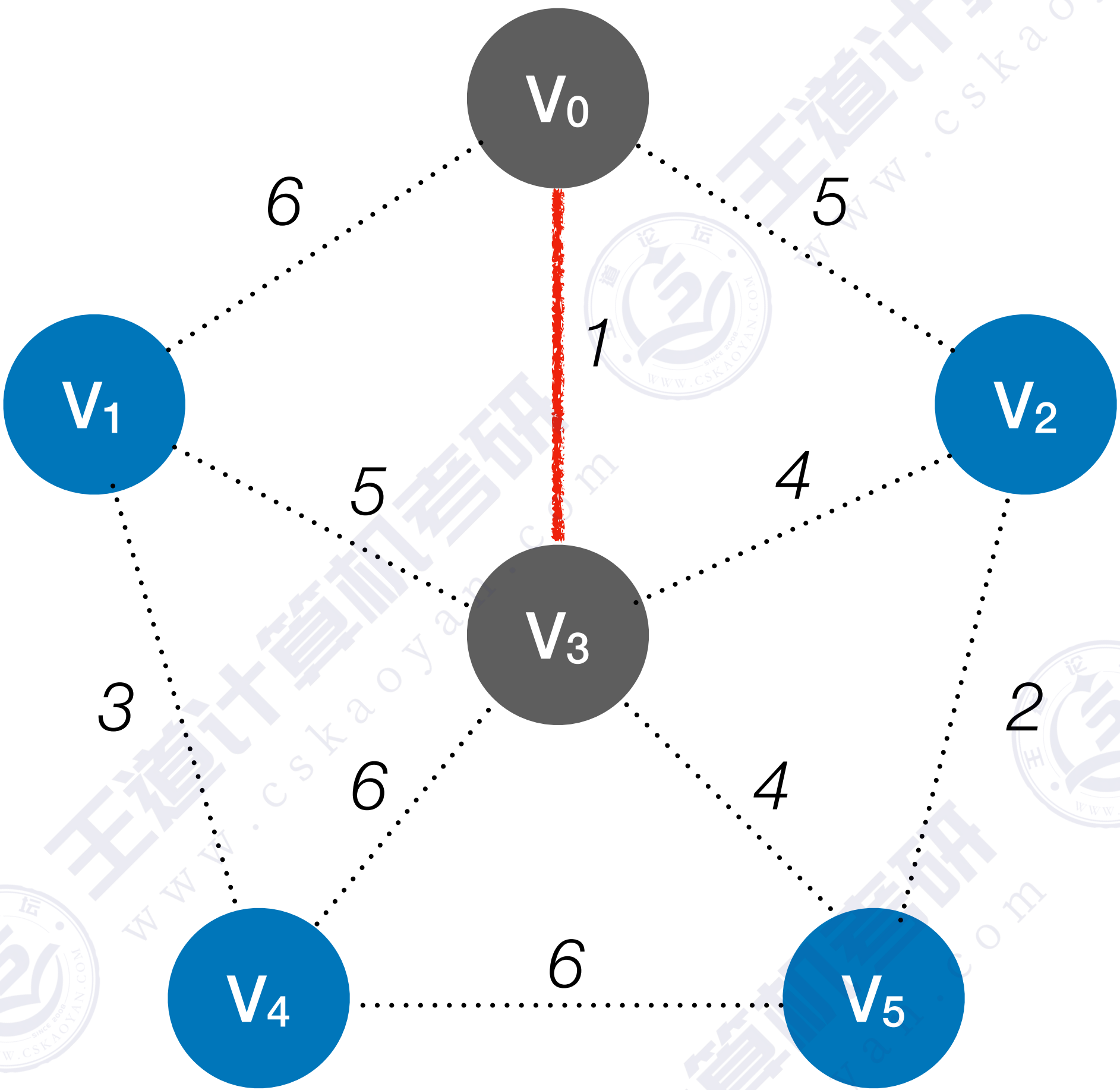
V0	V1	V2	V3	V4	V5
✓	✗	✗	✗	✗	✗

lowCost[6]

0	6	5	1	∞	∞
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第1轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗

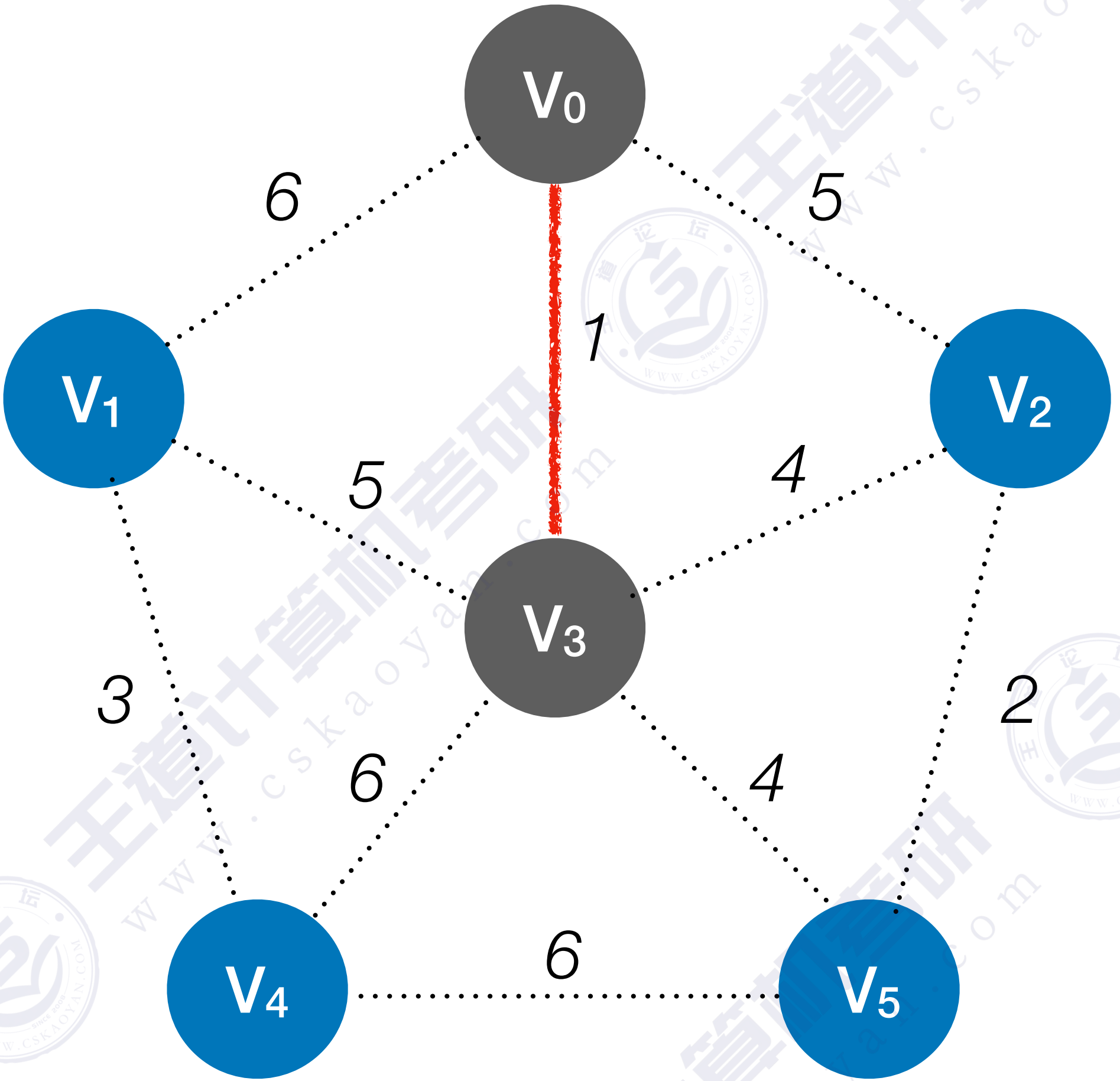


lowCost[6]

0	6	5	1	∞	∞
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第1轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗



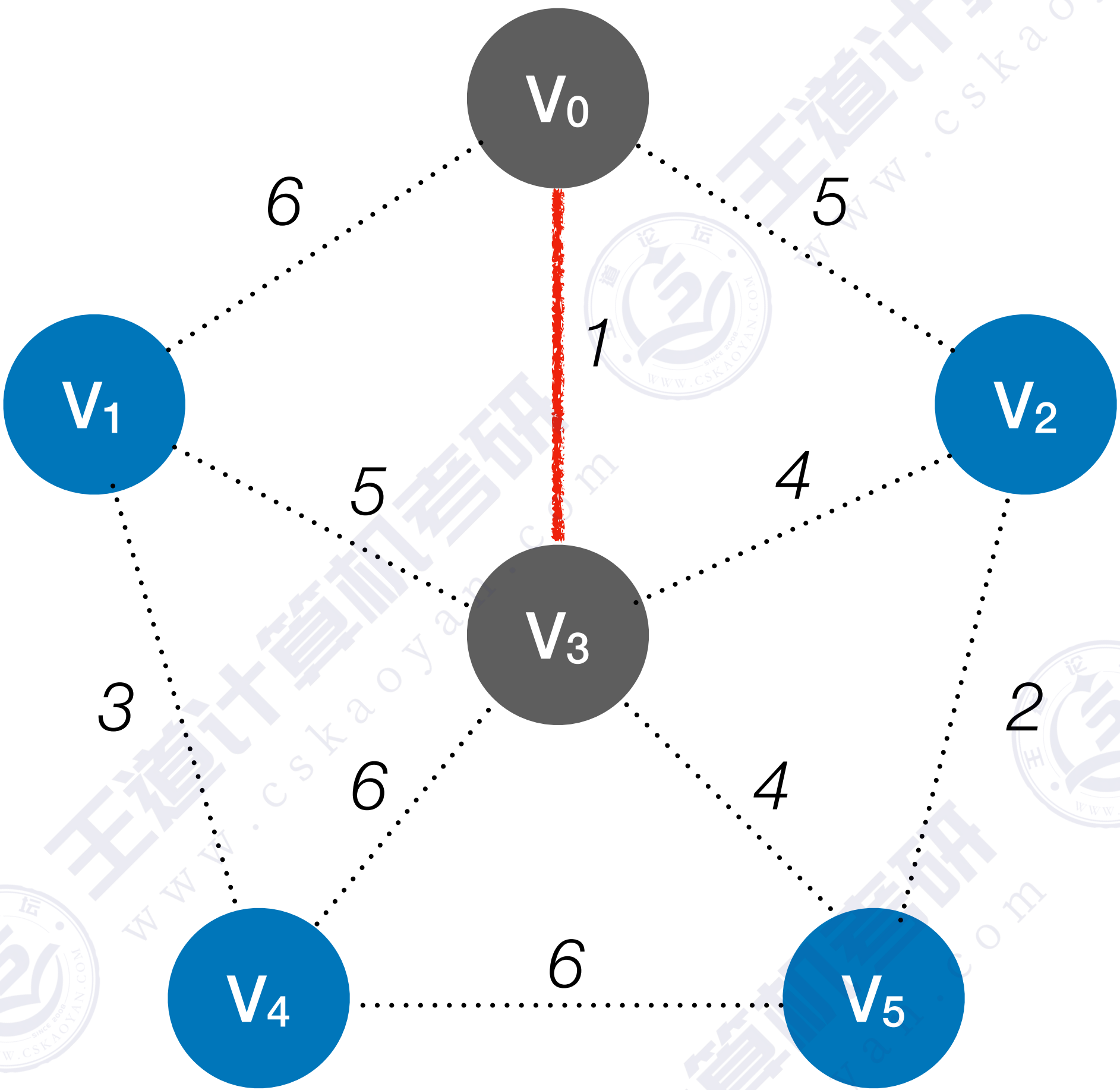
lowCost[6]

0	6	5	1	∞	∞
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第1轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗



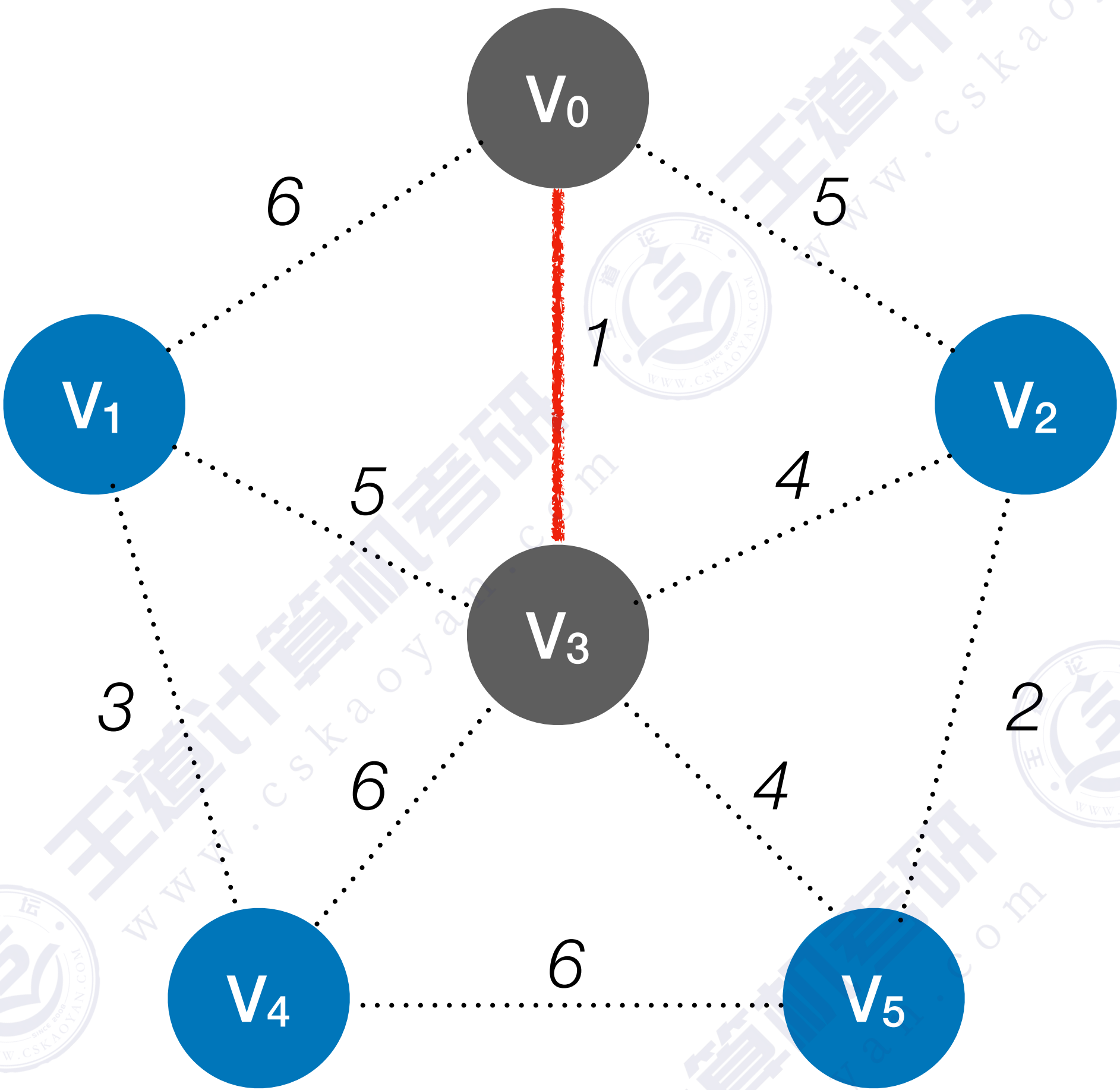
lowCost[6]

0	5	4	1	6	4
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第2轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

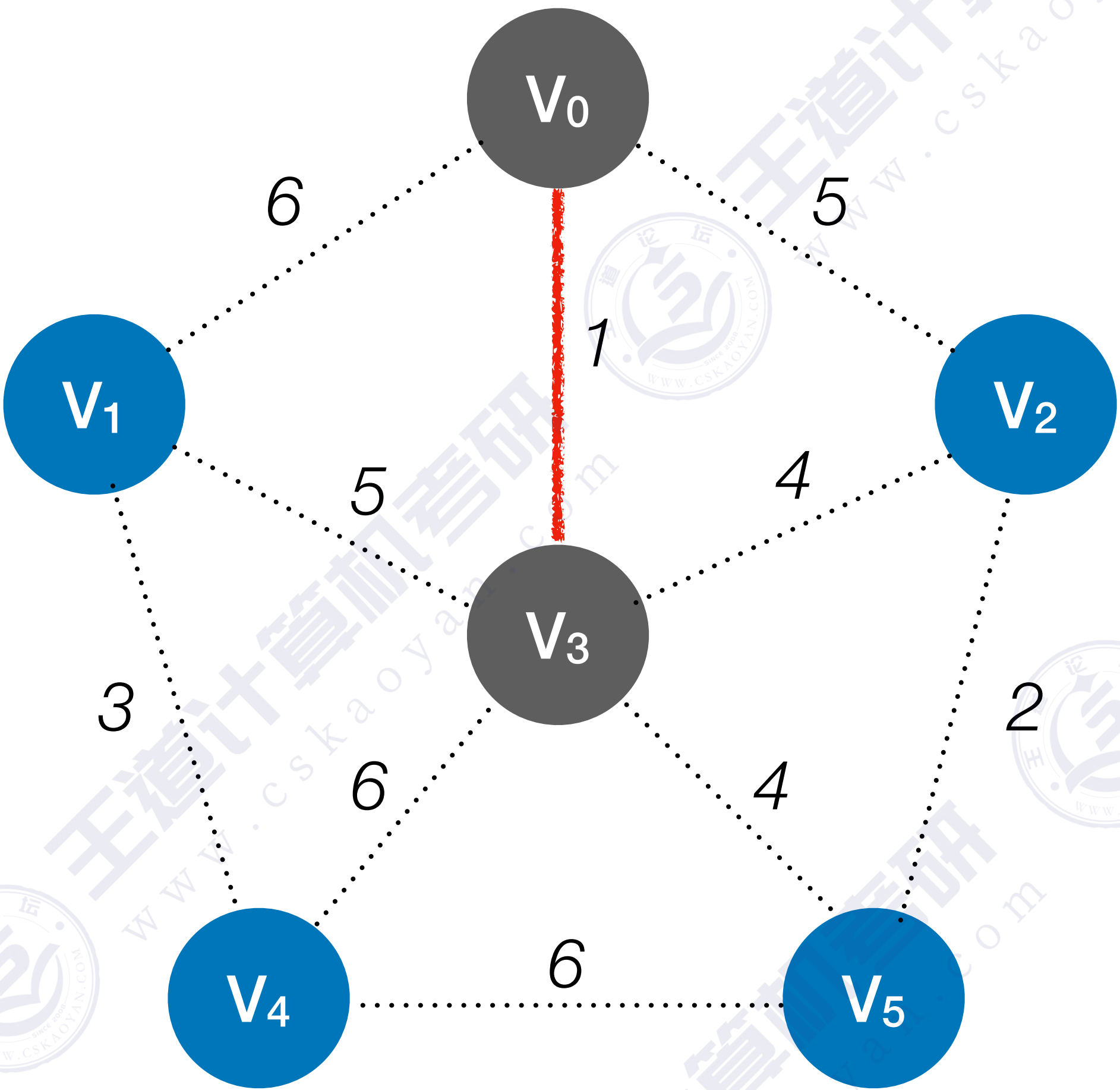
V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗

lowCost[6]

0	5	4	1	6	4
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第2轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

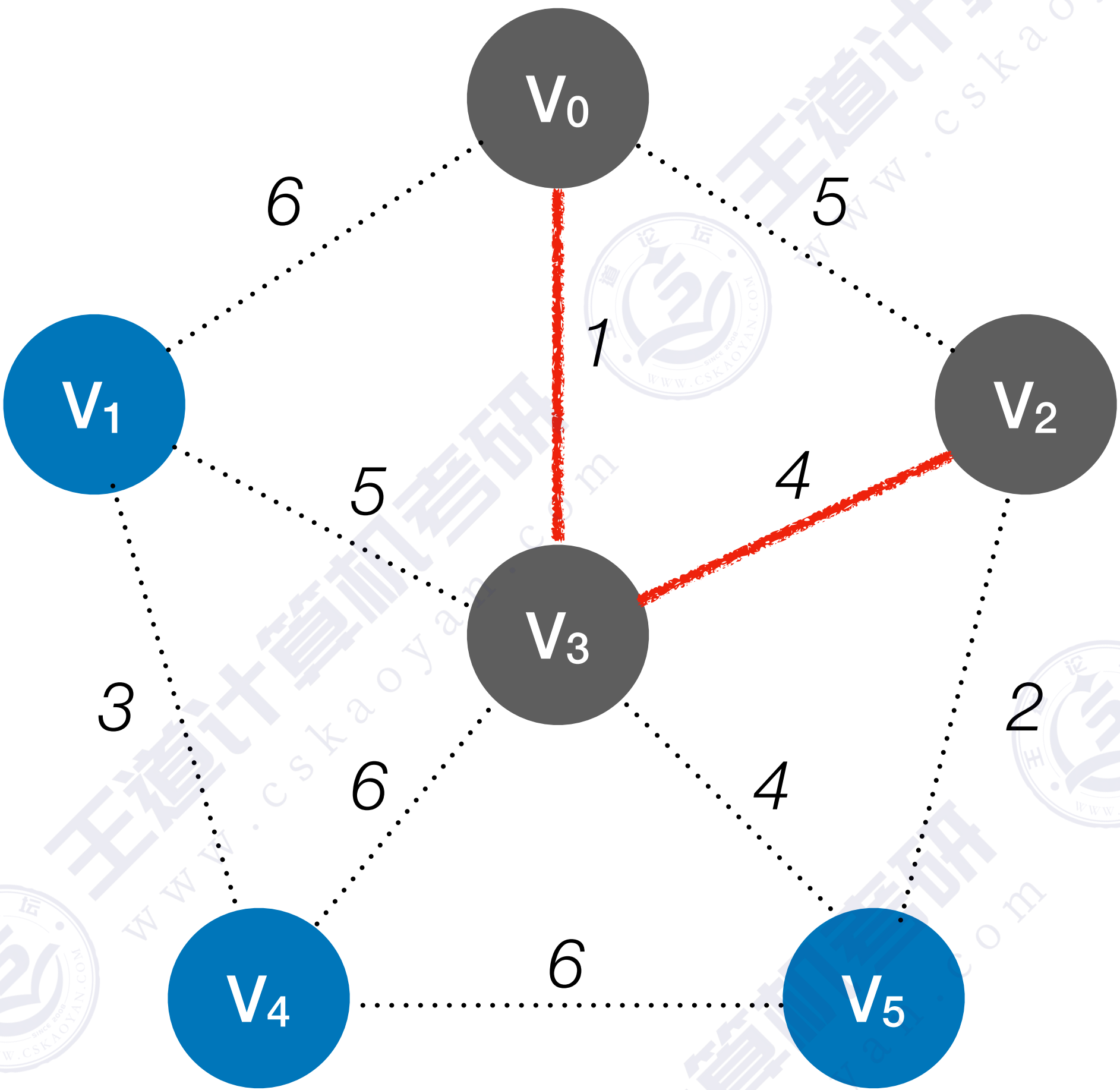
V0	V1	V2	V3	V4	V5
✓	✗	✗	✓	✗	✗

lowCost[6]

0	5	4	1	6	4
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第2轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗



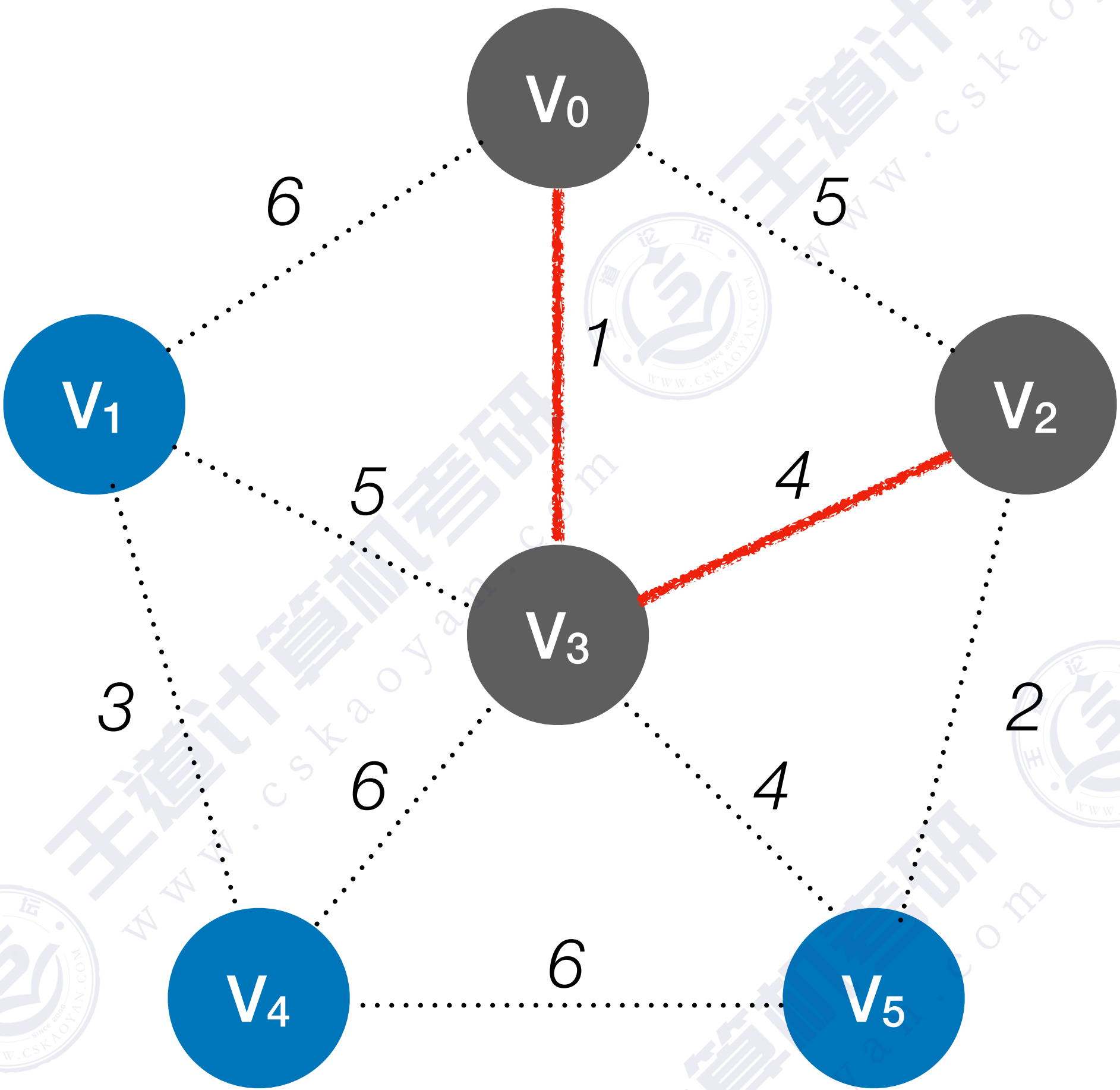
lowCost[6]

0	5	4	1	6	4
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第2轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗



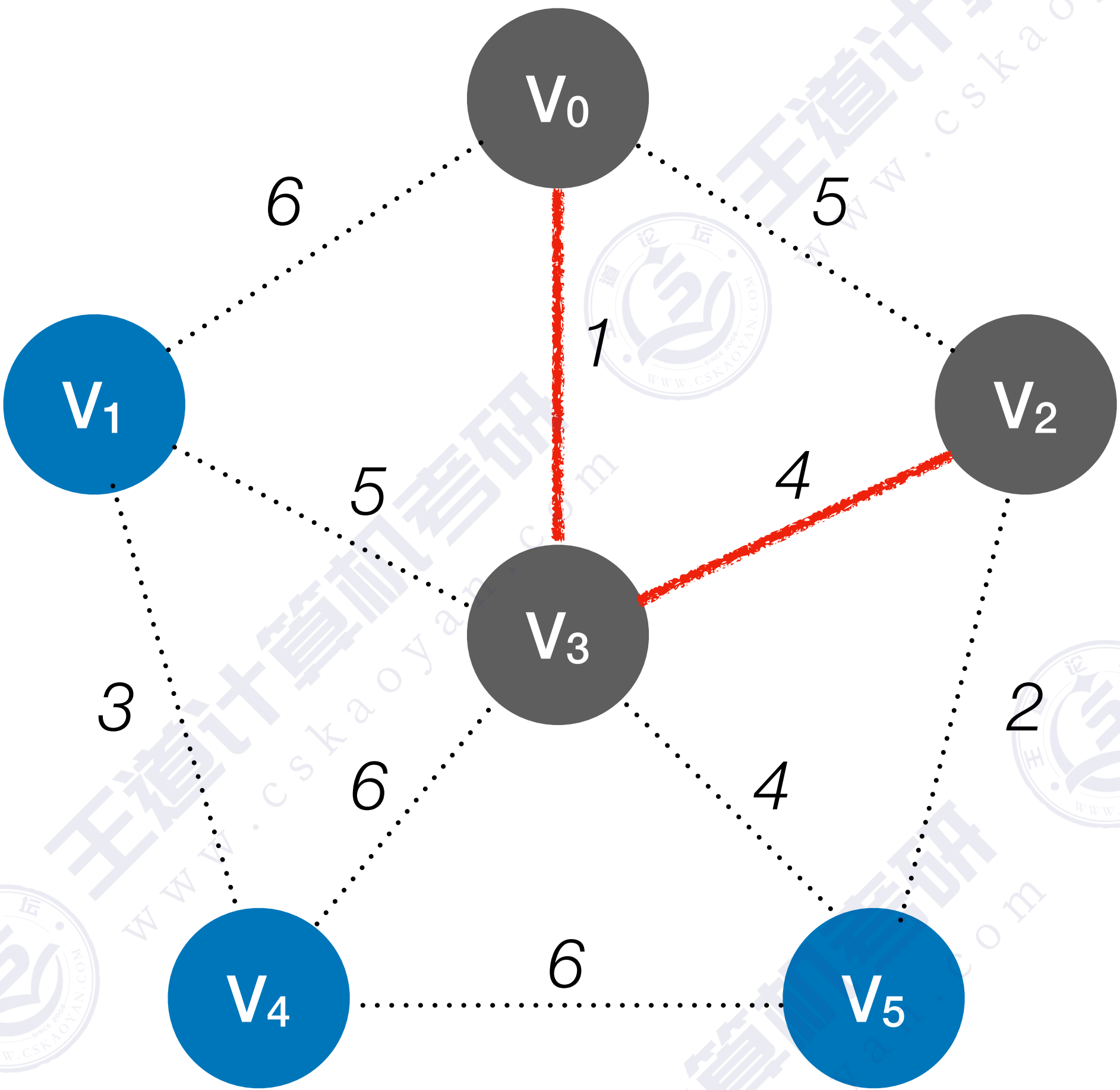
lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第3轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

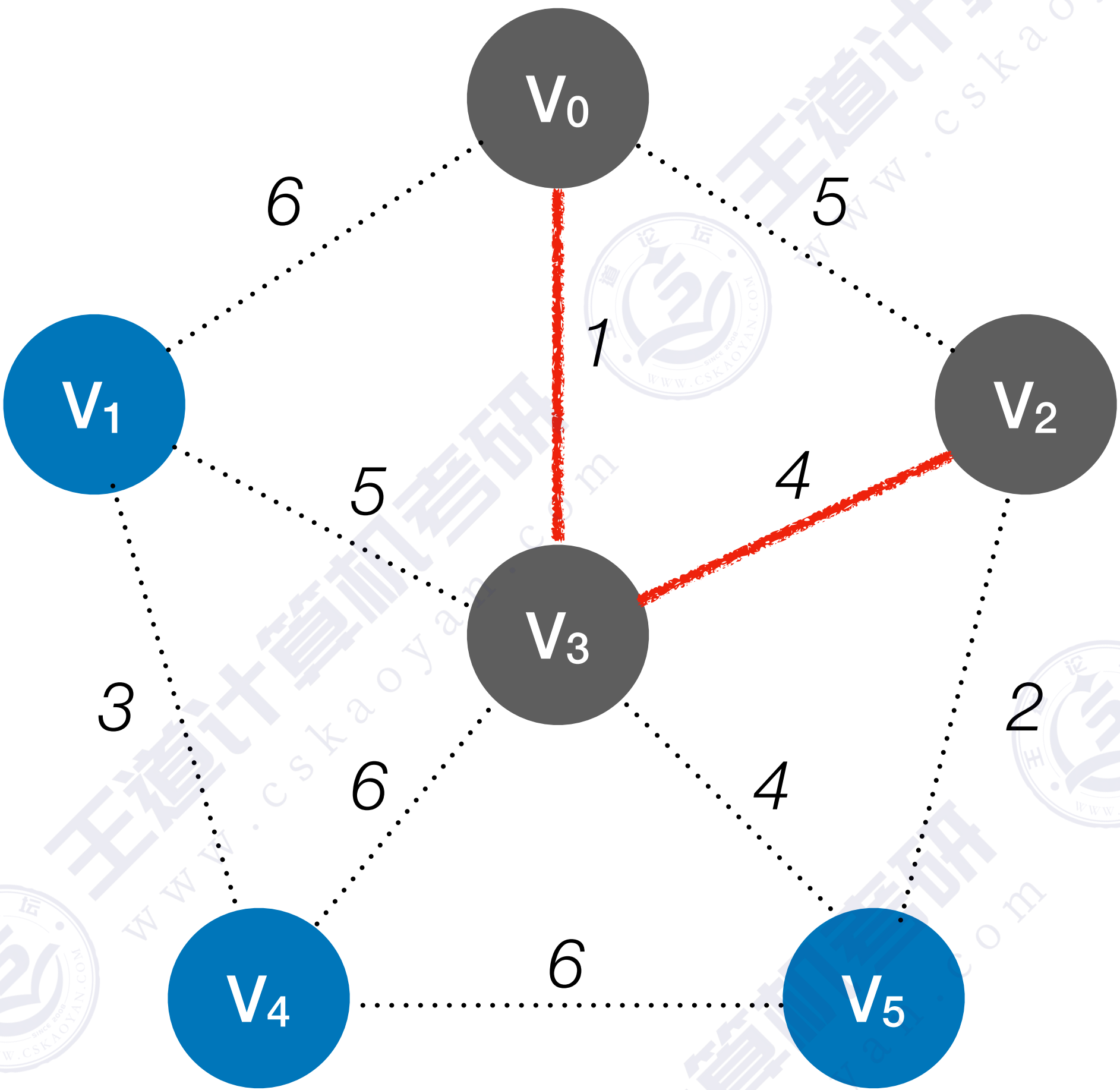
V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗

lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第3轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

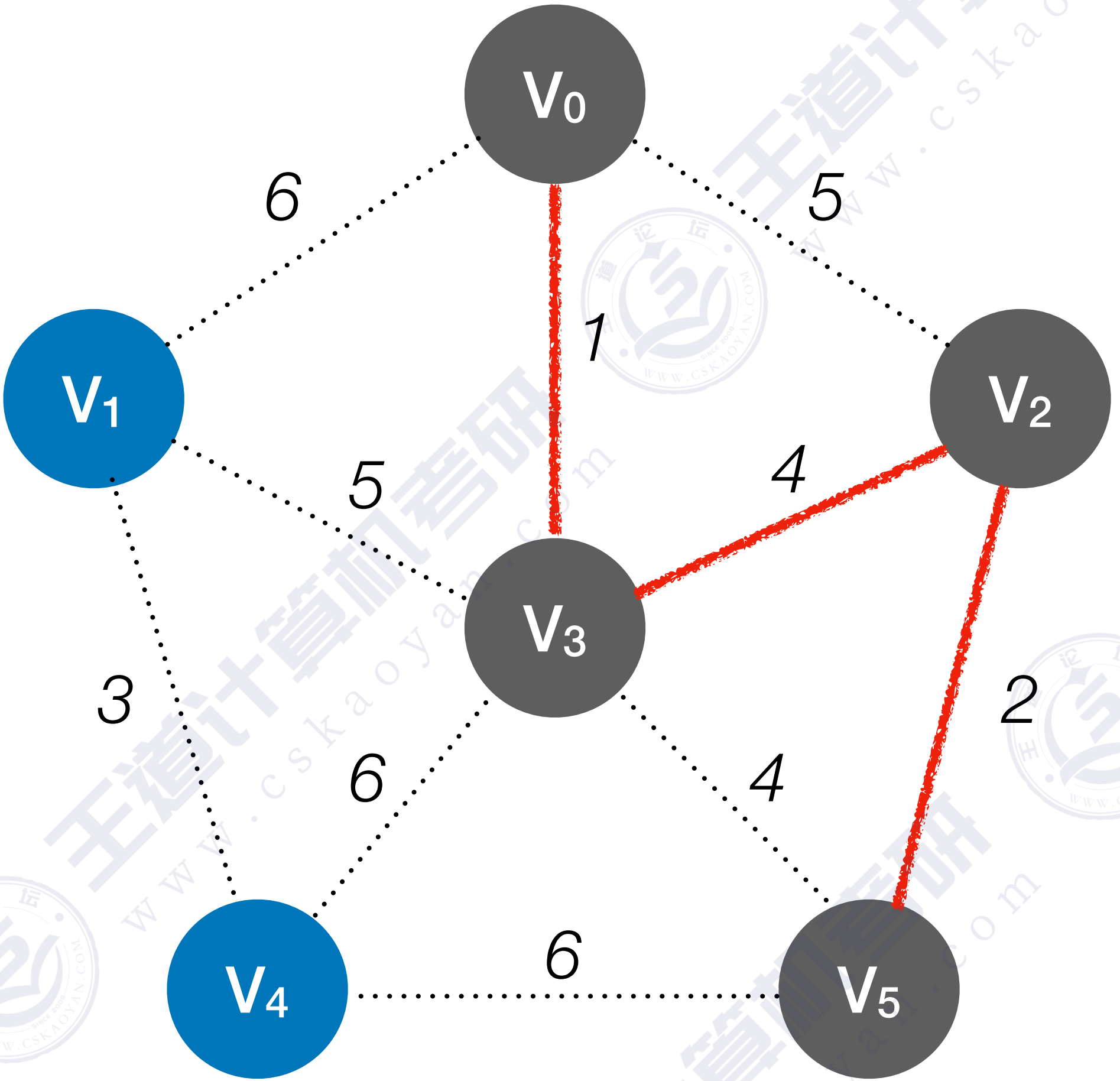
V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✗

lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第3轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✓

lowCost[6]

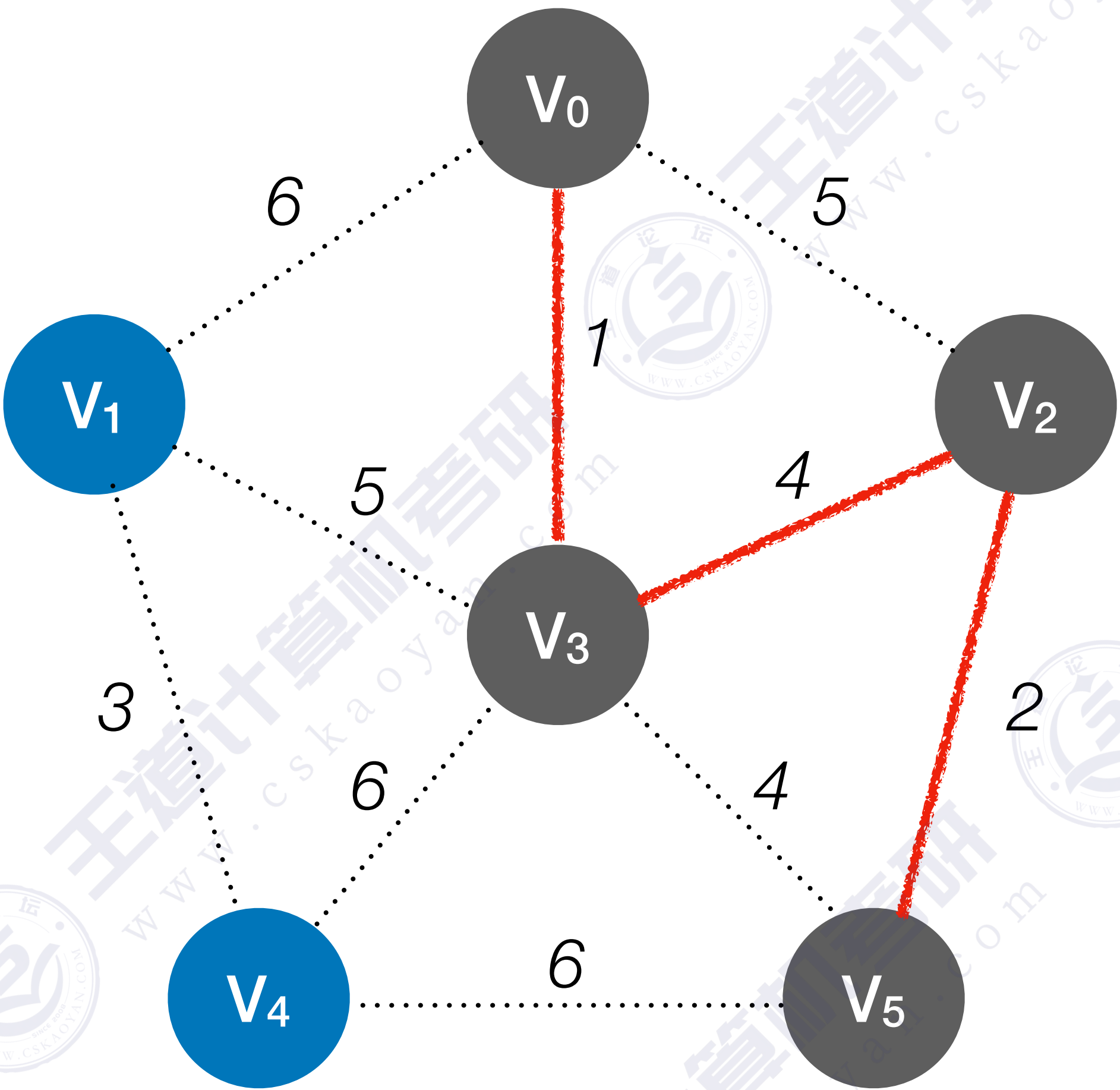
0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值



Prim 算法的实现思想



第4轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

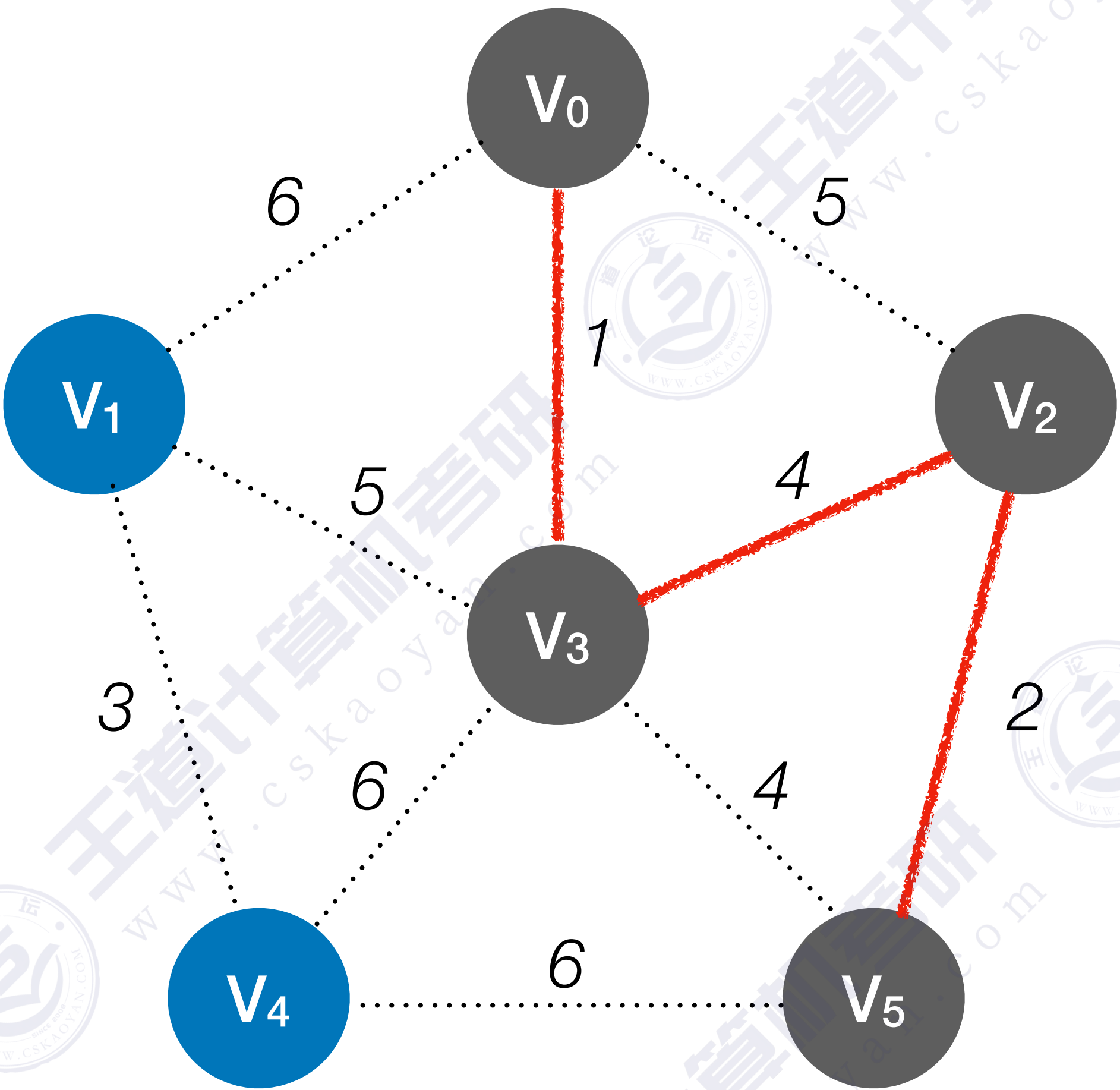
V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✓

lowCost[6]

各节点加入树的最低代价

0	5	4	1	6	2
---	---	---	---	---	---

Prim 算法的实现思想

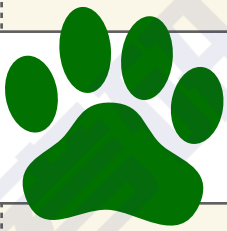


第4轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✗	✓	✓	✗	✓

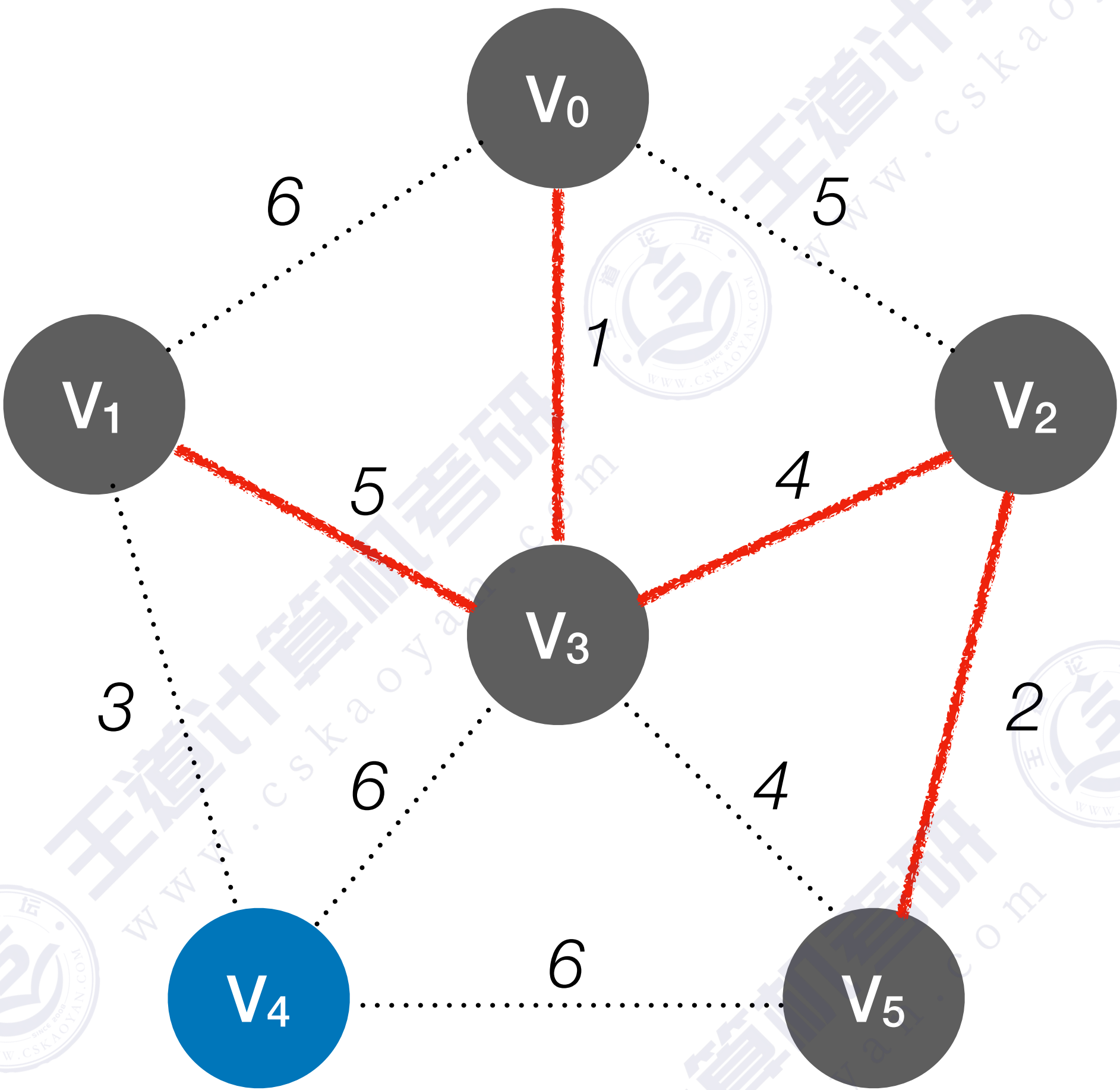


lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想

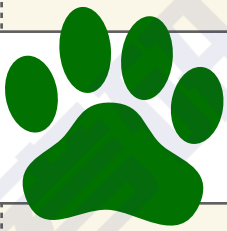


第4轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✗	✓

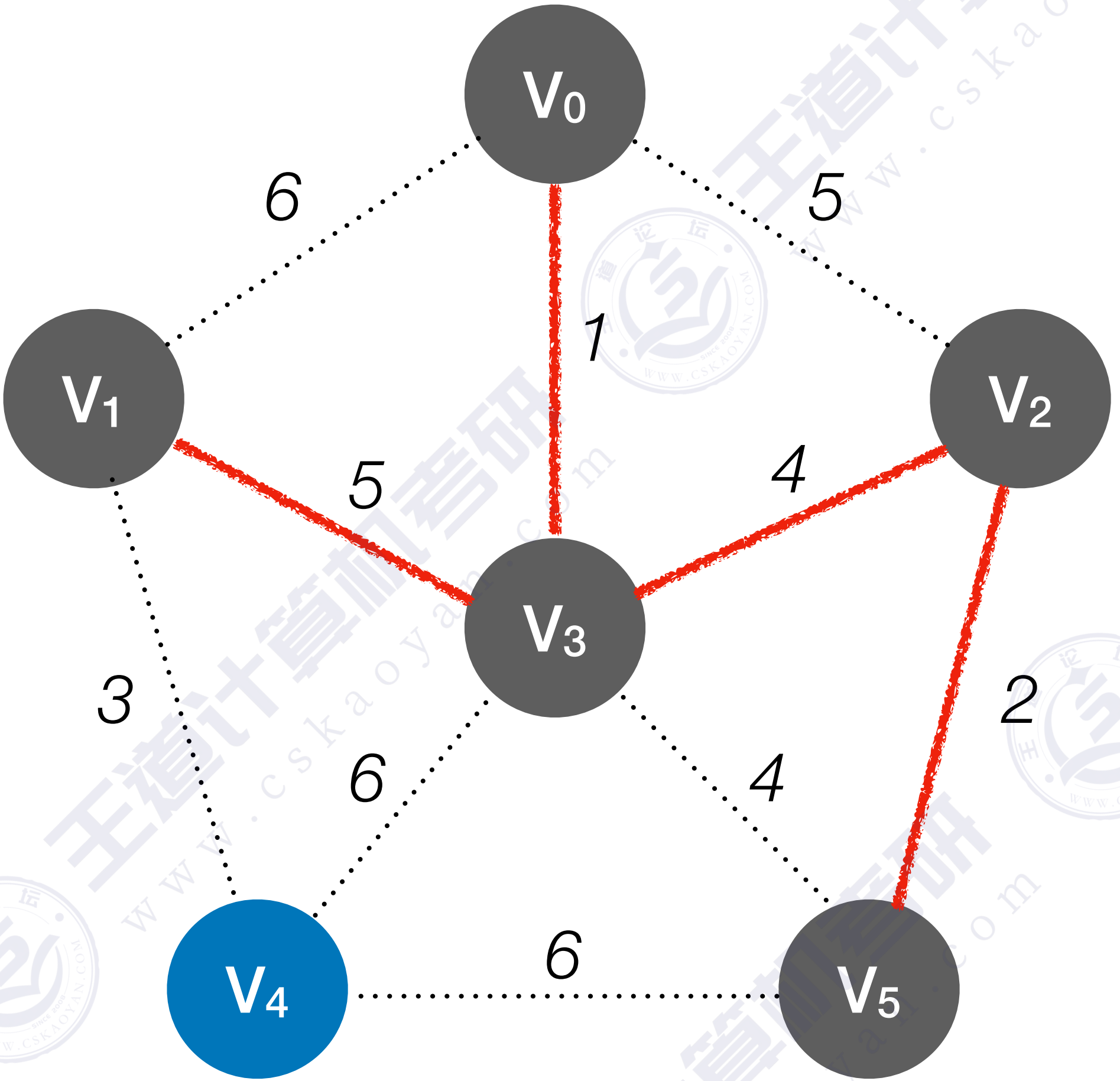


lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

Prim 算法的实现思想



第4轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✗	✓



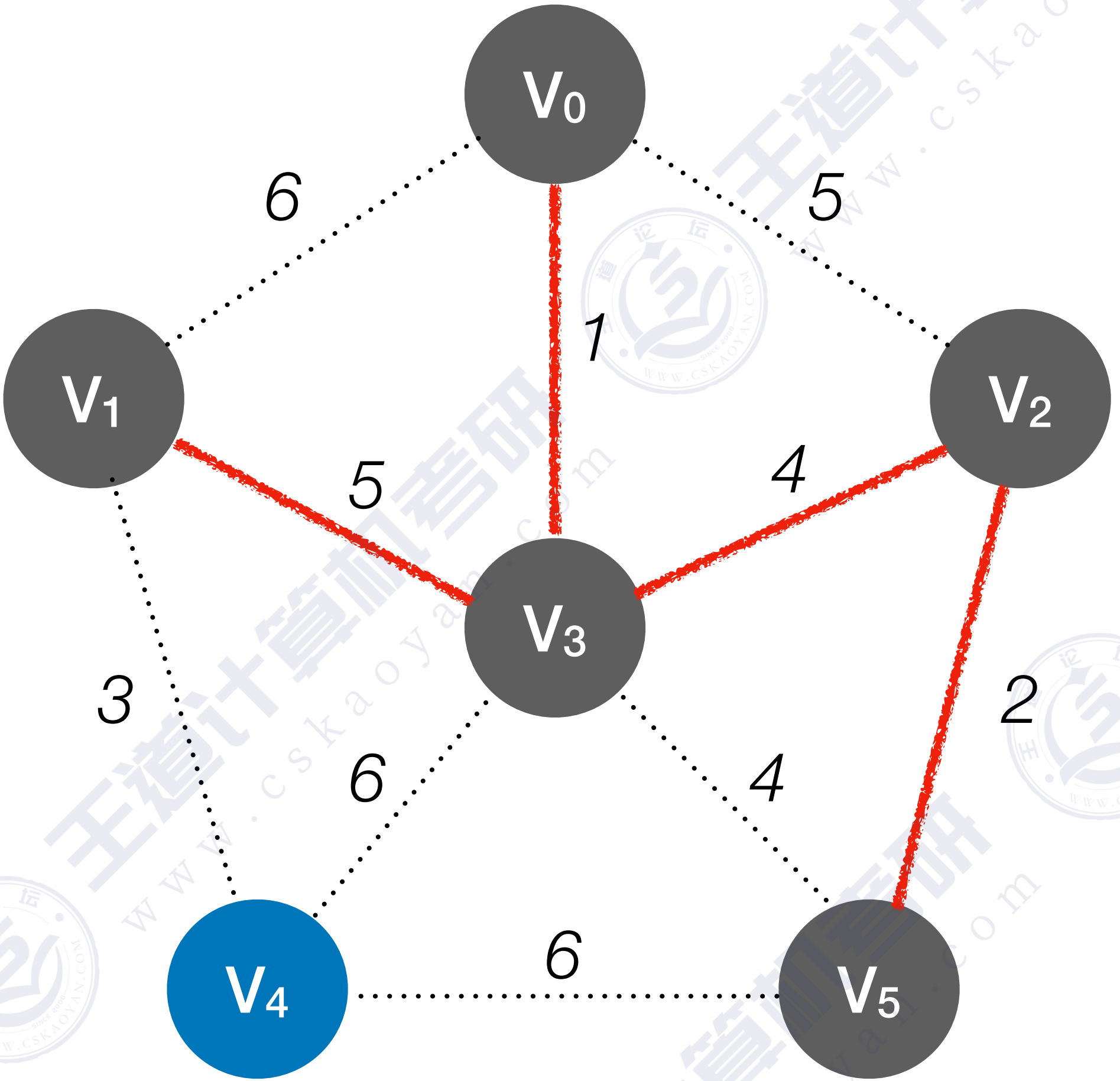
lowCost[6]

0	5	4	1	6	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第4轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✗	✓



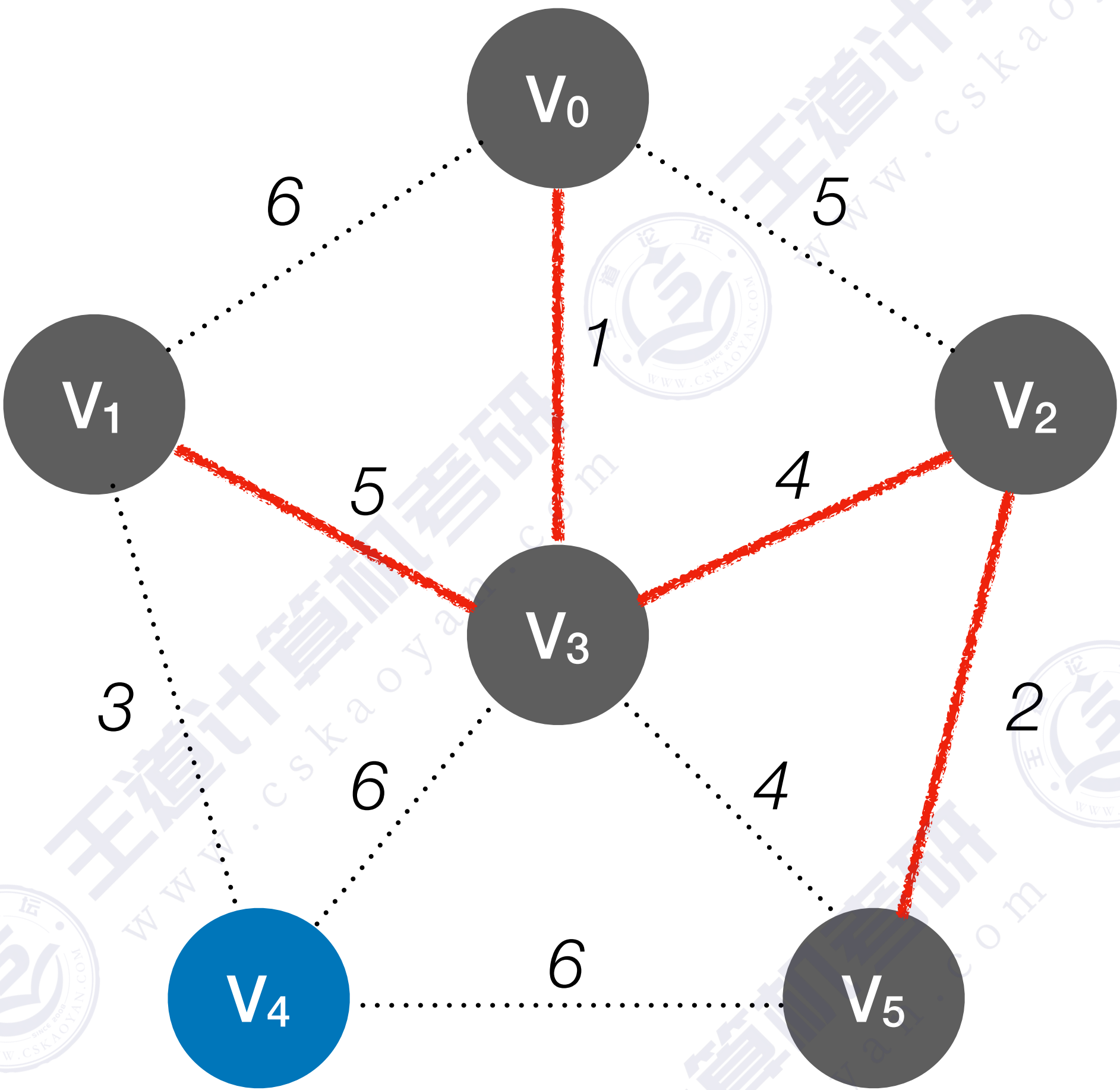
lowCost[6]

0	5	4	1	3	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第5轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✗	✓

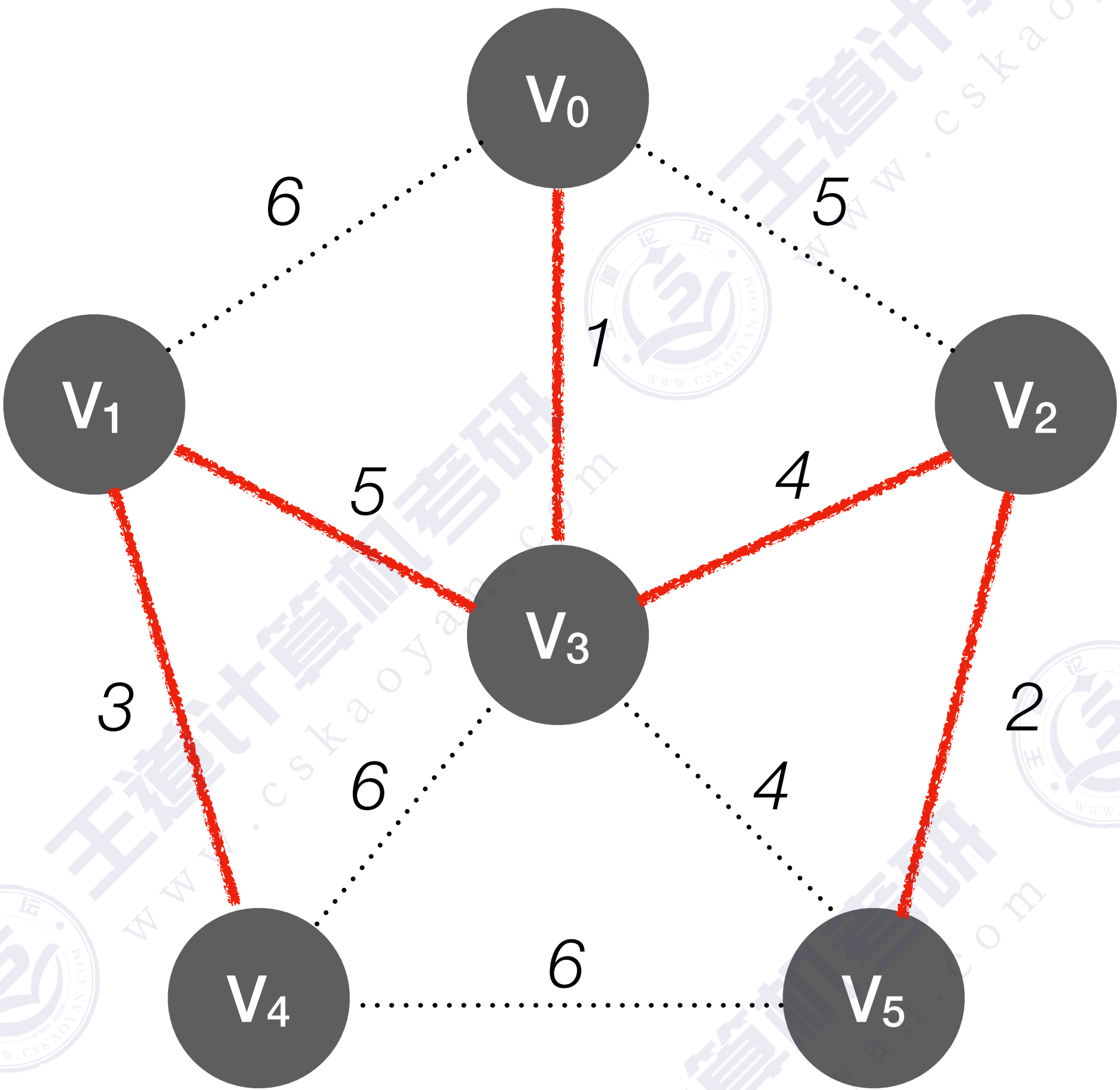
lowCost[6]

0	5	4	1	3	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想



第5轮：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点

标记各节点是否已加入树

isJoin[6]

V0	V1	V2	V3	V4	V5
✓	✓	✓	✓	✓	✓



lowCost[6]

0	5	4	1	3	2
---	---	---	---	---	---

各节点加入树的最低代价

再次循环遍历，更新还没加入的各个顶点的lowCost值

Prim 算法的实现思想

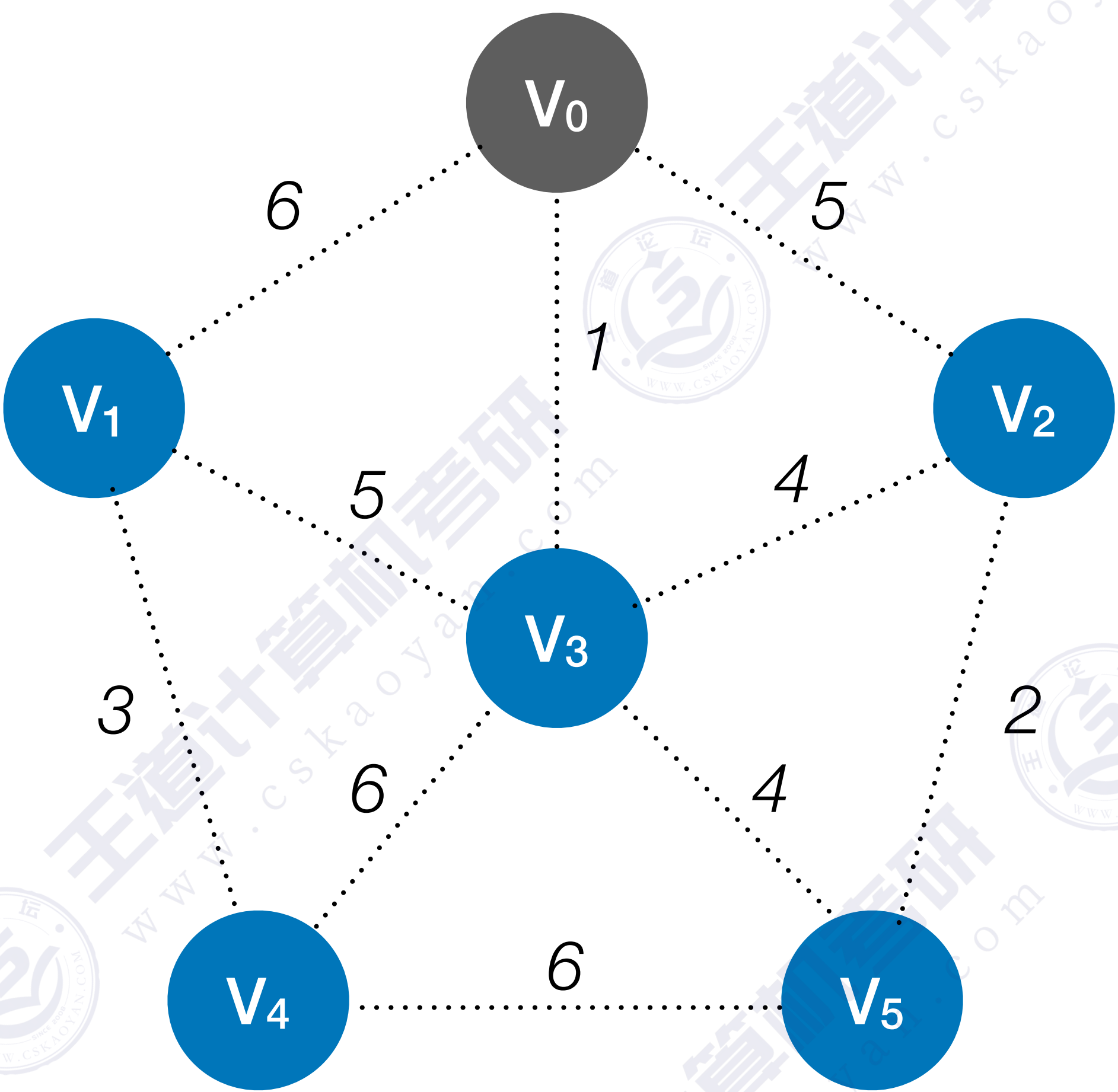
总时间复杂度
 $O(n^2)$, 即 $O(|V|^2)$

从 V_0 开始, 总共需要 $n-1$ 轮处理

每一轮处理: 循环遍历所有个结点, 找到lowCost最低的, 且还没加入树的顶点。

再次循环遍历, 更新还没加入的各个顶点的lowCost值

每一轮时间复杂度 $O(2n)$



$isJoin[6]$

V0	V1	V2	V3	V4	V5
✓	✗	✗	✗	✗	✗

$lowCost[6]$

0	6	5	1	∞	∞
---	---	---	---	----------	----------

Prim 算法 v.s. Kruskal 算法

Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

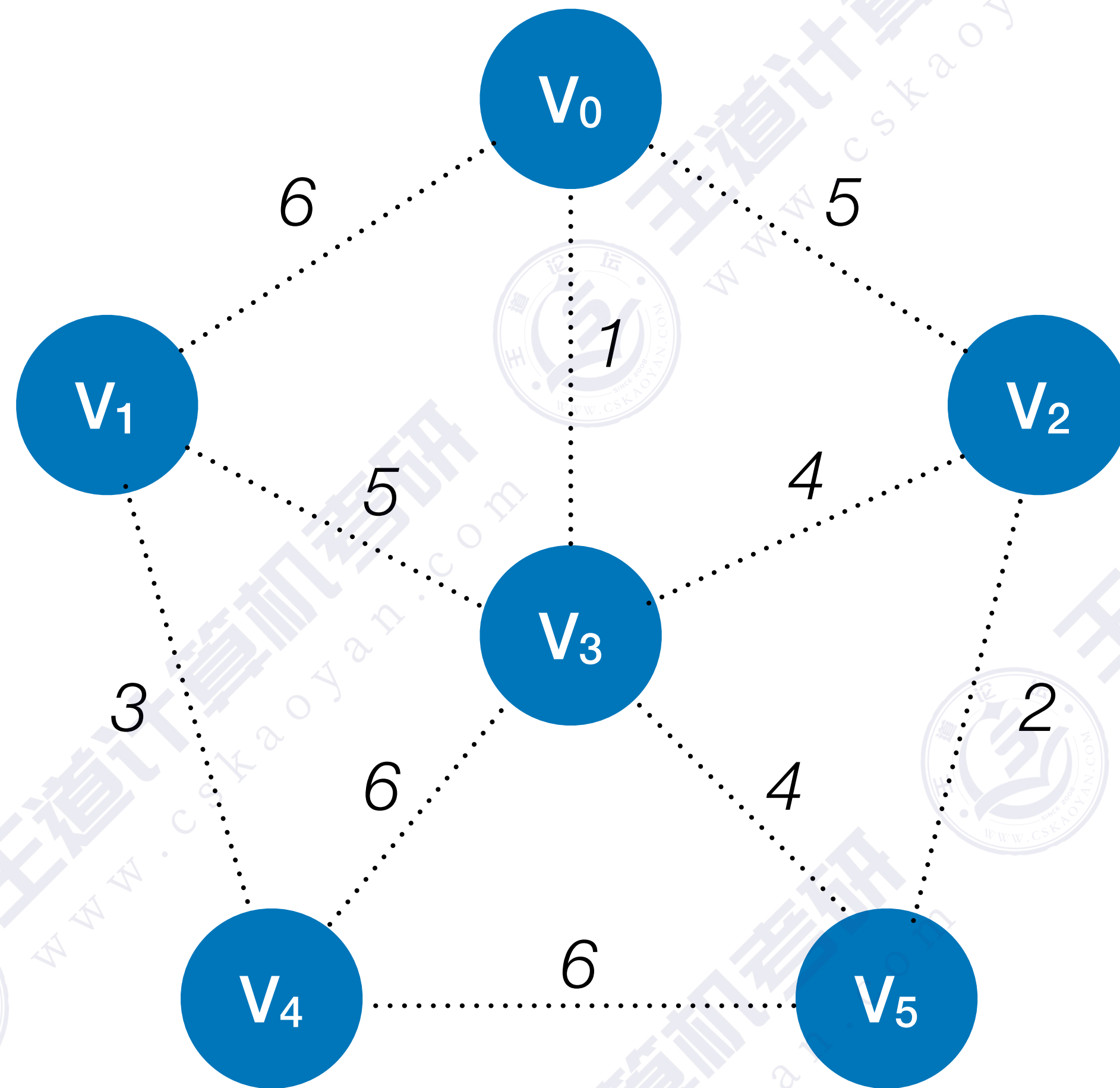
时间复杂度： $O(|V|^2)$
适合用于边稠密图

Kruskal 算法（克鲁斯卡尔）：

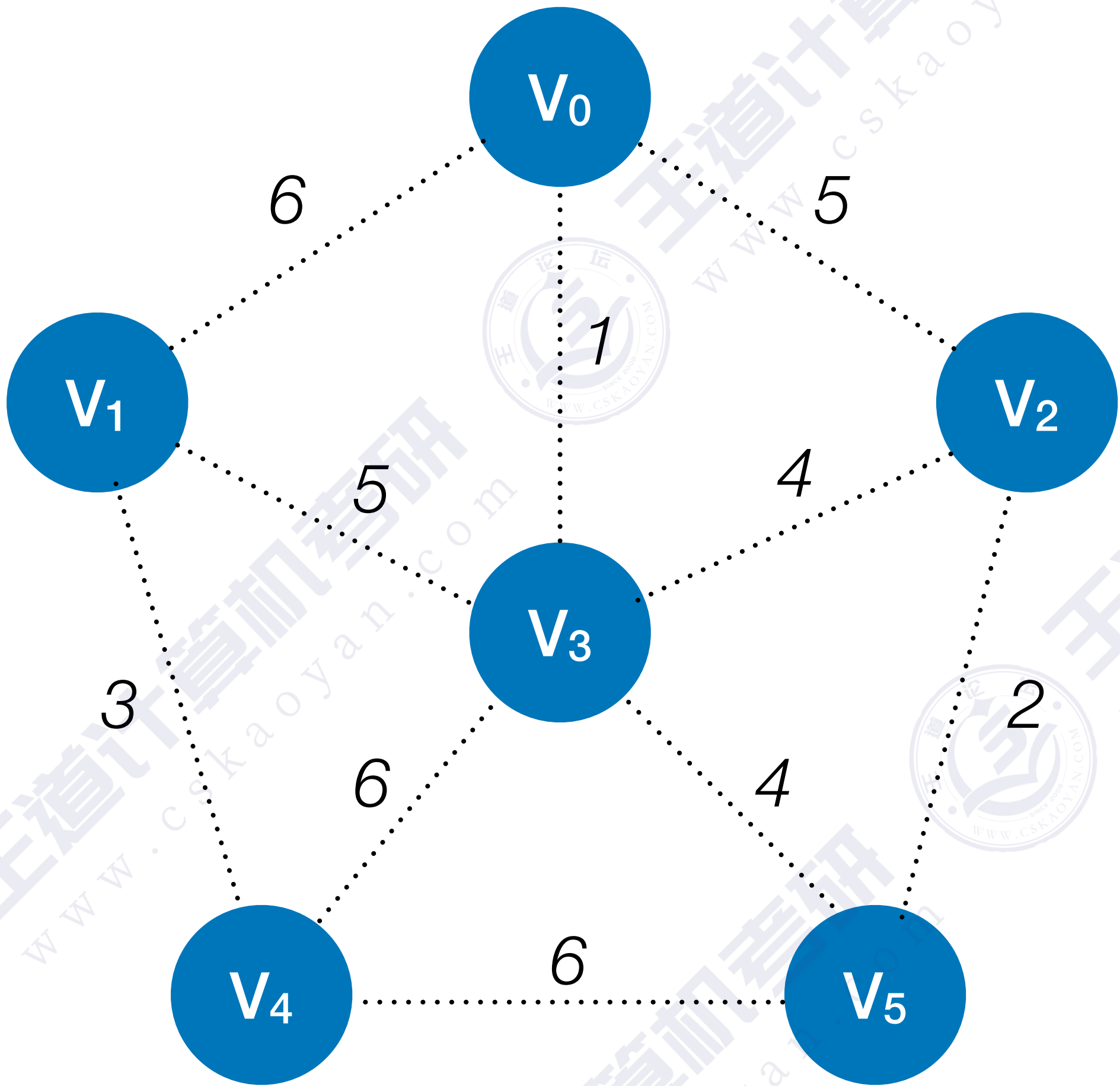
每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）
直到所有结点都连通

时间复杂度： $O(|E|\log_2|E|)$
适合用于边稀疏图

Kruskal 算法的实现思想



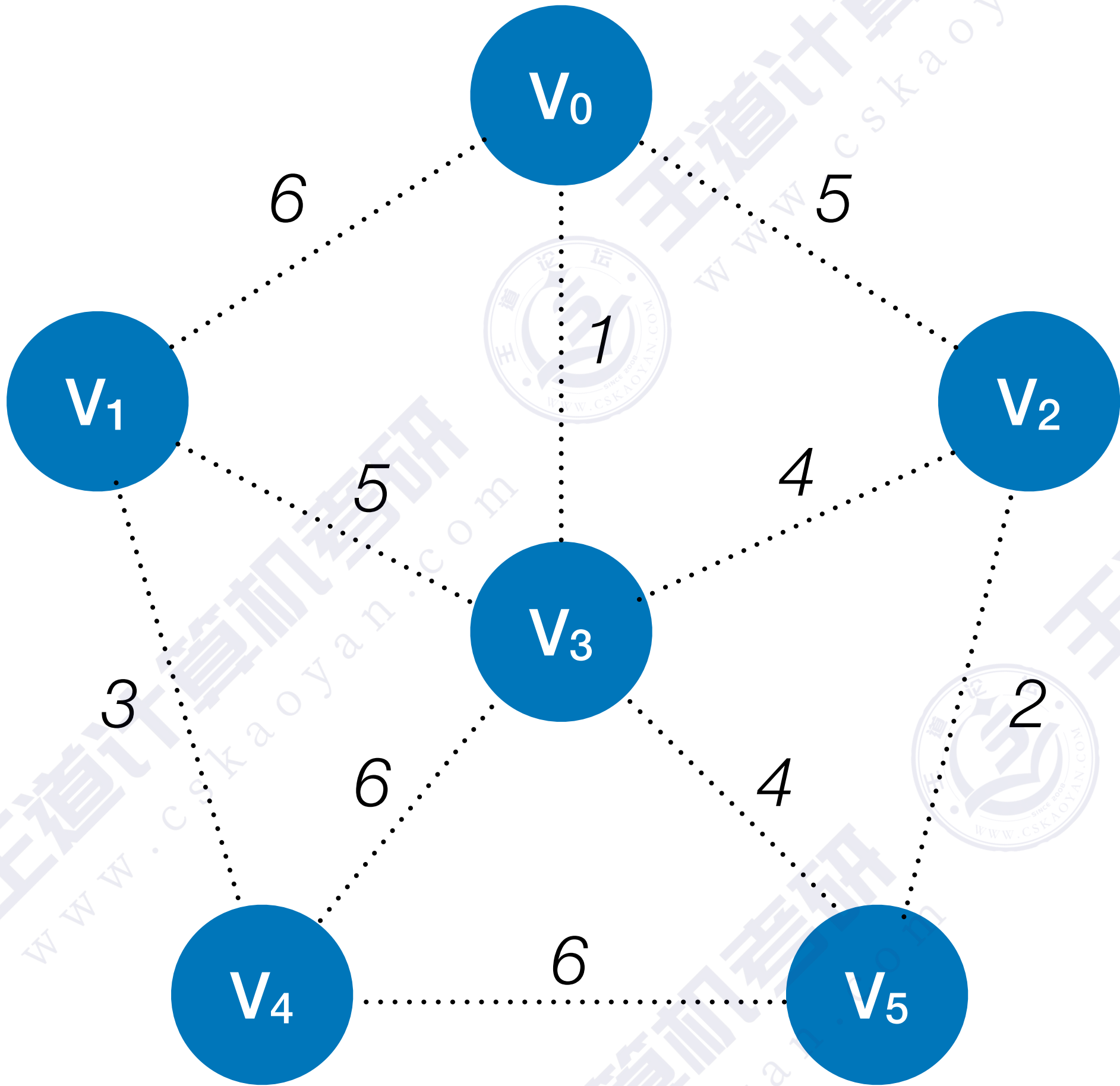
Kruskal 算法的实现思想



初始：将各条边按权值排序

weight	Vertex1	Vertex2
1	V_0	V_3
2	V_2	V_5
3	V_1	V_4
4	V_2	V_3
4	V_3	V_5
5	V_0	V_2
5	V_1	V_3
6	V_0	V_1
6	V_3	V_4
6	V_4	V_5

Kruskal 算法的实现思想



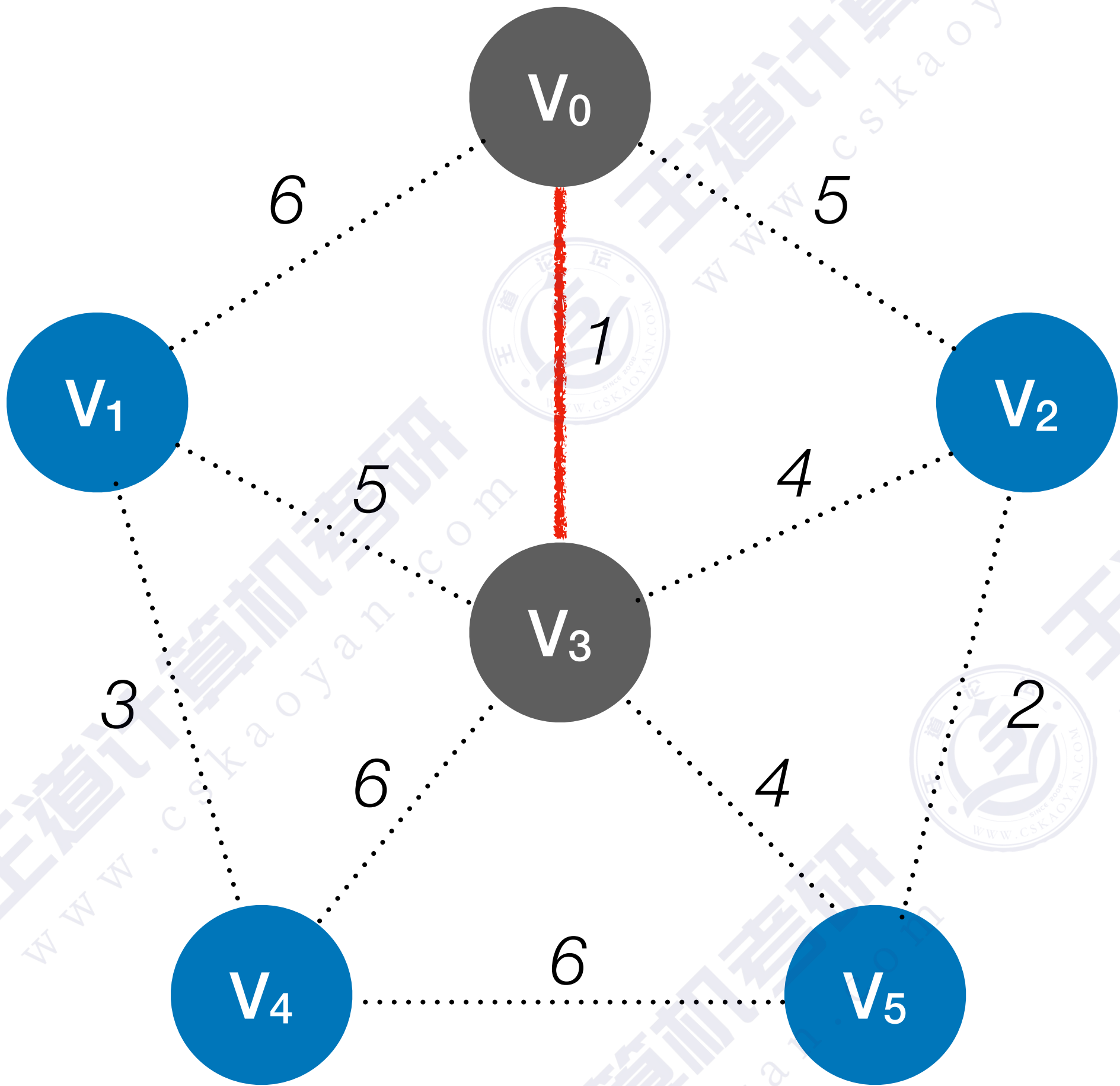
第1轮：检查第1条边的两个顶点是否连通（是否属于同一个集合）

	weight	Vertex1	Vertex2
	1	V_0	V_3
	2	V_2	V_5
	3	V_1	V_4
	4	V_2	V_3
	4	V_3	V_5
	5	V_0	V_2
	5	V_1	V_3
	6	V_0	V_1
	6	V_3	V_4
	6	V_4	V_5

Kruskal 算法的实现思想



第1轮：检查第1条边的两个顶点是否连通（是否属于同一个集合）



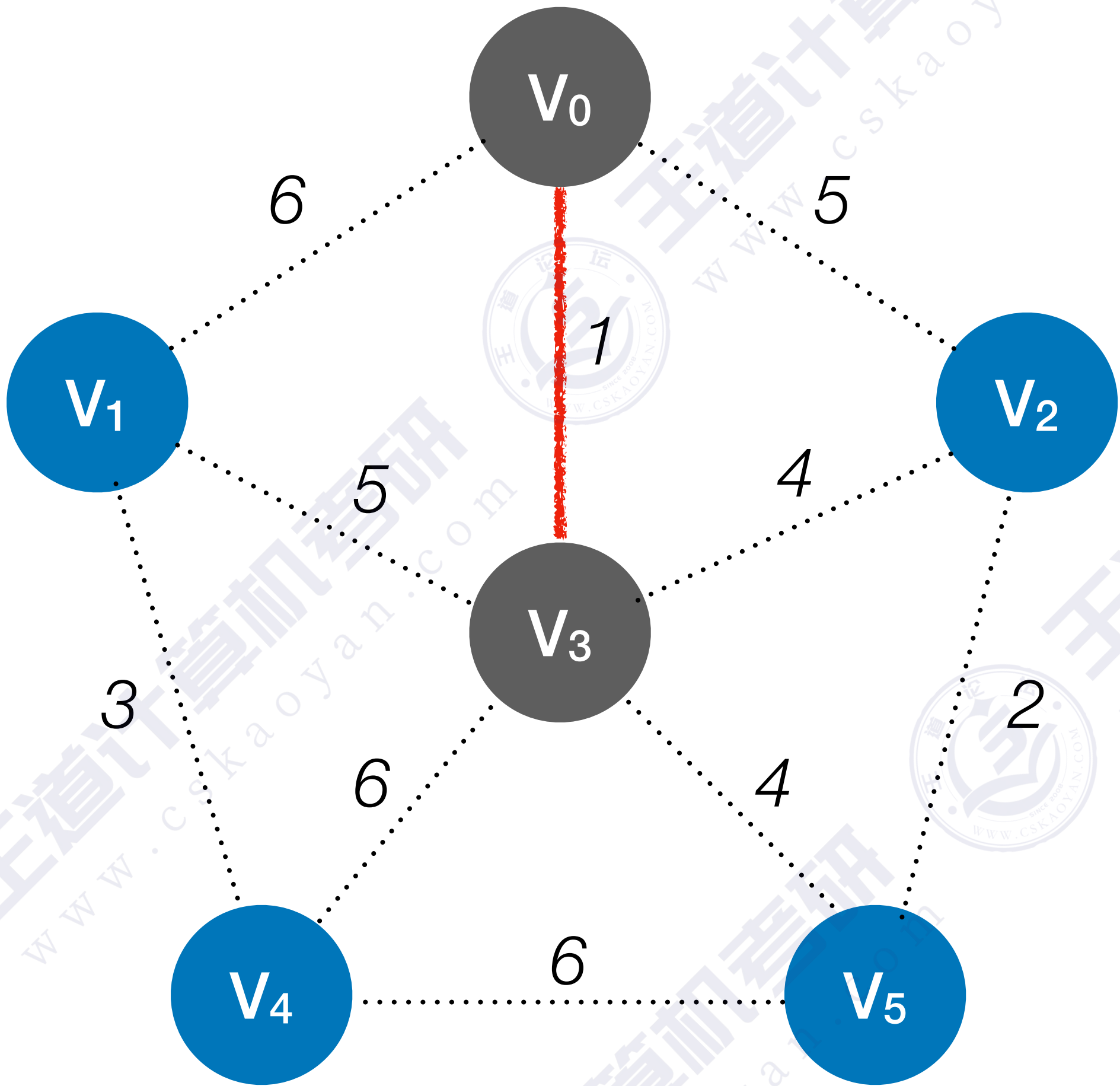
	weight	Vertex1	Vertex2
	1	V ₀	V ₃
	2	V ₂	V ₅
	3	V ₁	V ₄
	4	V ₂	V ₃
	4	V ₃	V ₅
	5	V ₀	V ₂
	5	V ₁	V ₃
	6	V ₀	V ₁
	6	V ₃	V ₄
	6	V ₄	V ₅

不连通，连起来！

Kruskal 算法的实现思想



第2轮：检查第2条边的两个顶点是否连通（是否属于同一个集合）

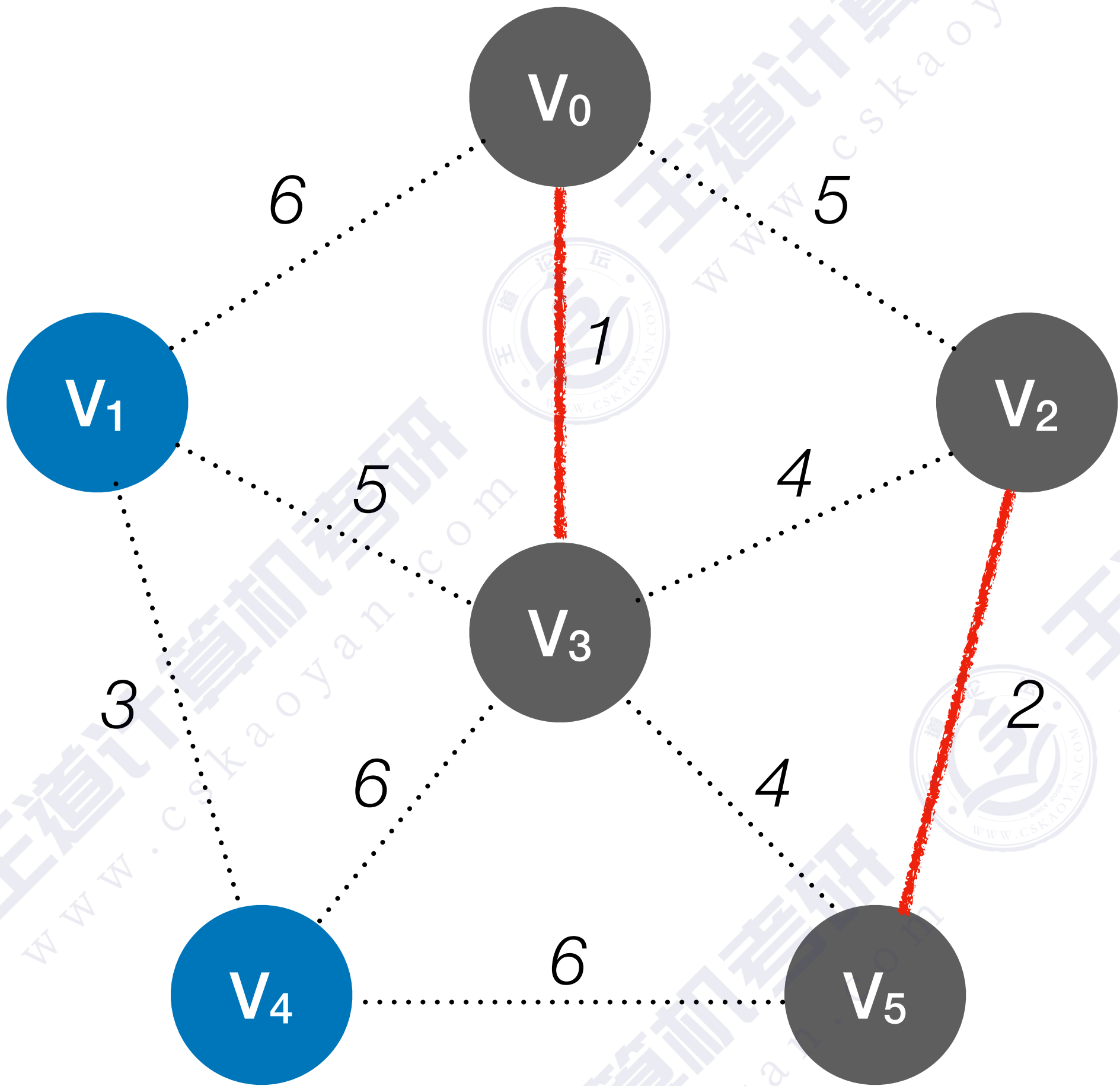


weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

Kruskal 算法的实现思想



第2轮：检查第2条边的两个顶点是否连通（是否属于同一个集合）



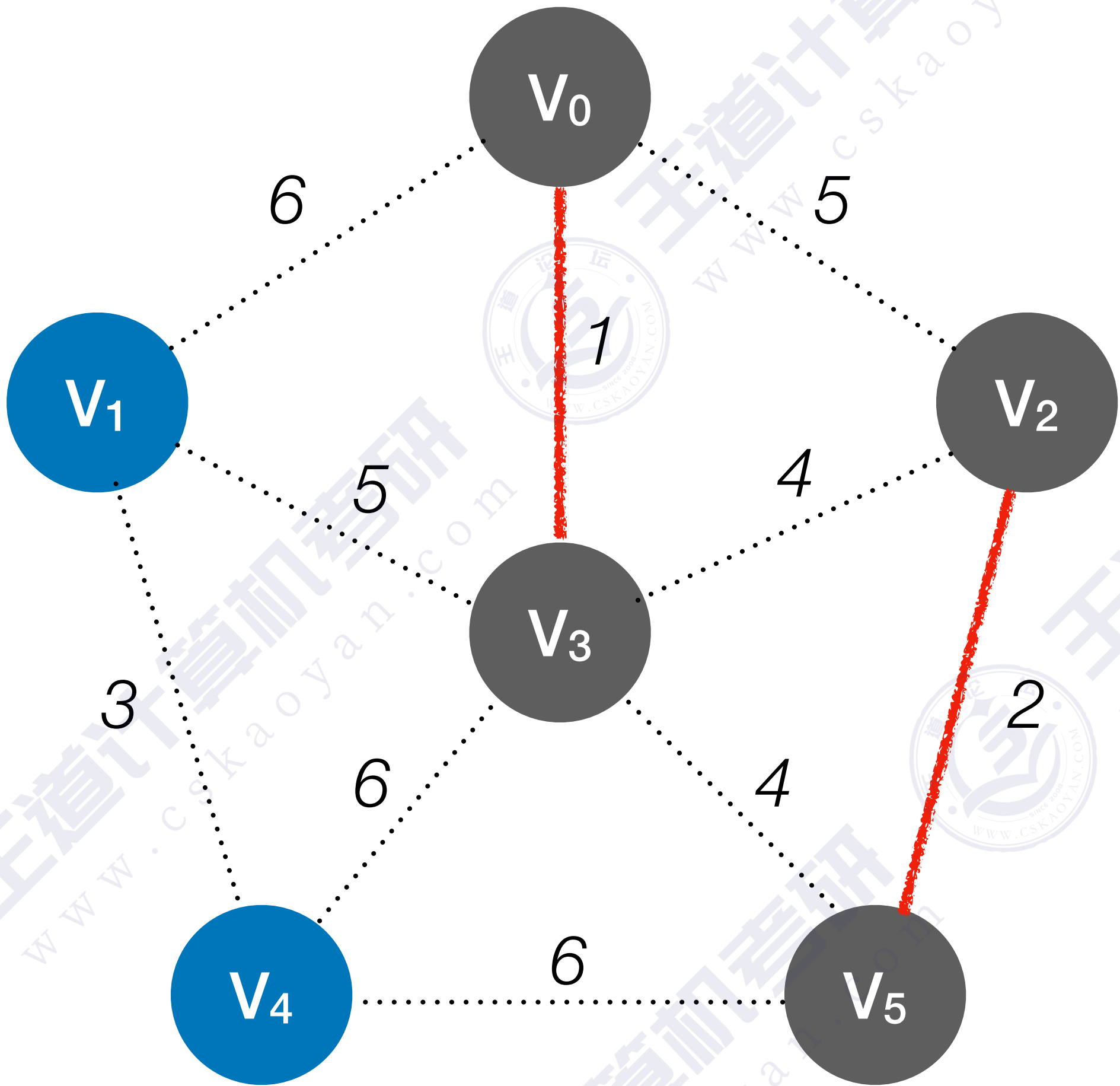
weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

不连通，连起来！

Kruskal 算法的实现思想

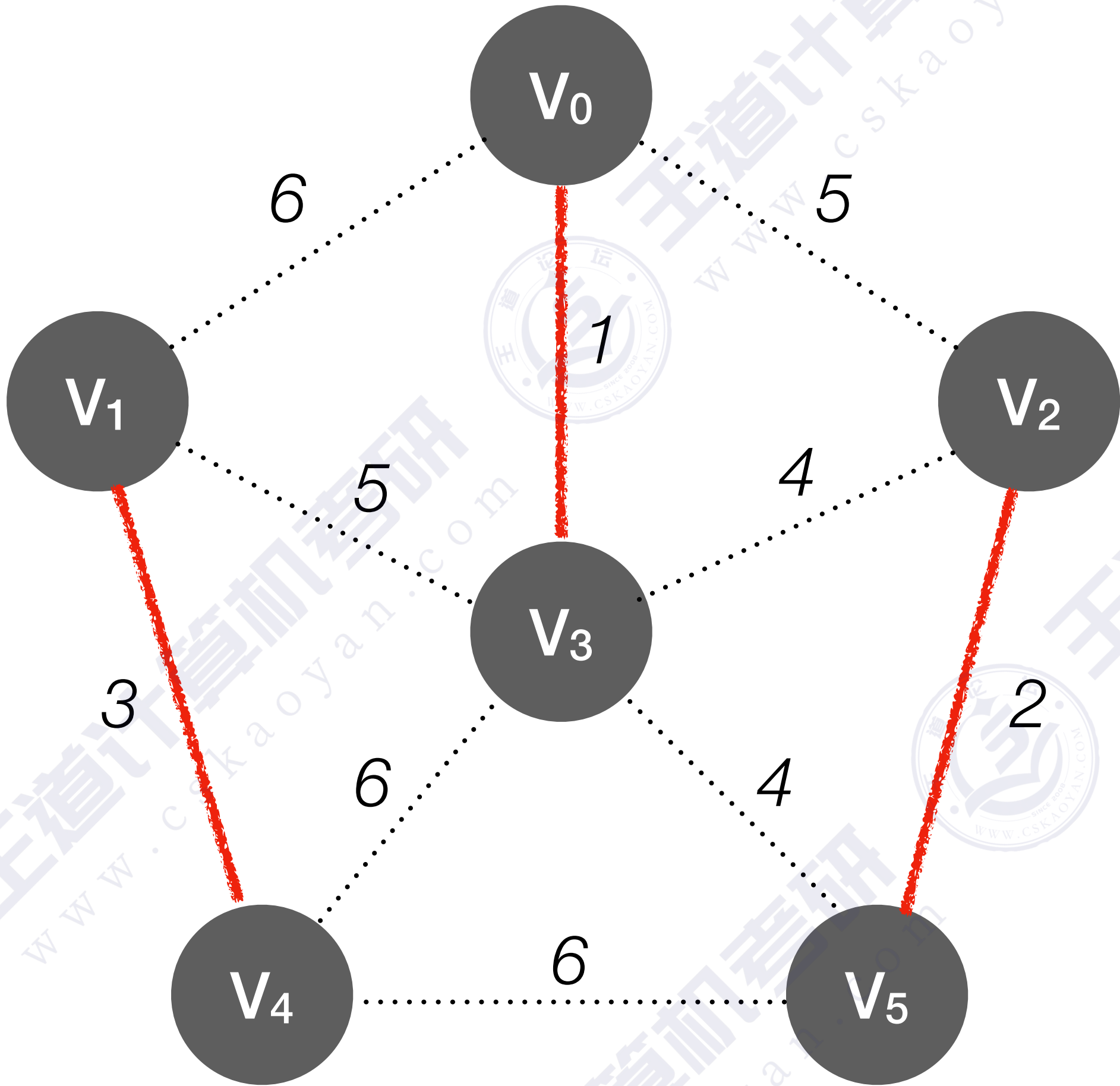


第3轮：检查第3条边的两个顶点是否连通（是否属于同一个集合）



weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

Kruskal 算法的实现思想



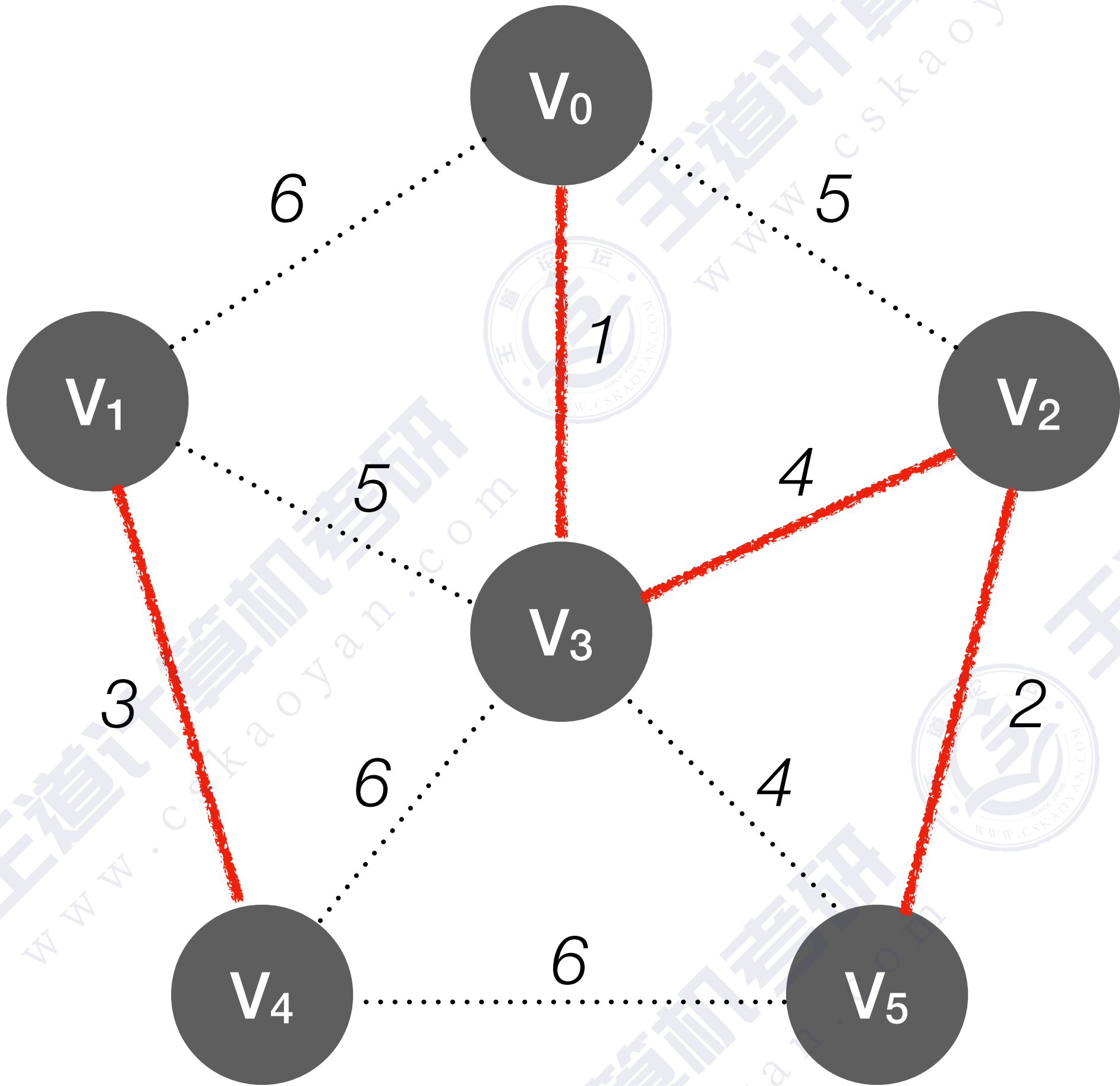
第3轮：检查第3条边的两个顶点是否连通（是否属于同一个集合）



weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

不连通，连起来！

Kruskal 算法的实现思想



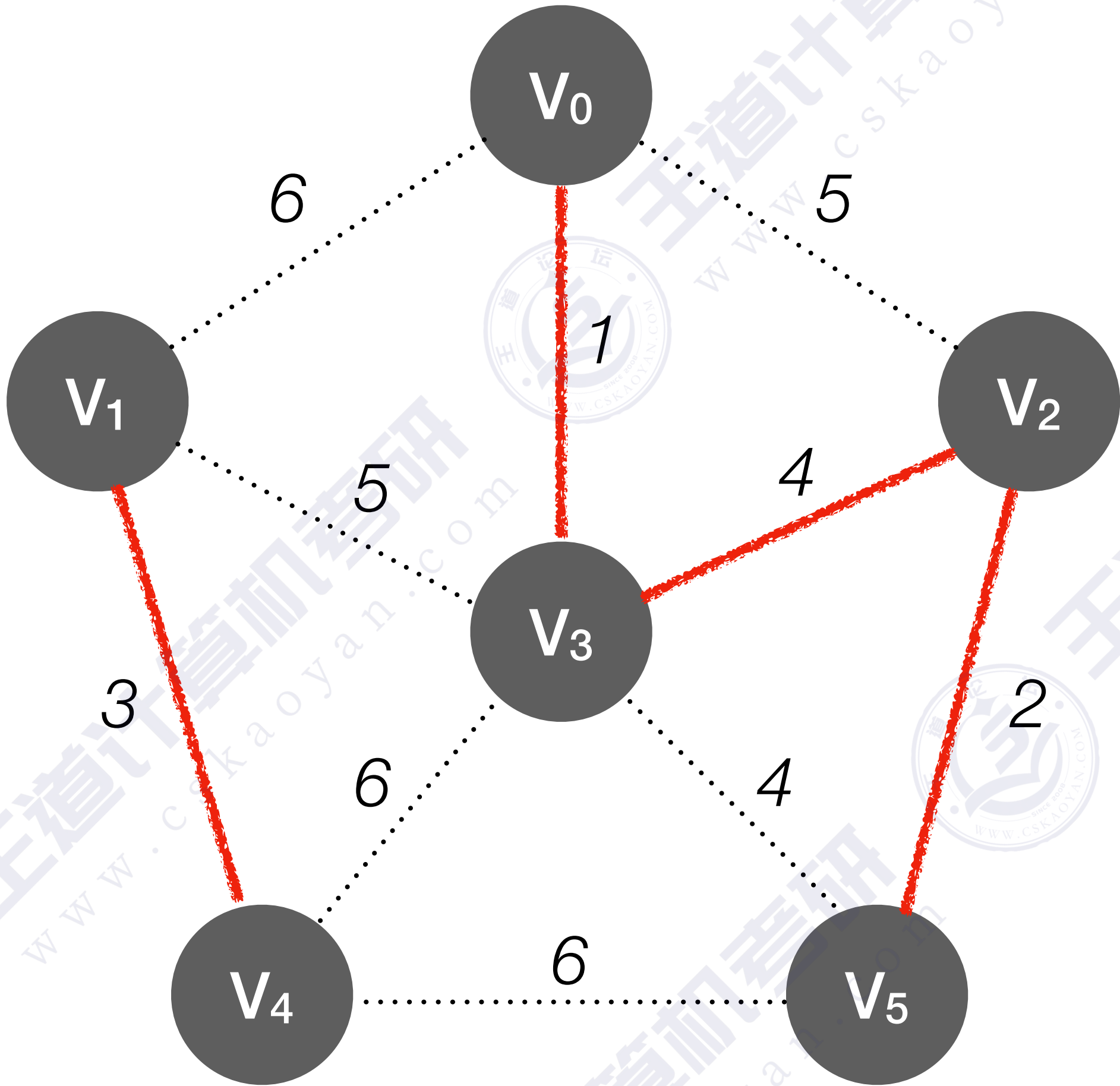
第4轮：检查第4条边的两个顶点是否连通（是否属于同一个集合）



weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

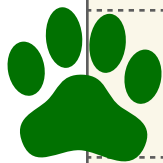
不连通，连起来！

Kruskal 算法的实现思想



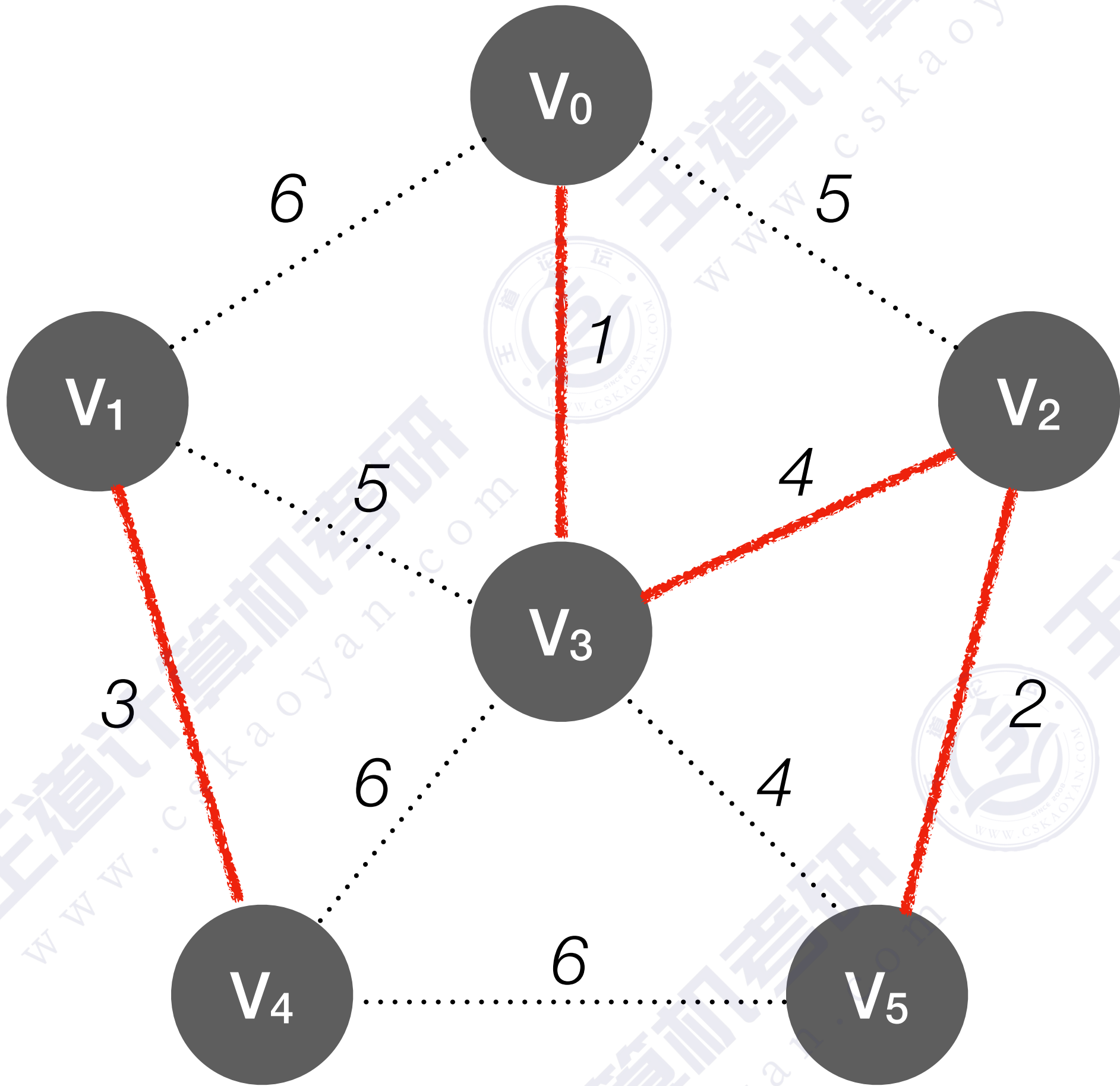
第5轮：检查第5条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅



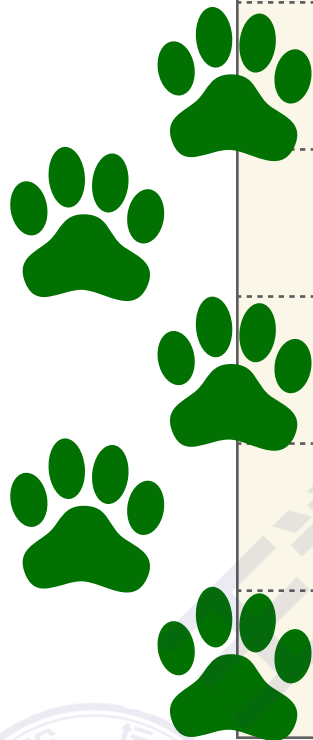
已连通，跳过！

Kruskal 算法的实现思想

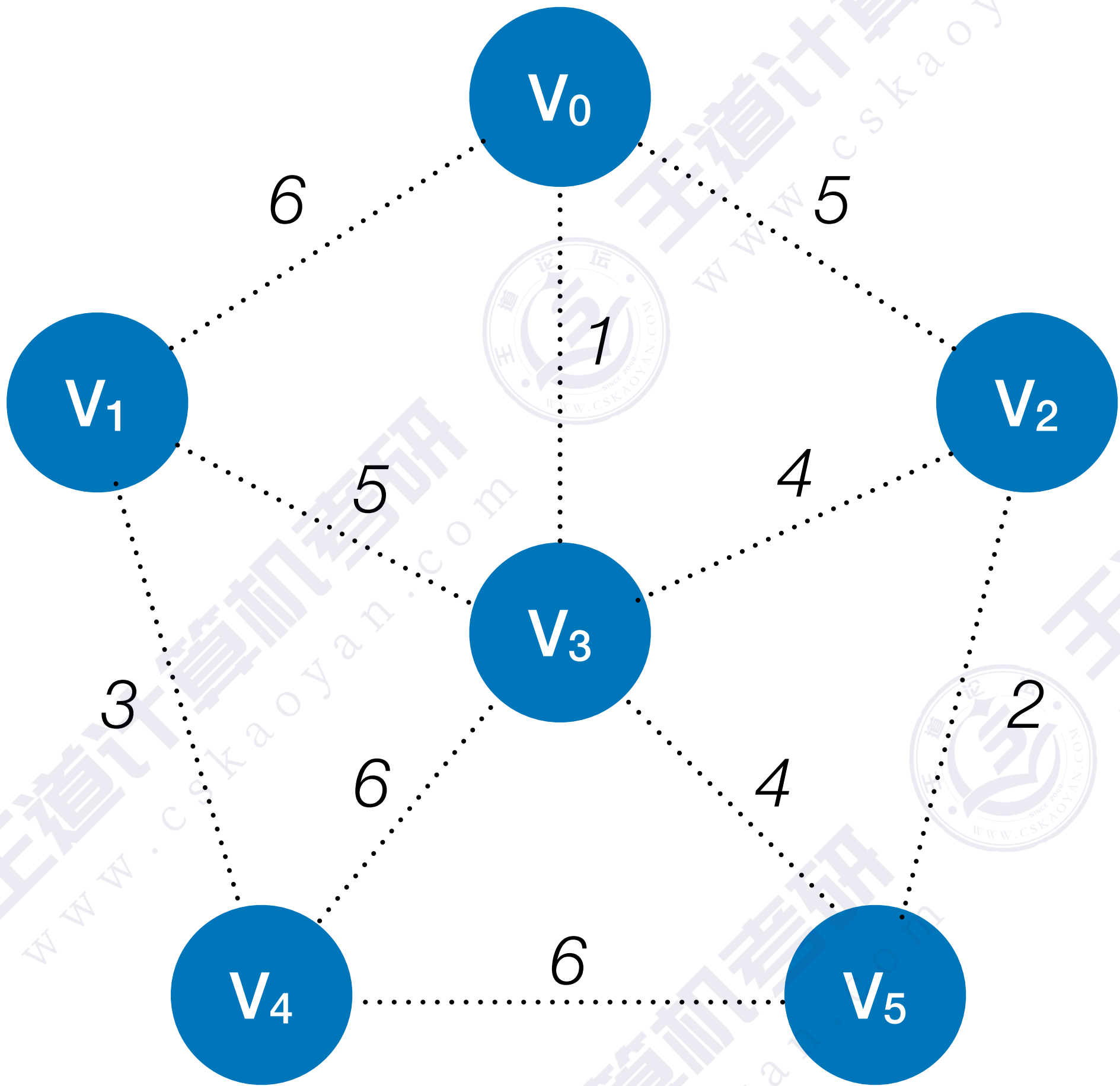


第6轮：检查第6条边的两个顶点是否连通（是否属于同一个集合）

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅



Kruskal 算法的实现思想



初始：将各条边按权值排序

weight	Vertex1	Vertex2
1	V ₀	V ₃
2	V ₂	V ₅
3	V ₁	V ₄
4	V ₂	V ₃
4	V ₃	V ₅
5	V ₀	V ₂
5	V ₁	V ₃
6	V ₀	V ₁
6	V ₃	V ₄
6	V ₄	V ₅

共执行 e 轮，每轮判断两个顶点是否属于同一集合，需要 $O(\log_2 e)$

总时间复杂度 $O(e \log_2 e)$

知识回顾与重要考点

对于一个带权连通无向图 $G = (V, E)$ ，生成树不同，每棵树的权（即树中所有边上的权值之和）也可能不同。设 R 为 G 的所有生成树的集合，若 T 为 R 中边的权值之和最小的生成树，则 T 称为 G 的最小生成树（*Minimum-Spanning-Tree, MST*）。

Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

时间复杂度： $O(|V|^2)$
适合用于边稠密图

Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）
直到所有结点都连通

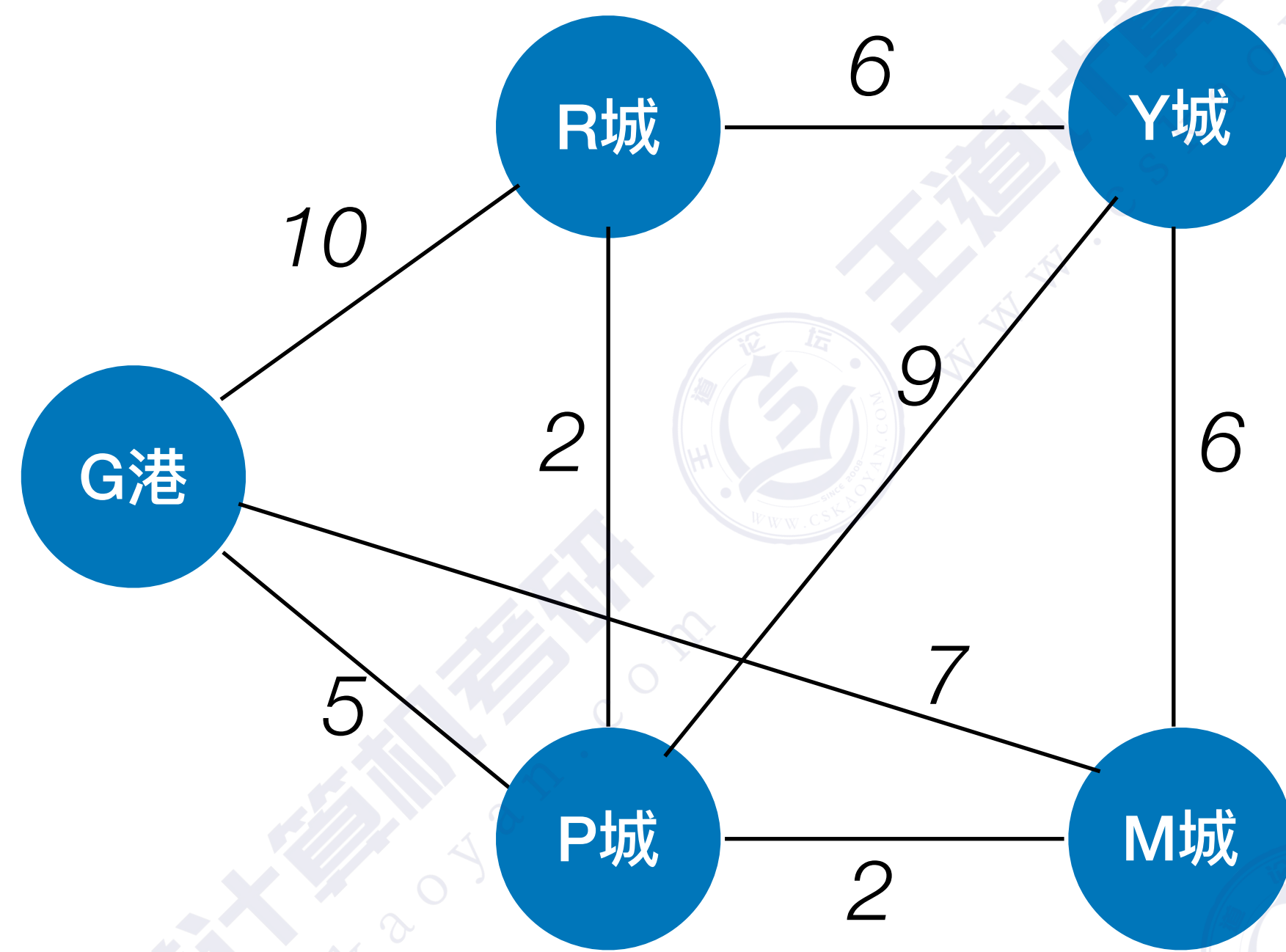
时间复杂度： $O(|E|\log_2|E|)$
适合用于边稀疏图

本节内容

最短路径

BFS算法

最短路径问题



“G港”是个物流集散中心，经常需要往各个城市运东西，怎么运送距离最近？——单源最短路径问题

各个城市之间也需要互相往来，相互之间怎么走距离最近？——每对顶点间的最短路径

最短路径问题

单源最短路径

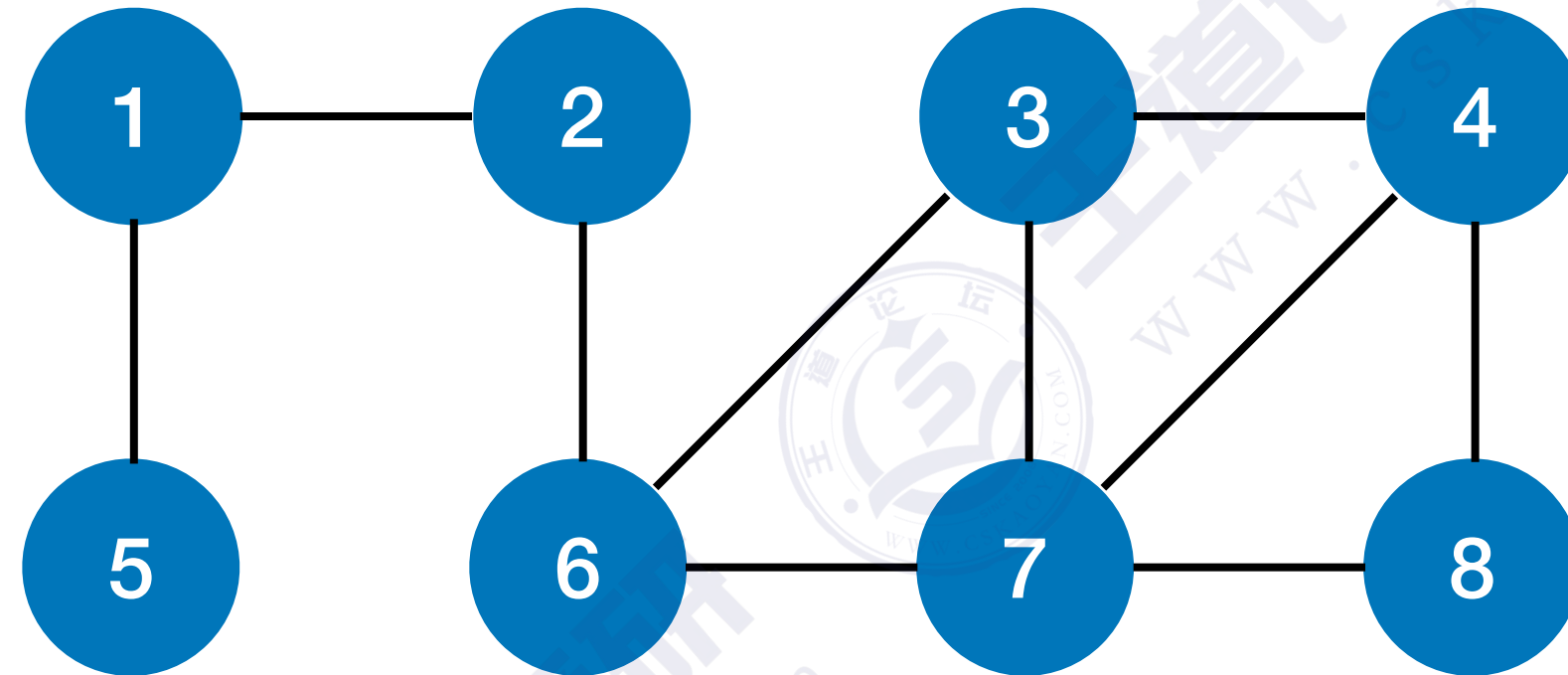
BFS 算法（无权图）

Dijkstra 算法（带权图、无权图）

各顶点间的最短路径

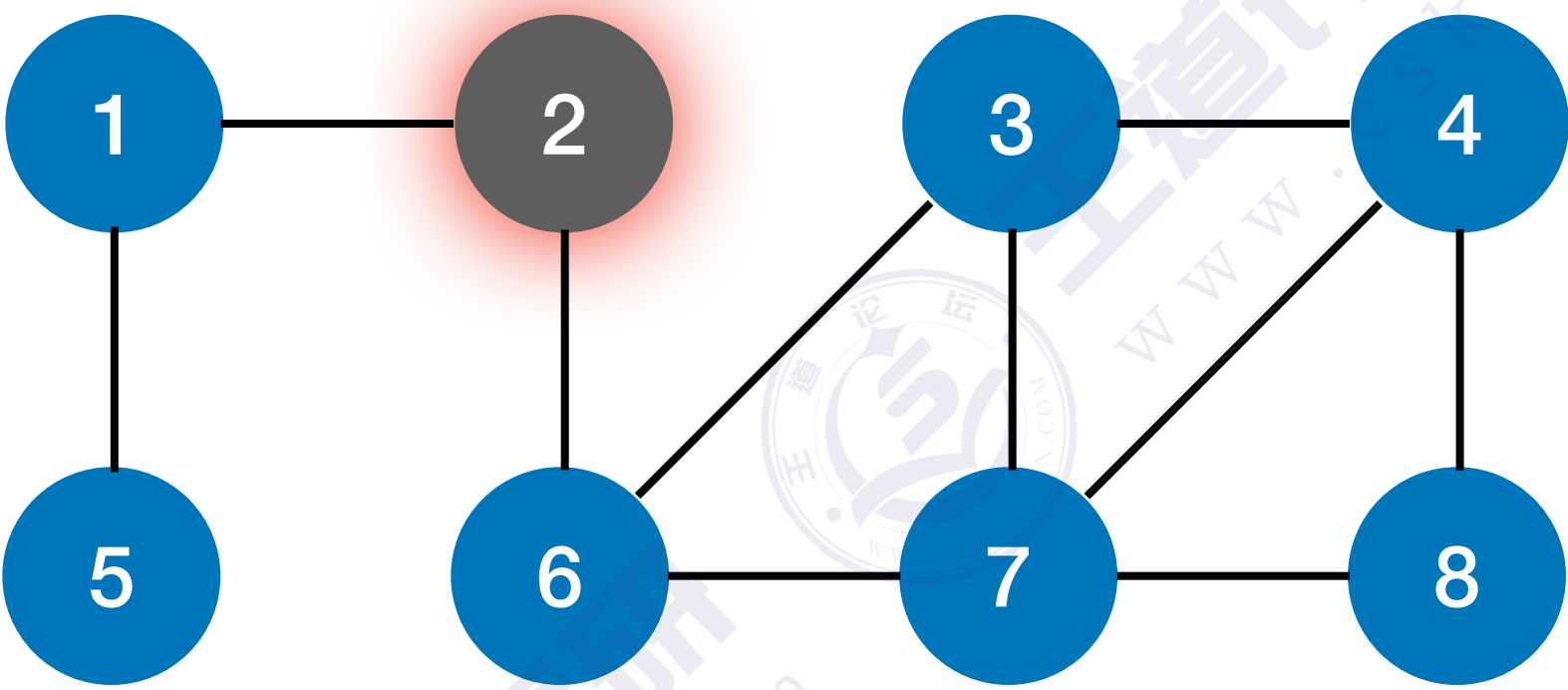
Floyd 算法（带权图、无权图）

BFS求无权图的单源最短路径

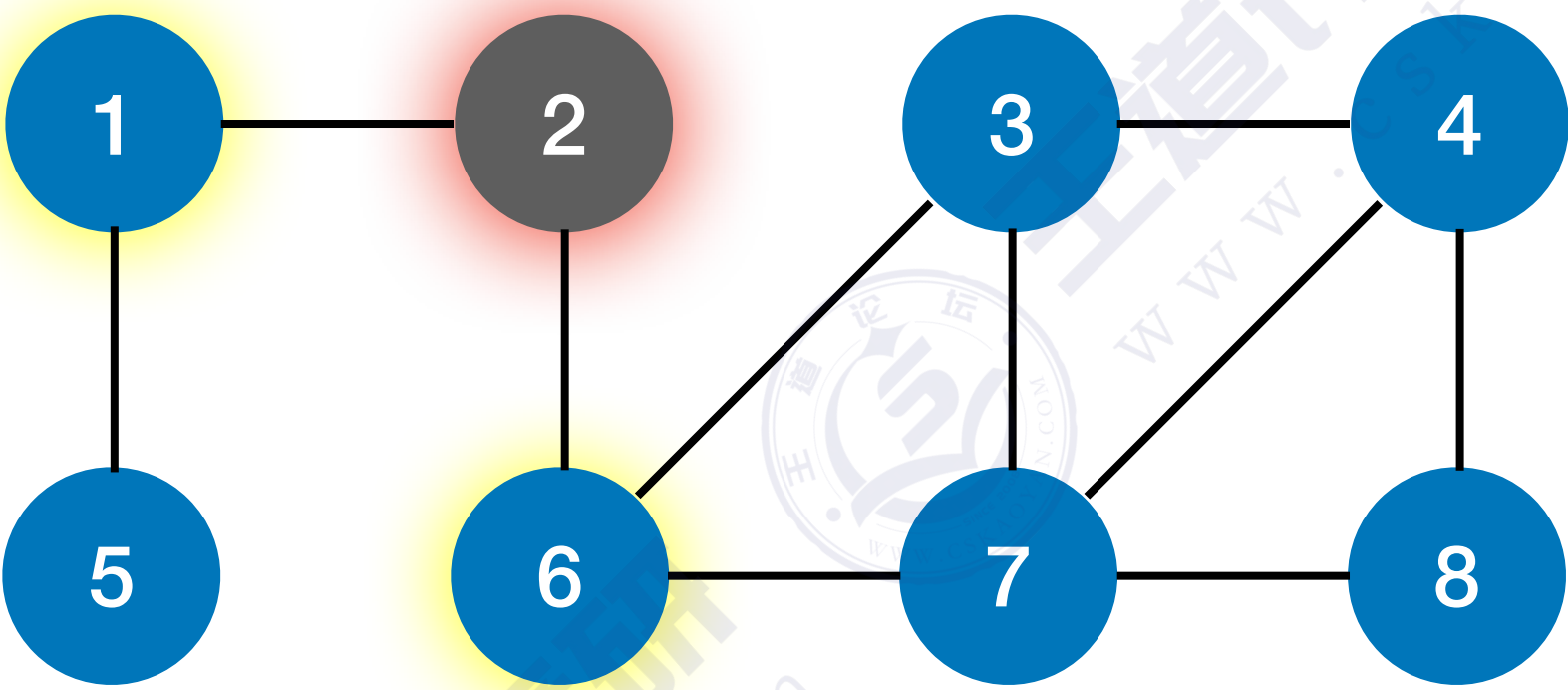


注：无权图可以视为一种特殊的带权图，只是每条边的权值都为1

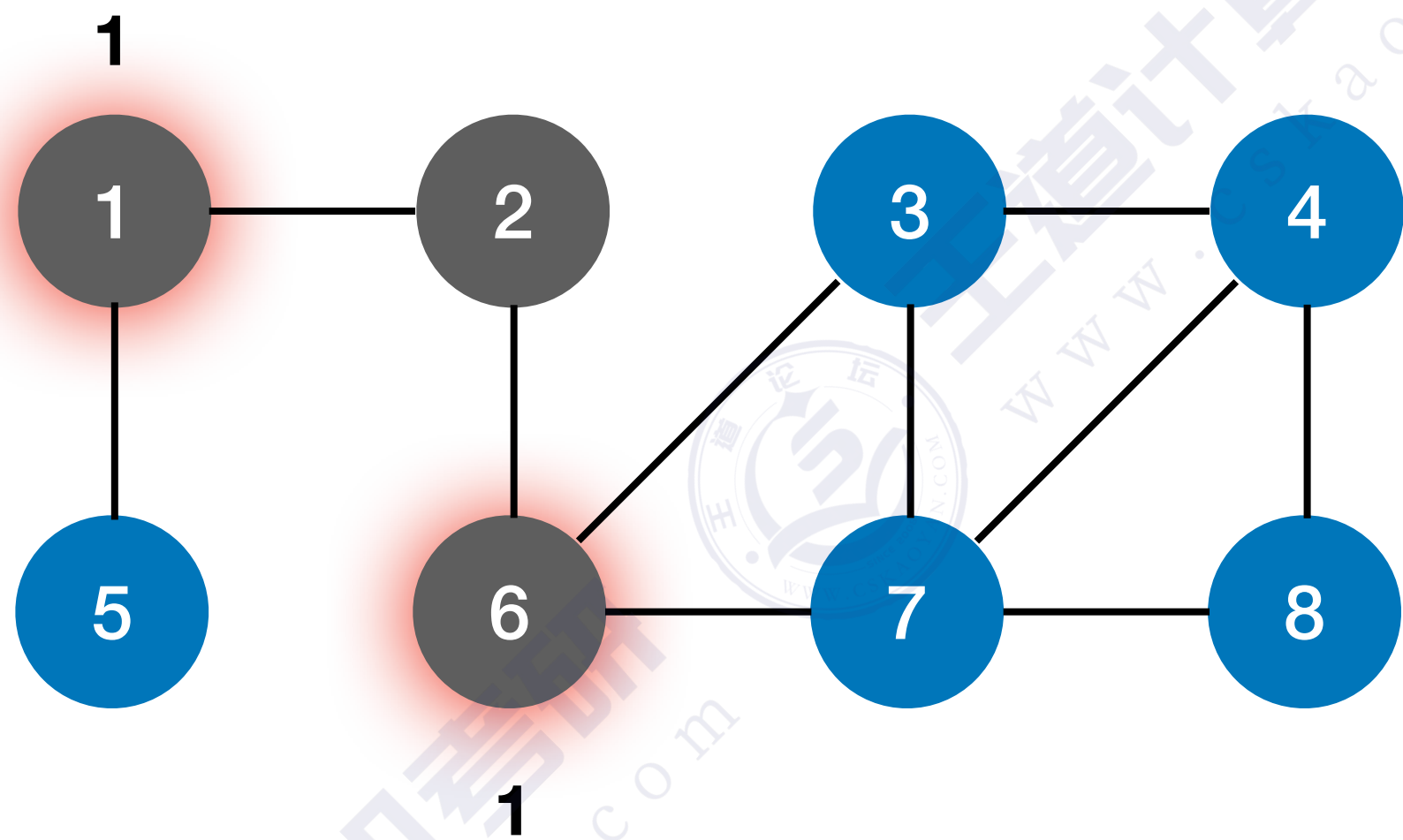
BFS求无权图的单源最短路径



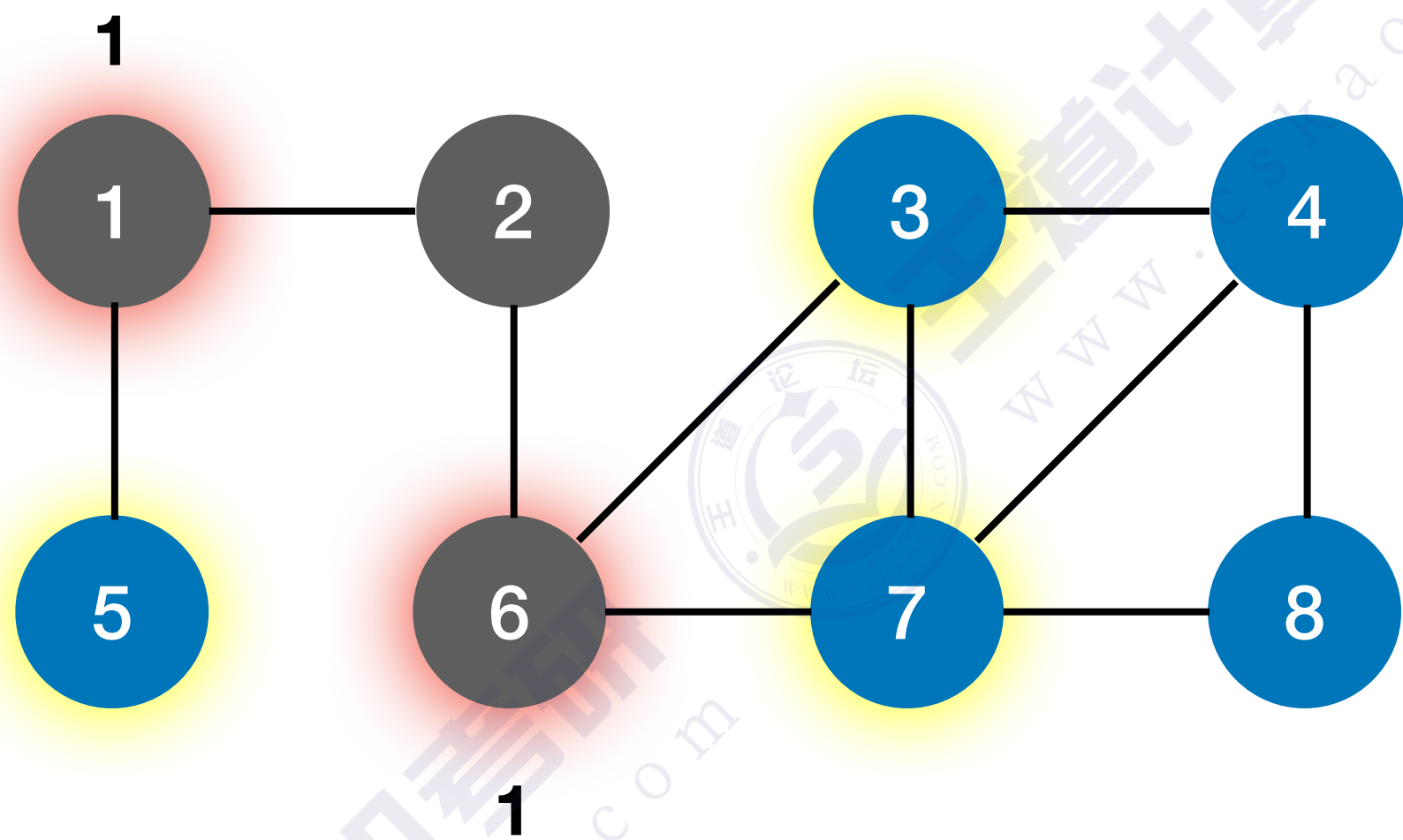
BFS求无权图的单源最短路径



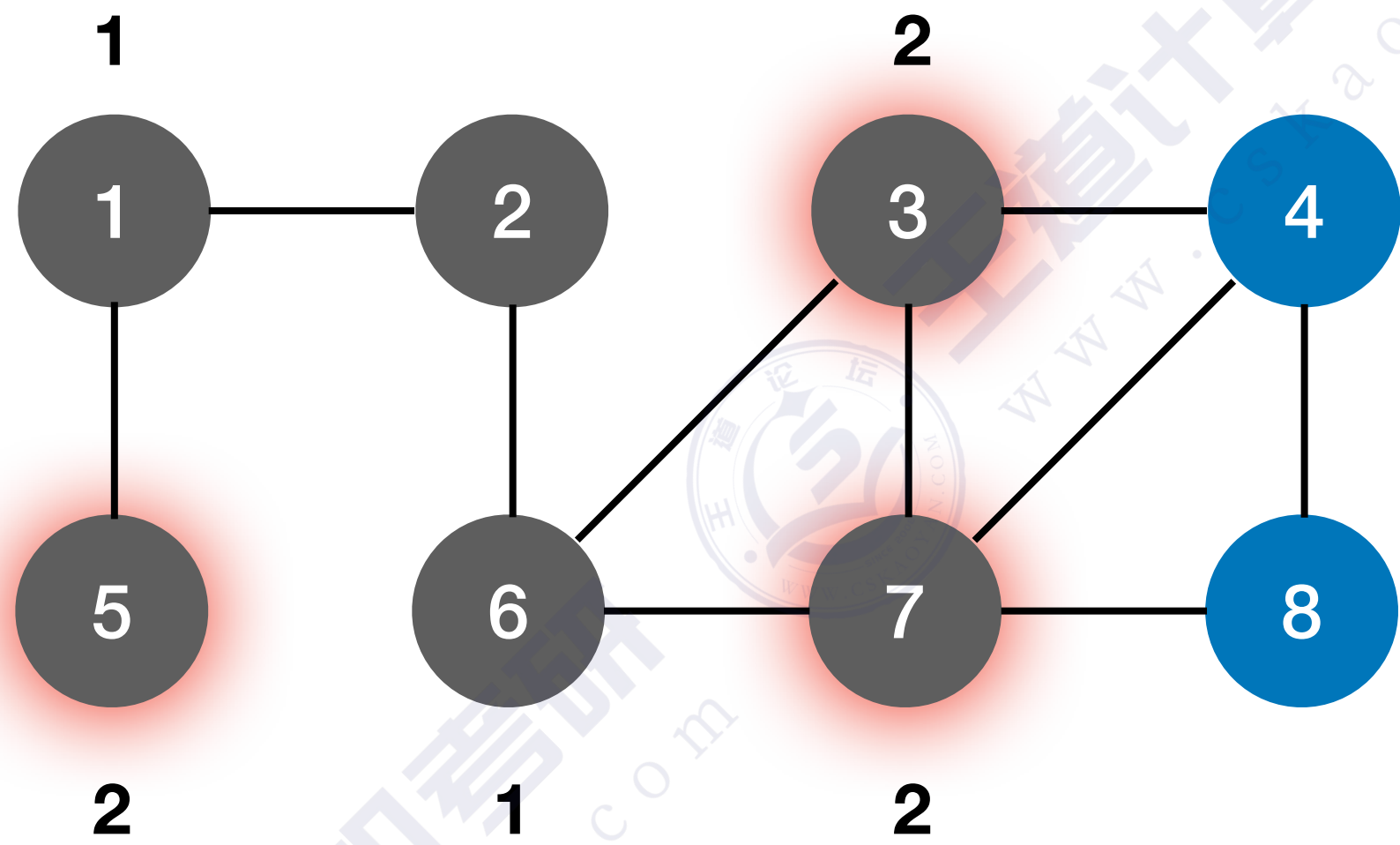
BFS求无权图的单源最短路径



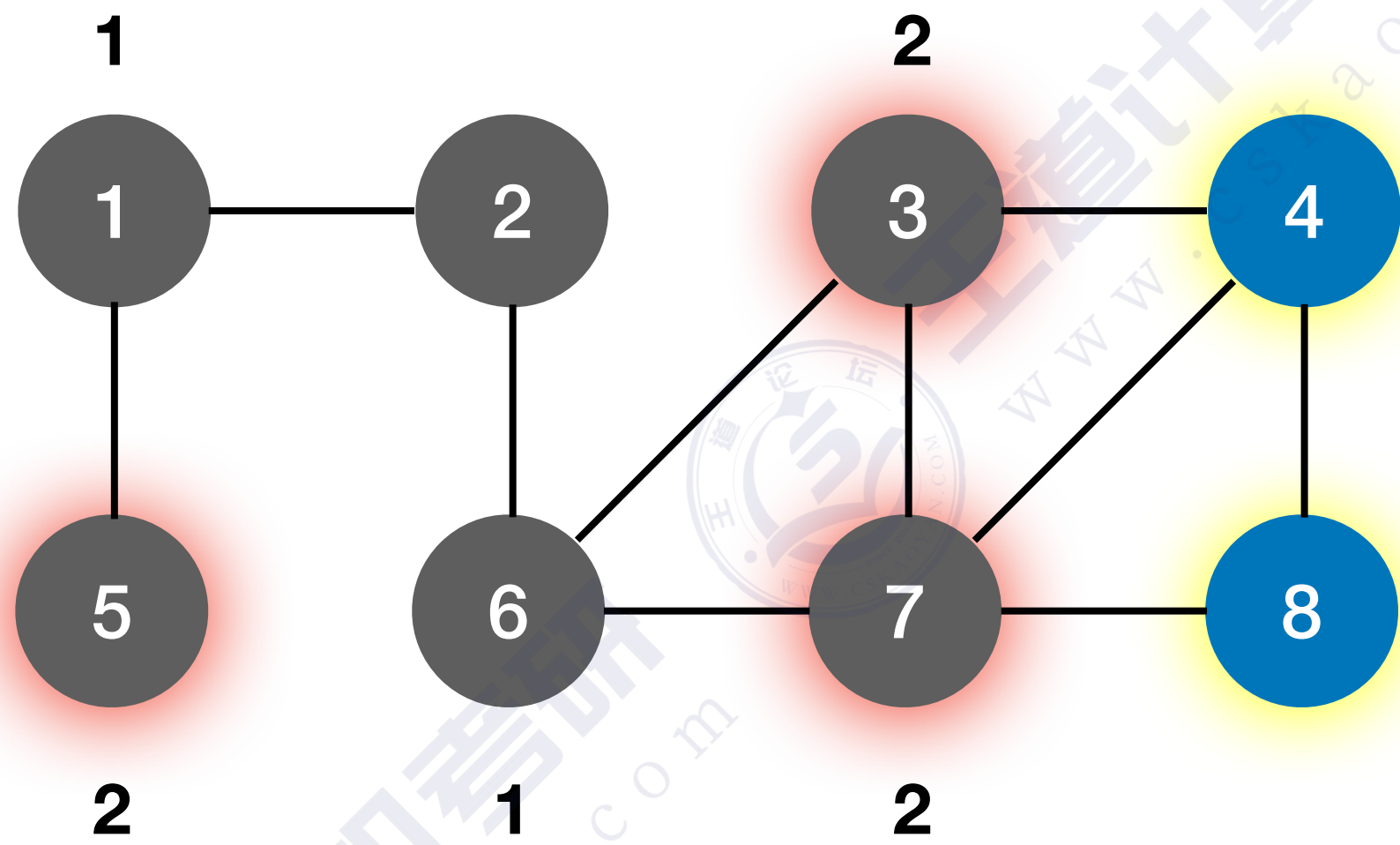
BFS求无权图的单源最短路径



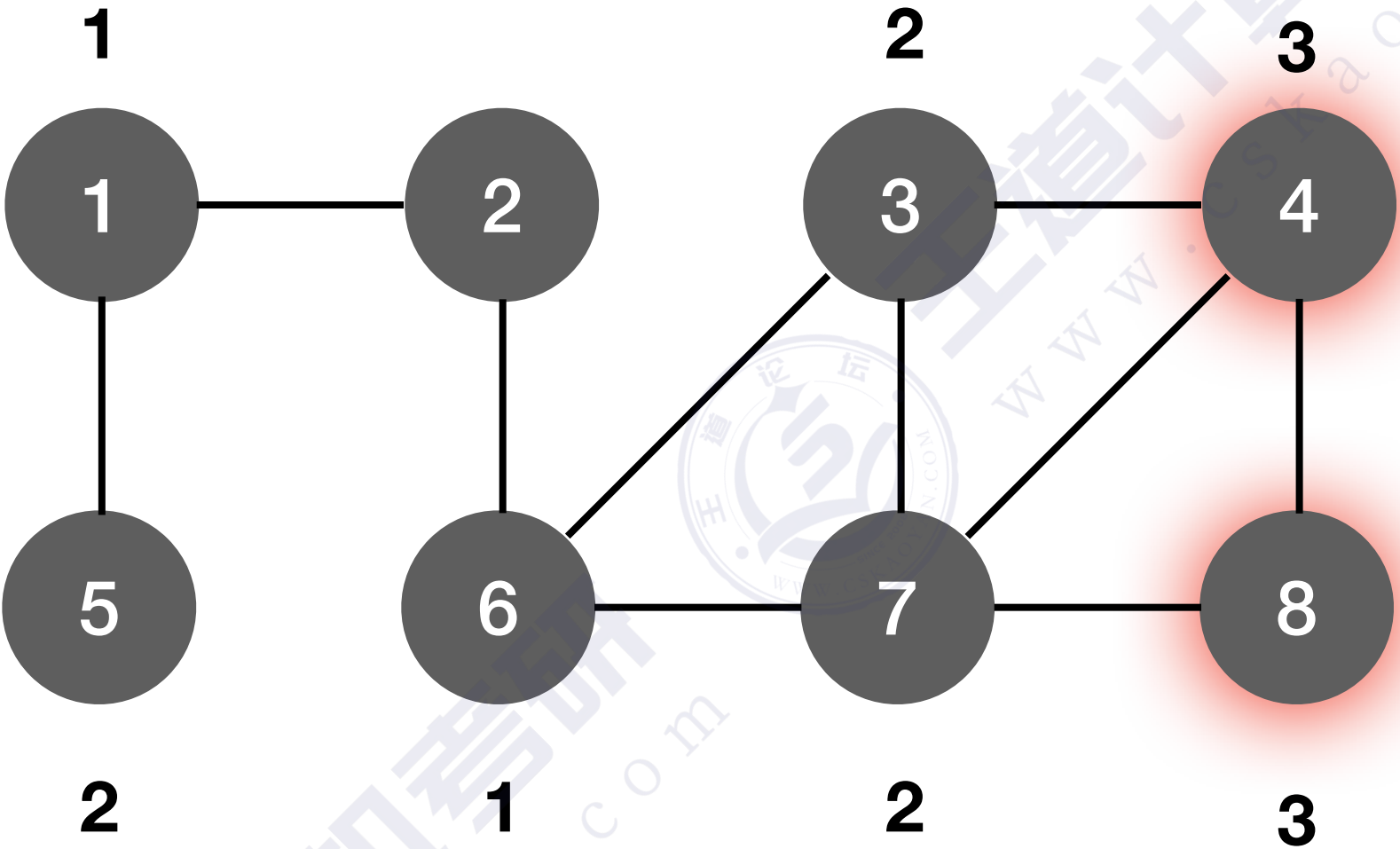
BFS求无权图的单源最短路径



BFS求无权图的单源最短路径

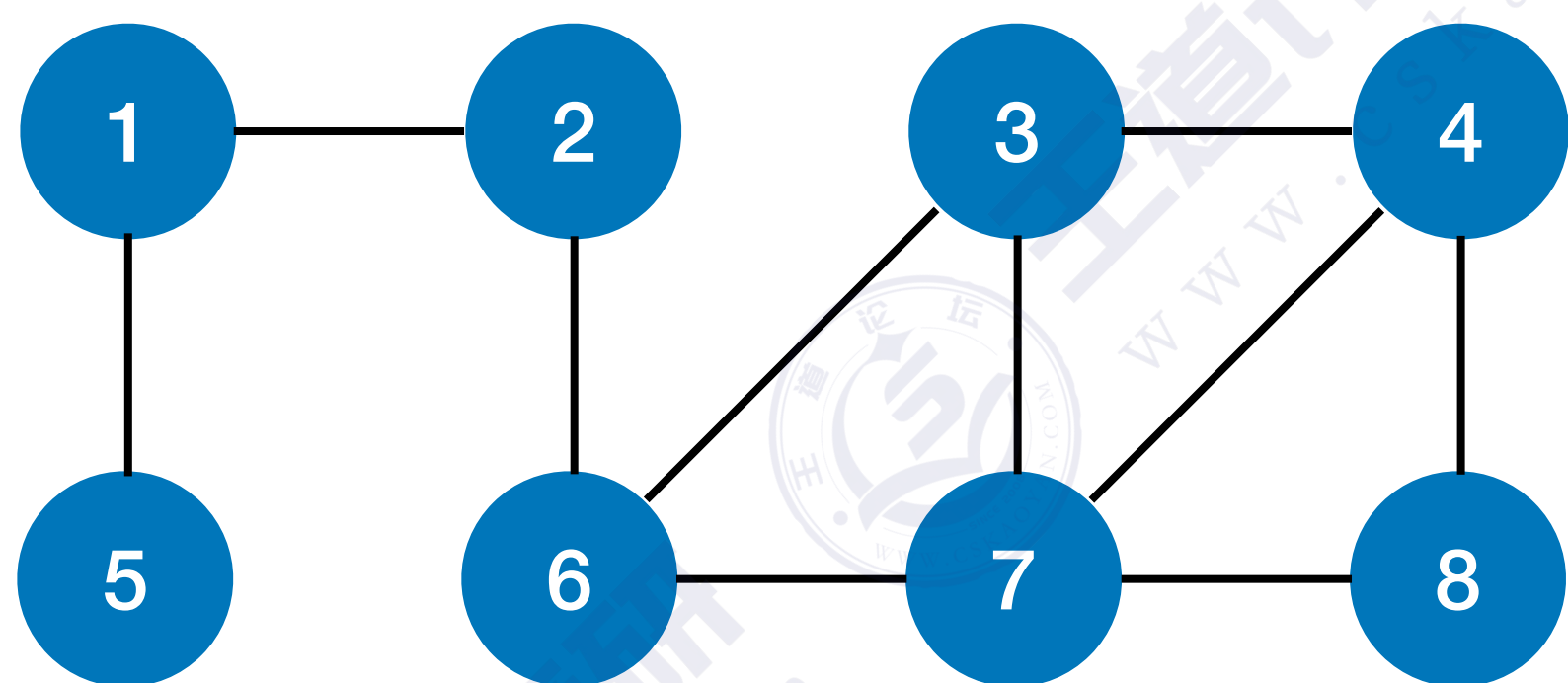


BFS求无权图的单源最短路径



代码实现

初始都为false

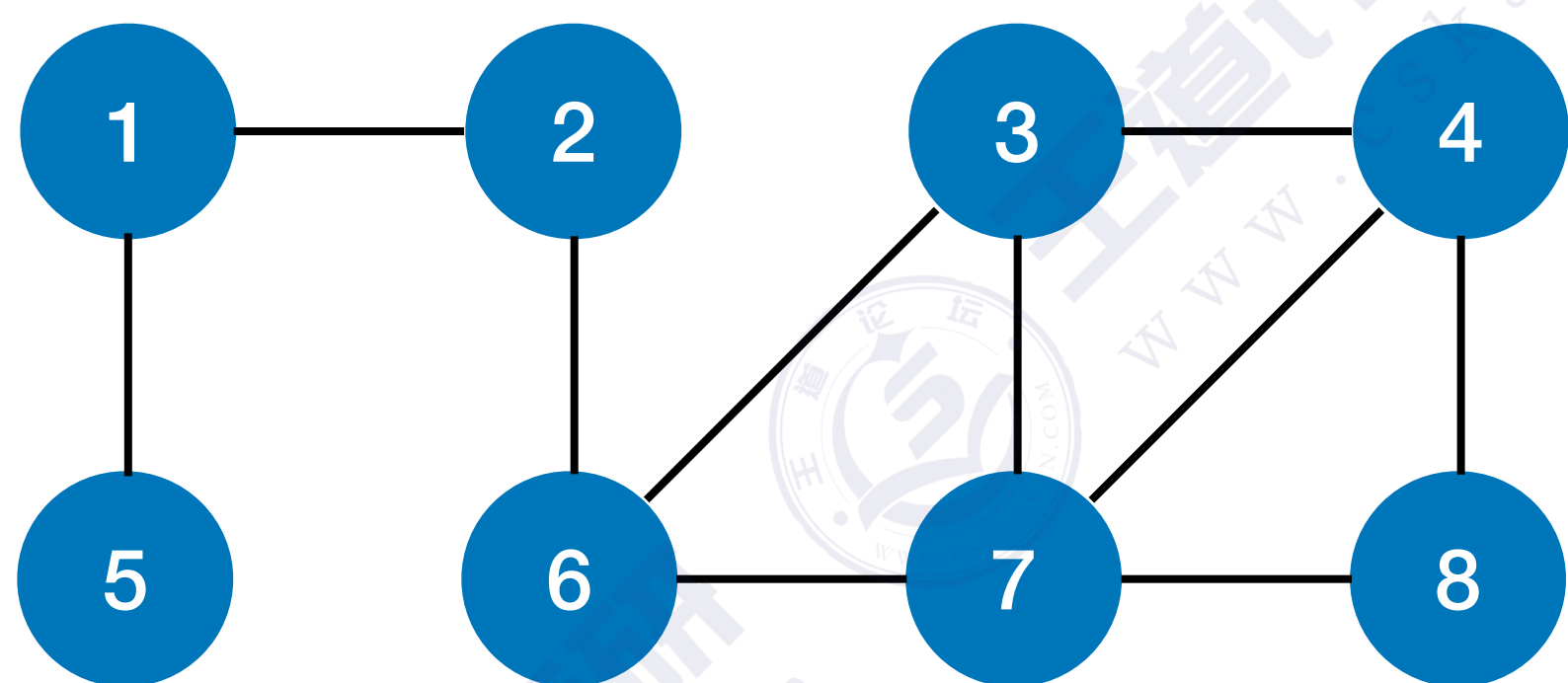


```
bool visited[MAX_VERTEX_NUM]; //访问标记数组

//广度优先遍历
void BFS(Graph G,int v){ //从顶点v出发，广度优先遍历图G
    visit(v); //访问初始顶点v
    visited[v]=TRUE; //对v做已访问标记
    Enqueue(Q,v); //顶点v入队列Q
    while(!isEmpty(Q)){
        Dequeue(Q,v); //顶点v出队列
        for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
            //检测v所有邻接点
            if(!visited[w]){ //w为v的尚未访问的邻接顶点
                visit(w); //访问顶点w
                visited[w]=TRUE; //对w做已访问标记
                Enqueue(Q,w); //顶点w入队列
            } //if
        } //while
    } }
```

	1	2	3	4	5	6	7	8
visited	false	false	false	false	false	false	false	false

代码实现

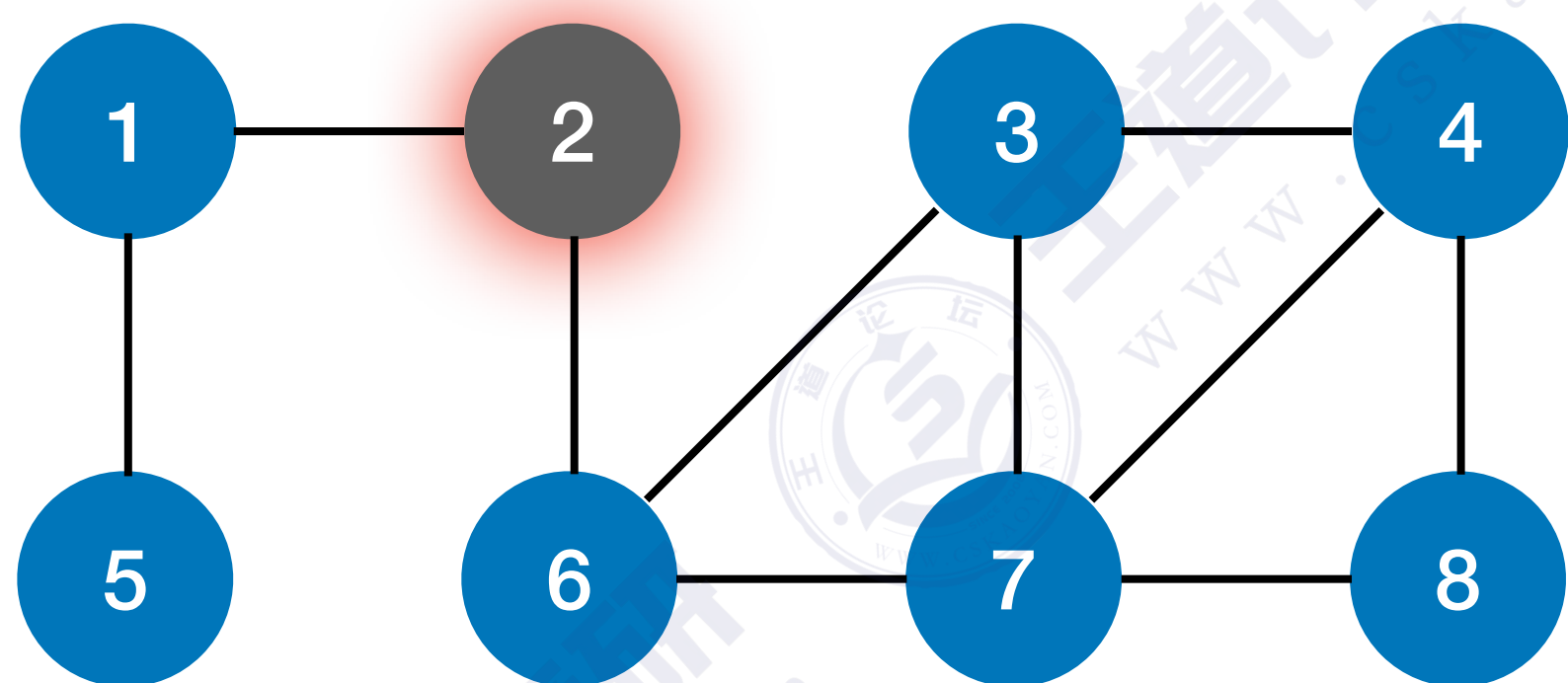


	1	2	3	4	5	6	7	8
d[]	∞	∞	∞	∞	∞	∞	∞	∞
path[]	-1	-1	-1	-1	-1	-1	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	false	false	false	false	false	false	false	false

代码实现



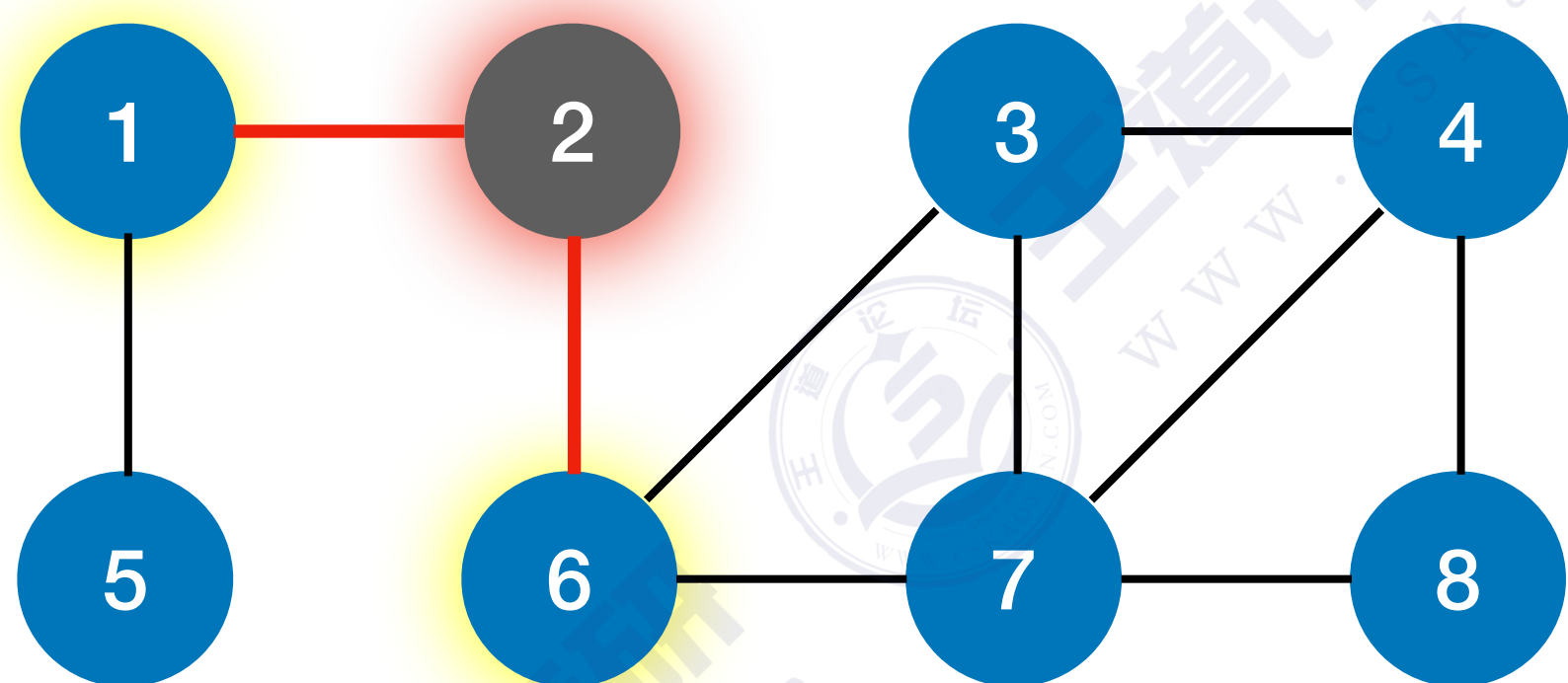
2

	1	2	3	4	5	6	7	8
d[]	∞	0	∞	∞	∞	∞	∞	∞
path[]	-1	-1	-1	-1	-1	-1	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	false	true	false	false	false	false	false	false

代码实现

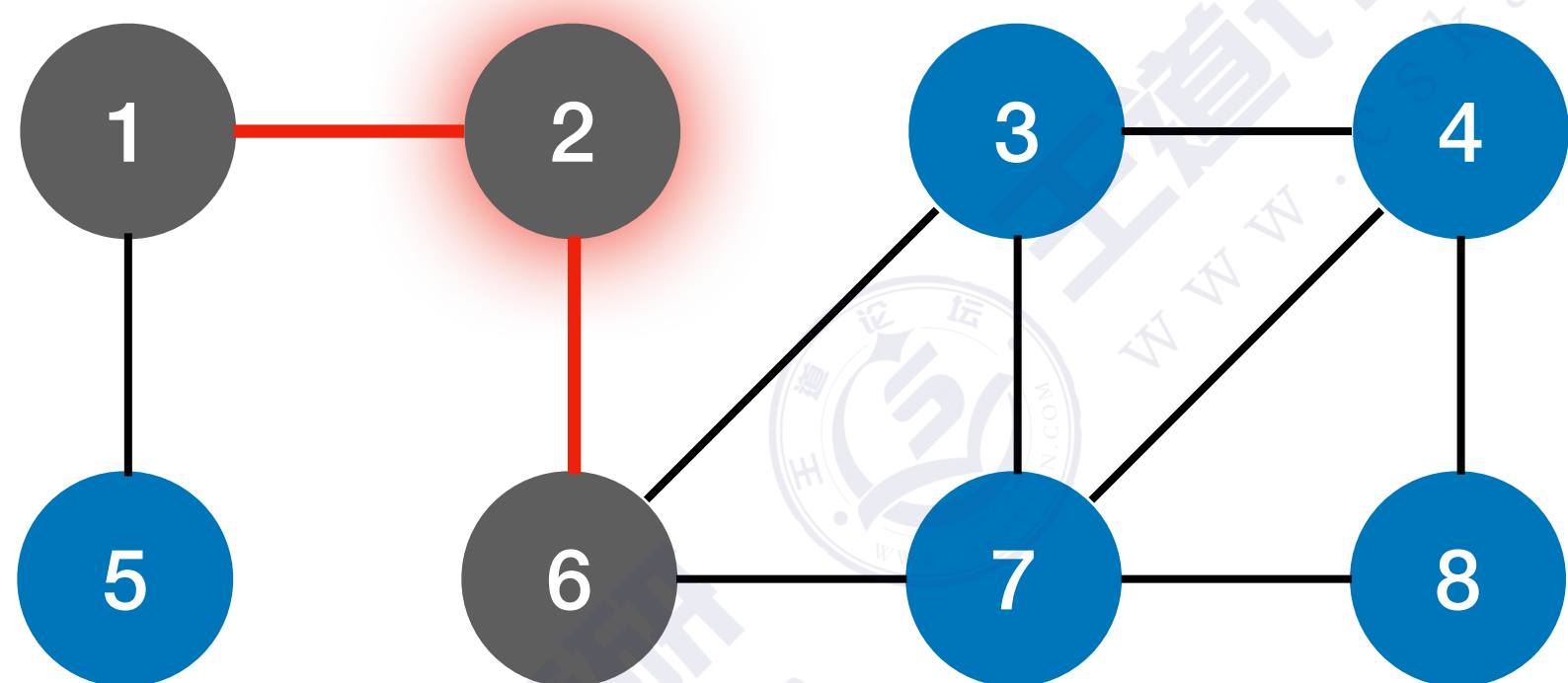


	1	2	3	4	5	6	7	8
d[]	∞	0	∞	∞	∞	∞	∞	∞
path[]	-1	-1	-1	-1	-1	-1	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	false	true	false	false	false	false	false	false

代码实现

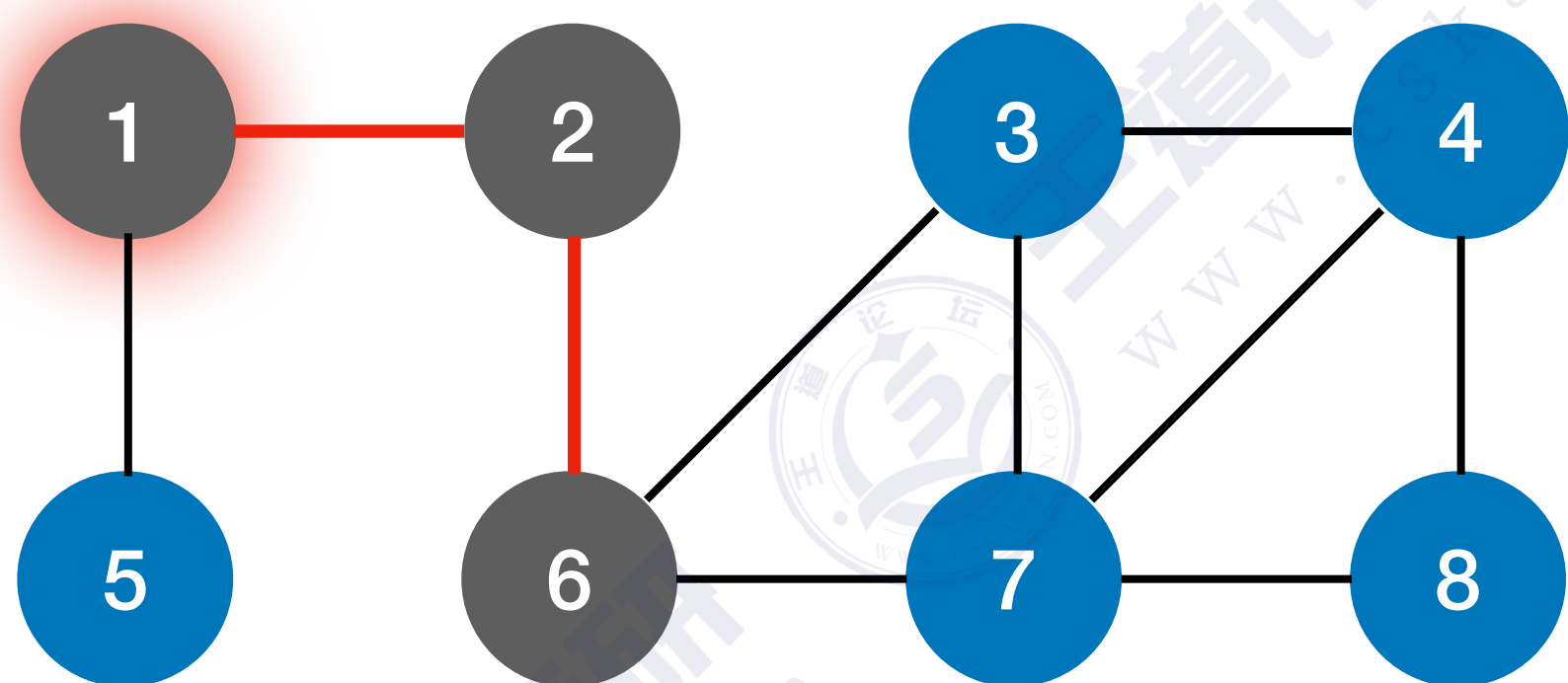


	1	2	3	4	5	6	7	8
d[]	1	0	∞	∞	∞	1	∞	∞
path[]	2	-1	-1	-1	-1	2	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	false	false	false	true	false	false

代码实现

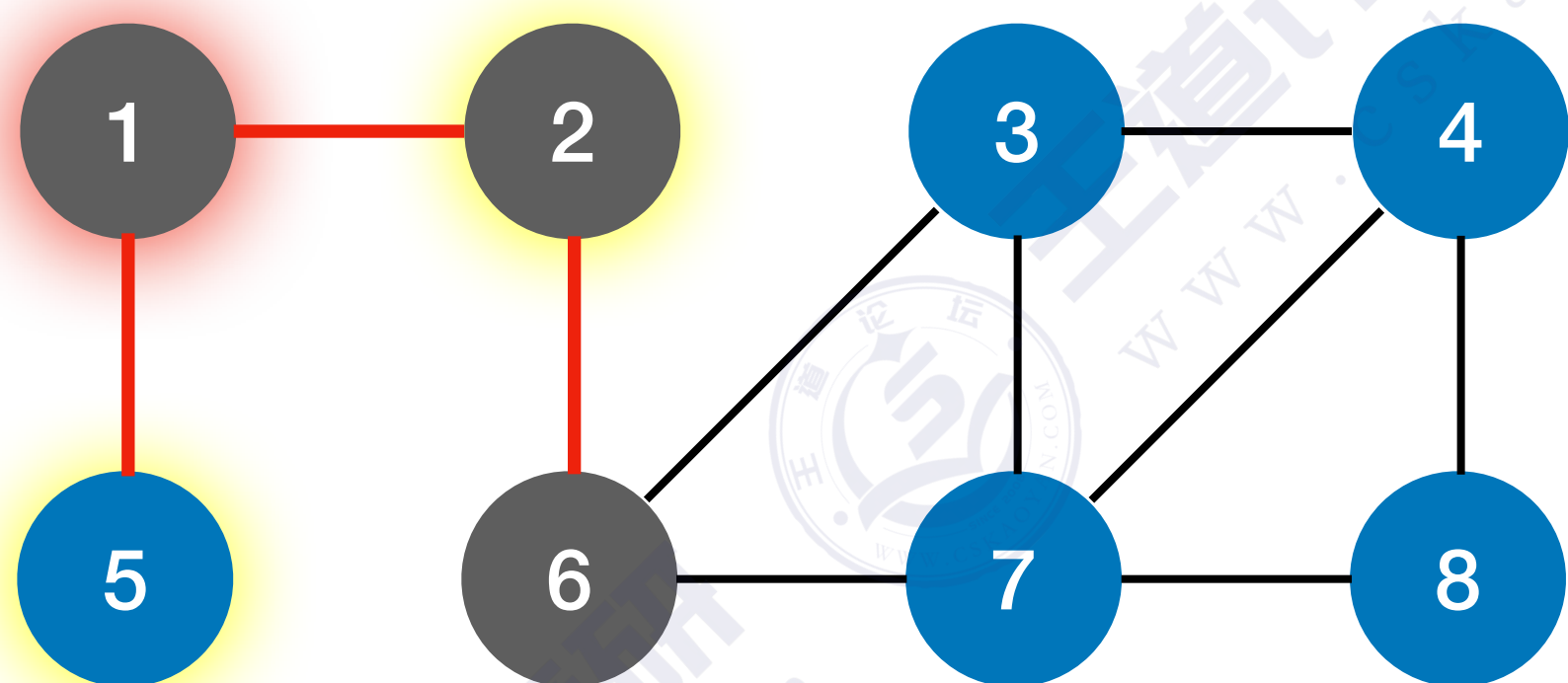


	1	2	3	4	5	6	7	8
d[]	1	0	∞	∞	∞	1	∞	∞
path[]	2	-1	-1	-1	-1	2	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	false	false	false	true	false	false

代码实现

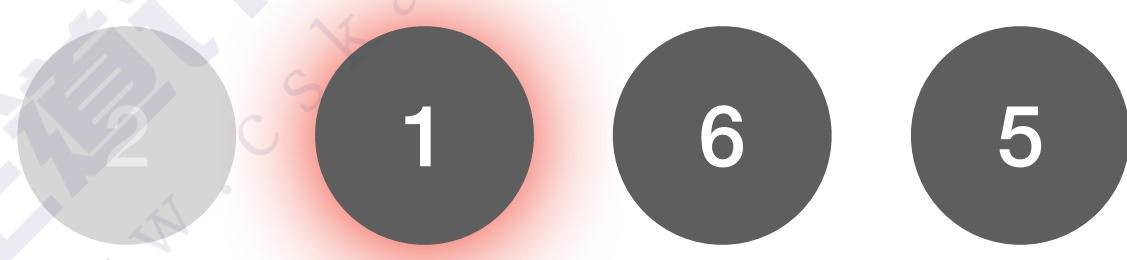
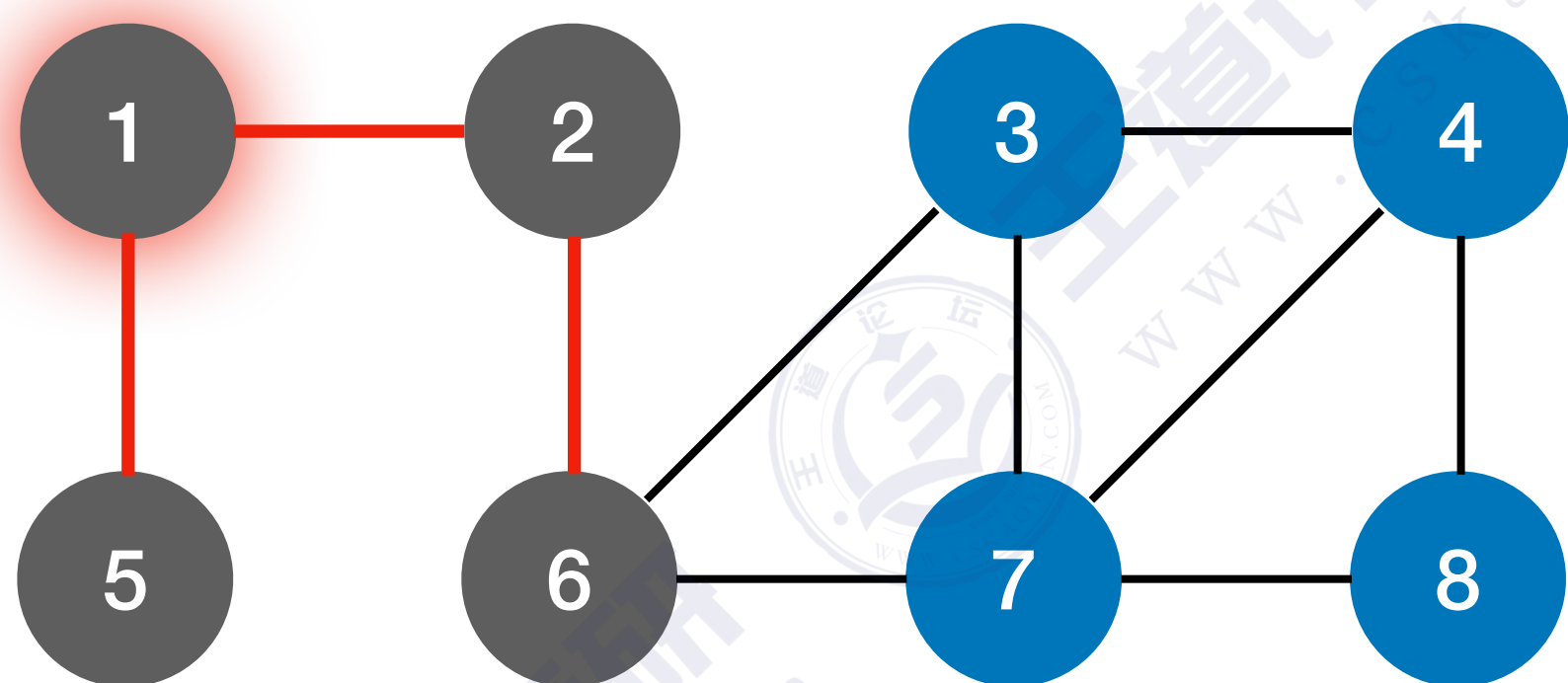


	1	2	3	4	5	6	7	8
d[]	1	0	∞	∞	∞	1	∞	∞
path[]	2	-1	-1	-1	-1	2	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	false	false	false	true	false	false

代码实现

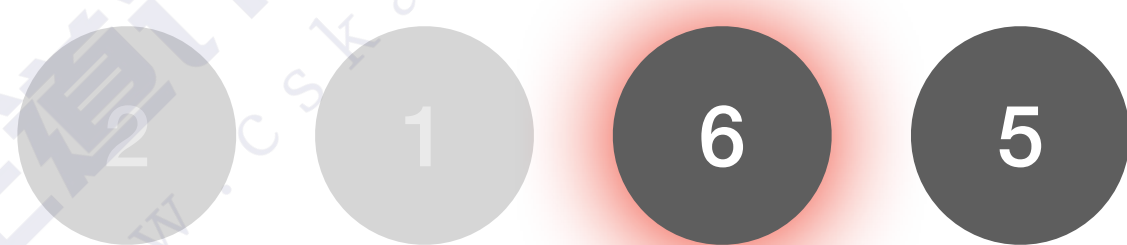
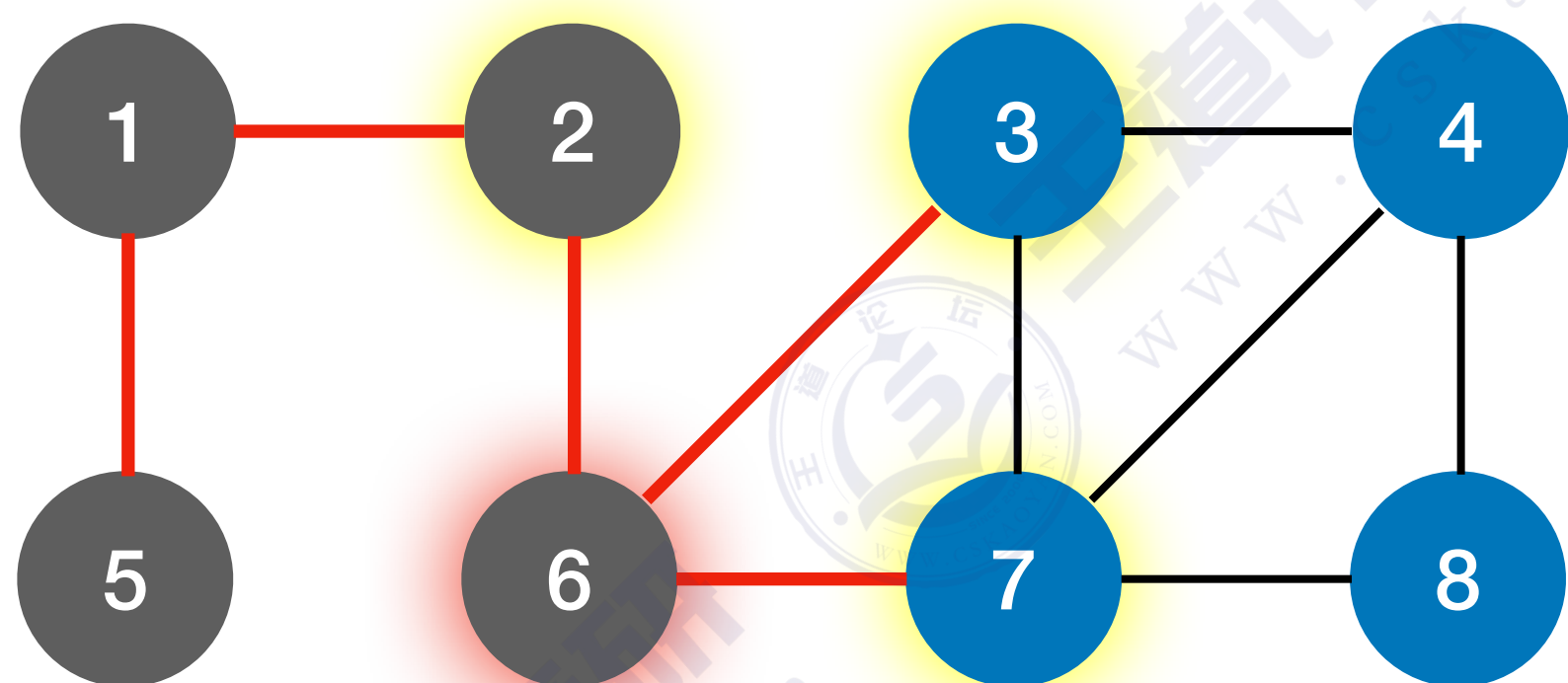


	1	2	3	4	5	6	7	8
d[]	1	0	∞	∞	2	1	∞	∞
path[]	2	-1	-1	-1	1	2	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	false	false	true	true	false	false

代码实现

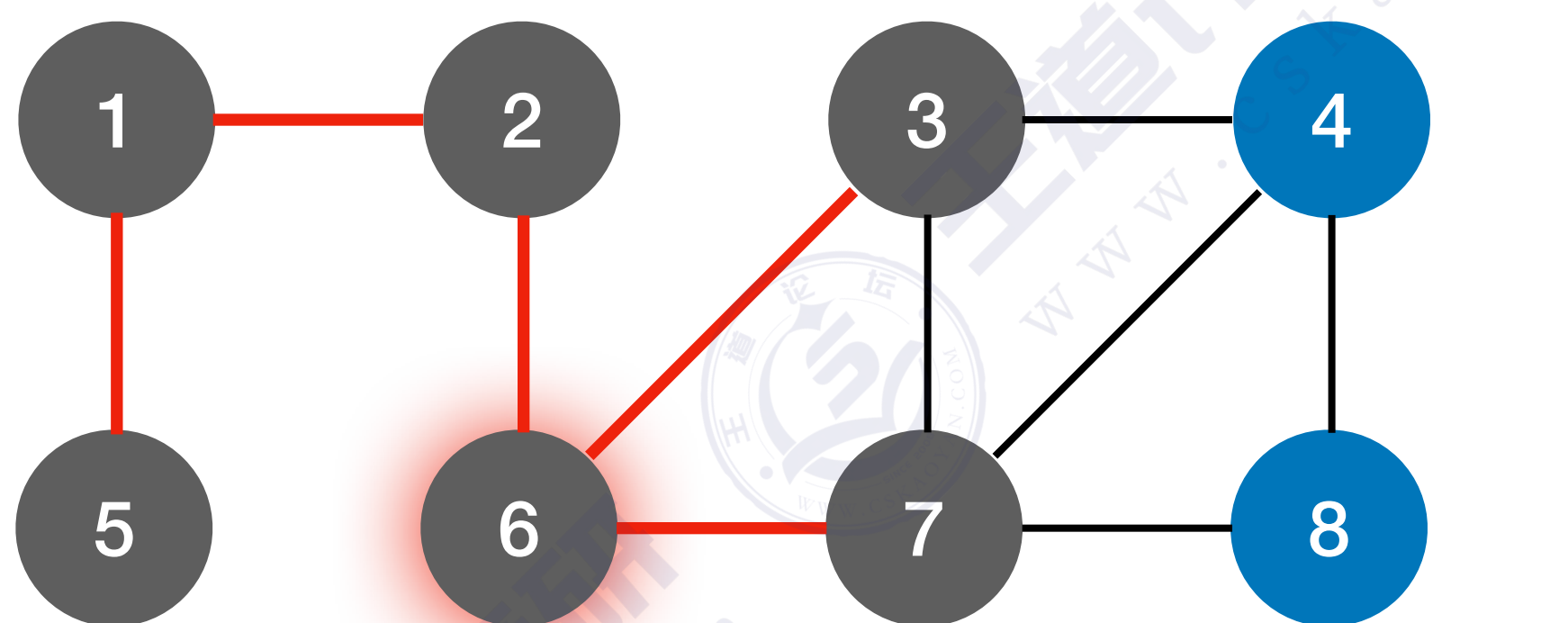


	1	2	3	4	5	6	7	8
d[]	1	0	∞	∞	2	1	∞	∞
path[]	2	-1	-1	-1	1	2	-1	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	false	false	true	true	false	false

代码实现

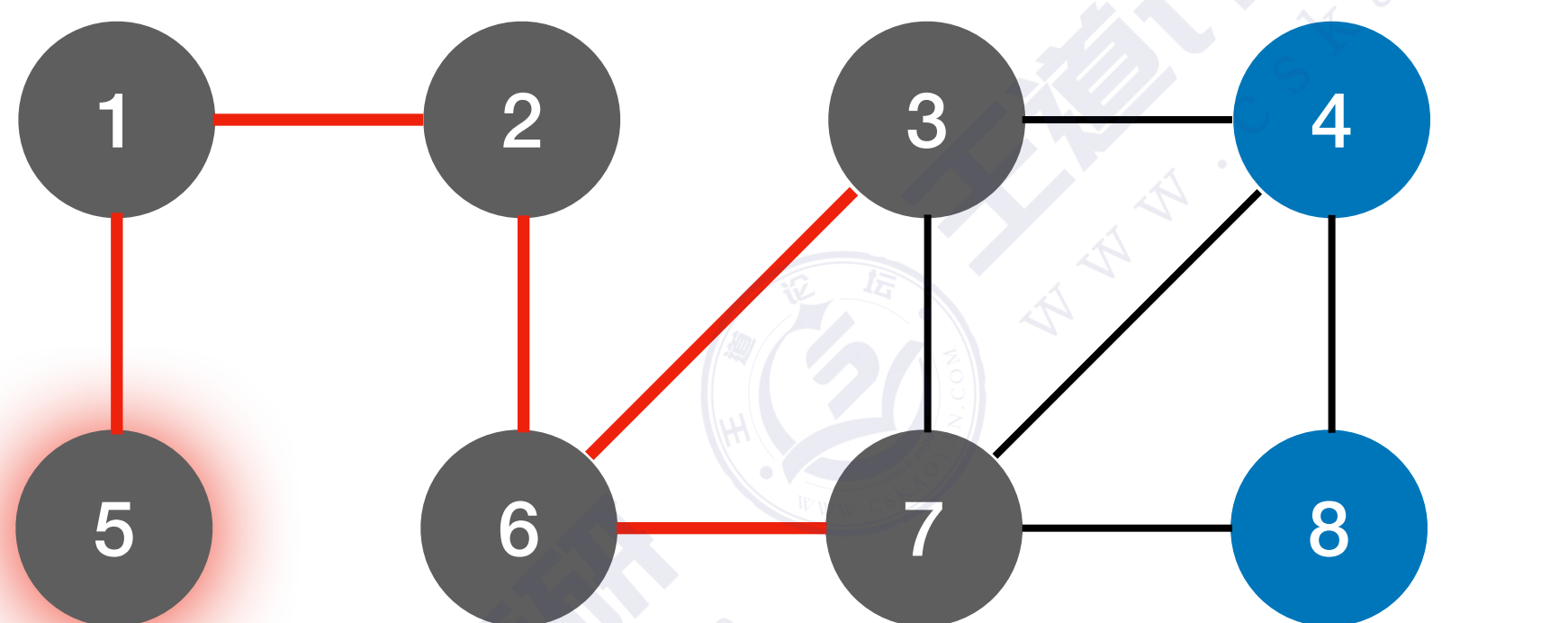


	1	2	3	4	5	6	7	8
d[]	1	0	2	∞	2	1	2	∞
path[]	2	-1	6	-1	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	false	true	true	true	false

代码实现

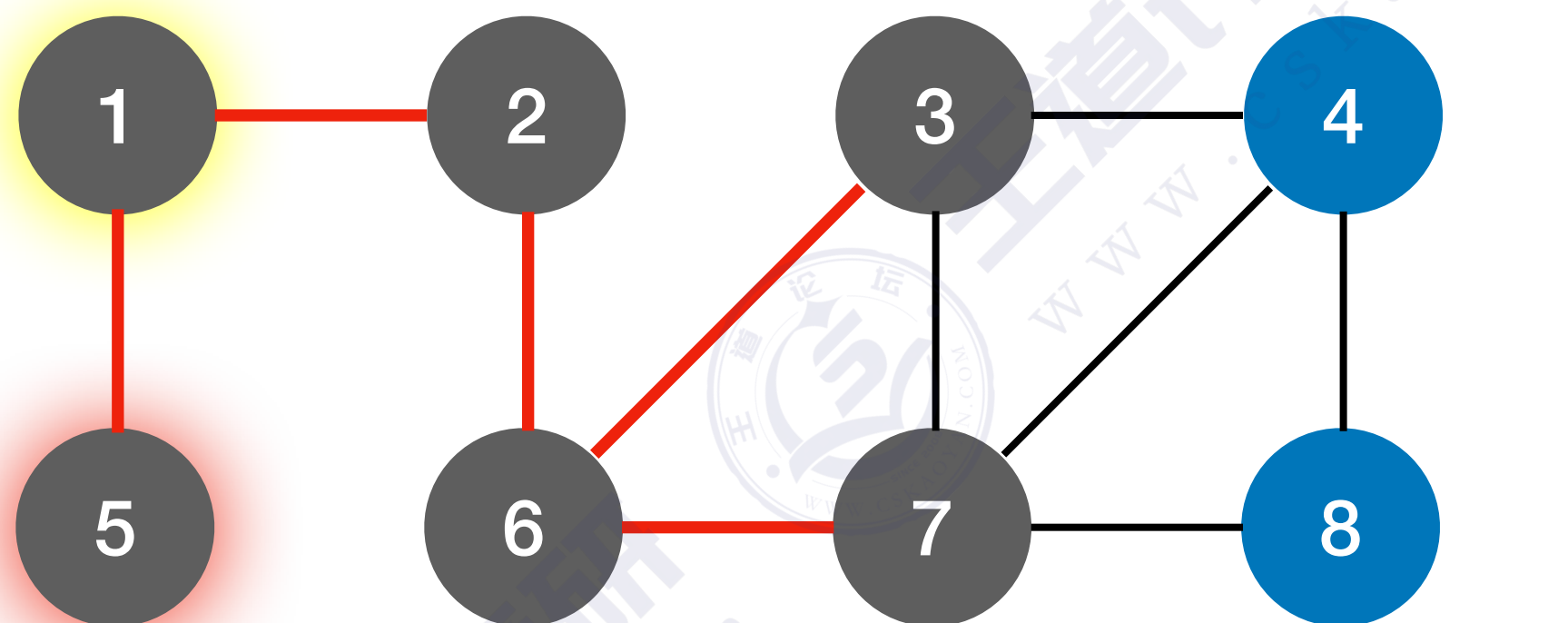


	1	2	3	4	5	6	7	8
d[]	1	0	2	∞	2	1	2	∞
path[]	2	-1	6	-1	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	false	true	true	true	false

代码实现



	1	2	3	4	5	6	7	8
d[]	1	0	2	∞	2	1	2	∞
path[]	2	-1	6	-1	1	2	6	-1

//求顶点 u 到其他顶点的最短路径

```
void BFS_MIN_Distance(Graph G,int u){
```

// $d[i]$ 表示从 u 到 i 结点的最短路径

```
for(i=0;i<G.vexnum;++i){
```

```
d[i]=∞; //初始化路径长度
```

```
path[i]=-1; //最短路径从哪个顶点过来
```

}

```
d[u]=0;
```

```
visited[u]=TRUE;
```

```
EnQueue(Q,u);
```

```
while(!isEmpty(Q)){
```

```
DeQueue(Q,u);
```

```
for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
```

```
if(!visited[w]){
```

```
d[w]=d[u]+1;
```

```
path[w]=u;
```

```
visited[w]=TRUE:
```

EnQueue(0,w):

```
    } //if
```

```
//while
```

}

//BFS算法主过程

```
//队头元素u出队
```

// w 为 u 的尚未访问的邻接顶点

```
// 路径长度加1
```

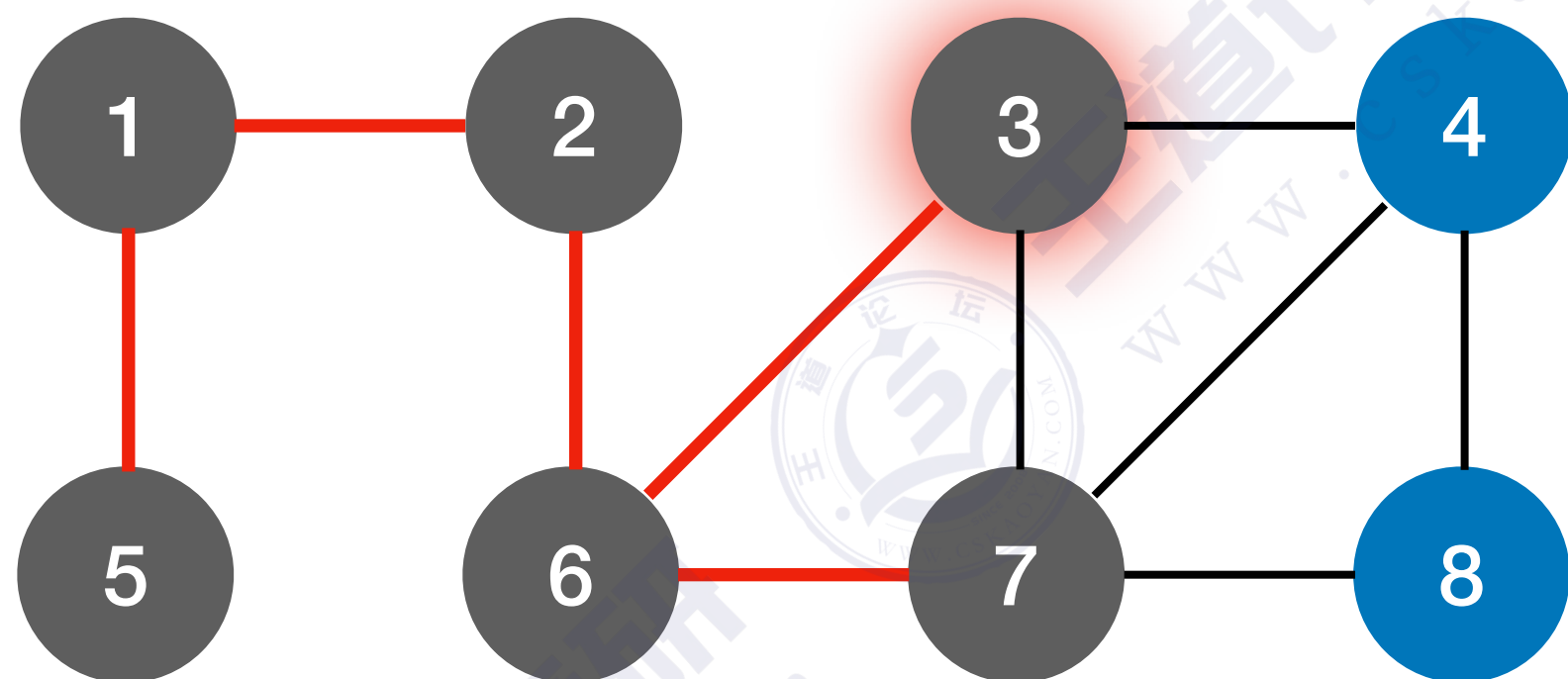
```
//最短路径应从u到w
```

```
//设已访问标记
```

```
//顶点w入队
```

	1	2	3	4	5	6	7	8
visited	true	true	true	false	true	true	true	false

代码实现



	1	2	3	4	5	6	7	8
d[]	1	0	2	∞	2	1	2	∞
path[]	2	-1	6	-1	1	2	6	-1

//求顶点 u 到其他顶点的最短路径

```
void BFS_MIN_Distance(Graph G,int u){
```

// $d[i]$ 表示从 u 到 i 结点的最短路径

```
for(i=0;i<G.vexnum;++i){
```

```
d[i]=∞; //初始化路径长度
```

```
path[i]=-1; //最短路径从哪个顶点过来
```

}

```
d[u]=0;
```

```
visited[u]=TRUE;
```

```
EnQueue(Q,u);
```

```
while(!isEmpty(Q)){
```

DeQueue(Q,u);

```
for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
```

```
if(!visited[w]){
```

```
d[w]=d[u]+1;
```

```
path[w]=u;
```

```
visited[w]=TRUE;
```

EnQueue(0,w):

```
    } // if
```

```
//while
```

}

//BFS算法主过程

```
//队头元素u出队
```

//w为u的尚未访问的邻接顶点

```
// 路径长度加1
```

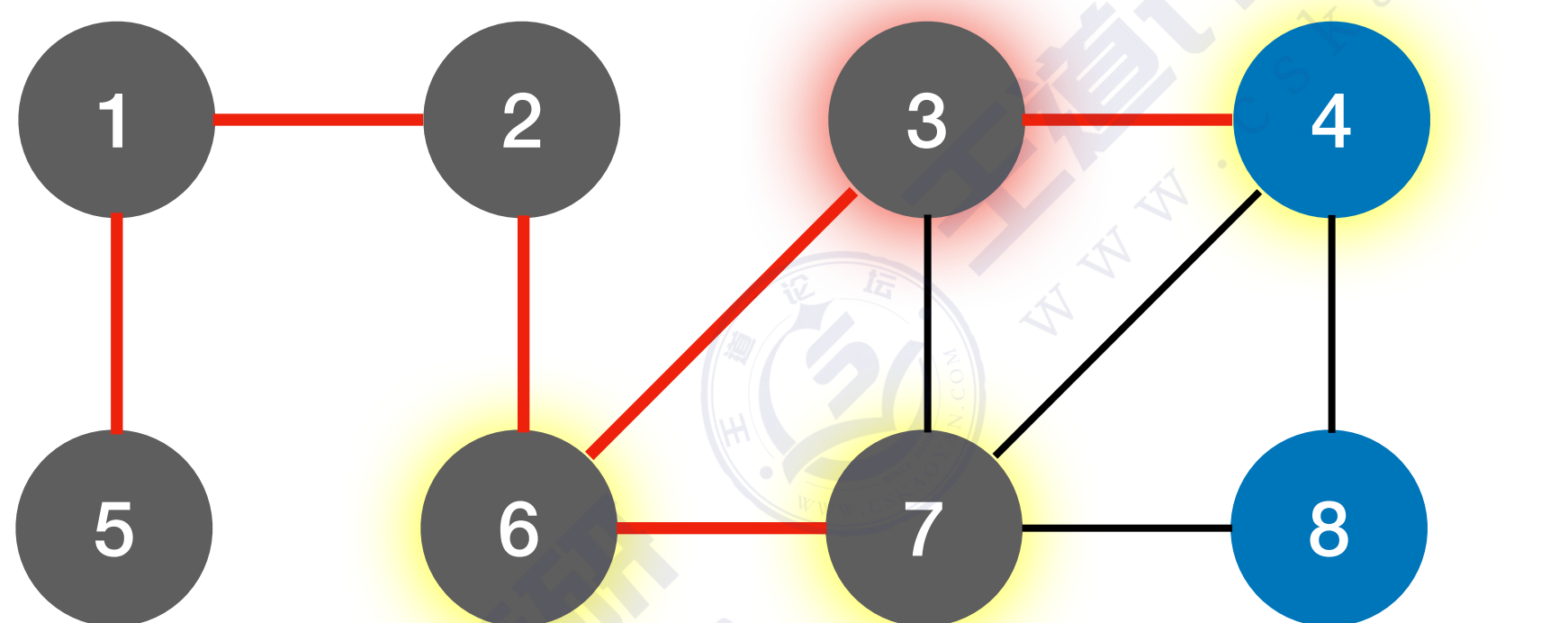
```
//最短路径应从u到w
```

```
//设已访问标记
```

```
//顶点w入队
```

	1	2	3	4	5	6	7	8
visited	true	true	true	false	true	true	true	false

代码实现

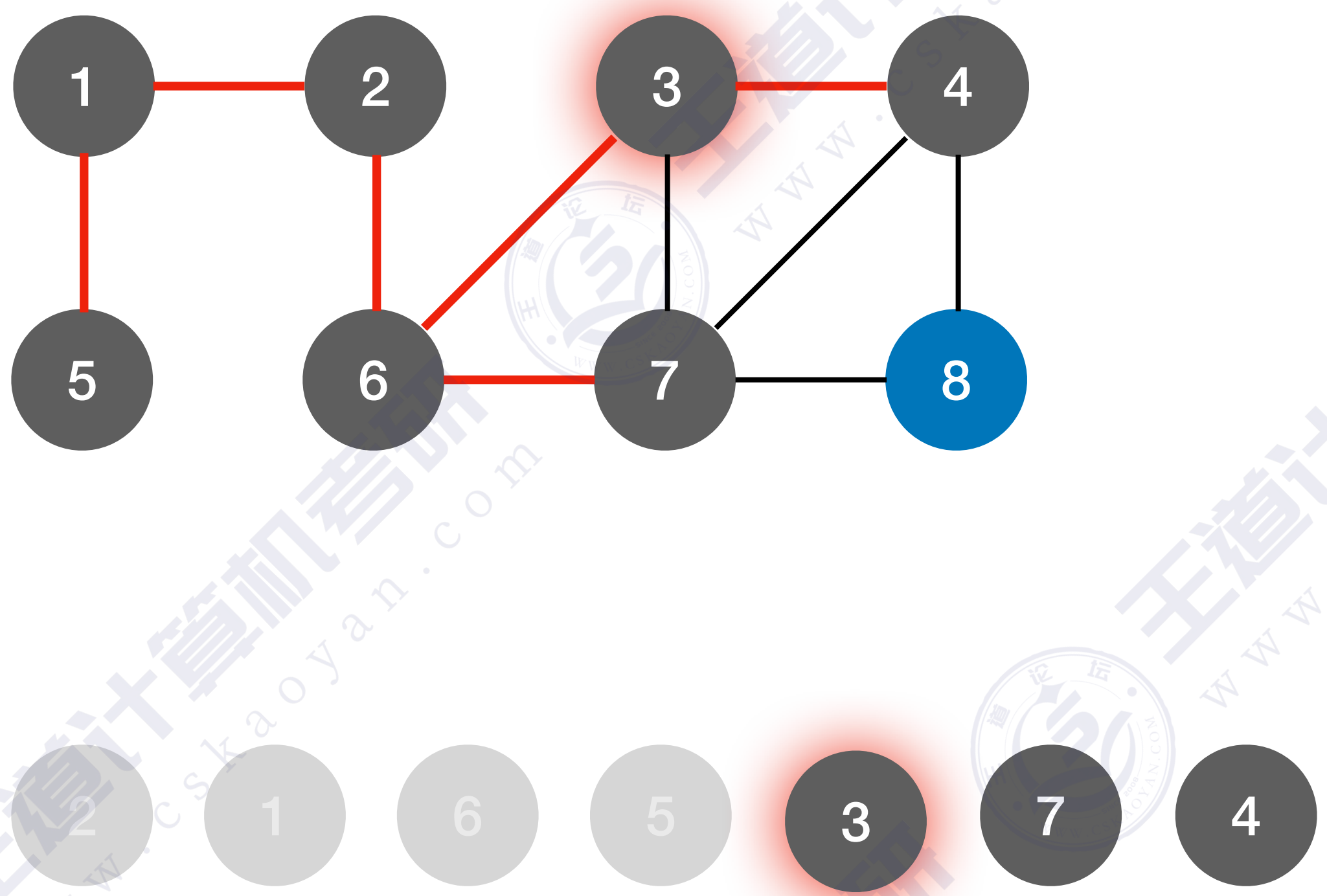


	1	2	3	4	5	6	7	8
d[]	1	0	2	∞	2	1	2	∞
path[]	2	-1	6	-1	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	false	true	true	true	false

代码实现

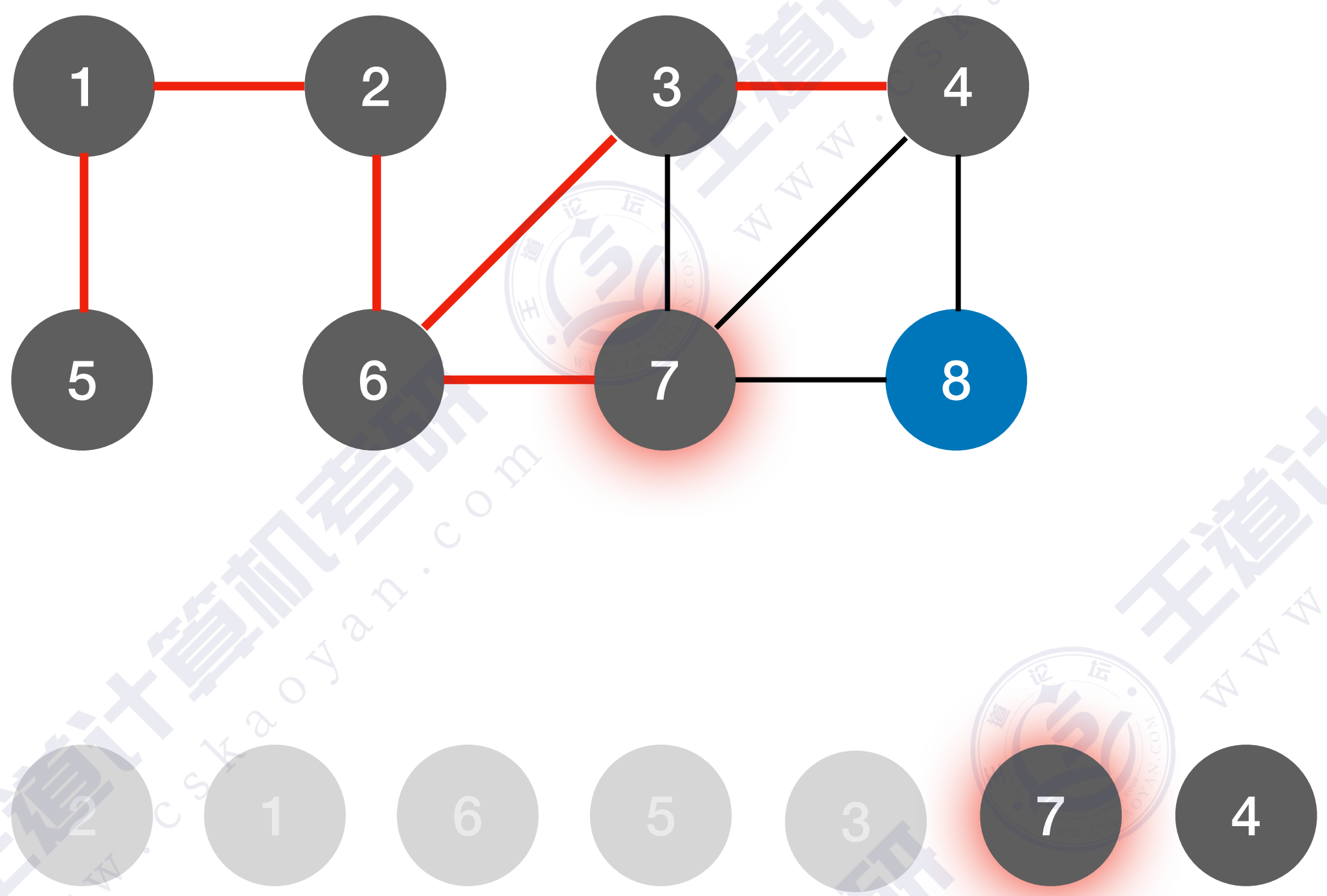


	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	∞
path[]	2	-1	6	3	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	false

代码实现

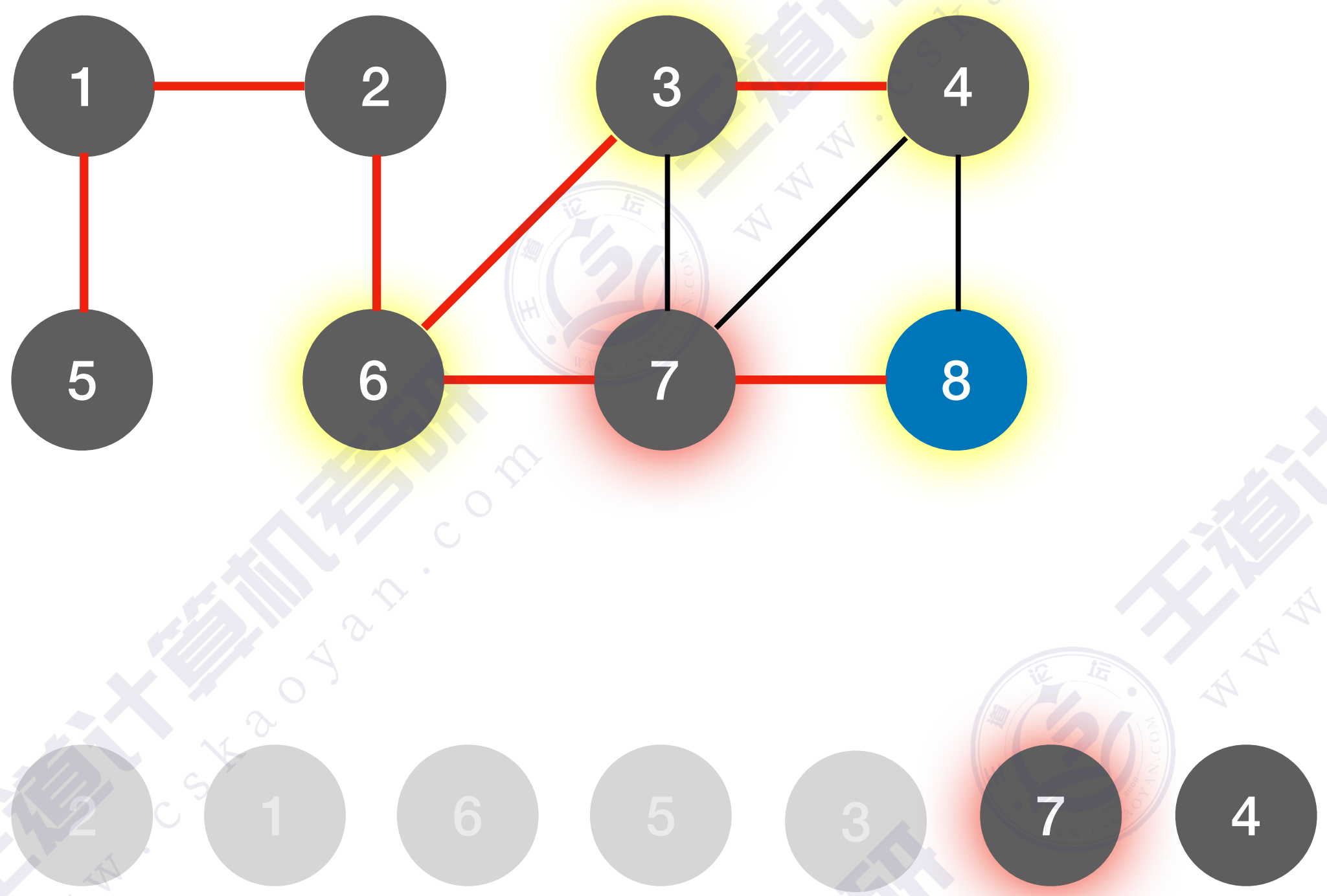


	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	∞
path[]	2	-1	6	3	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	false

代码实现

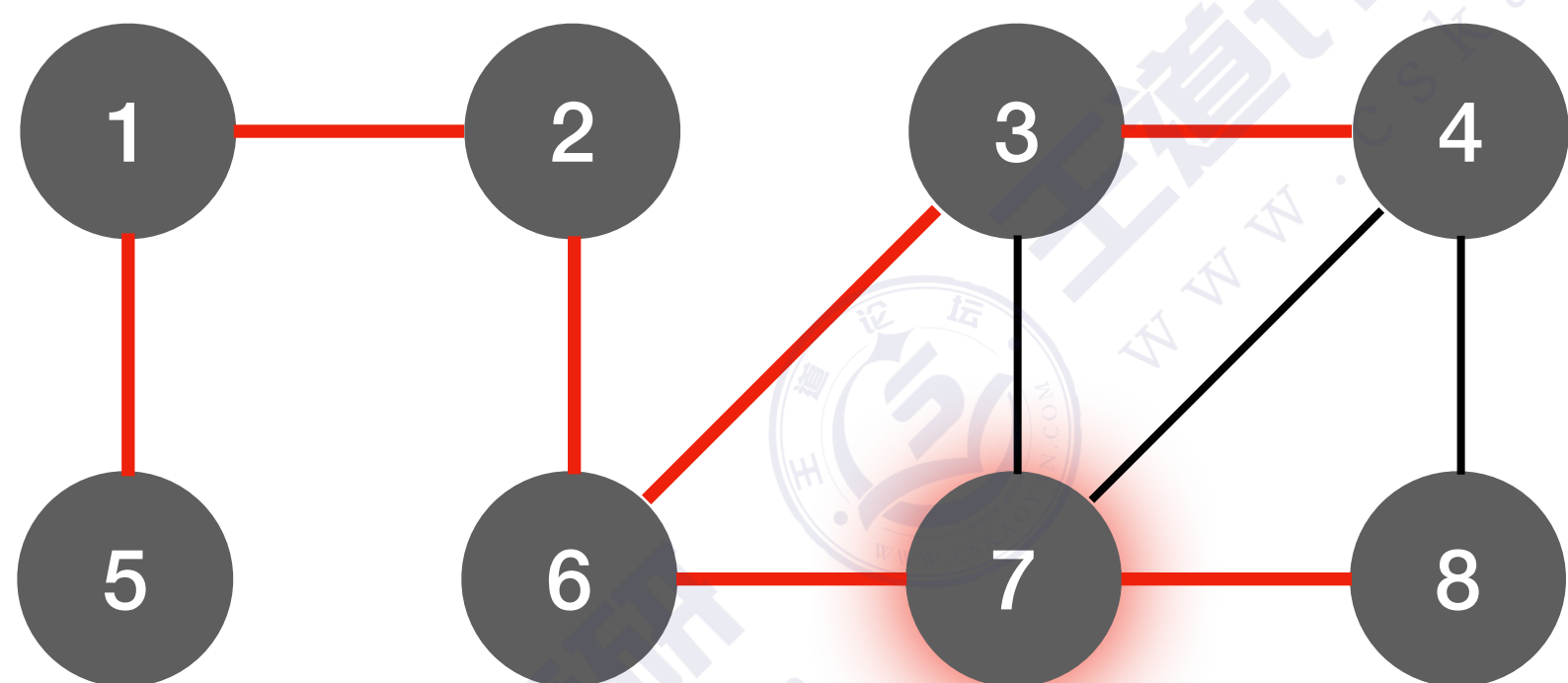


	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	∞
path[]	2	-1	6	3	1	2	6	-1

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	false

代码实现

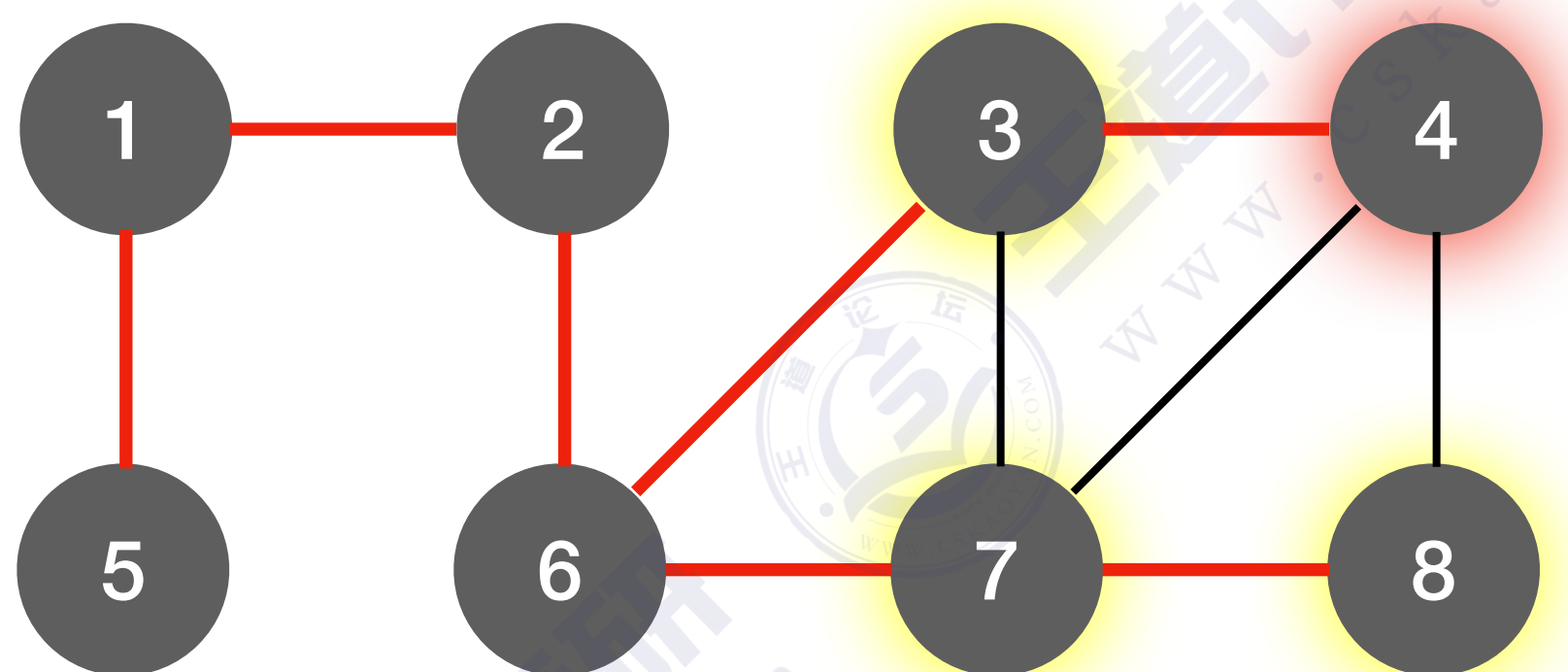


	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	3
path[]	2	-1	6	3	1	2	6	7

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	true

代码实现

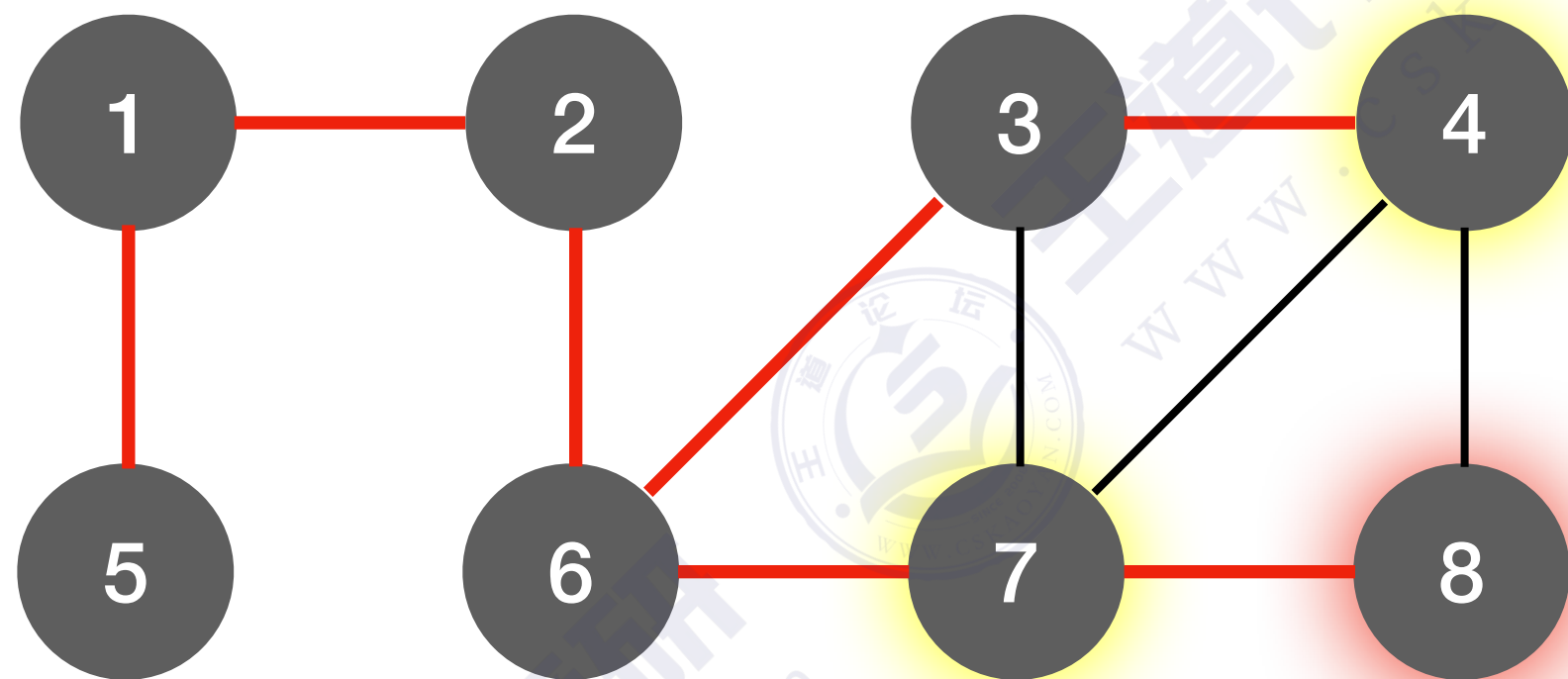


	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	3
path[]	2	-1	6	3	1	2	6	7

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞; //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){
        DeQueue(Q,u);
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w)){
            if(!visited[w]){
                d[w]=d[u]+1;
                path[w]=u;
                visited[w]=TRUE;
                EnQueue(Q,w);
            }
        }
    }
}
```

	1	2	3	4	5	6	7	8
visited	true	true	true	true	true	true	true	true

代码实现



	1	2	3	4	5	6	7	8
d[]	1	0	2	3	2	1	2	3
path[]	2	-1	6	3	1	2	6	7

//求顶点 u 到其他顶点的最短路径

```
void BFS_MIN_Distance(Graph G, int u){
```

// $d[i]$ 表示从 u 到 i 结点的最短路径

```
for(i=0;i<G.vexnum;++i){
```

```
d[i]=∞; //初始化路径长度
```

```
path[i]=-1; //最短路径从哪个顶点过来
```

}

```
d[u]=0;
```

```
visited[u]=TRUE;
```

```
EnQueue(Q,u);
```

```
while(!isEmpty(Q)){
```

DeQueue(Q,u);

```
for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
```

```
if (!visited[w]) {
```

```
d[w]=d[u]+1;
```

```
path[w]=u;
```

```
visited[w]=TRUE;
```

```
EnQueue(Q,w);
```

```
}//if
```

```
}//while
```

}

//BFS算法主过程

//队头元素 u 出队

//w为u的尚未访问的邻接顶点

```
//路径长度加1
```

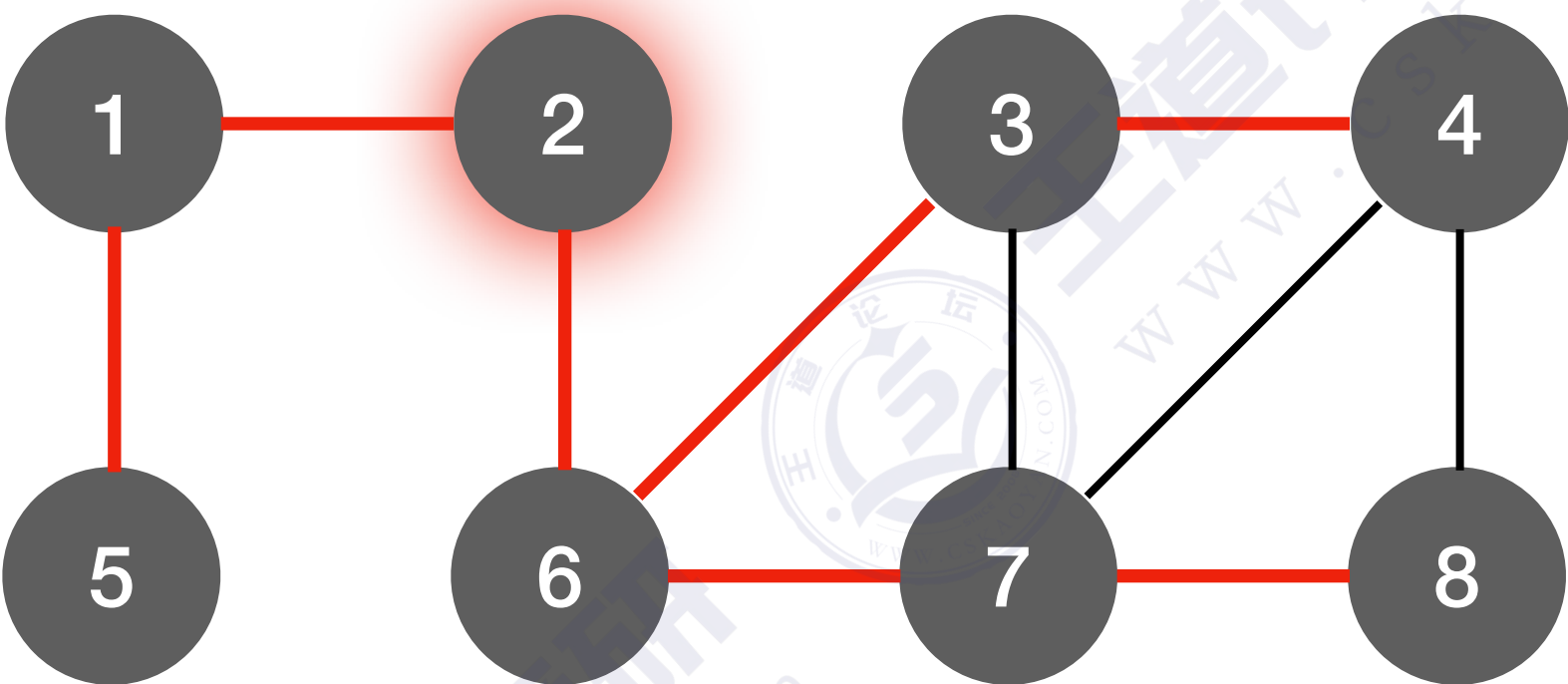
```
//最短路径应从u到w
```

```
//设已访问标记
```

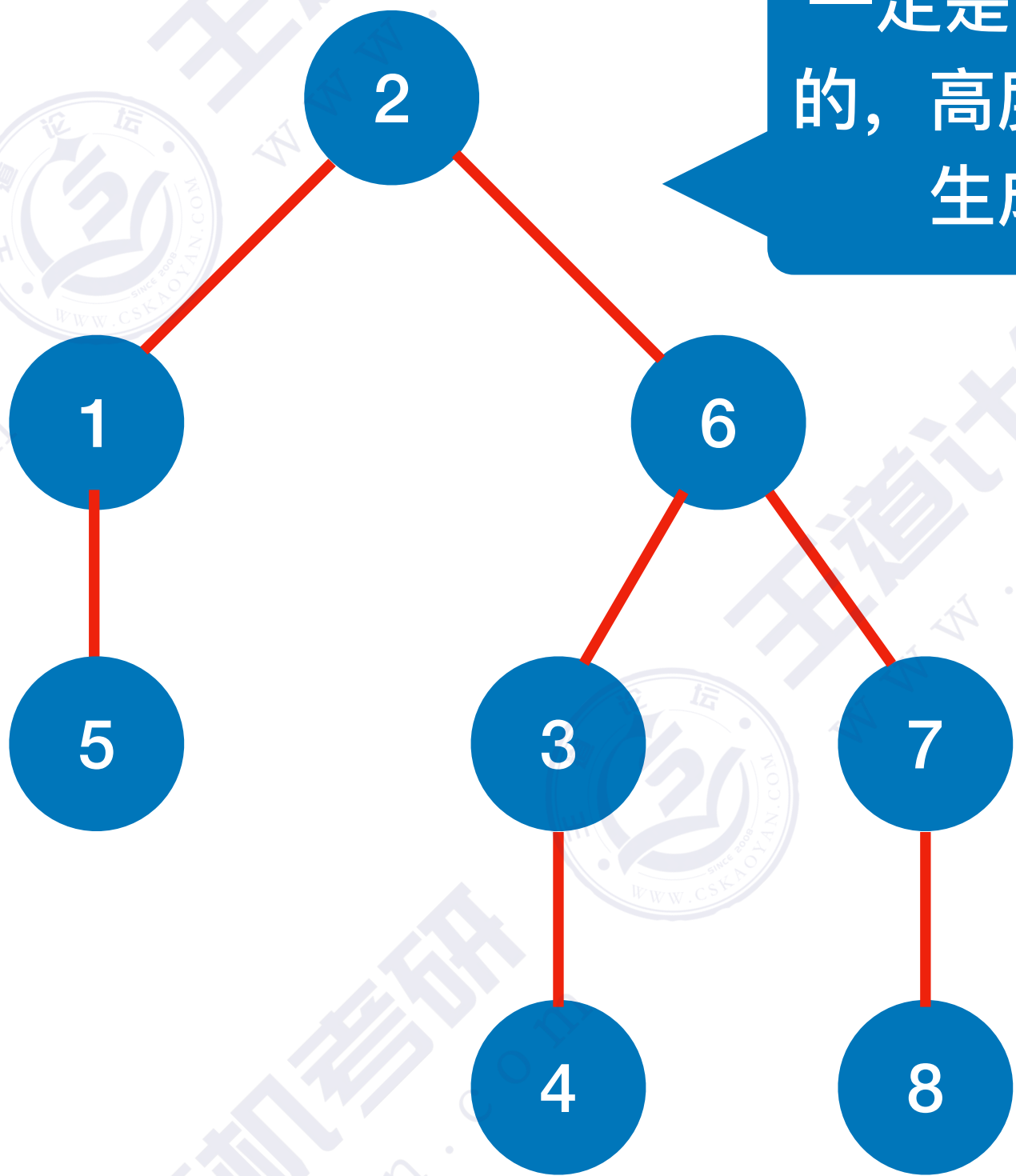
```
//顶点w入队
```

[illegible]

知识点回顾与重要考点



广度优先生成树



一定是以2为根的，高度最小的生成树

就是对BFS的小修改，在visit一个顶点时，修改其最短路径长度 $d[]$ 并在 $path[]$ 记录前驱结点

	1	2	3	4	5	6	7	8
$d[]$	1	0	2	3	2	1	2	3
$path[]$	2	-1	6	3	1	2	6	7

2到8的最短路径长度 = $d[8] = 3$

通过 $path$ 数组可知，2到8的最短路径为：8 $\leftarrow$ 7 $\leftarrow$ 6 $\leftarrow$ 2

本节内容

最短路径

Dijkstra算法

迪杰斯特拉



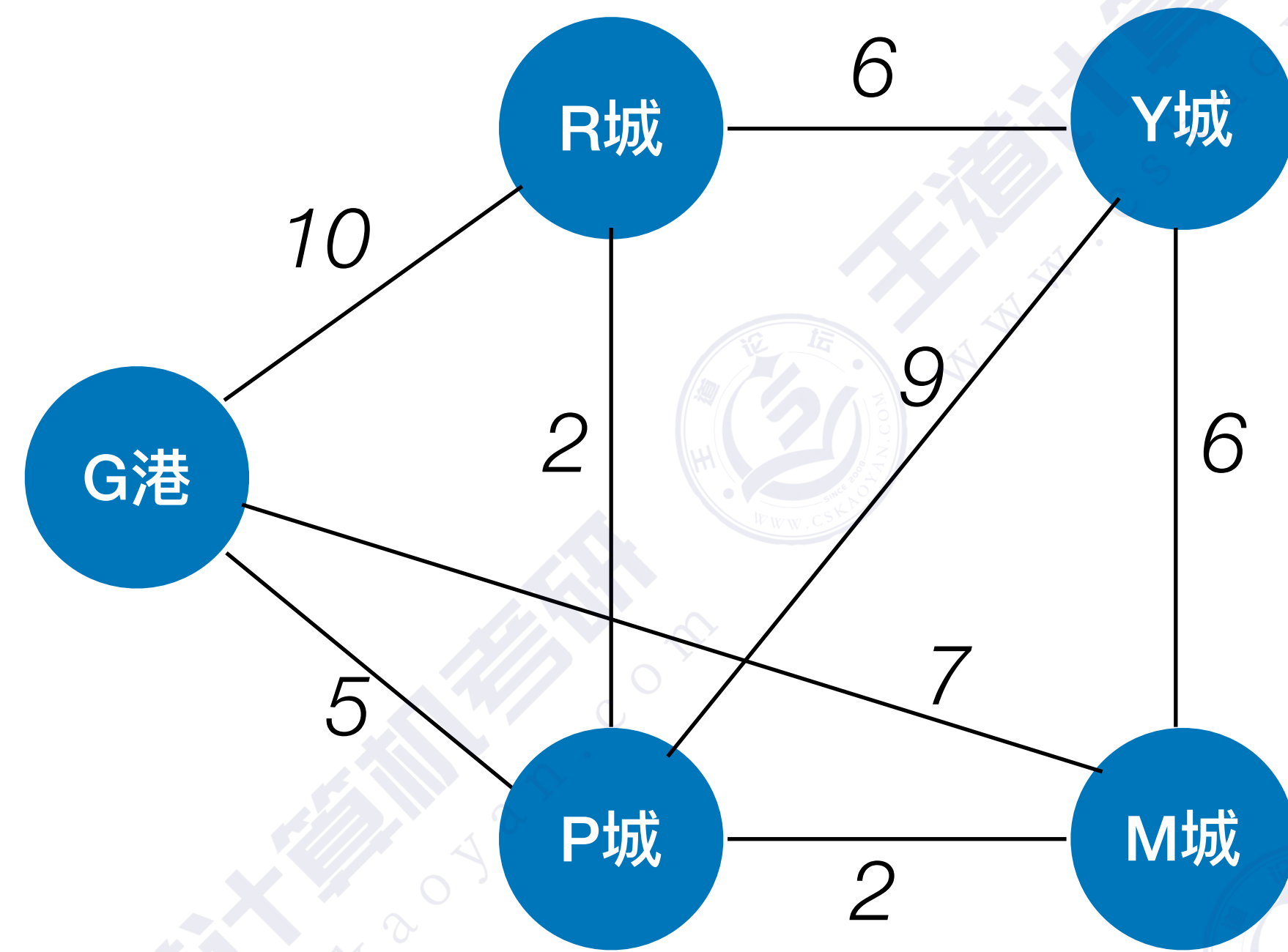
艾兹格·W·迪杰斯特拉
Edsger Wybe Dijkstra
(1930~2002)



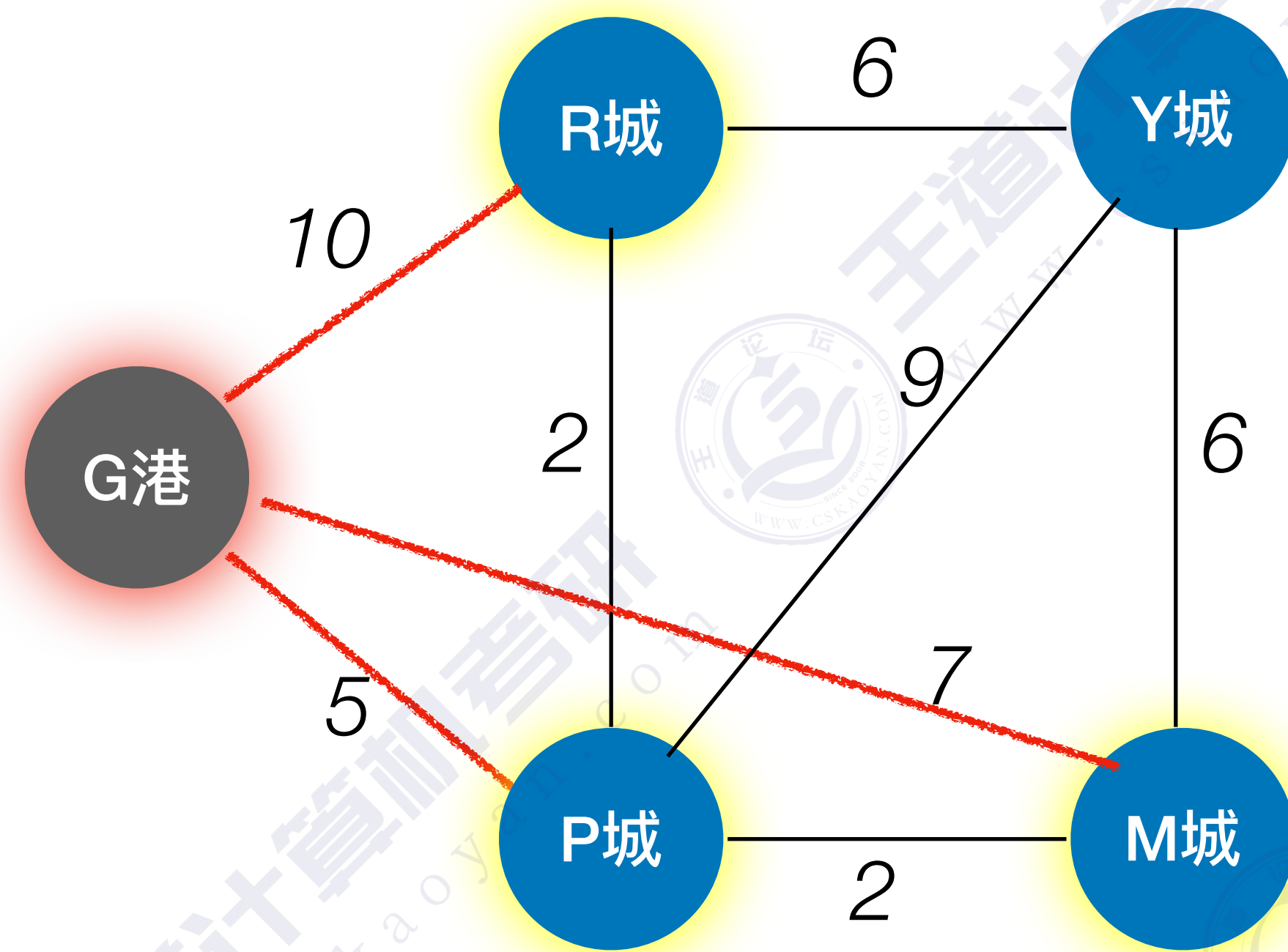
1972年图灵奖得主

- 提出“goto 有害理论”——操作系统，虚拟存储技术
- 信号量机制PV原语——操作系统，进程同步
- 银行家算法——操作系统，死锁
- 解决哲学家进餐问题——操作系统，死锁
- Dijkstra最短路径算法——数据结构大题、小题

BFS算法的局限性



BFS算法的局限性



带权路径长度——当图是带权图时，一条路径上所有边的权值之和，称为该路径的带权路径长度

BFS算法求单源最短路径只适用于无权图，或所有边的权值都相同的图

最短路径问题

单源最短路径

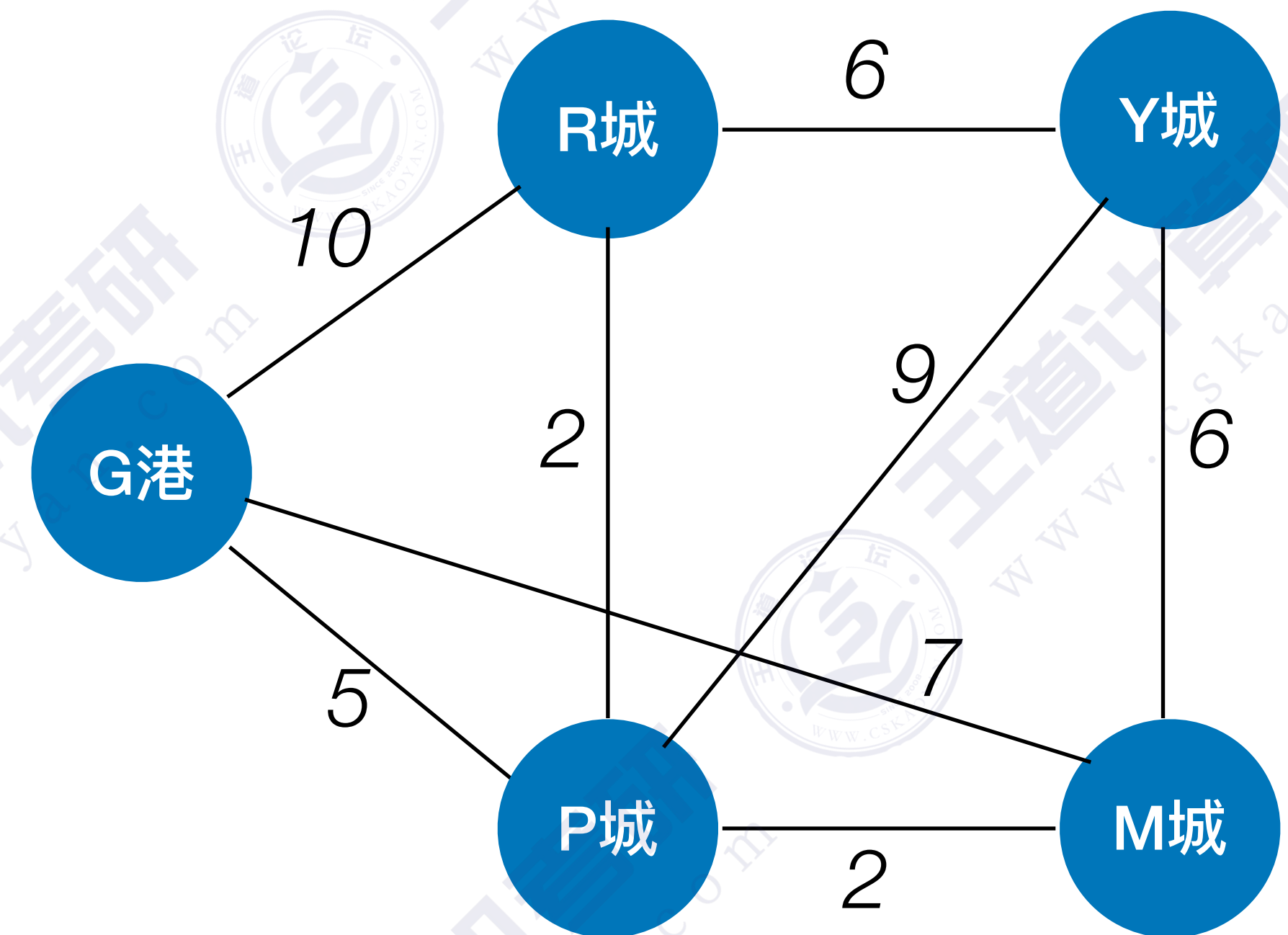
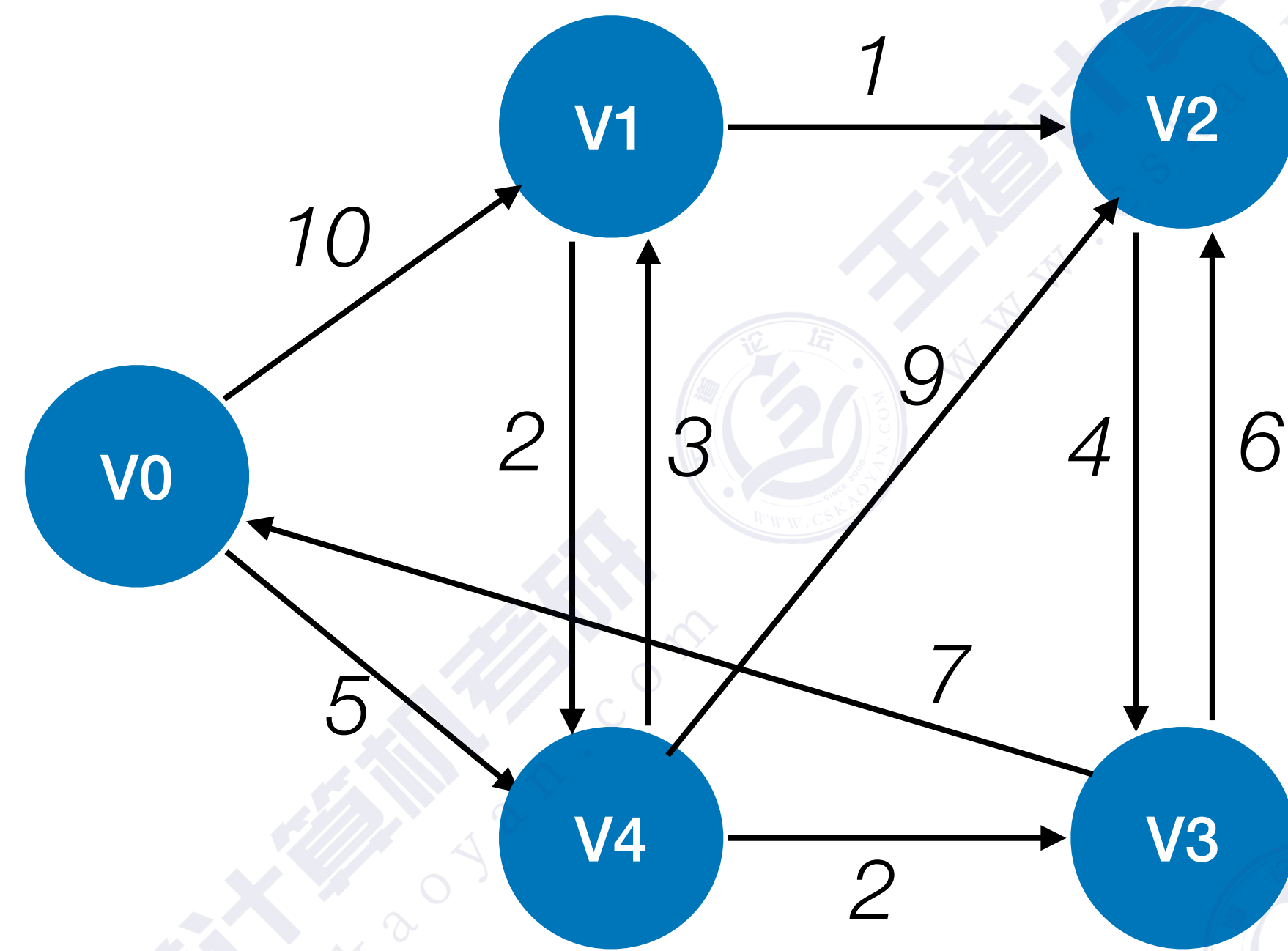
BFS 算法（无权图）

Dijkstra 算法（带权图、无权图）

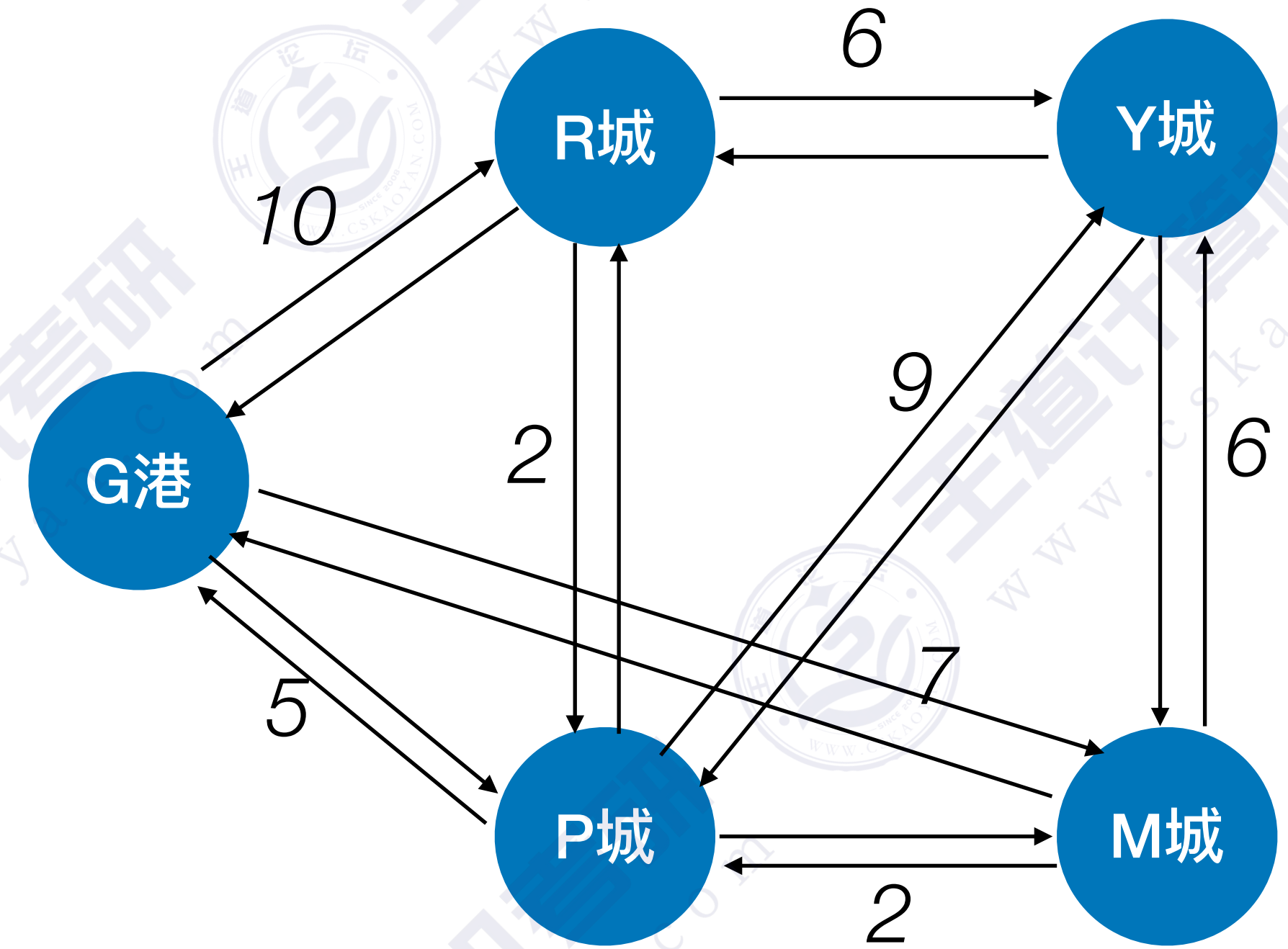
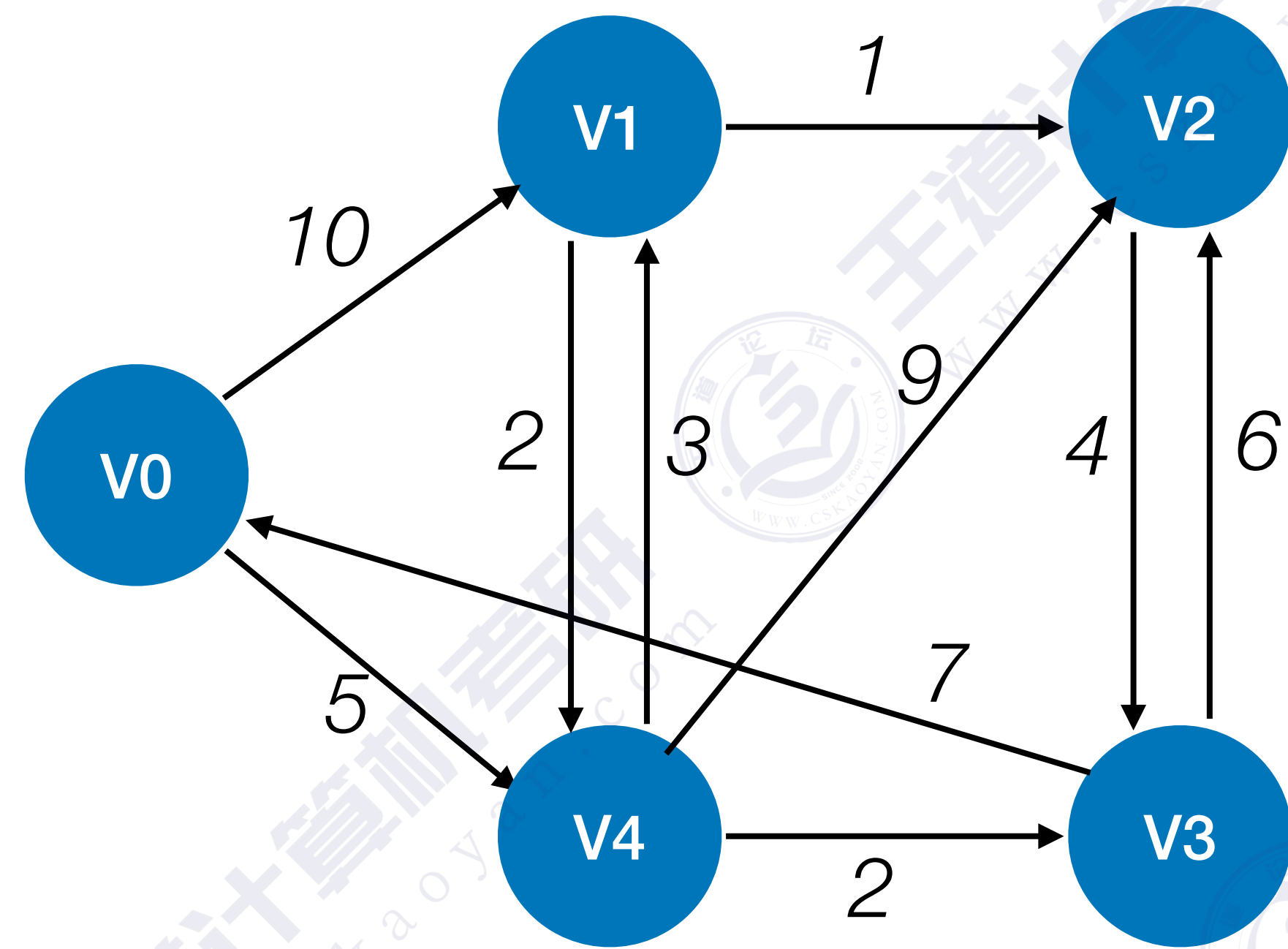
各顶点间的最短路径

Floyd 算法（带权图、无权图）

Dijkstra算法



Dijkstra算法



Dijkstra算法

初始：从V₀开始，初始化三个数组信息如下👉

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✗

0	10	∞	∞	5
---	----	---	---	---

-1	0	-1	-1	0
----	---	----	----	---

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

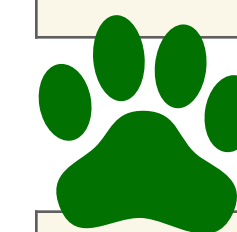
路径上
的前驱

$final[5]$

$dist[5]$

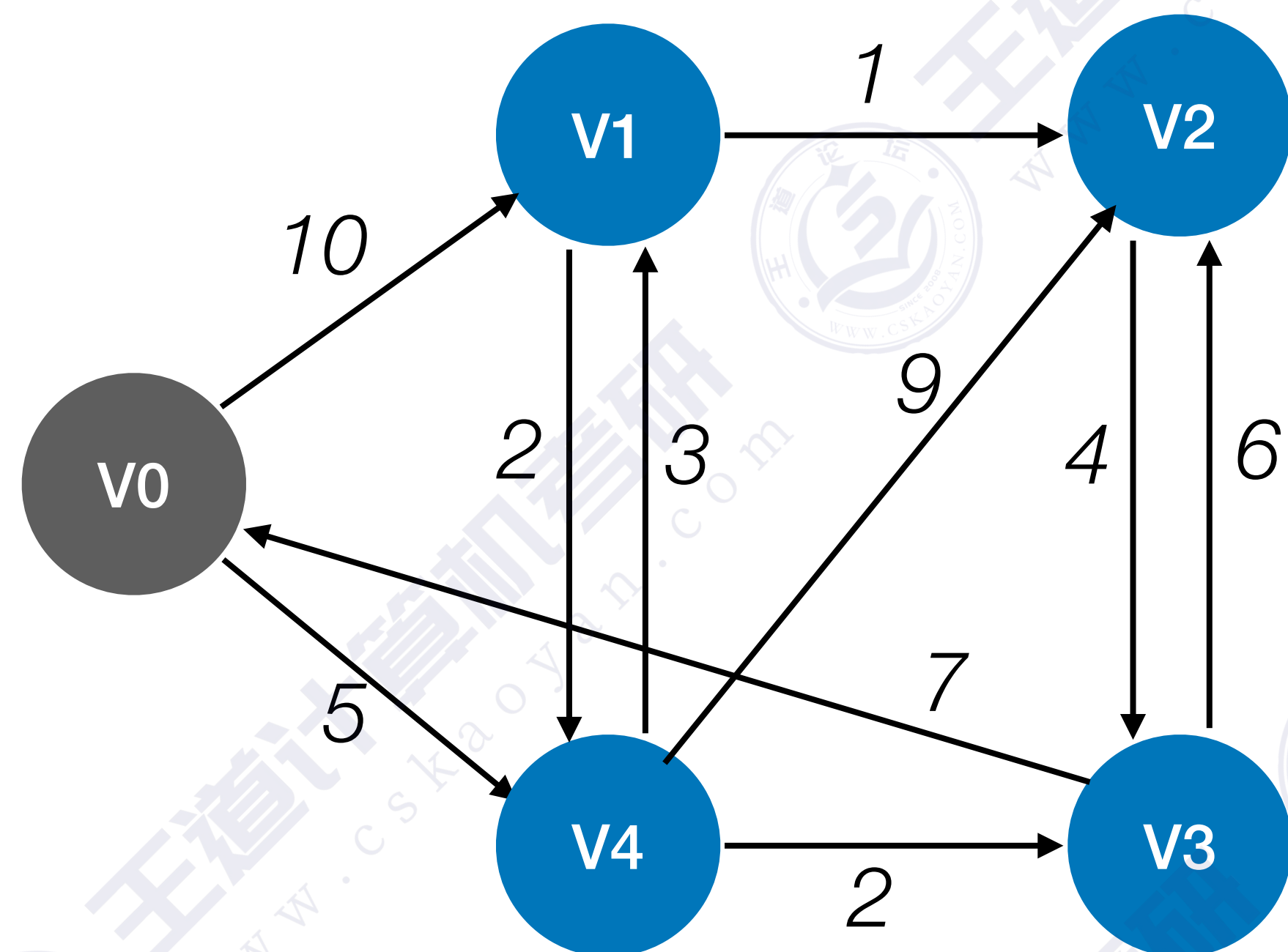
$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✗



0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---



Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✗

0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---

检查所有邻接自 V_i 的顶点，若其 final 值为false，则更新 dist 和 path 信息

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---

检查所有邻接自 V_i 的顶点，若其 final 值为false，则更新 dist 和 path 信息

Dijkstra算法

第1轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

0	8	14	7	5
---	---	----	---	---

-1	4	4	4	0
----	---	---	---	---

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓

0	8	14	7	5
---	---	----	---	---

-1	4	4	4	0
----	---	---	---	---

Dijkstra算法

第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

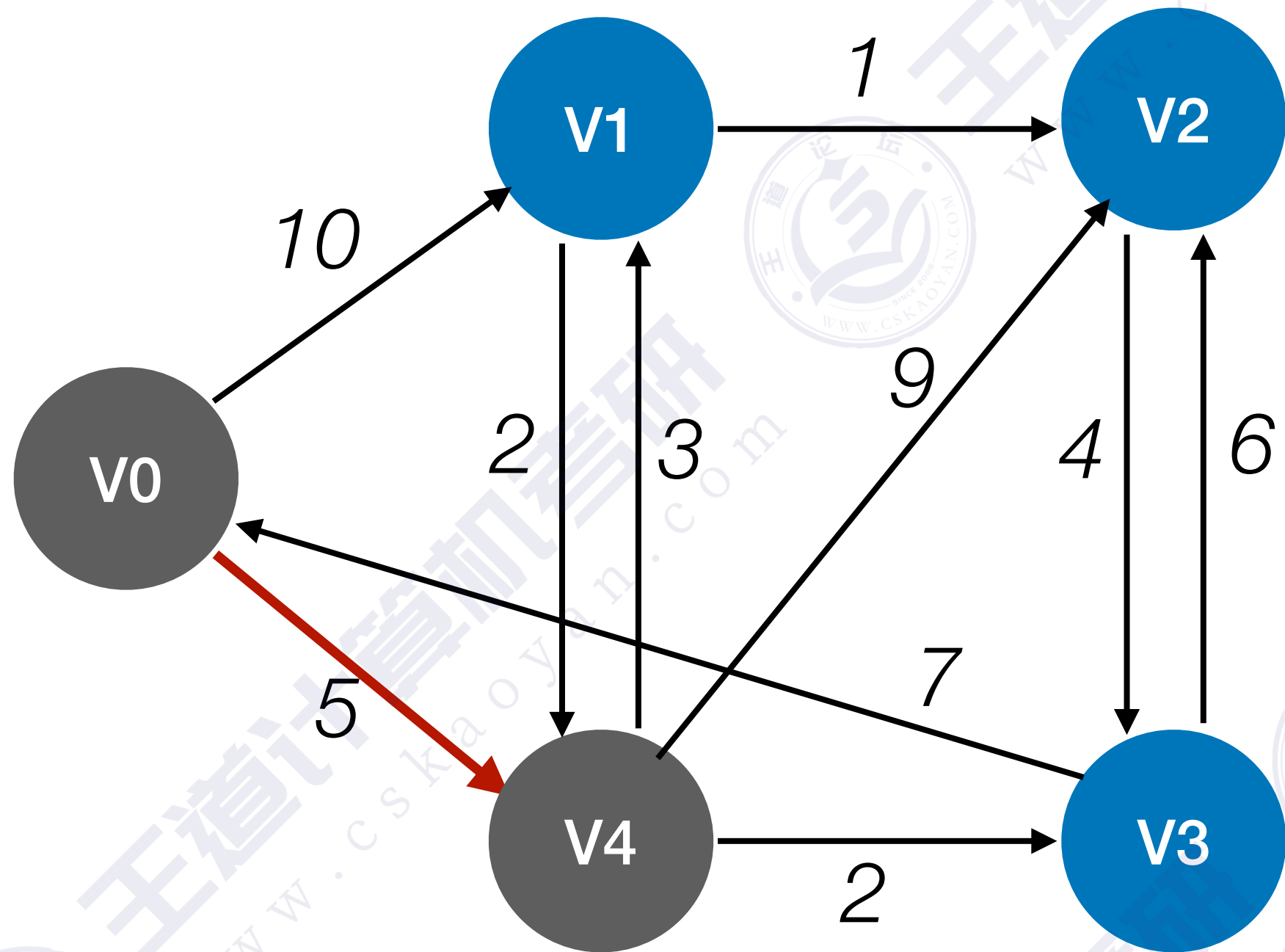
$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✓



0	8	14	7	5
---	---	----	---	---

-1	4	4	4	0
----	---	---	---	---



Dijkstra算法

第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

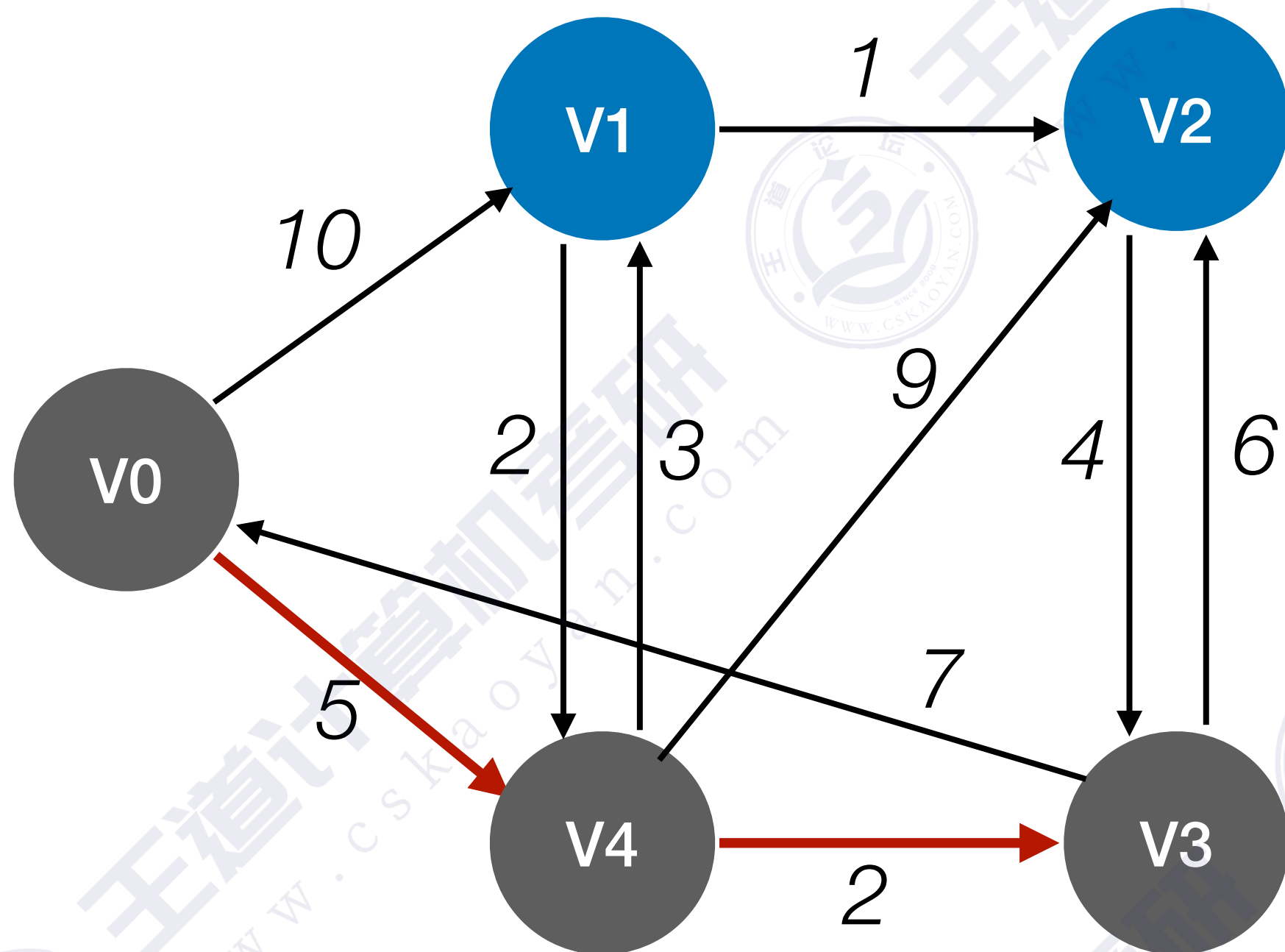
$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓



0	8	14	7	5
---	---	----	---	---

-1	4	4	4	0
----	---	---	---	---



Dijkstra算法

第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

最短路径长度

路径上的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓

0	8	14	7	5
---	---	----	---	---

-1	4	4	4	0
----	---	---	---	---

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第2轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓

0	8	13	7	5
---	---	----	---	---

-1	4	3	4	0
----	---	---	---	---

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓

0	8	13	7	5
---	---	----	---	---

-1	4	3	4	0
----	---	---	---	---

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✓	✓
0	8	13	7	5
-1	4	3	4	0

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否已找到最短路径

最短路径长度

路径上的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	13	7	5
-1	4	3	4	0

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	13	7	5
-1	4	3	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	13	7	5
-1	4	3	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第3轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓
0	8	9	7	5
-1	4	1	4	0

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，
则更新 $dist$ 和 $path$ 信息

Dijkstra算法

第4轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

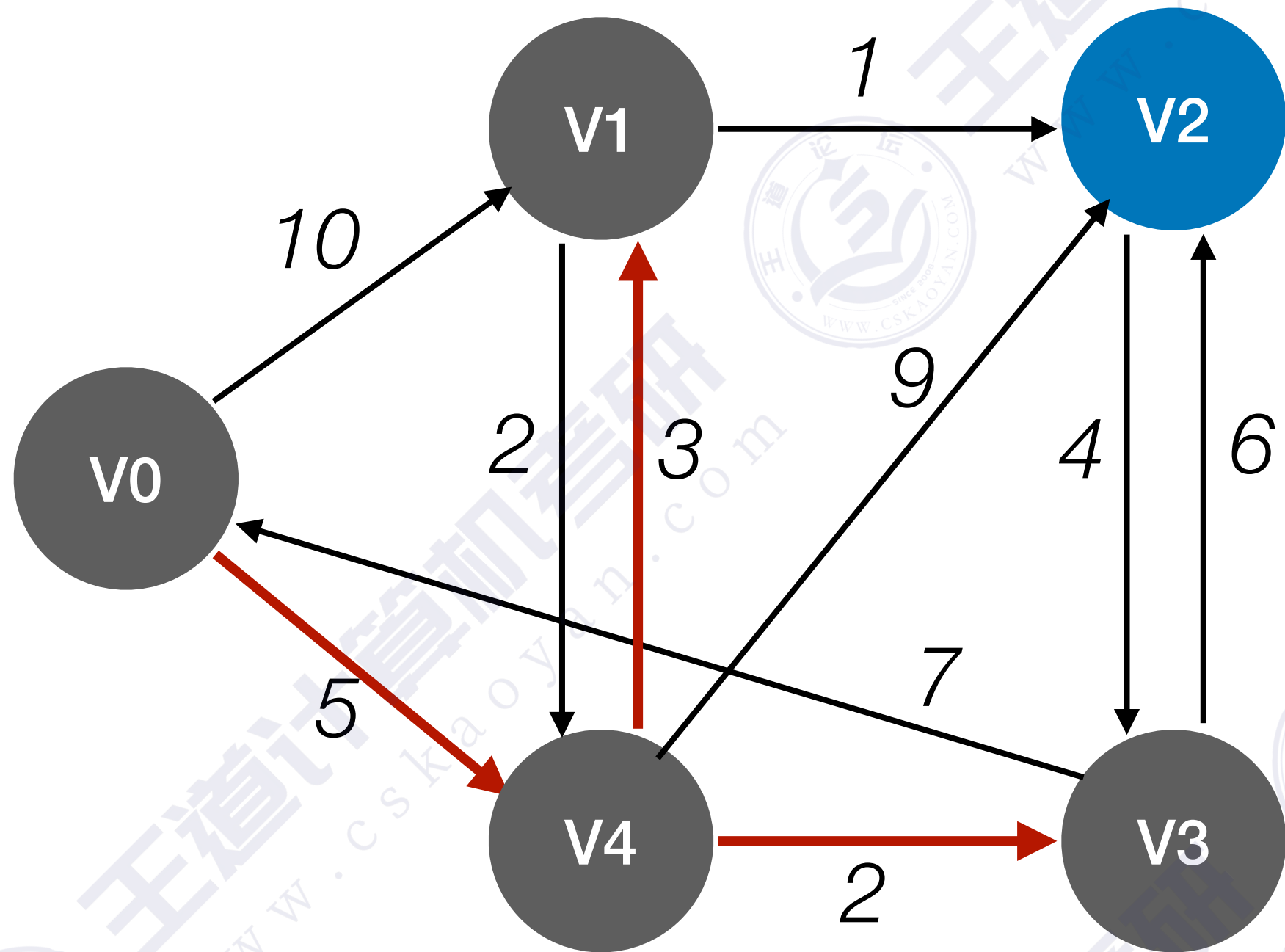
$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓

0	8	9	7	5
---	---	---	---	---

-1	4	1	4	0
----	---	---	---	---



Dijkstra算法

第4轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✗	✓	✓

0	8	9	7	5
---	---	---	---	---

-1	4	1	4	0
----	---	---	---	---

Dijkstra算法

第4轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✓	✓	✓	✓

0	8	9	7	5
---	---	---	---	---

-1	4	1	4	0
----	---	---	---	---

Dijkstra算法

第4轮：循环遍历所有结点，找到还没确定最短路径，且dist 最小的顶点 V_i ，令 $final[i]=ture$ 。

标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

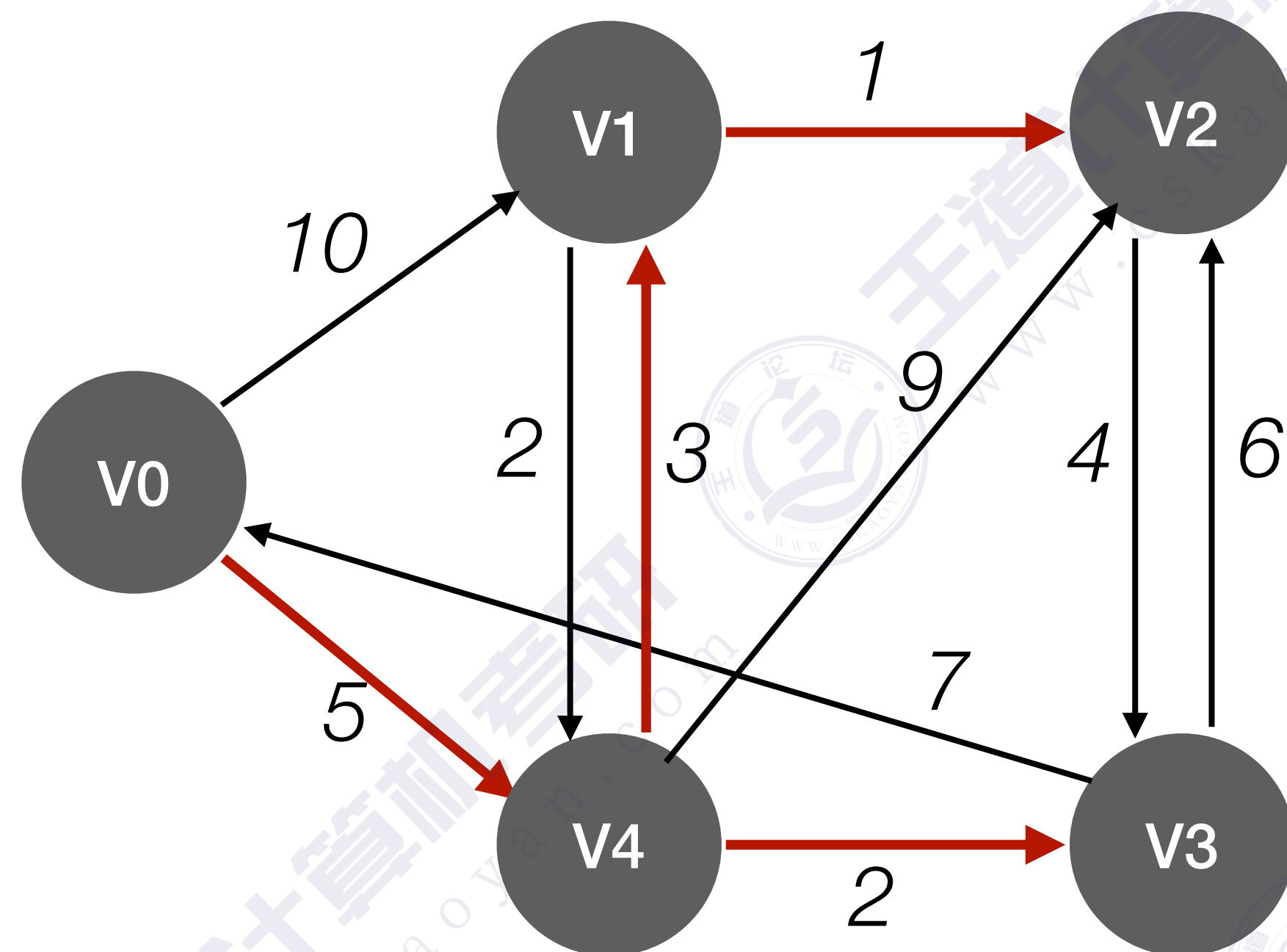
V0	V1	V2	V3	V4
✓	✓	✓	✓	✓

0	8	9	7	5
---	---	---	---	---

-1	4	1	4	0
----	---	---	---	---

检查所有邻接自 V_i 的顶点，若其 $final$ 值为false，则更新 $dist$ 和 $path$ 信息

如何使用数组信息？



标记各顶点是否
已找到最短路径

$final[5]$

V0	V1	V2	V3	V4
✓	✓	✓	✓	✓

最短路
径长度

$dist[5]$

0	8	9	7	5
---	---	---	---	---

路径上
的前驱

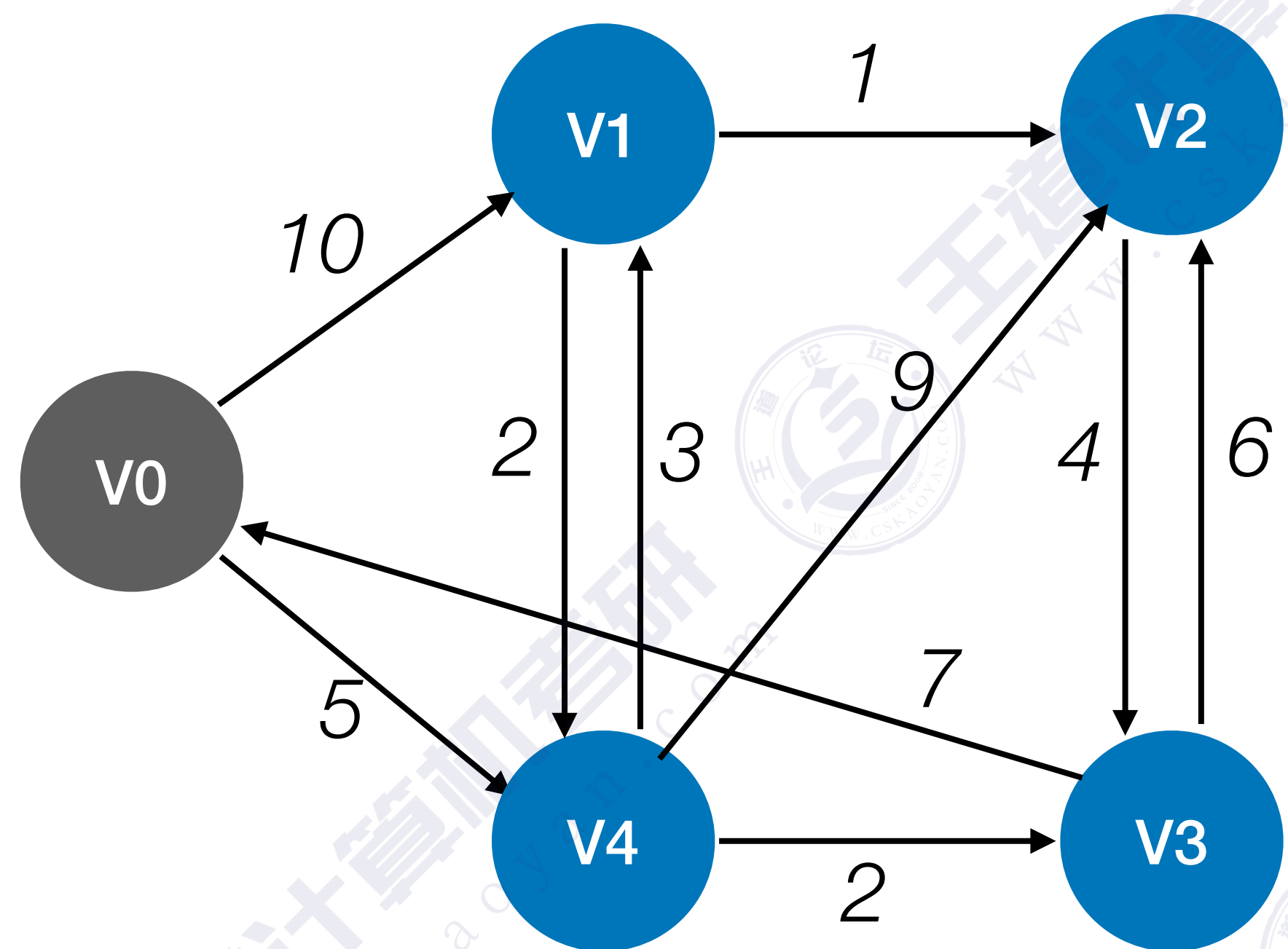
$path[5]$

-1	4	1	4	0
----	---	---	---	---

V0到V2 的最短(带权)路径长度为： $dist[2] = 9$

通过 $path[]$ 可知，V0到V2 的最短(带权)路径： $V2 \leftarrow V1 \leftarrow V4 \leftarrow V0$

Dijkstra算法的时间复杂度



标记各顶点是否
已找到最短路径

最短路
径长度

路径上
的前驱

$final[5]$

$dist[5]$

$path[5]$

V0	V1	V2	V3	V4
✓	✗	✗	✗	✗

0	10	∞	∞	5
---	----	----------	----------	---

-1	0	-1	-1	0
----	---	----	----	---

初始：若从 V_0 开始，令 $final[0]=ture$; $dist[0]=0$; $path[0]=-1$ 。

其余顶点 $final[k]=false$; $dist[k]=arcs[0][k]$; $path[k]= (arcs[0][k]==\infty) ? -1 : 0$

时间复杂度：
 $O(n^2)$ 即 $O(|V|^2)$

n-1轮处理：循环遍历所有顶点，找到还没确定最短路径，且 $dist$ 最小的顶点 V_i ，令 $final[i]=ture$ 。并检查所有邻接自 V_i 的顶点，对于邻接自 V_i 的顶点 V_j ，若 $final[j]==false$ 且 $dist[i]+arcs[i][j] < dist[j]$ ，则令 $dist[j]=dist[i]+arcs[i][j]$; $path[j]=i$ 。（注： $arcs[i][j]$ 表示 V_i 到 V_j 的弧的权值）

对比：Prim 算法的实现思想

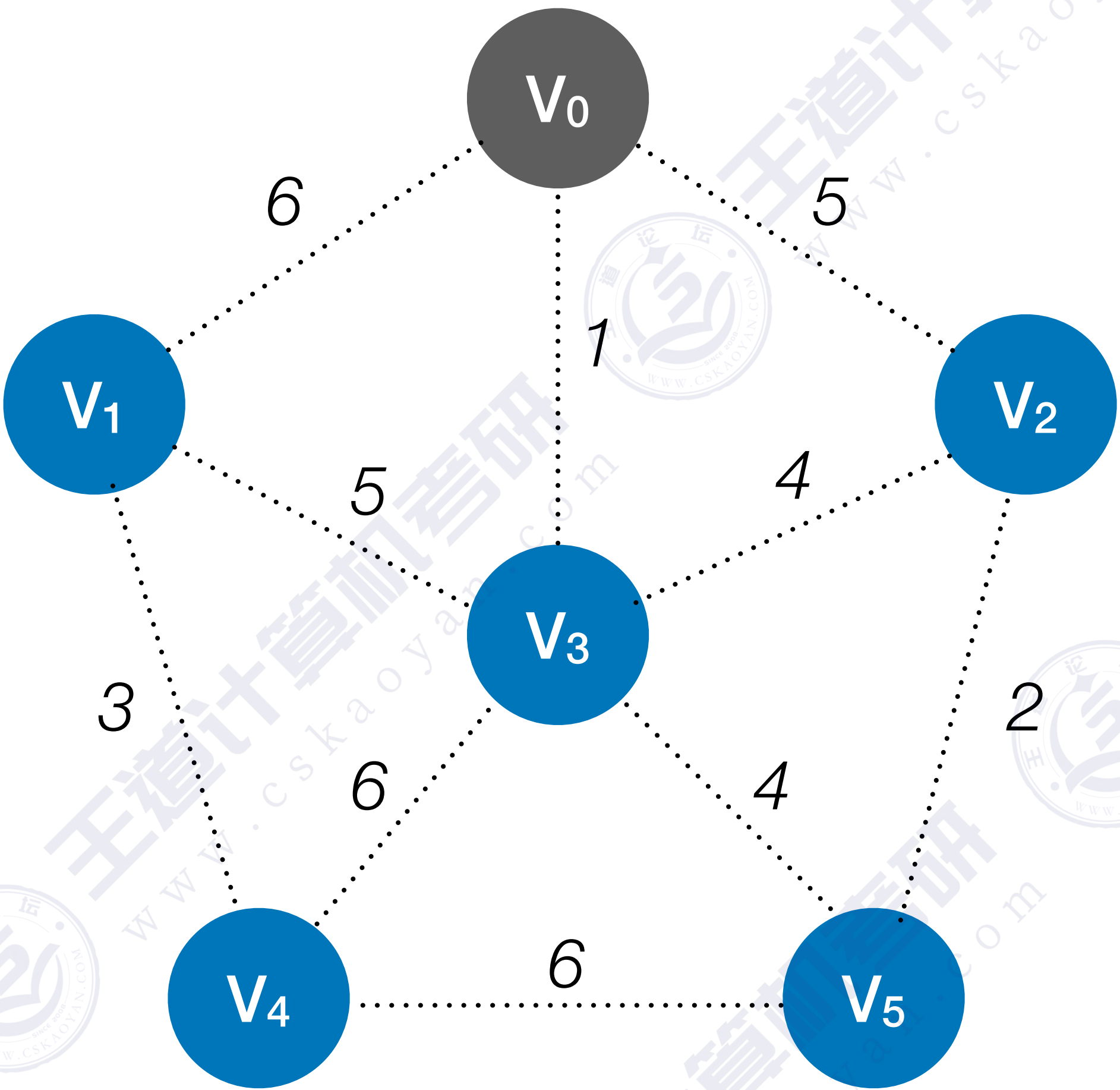
总时间复杂度
 $O(n^2)$ ，即 $O(|V|^2)$

从 V_0 开始，总共需要 $n-1$ 轮处理

每一轮处理：循环遍历所有个结点，找到lowCost最低的，且还没加入树的顶点。

再次循环遍历，更新还没加入的各个顶点的lowCost值

每一轮时间复杂度 $O(2n)$



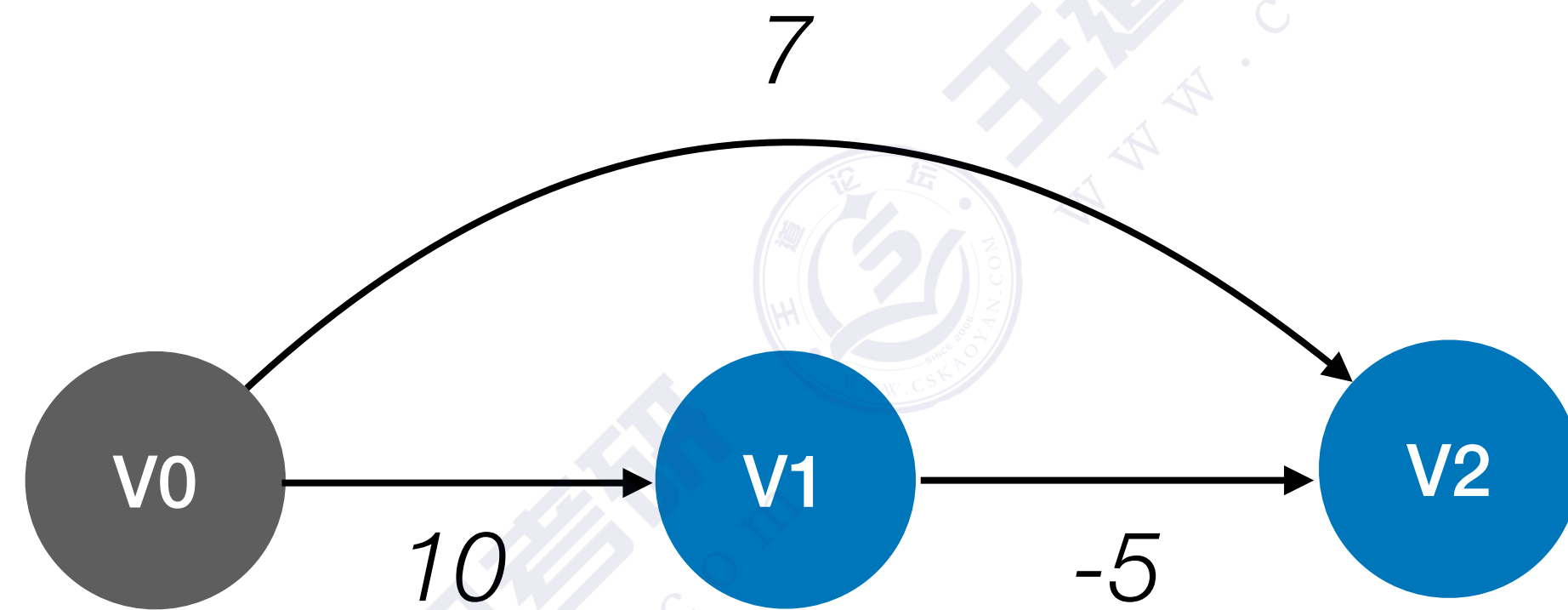
$isJoin[6]$

V0	V1	V2	V3	V4	V5
✓	✗	✗	✗	✗	✗

$lowCost[6]$

0	6	5	1	∞	∞
---	---	---	---	----------	----------

用于负权值带权图



标记各顶点是否
已找到最短路径

$final[3]$

V0	V1	V2
✓	✗	✗

最短路
径长度

$dist[3]$

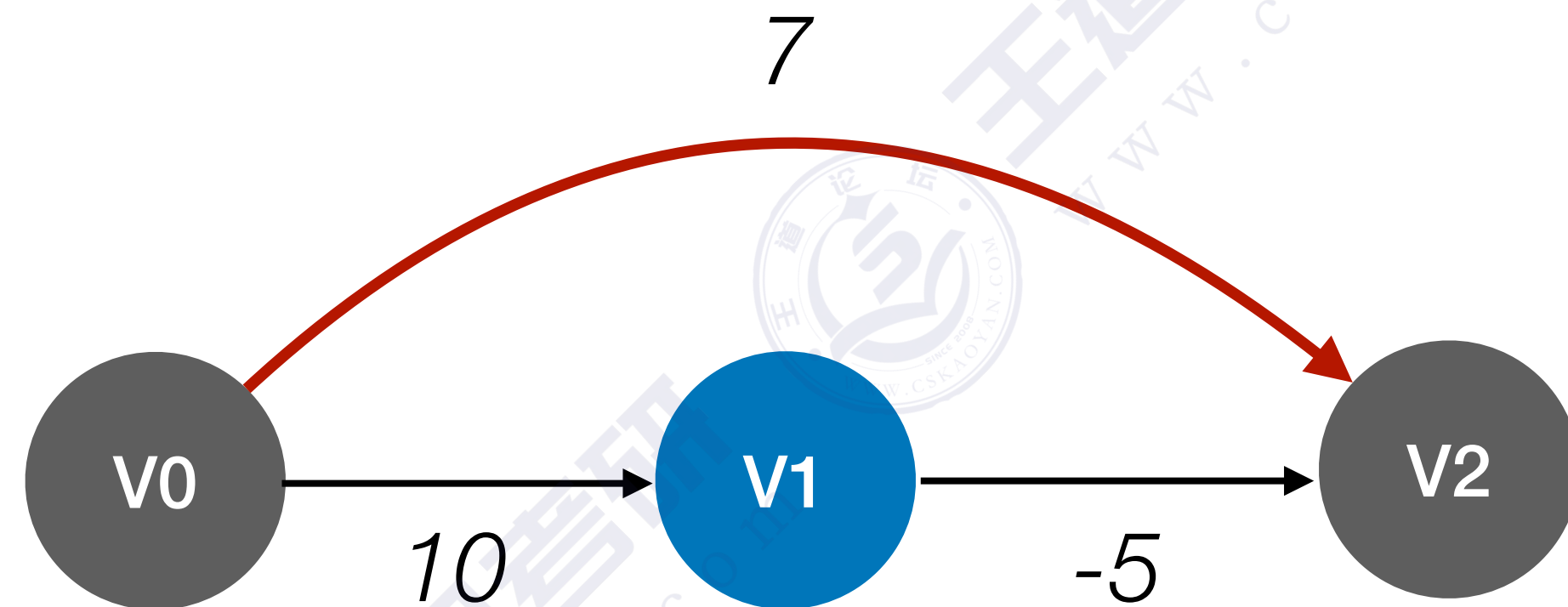
0	10	7
---	----	---

路径上
的前驱

$path[3]$

-1	0	0
----	---	---

用于负权值带权图



标记各顶点是否
已找到最短路径

$final[3]$

V0	V1	V2
✓	✗	✓

最短路
径长度

$dist[3]$

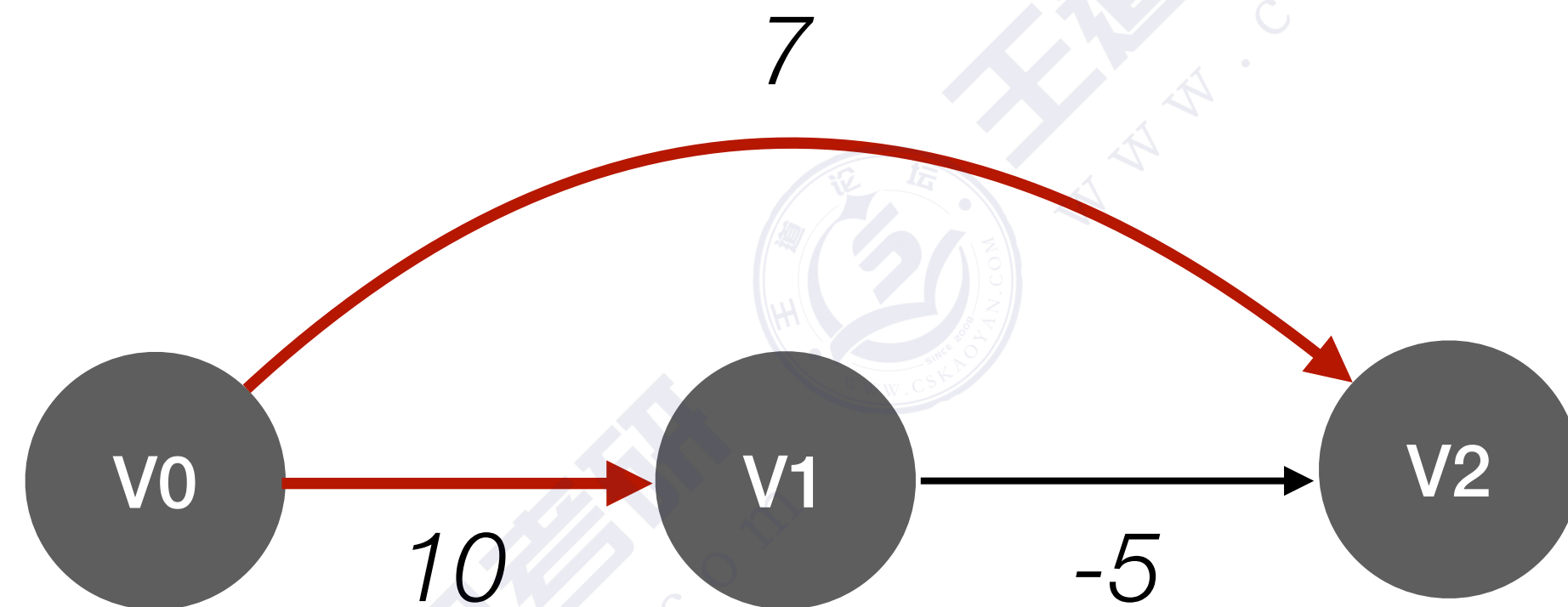
0	10	7
---	----	---

路径上
的前驱

$path[3]$

-1	0	0
----	---	---

用于负权值带权图



标记各顶点是否
已找到最短路径

$final[3]$

V0	V1	V2
✓	✓	✓

最短路
径长度

$dist[3]$

0	10	7
---	----	---

路径上
的前驱

$path[3]$

-1	0	0
----	---	---

事实上V0到V2 的最短带权路径长度为 5

结论: Dijkstra 算法不适用于有负权值的带权图

本节内容

最短路径

Floyd算法

Robert W. Floyd



罗伯特·弗洛伊德
(1936－2001) Robert W. Floyd


1978年图灵奖得主

- Floyd算法 (Floyd-Warshall算法)
- 堆排序算法

最短路径问题

单源最短路径

BFS 算法 (无权图)

Dijkstra 算法 (带权图、无权图)

各顶点间的最短路径

Floyd 算法 (带权图、无权图)

Floyd算法



Floyd算法：求出每一对顶点之间的最短路径

使用动态规划思想，将问题的求解分为多个阶段

对于n个顶点的图G，求任意一对顶点 $V_i \rightarrow V_j$ 之间的最短路径可分为如下几个阶段：

#初始：不允许在其他顶点中转，最短路径是？

#0：若允许在 V_0 中转，最短路径是？

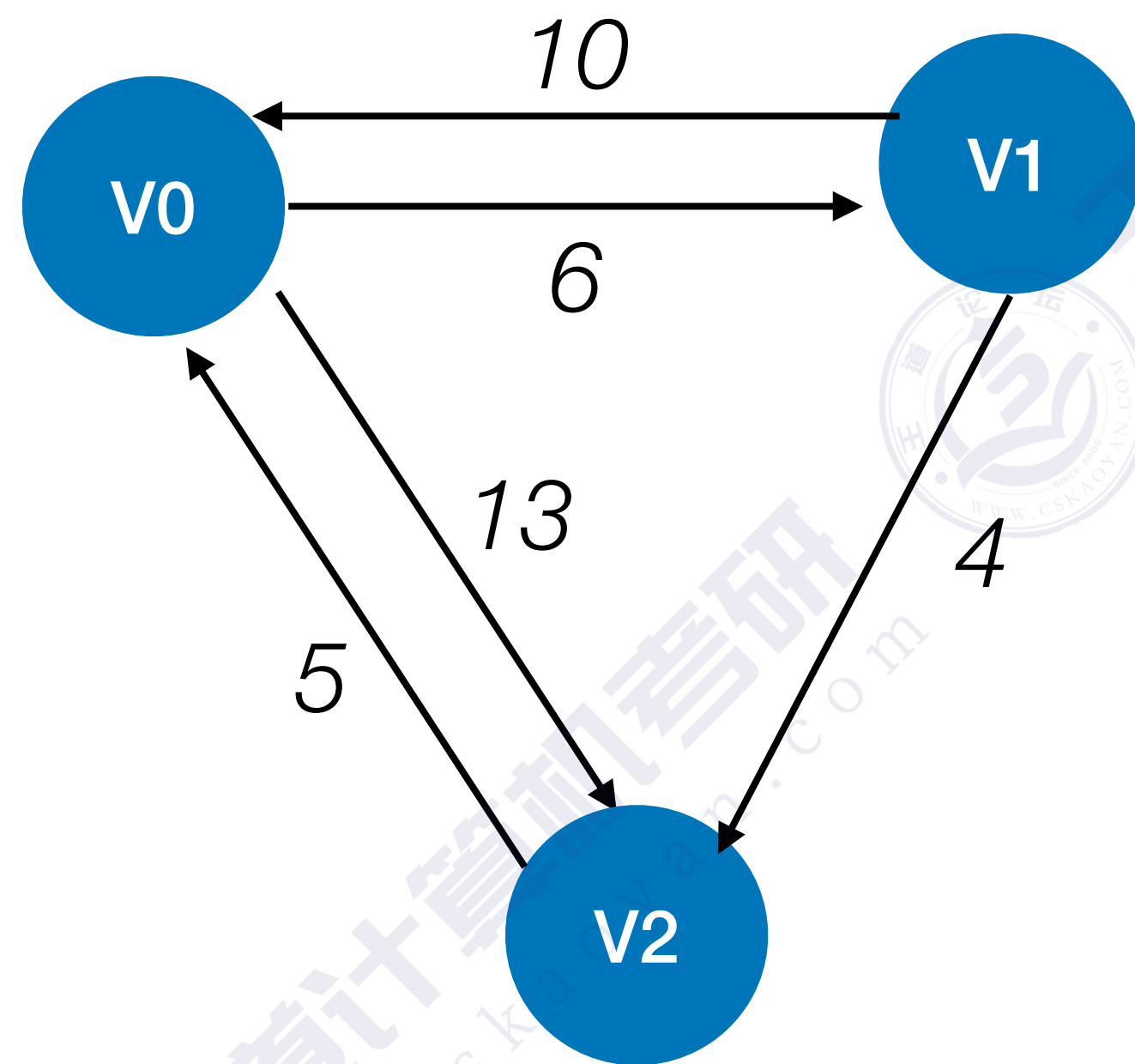
#1：若允许在 V_0 、 V_1 中转，最短路径是？

#2：若允许在 V_0 、 V_1 、 V_2 中转，最短路径是？

...

#n-1：若允许在 V_0 、 V_1 、 V_2 V_{n-1} 中转，最短路径是？

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(-1)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	∞	0

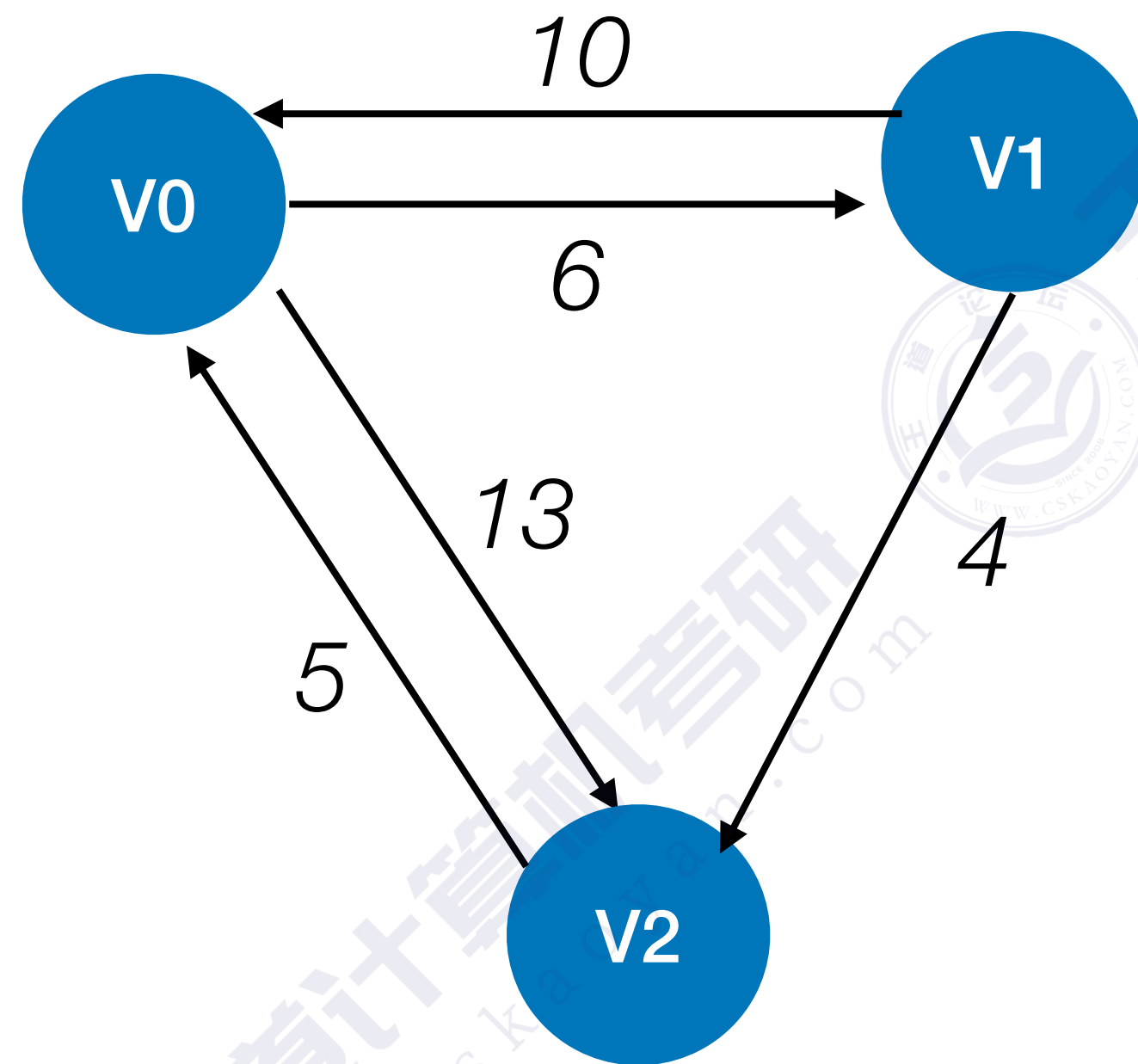
两个顶点之间的中转点

$path^{(-1)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1

#初始：不允许在其他顶点中转，最短路径是？

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(-1)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	∞	0

两个顶点之间的中转点

$path^{(-1)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1

#0: 若允许在 **V₀** 中转，最短路径是? —— 求 $A^{(0)}$ 和 $path^{(0)}$

若
则

$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$

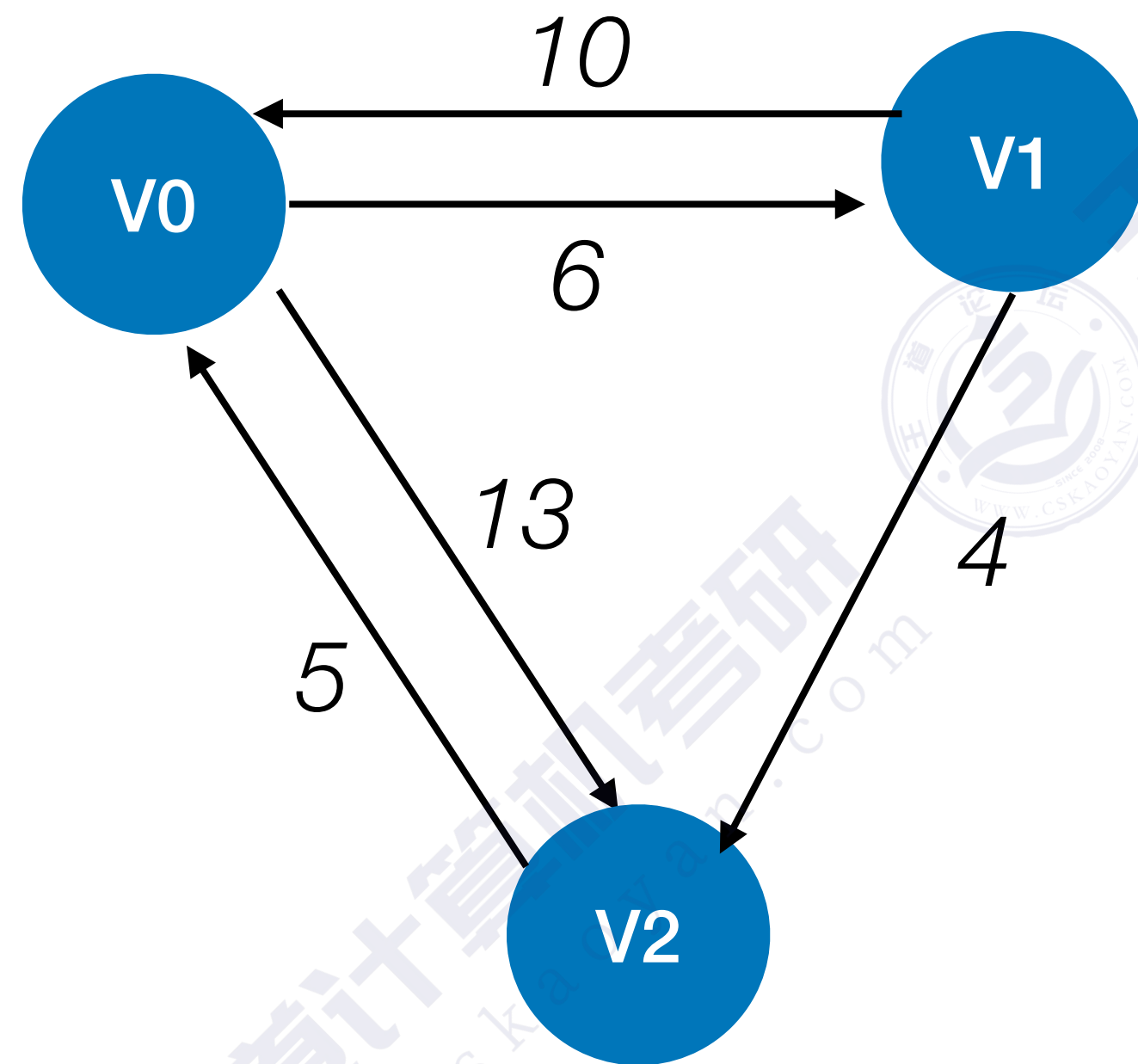
否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

$A^{(-1)}[2][1] > A^{(-1)}[2][0] + A^{(-1)}[0][1] = 11$

$A^{(0)}[2][1] = 11$

$path^{(0)}[2][1] = 0;$

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(-1)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	∞	0

两个顶点之间的中转点

$path^{(-1)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1

#0: 若允许在 **V0** 中转，最短路径是？——求 $A^{(0)}$ 和 $path^{(0)}$

若
则

$$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$$

$$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$$

$$path^{(k)}[i][j] = k$$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

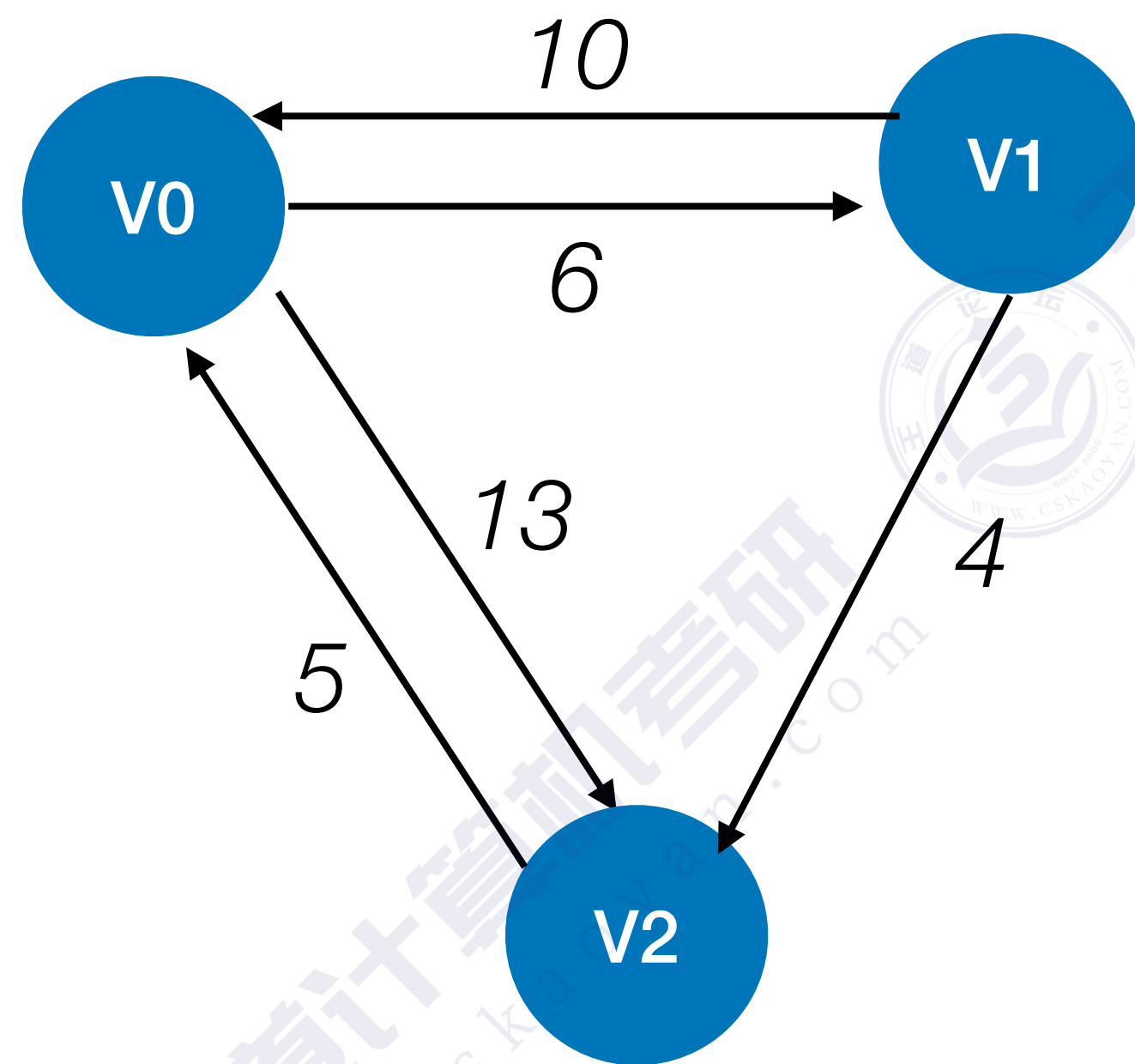
$A^{(0)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	11	0

$path^{(0)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	0	-1

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(0)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	11	0

两个顶点之间的中转点

$path^{(0)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	0	-1

#1: 若允许在 V_0 、 V_1 中转，最短路径是？——求 $A^{(1)}$ 和 $path^{(1)}$

若
则

$$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$$

$$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$$

$$path^{(k)}[i][j] = k$$

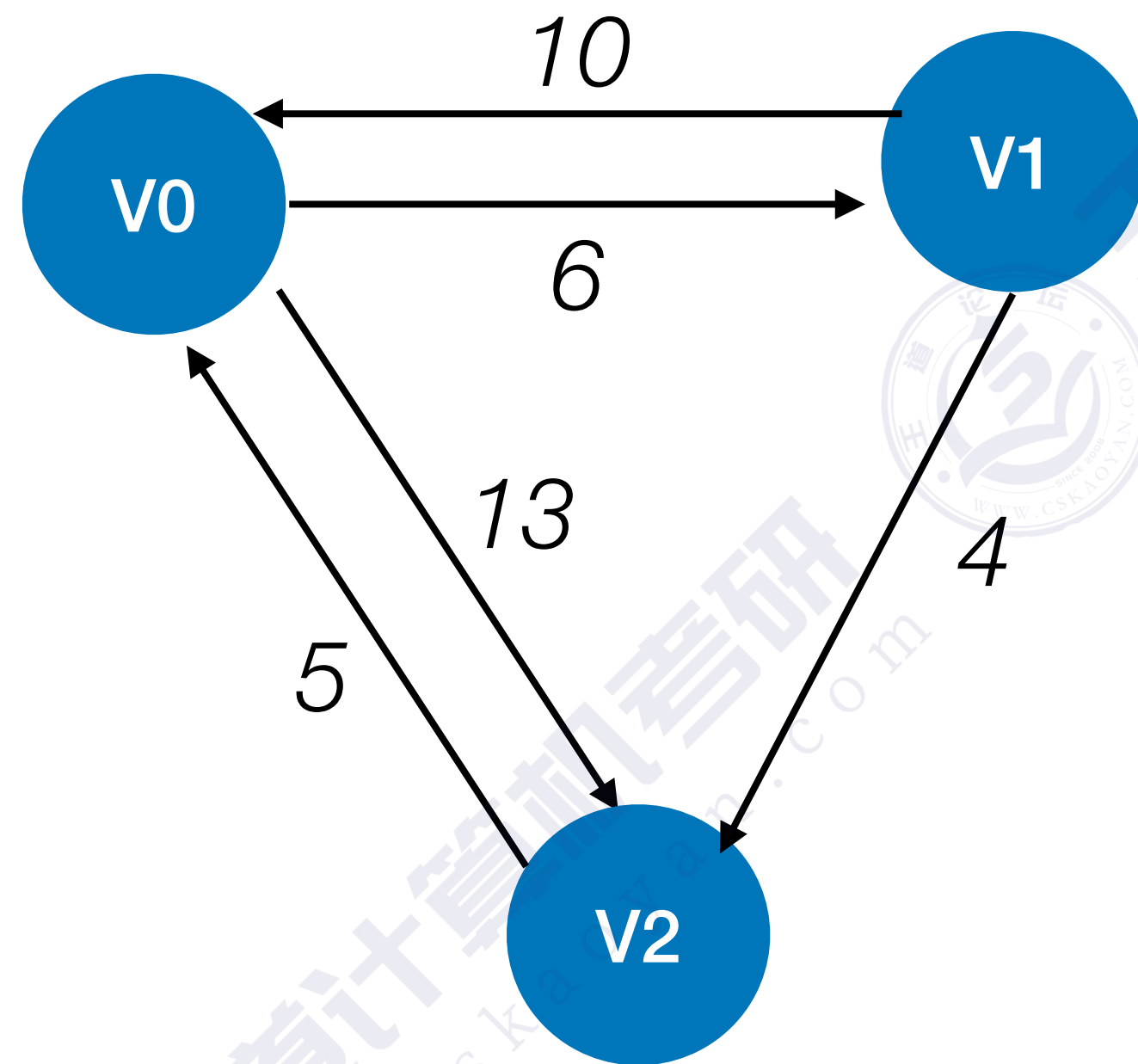
否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

$$A^{(0)}[0][2] > A^{(0)}[0][1] + A^{(0)}[1][2] = 10$$

$$A^{(1)}[0][2] = 10$$

$$path^{(1)}[0][2] = 1;$$

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(0)} =$

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	11	0

两个顶点之间的中转点

$path^{(0)} =$

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	0	-1

#1: 若允许在 V_0 、 V_1 中转，最短路径是？——求 $A^{(1)}$ 和 $path^{(1)}$

$A^{(1)} =$

	V0	V1	V2
V0	0	6	10
V1	10	0	4
V2	5	11	0

$path^{(1)} =$

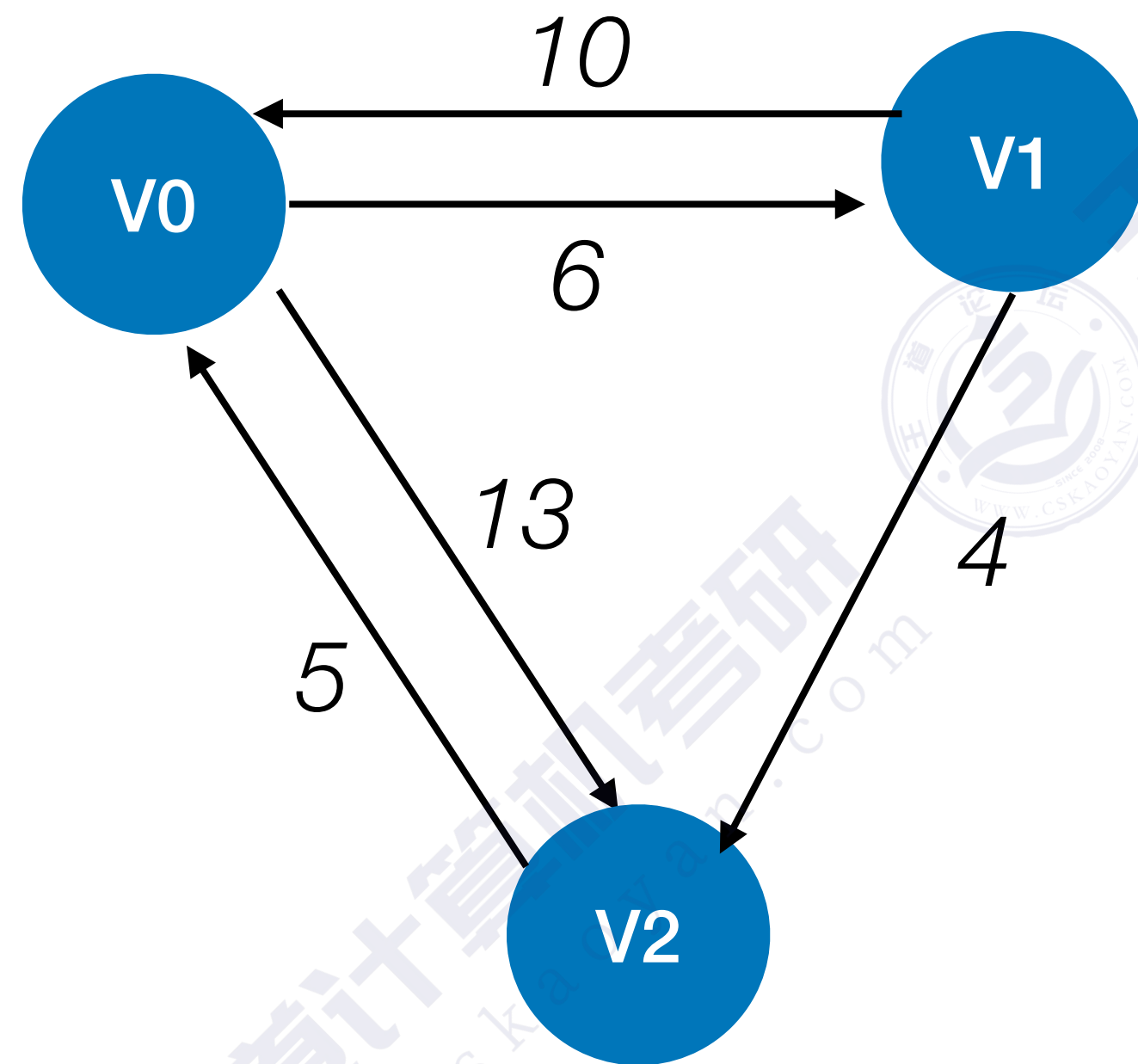
	V0	V1	V2
V0	-1	-1	1
V1	-1	-1	-1
V2	-1	0	-1

若
则

$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(1)} =$

	V0	V1	V2
V0	0	6	10
V1	10	0	4
V2	5	11	0

两个顶点之间的中转点

$path^{(1)} =$

	V0	V1	V2
V0	-1	-1	1
V1	-1	-1	-1
V2	-1	0	-1

#2: 若允许在 V_0 、 V_1 、 V_2 中转，最短路径是？——求 $A^{(2)}$ 和 $path^{(2)}$

若
则

$$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$$

$$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$$

$$path^{(k)}[i][j] = k$$

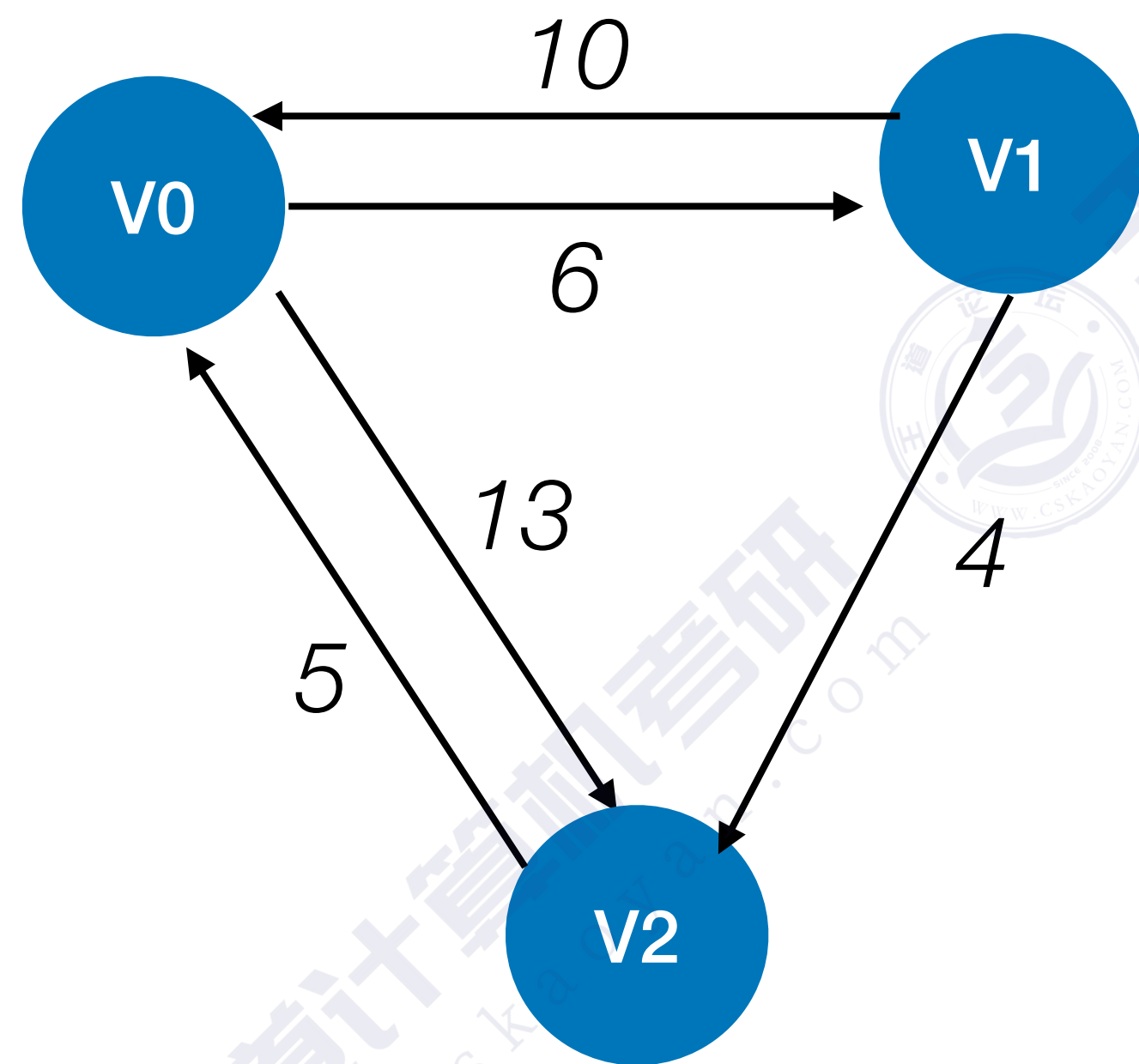
否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

$$A^{(1)}[1][0] > A^{(1)}[1][2] + A^{(1)}[2][0] = 9$$

$$A^{(2)}[1][0] = 9$$

$$path^{(2)}[1][0] = 2;$$

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(1)} =$

	V0	V1	V2
V0	0	6	10
V1	10	0	4
V2	5	11	0

两个顶点之间的中转点

$path^{(1)} =$

	V0	V1	V2
V0	-1	-1	1
V1	-1	-1	-1
V2	-1	0	-1

#2: 若允许在 V_0 、 V_1 、 V_2 中转，最短路径是？——求 $A^{(2)}$ 和 $path^{(2)}$

$A^{(2)} =$

	V0	V1	V2
V0	0	6	10
V1	9	0	4
V2	5	11	0

$path^{(2)} =$

	V0	V1	V2
V0	-1	-1	1
V1	2	-1	-1
V2	-1	0	-1

若
则

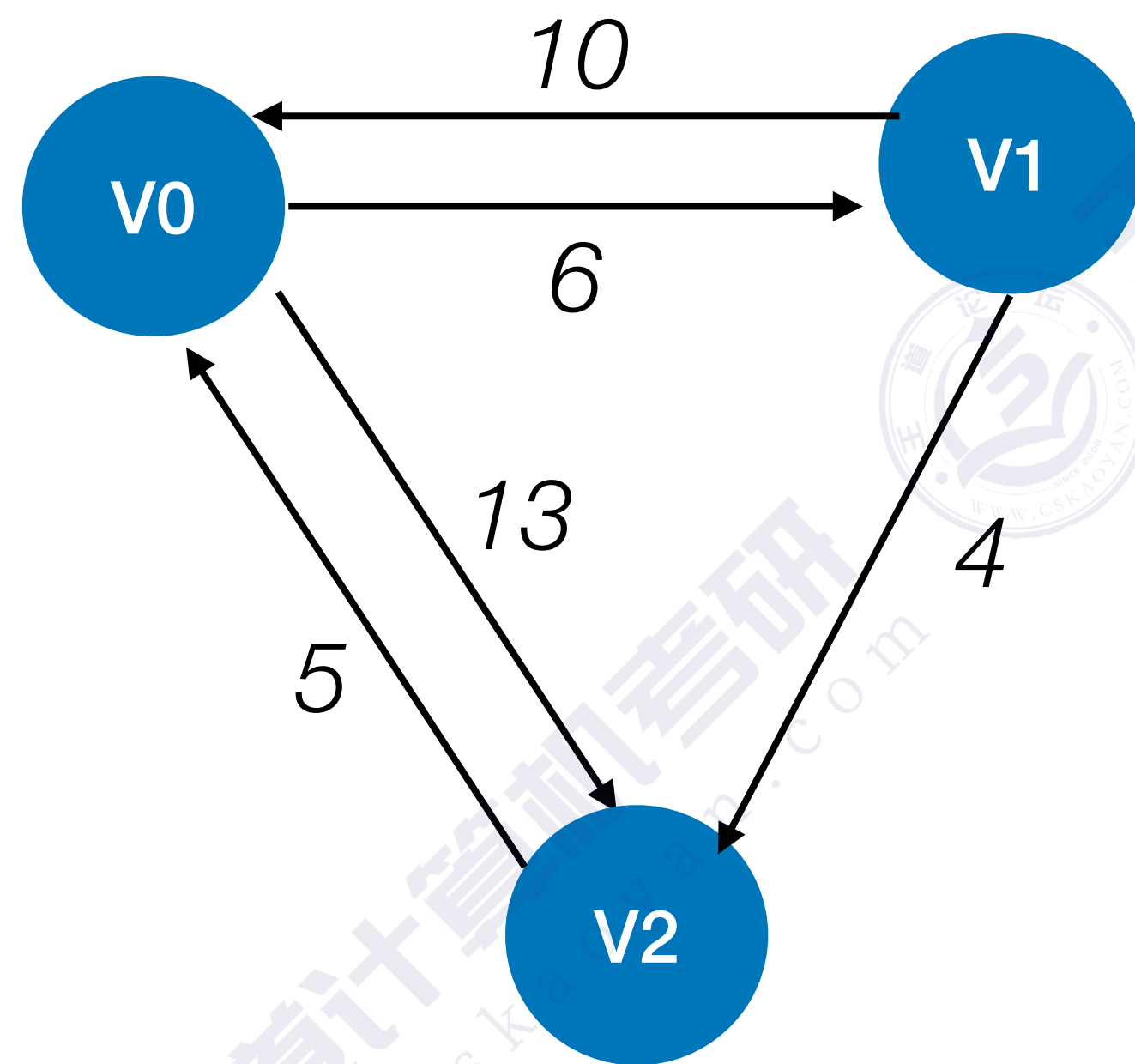
$$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$$

$$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$$

$$path^{(k)}[i][j] = k$$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法



目前来看，各顶点间的最短路径长度

$A^{(2)} =$

	V0	V1	V2
V0	0	6	10
V1	9	0	4
V2	5	11	0

两个顶点之间的中转点

$path^{(2)} =$

	V0	V1	V2
V0	-1	-1	1
V1	2	-1	-1
V2	-1	0	-1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

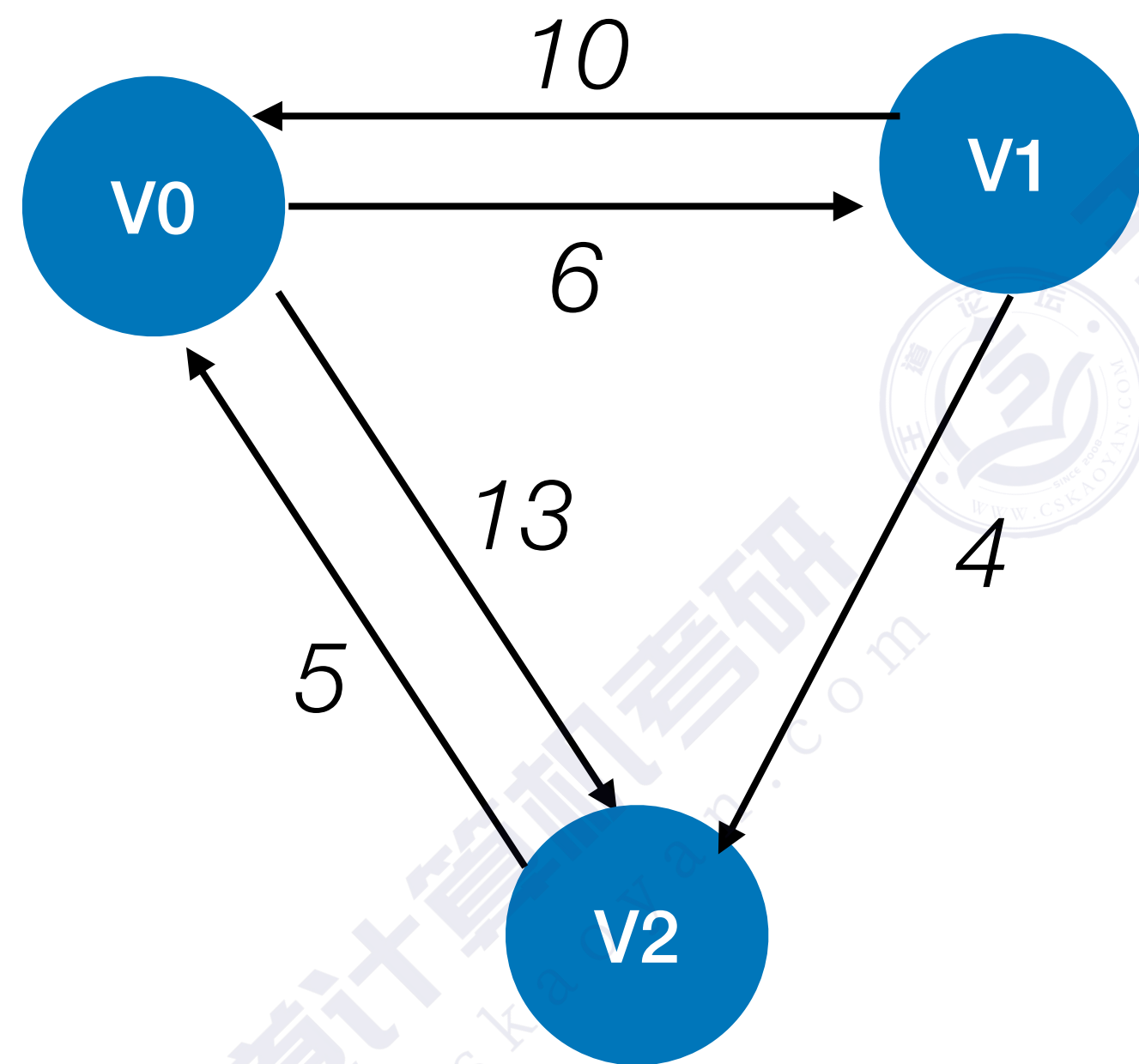
从 $A^{(-1)}$ 和 $path^{(-1)}$ 开始，经过 n 轮递推，得到 $A^{(n-1)}$ 和 $path^{(n-1)}$

根据 $A^{(2)}$ 可知，V1到V2 最短路径长度为 4，
根据 $path^{(2)}$ 可知，完整路径信息为 V1_V2

根据 $A^{(2)}$ 可知，V0到V2 最短路径长度为 10，
根据 $path^{(2)}$ 可知，完整路径信息为 V0_V1_V2

根据 $A^{(2)}$ 可知，V1到V0 最短路径长度为 9，
根据 $path^{(2)}$ 可知，完整路径信息为 V1_V2_V0

Floyd算法核心代码



A=

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	∞	0

path=

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1

//.....准备工作, 根据图的信息初始化矩阵 A 和 path (如上图)

for (int k=0; k<n; k++){ //考虑以 vk 作为中转点

for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号

for (int j=0; j<n; j++){

if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短

A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度

path[i][j]=k; //中转点

}

}

}

}

时间复杂度, $O(|V|^3)$

空间复杂度, $O(|V|^2)$

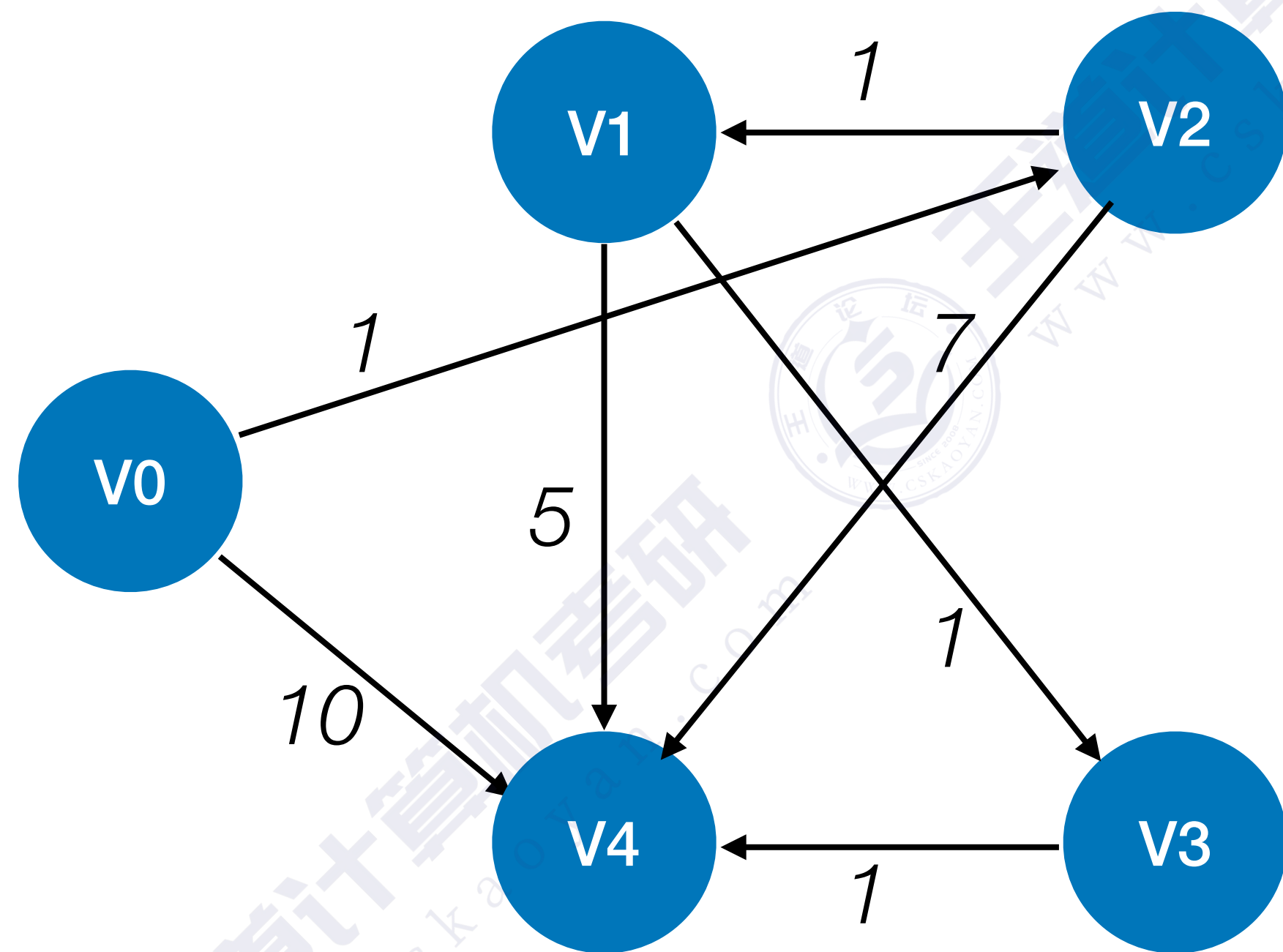
若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$

则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$

$path^{(k)}[i][j] = k$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(-1)} =$

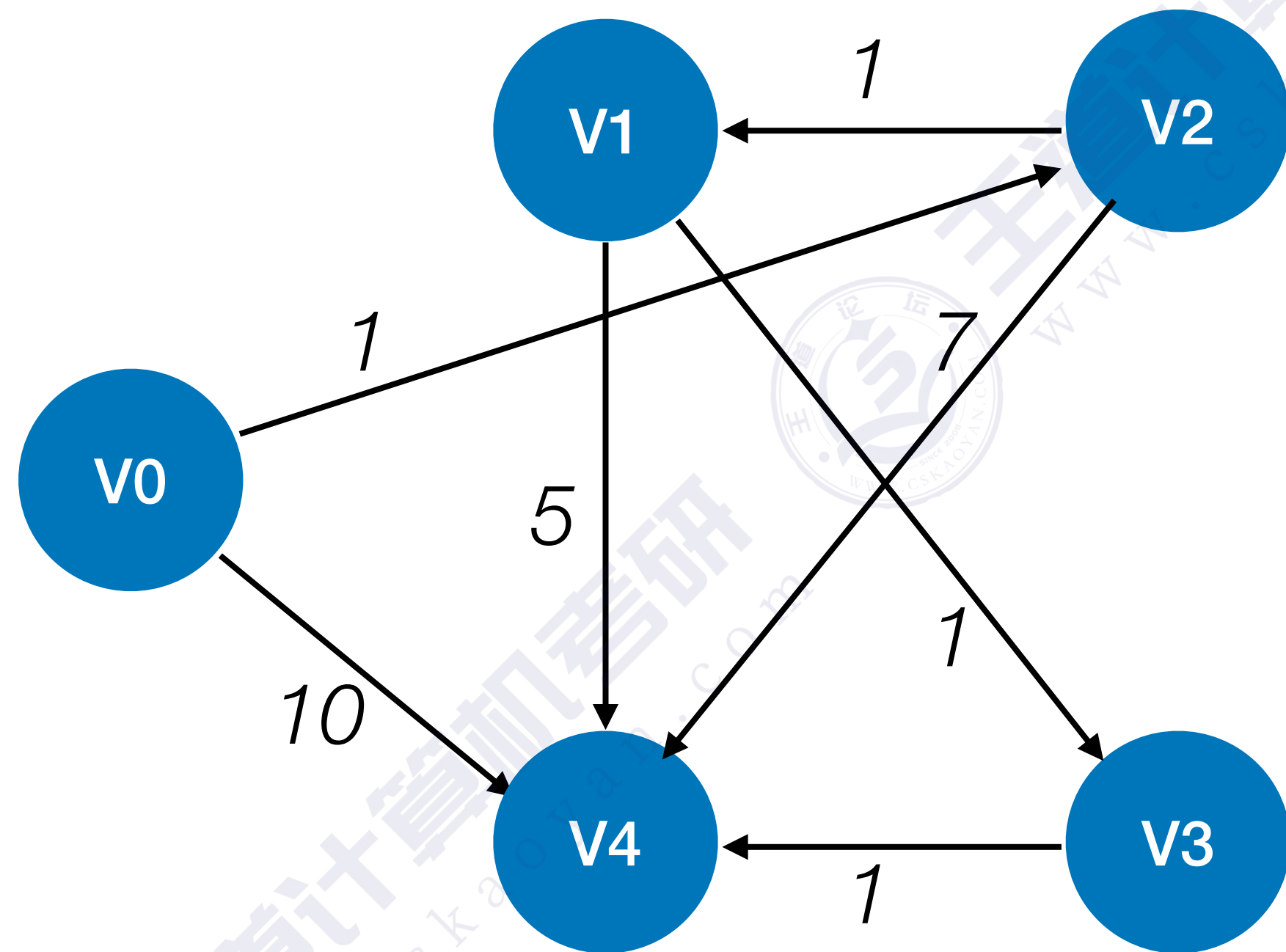
	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(-1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#初始：不允许在其他顶点中转，最短路径是？

Floyd算法实例



$A^{(-1)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(-1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#0: 若允许在 **V0** 中转, 最短路径是? ——求 $A^{(0)}$ 和 $path^{(0)}$

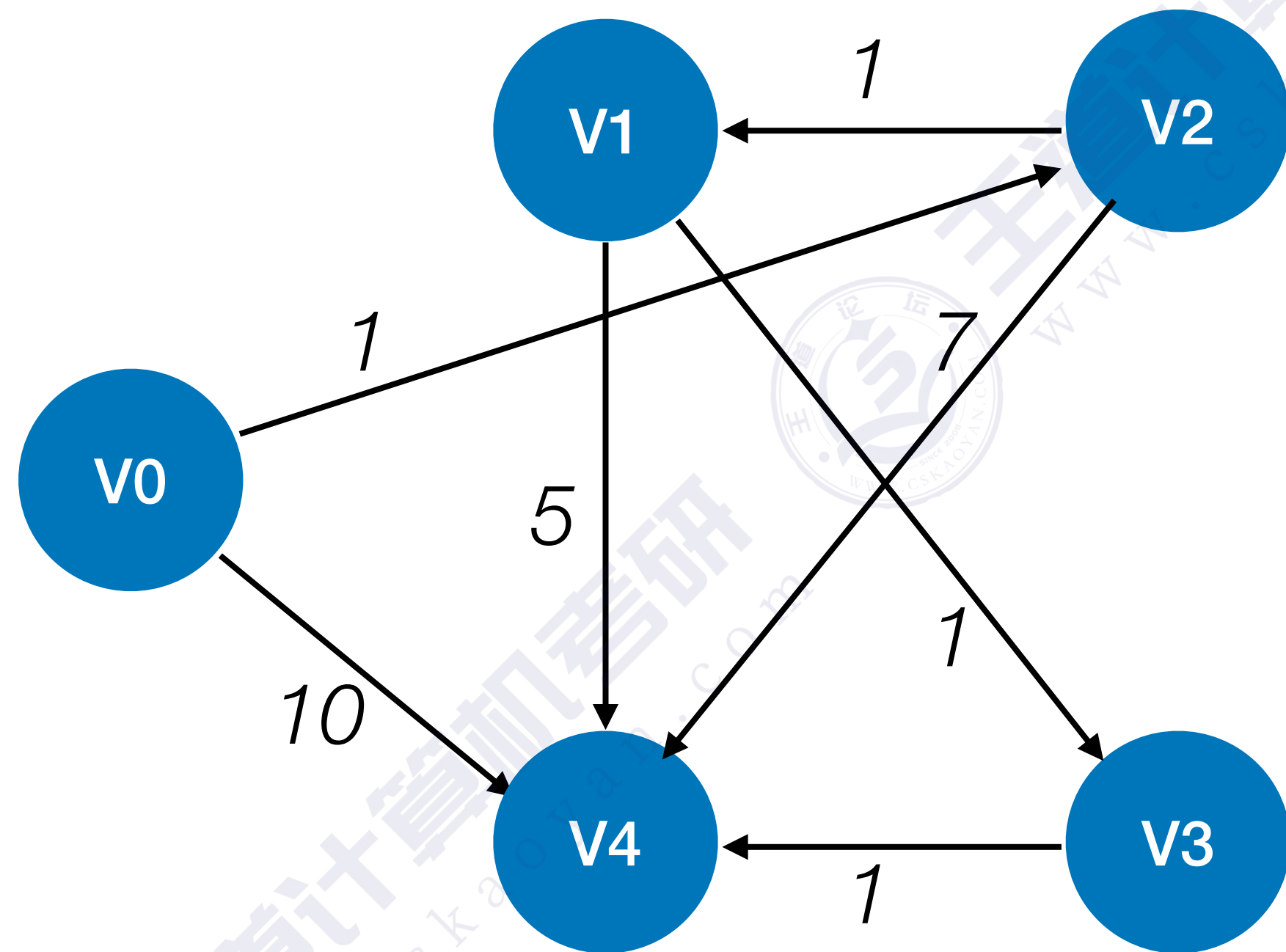
```

for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号
    for (int j=0; j<n; j++){
        if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
            A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
            path[i][j]=k; //中转点
        }
    }
}
    
```

k=0

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(-1)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(-1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#0: 若允许在 **V0** 中转, 最短路径是? ——求 $A^{(0)}$ 和 $path^{(0)}$

$A^{(0)} =$

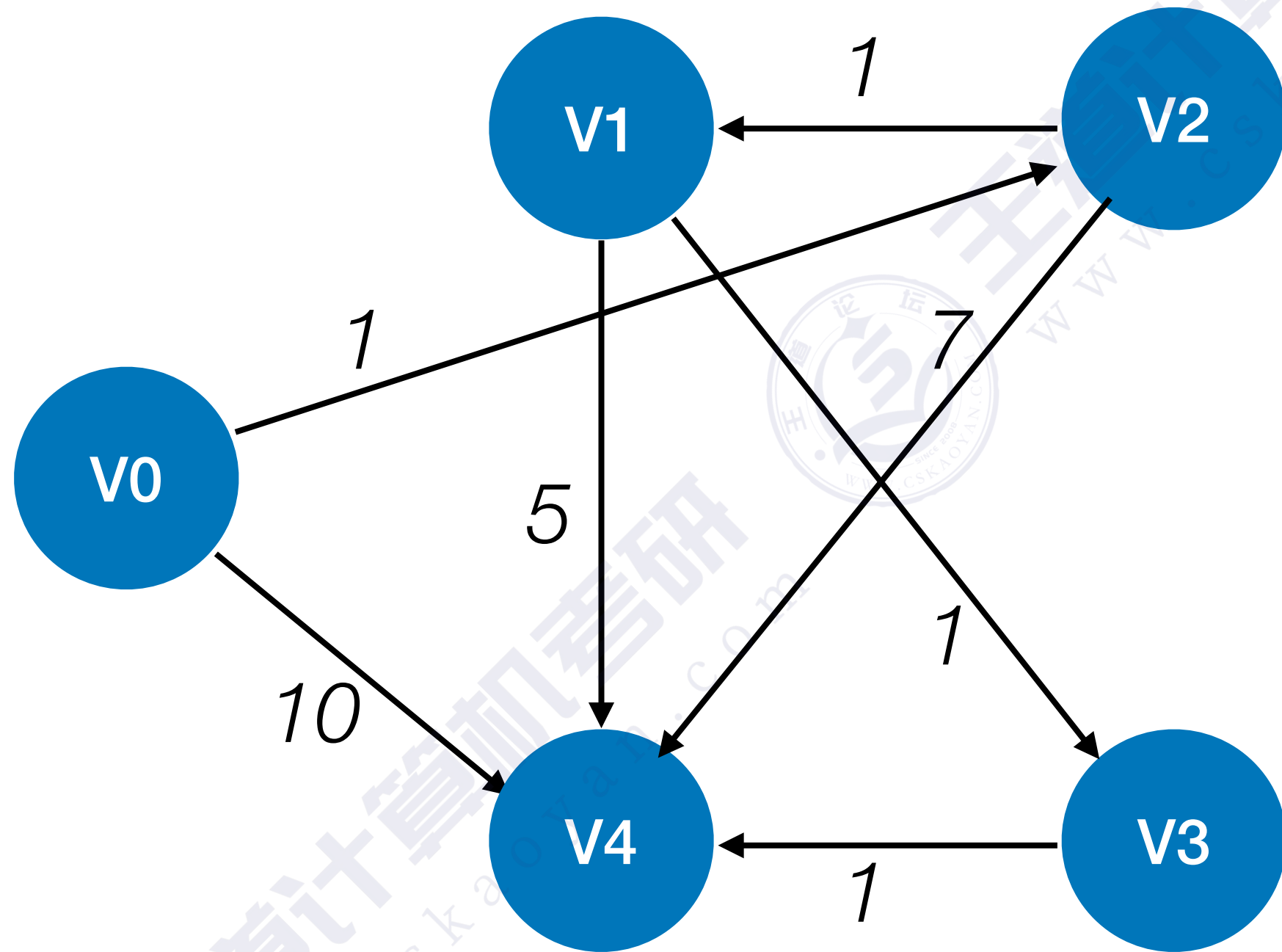
	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(0)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(0)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(0)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

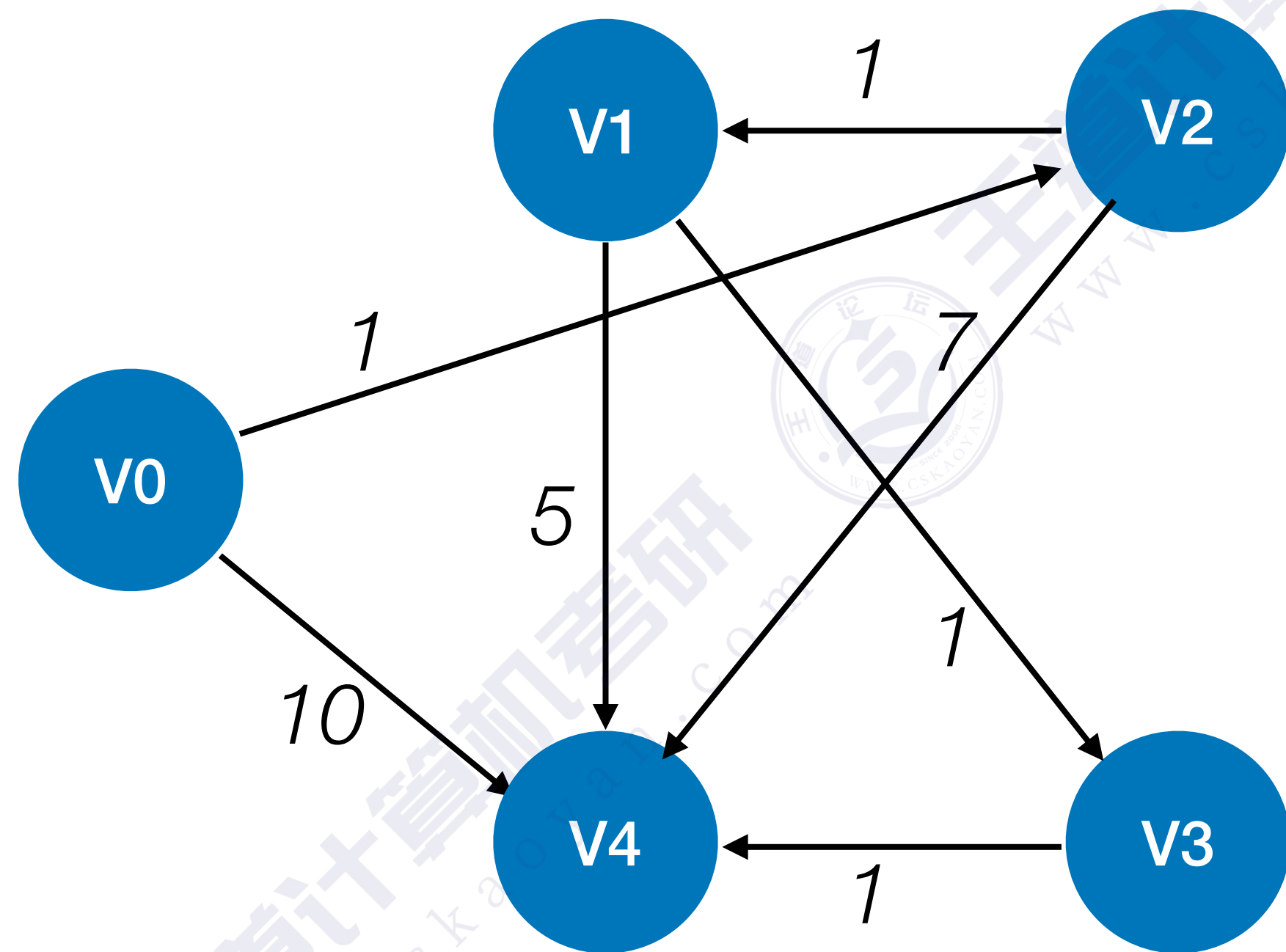
#1: 若允许在 V_0 、 V_1 中转, 最短路径是? —— 求 $A^{(1)}$ 和 $path^{(1)}$

```
for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号
    for (int j=0; j<n; j++){
        if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
            A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
            path[i][j]=k; //中转点
        }
    }
}
```

k=1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(0)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(0)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#1: 若允许在 V_0 、 V_1 中转, 最短路径是? ——求 $A^{(1)}$ 和 $path^{(1)}$

$$A^{(0)}[2][3] > A^{(0)}[2][1] + A^{(0)}[1][3] = 2$$

$$A^{(1)}[2][3] = 2$$

$$path^{(1)}[2][3] = 1;$$

$$A^{(0)}[2][4] > A^{(0)}[2][1] + A^{(0)}[1][4] = 6$$

$$A^{(1)}[2][4] = 6$$

$$path^{(1)}[2][4] = 1;$$

若
则

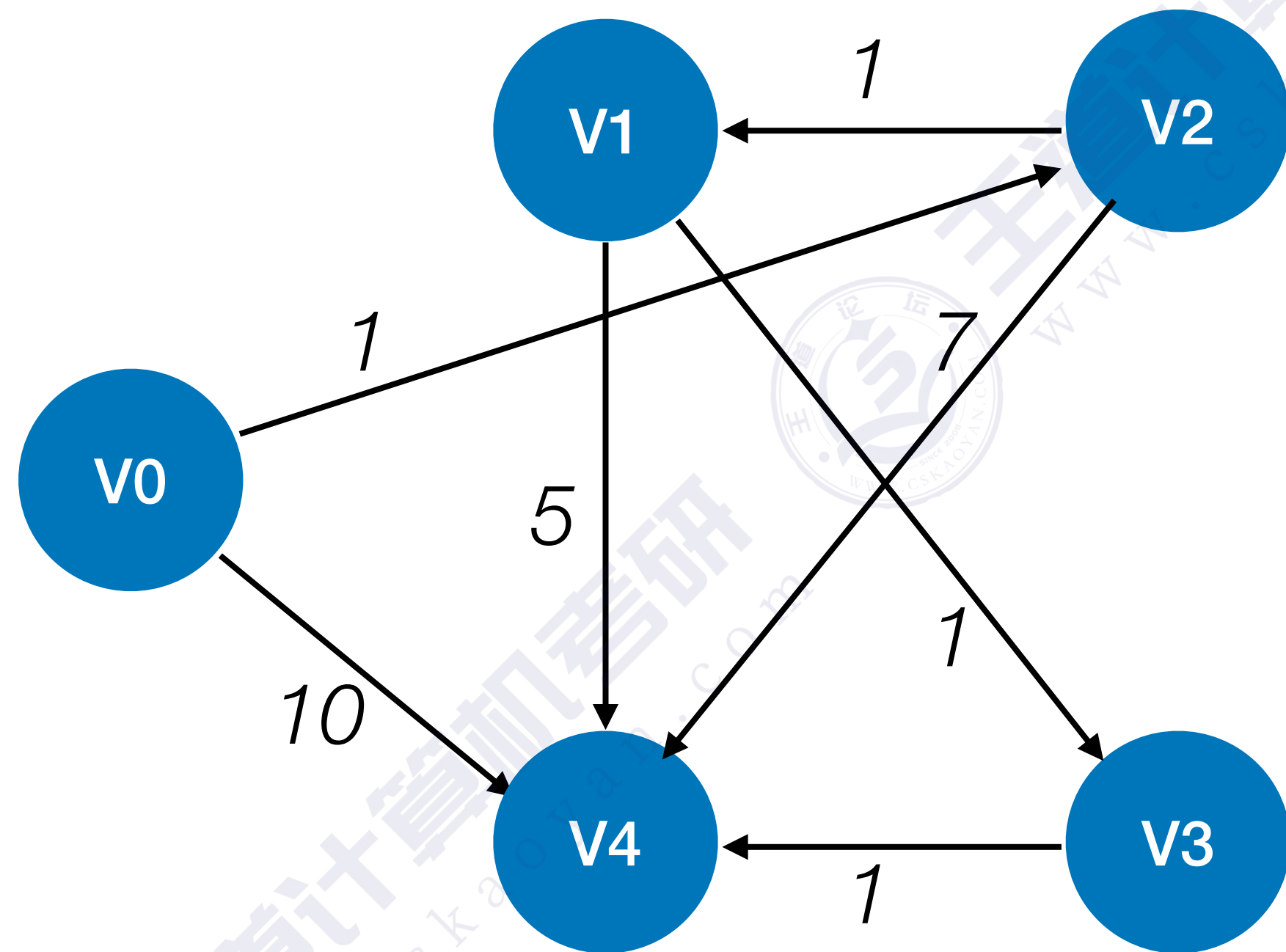
$$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$$

$$A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$$

$$path^{(k)}[i][j] = k$$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(0)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	∞	7
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(0)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	-1	-1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#1: 若允许在 V_0 、 V_1 中转, 最短路径是? —— 求 $A^{(1)}$ 和 $path^{(1)}$

$A^{(1)} =$

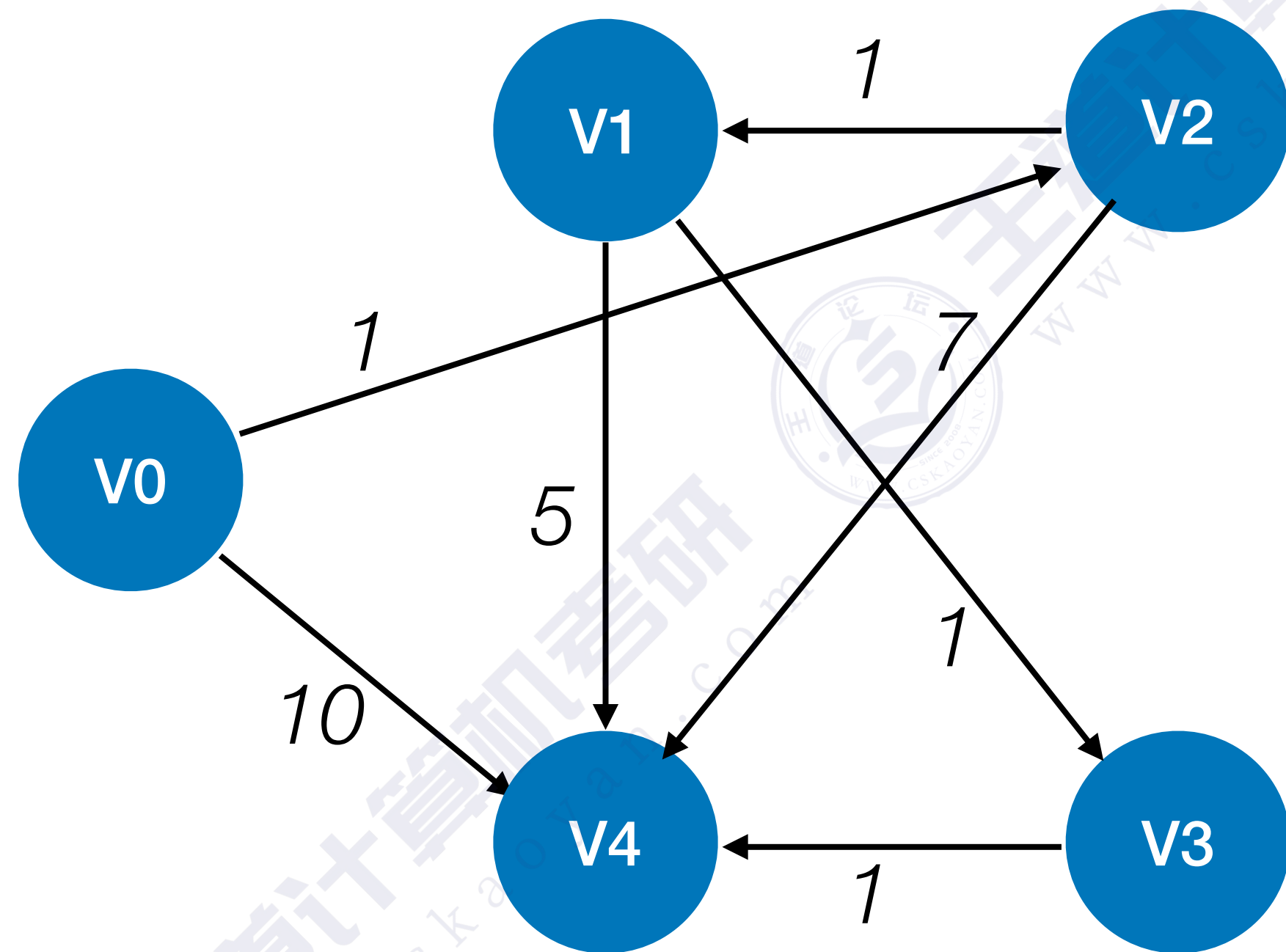
	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(1)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#2: 若允许在 V_0 、 V_1 、 V_2 中转, 最短路径是? —— 求 $A^{(2)}$ 和 $path^{(2)}$

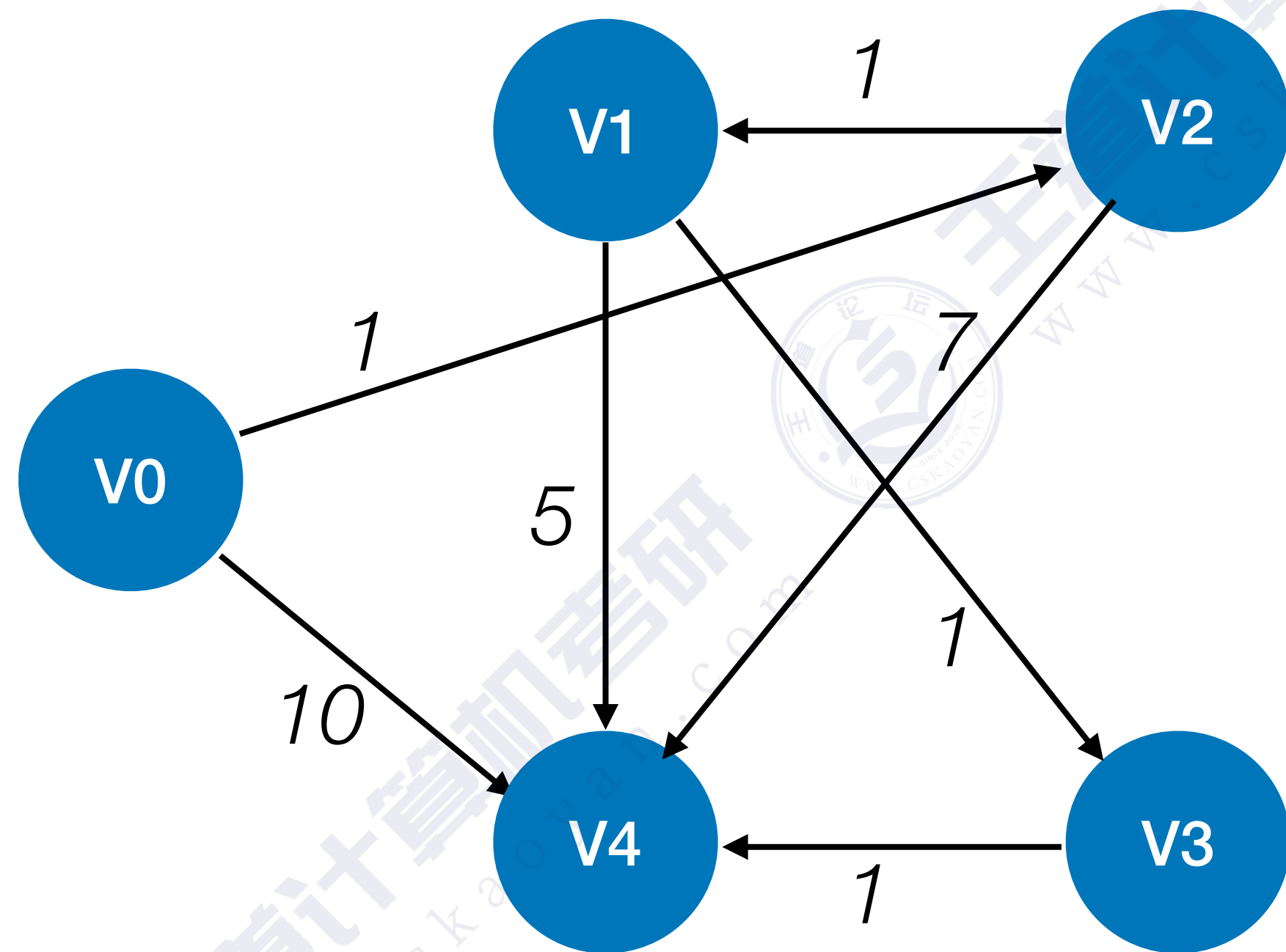
```

for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号
    for (int j=0; j<n; j++){
        if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
            A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
            path[i][j]=k; //中转点
        }
    }
}
    
```

k=2

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(1)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#2: 若允许在 V_0 、 V_1 、 V_2 中转, 最短路径是? —— 求 $A^{(2)}$ 和 $path^{(2)}$

$$A^{(1)}[0][1] > A^{(1)}[0][2] + A^{(1)}[2][1] = 2$$

$$A^{(2)}[0][1] = 2; path^{(2)}[0][1] = 2;$$

$$A^{(1)}[0][3] > A^{(1)}[0][2] + A^{(1)}[2][3] = 3$$

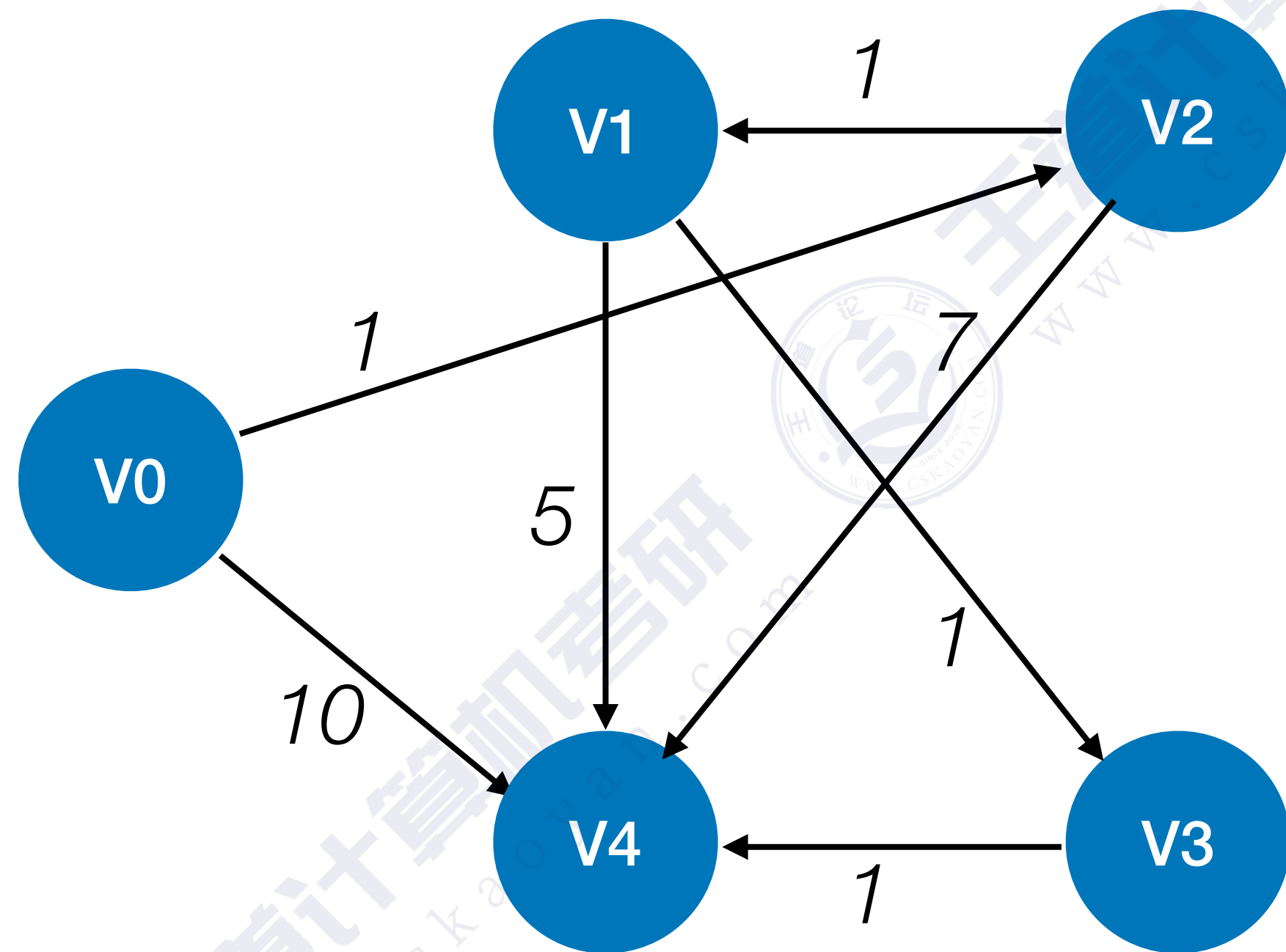
$$A^{(2)}[0][3] = 3; path^{(2)}[0][3] = 2;$$

$$A^{(1)}[0][4] > A^{(1)}[0][2] + A^{(1)}[2][4] = 7$$

$$A^{(2)}[0][4] = 7; path^{(2)}[0][4] = 2;$$

若
则 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(1)} =$

	V0	V1	V2	V3	V4
V0	0	∞	1	∞	10
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(1)} =$

	V0	V1	V2	V3	V4
V0	-1	-1	-1	-1	-1
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#2: 若允许在 V_0 、 V_1 、 V_2 中转, 最短路径是? —— 求 $A^{(2)}$ 和 $path^{(2)}$

$A^{(2)} =$

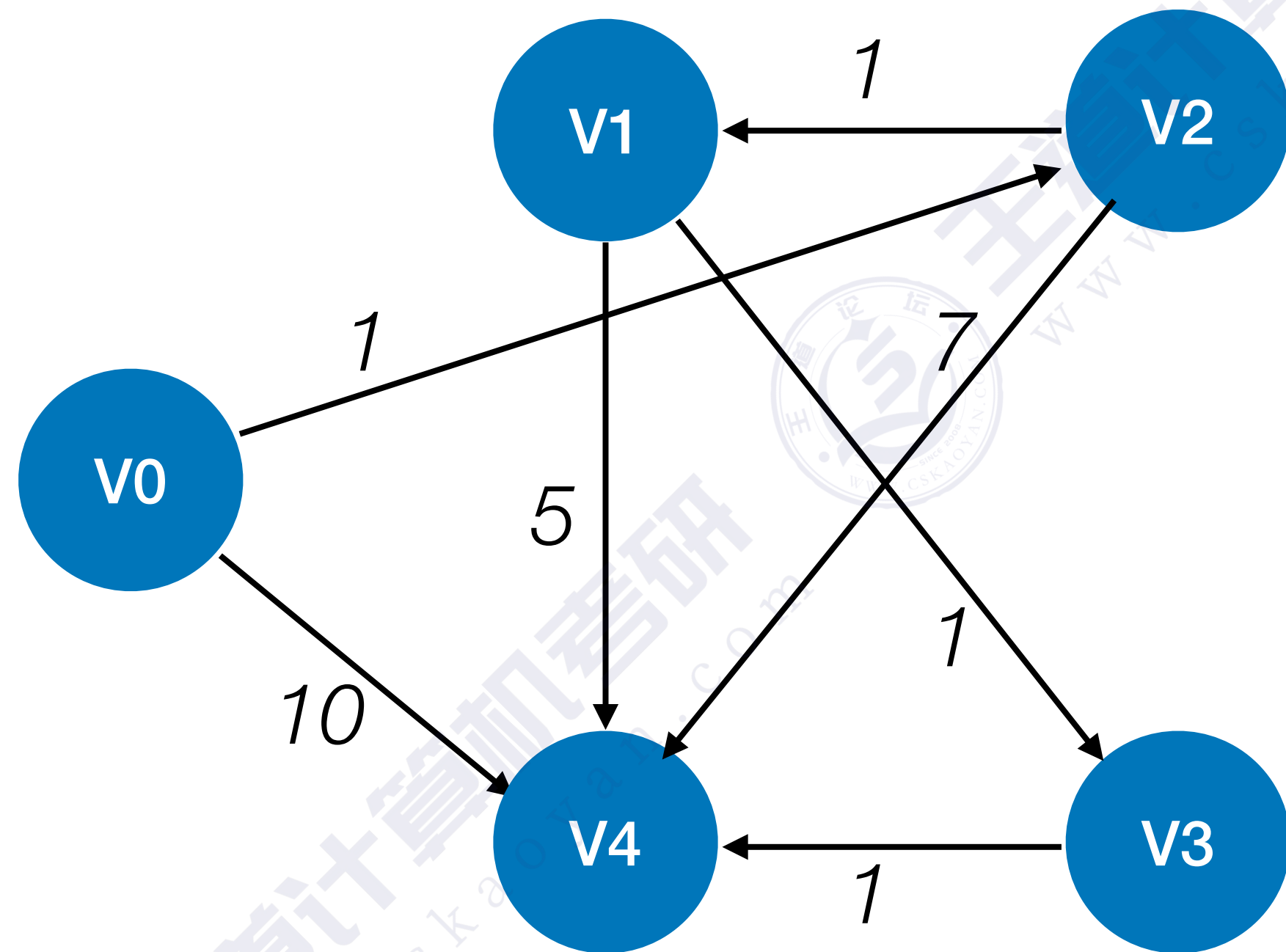
	V0	V1	V2	V3	V4
V0	0	2	1	3	7
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(2)} =$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	2
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$A^{(2)} =$

	V0	V1	V2	V3	V4
V0	0	2	1	3	7
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(2)} =$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	2
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#3: 若允许在 V_0 、 V_1 、 V_2 、 V_3 中转，最短路径是？——求 $A^{(3)}$ 和 $path^{(3)}$

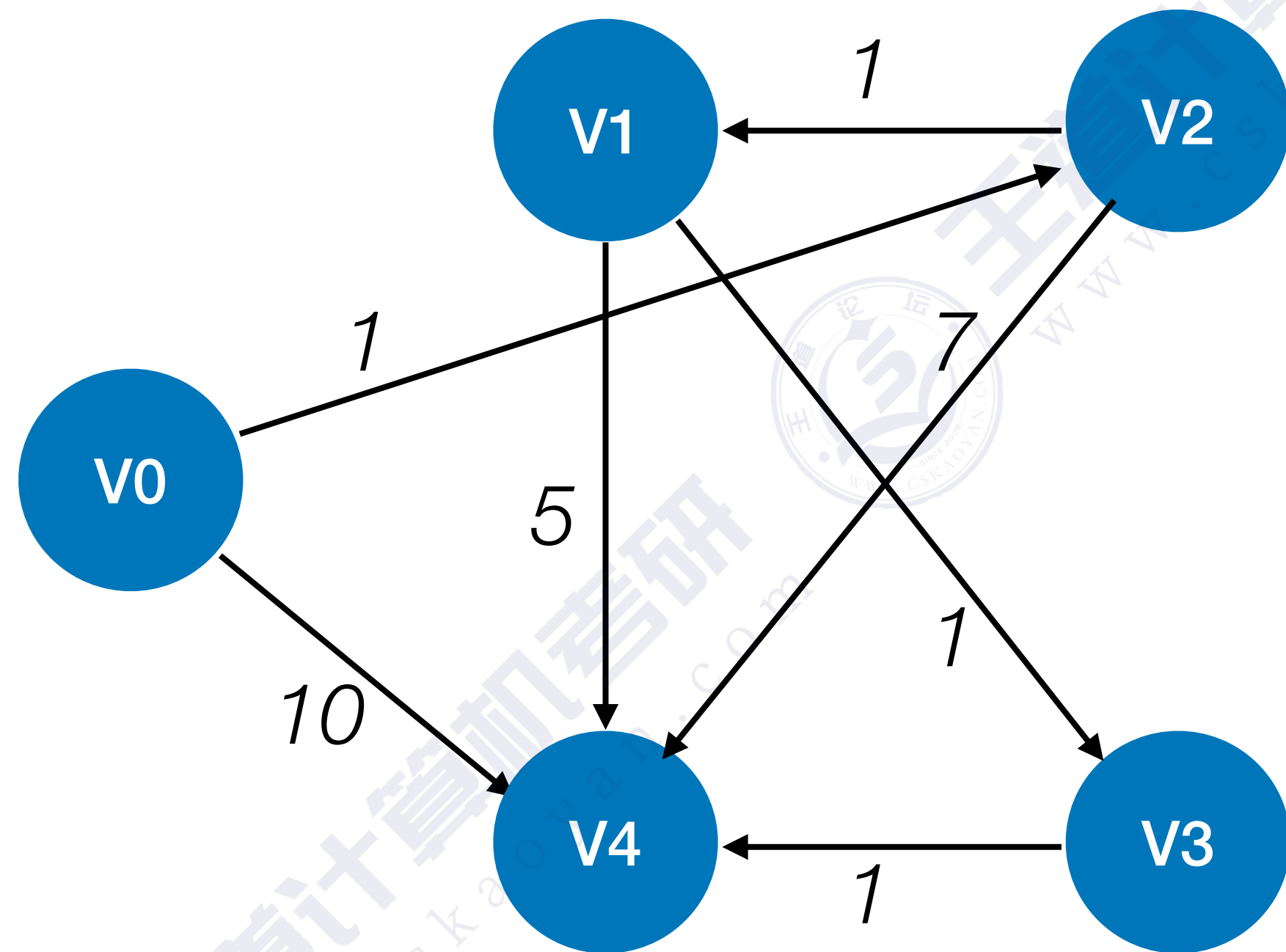
```

for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号
    for (int j=0; j<n; j++){
        if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
            A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
            path[i][j]=k; //中转点
        }
    }
}
    
```

k=3

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $path^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$$A^{(2)} =$$

	V0	V1	V2	V3	V4
V0	0	2	1	3	7
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$$\text{path}^{(2)} =$$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	2
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#3: 若允许在 V_0 、 V_1 、 V_2 、 V_3 中转，最短路径是？——求 $A^{(3)}$ 和 $\text{path}^{(3)}$

$$A^{(2)}[0][4] > A^{(2)}[0][3] + A^{(2)}[3][4] = 4$$

$$A^{(3)}[0][4] = 4; \text{path}^{(3)}[0][4] = 3;$$

$$A^{(2)}[1][4] > A^{(2)}[1][3] + A^{(2)}[3][4] = 2$$

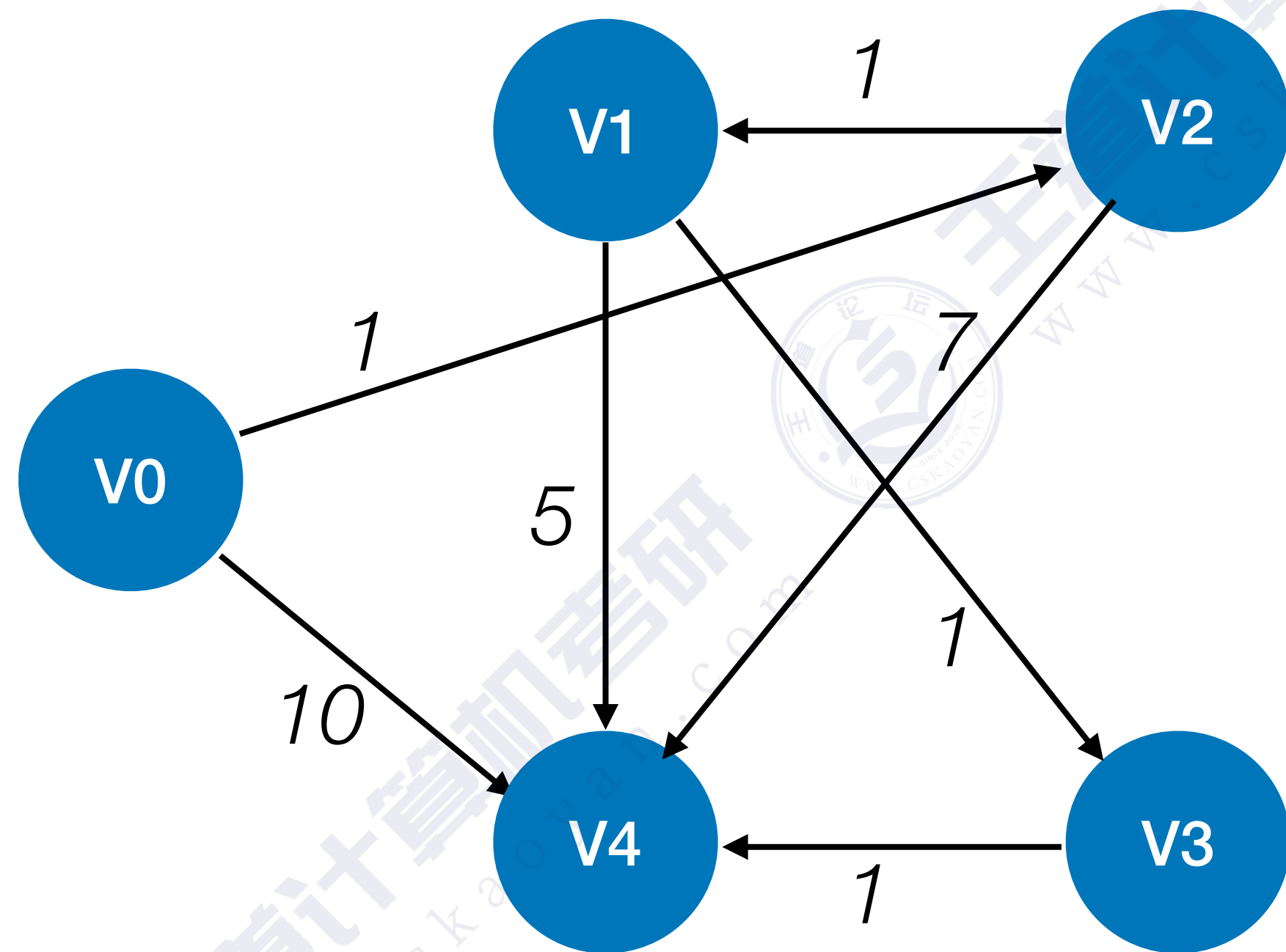
$$A^{(3)}[1][4] = 2; \text{path}^{(3)}[1][4] = 3;$$

$$A^{(2)}[2][4] > A^{(2)}[2][3] + A^{(2)}[3][4] = 3$$

$$A^{(3)}[2][4] = 3; \text{path}^{(3)}[2][4] = 3;$$

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $\text{path}^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $\text{path}^{(k)}$ 保持原值

Floyd算法实例



$A^{(2)} =$

	V0	V1	V2	V3	V4
V0	0	2	1	3	7
V1	∞	0	∞	1	5
V2	∞	1	0	2	6
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(2)} =$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	2
V1	-1	-1	-1	-1	-1
V2	-1	-1	-1	1	1
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#3: 若允许在 V_0 、 V_1 、 V_2 、 V_3 中转，最短路径是？——求 $A^{(3)}$ 和 $path^{(3)}$

$A^{(3)} =$

	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$path^{(3)} =$

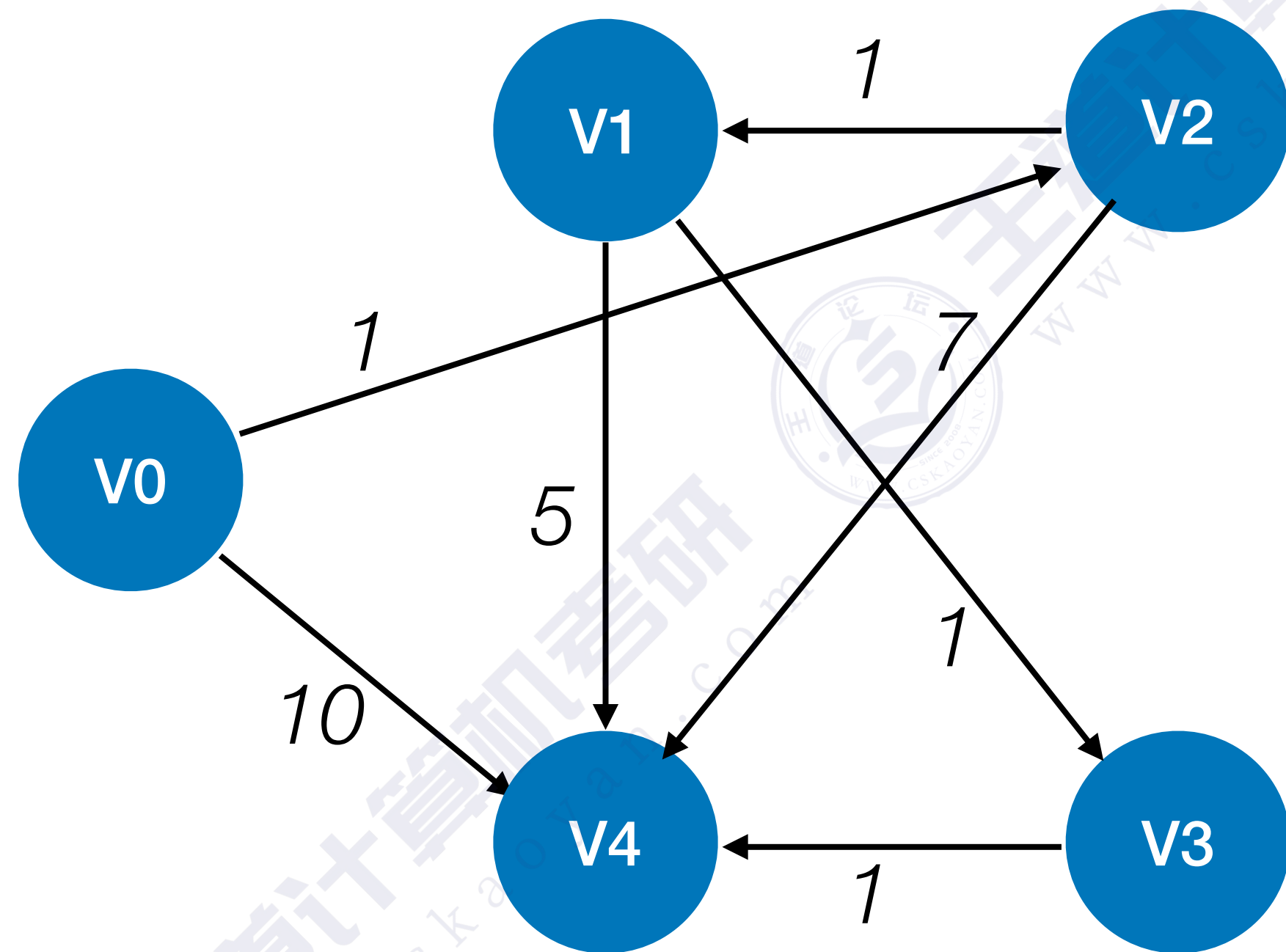
	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

若
则

$A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j];$
 $path^{(k)}[i][j] = k$

否则 $A^{(k)}$ 和 $path^{(k)}$ 保持原值

Floyd算法实例



$$A^{(3)} =$$

	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$$\text{path}^{(3)} =$$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#4: 若允许在 V_0 、 V_1 、 V_2 、 V_3 、 V_4 中转，最短路径是？——求 $A^{(4)}$ 和 $\text{path}^{(4)}$

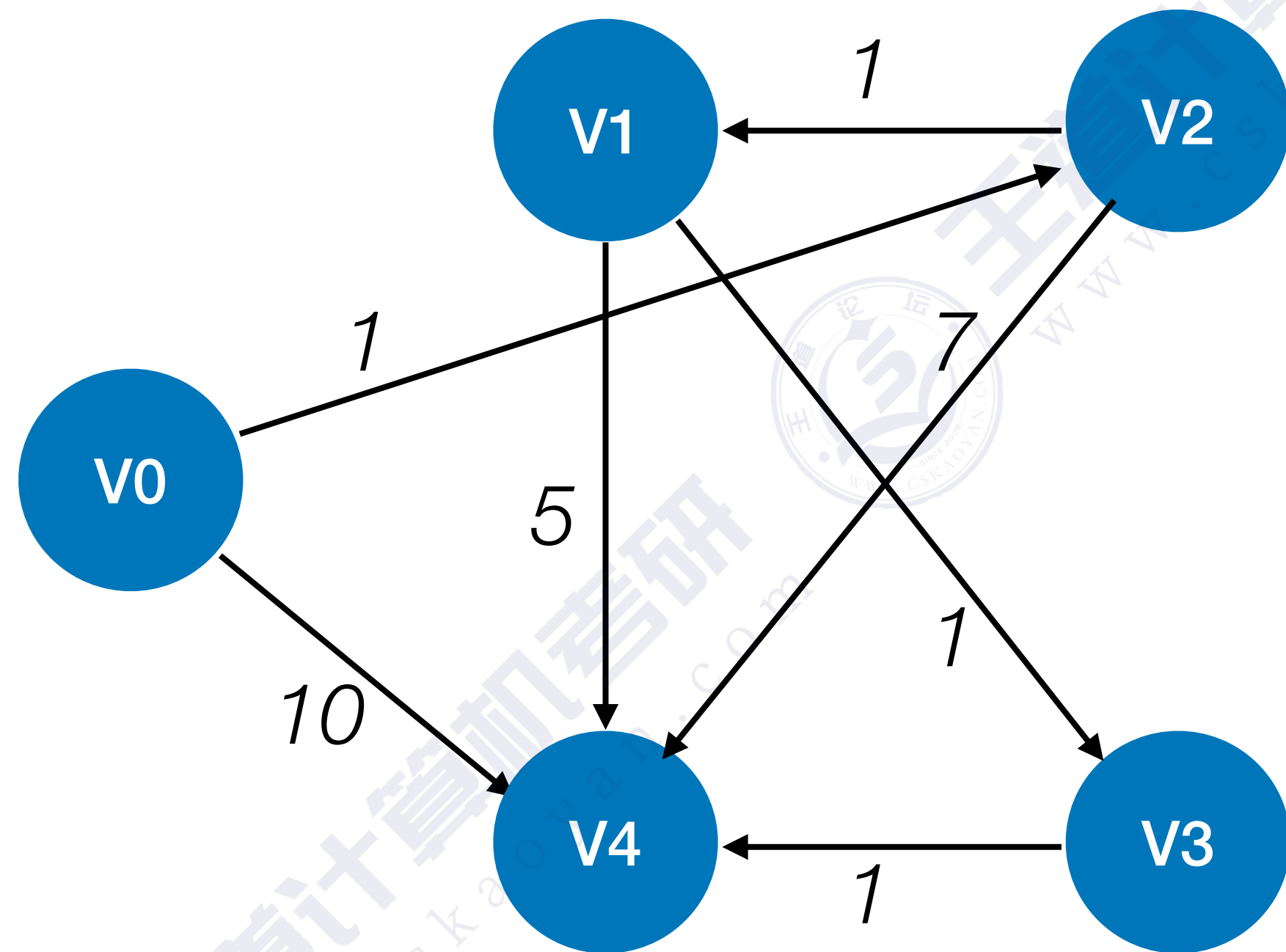
```

for(int i=0; i<n; i++) { //遍历整个矩阵, i为行号, j为列号
    for (int j=0; j<n; j++){
        if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
            A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
            path[i][j]=k; //中转点
        }
    }
}
    
```

k=4

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $\text{path}^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $\text{path}^{(k)}$ 保持原值

Floyd算法实例



$$A^{(3)} =$$

	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$$\text{path}^{(3)} =$$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

#4: 若允许在 V_0 、 V_1 、 V_2 、 V_3 、 V_4 中转，最短路径是？——求 $A^{(4)}$ 和 $\text{path}^{(4)}$

$$A^{(4)} =$$

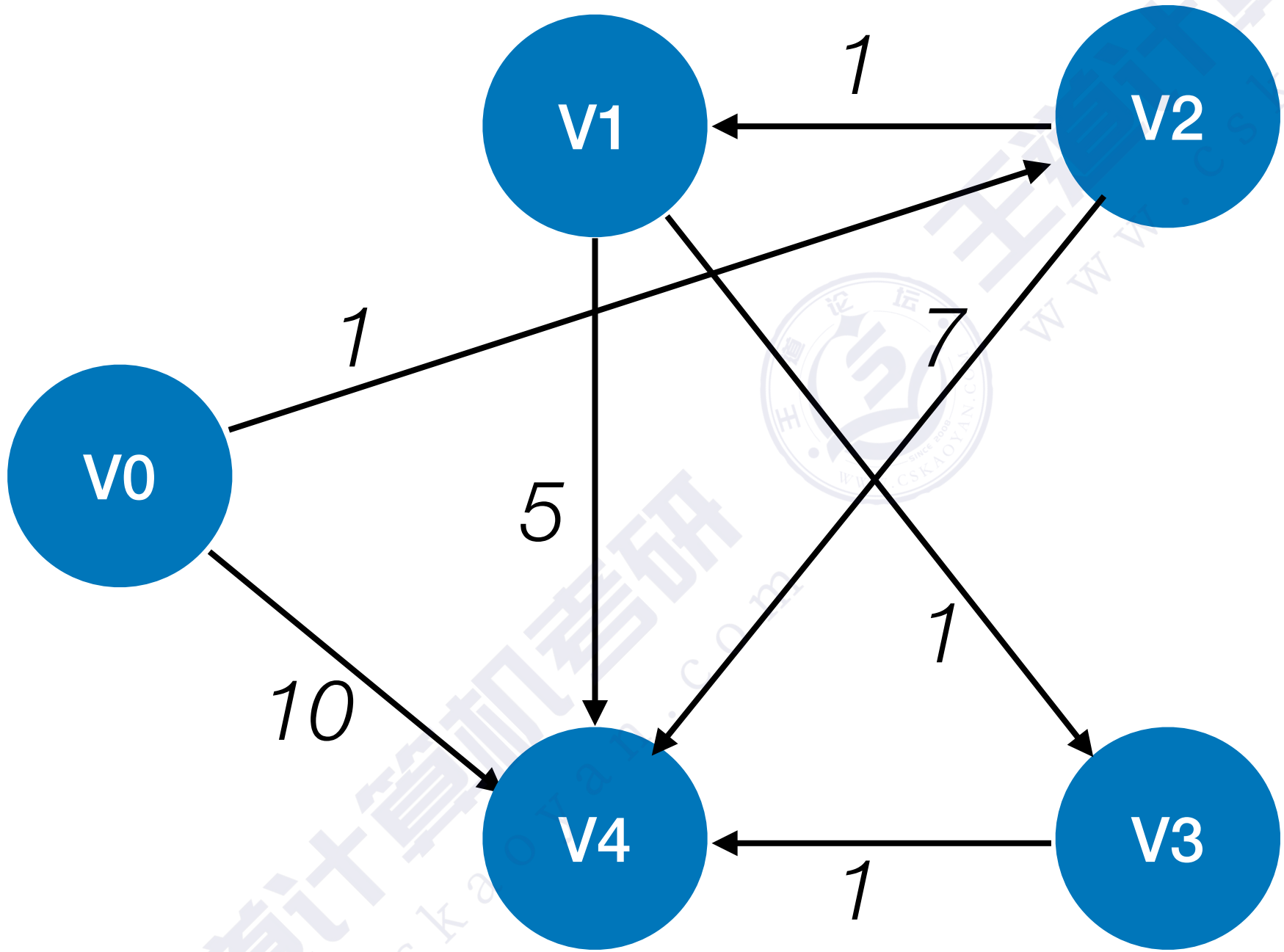
	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

$$\text{path}^{(4)} =$$

	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

若 $A^{(k-1)}[i][j] > A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$
 则 $A^{(k)}[i][j] = A^{(k-1)}[i][k] + A^{(k-1)}[k][j]$;
 $\text{path}^{(k)}[i][j] = k$
 否则 $A^{(k)}$ 和 $\text{path}^{(k)}$ 保持原值

Floyd算法实例



A =

	V0	V1	V2	V3	V4
V0	0	2	1	3	4
V1	∞	0	∞	1	2
V2	∞	1	0	2	3
V3	∞	∞	∞	0	1
V4	∞	∞	∞	∞	0

path =

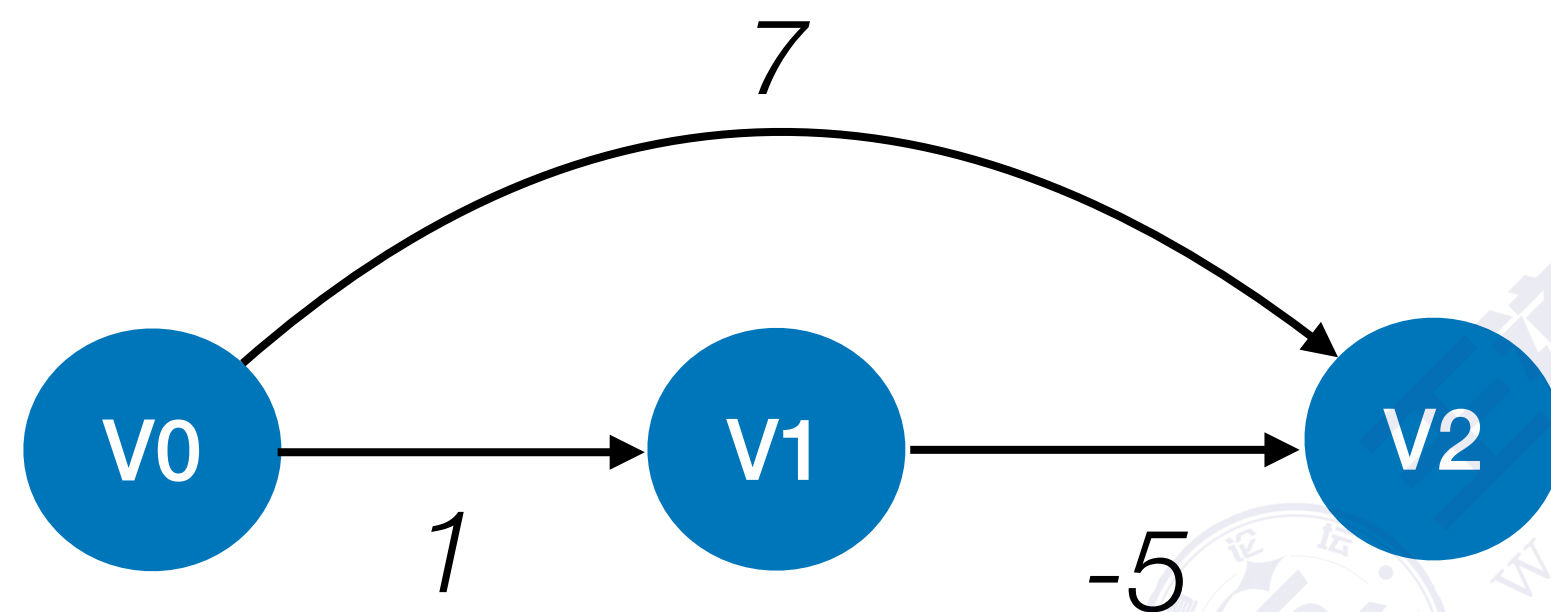
	V0	V1	V2	V3	V4
V0	-1	2	-1	2	3
V1	-1	-1	-1	-1	3
V2	-1	-1	-1	1	3
V3	-1	-1	-1	-1	-1
V4	-1	-1	-1	-1	-1

V0到V4 最短路径长度为 $A[0][4]=4$

通过path矩阵递归地找到完整路径：

V0			V3	V4
V0	V2		V3	V4
V0	V2	V1	V3	V4

练习：Floyd算法用于负权图



A=

	V0	V1	V2
V0	0	1	7
V1	∞	0	-5
V2	∞	∞	0

path=

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1



OK



很强

Floyd算法可以用于
负权值带权图

//.....准备工作，根据图的信息初始化矩阵 A 和 path (如上图)

```
for (int k=0; k<n; k++){ //考虑以 vk 作为中转点
```

```
for(int i=0; i<n; i++){ //遍历整个矩阵，i为行号，j为列号
```

```
for (int j=0; j<n; j++){
```

```
if (A[i][j]>A[i][k]+A[k][j]){ //以 vk 为中转点的路径更短
```

```
A[i][j]=A[i][k]+A[k][j]; //更新最短路径长度
```

```
path[i][j]=k; //中转点
```

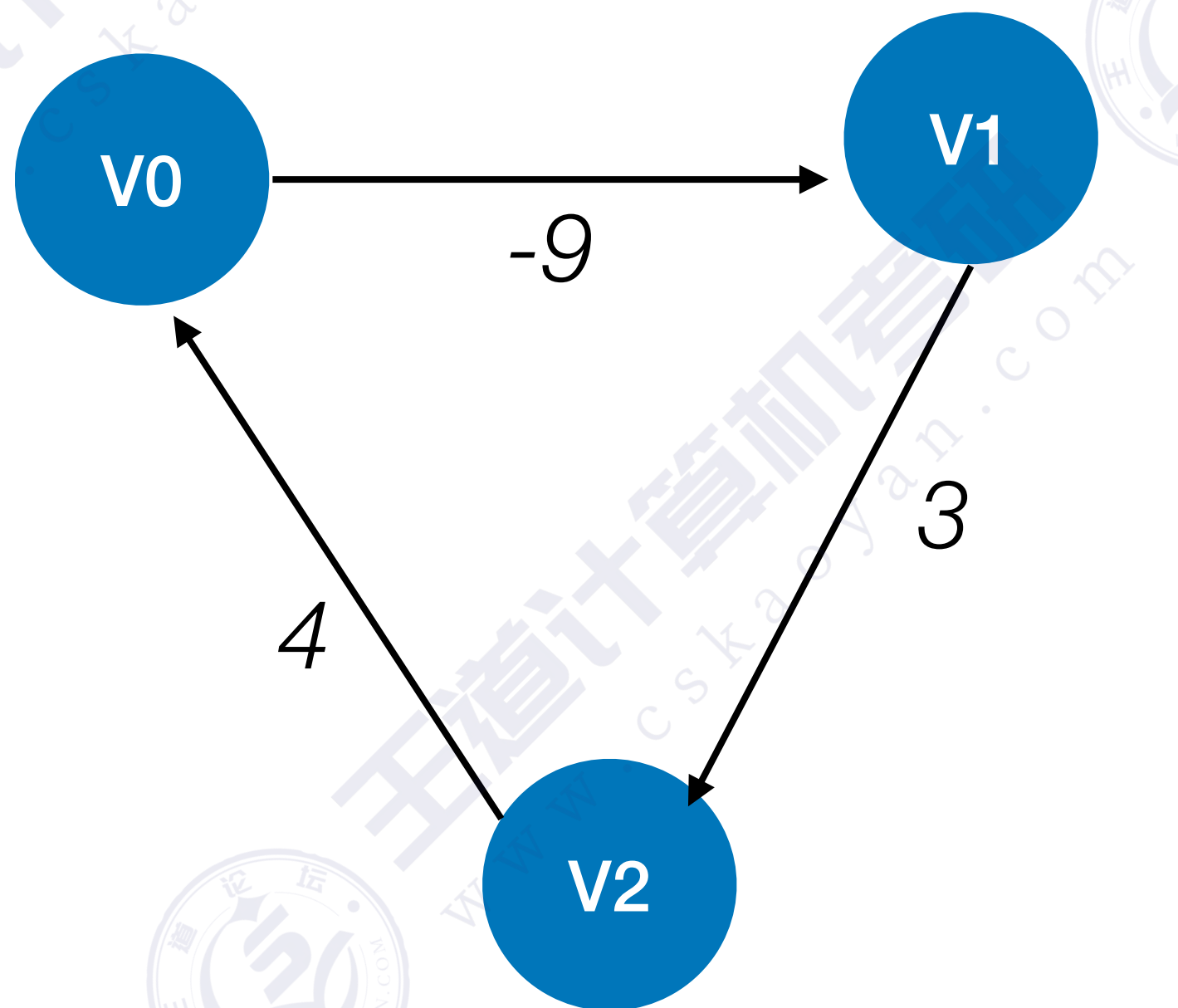
```
}
```

```
}
```

```
}
```

```
}
```

不能解决的问题



Floyd 算法不能解决带有“负权回路”的图（有负权值的边组成回路），这种图有可能没有最短路径

知识点回顾与重要考点

	BFS 算法	Dijkstra 算法	Floyd 算法
无权图	✓	✓	✓
带权图	✗	✓	✓
带负权值的图	✗	✗	✓
带负权回路的图	✗	✗	✗
时间复杂度	$O(V ^2)$ 或 $O(V + E)$	$O(V ^2)$	$O(V ^3)$
通常用于	求无权图的单源最短路径	求带权图的单源最短路径	求带权图中各顶点间的最短路径

注：也可用 Dijkstra 算法求所有顶点间的最短路径，重复 $|V|$ 次即可，总的时间复杂度也是 $O(|V|^3)$

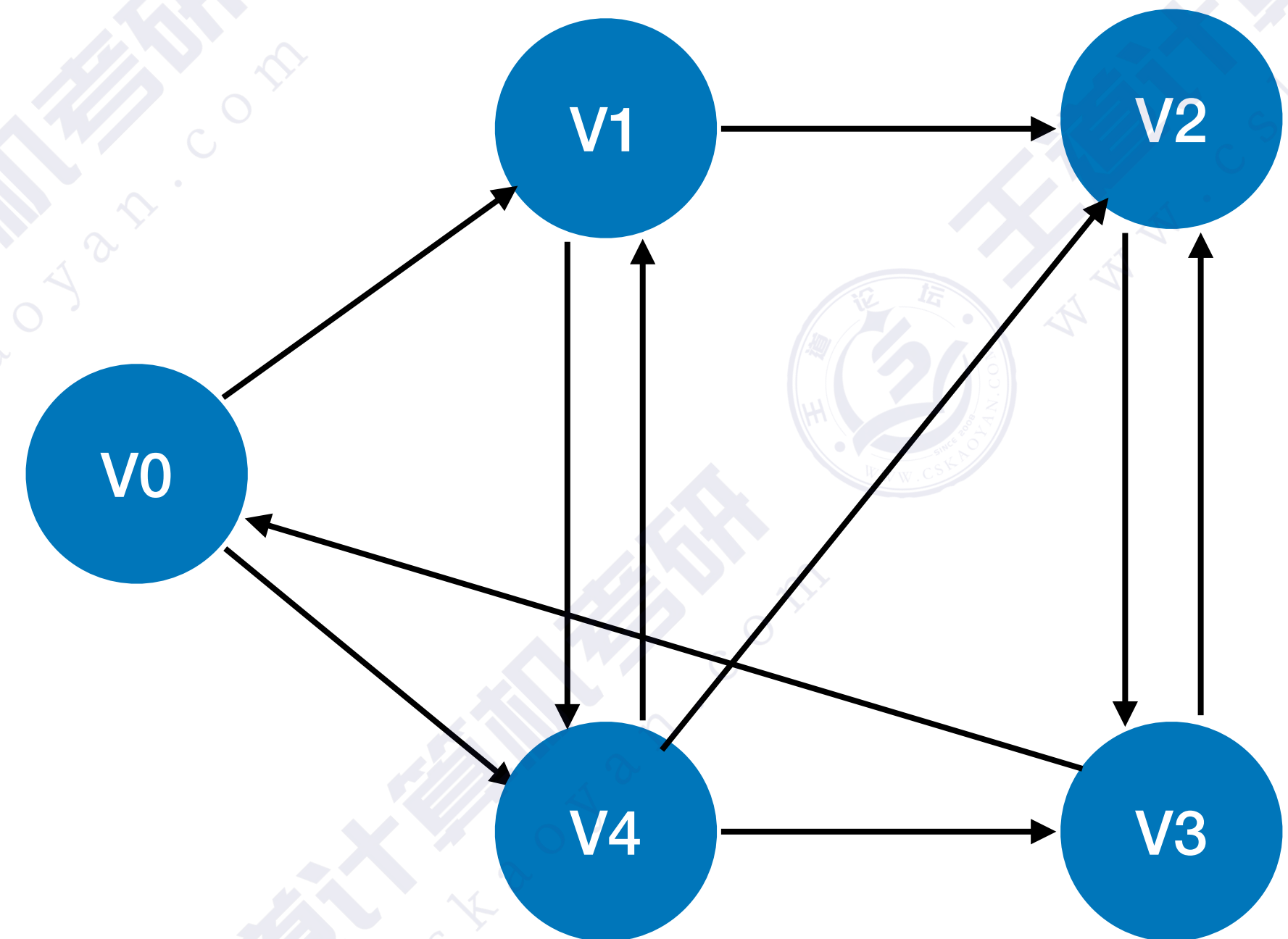
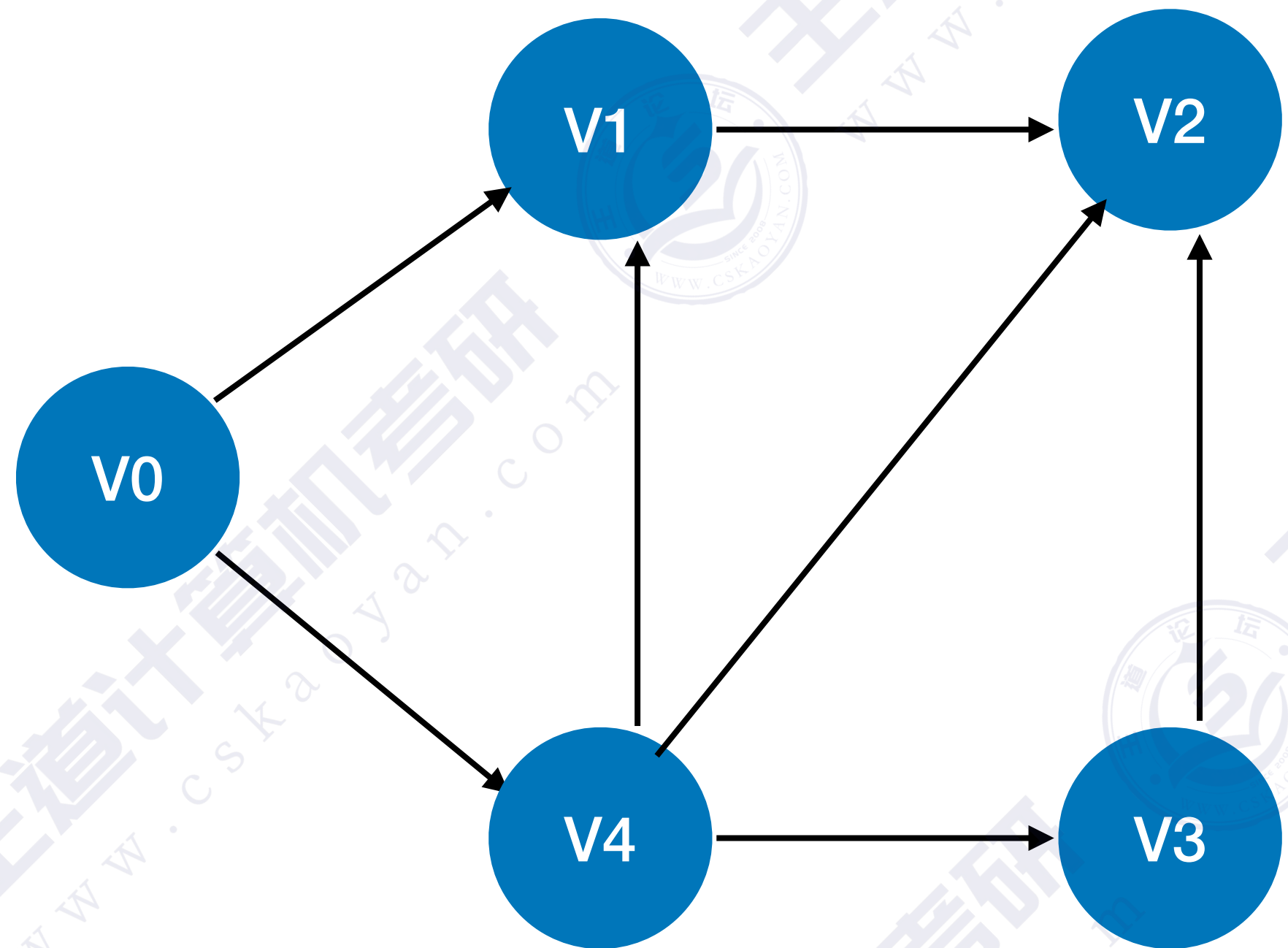
本节内容

有向无环图

描述表达式

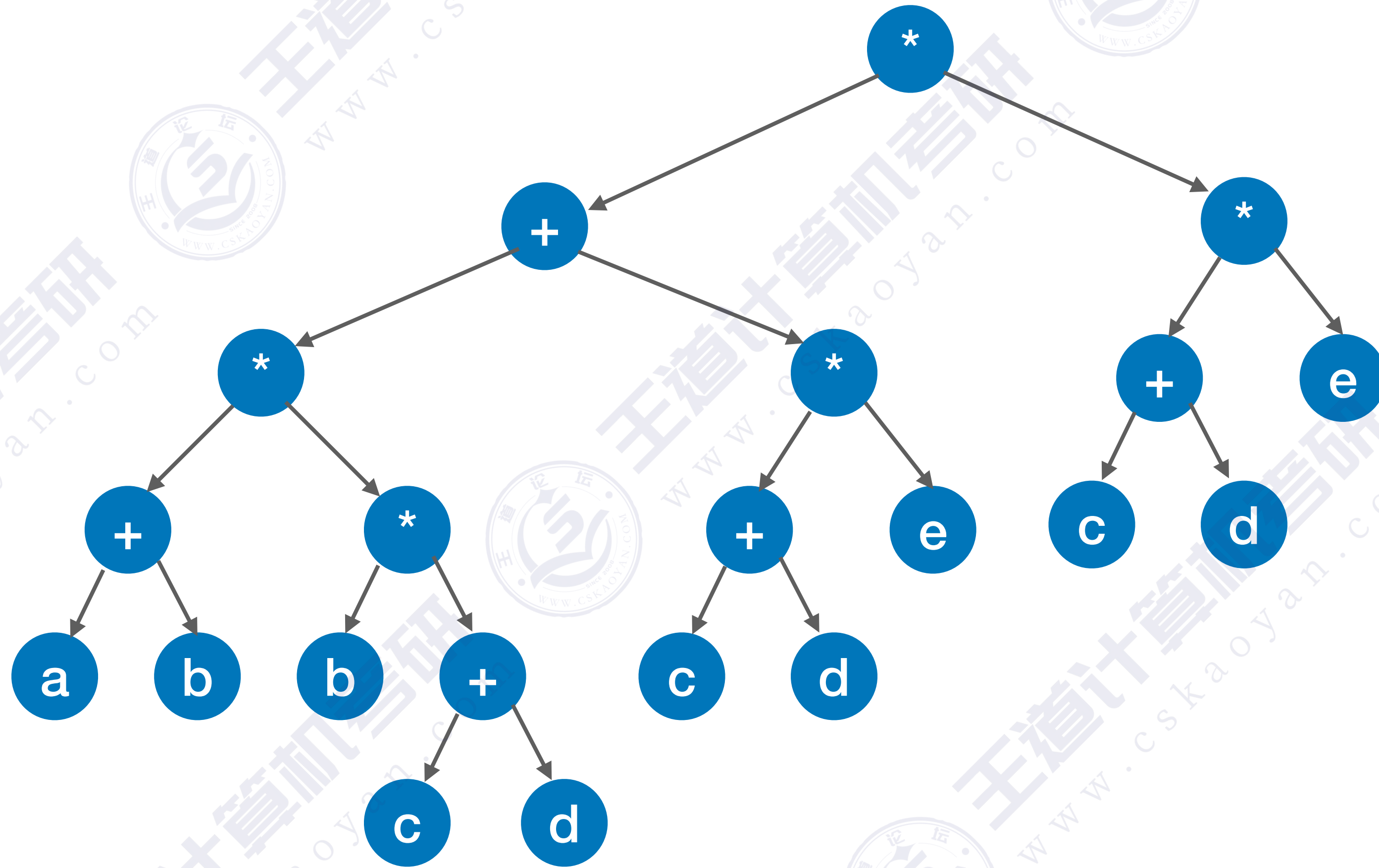
有向无环图 (DAG)

有向无环图：若一个有向图中不存在环，则称为有向无环图，简称DAG图 (Directed Acyclic Graph)



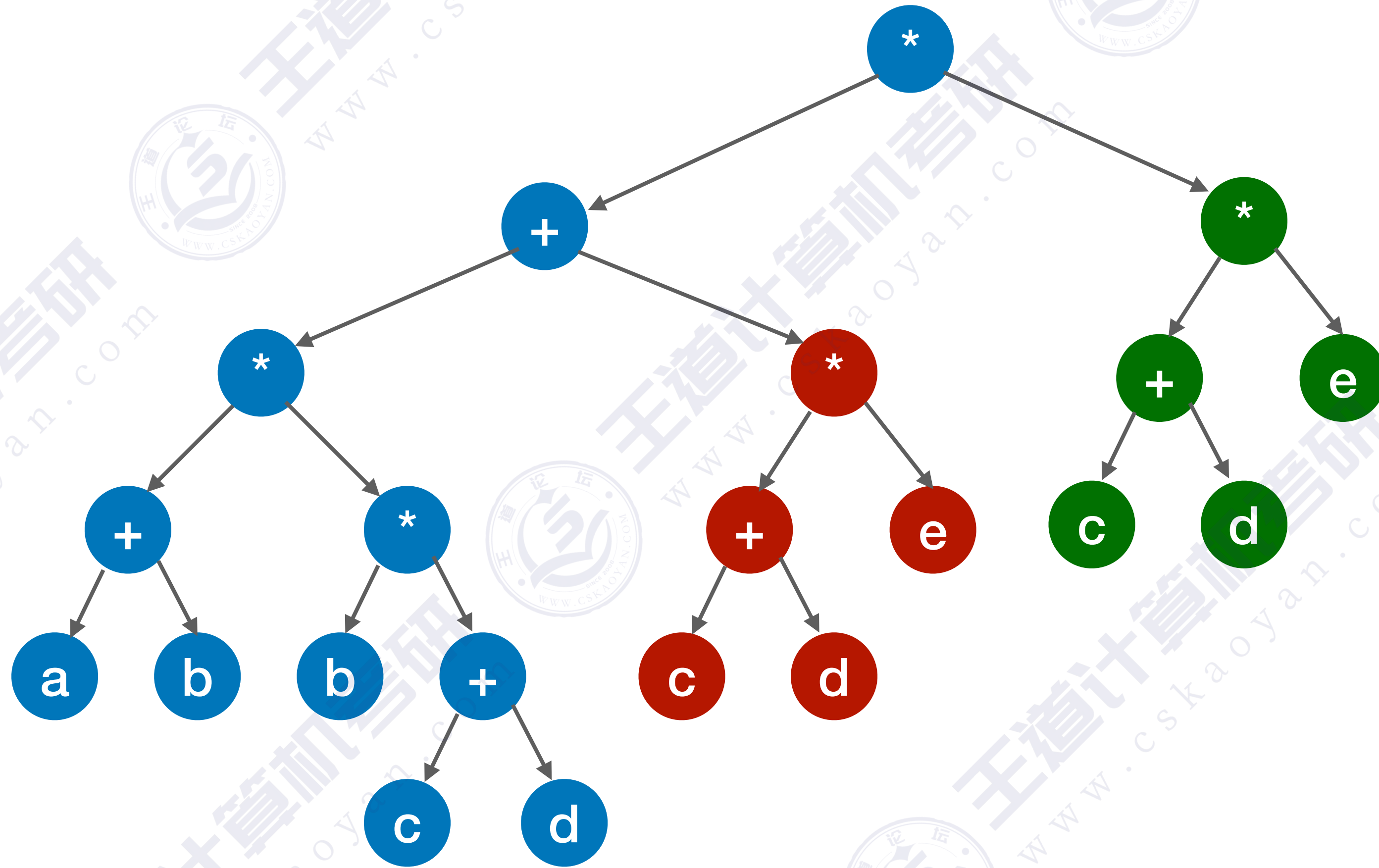
DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



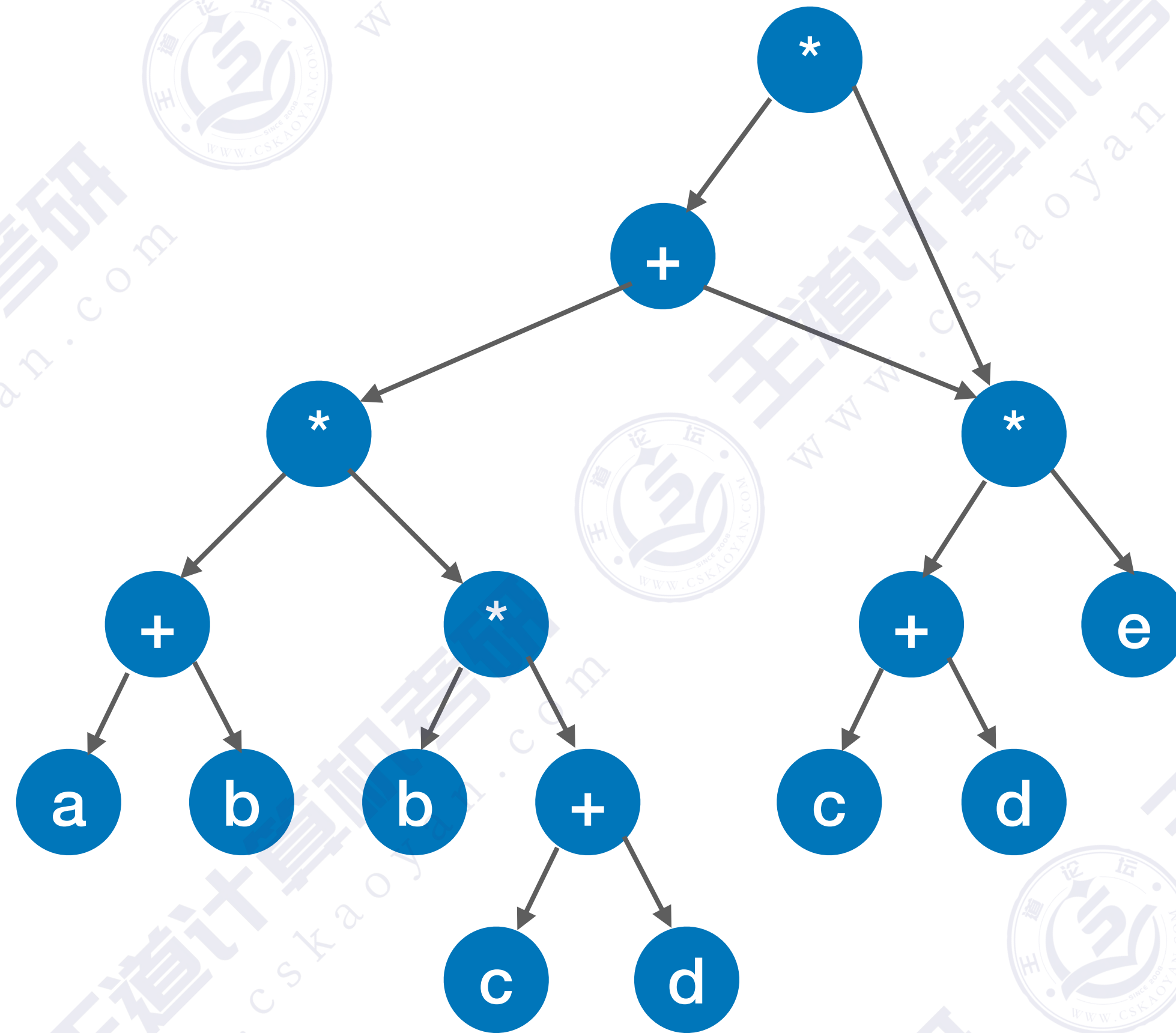
DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



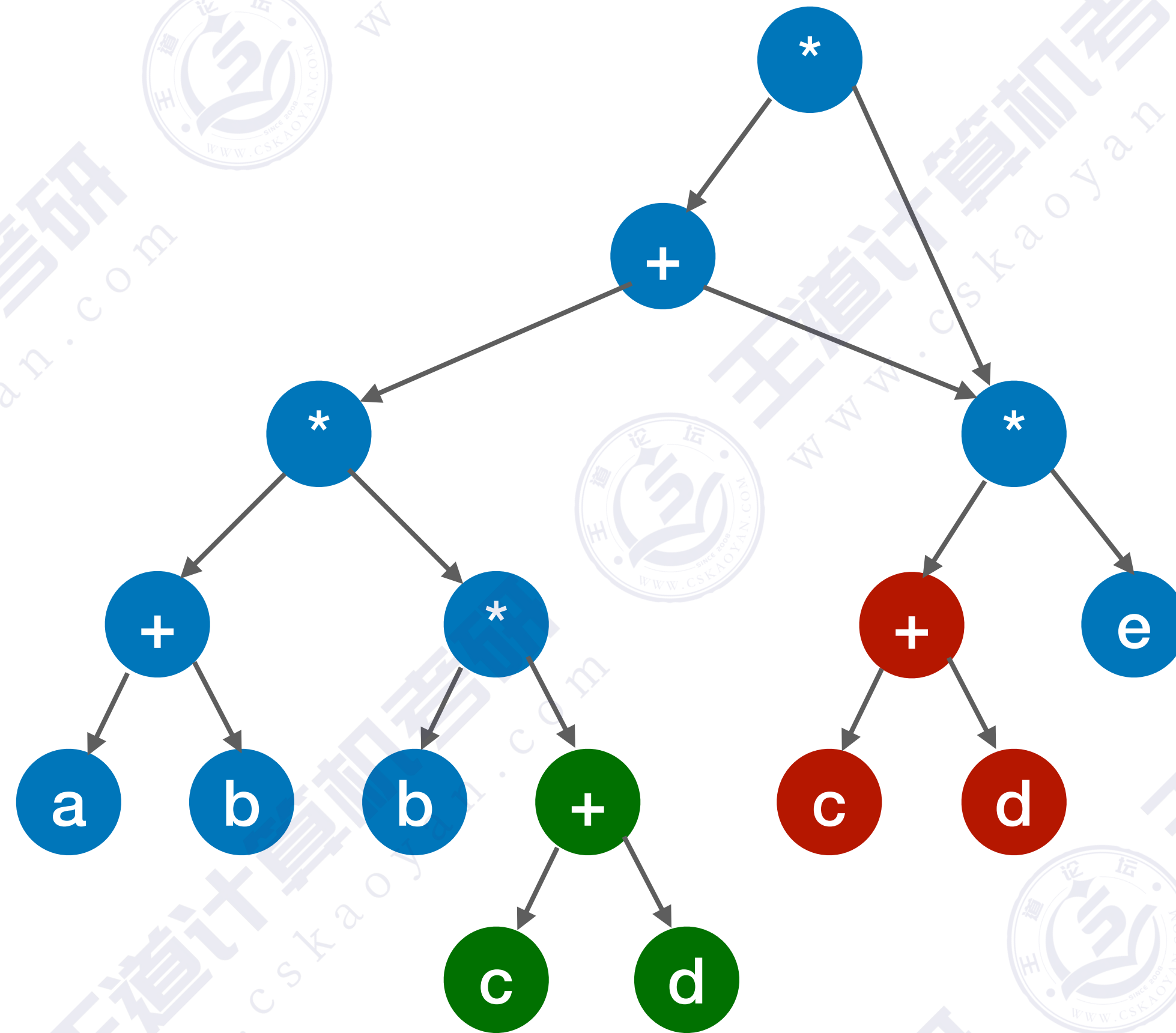
DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

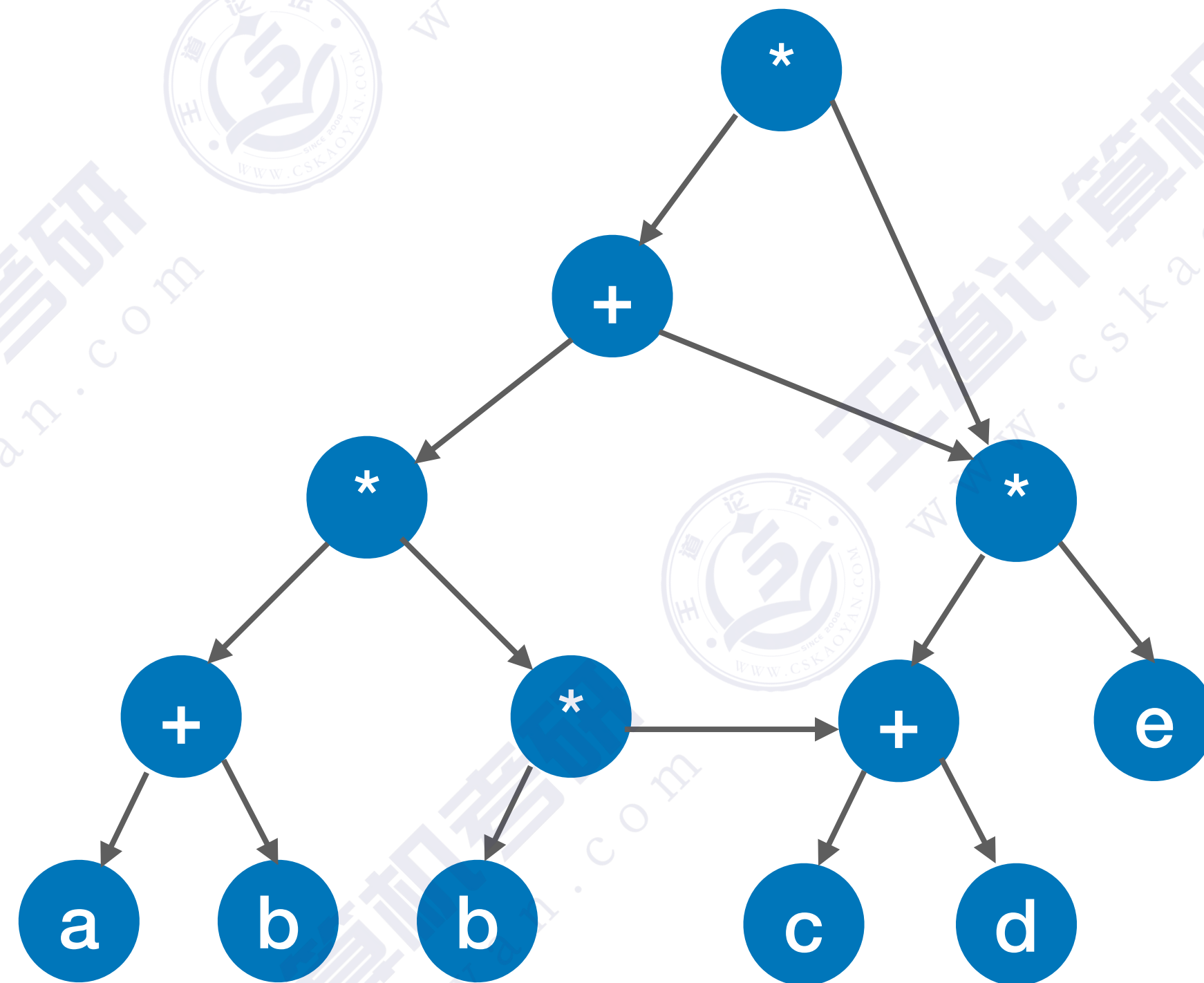


DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

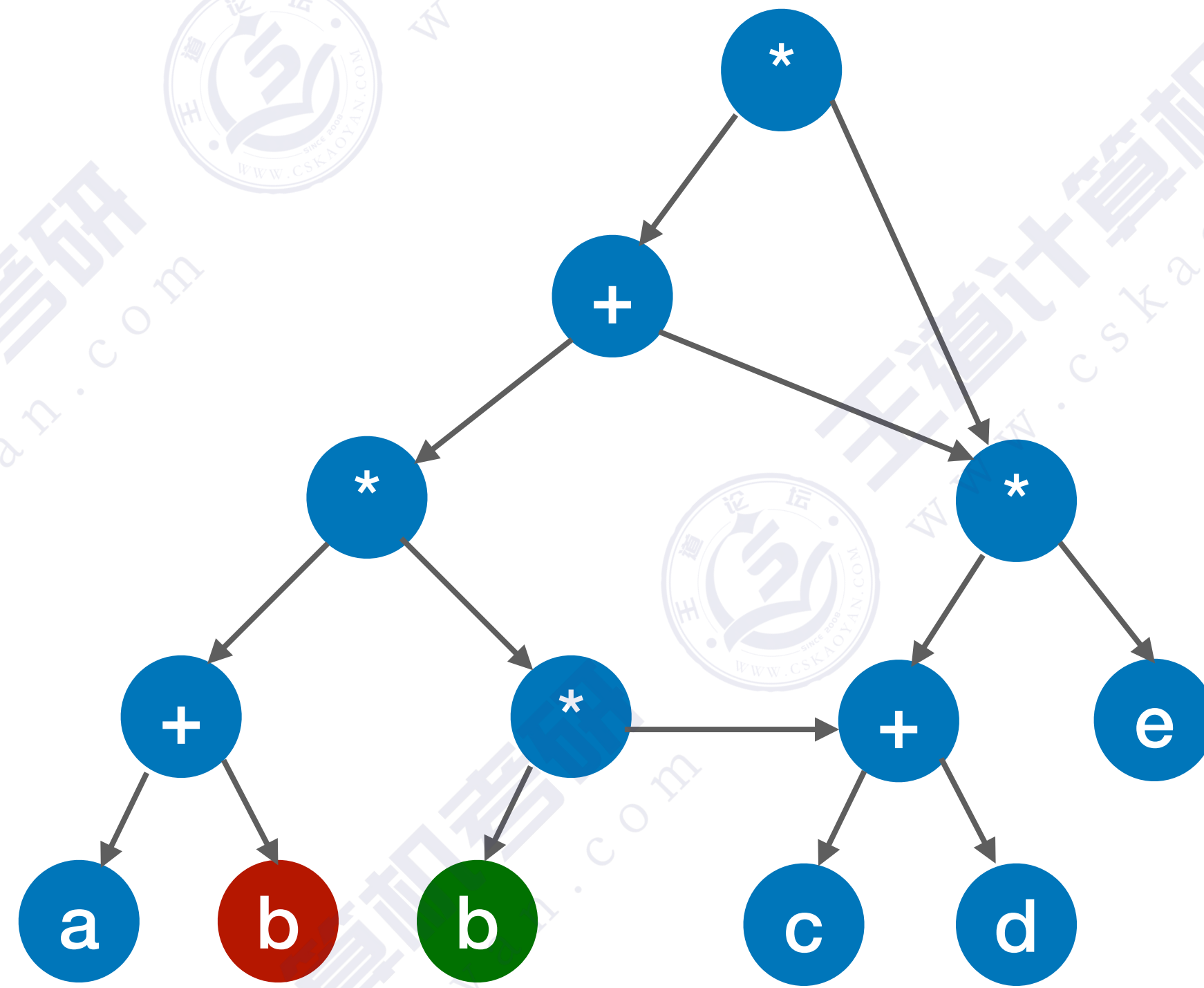


$$((a + b) * (b * (c + d))) + (c + d) * e) * ((c + d) * e)$$



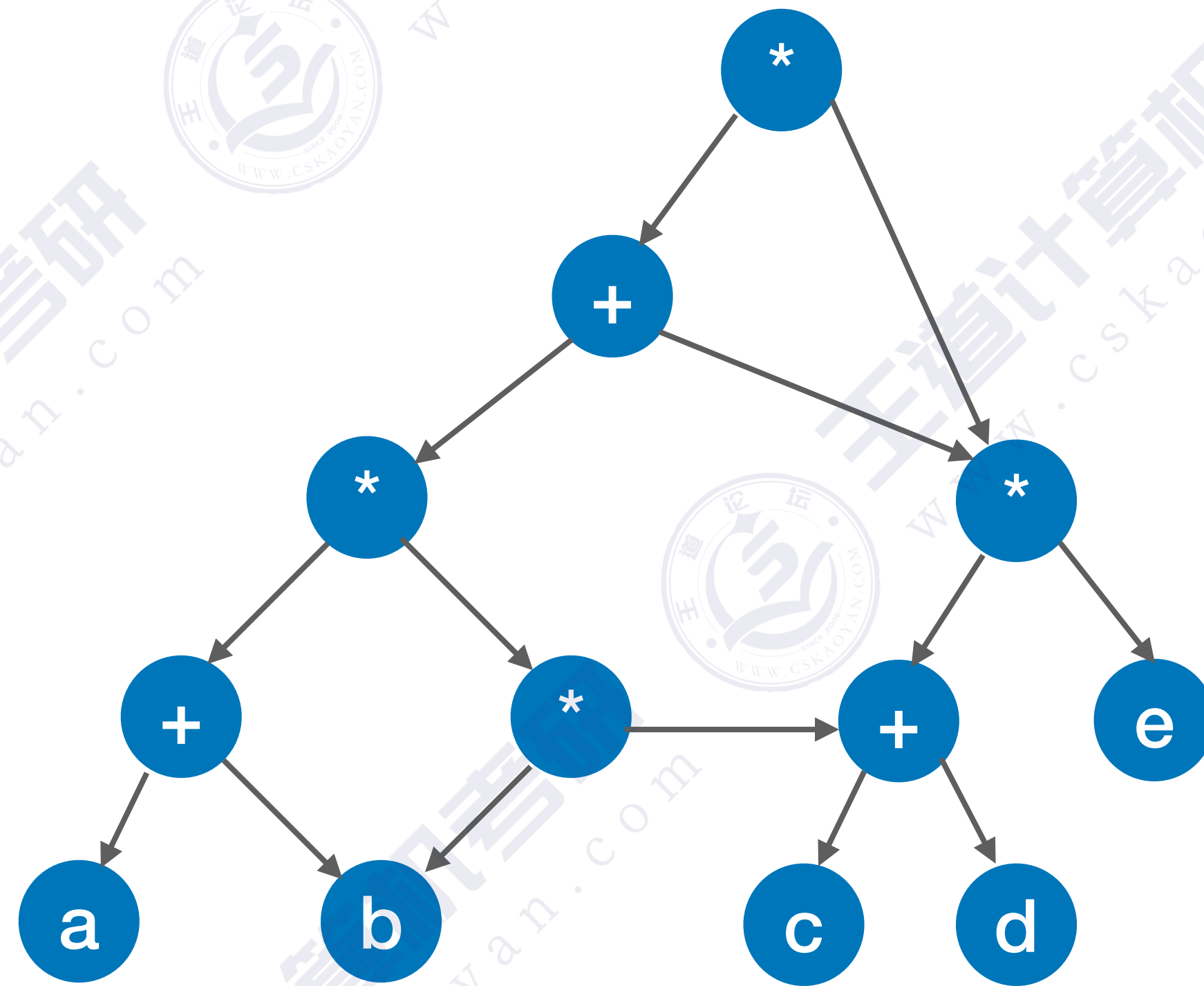
DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



DAG描述表达式

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



DAG描述表达式

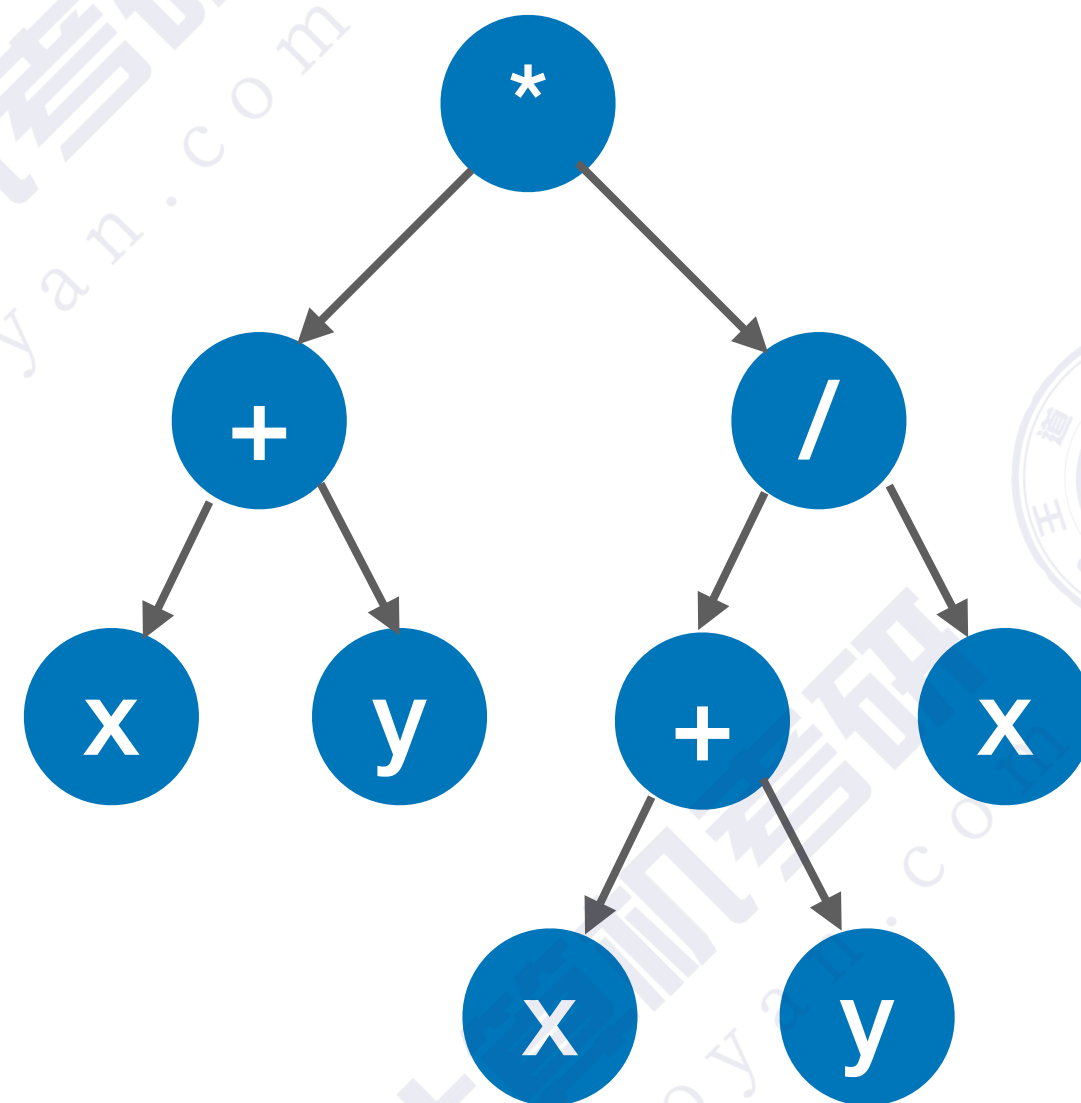
29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

C. 8

D. 9



DAG描述表达式

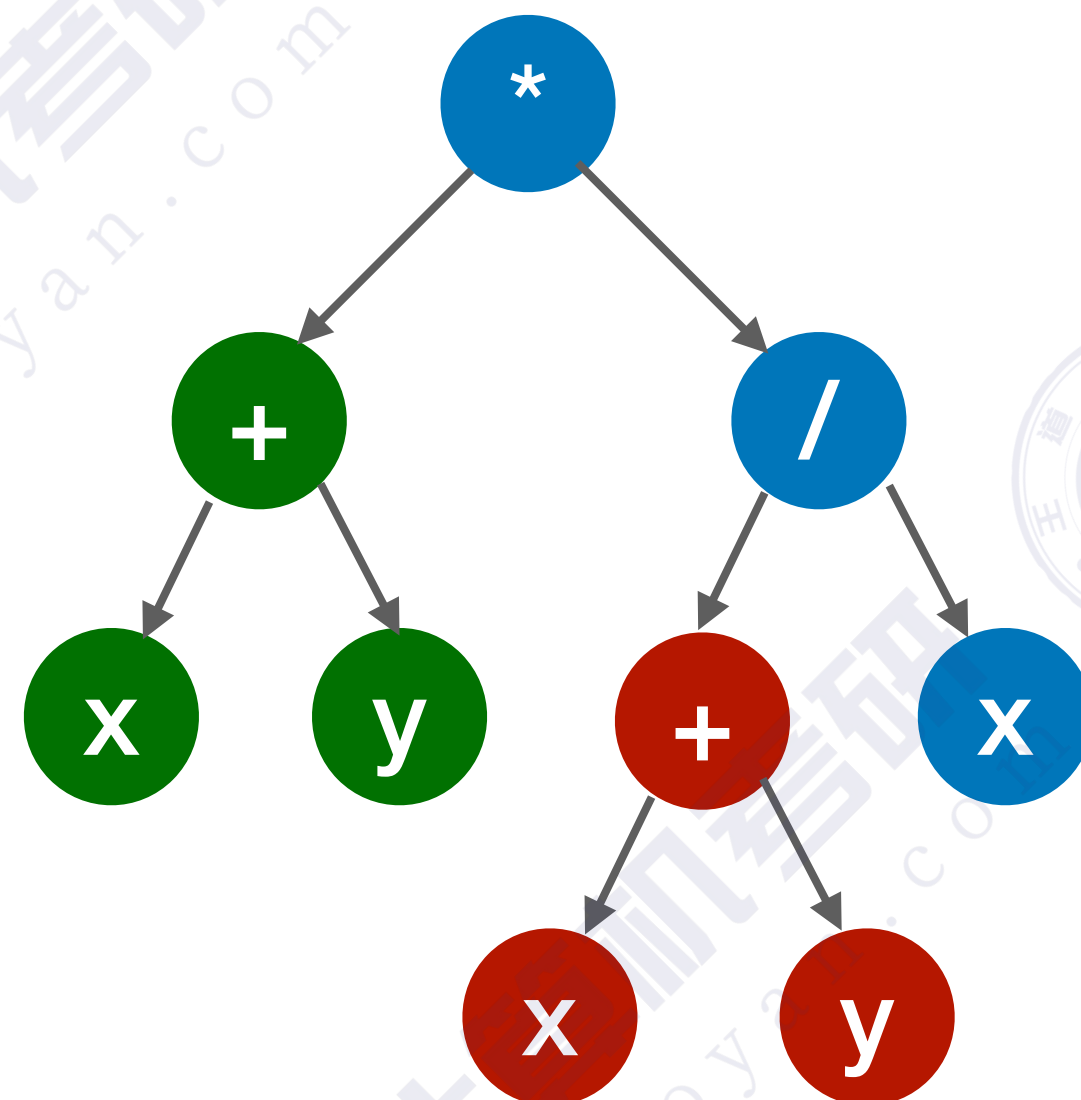
29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

C. 8

D. 9



DAG描述表达式

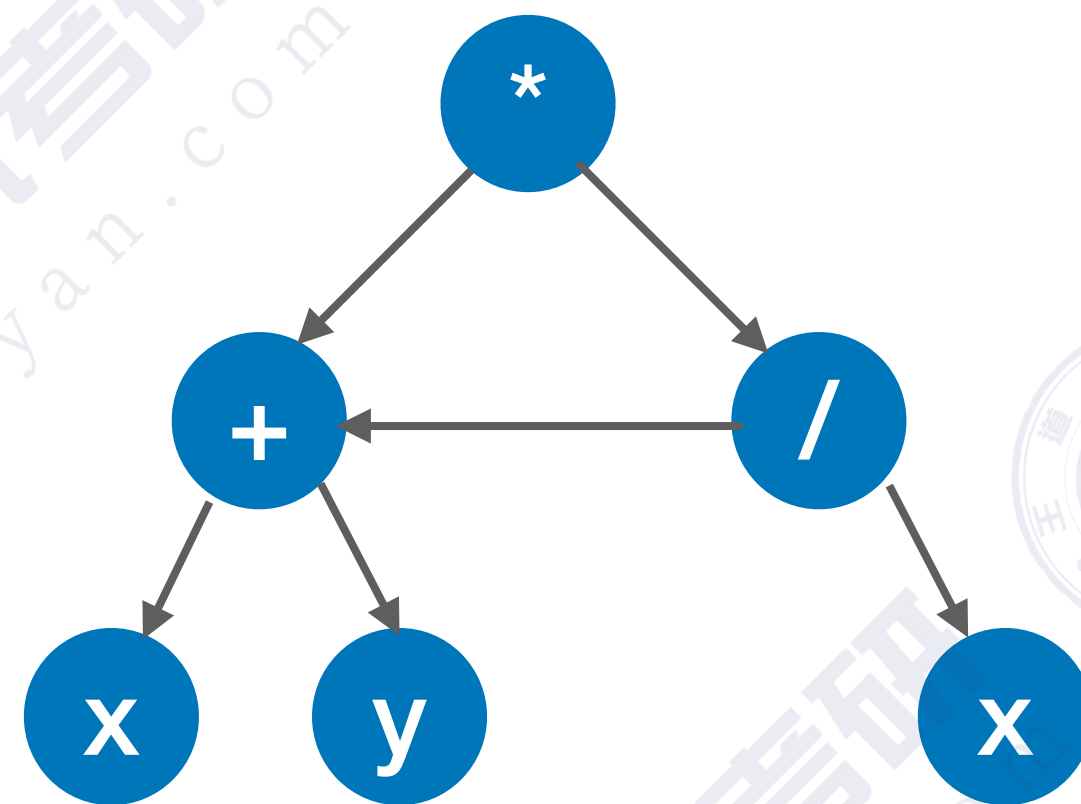
29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

C. 8

D. 9



DAG描述表达式

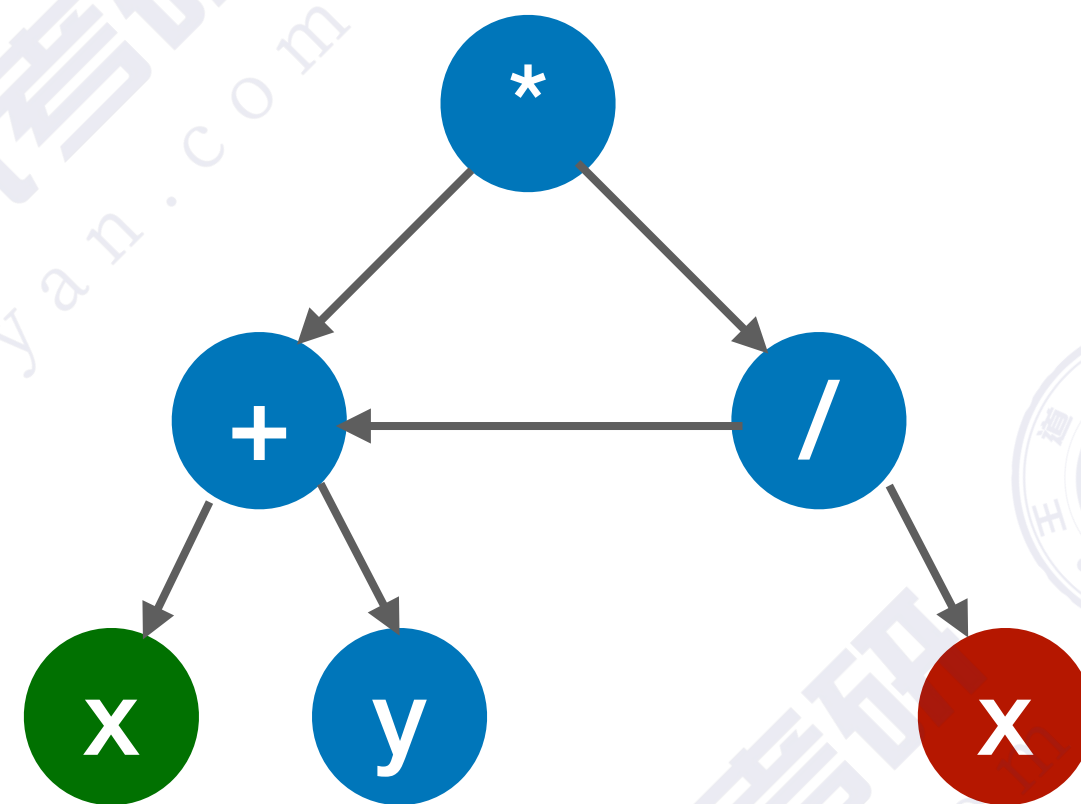
29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

C. 8

D. 9



DAG描述表达式

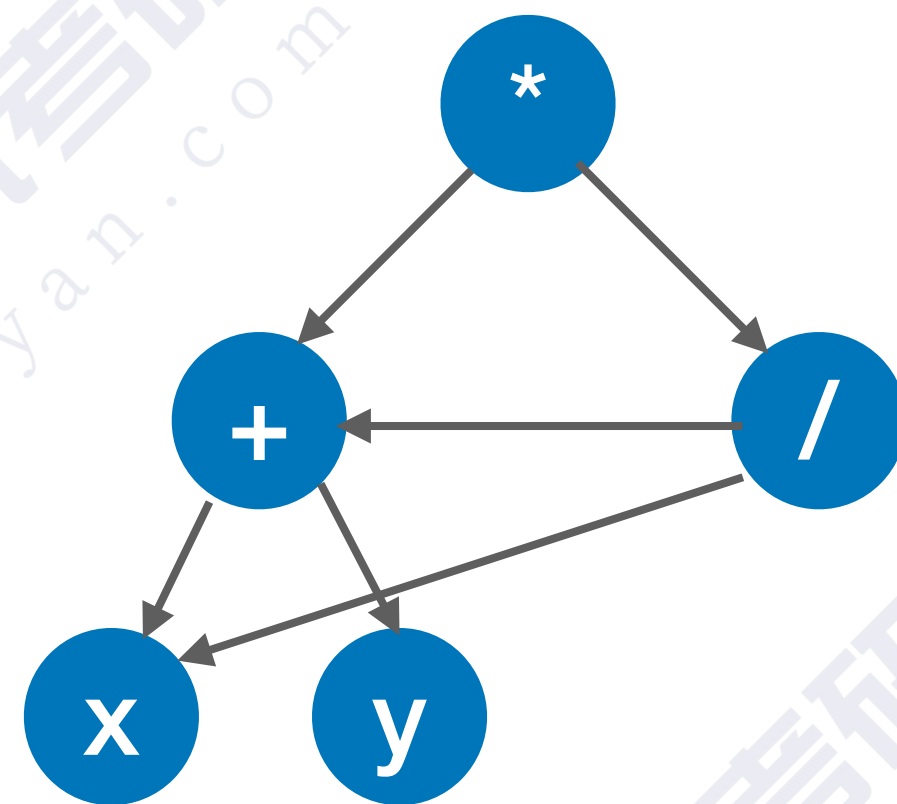
29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

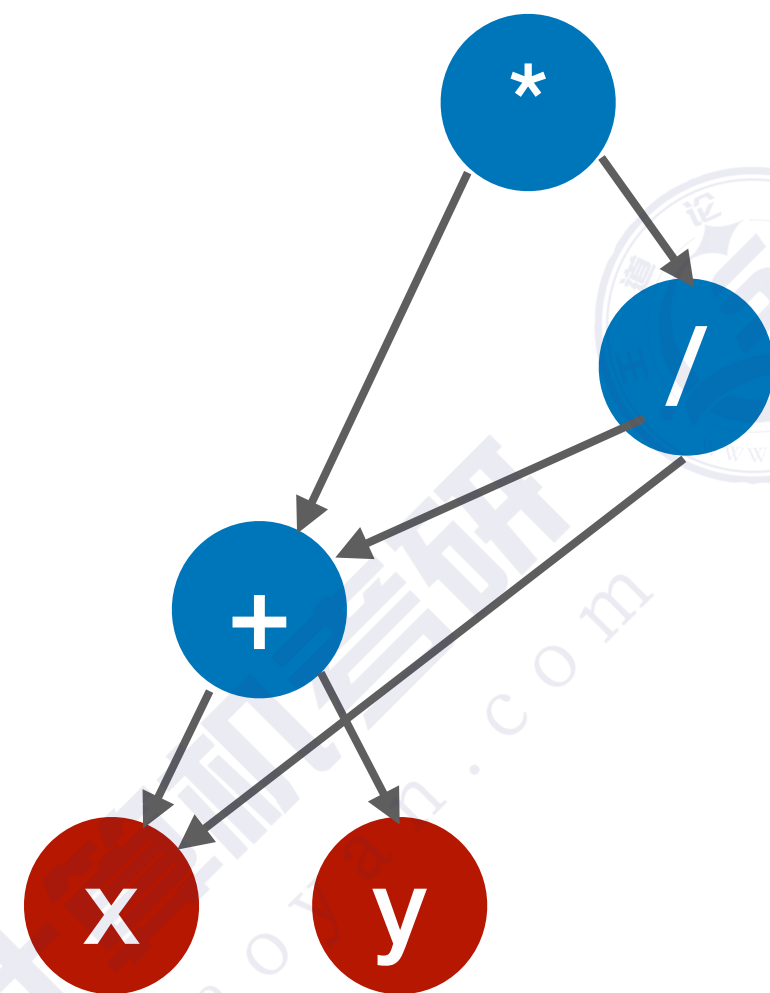
C. 8

D. 9



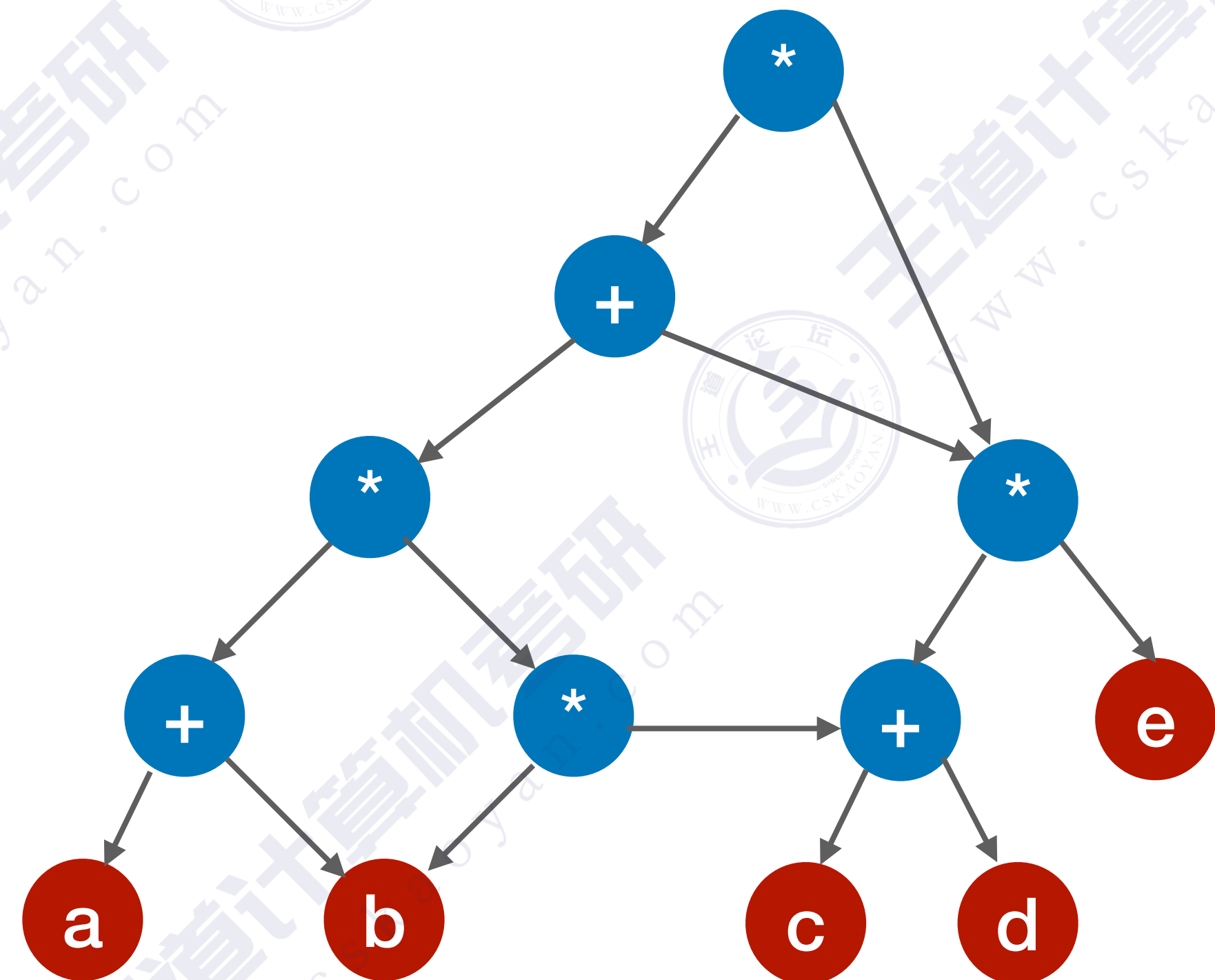
DAG描述表达式

$$(x + y)((x + y) / x)$$



顶点中不可能出现重复的操作数

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

Step 1: 把各个操作数不重复地排成一排

解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

Step 1: 把各个操作数不重复地排成一排

a

b

c

d

e

解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

①

④

③

②

⑦

⑤

⑥

⑩

⑧

⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

a

b

c

d

e

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”

b

d

e

解题方法

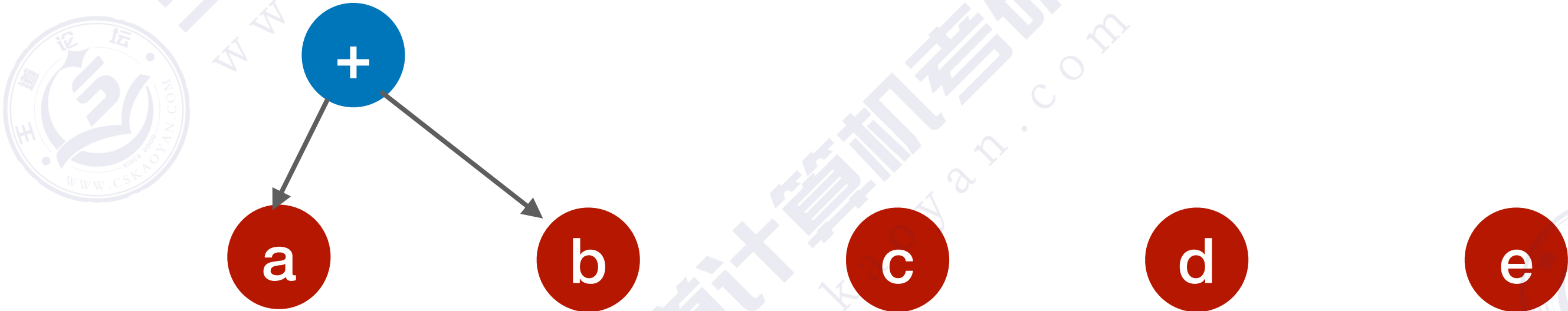
$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”



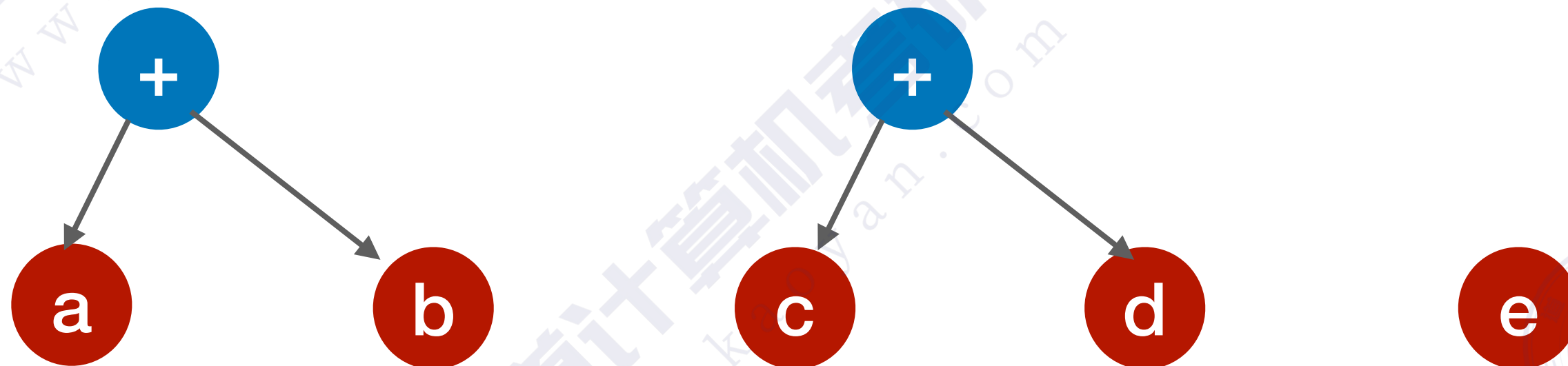
解题方法

$$\begin{array}{cccccccccc} ((a+b) * (b * (c+d)) + (c+d) * e) * ((c+d) * e) \\ \textcircled{1} \quad \textcircled{4} \quad \textcircled{3} \quad \textcircled{2} \quad \textcircled{7} \quad \textcircled{5} \quad \textcircled{6} \quad \textcircled{10} \quad \textcircled{8} \quad \textcircled{9} \end{array}$$

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”



解题方法

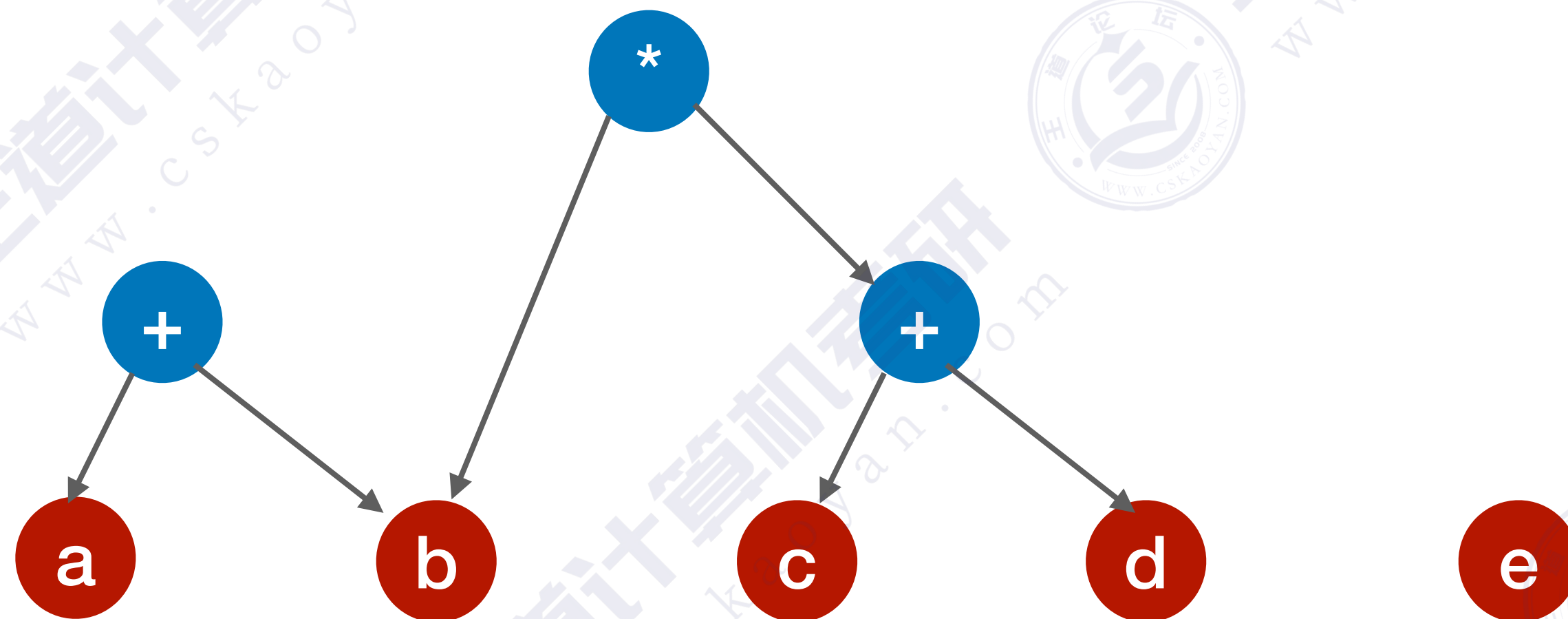
$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”



解题方法

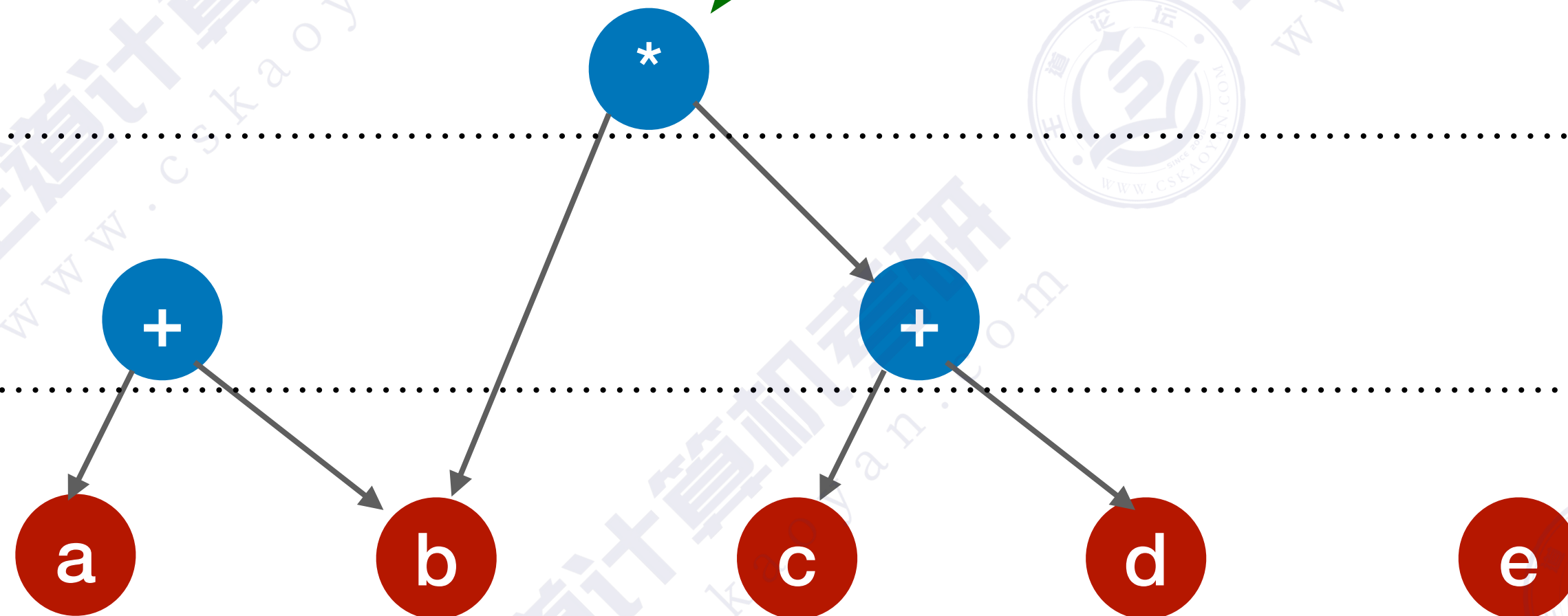
$$\begin{array}{cccccccccc} ((a+b) * (b * (c+d)) + (c+d) * e) * ((c+d) * e) \\ \textcircled{1} \quad \textcircled{4} \quad \textcircled{3} \quad \textcircled{2} \quad \textcircled{7} \quad \textcircled{5} \quad \textcircled{6} \quad \textcircled{10} \quad \textcircled{8} \quad \textcircled{9} \end{array}$$

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

这个运算要基于下面一层运算的结果来进行



解题方法

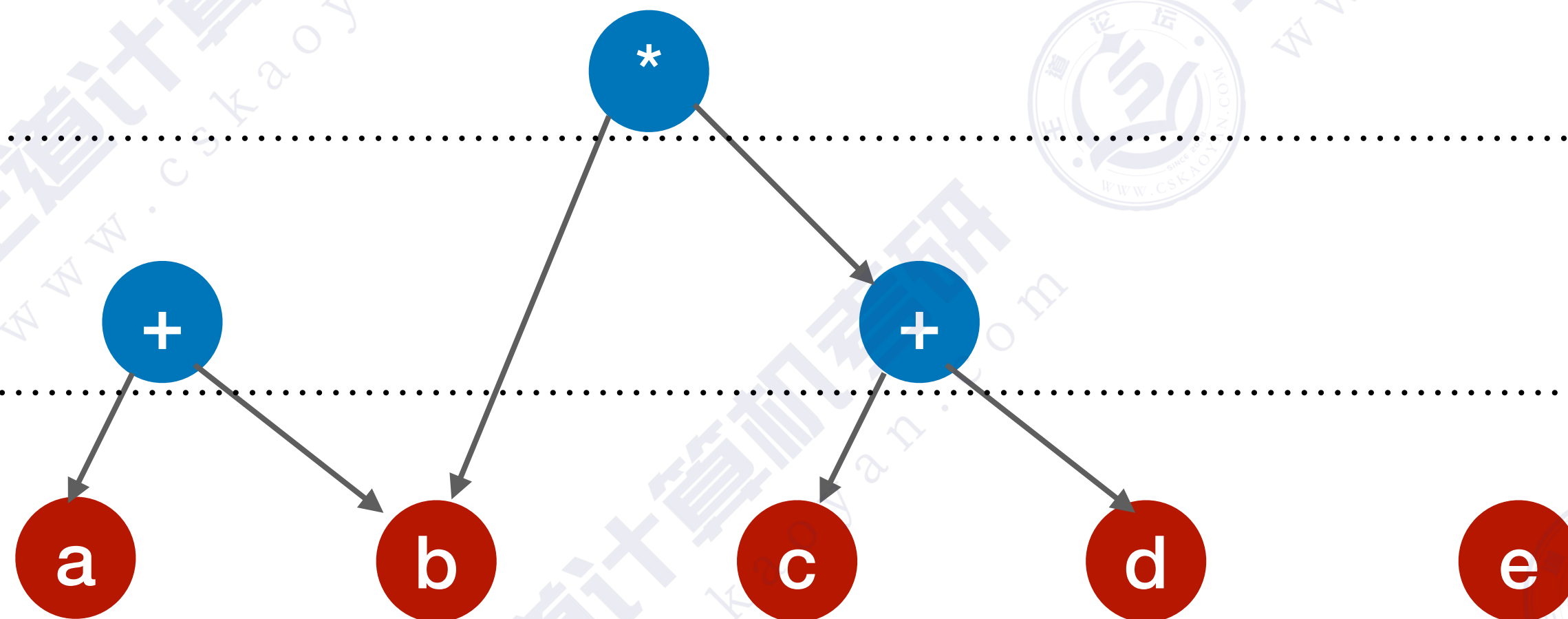
$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”



解题方法

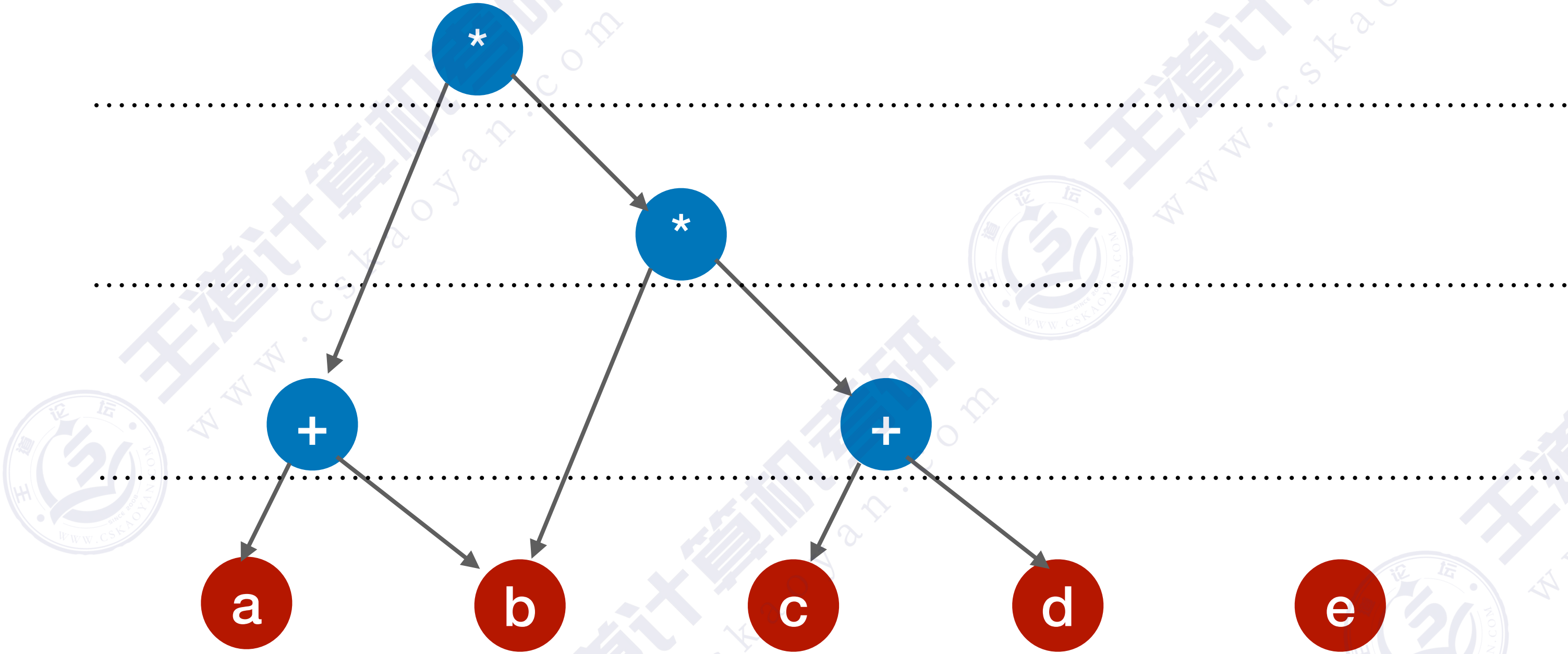
$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”



解题方法

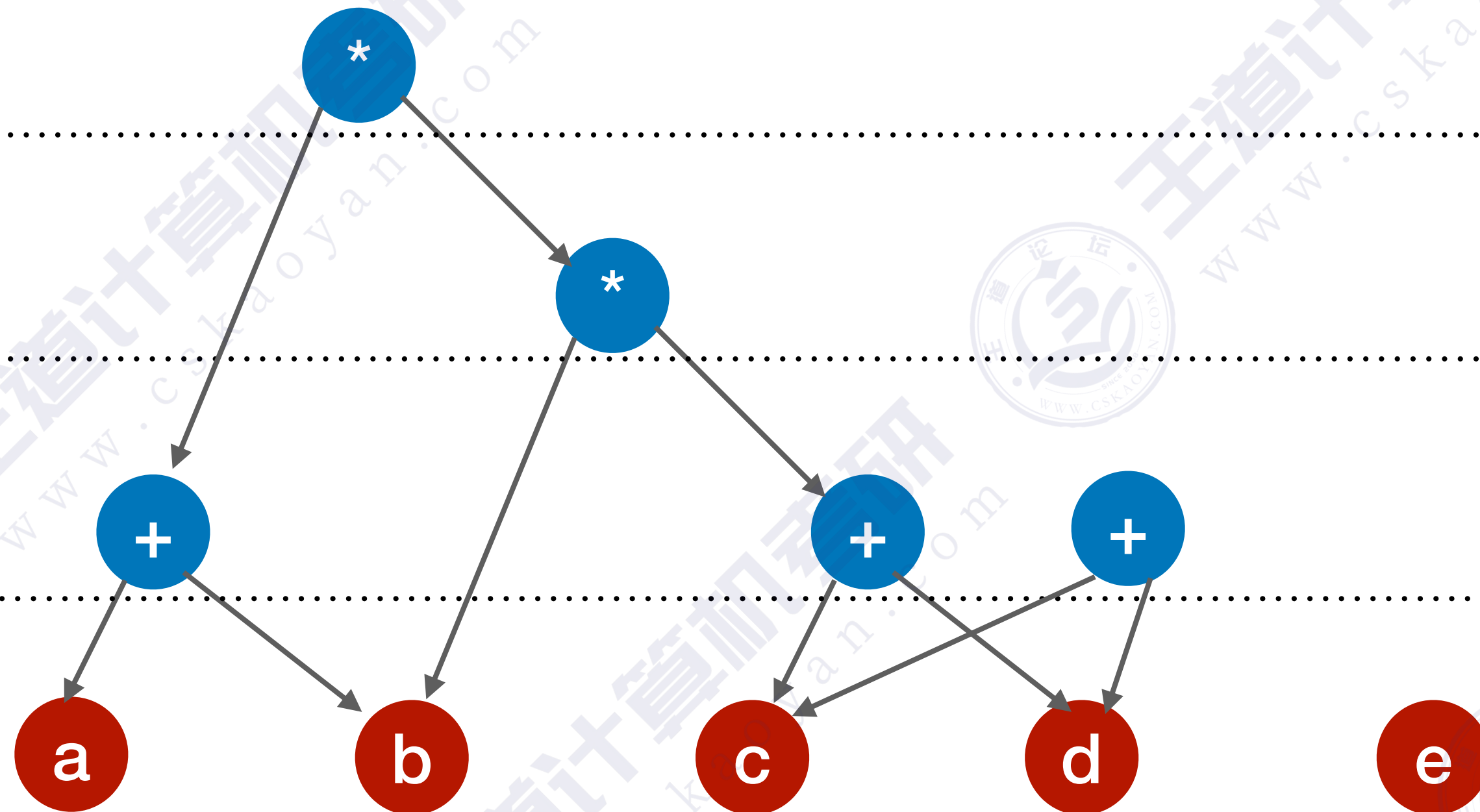
$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”



① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



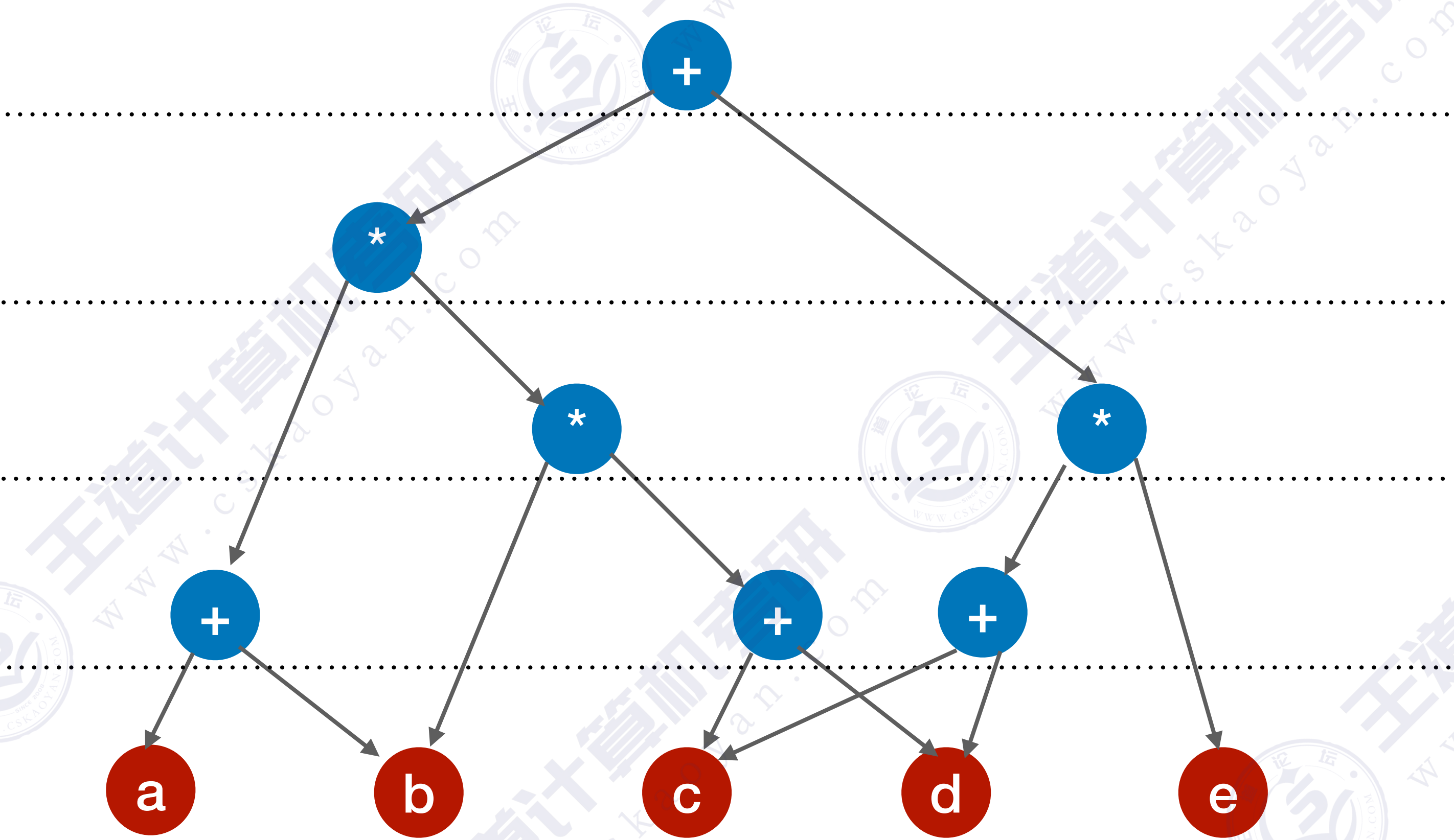
Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”

解题方法

$((a + b) * (b * (c + d))) + (c + d) * e) * ((c + d) * e)$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 3: 按顺序加入运算符，注意“分层”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 3: 按顺序加入运算符，注意“分层”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 3: 按顺序加入运算符，注意“分层”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

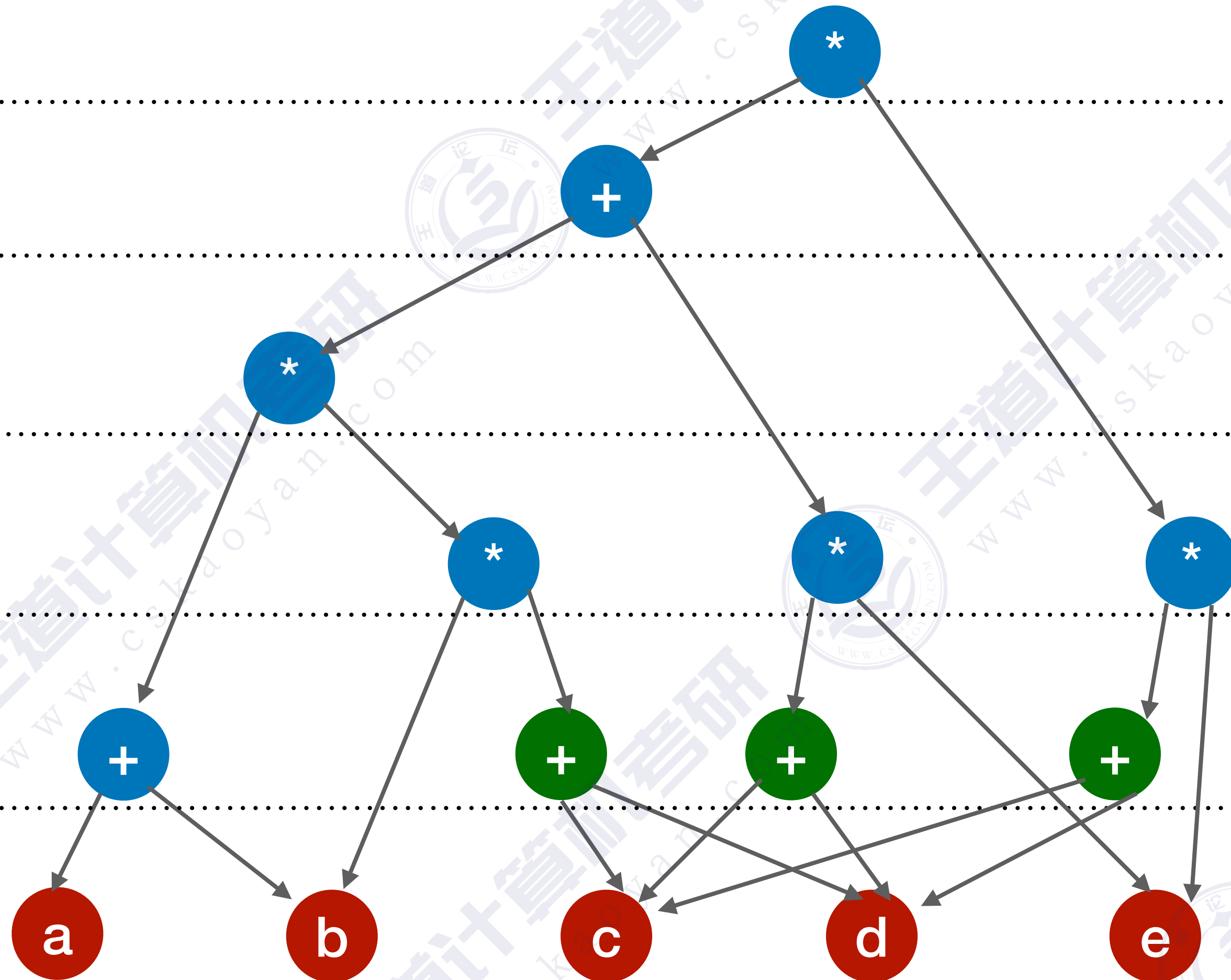
Step 3: 按顺序加入运算符，注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

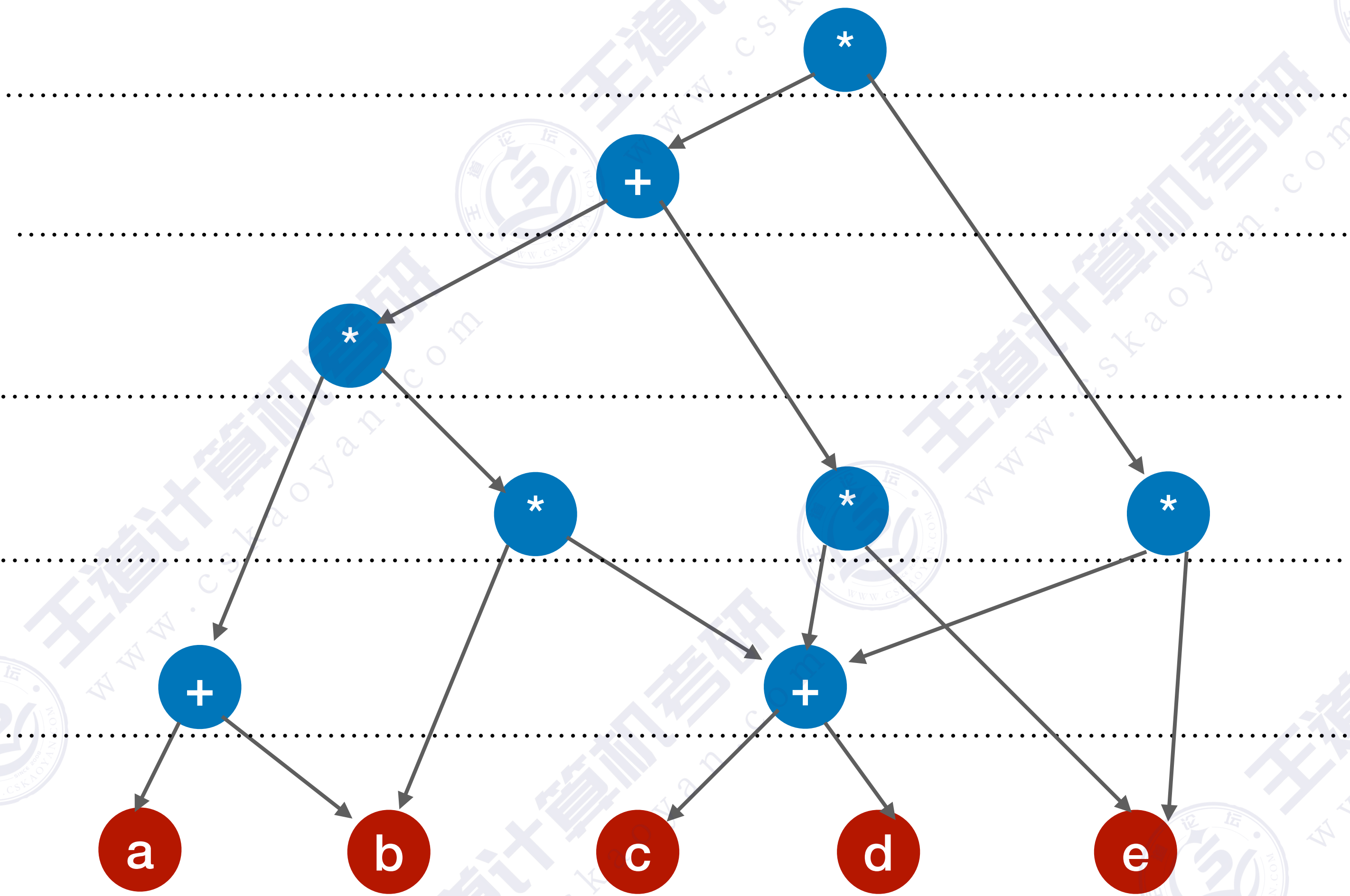
Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

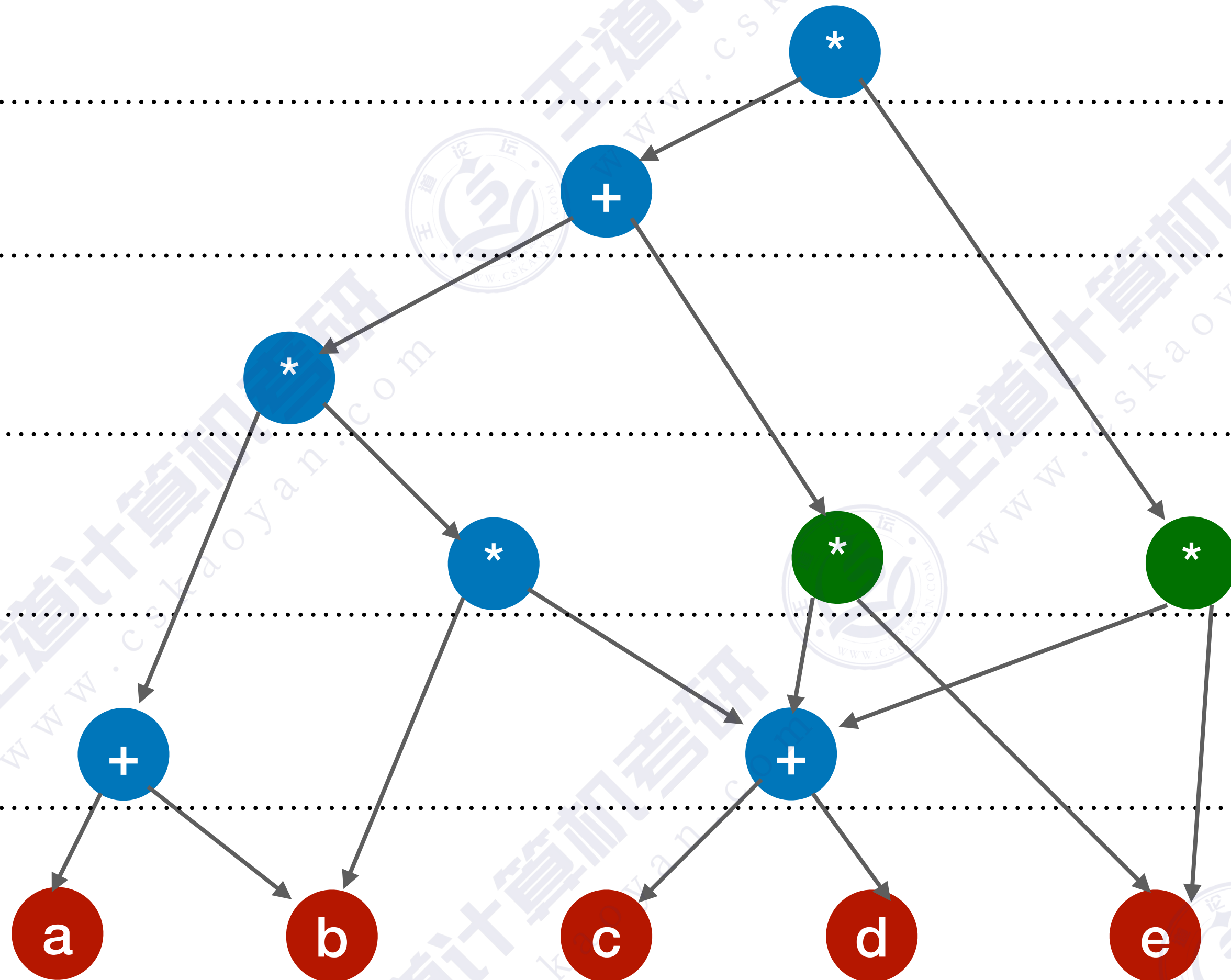
Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

解题方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

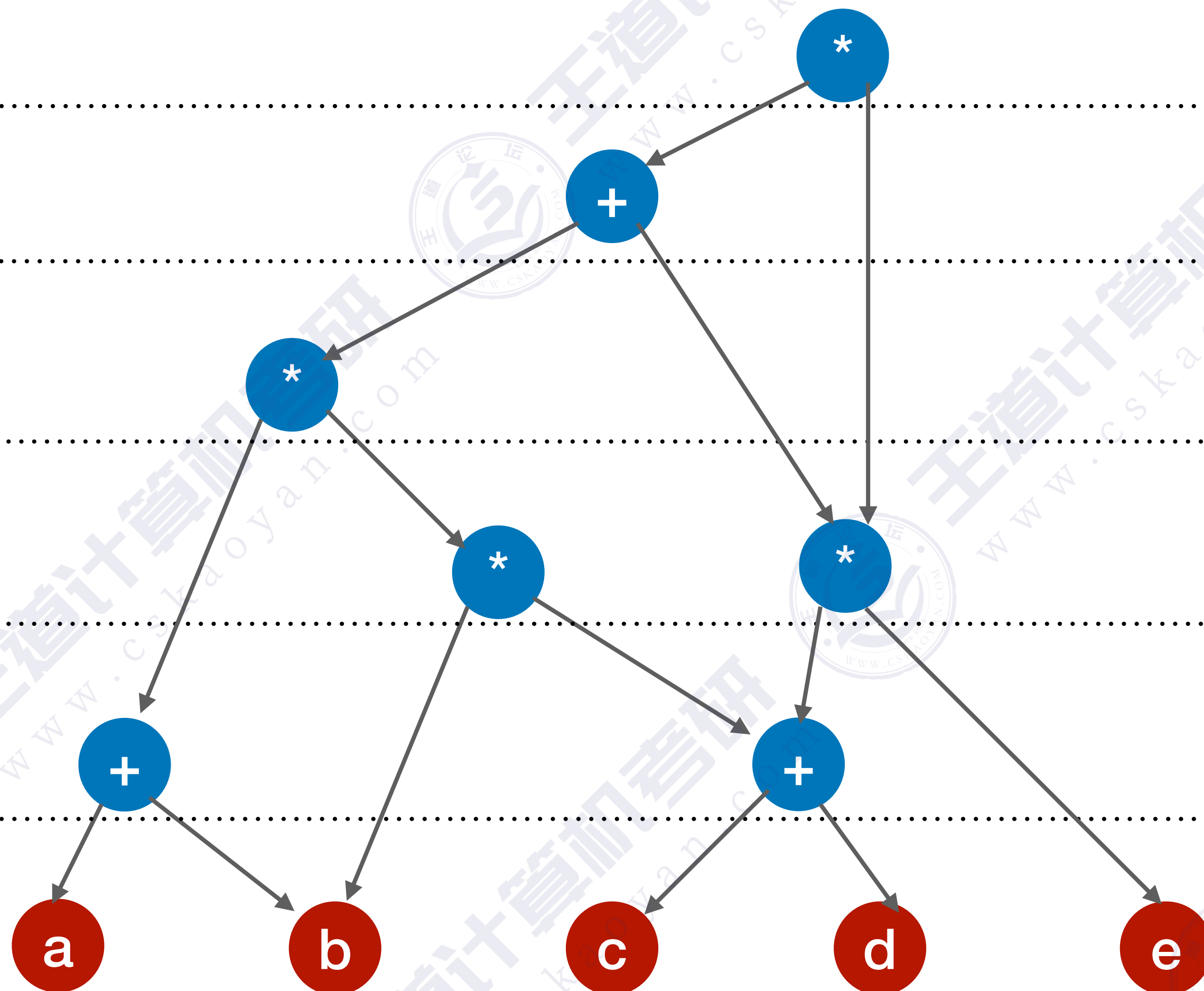
Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

解題方法

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$

① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨



Step 1: 把各个操作数不重复地排成一排

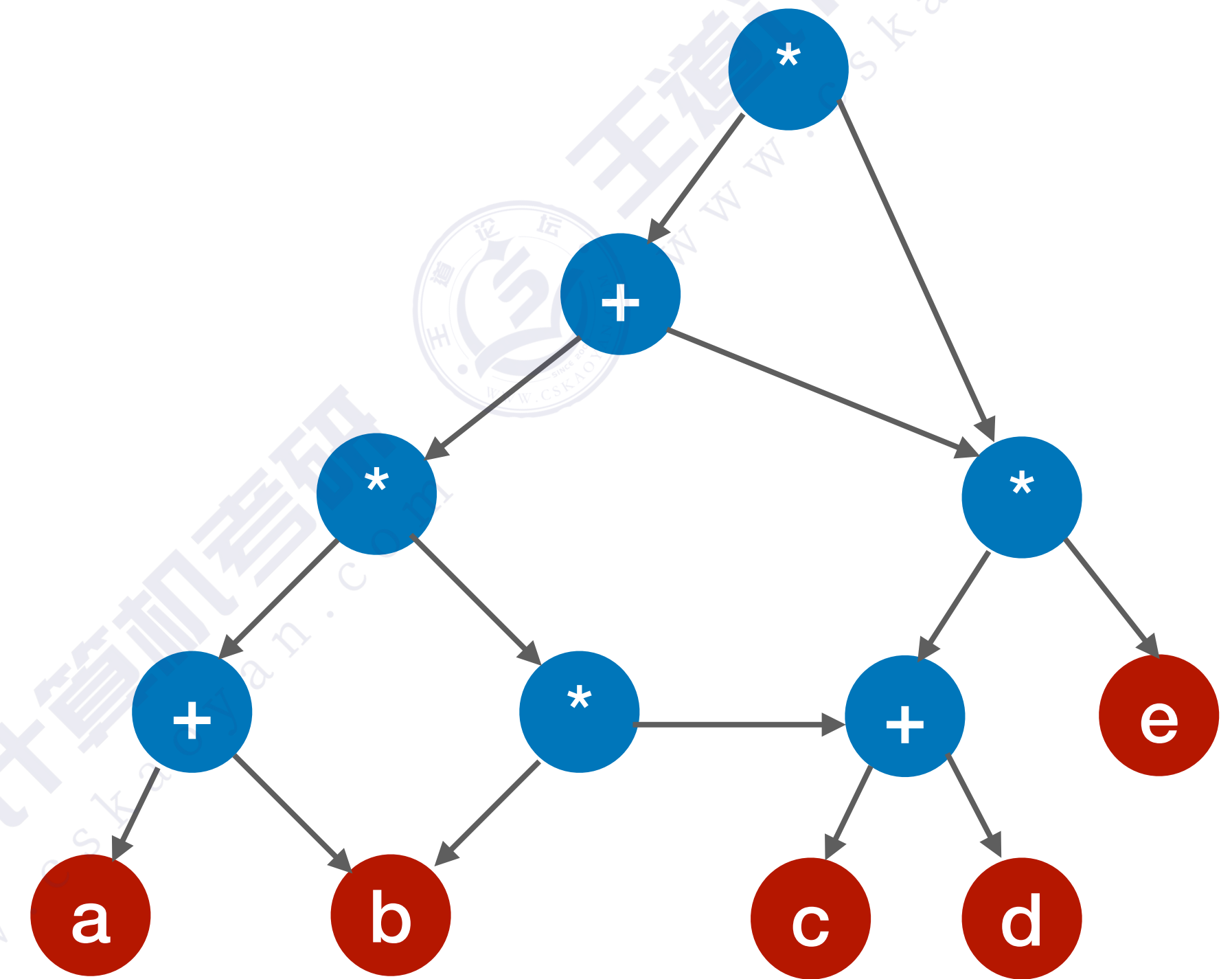
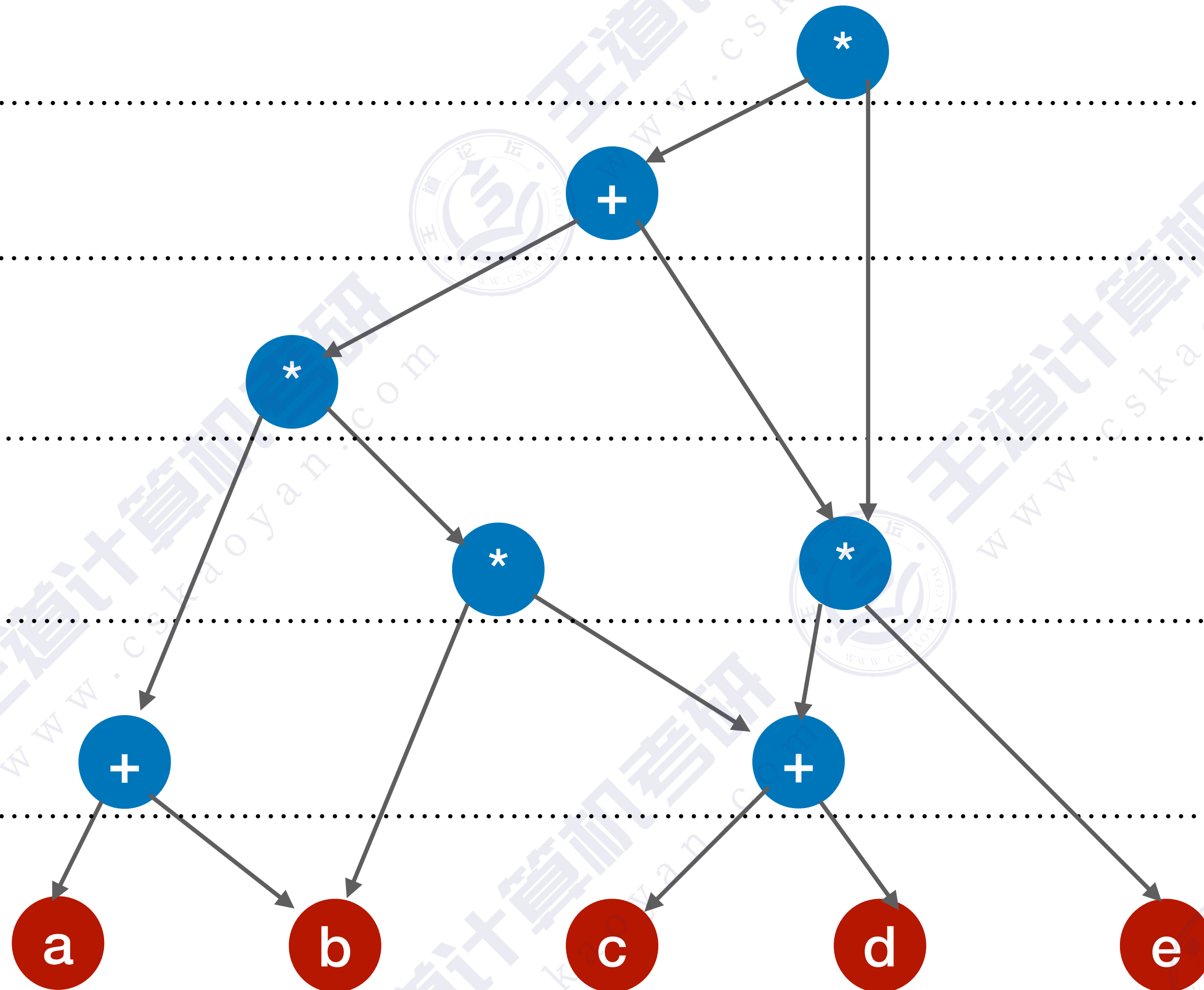
Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“分层”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

对比一哈子

$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$



练习

$$(a * b) * (a * b) * (a * b) * c$$

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

练习

$$(a * b) * (a * b) * (a * b) * c$$

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

a

b

c

练习

$$\underset{\textcircled{1}}{(a * b)} \underset{\textcircled{4}}{*} \underset{\textcircled{2}}{(a * b)} \underset{\textcircled{5}}{*} \underset{\textcircled{3}}{(a * b)} \underset{\textcircled{6}}{*} c$$

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

a

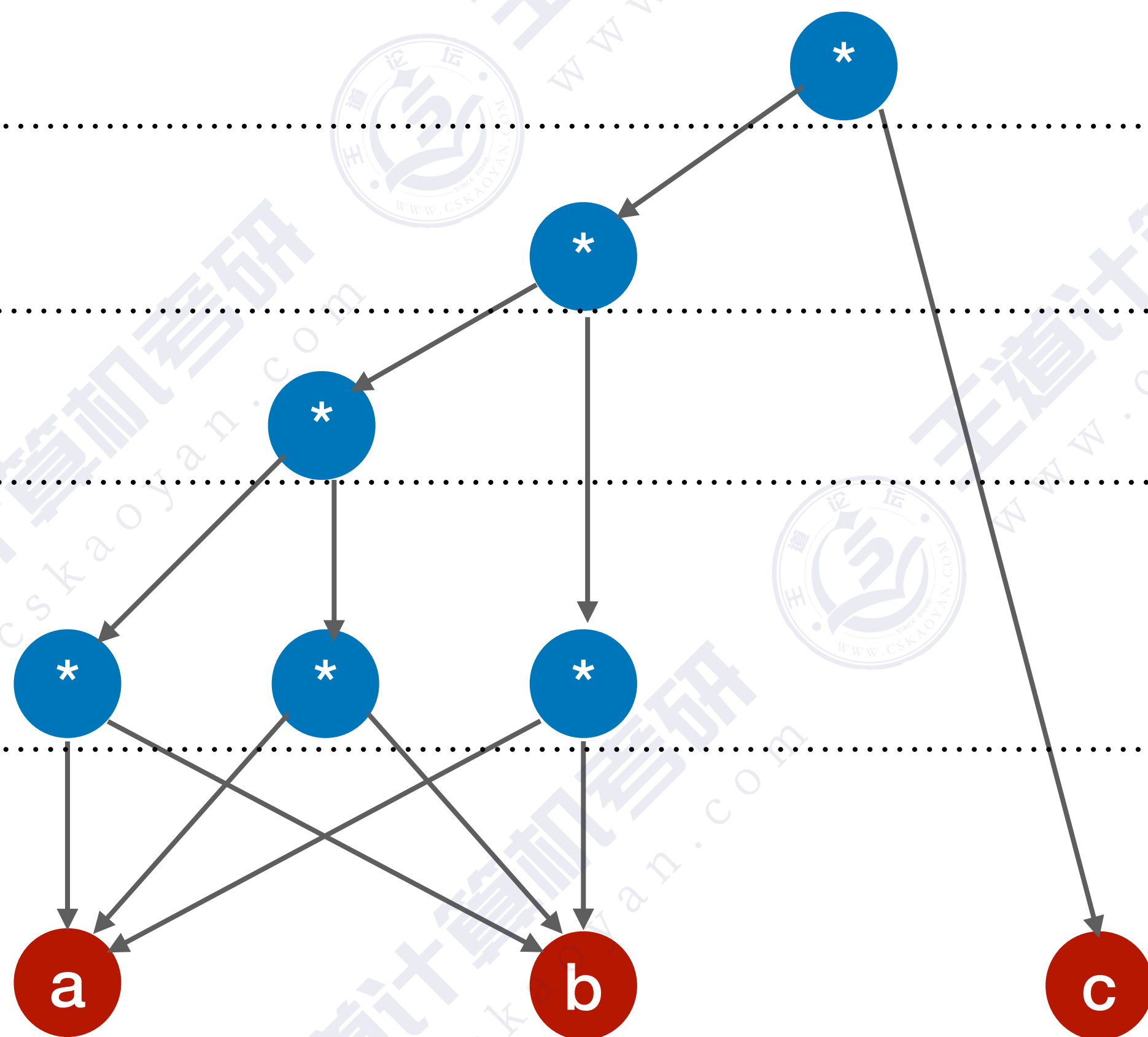
b

c

练习

$$(a * b) * (a * b) * (a * b) * c$$

① ④ ② ⑤ ③ ⑥



Step 1: 把各个操作数不重复地排成一排

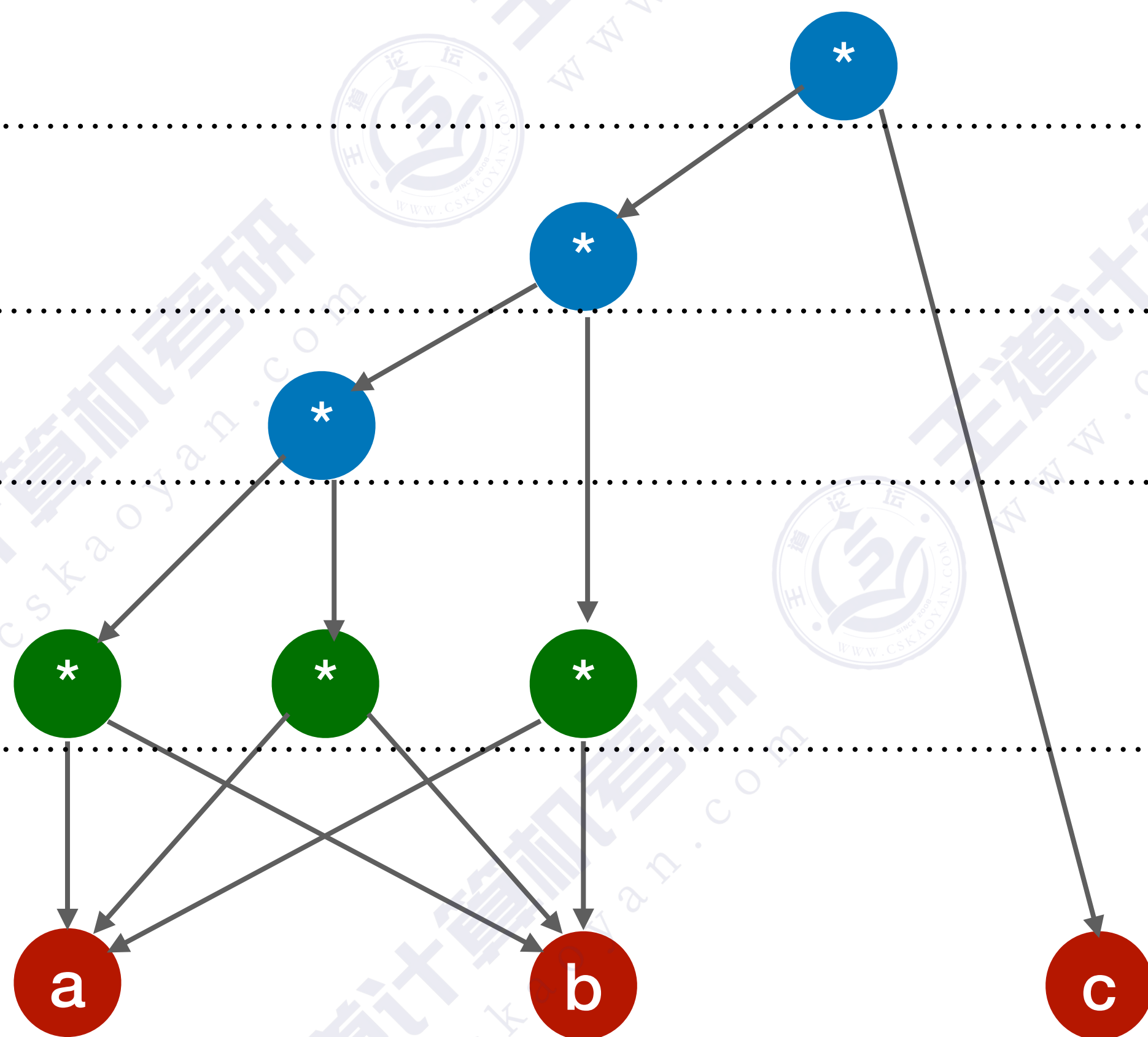
Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

练习

$$\underset{\textcircled{1}}{(a * b)} \underset{\textcircled{4}}{*} \underset{\textcircled{2}}{(a * b)} \underset{\textcircled{5}}{*} \underset{\textcircled{3}}{(a * b)} \underset{\textcircled{6}}{*} c$$



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

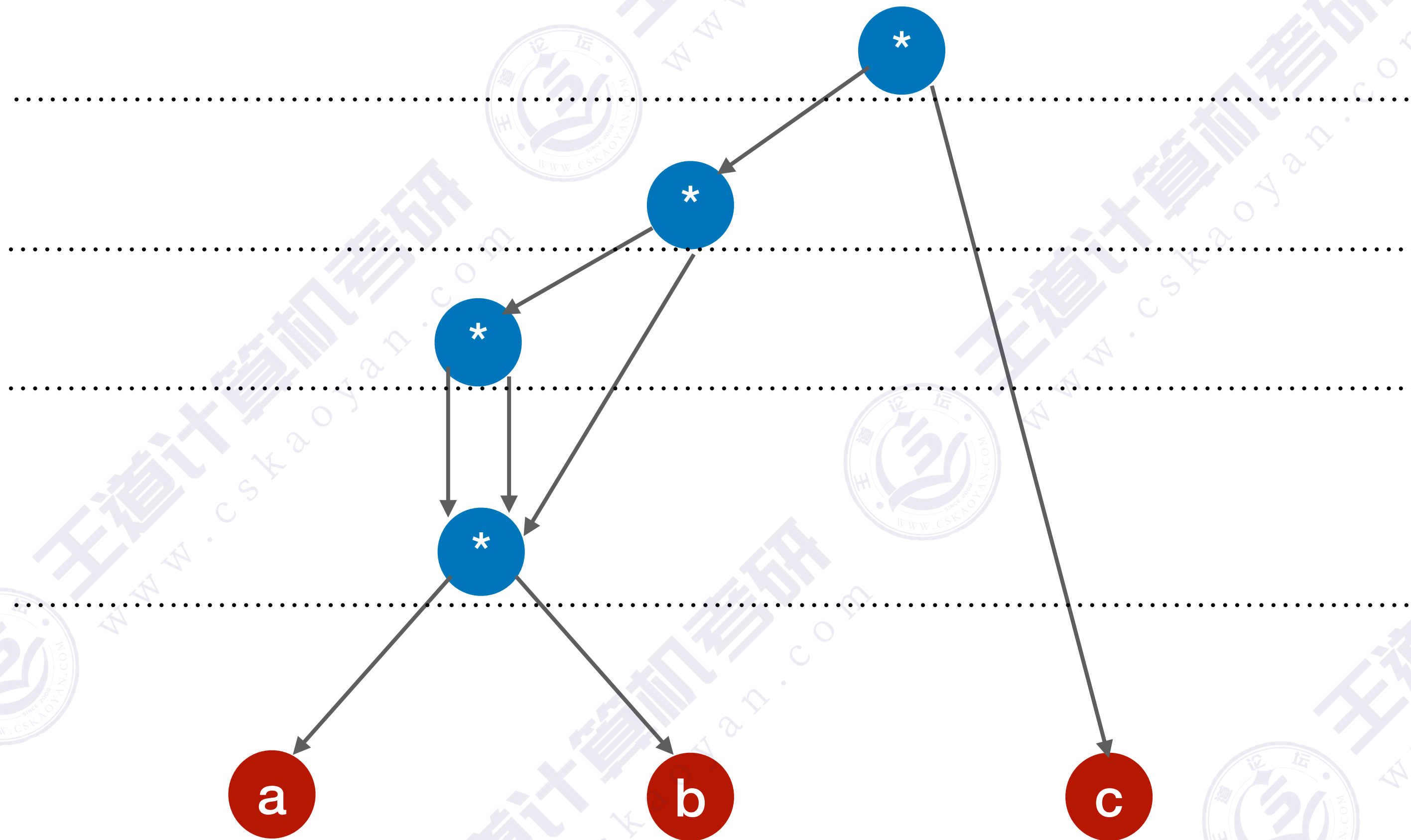
Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

练习

$$(a * b) * (a * b) * (a * b) * c$$

① ④ ② ⑤ ③ ⑥



Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

知识点回顾与重要考点

29. 【2019 统考真题】用有向无环图描述表达式 $(x+y)((x+y)/x)$ ，需要的顶点个数至少是 ()。

A. 5

B. 6

C. 8

D. 9

Step 1: 把各个操作数不重复地排成一排

Step 2: 标出各个运算符的生效顺序（先后顺序有点出入无所谓）

Step 3: 按顺序加入运算符，注意“**分层**”

Step 4: 从底向上逐层检查同层的运算符是否可以合体

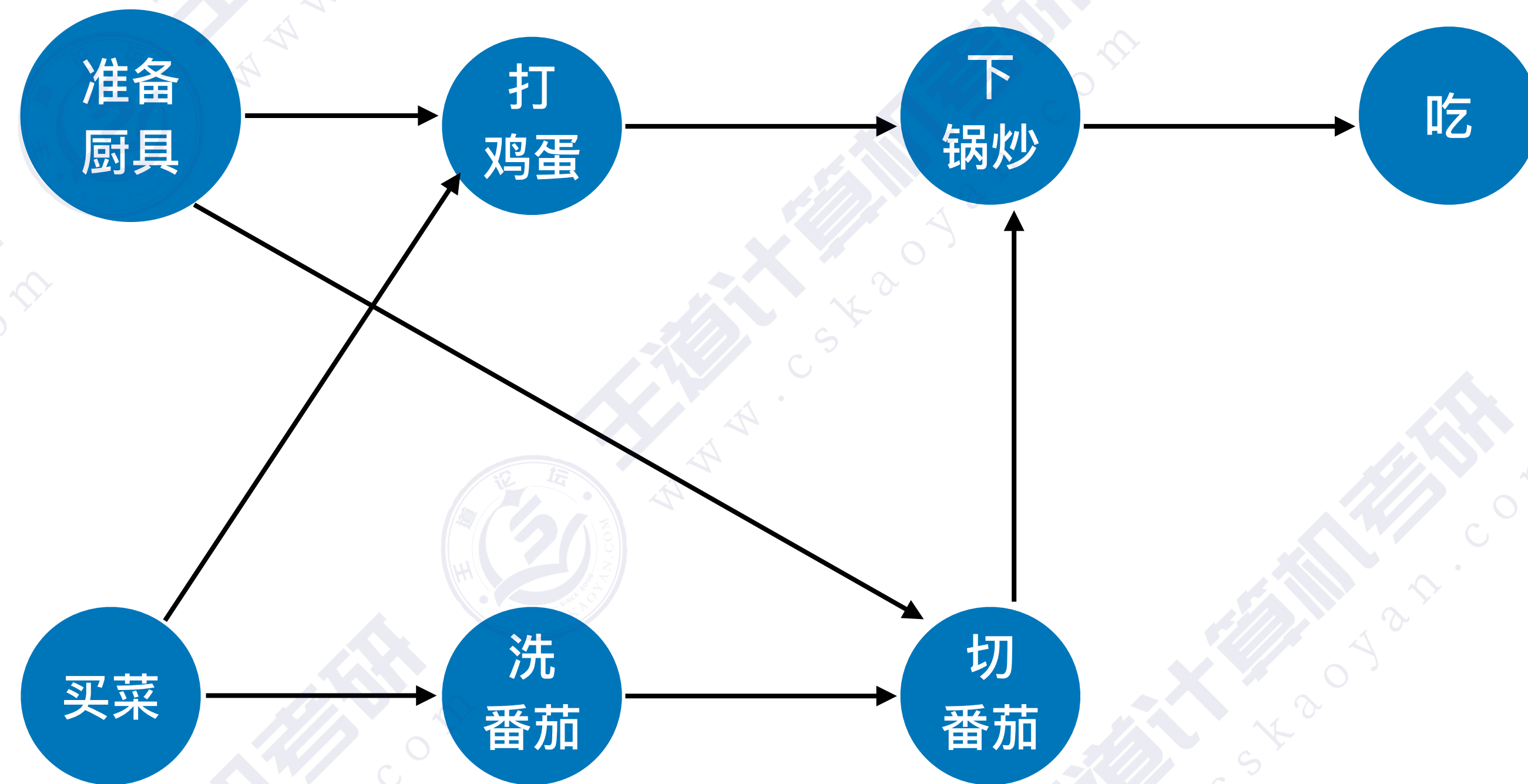
本节内容

拓扑排序

AOV网

AOV网(Activity On Vertex NetWork, 用顶点表示活动的网):

用DAG图(有向无环图)表示一个工程。顶点表示活动, 有向边 $\langle V_i, V_j \rangle$ 表示活动 V_i 必须先于活动 V_j 进行

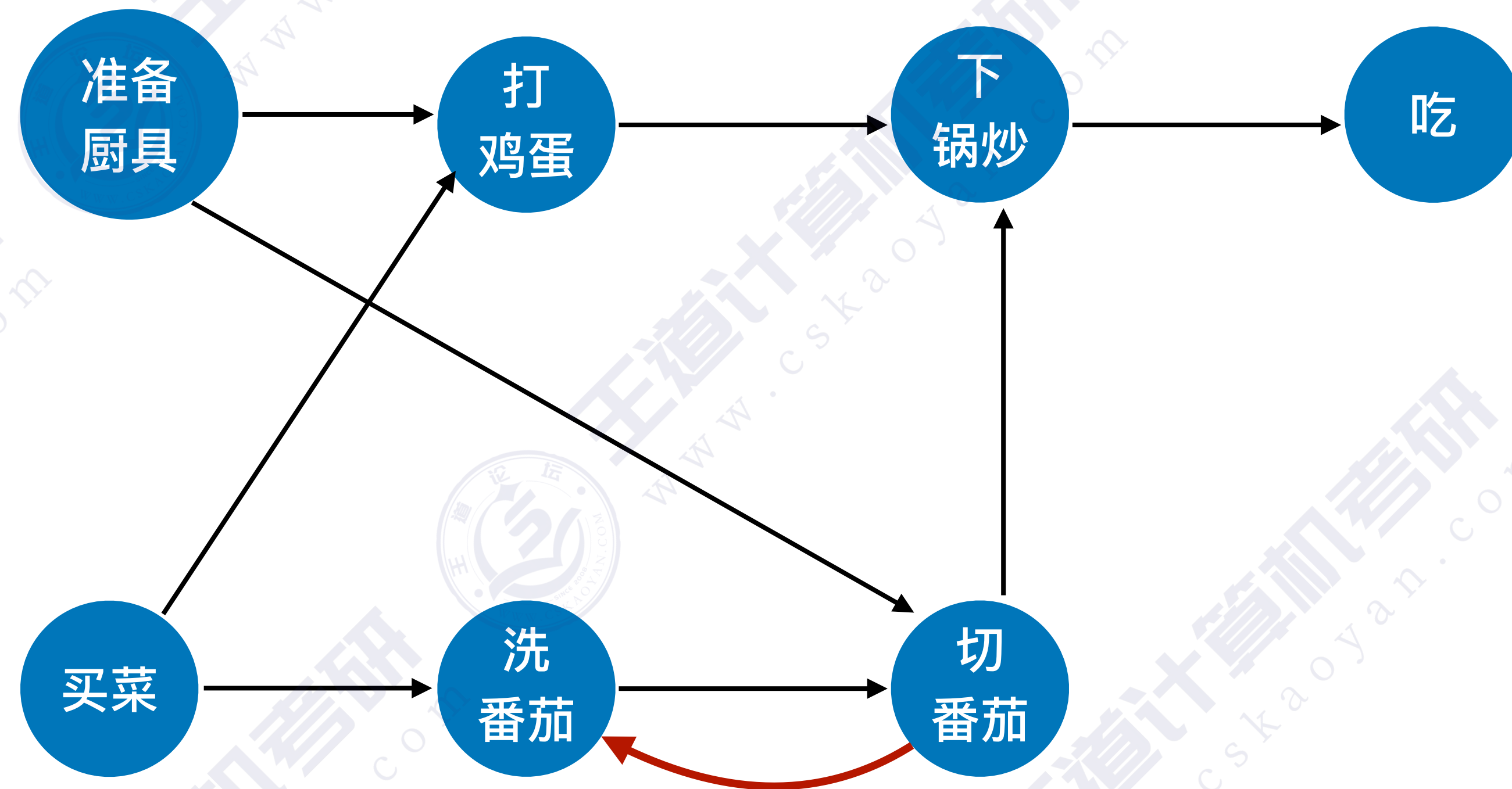


表示“番茄炒蛋工程”的AOV网

AOV网

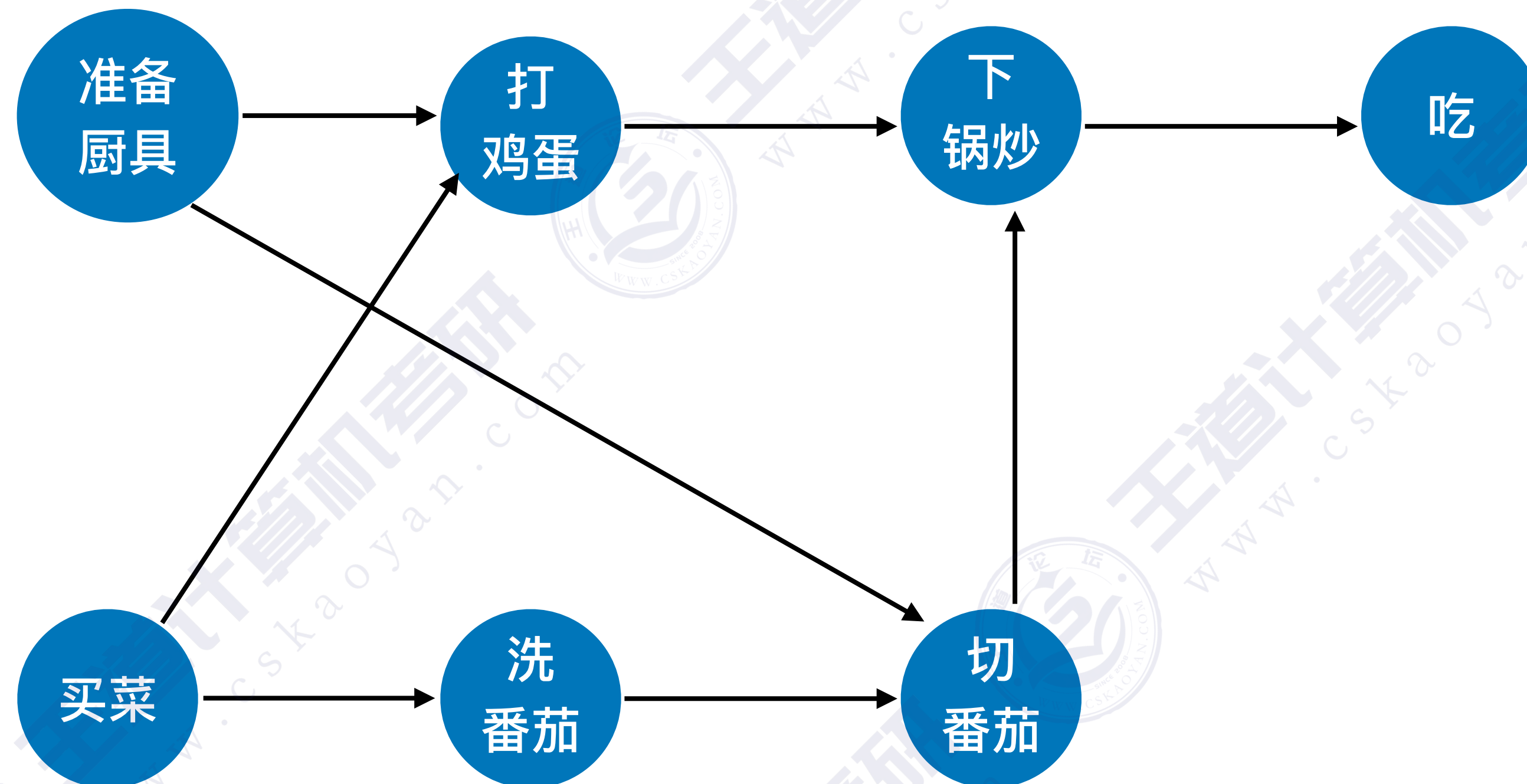
AOV网 (Activity On Vertex NetWork, 用顶点表示活动的网):

用DAG图 (有向无环图) 表示一个工程。顶点表示活动, 有向边 $\langle V_i, V_j \rangle$ 表示活动 V_i 必须先于活动 V_j 进行



不是AOV网

拓扑排序

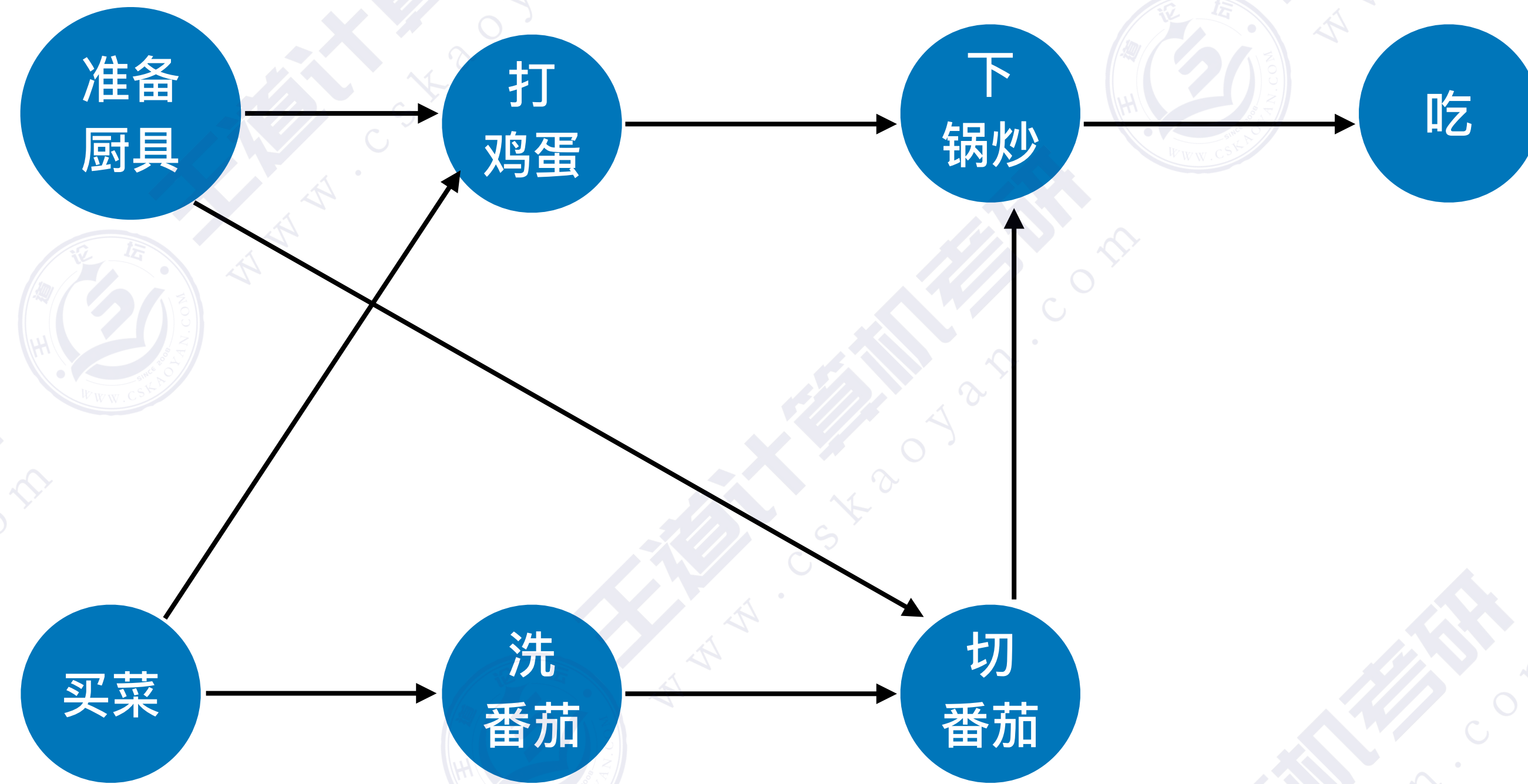


拓扑排序：在图论中，由一个**有向无环图**的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：

- ① 每个顶点出现且只出现一次。
- ② 若顶点A在序列中排在顶点B的前面，则在图中不存在从顶点B到顶点A的路径。

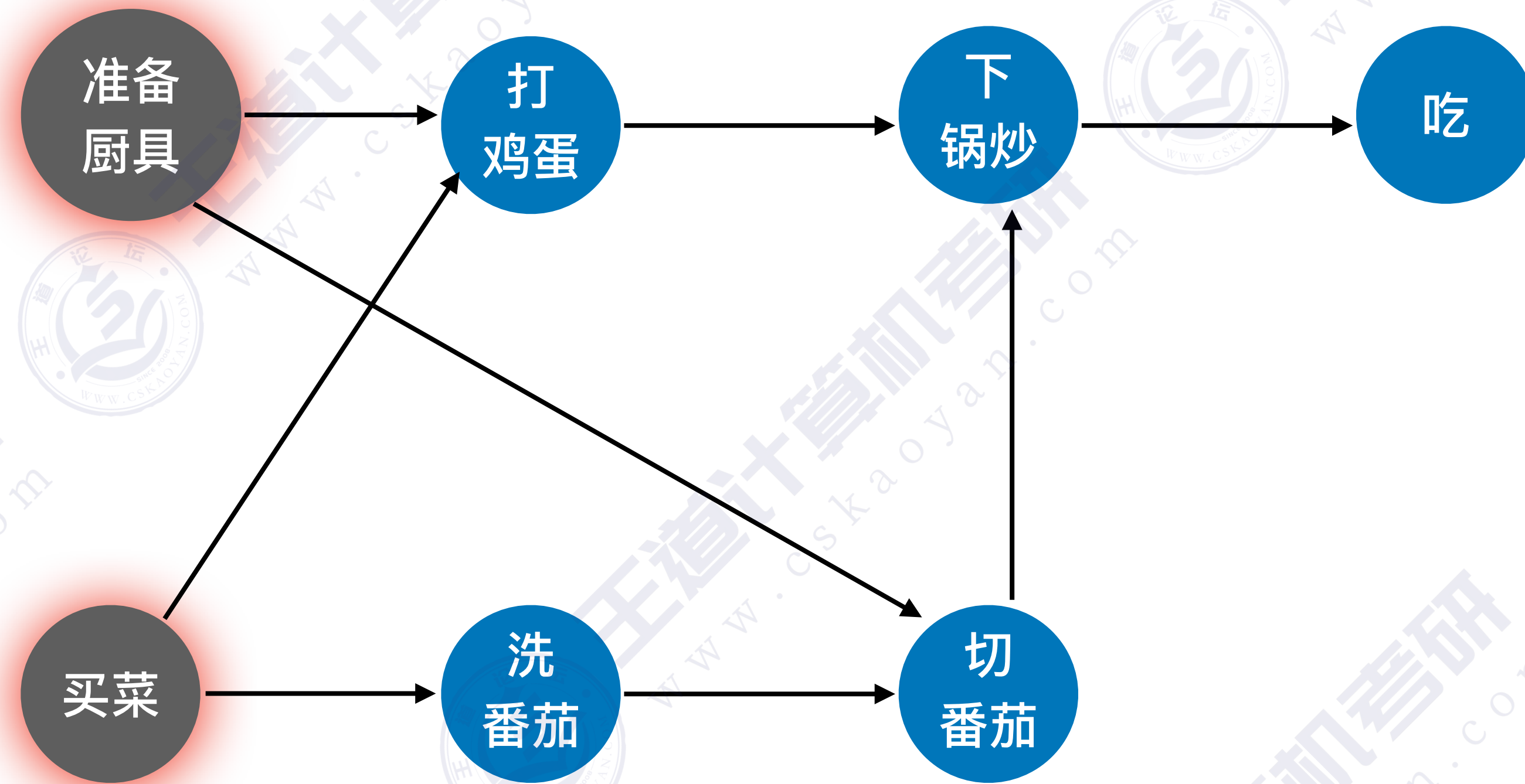
或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点A到顶点B的路径，则在排序中顶点B出现在顶点A的后面。每个AOV网都有一个或多个拓扑排序序列。

拓扑排序



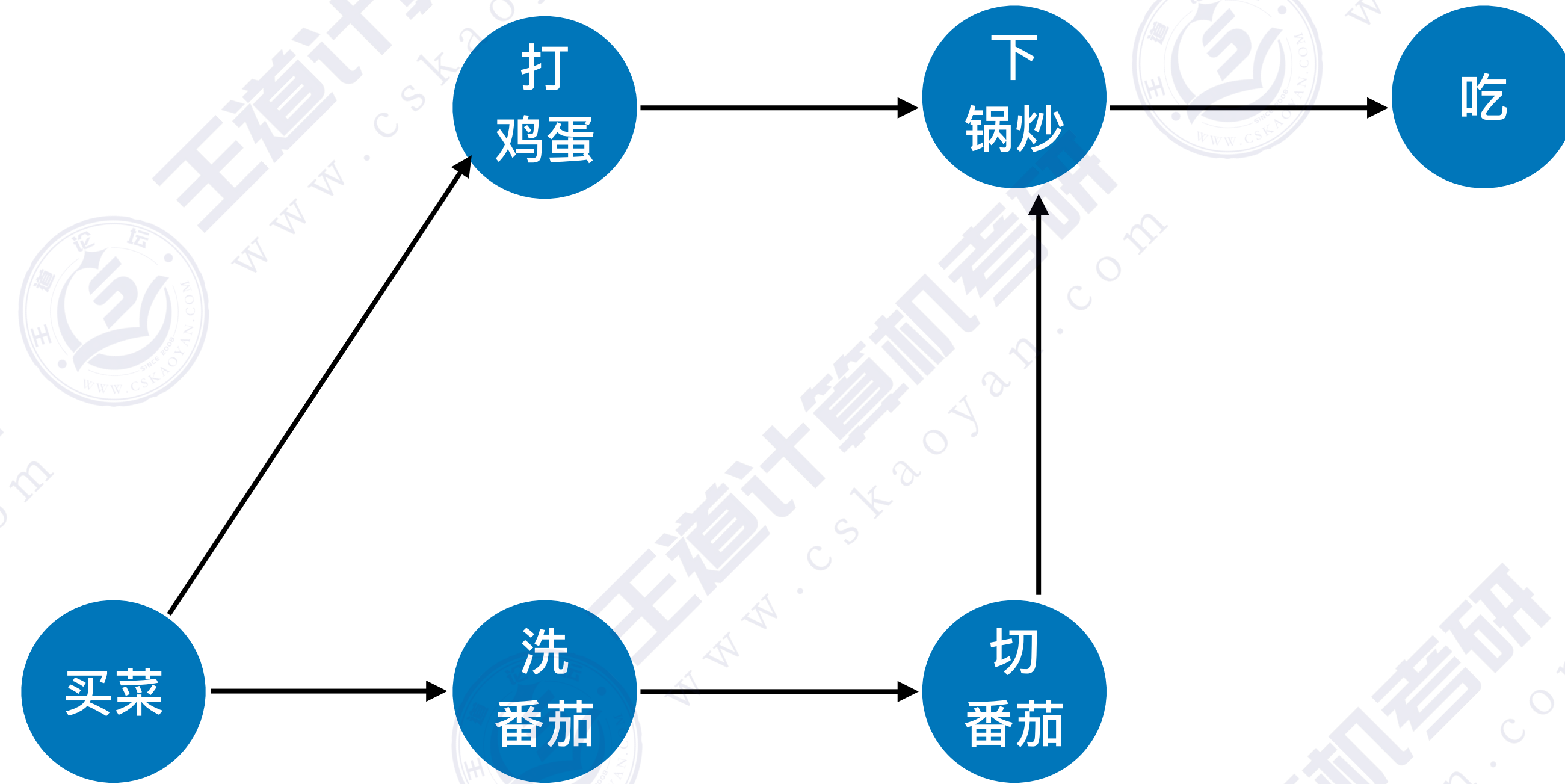
拓扑排序：找到做事的先后顺序

拓扑排序



拓扑排序：找到做事的先后顺序

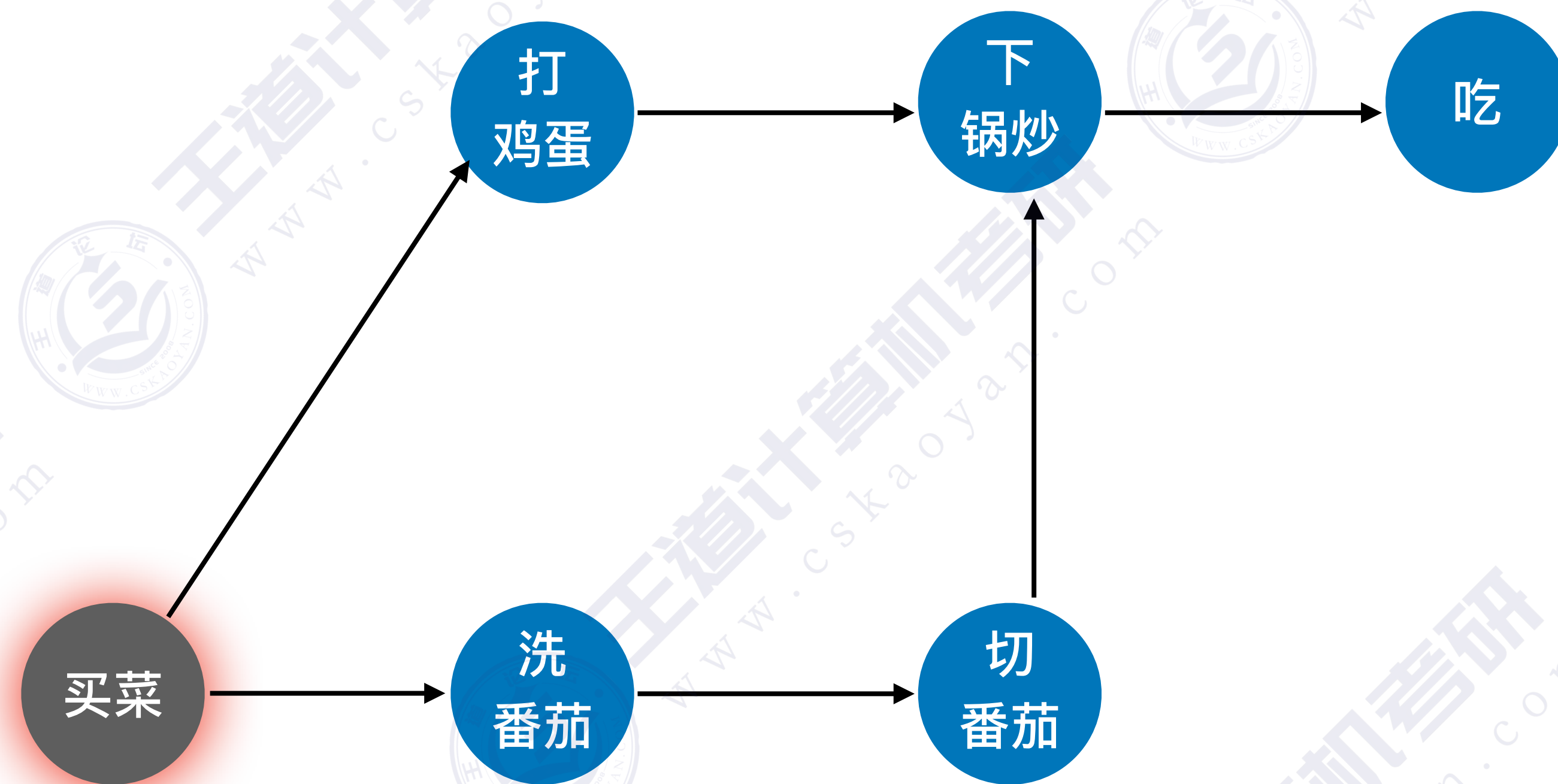
拓扑排序



拓扑排序：找到做事的先后顺序

准备
厨具

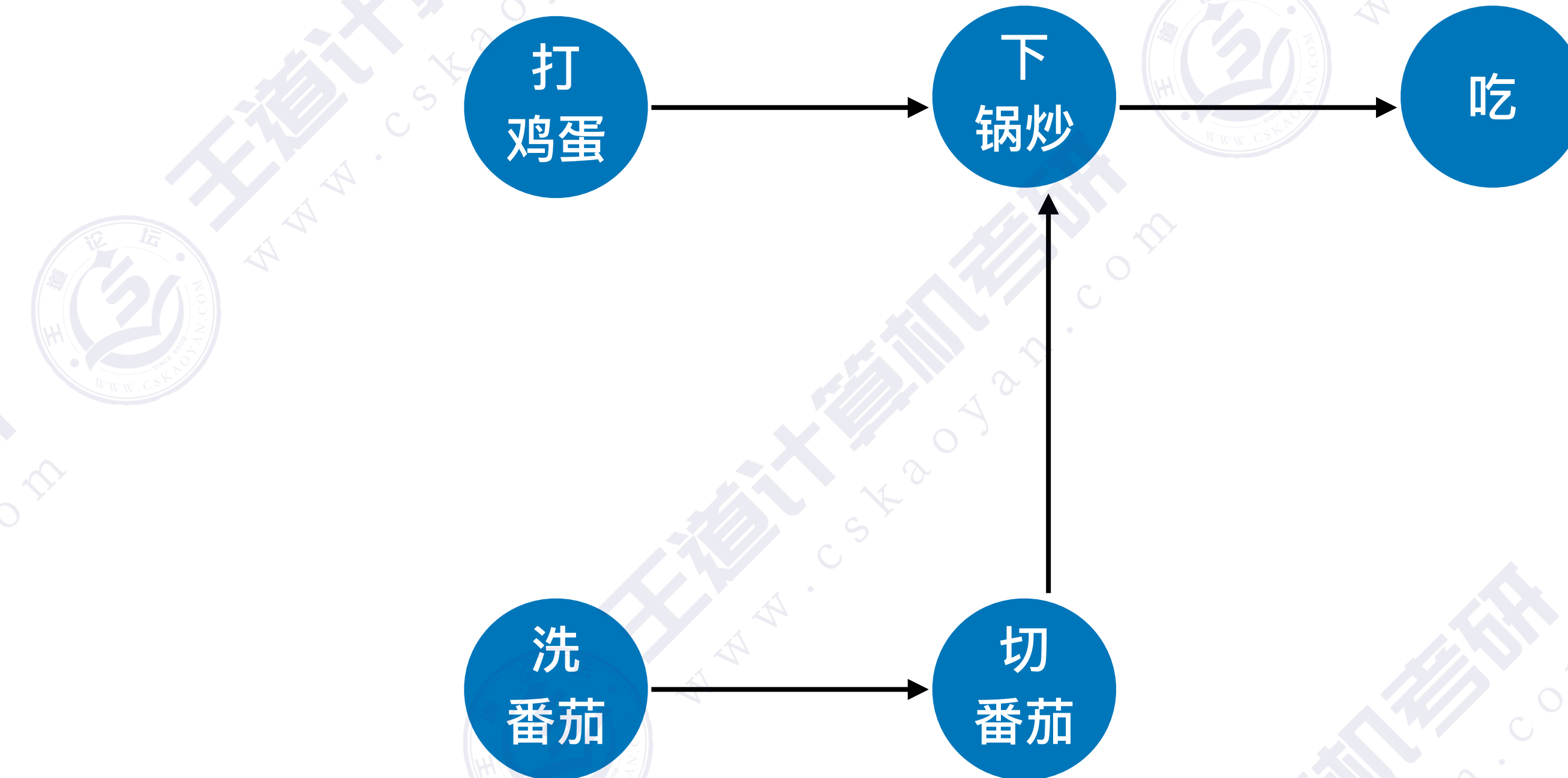
拓扑排序



拓扑排序：找到做事的先后顺序

准备
厨具

拓扑排序

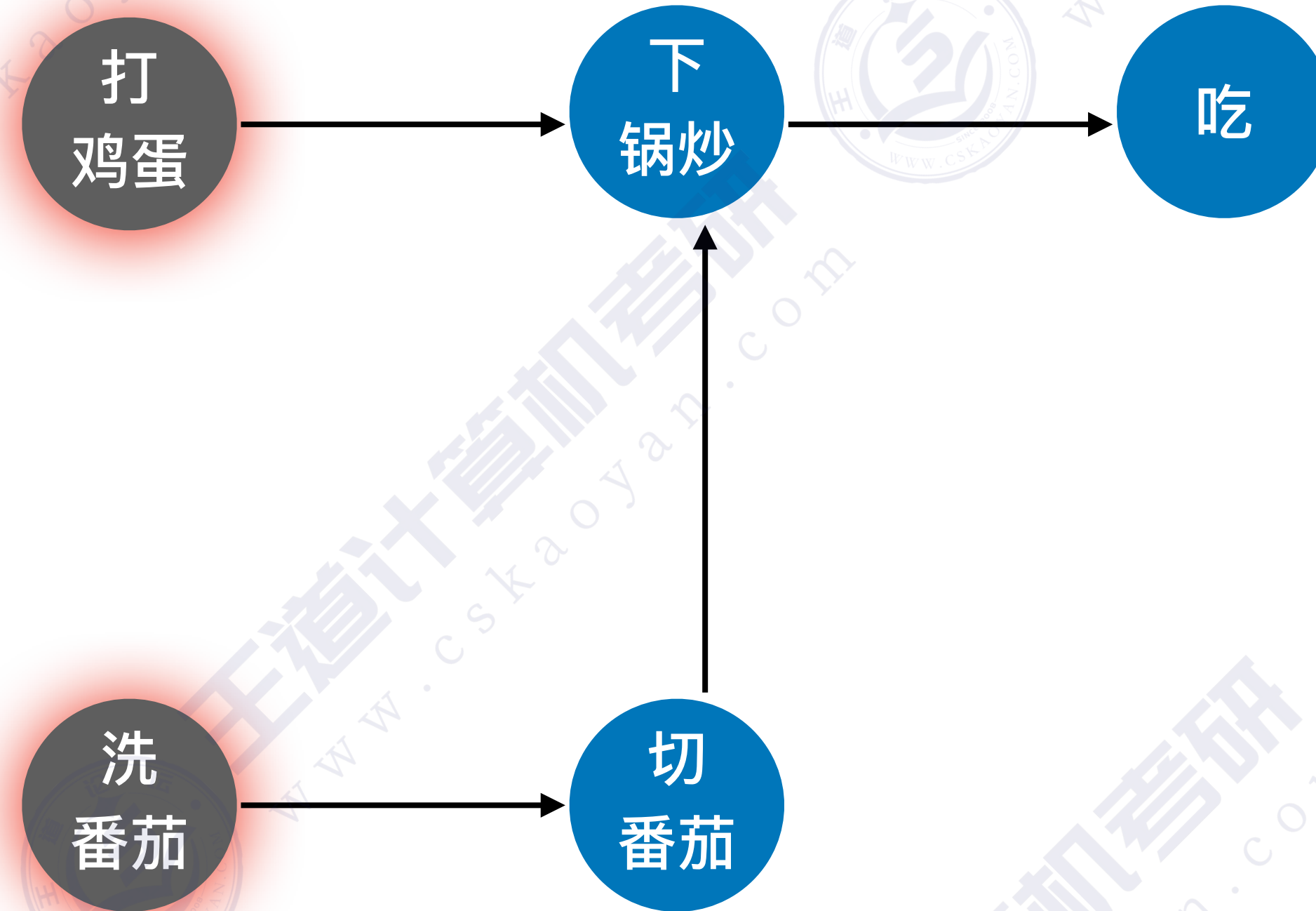


拓扑排序：找到做事的先后顺序

准备
厨具

买菜

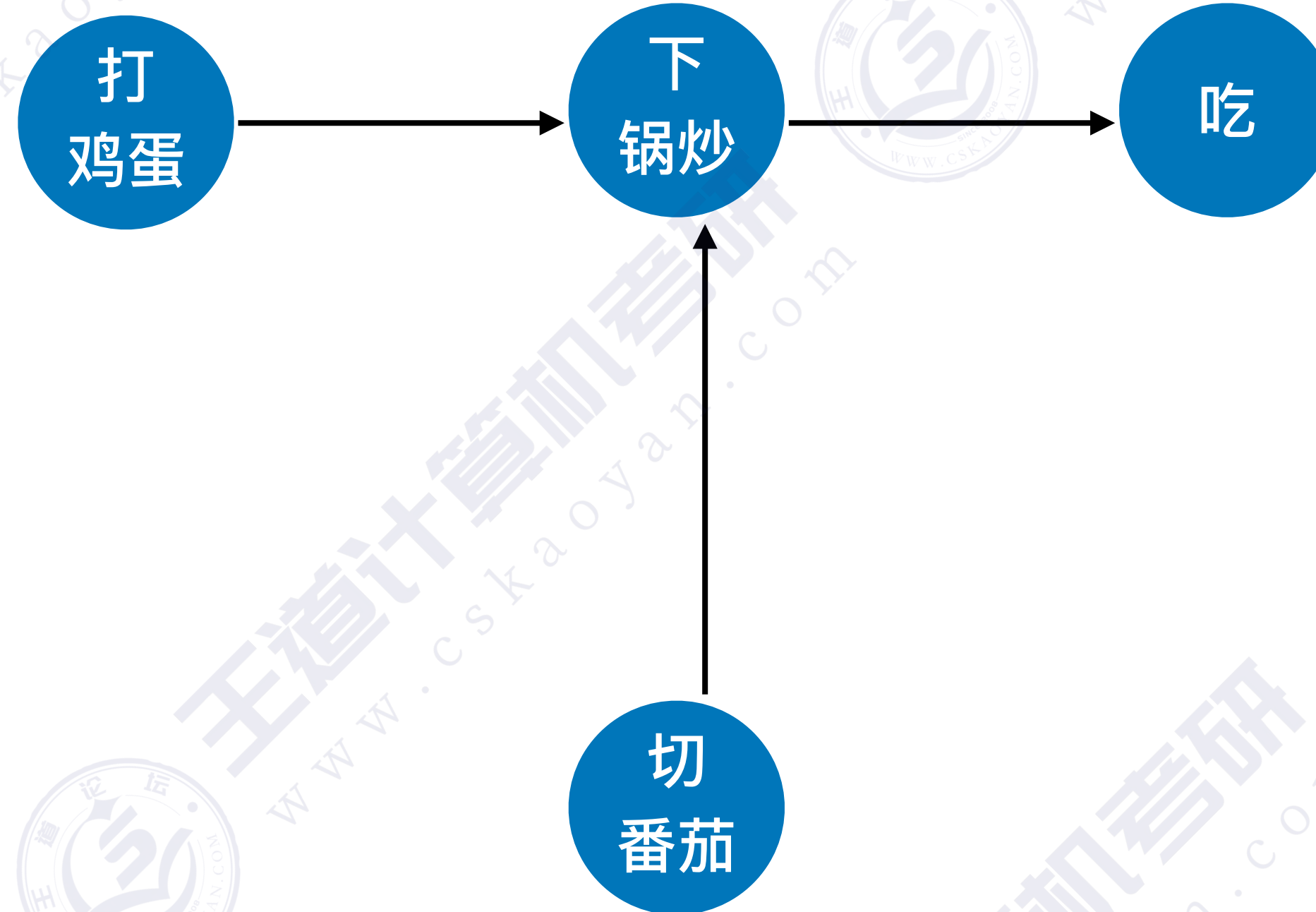
拓扑排序



拓扑排序：找到做事的先后顺序



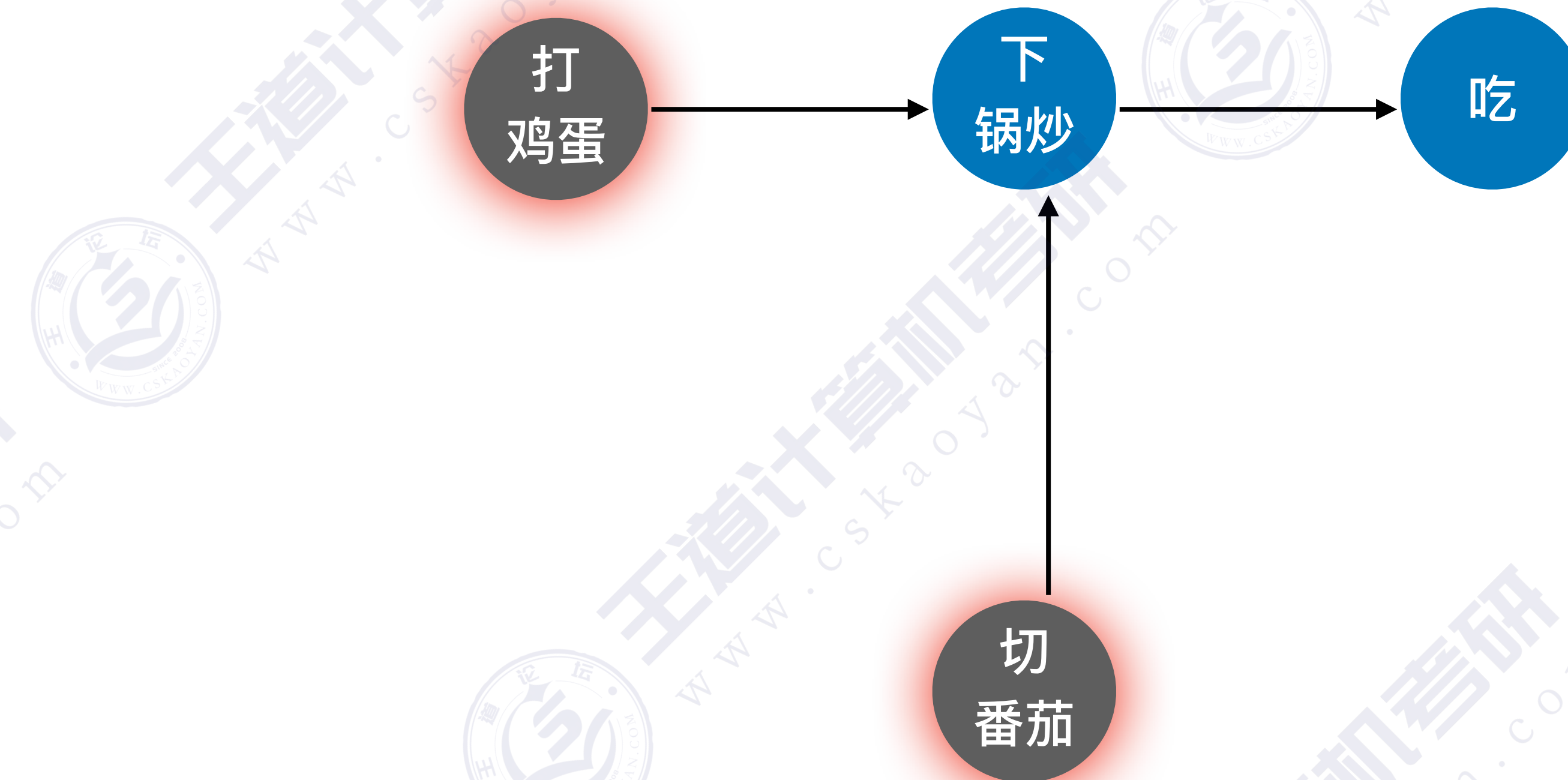
拓扑排序



拓扑排序：找到做事的先后顺序



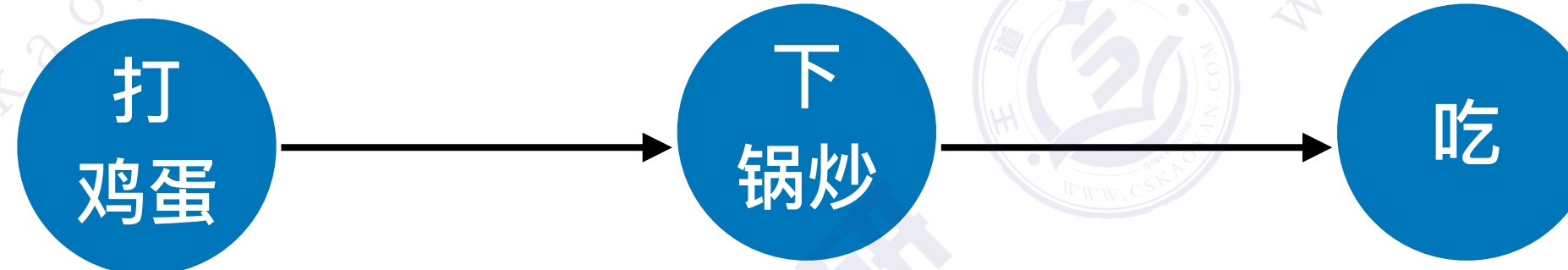
拓扑排序



拓扑排序：找到做事的先后顺序



拓扑排序



拓扑排序：找到做事的先后顺序



拓扑排序



打鸡蛋

下锅炒

吃

拓扑排序：找到做事的先后顺序

准备
厨具

买菜

洗
番茄

切
番茄

拓扑排序



拓扑排序：找到做事的先后顺序



拓扑排序



拓扑排序：找到做事的先后顺序



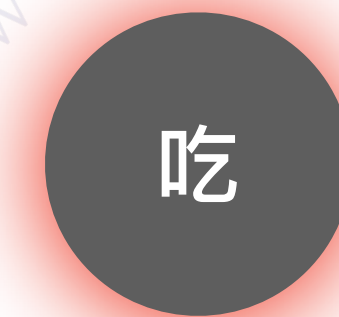
拓扑排序



拓扑排序：找到做事的先后顺序



拓扑排序



拓扑排序：找到做事的先后顺序



拓扑排序



拓扑排序的实现：

- ① 从AOV网中选择一个没有前驱（入度为0）的顶点并输出。
- ② 从网中删除该顶点和所有以它为起点的有向边。
- ③ 重复①和②直到当前的AOV网为空或当前网中不存在无前驱的顶点为止。

准备
厨具

买菜

洗
番茄

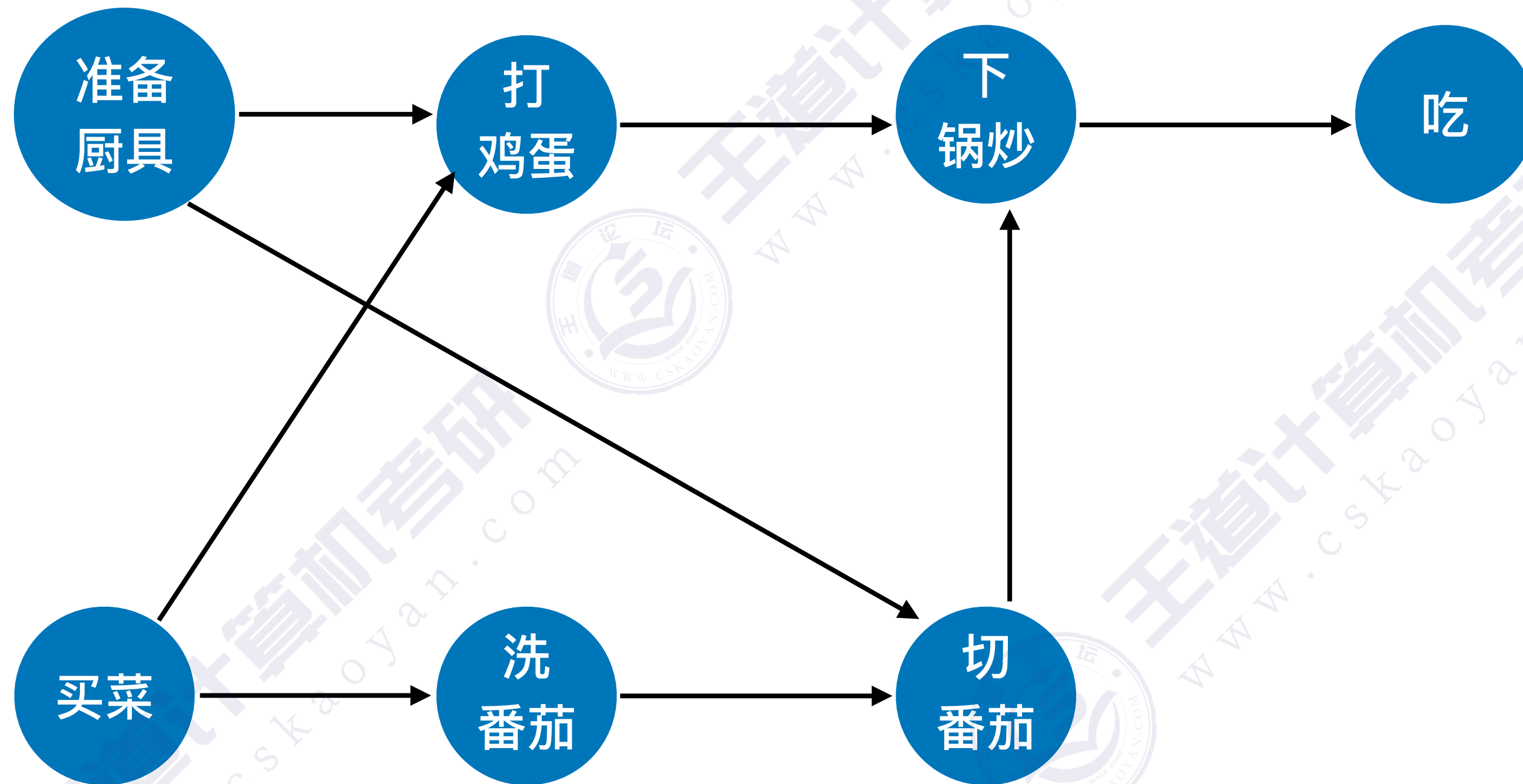
切
番茄

打
鸡蛋

下
锅炒

吃

拓扑排序



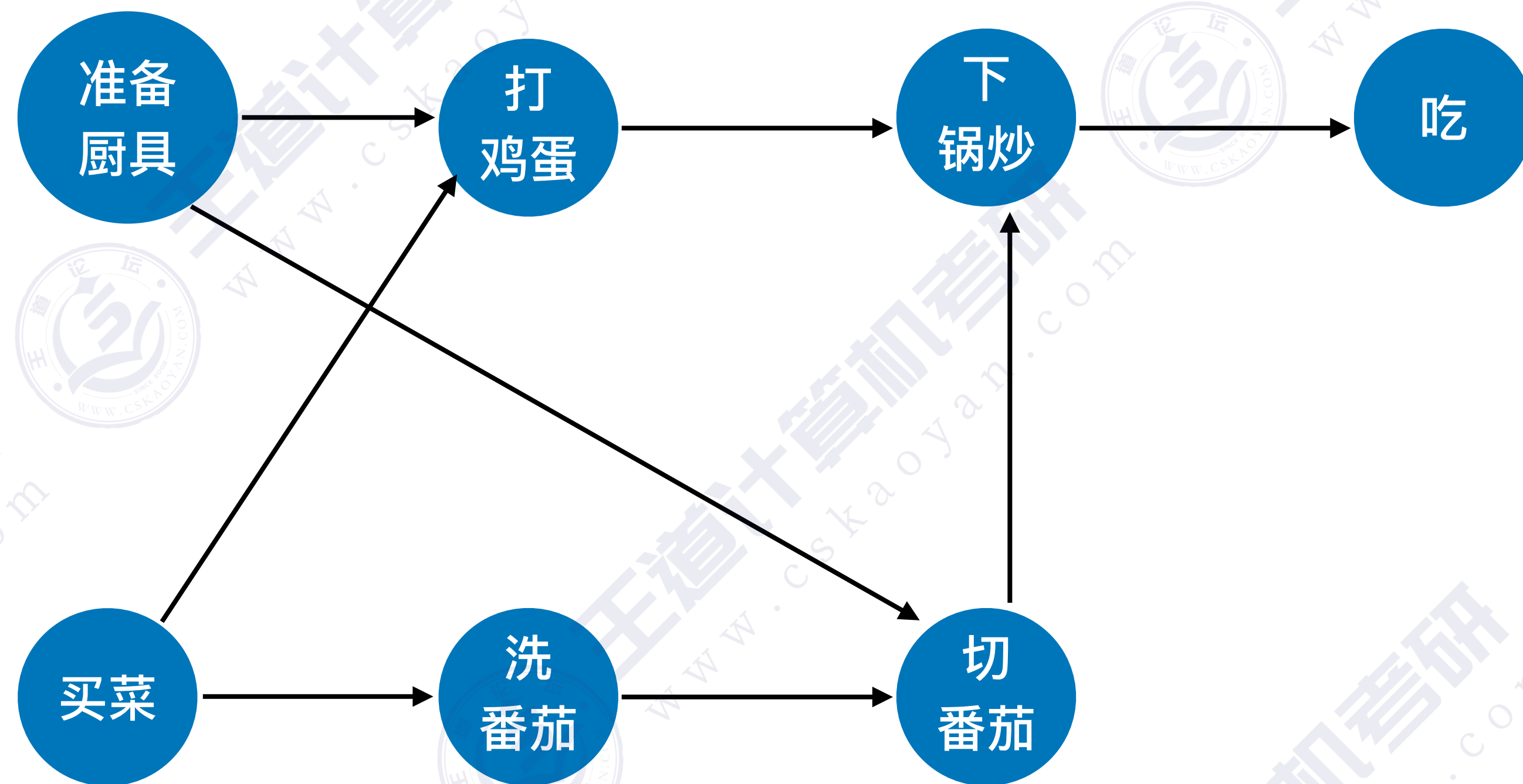
拓扑排序：在图论中，由一个**有向无环图**的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：

- ① 每个顶点出现且只出现一次。
- ② 若顶点A在序列中排在顶点B的前面，则在图中不存在从顶点B到顶点A的路径。

或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点A到顶点B的路径，则在排序中顶点B出现在顶点A的后面。**每个AOV网都有一个或多个拓扑排序序列。**



拓扑排序

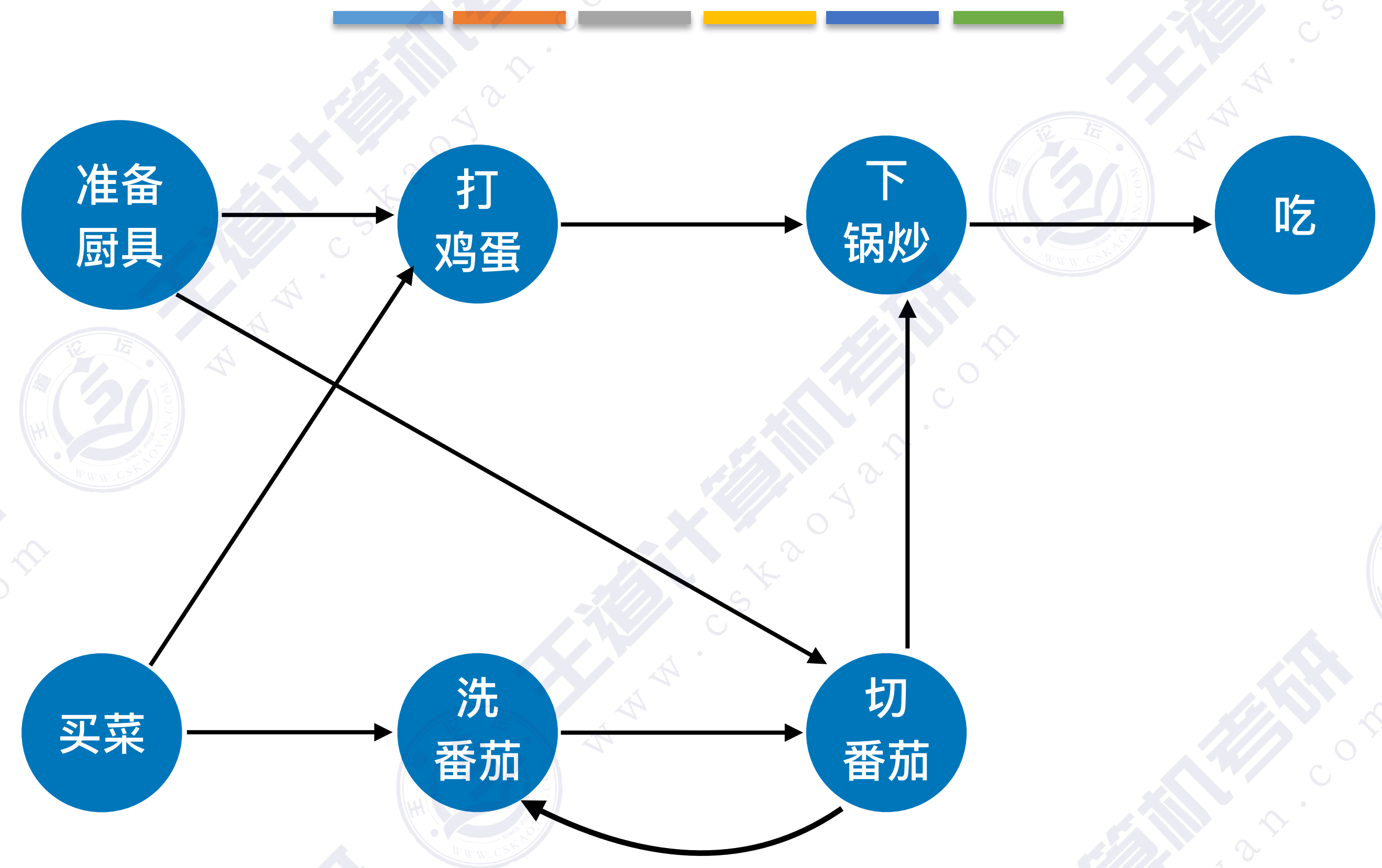


拓扑排序的实现:

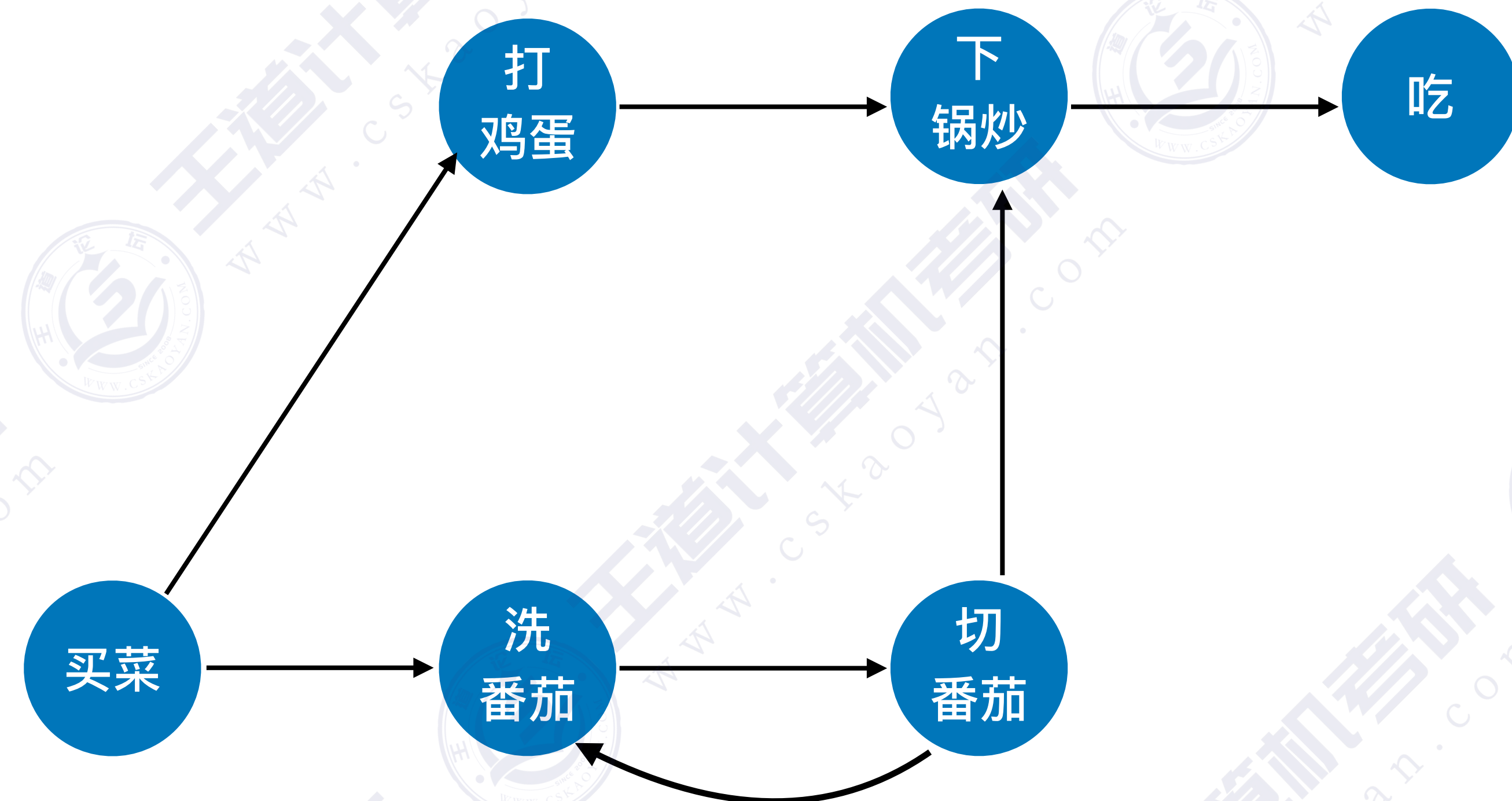
- ① 从AOV网中选择一个没有前驱的顶点并输出。
- ② 从网中删除该顶点和所有以它为起点的有向边。
- ③ 重复①和②直到当前的AOV网为空或当前网中不存在无前驱的顶点为止。

说明有回路

对有回路的图进行拓扑排序

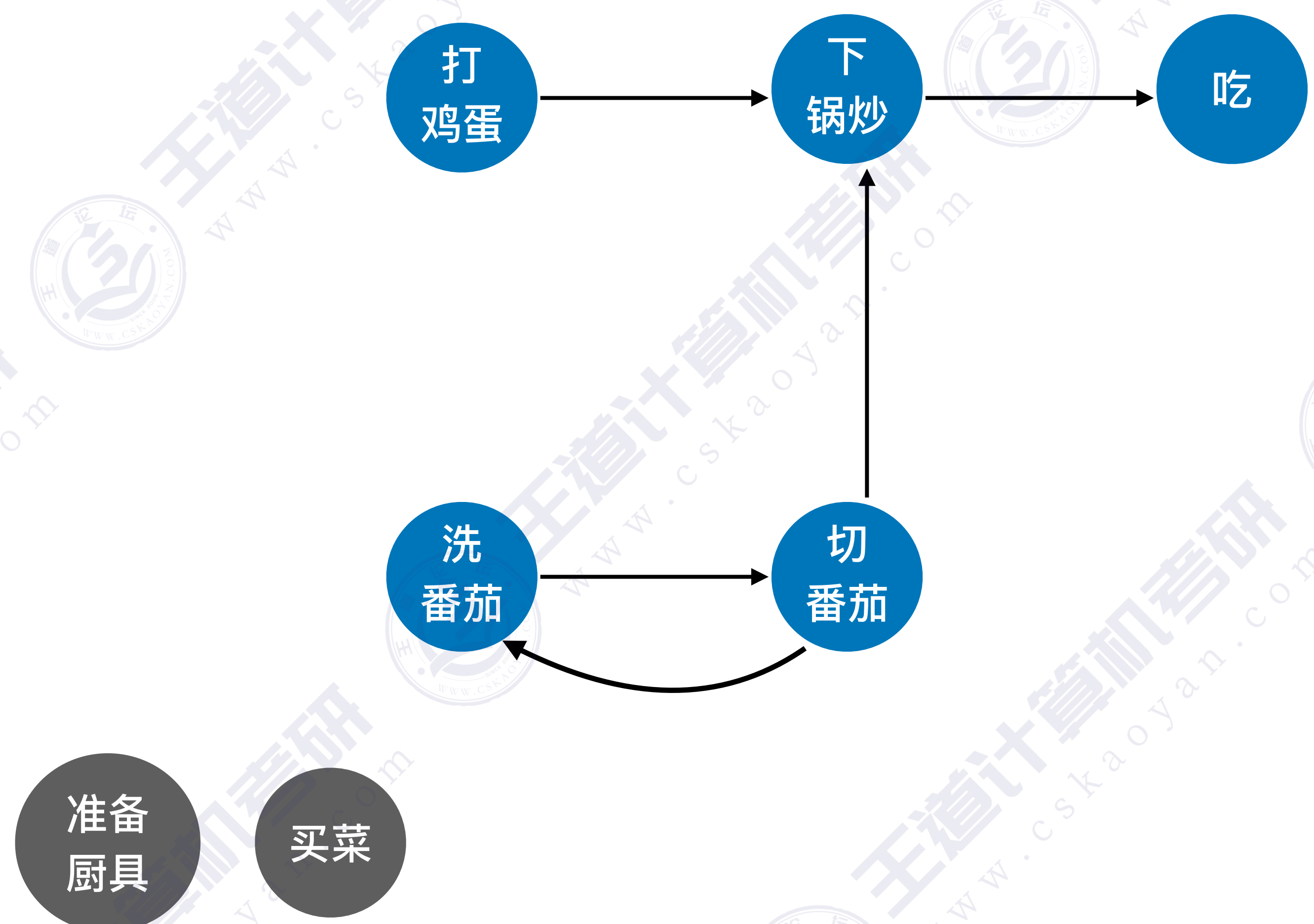


对有回路的图进行拓扑排序

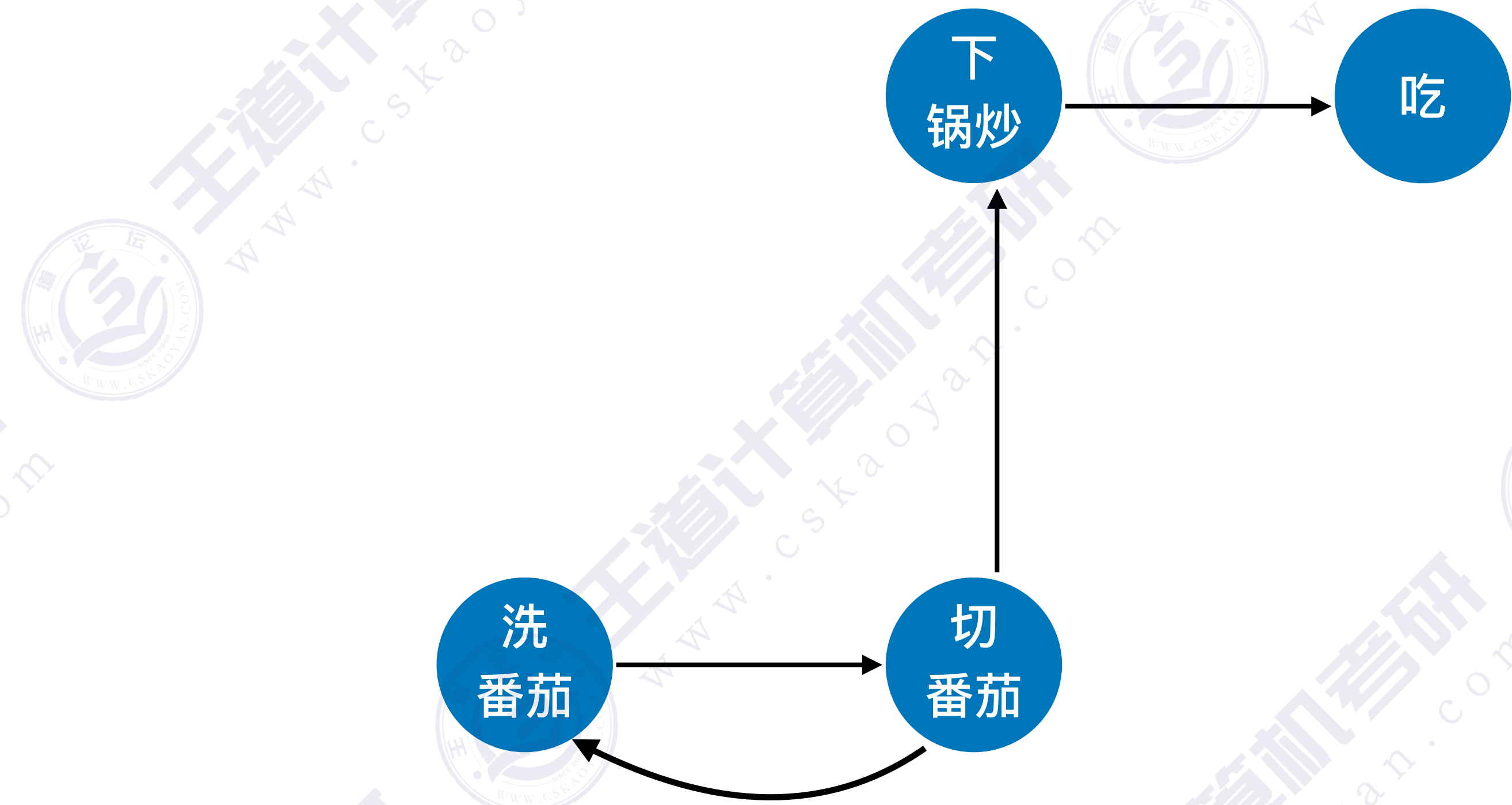


准备
厨具

对有回路的图进行拓扑排序



对有回路的图进行拓扑排序

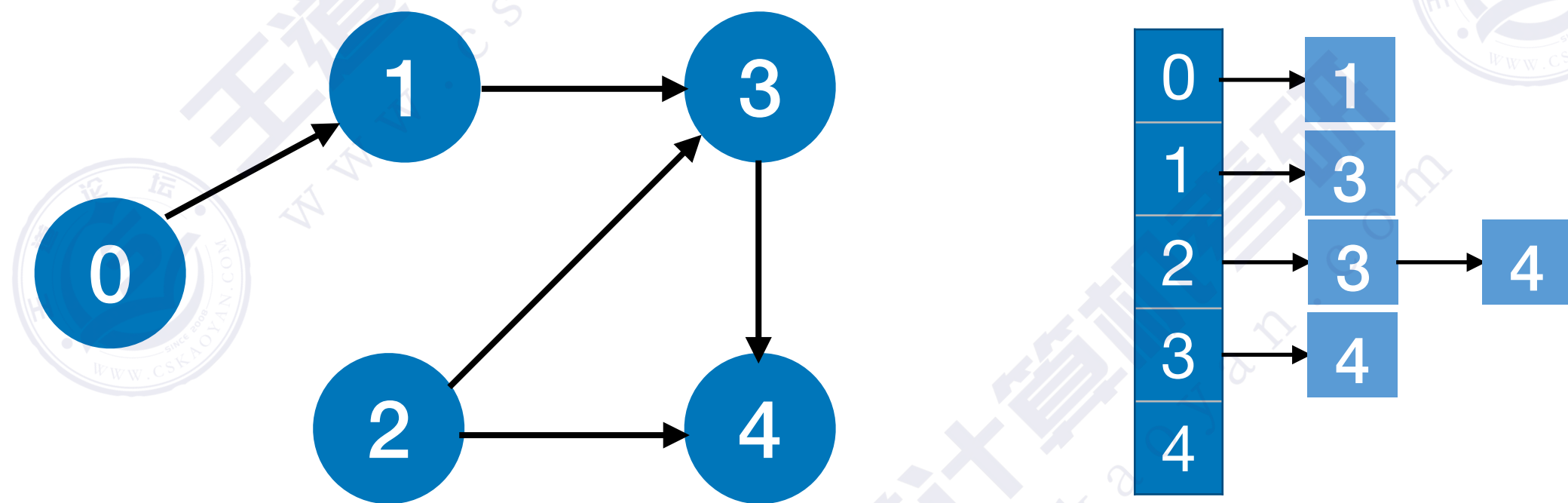


当前所有顶点入度>0,
说明原图存在回路


```

#define MaxVertexNum 100    //图中顶点数目的最大值
typedef struct ArcNode{    //边表结点
    int adjvex;            //该弧所指向的顶点的位置
    struct ArcNode *nextarc; //指向下一条弧的指针
    //InfoType info;        //网的边权值
}ArcNode;
typedef struct VNode{    //顶点表结点
    VertexType data;    //顶点信息
    ArcNode *firstarc;  //指向第一条依附该顶点的弧的指针
}VNode, AdjList[MaxVertexNum];
typedef struct{
    AdjList vertices;    //邻接表
    int vexnum, arcnum;  //图的顶点数和弧数
} Graph;                //Graph是以邻接表存储的图类型

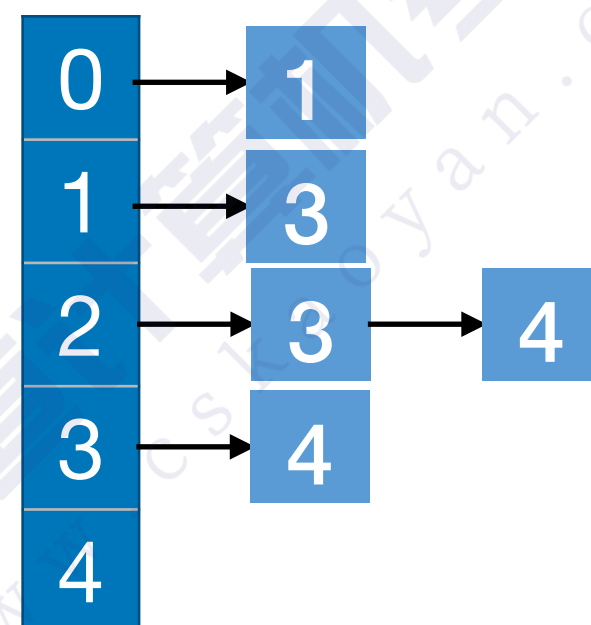
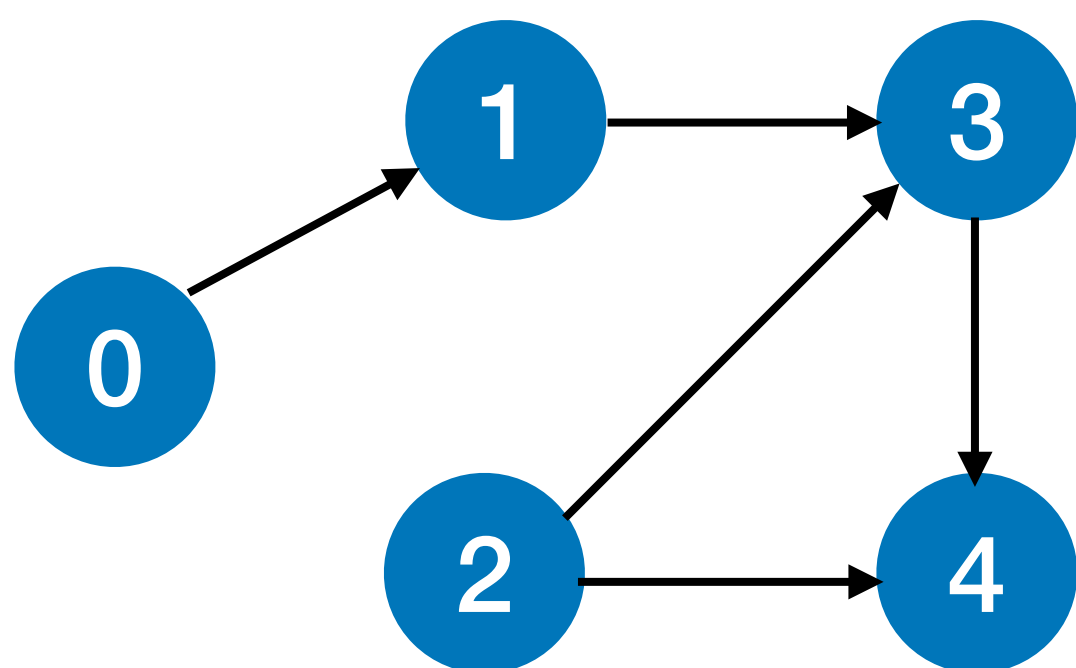
```



```

bool TopologicalSort(Graph G){
    InitStack(S);    //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);    //将所有入度为0的顶点进栈
    }
    int count=0;    //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);    //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    //while
    if(count<G.vexnum)
        return false;    //排序失败，有向图中有回路
    else
        return true;    //拓扑排序成功
}

```

indegree[]

0
1
0
2
2

当前顶
点入度

print[]

-1
-1
-1
-1
-1

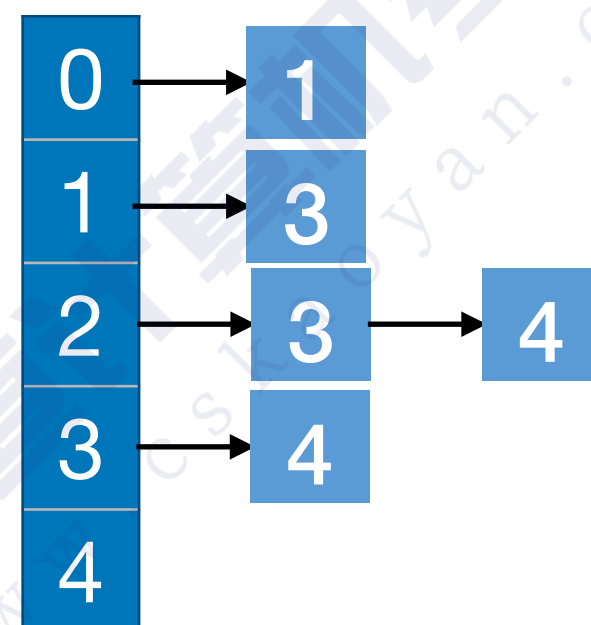
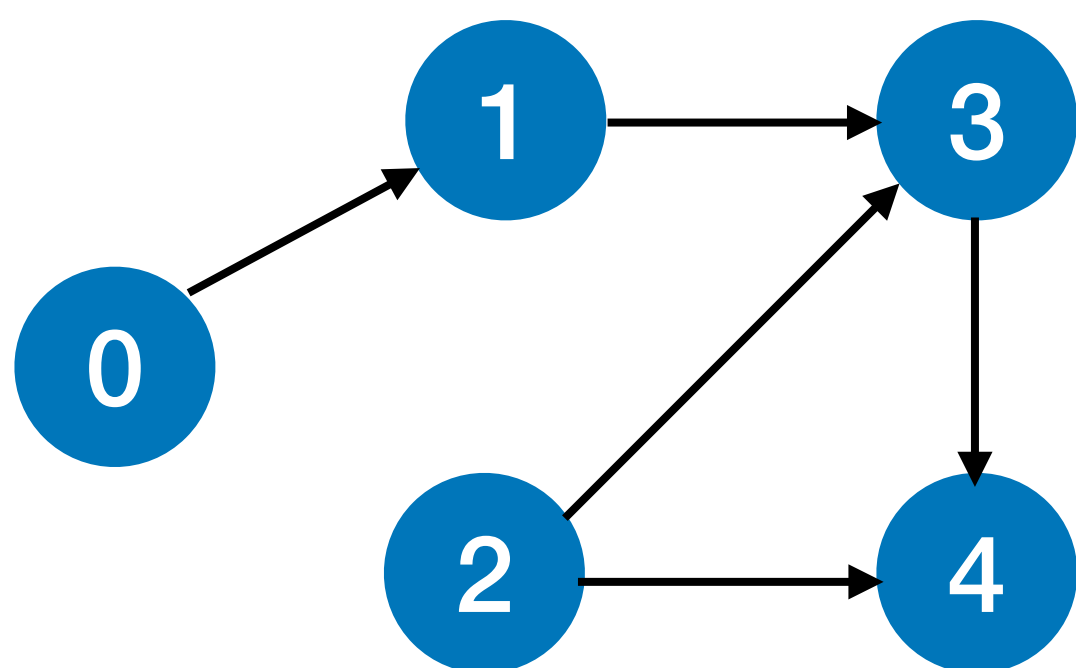
记录拓
扑序列

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```



indegree[]

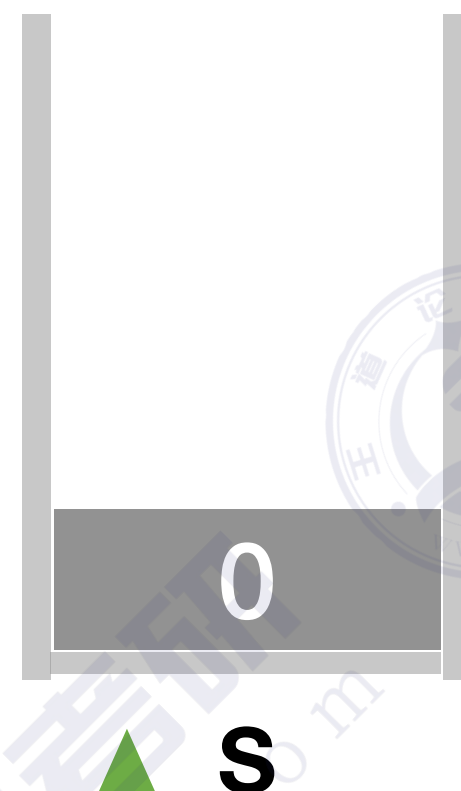
0
1
0
2
2

当前顶
点入度

print[]

-1
-1
-1
-1
-1

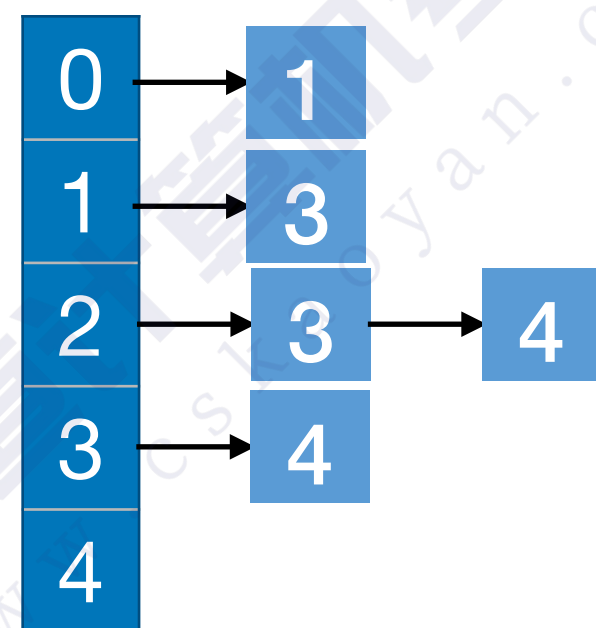
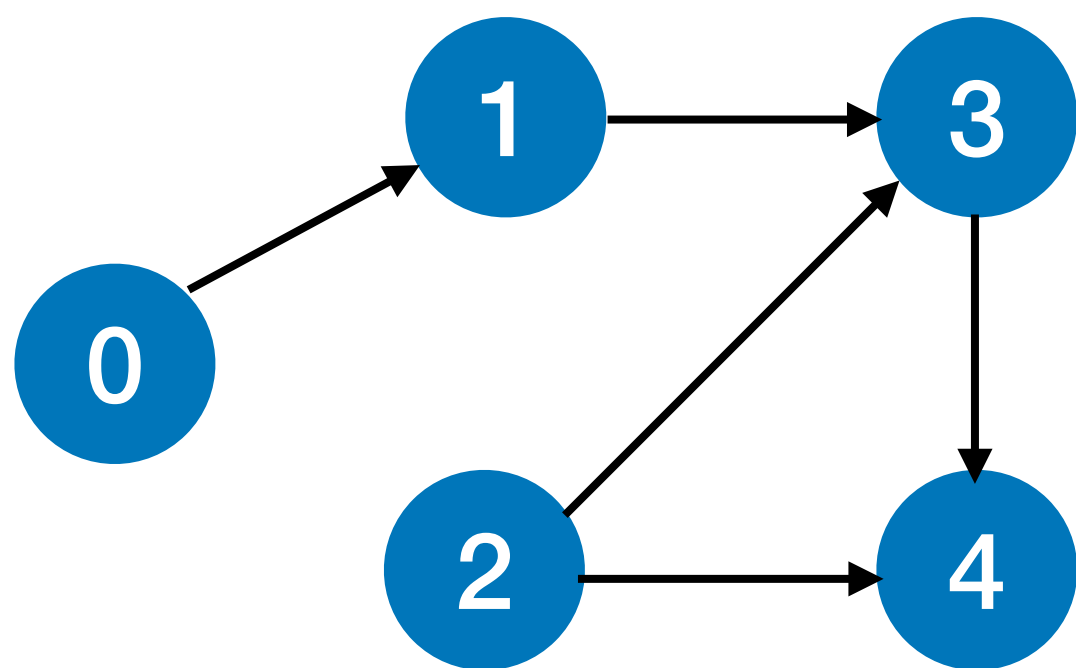
记录拓
扑序列



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

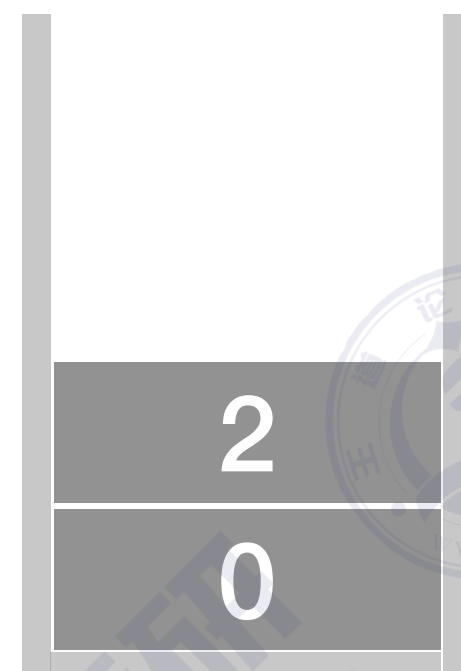
0
1
0
2
2

当前顶
点入度

print[]

-1
-1
-1
-1
-1

记录拓
扑序列

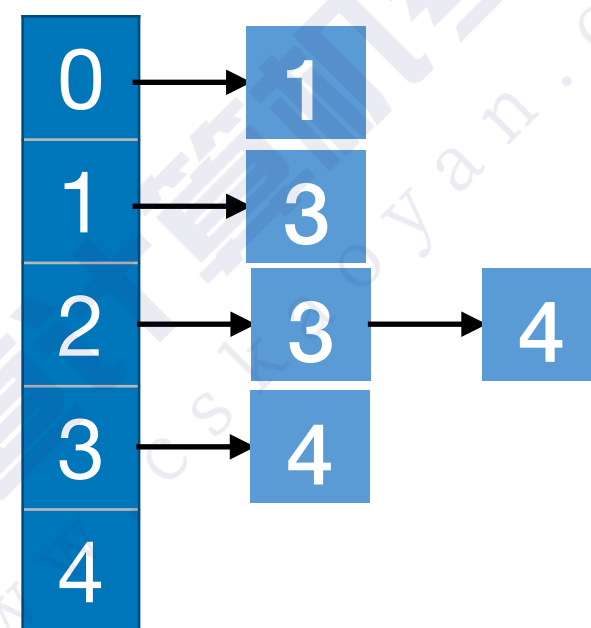
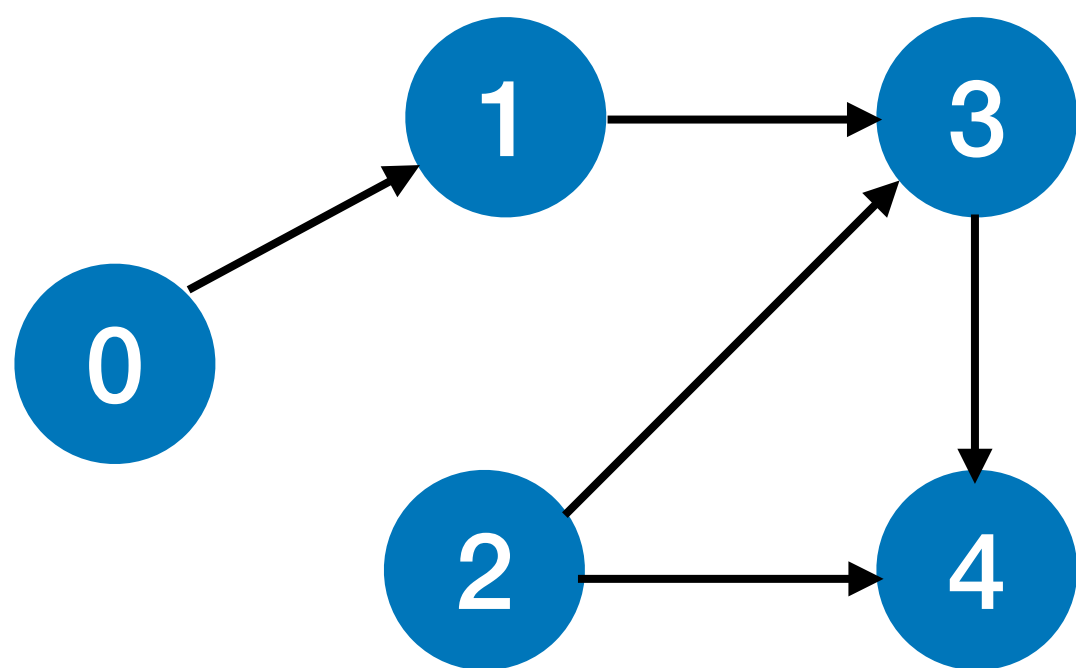


S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);      //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);  //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```



indegree[]

0
1
0
2
2

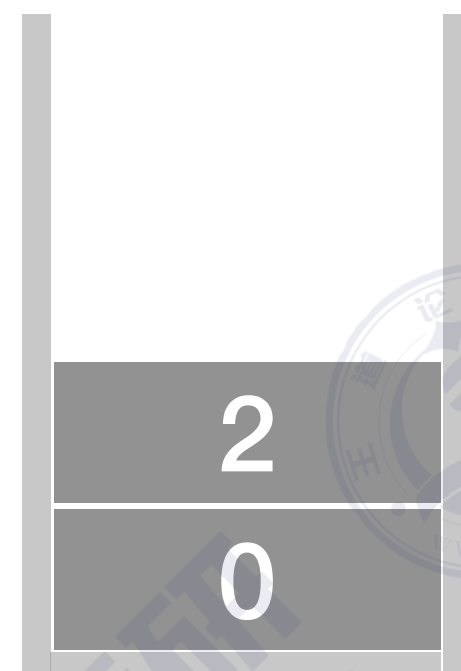
当前顶
点入度

print[]

-1
-1
-1
-1
-1

记录拓
扑序列

← count

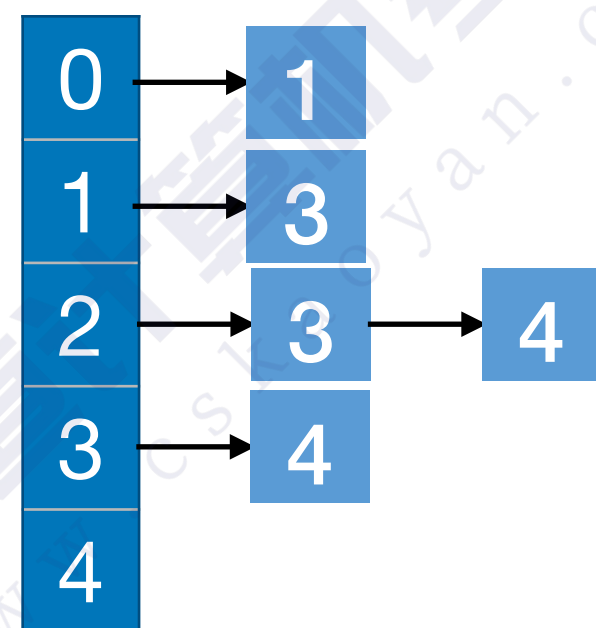
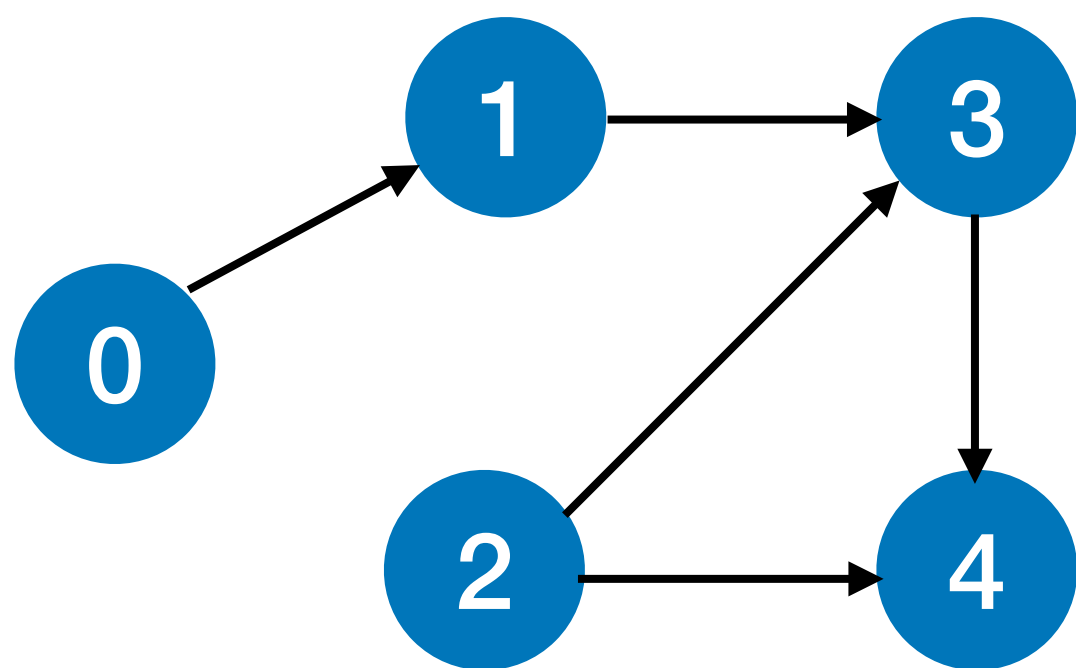


S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
1
0
2
2

当前顶
点入度

print[]

2
-1
-1
-1
-1

记录拓
扑序列

← count

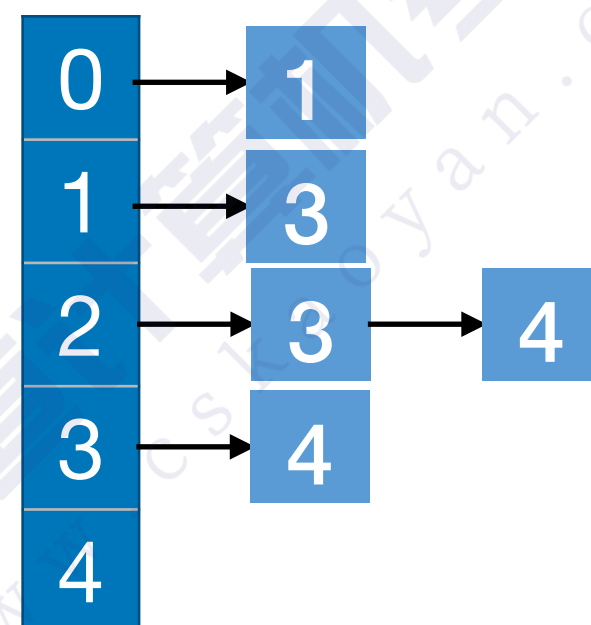
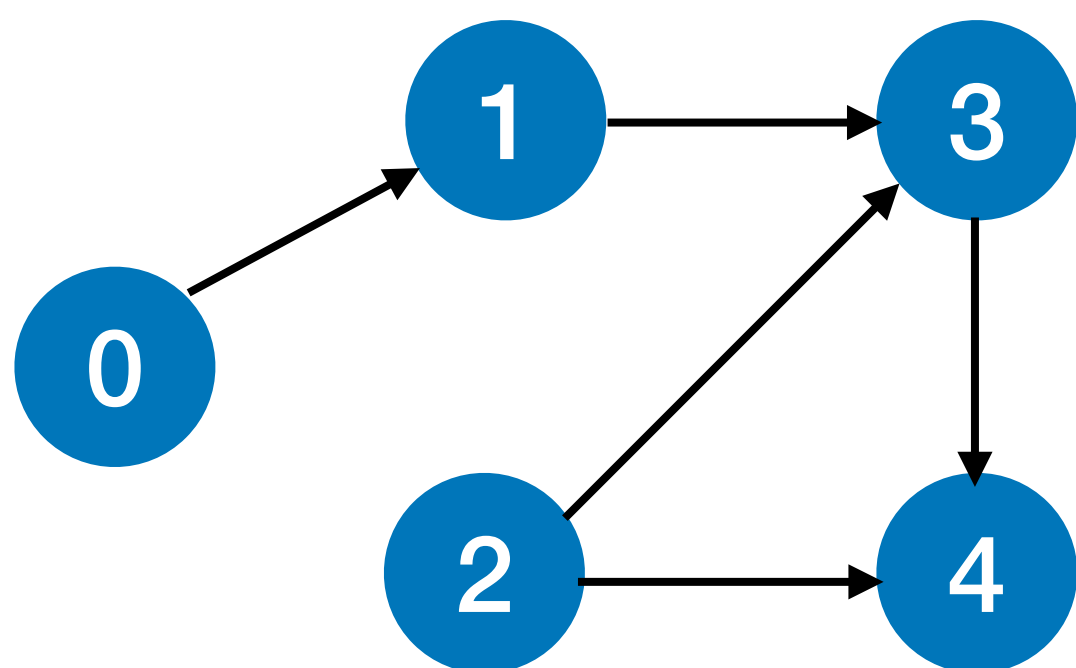
S



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
1
0
2
2

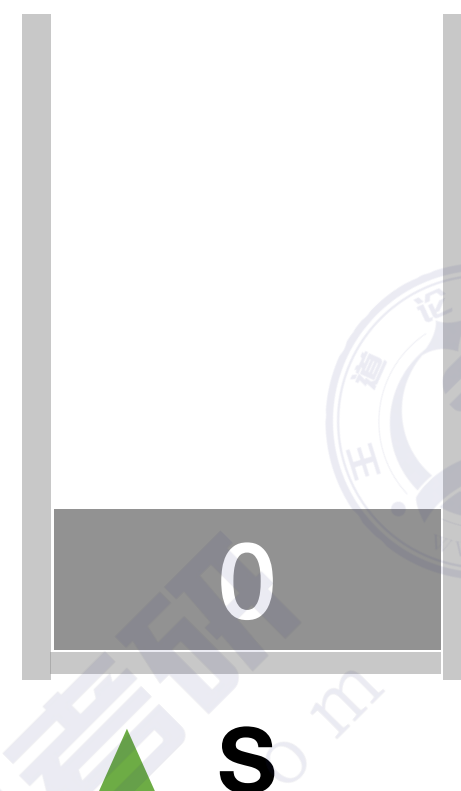
当前顶
点入度

print[]

2
-1
-1
-1
-1

记录拓
扑序列

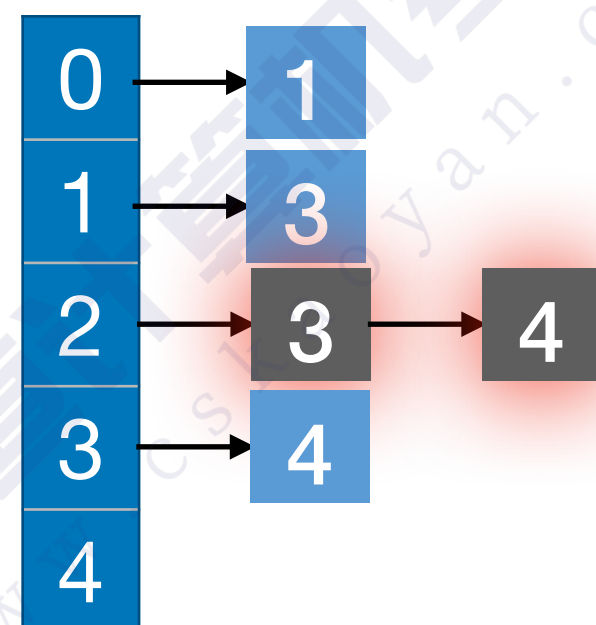
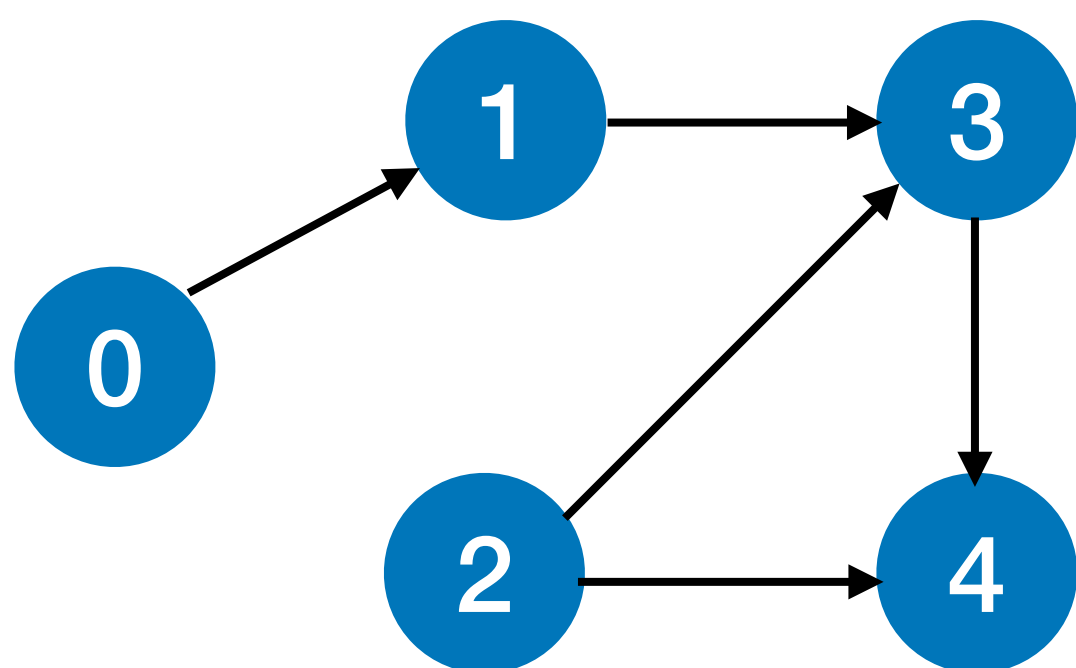
← count



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);      //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);  //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
1
0
2
2

当前顶
点入度

print[]

2
-1
-1
-1
-1

记录拓
扑序列

count

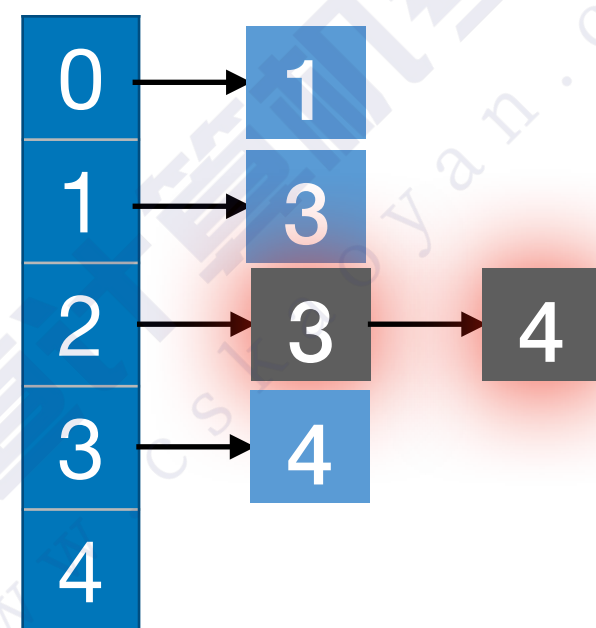
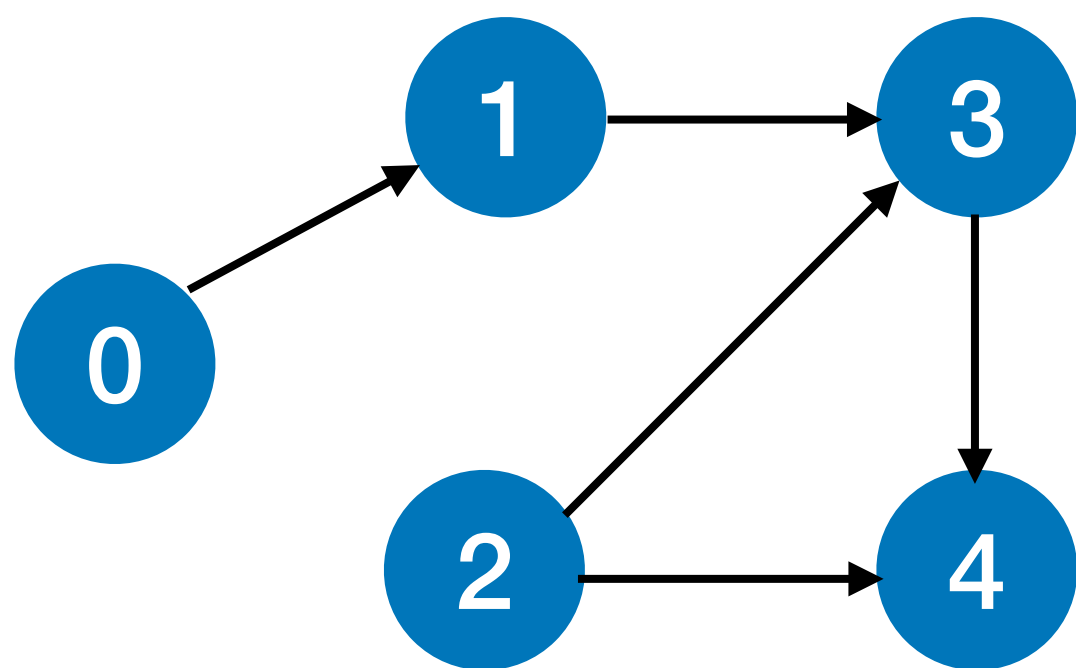
S



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
1
0
1
1

当前顶
点入度

print[]

2
-1
-1
-1
-1

记录拓
扑序列

count

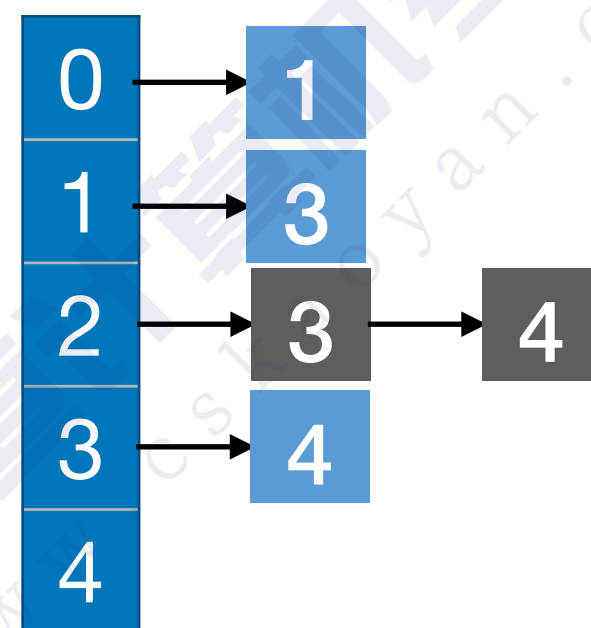
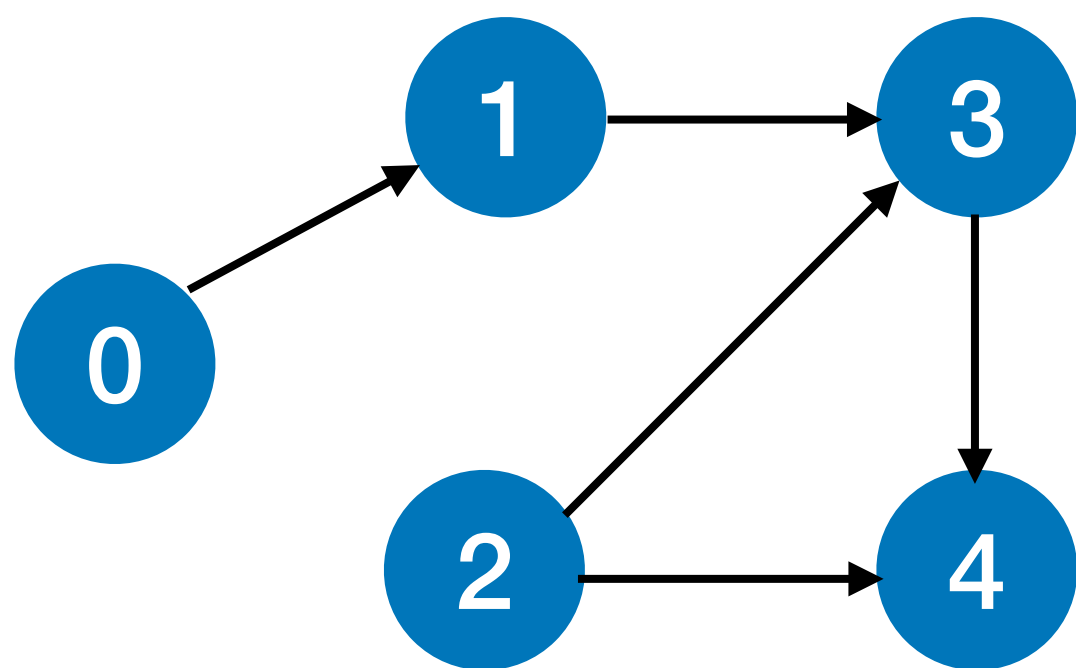
S



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
1
0
1
1

当前顶
点入度

print[]

2
-1
-1
-1
-1

记录拓
扑序列

count

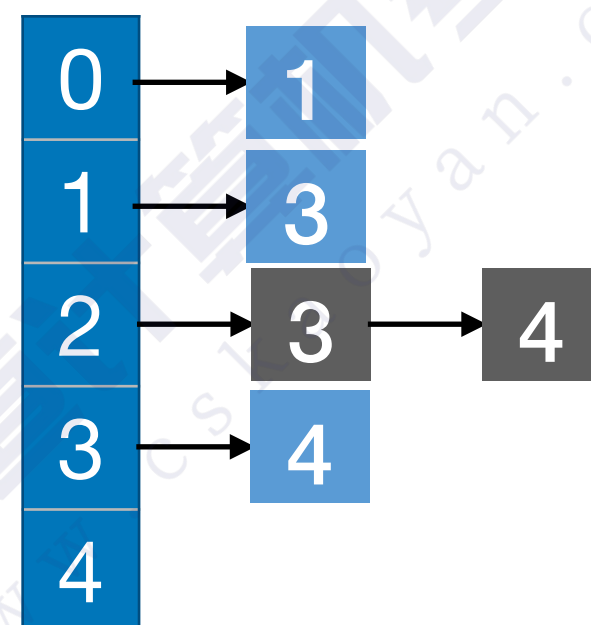
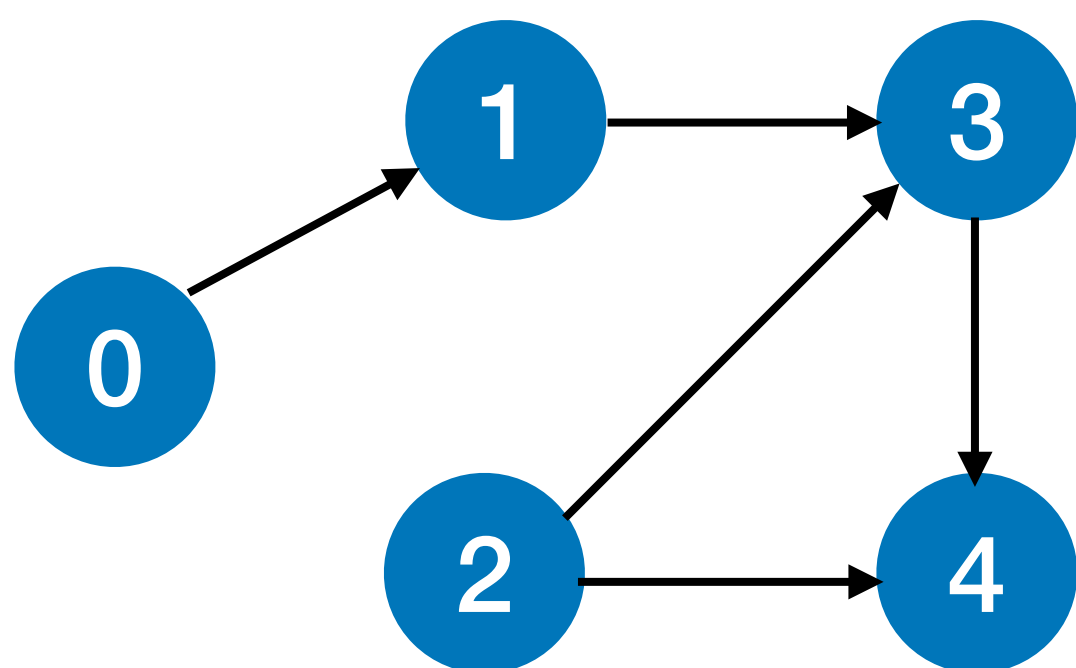
S



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);      //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);  //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
1
0
1
1

当前顶
点入度

print[]

2
0
-1
-1
-1

记录拓
扑序列

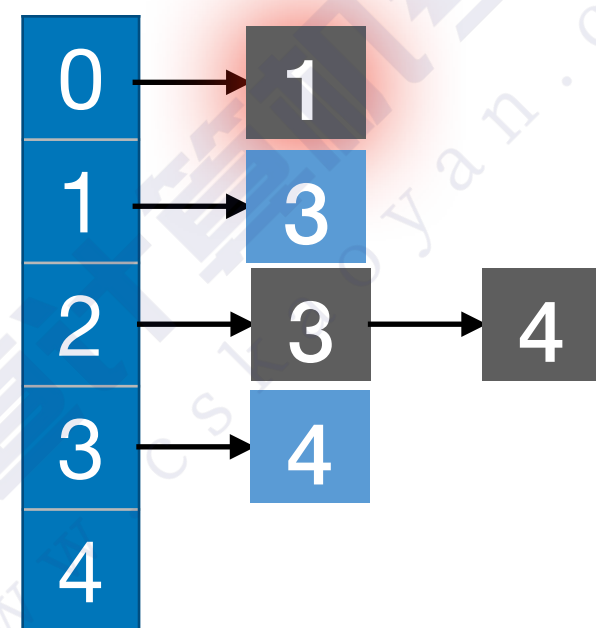
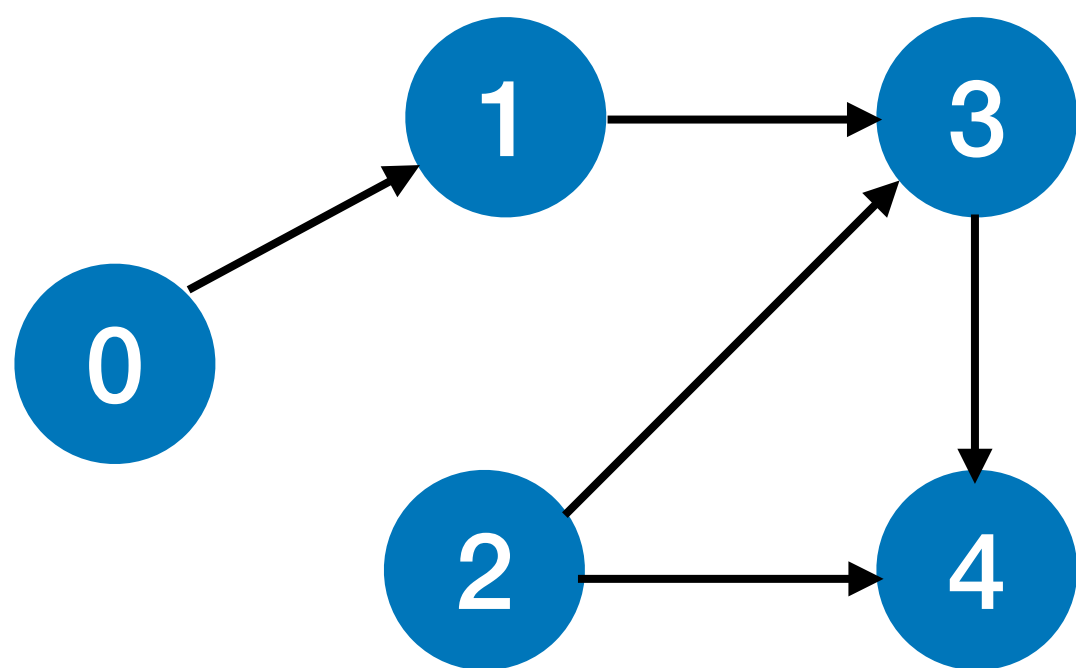
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;        //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```



indegree[]

0
1
0
1
1

当前顶
点入度

print[]

2
0
-1
-1
-1

记录拓
扑序列

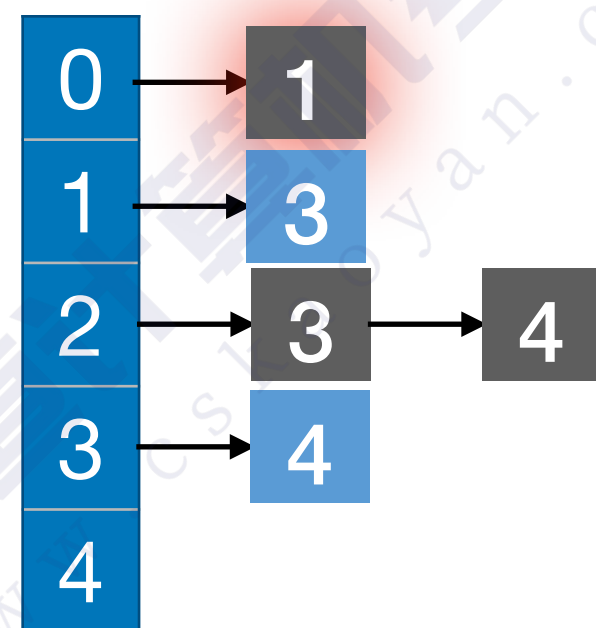
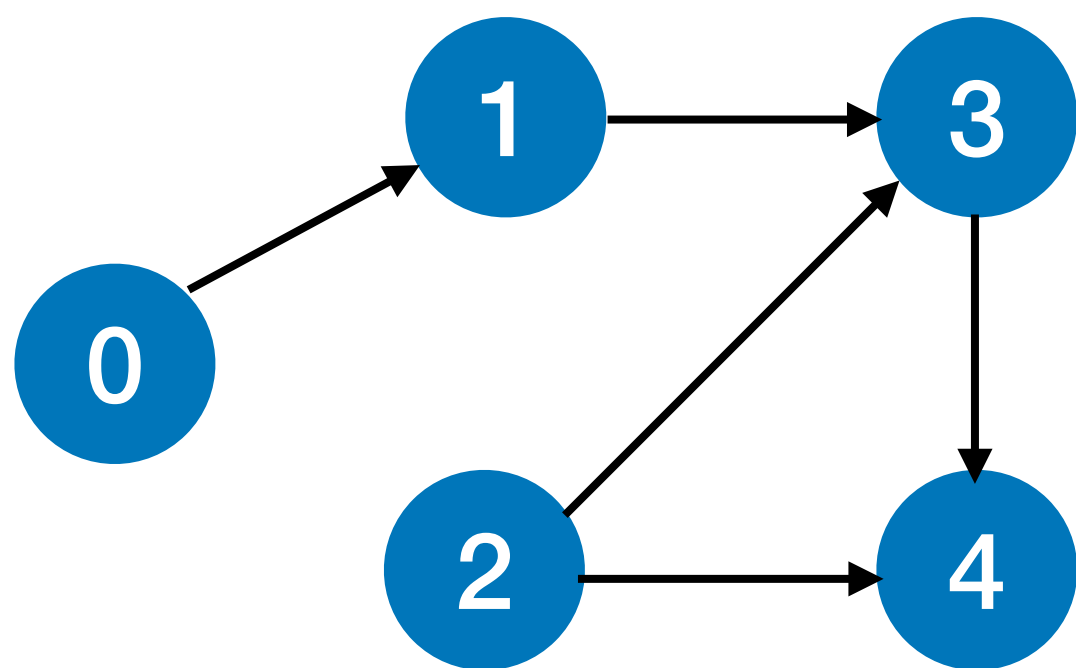
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
1
1

当前顶
点入度

print[]

2
0
-1
-1
-1

记录拓
扑序列

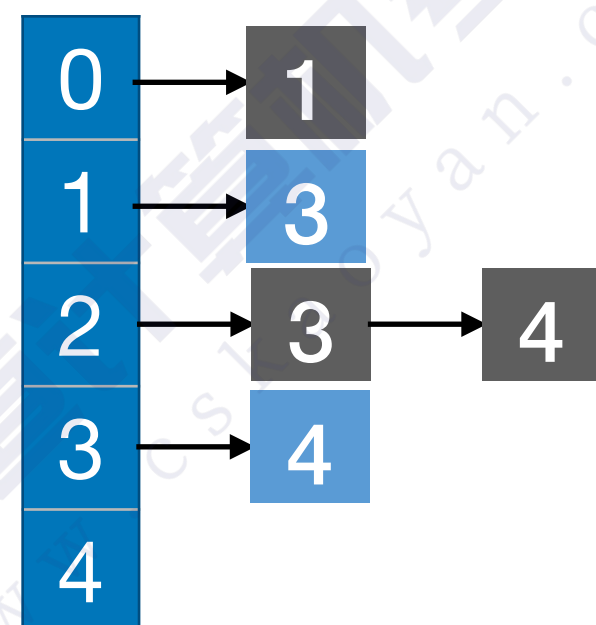
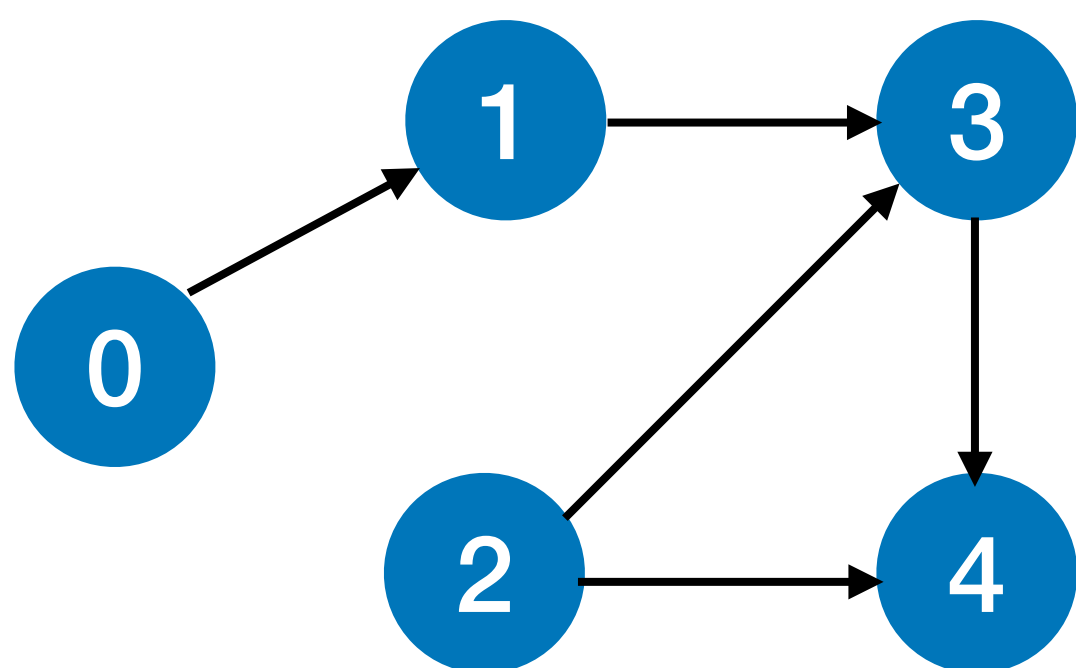
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;        //排序失败，有向图中有回路
    else
        return true;         //拓扑排序成功
}
  
```



indegree[]

0
0
0
1
1

当前顶
点入度

print[]

2
0
-1
-1
-1

记录拓
扑序列

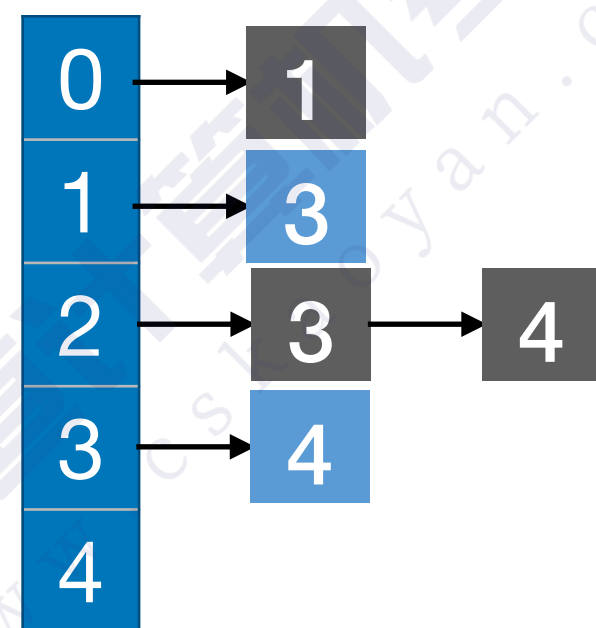
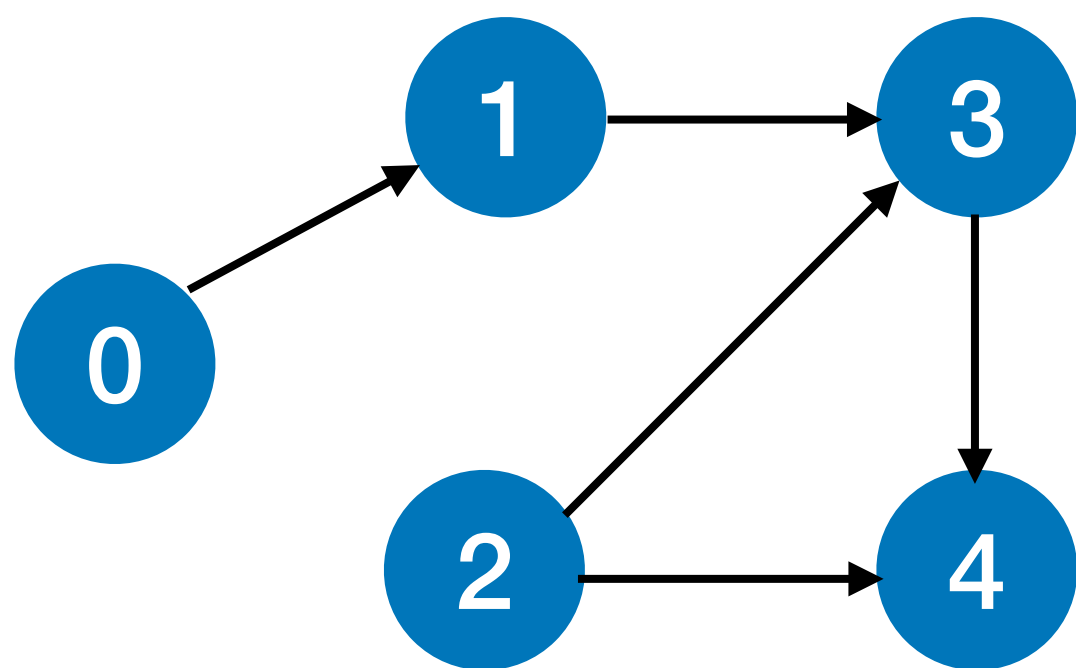
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
1
1

当前顶
点入度

print[]

2
0
1
-1
-1

记录拓
扑序列

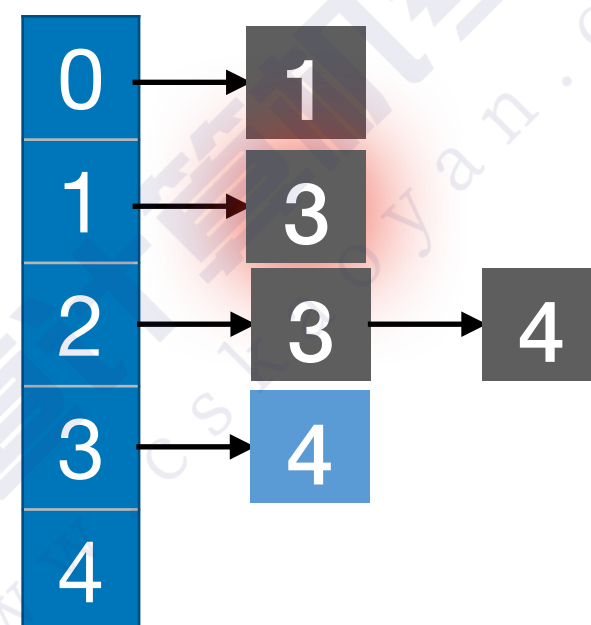
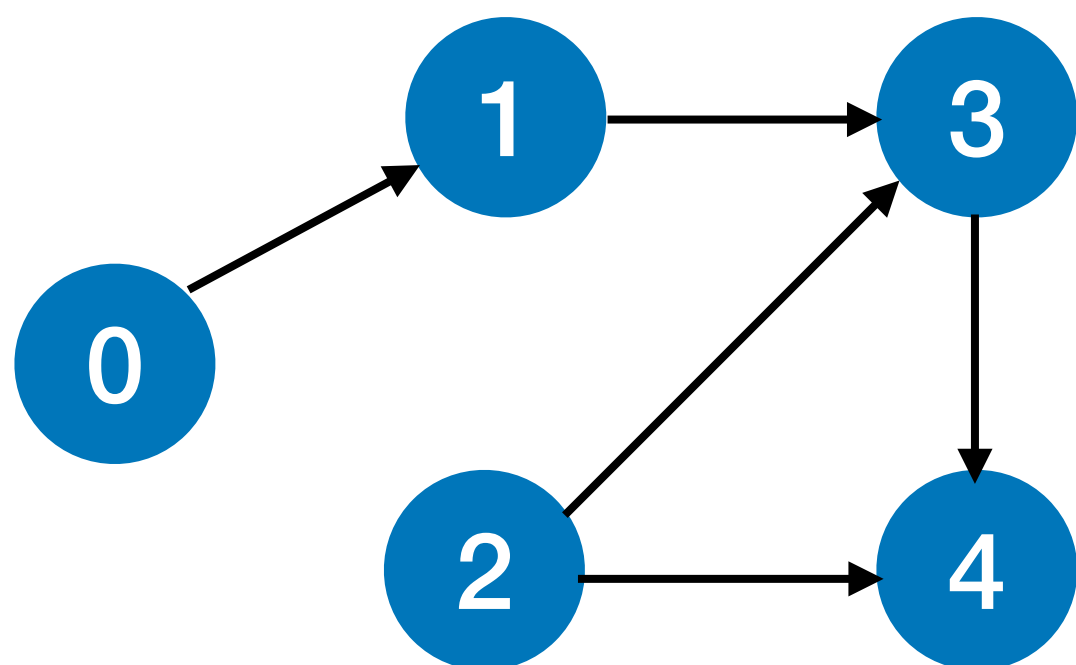
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```



indegree[]

0
0
0
1
1

当前顶
点入度

print[]

2
0
1
-1
-1

记录拓
扑序列

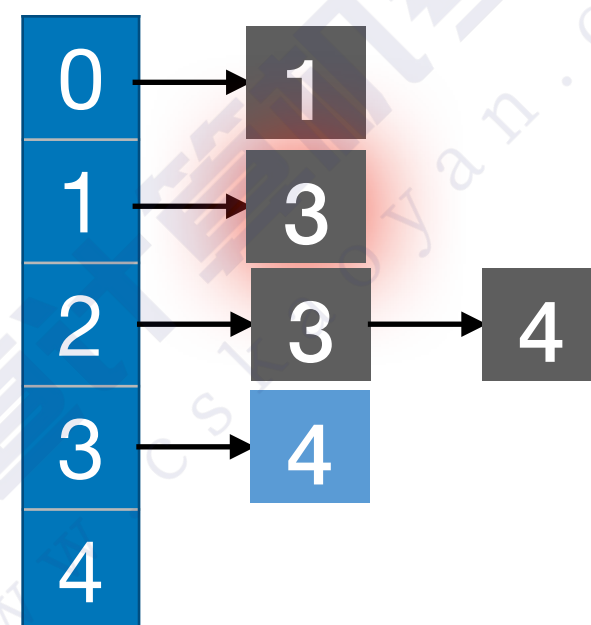
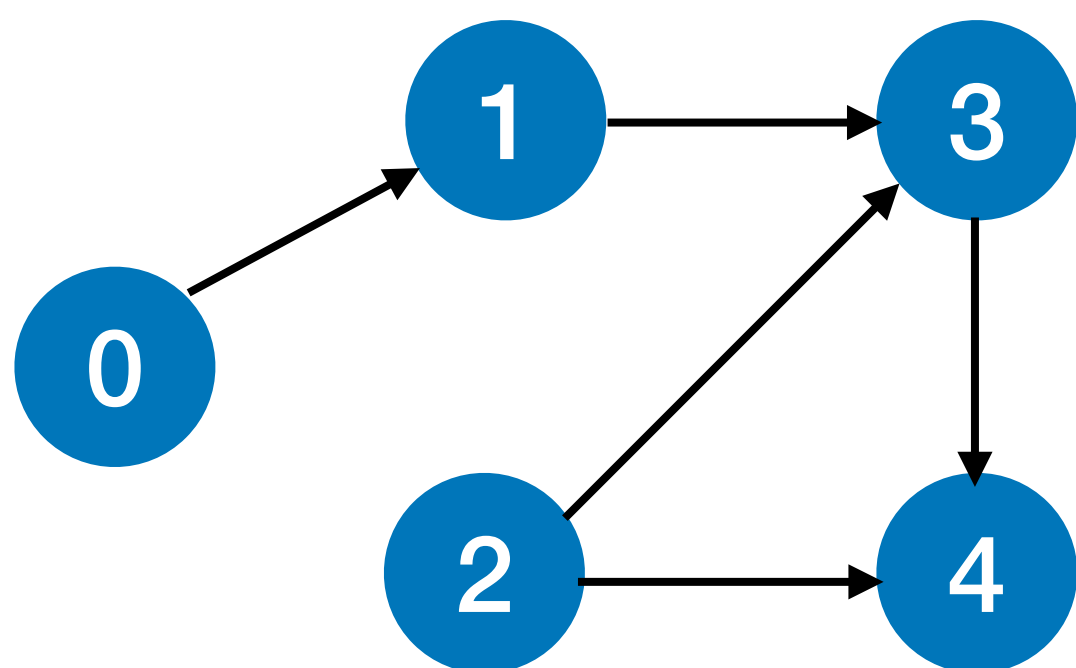
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);      //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);  //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
1

当前顶
点入度

print[]

2
0
1
-1
-1

记录拓
扑序列

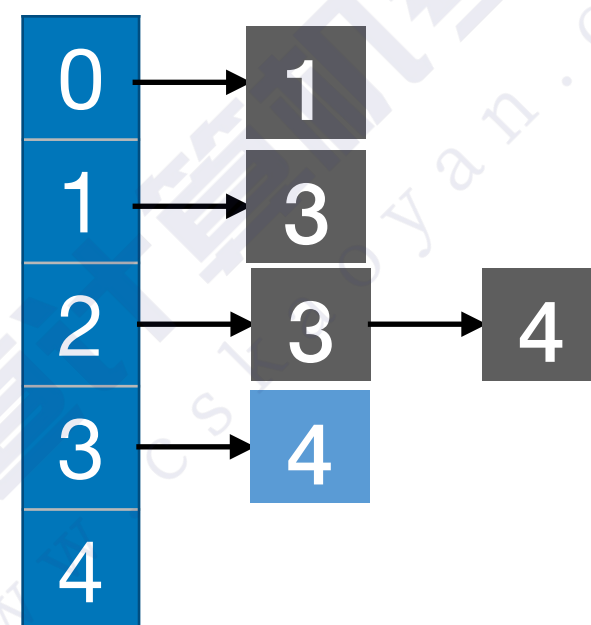
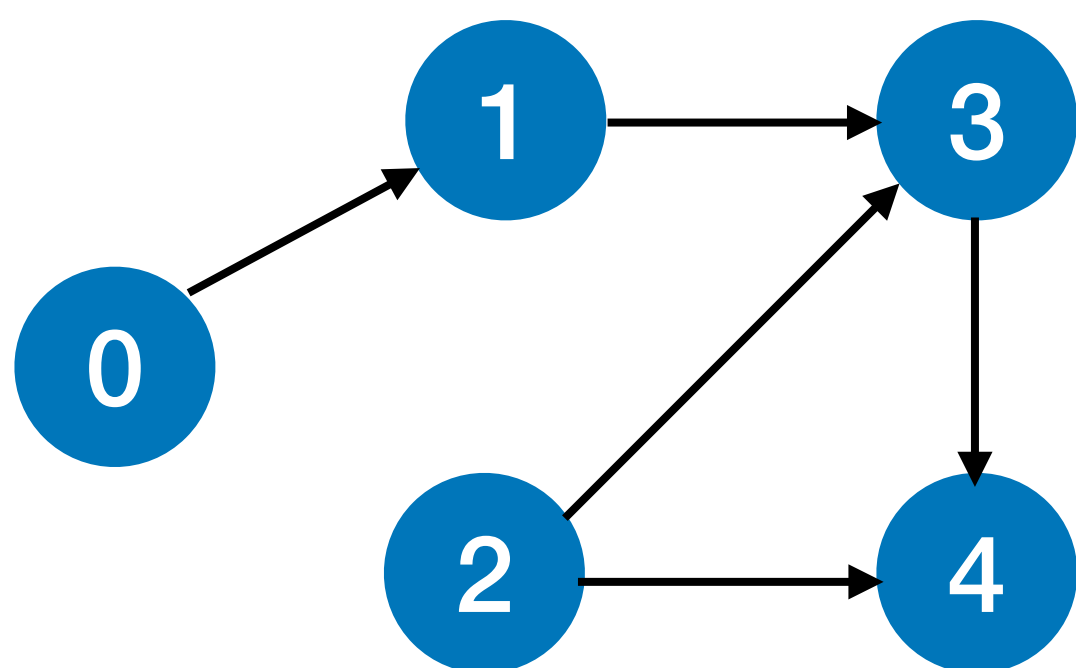
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```



indegree[]

0
0
0
0
1

当前顶
点入度

print[]

2
0
1
-1
-1

记录拓
扑序列

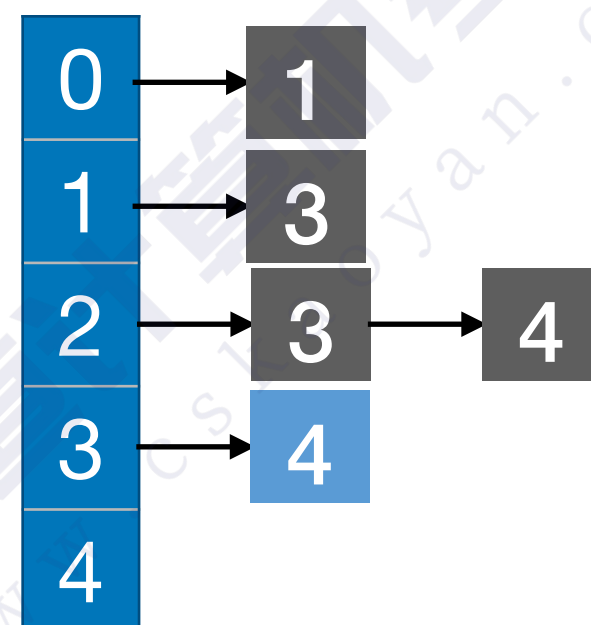
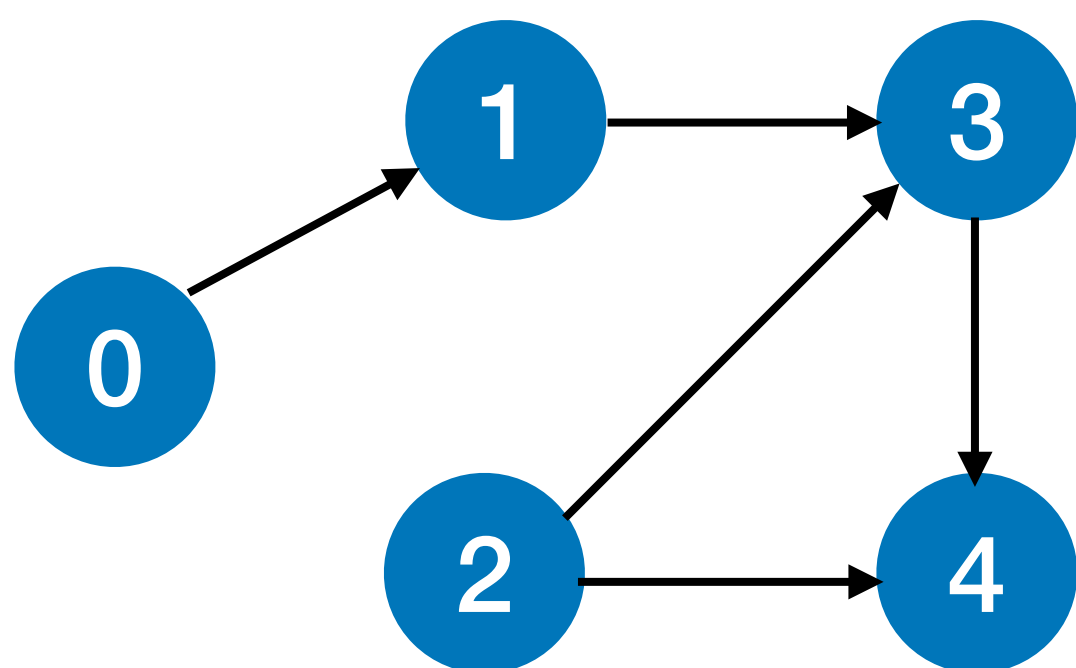
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
1

当前顶
点入度

print[]

2
0
1
3
-1

记录拓
扑序列

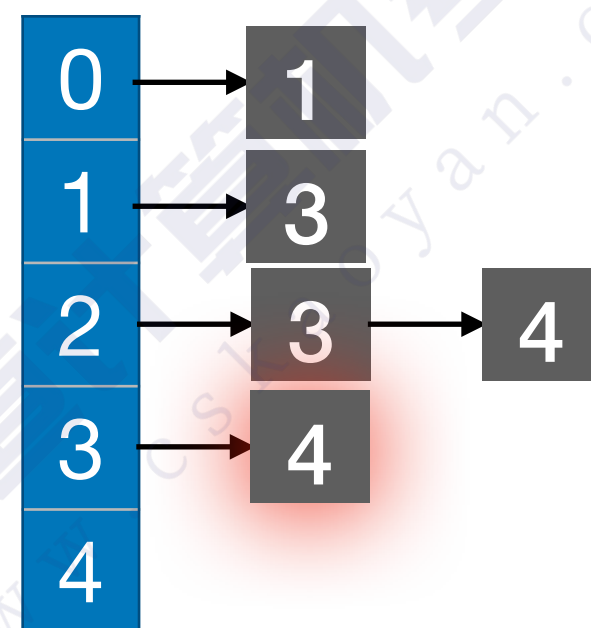
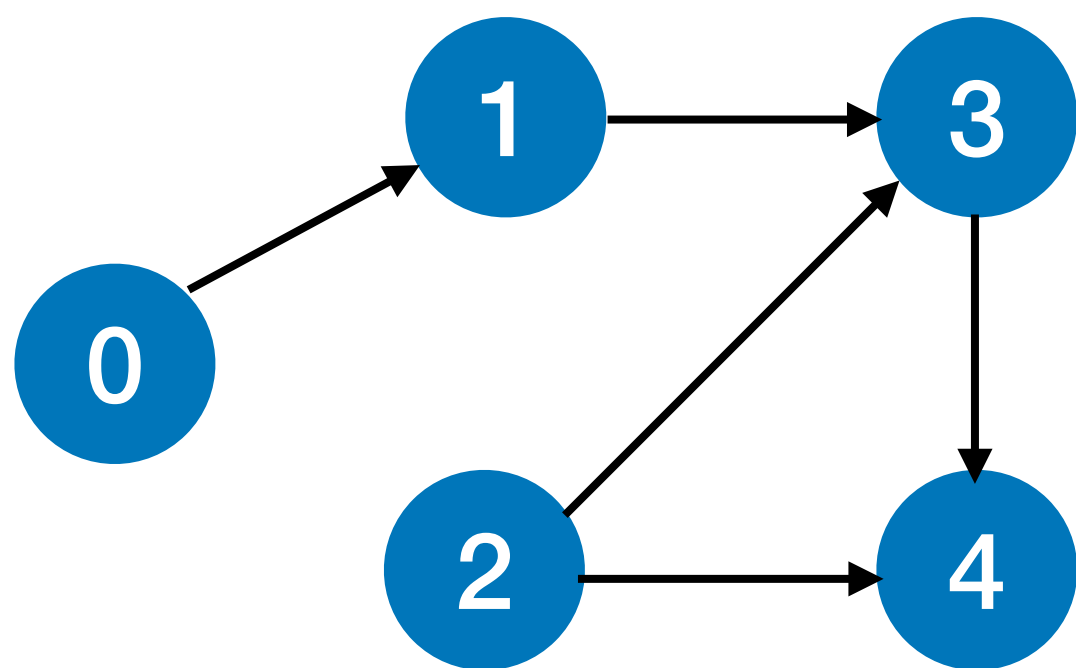
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);          //栈顶元素出栈
        print[count++]=i;   //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;      //排序失败，有向图中有回路
    else
        return true;       //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
1

当前顶
点入度

print[]

2
0
1
3
-1

记录拓
扑序列

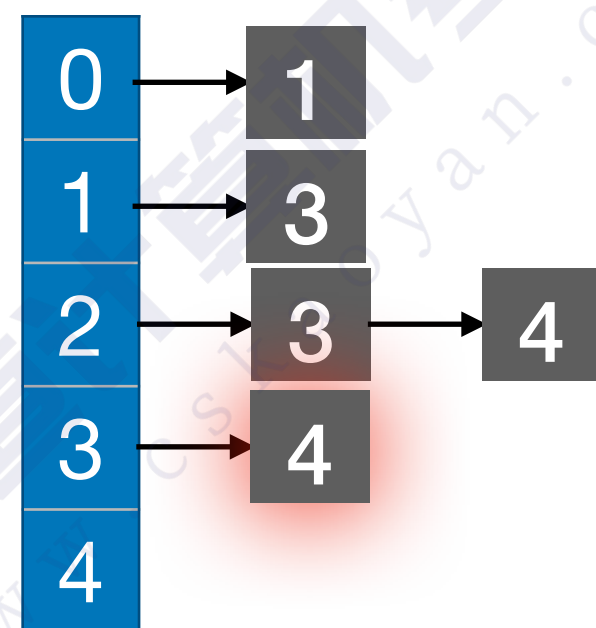
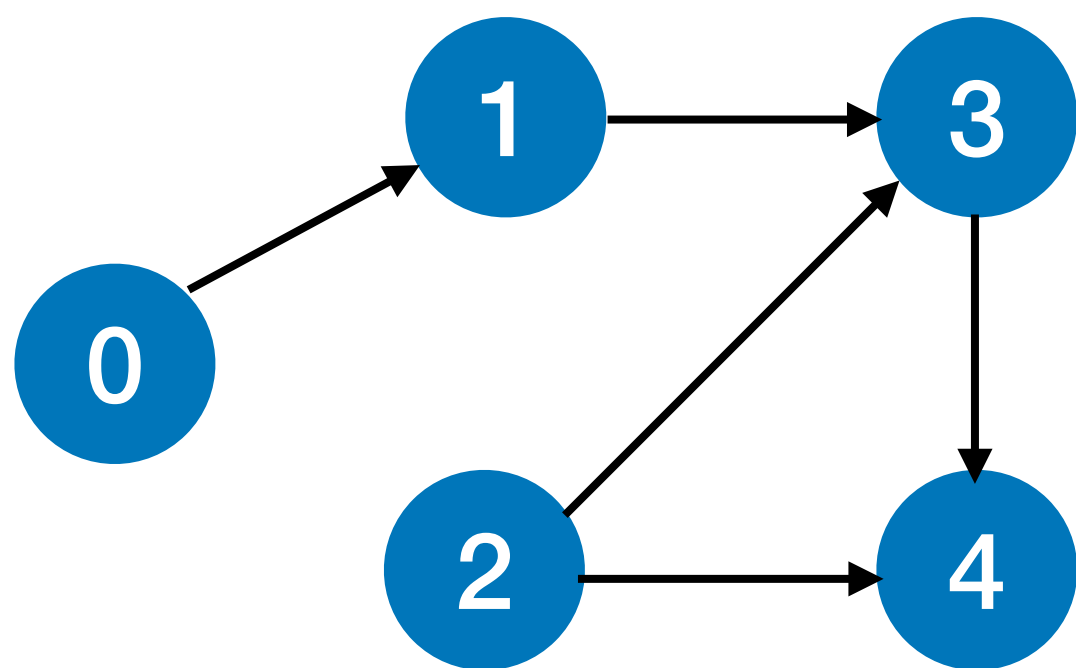
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;        //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){    //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
0

当前顶
点入度

print[]

2
0
1
3
-1

记录拓
扑序列

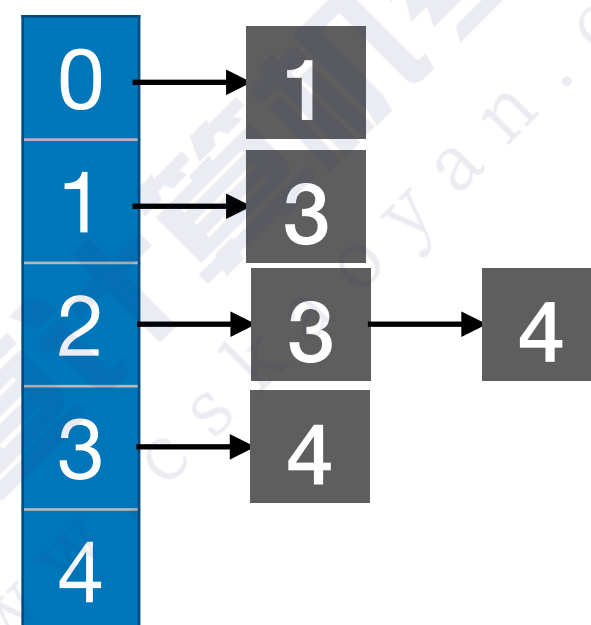
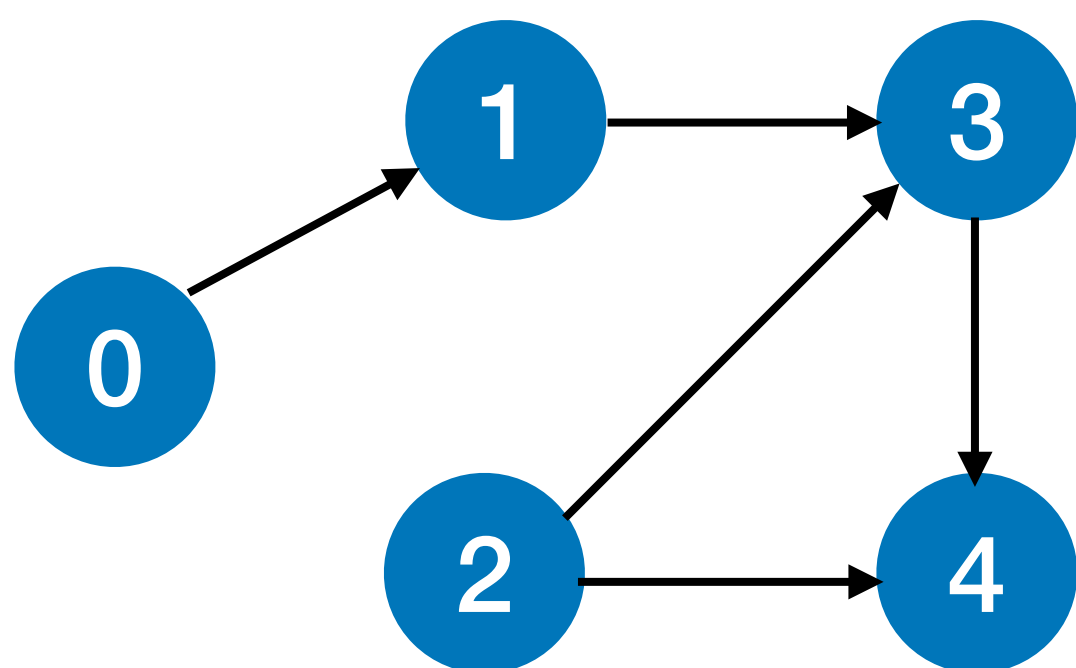
count

S

保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
0

当前顶
点入度

print[]

2
0
1
3
-1

记录拓
扑序列

count

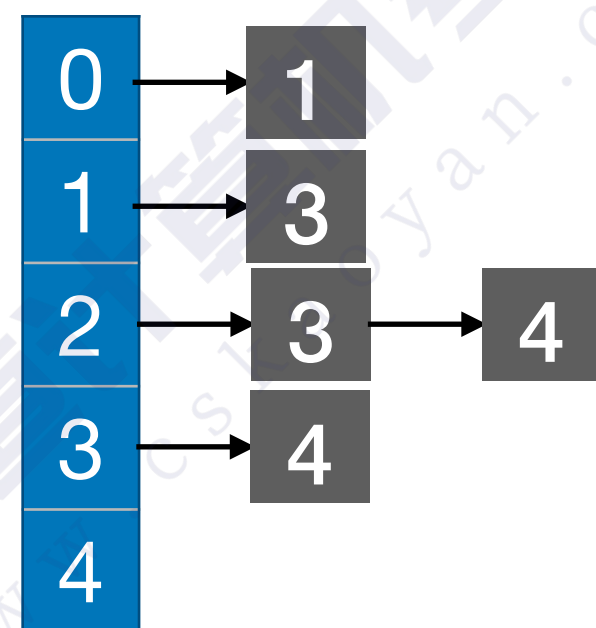
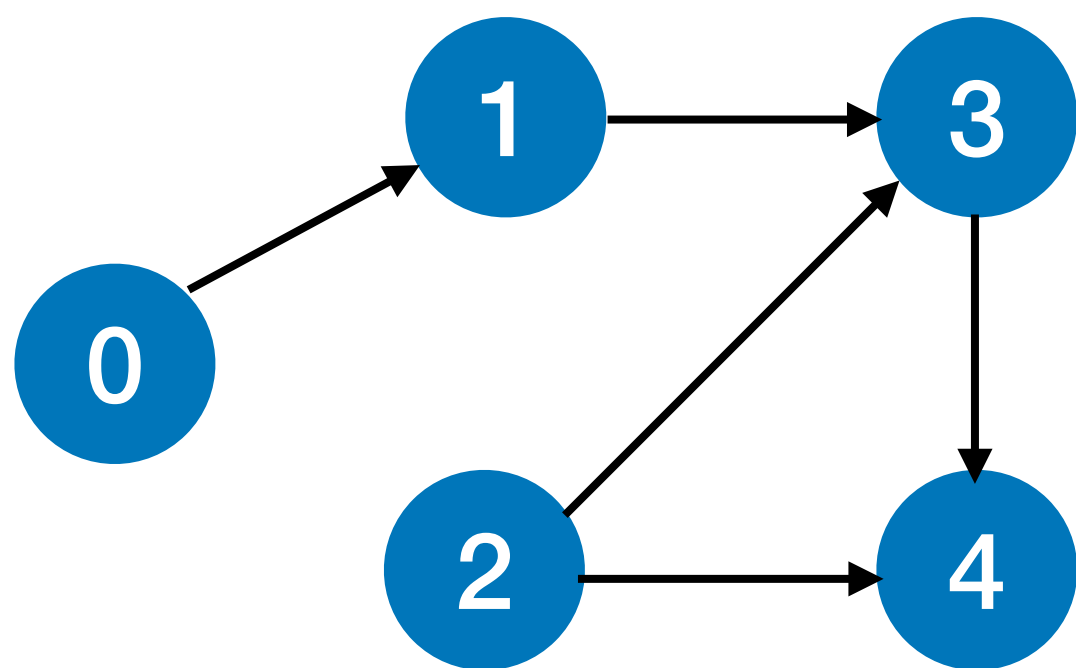
S



保存度为0的顶点
(也可用队列)

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
    }
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);    //入度为0，则入栈
        }
    } //while
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```

indegree[]

0
0
0
0
0

当前顶
点入度

print[]

2
0
1
3
4

记录拓
扑序列

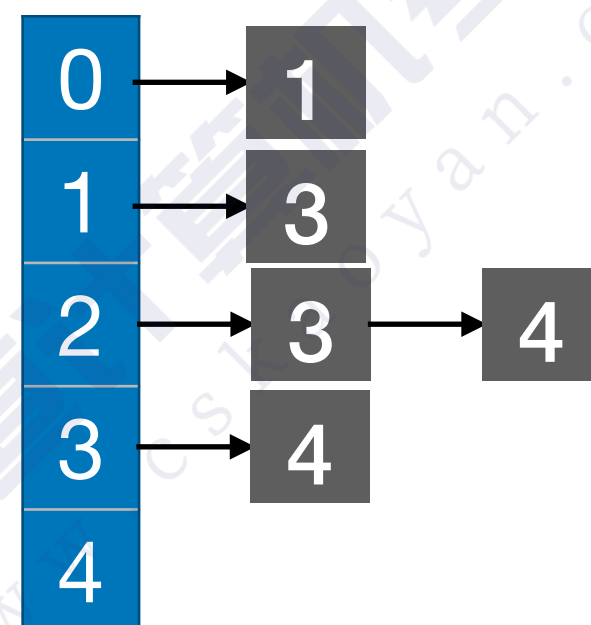
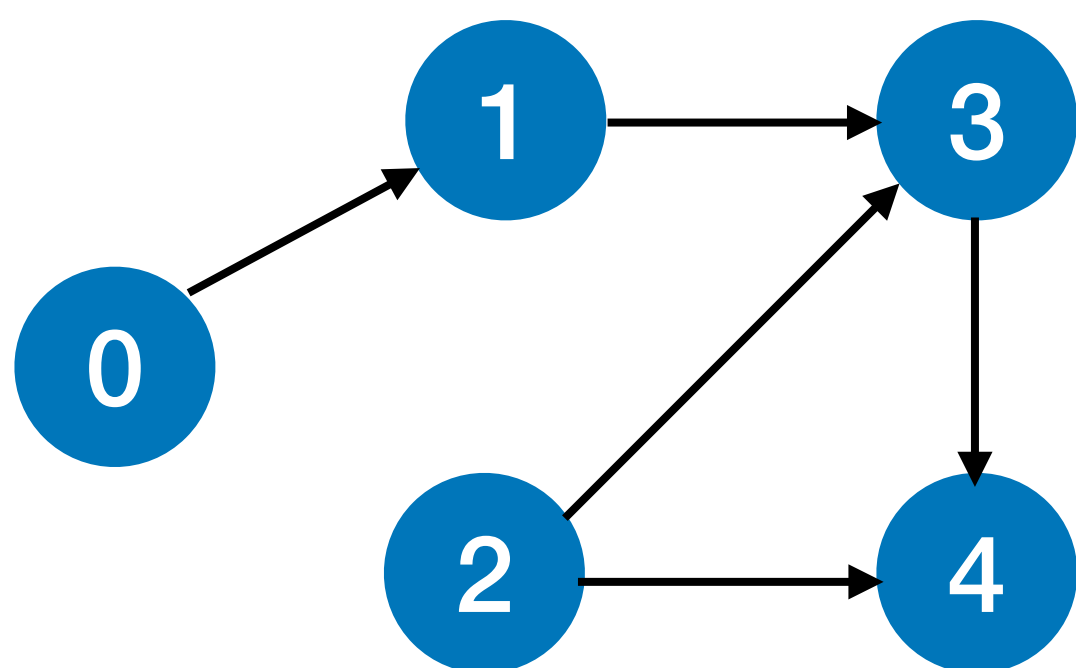
count

保存度为0的顶点
(也可用队列)

S

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
    }
    int count=0;           //计数，记录当前已经输出的顶点数
    while(!IsEmpty(S)){     //栈不空，则存在入度为0的顶点
        Pop(S,i);           //栈顶元素出栈
        print[count++]=i;    //输出顶点i
        for(p=G.vertices[i].firstarc;p;p=p->nextarc){
            //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
            v=p->adjvex;
            if(!(--indegree[v]))
                Push(S,v);   //入度为0，则入栈
        }
    }
    if(count<G.vexnum)
        return false;       //排序失败，有向图中有回路
    else
        return true;        //拓扑排序成功
}
  
```



每个顶点都需要处理一次

每条边都需要处理一次

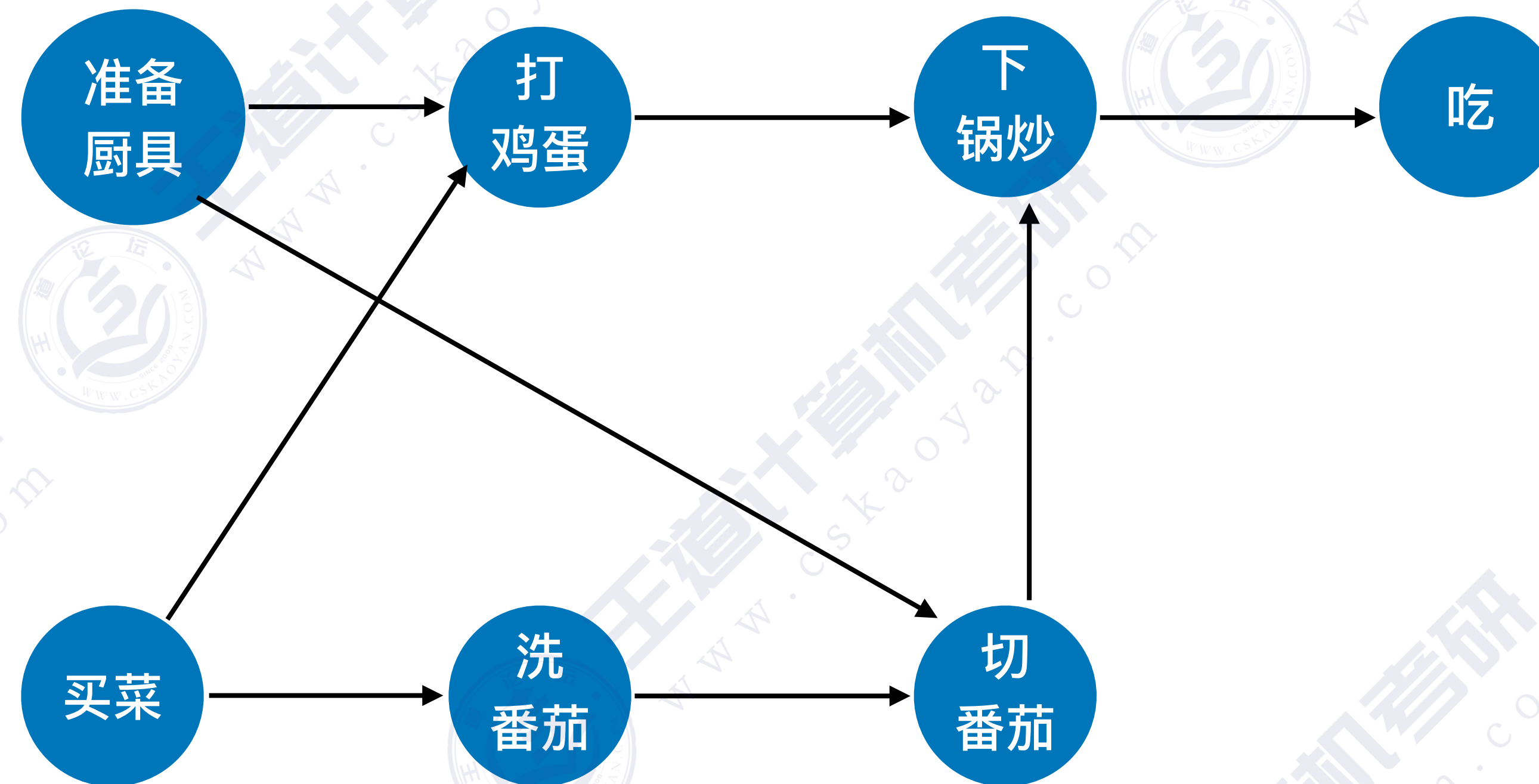
时间复杂度: $O(|V|+|E|)$

若采用邻接矩阵, 则需 $O(|V|^2)$

```

bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈, 存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数, 记录当前已经输出的顶点数
        while(!IsEmpty(S)){  //栈不空, 则存在入度为0的顶点
            Pop(S,i);         //栈顶元素出栈
            print[count++]=i;  //输出顶点i
            for(p=G.vertices[i].firstarc;p;p=p->nextarc){
                //将所有i指向的顶点的入度减1, 并且将入度减为0的顶点压入栈s
                v=p->adjvex;
                if(!(--indegree[v]))
                    Push(S,v); //入度为0, 则入栈
            }
        } //while
        if(count<G.vexnum)
            return false;     //排序失败, 有向图中有回路
        else
            return true;       //拓扑排序成功
    }
}
  
```

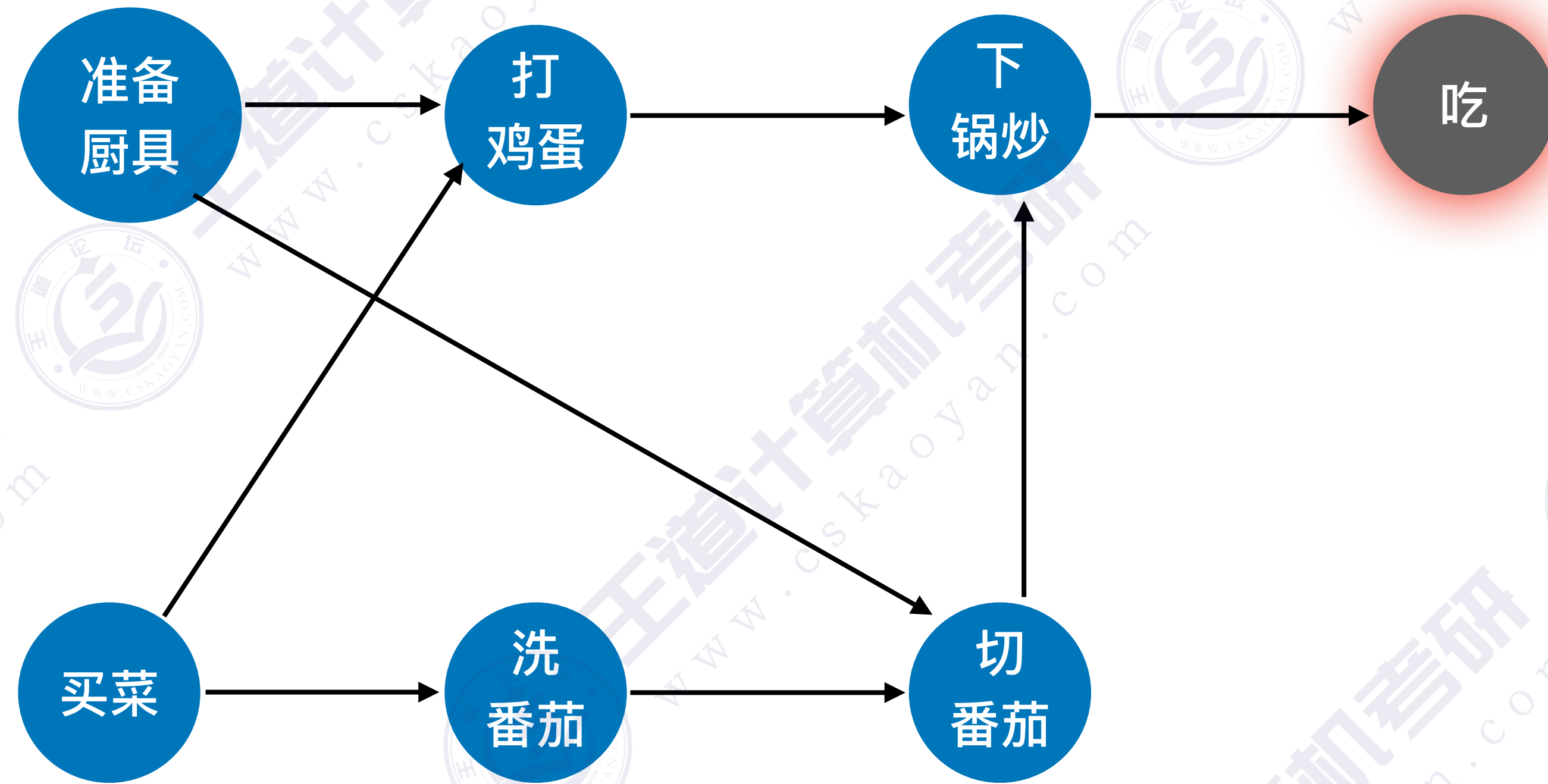

逆拓扑排序



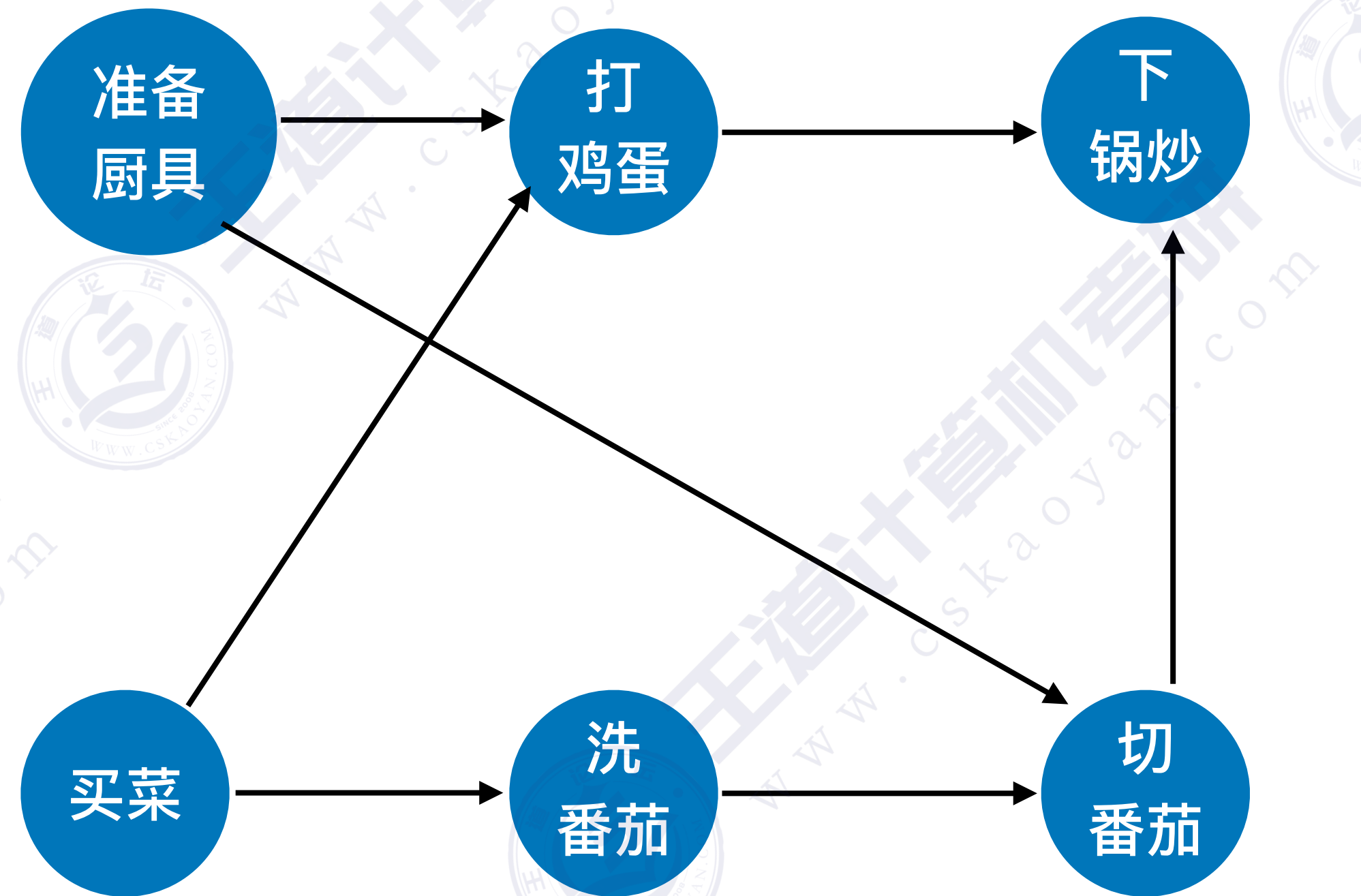
对一个AOV网，如果采用下列步骤进行排序，则称之为**逆拓扑排序**：

- ① 从AOV网中选择一个没有后继（**出度为0**）的顶点并输出。
- ② 从网中删除该顶点和所有以它为终点的有向边。
- ③ 重复①和②直到当前的AOV网为空。

逆拓扑排序

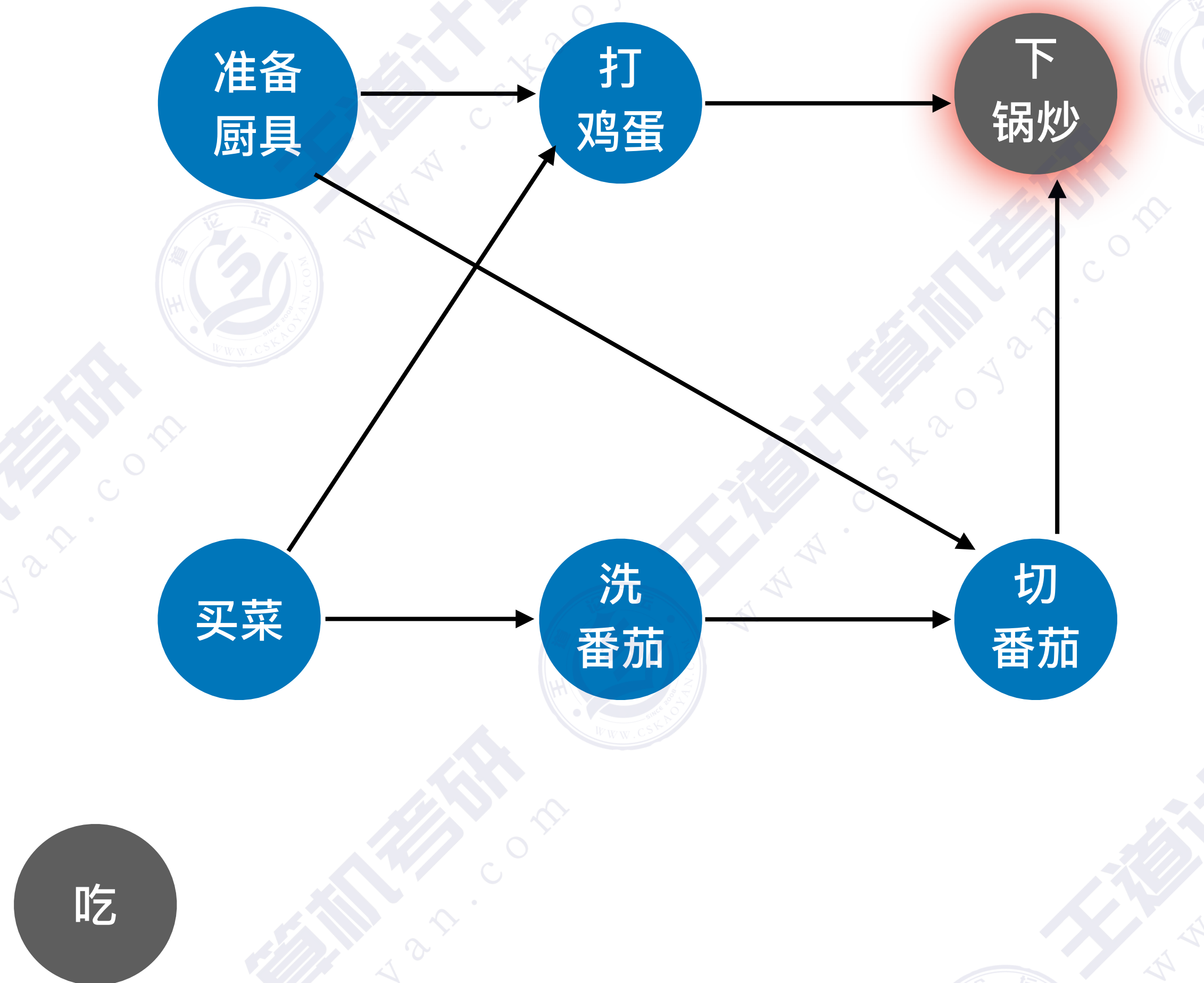


逆拓扑排序

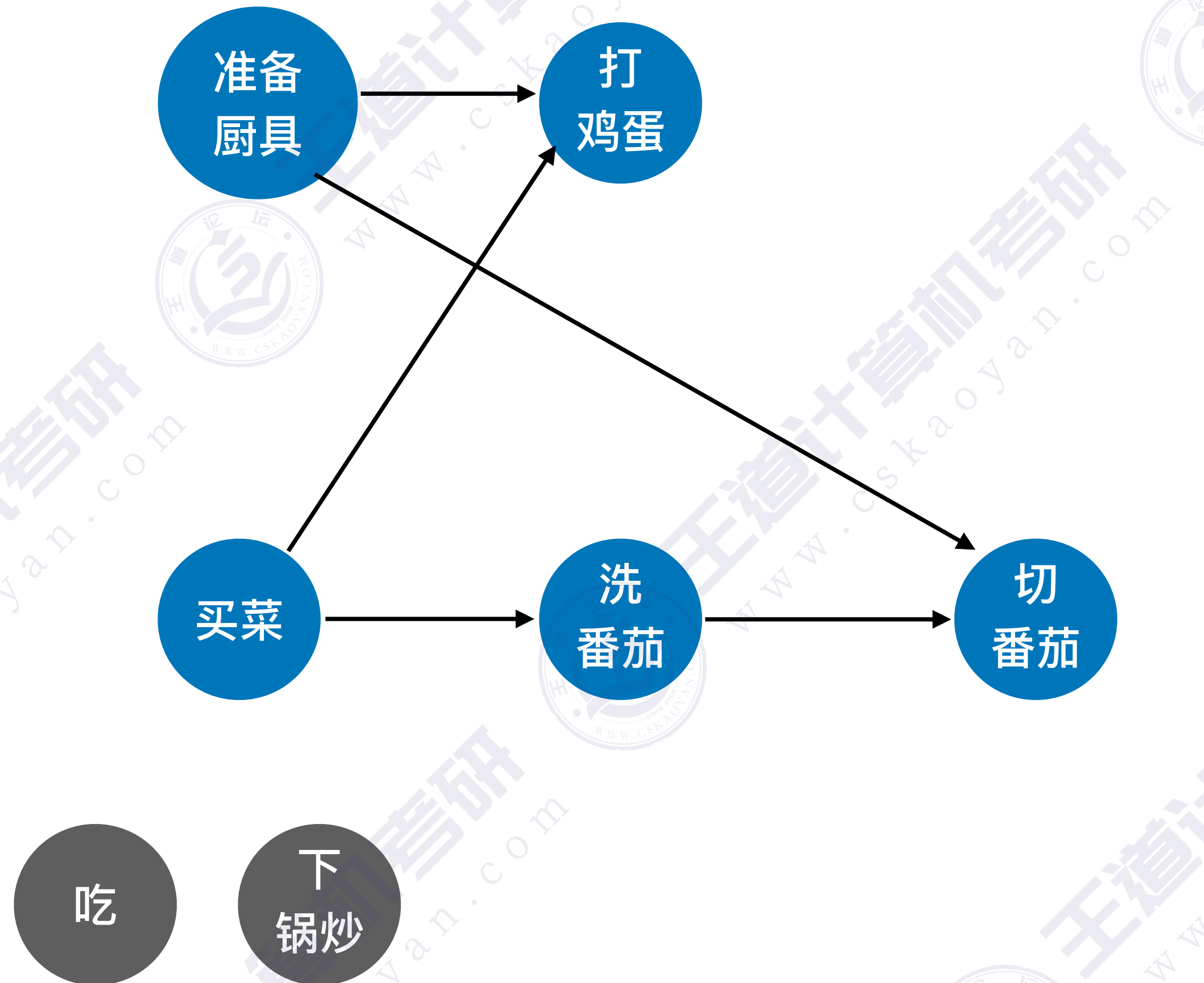


吃

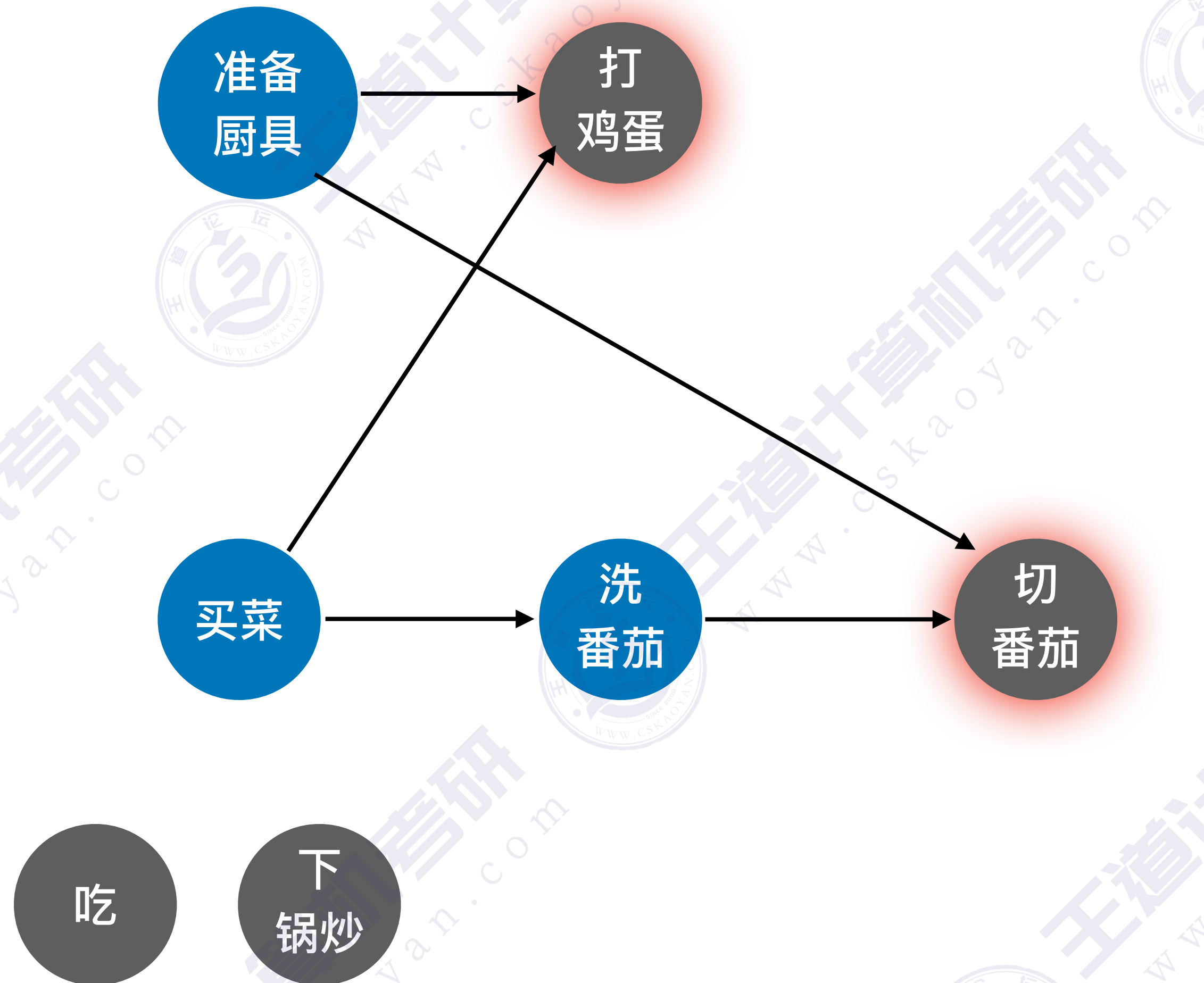
逆拓扑排序



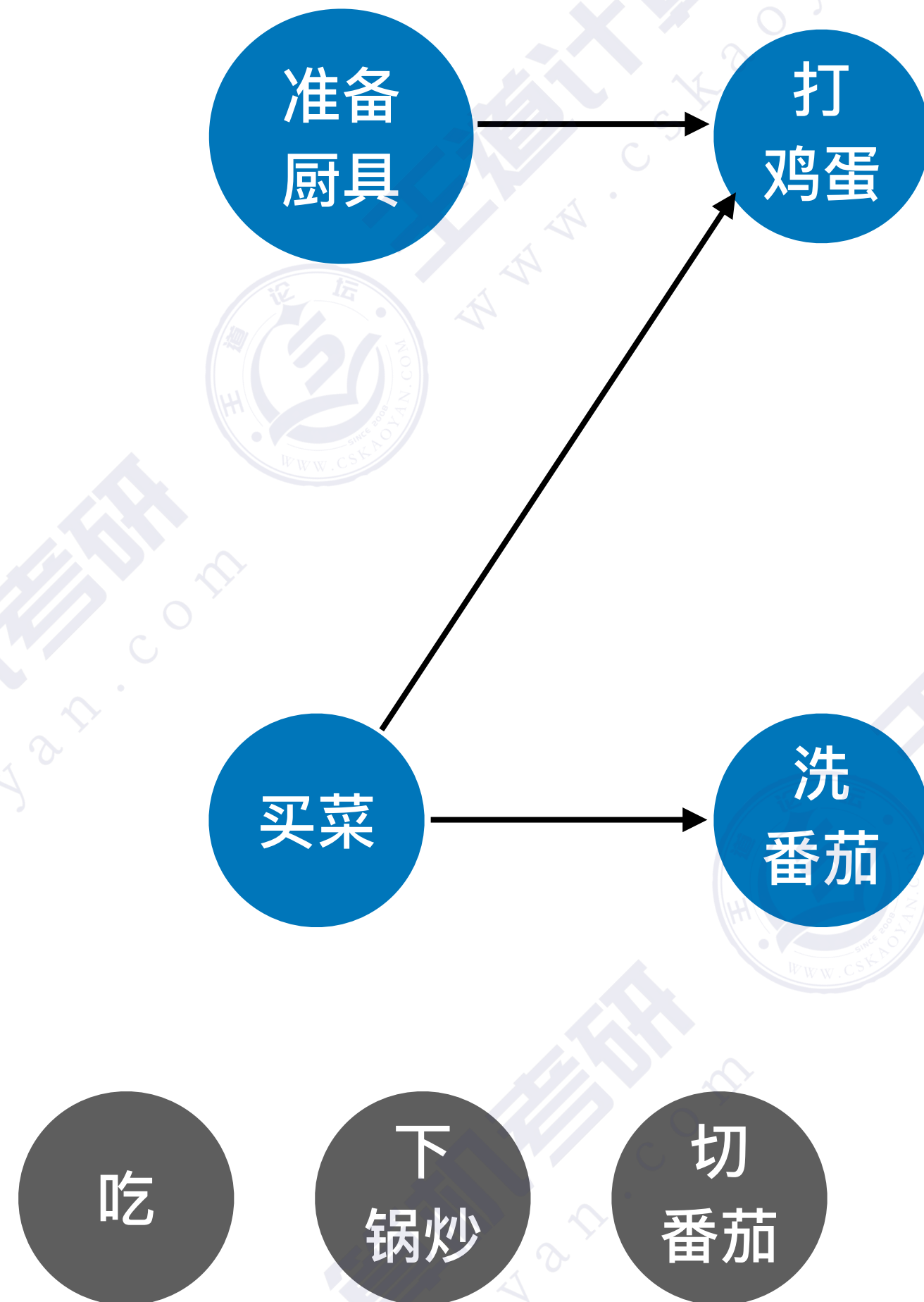
逆拓扑排序



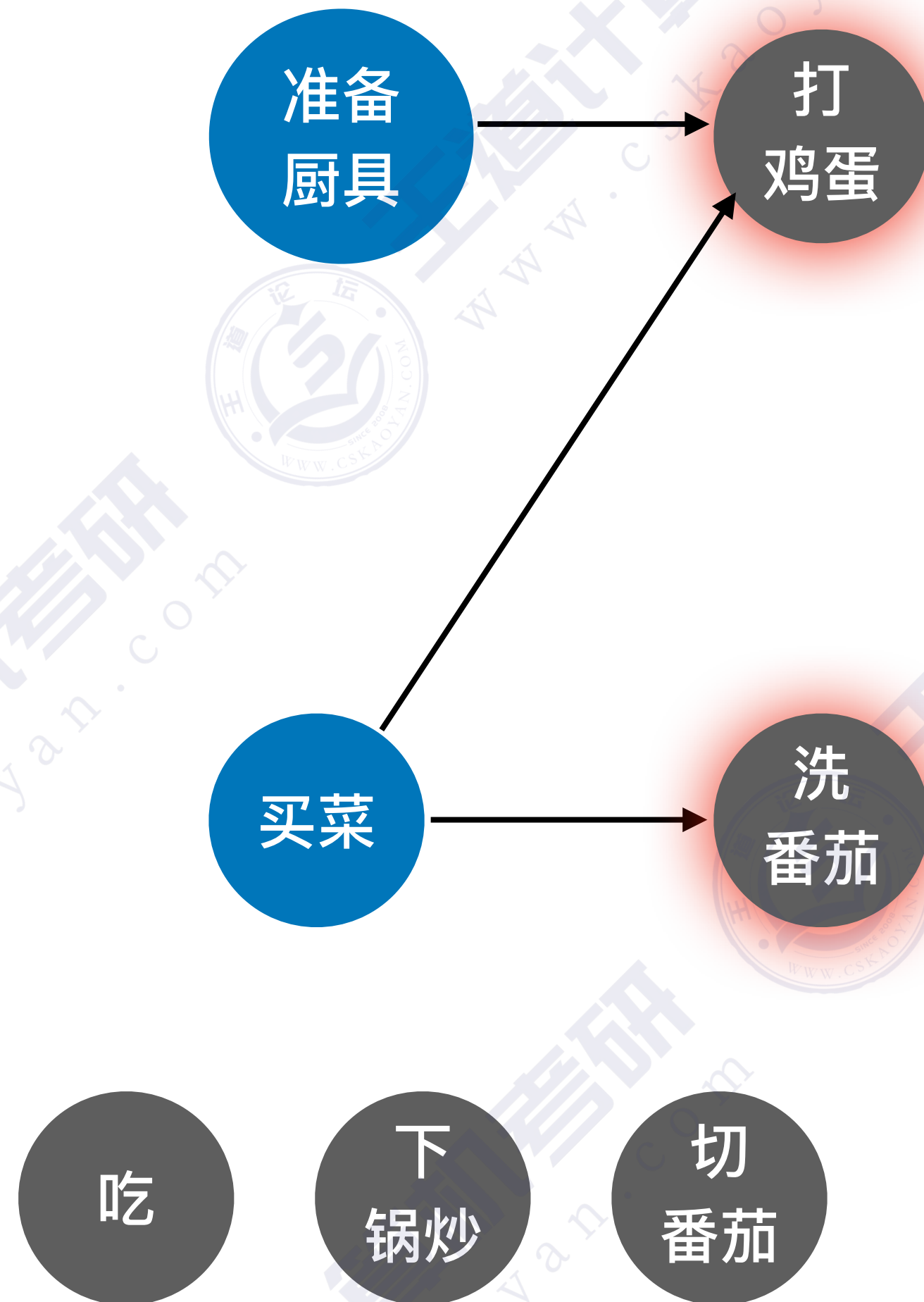
逆拓扑排序



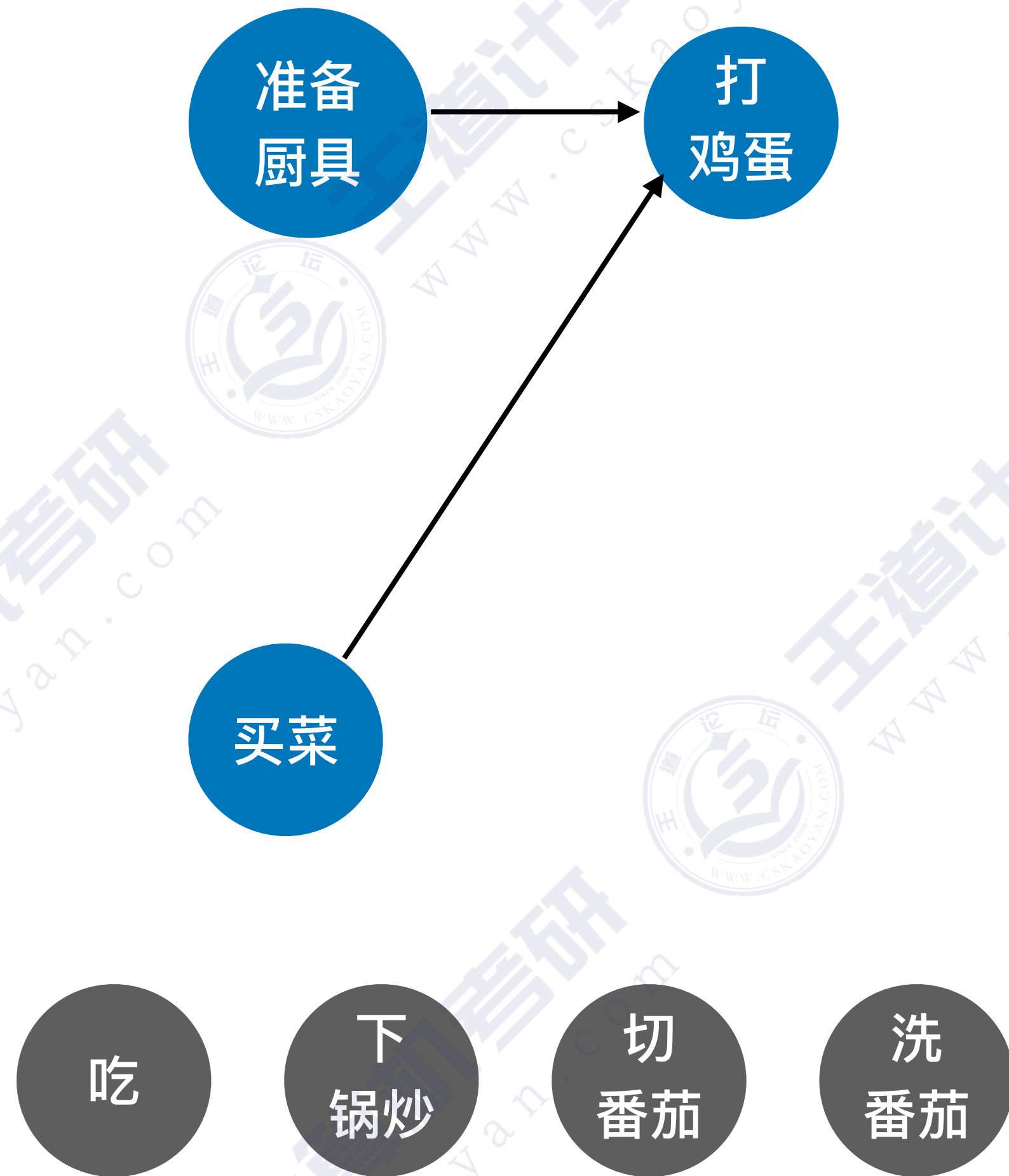
逆拓扑排序



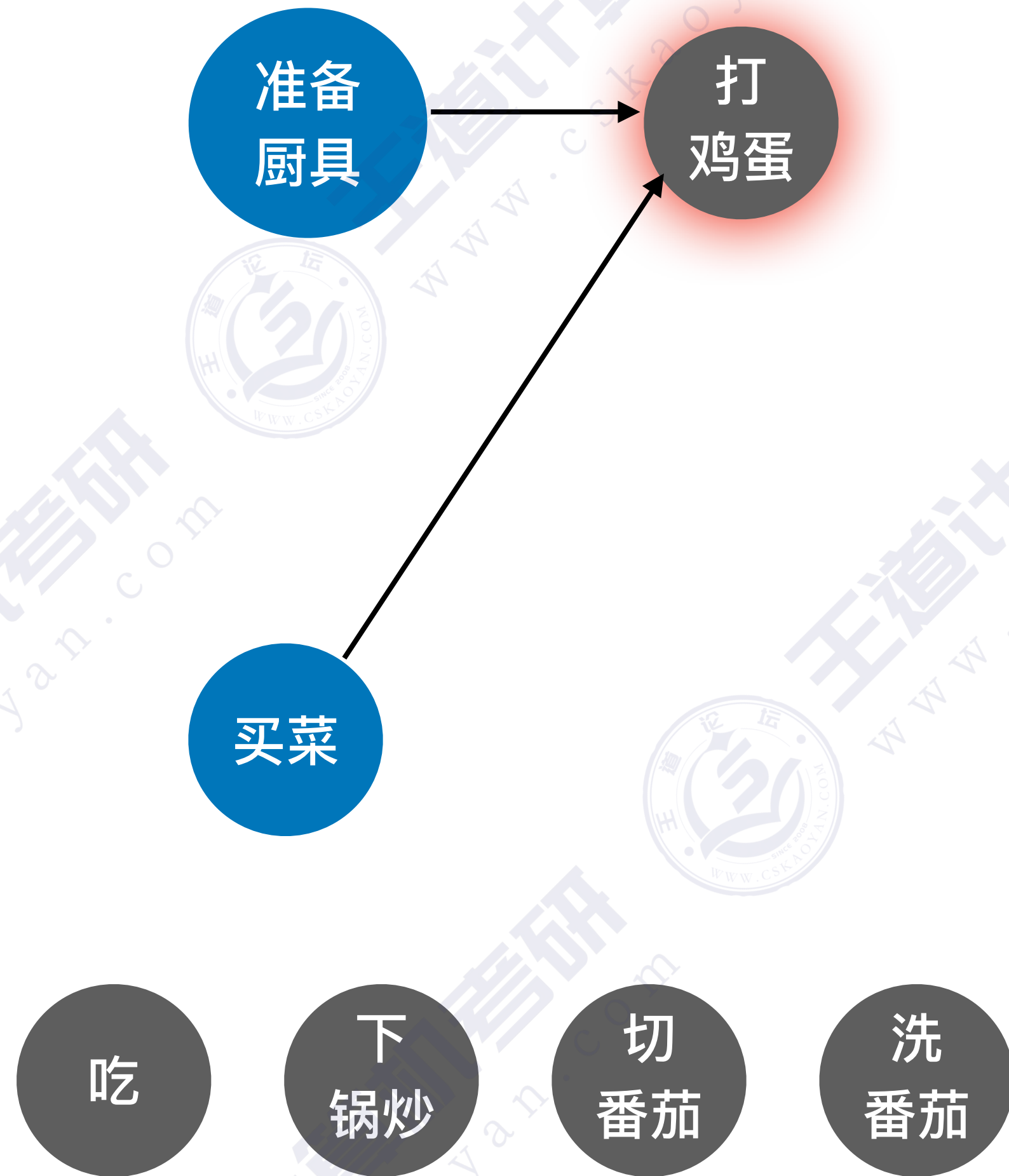
逆拓扑排序



逆拓扑排序



逆拓扑排序



逆拓扑排序



准备
厨具

买菜

吃

下
锅炒

切
番茄

洗
番茄

打
鸡蛋

逆拓扑排序



准备
厨具

买菜

吃

下
锅炒

切
番茄

洗
番茄

打
鸡蛋

逆拓扑排序



对一个AOV网**逆拓扑排序**:

- ① 从AOV网中选择一个没有后继（**出度为0**）的顶点并输出。
- ② 从网中删除该顶点和所有以它为终点的有向边。
- ③ 重复①和②直到当前的AOV网为空。

吃

下
锅炒

切
番茄

洗
番茄

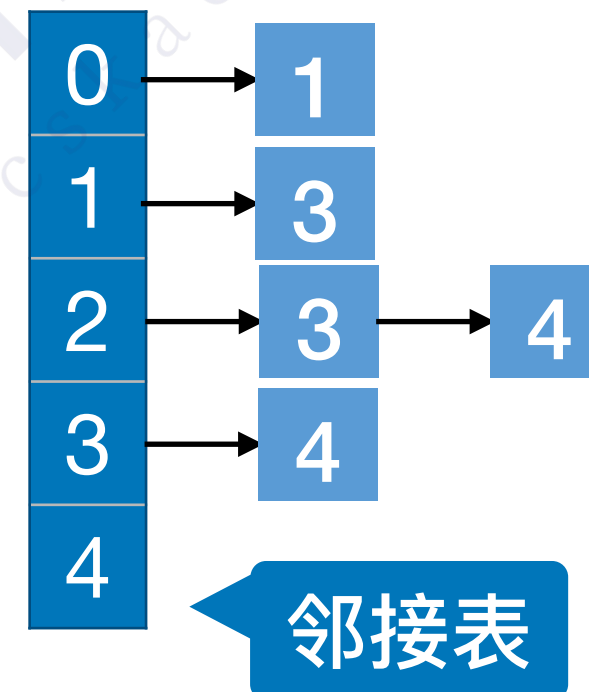
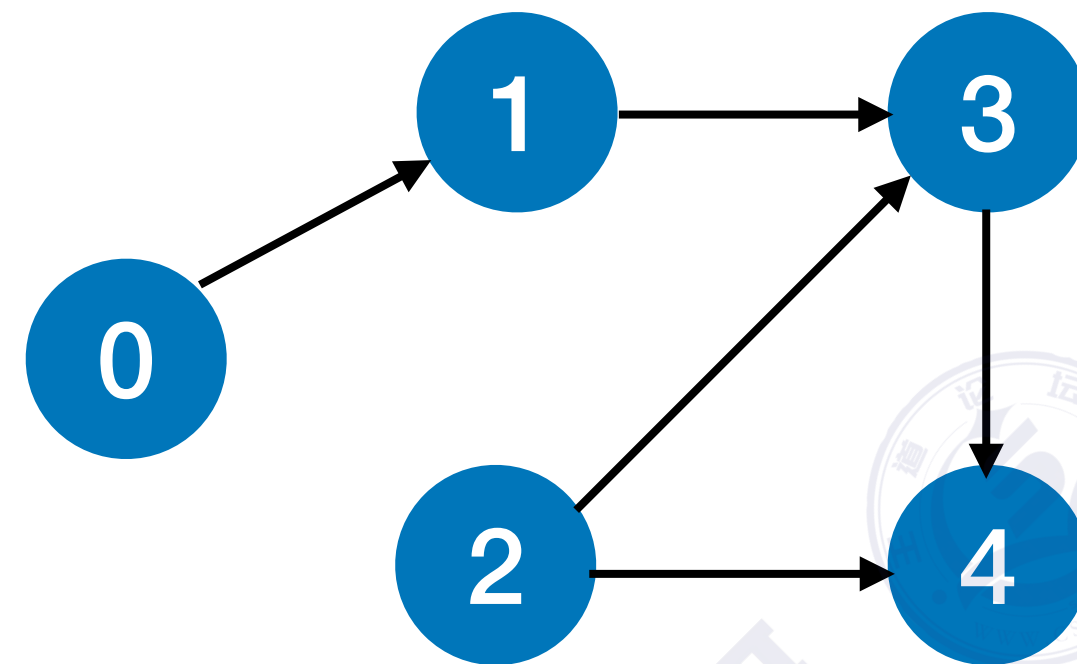
打
鸡蛋

准备
厨具

买菜

逆拓扑排序的实现

拓扑排序的实现

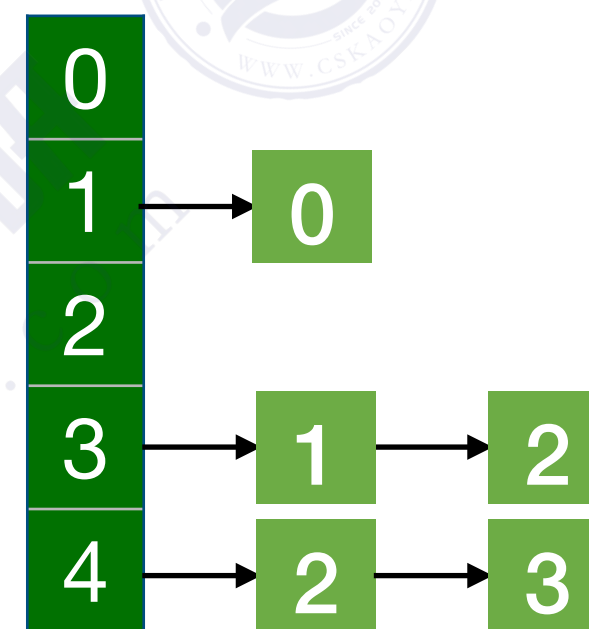


练习：模仿拓扑排序的思想实现**逆拓扑排序**

思考：使用不同的存储结构来对时间复杂度的影响

	0	1	2	3	4
0	0	1	0	0	0
1	0	0	0	1	0
2	0	0	0	1	1
3	0	0	0	0	1
4	0	0	0	0	0

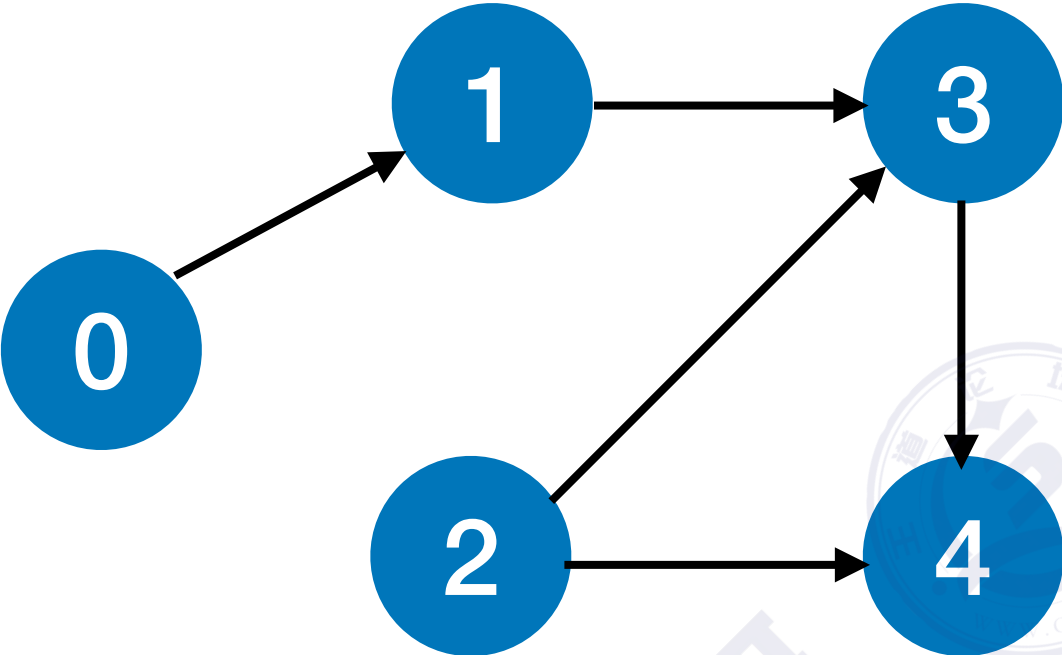
邻接矩阵



逆邻接表

```
bool TopologicalSort(Graph G){
    InitStack(S);           //初始化栈，存储入度为0的顶点
    for(int i=0;i<G.vexnum;i++){
        if(indegree[i]==0)
            Push(S,i);       //将所有入度为0的顶点进栈
        int count=0;         //计数，记录当前已经输出的顶点数
        while(!IsEmpty(S)){  //栈不空，则存在入度为0的顶点
            Pop(S,i);         //栈顶元素出栈
            print[count++]=i;  //输出顶点i
            for(p=G.vertices[i].firstarc;p;p=p->nextarc){
                //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈s
                v=p->adjvex;
                if(!(--indegree[v]))
                    Push(S,v); //入度为0，则入栈
            }
        } //while
        if(count<G.vexnum)
            return false;     //排序失败，有向图中有回路
        else
            return true;      //拓扑排序成功
    }
}
```


逆拓扑排序的实现 (DFS算法)

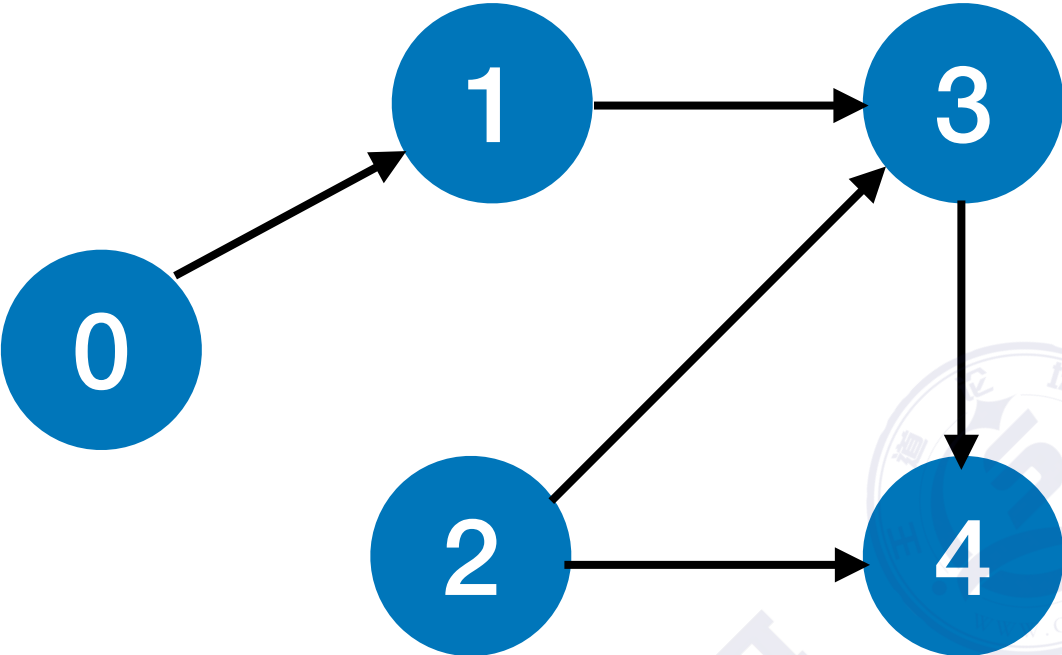


递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visit(v);  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
}
```

//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历
//从顶点v出发，深度优先遍历图G
//访问顶点v
//设已访问标记
//w为u的尚未访问的邻接顶点
//if

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

//初始化已访问标记数据

//本代码中是从v=0开始遍历

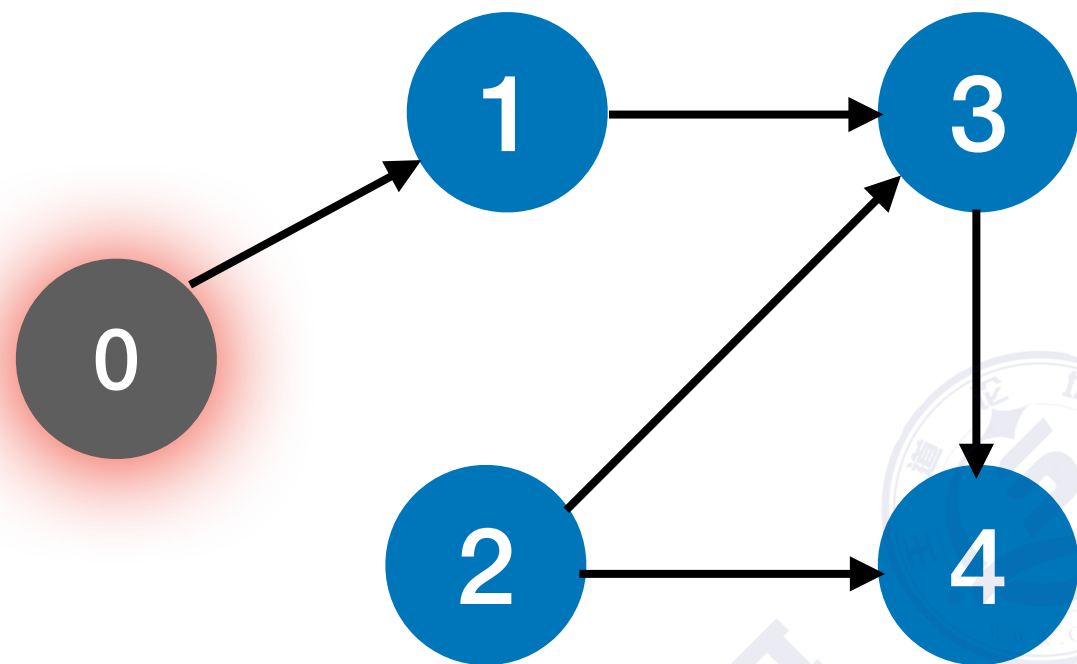
//从顶点v出发，深度优先遍历图G

//设已访问标记

//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)



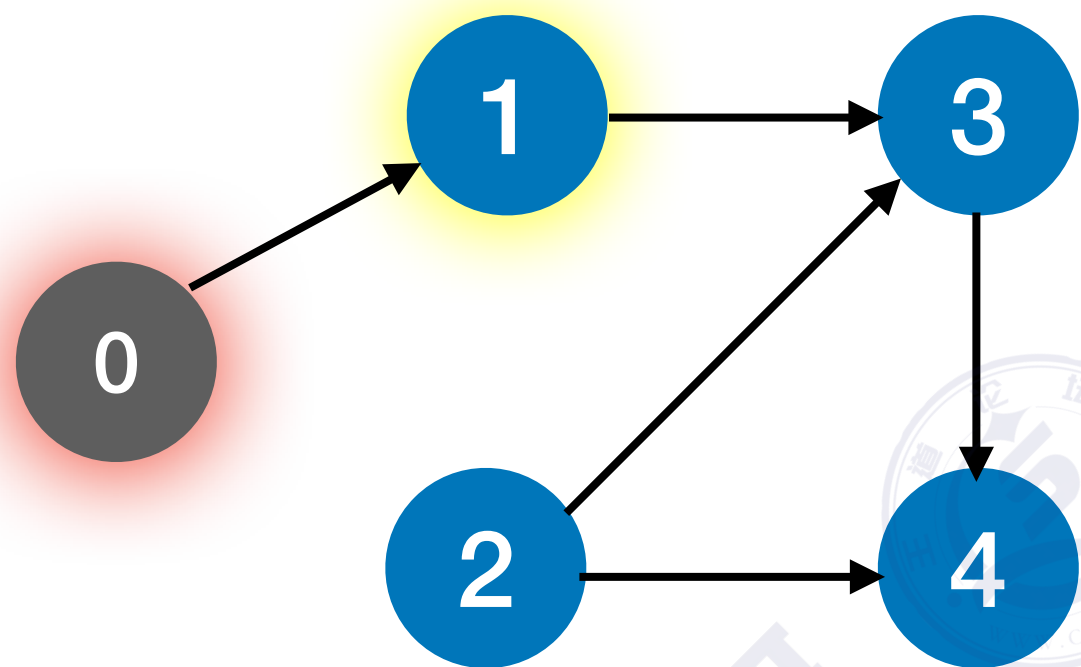
```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)

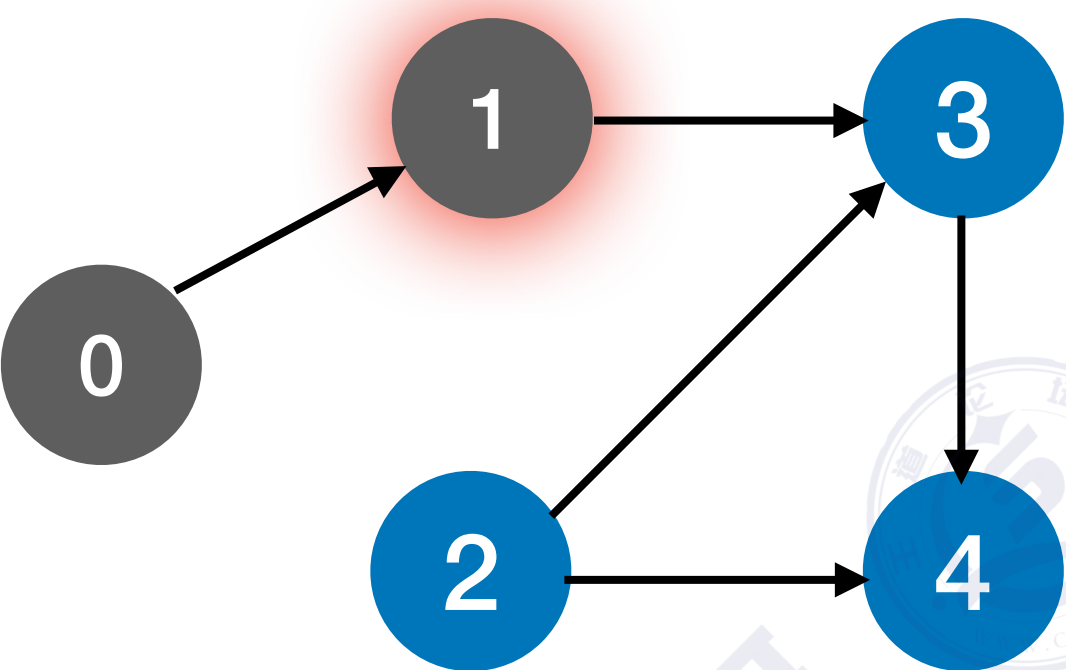


递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历
//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点
//输出顶点

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

//初始化已访问标记数据

//本代码中是从v=0开始遍历

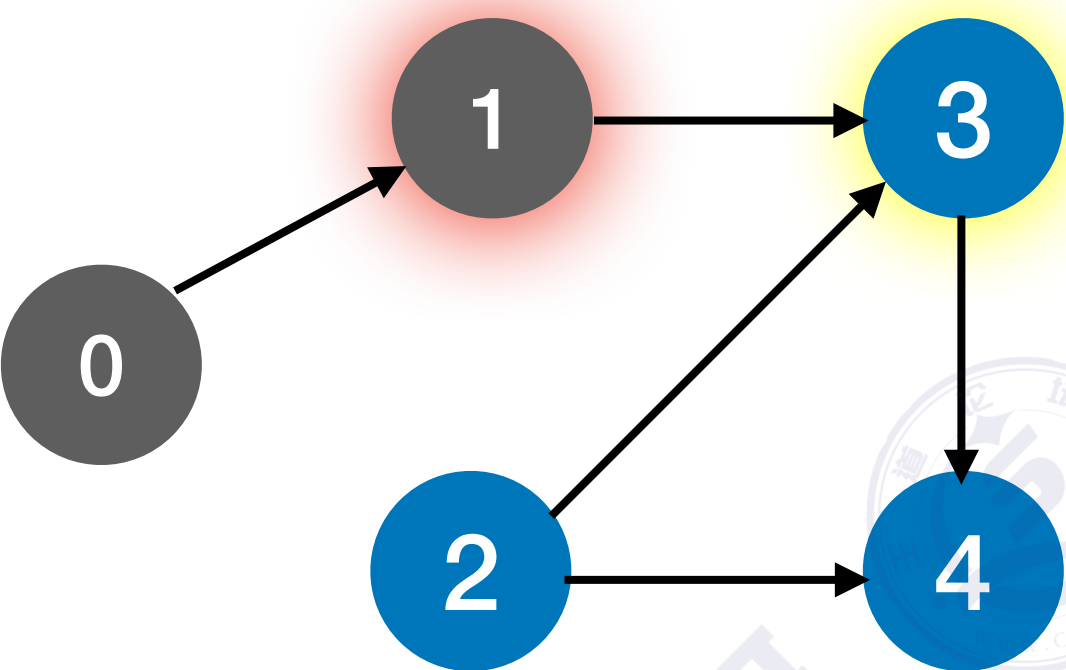
//从顶点v出发，深度优先遍历图G

//设已访问标记

//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)

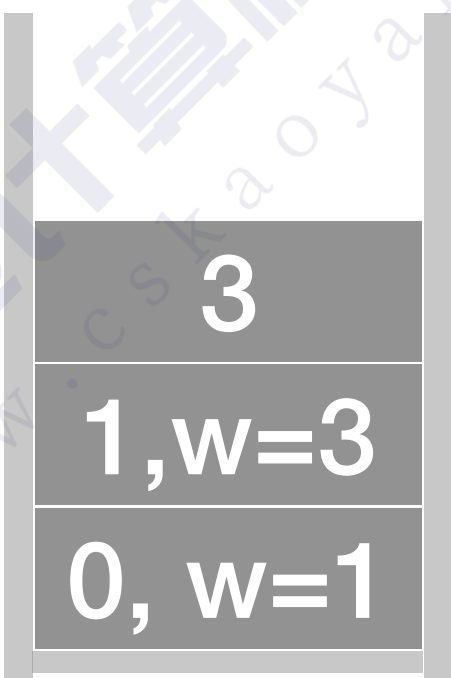
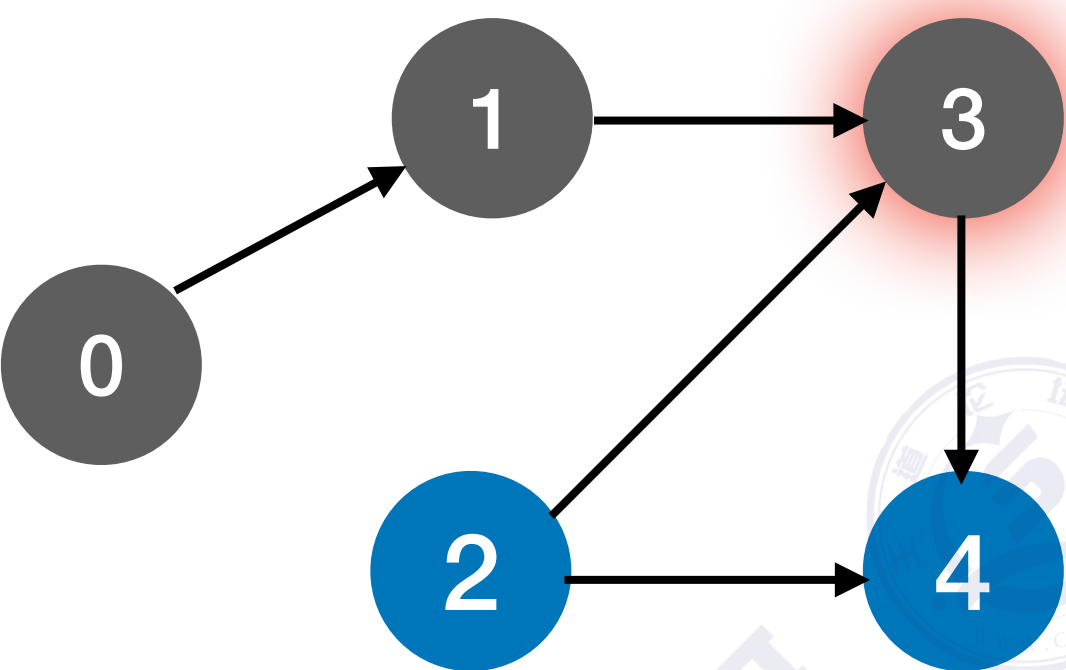


递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历
//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点
//输出顶点

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

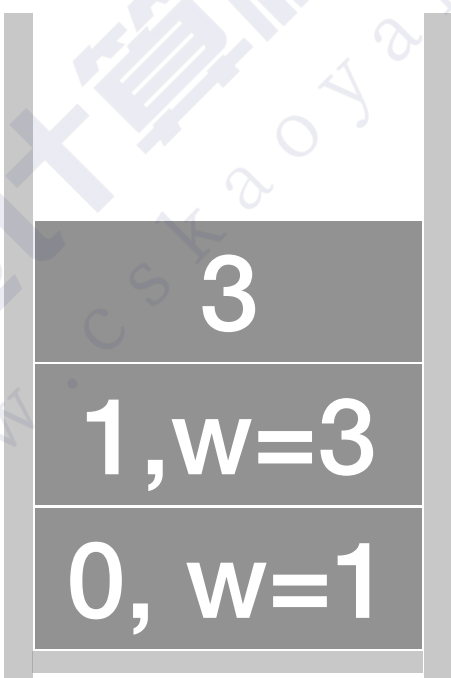
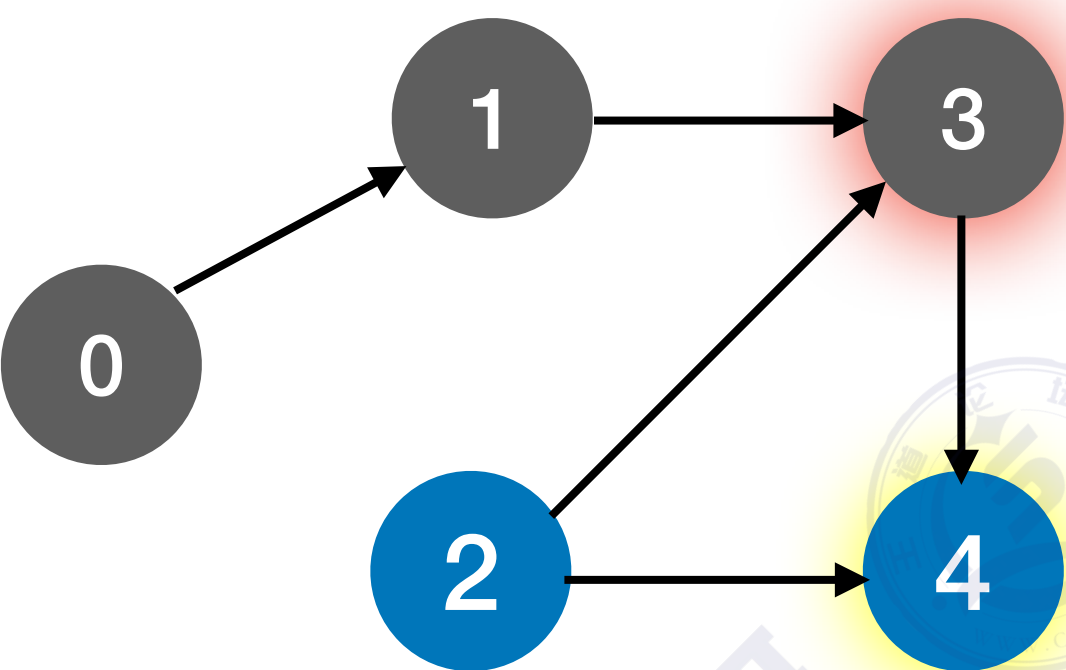
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记

//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

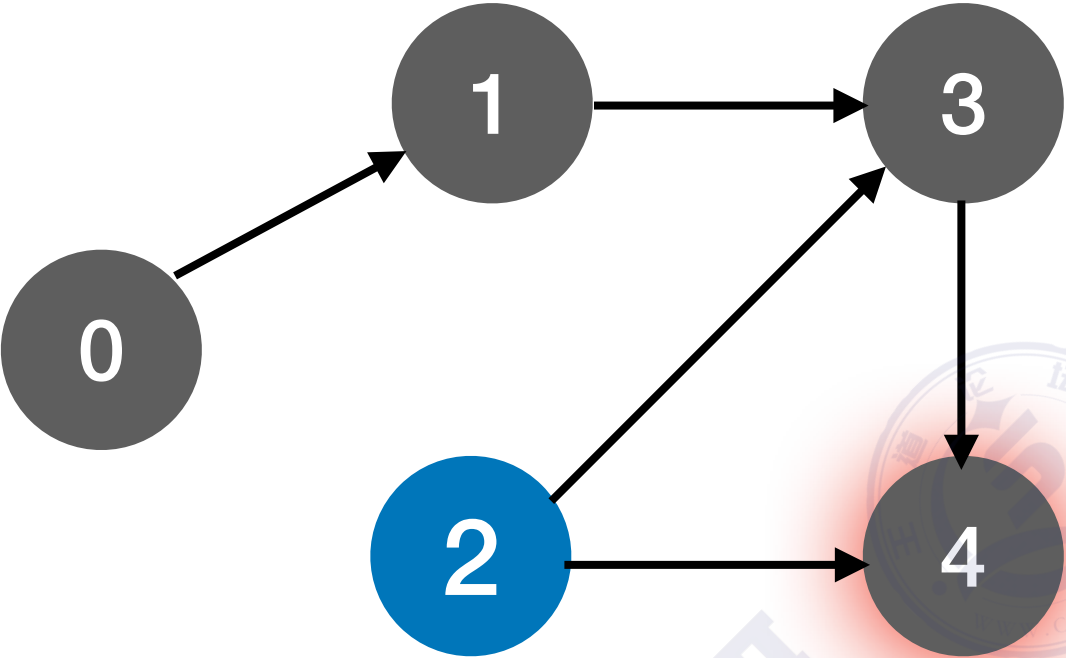
//对图G进行深度优先遍历

//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)



4
3,w=4
1,w=3
0, w=1

递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

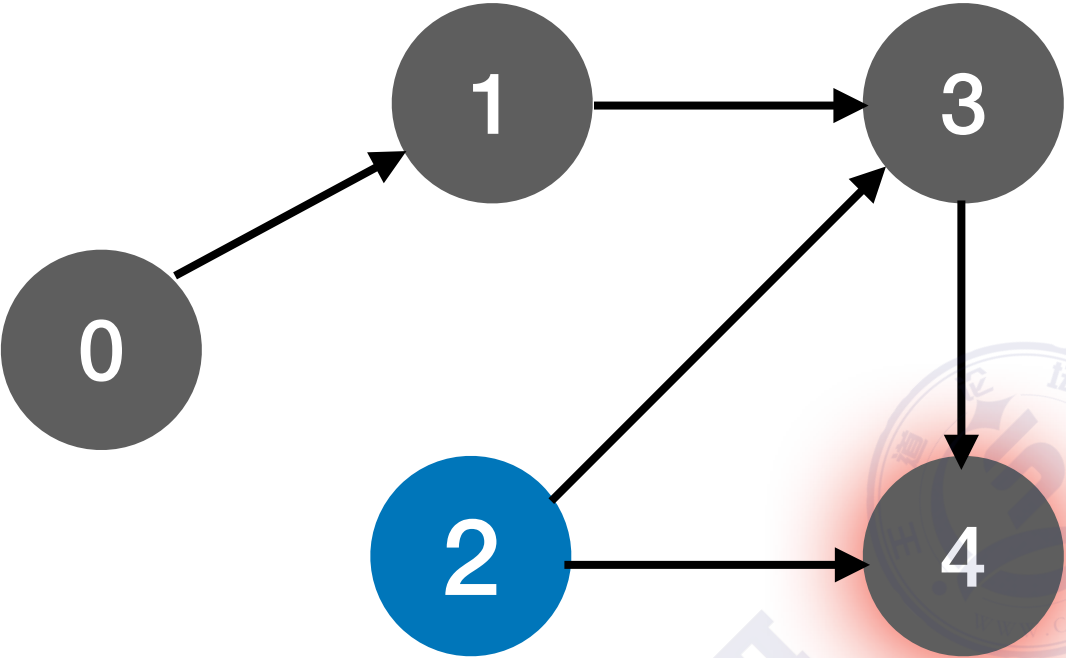
//对图G进行深度优先遍历

//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序的实现 (DFS算法)



4
3,w=4
1,w=3
0, w=1

递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    //if  
    print(v);  
}
```

//对图G进行深度优先遍历

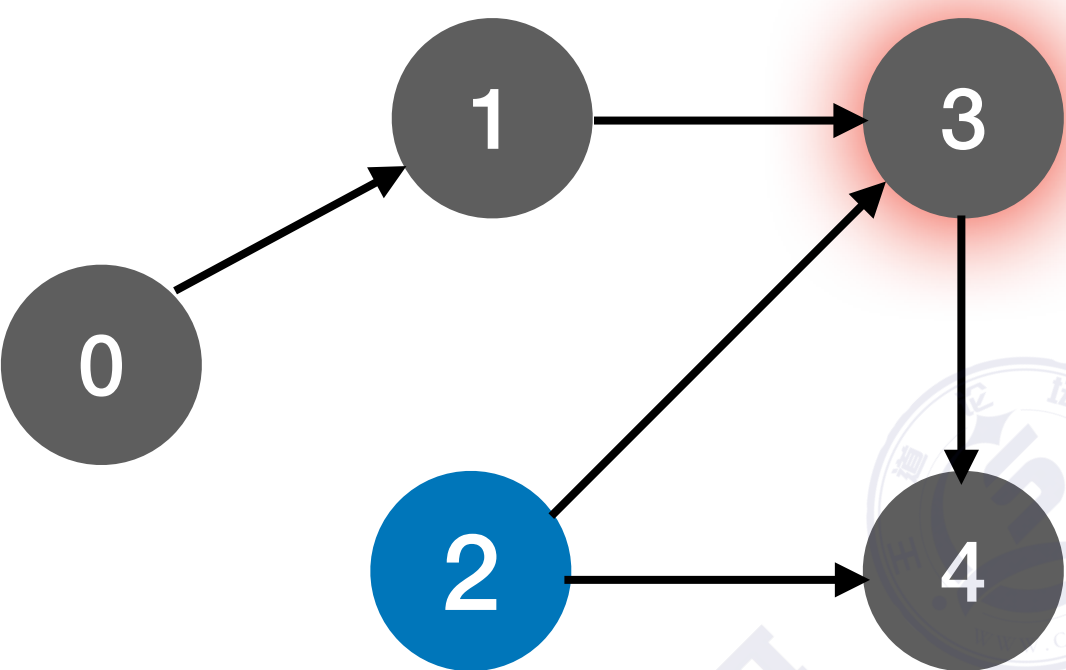
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列：4

逆拓扑排序的实现 (DFS算法)



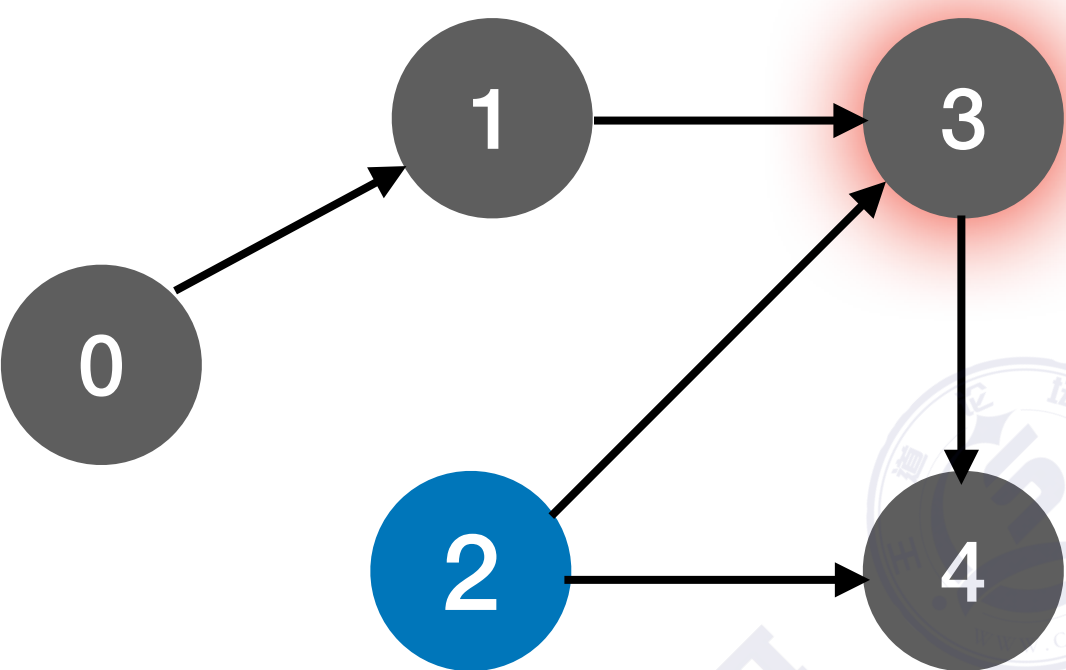
3,w=4
1,w=3
0, w=1

递归栈

```
void DFSTraverse(Graph G){  
    //对图G进行深度优先遍历  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    //初始化已访问标记数据  
    //本代码中是从v=0开始遍历  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    //从顶点v出发, 深度优先遍历图G  
    //设已访问标记  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            //w为u的尚未访问的邻接顶点  
            DFS(G,w);  
        }  
    //if  
    print(v);  
    //输出顶点  
}
```

逆拓扑排序序列: 4

逆拓扑排序的实现 (DFS算法)



3,w=4
1,w=3
0, w=1

递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

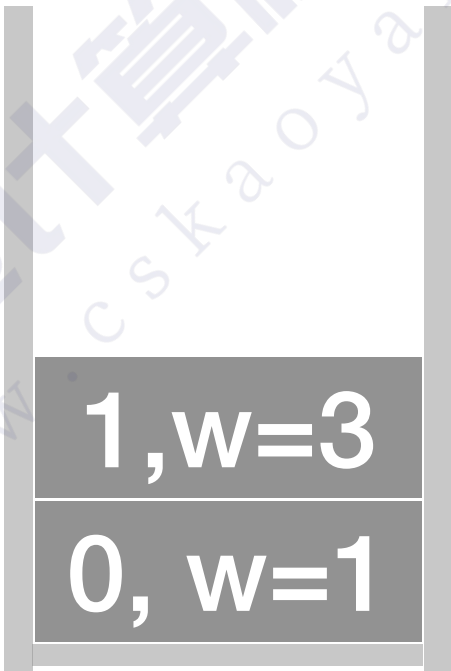
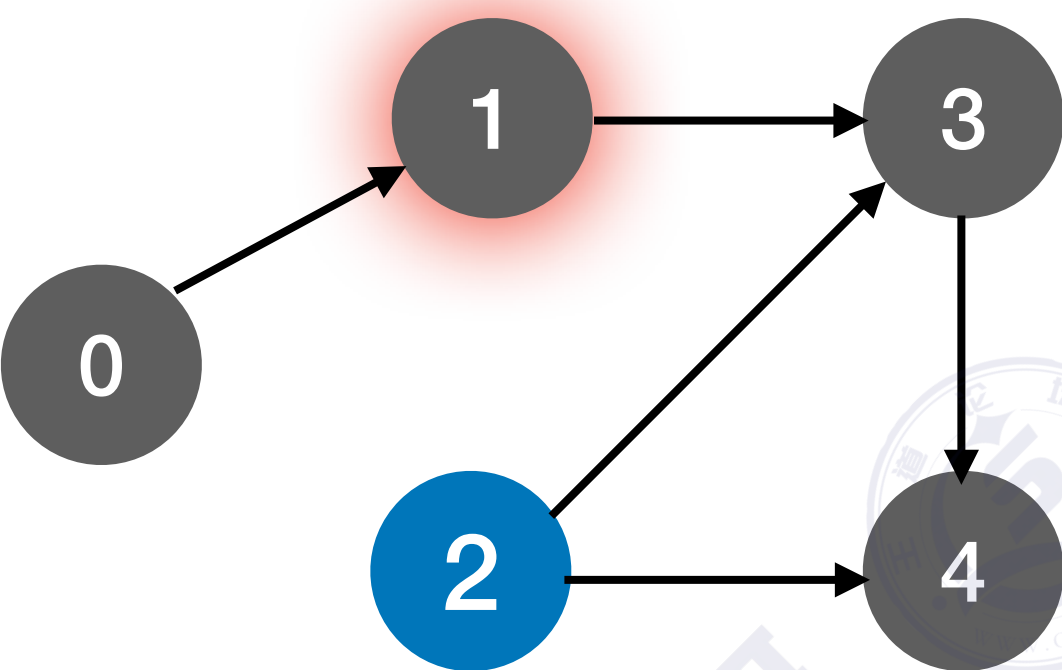
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3

逆拓扑排序的实现 (DFS算法)

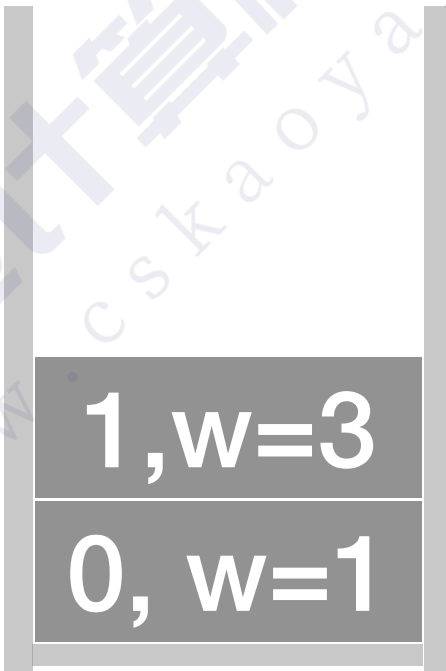
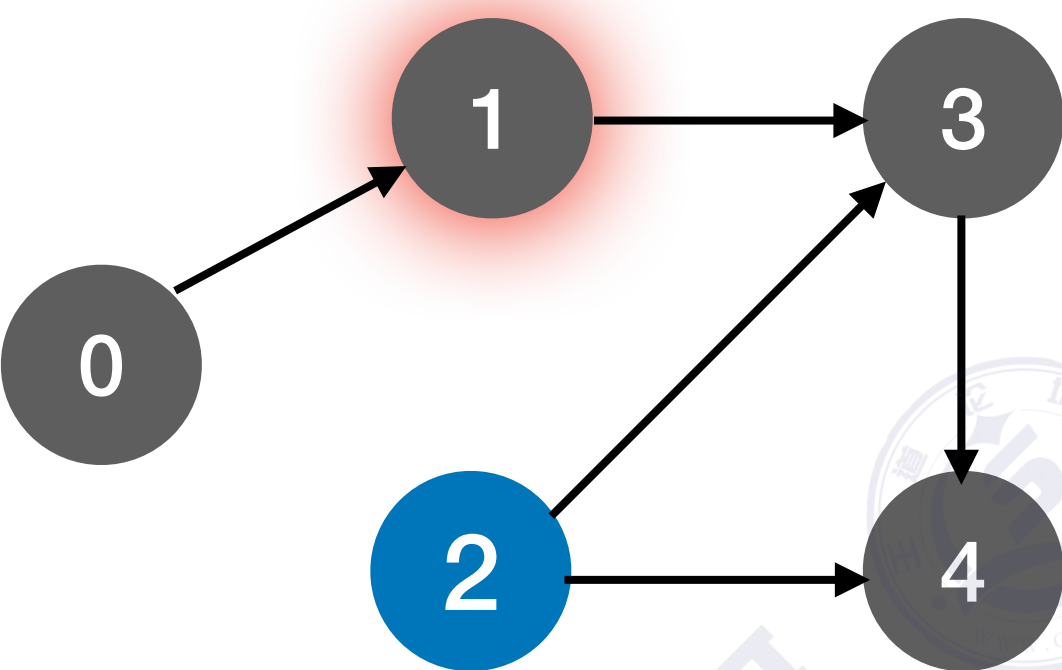


递归栈

```
void DFSTraverse(Graph G){  
    //对图G进行深度优先遍历  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    //初始化已访问标记数据  
    //本代码中是从v=0开始遍历  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    //从顶点v出发, 深度优先遍历图G  
    //设已访问标记  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            //w为u的尚未访问的邻接顶点  
            DFS(G,w);  
        }  
    //if  
    print(v);  
    //输出顶点  
}
```

逆拓扑排序序列: 4 3

逆拓扑排序的实现 (DFS算法)

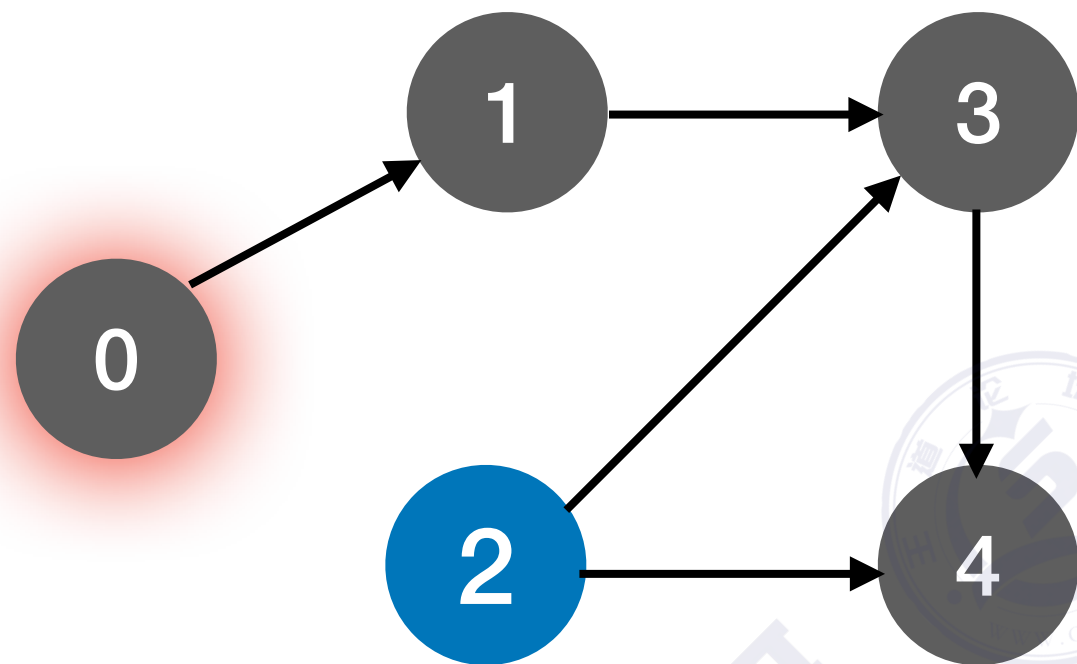


递归栈

```
void DFSTraverse(Graph G){  
    //对图G进行深度优先遍历  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    //初始化已访问标记数据  
    //本代码中是从v=0开始遍历  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    //从顶点v出发, 深度优先遍历图G  
    //设已访问标记  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            //w为u的尚未访问的邻接顶点  
            DFS(G,w);  
        }  
    //if  
    print(v);  
    //输出顶点  
}
```

逆拓扑排序序列: 4 3 1

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

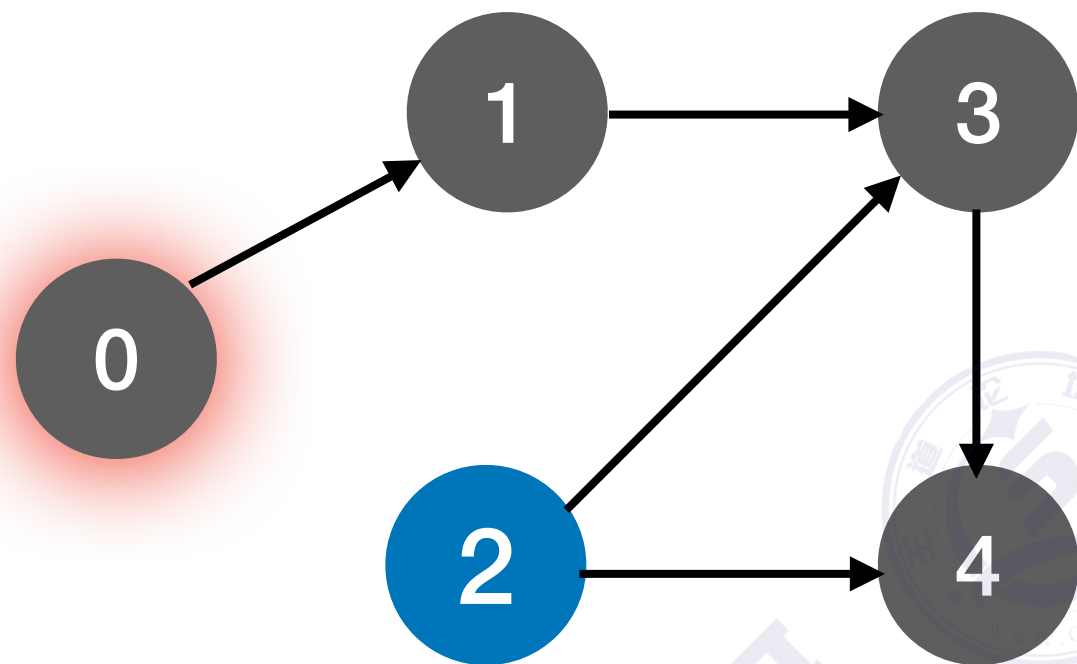
//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3 1

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

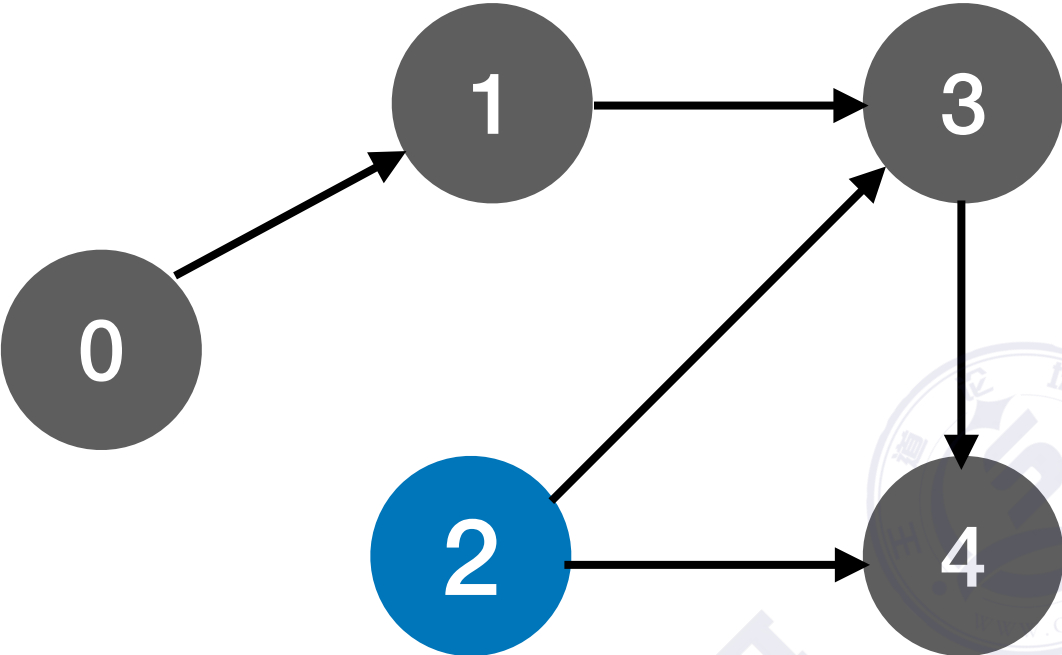
//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3 1 0

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

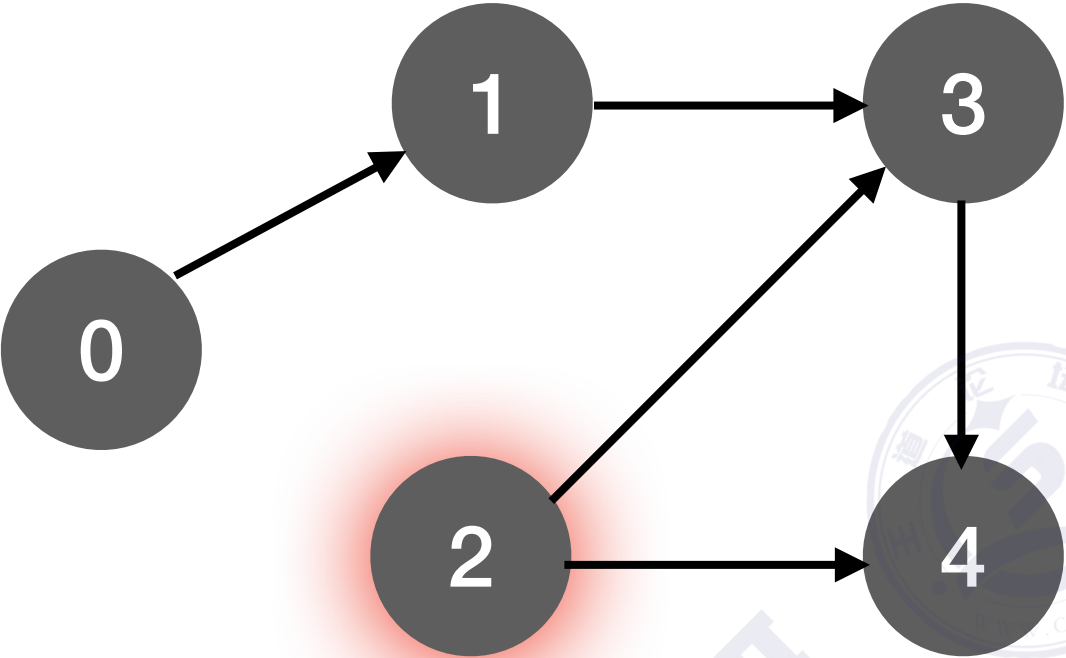
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3 1 0

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

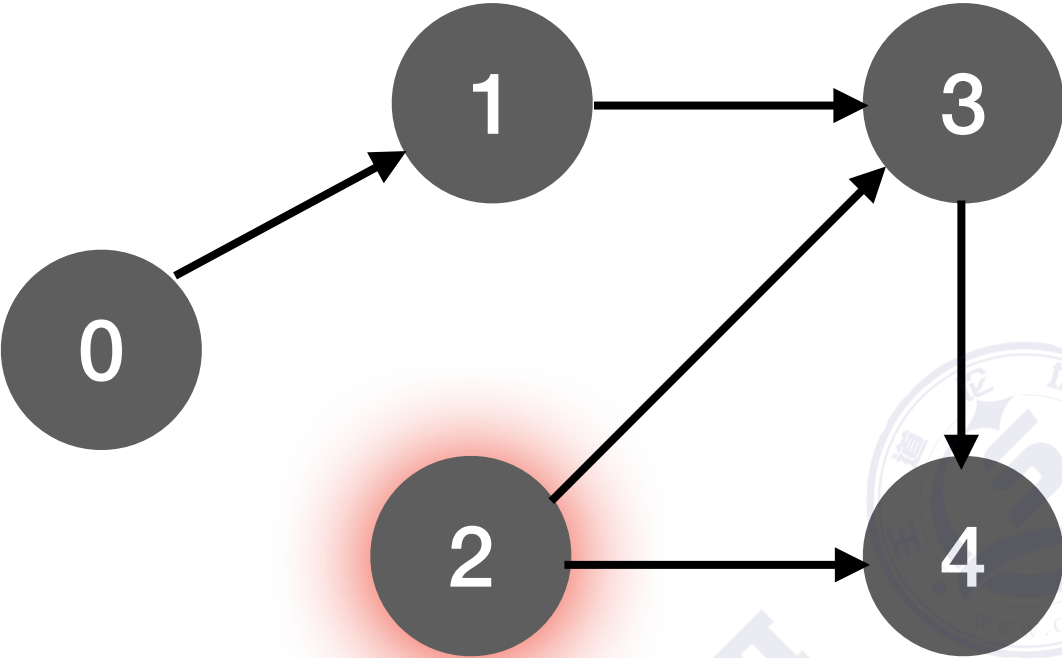
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3 1 0

逆拓扑排序的实现 (DFS算法)



递归栈

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    print(v);  
}
```

//对图G进行深度优先遍历

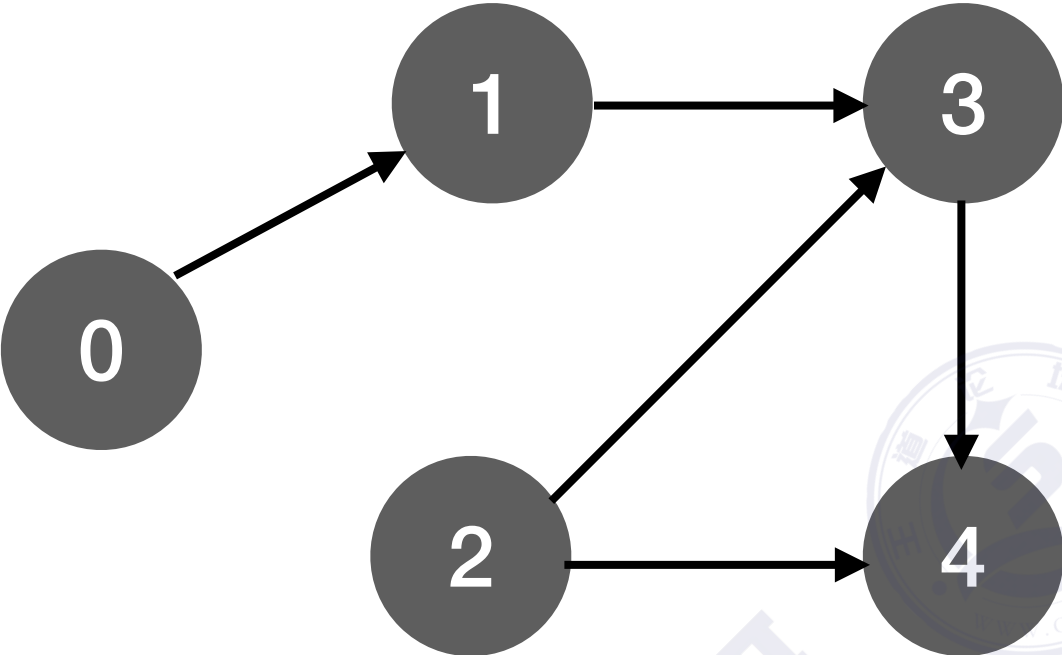
//初始化已访问标记数据
//本代码中是从v=0开始遍历

//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点

//输出顶点

逆拓扑排序序列: 4 3 1 0 2

逆拓扑排序的实现 (DFS算法)



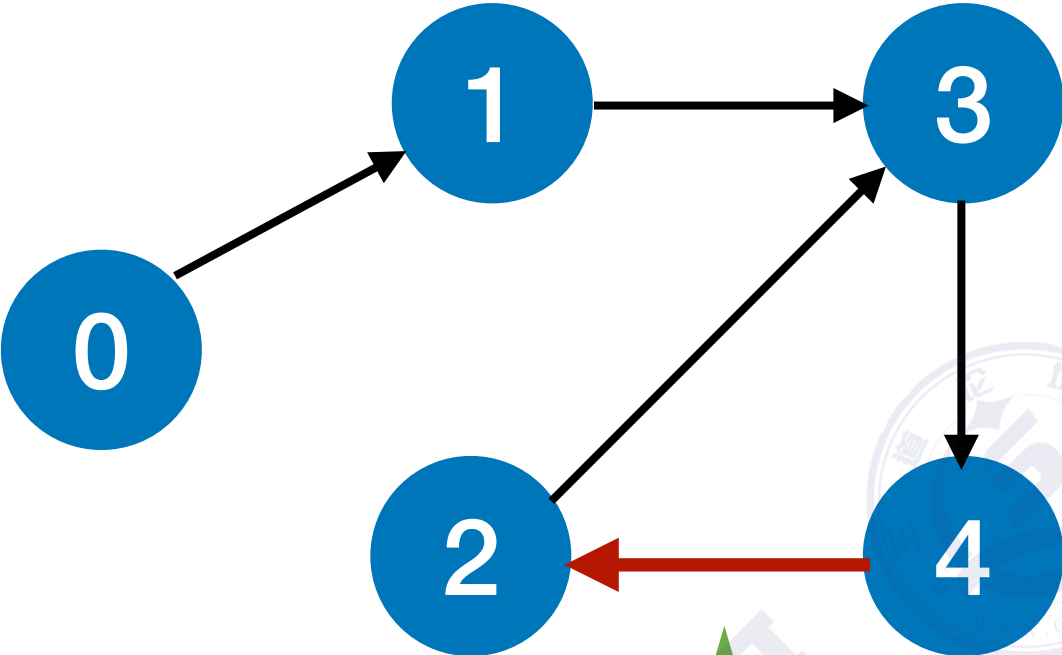
递归栈

DFS实现逆拓扑排序：
在顶点退栈前输出

```
void DFSTraverse(Graph G){  
    //对图G进行深度优先遍历  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    //初始化已访问标记数据  
    //本代码中是从v=0开始遍历  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    //从顶点v出发，深度优先遍历图G  
    //设已访问标记  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            //w为u的尚未访问的邻接顶点  
            DFS(G,w);  
        }  
    //if  
    print(v);  
    //输出顶点  
}
```

逆拓扑排序序列：4 3 1 0 2

逆拓扑排序的实现 (DFS算法)



思考：如果存在回路，则不存在逆拓扑排序序列，如何判断回路？



递归栈

DFS实现逆拓扑排序：
在顶点退栈前输出

```
void DFSTraverse(Graph G){  
    for(v=0;v<G.vexnum;++v)  
        visited[v]=FALSE;  
    for(v=0;v<G.vexnum;++v)  
        if(!visited[v])  
            DFS(G,v);  
}  
  
void DFS(Graph G,int v){  
    visited[v]=TRUE;  
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))  
        if(!visited[w]){  
            DFS(G,w);  
        }  
    //if  
    print(v);  
}
```

//对图G进行深度优先遍历
//初始化已访问标记数据
//本代码中是从v=0开始遍历
//从顶点v出发，深度优先遍历图G
//设已访问标记
//w为u的尚未访问的邻接顶点
//输出顶点

知识点回顾与重要考点

顶点代表活动，有向边 $\langle V_i, V_j \rangle$ 表示活动 V_i 必须先于 V_j 进行

AOV 网

AOV 网一定是 DAG 图，不能有环

拓扑排序

- ① 从 AOV 网中选择一个没有前驱（入度为 0）的顶点并输出
- ② 从网中删除该顶点和所有以它为起点的有向边
- ③ 重复 ① 和 ② 直到当前的 AOV 网为空

逆拓扑排序

- ① 从 AOV 网中选择一个没有后继（出度为 0）的顶点并输出
- ② 从网中删除该顶点和所有以它为终点的有向边
- ③ 重复 ① 和 ② 直到当前的 AOV 网为空

另一种实现方式：用 DFS 实现拓扑排序/逆拓扑排序

性质

- 拓扑排序、逆拓扑排序序列可能不唯一
- 若图中有环，则不存在拓扑排序序列/逆拓扑排序序列

拓扑排序

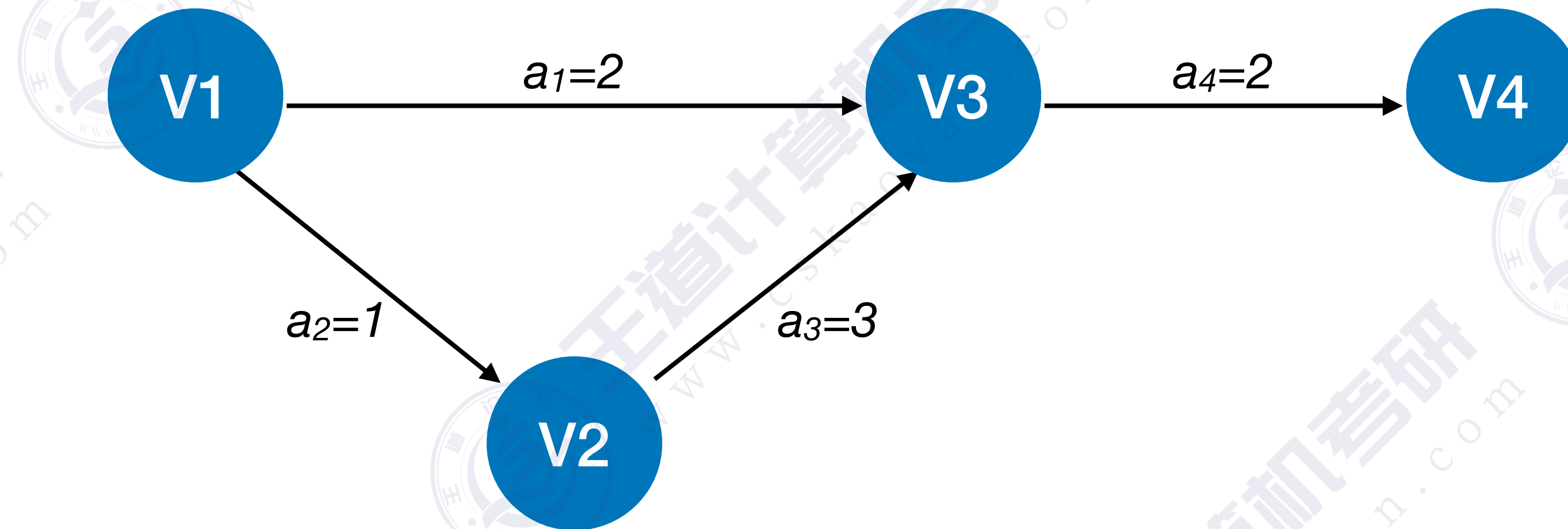
本节内容

关键路径

AOE网

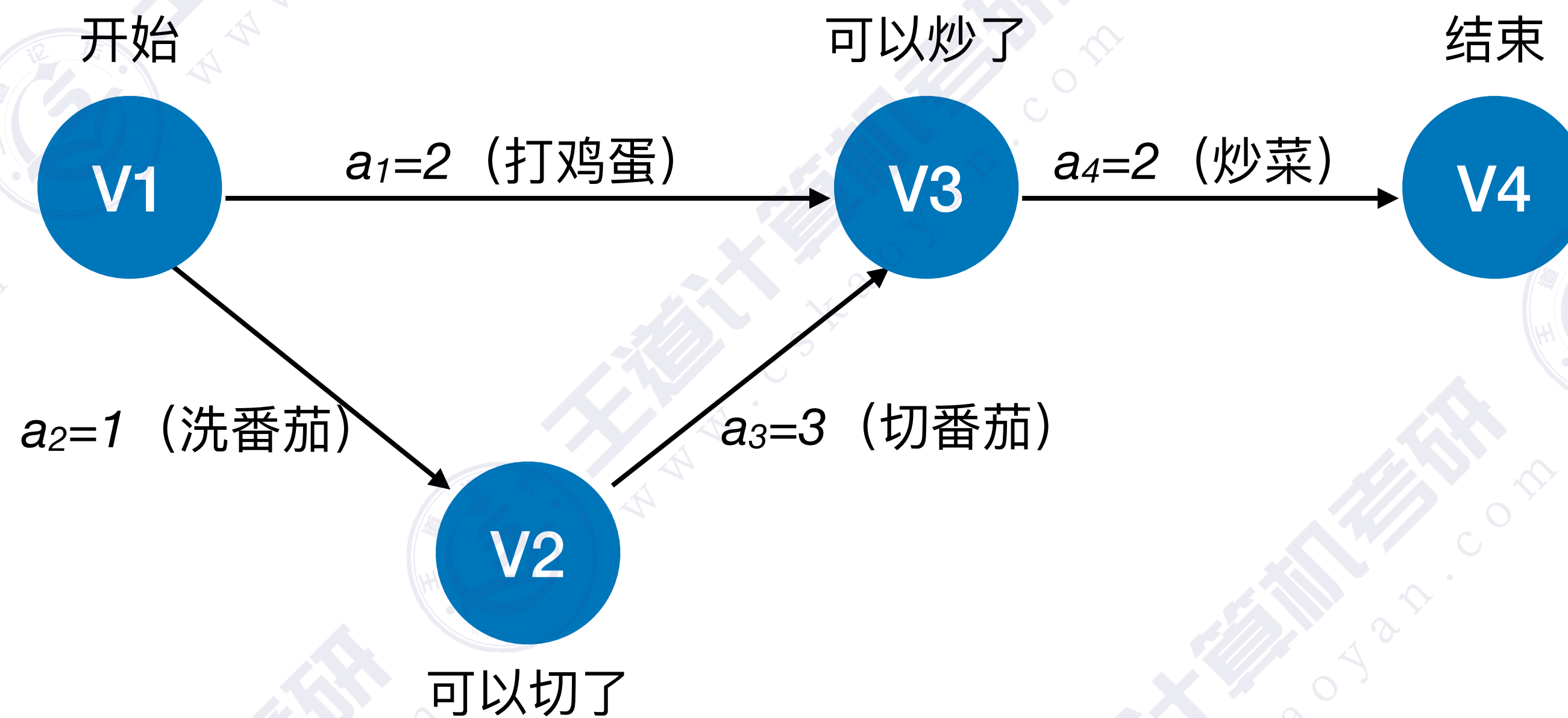


在带权有向图中，以**顶点表示事件**，以**有向边表示活动**，以**边上的权值表示完成该活动的开销**（如完成活动所需的时间），称之为用边表示活动的网络，简称**AOE网** (Activity On Edge NetWork)



AOE网

在带权有向图中，以**顶点表示事件**，以**有向边表示活动**，以**边上的权值表示完成该活动的开销**（如完成活动所需的时间），称之为用边表示活动的网络，简称**AOE网** (Activity On Edge NetWork)

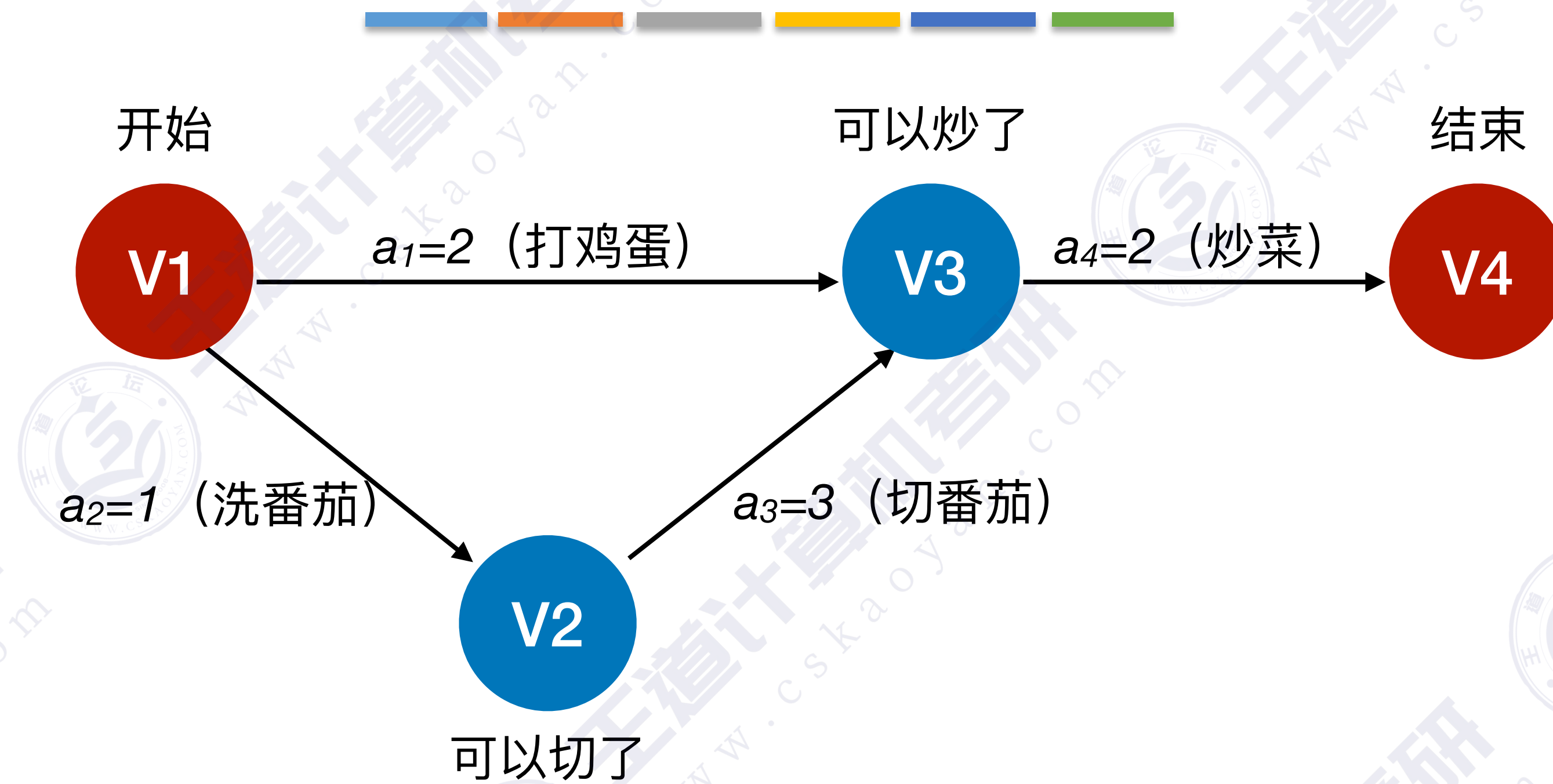


AOE网具有以下两个性质：

- ① 只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始；
- ② 只有在进入某顶点的各有向边所代表的活动都已结束时，该顶点所代表的事件才能发生。

另外，有些活动是可以并行进行的

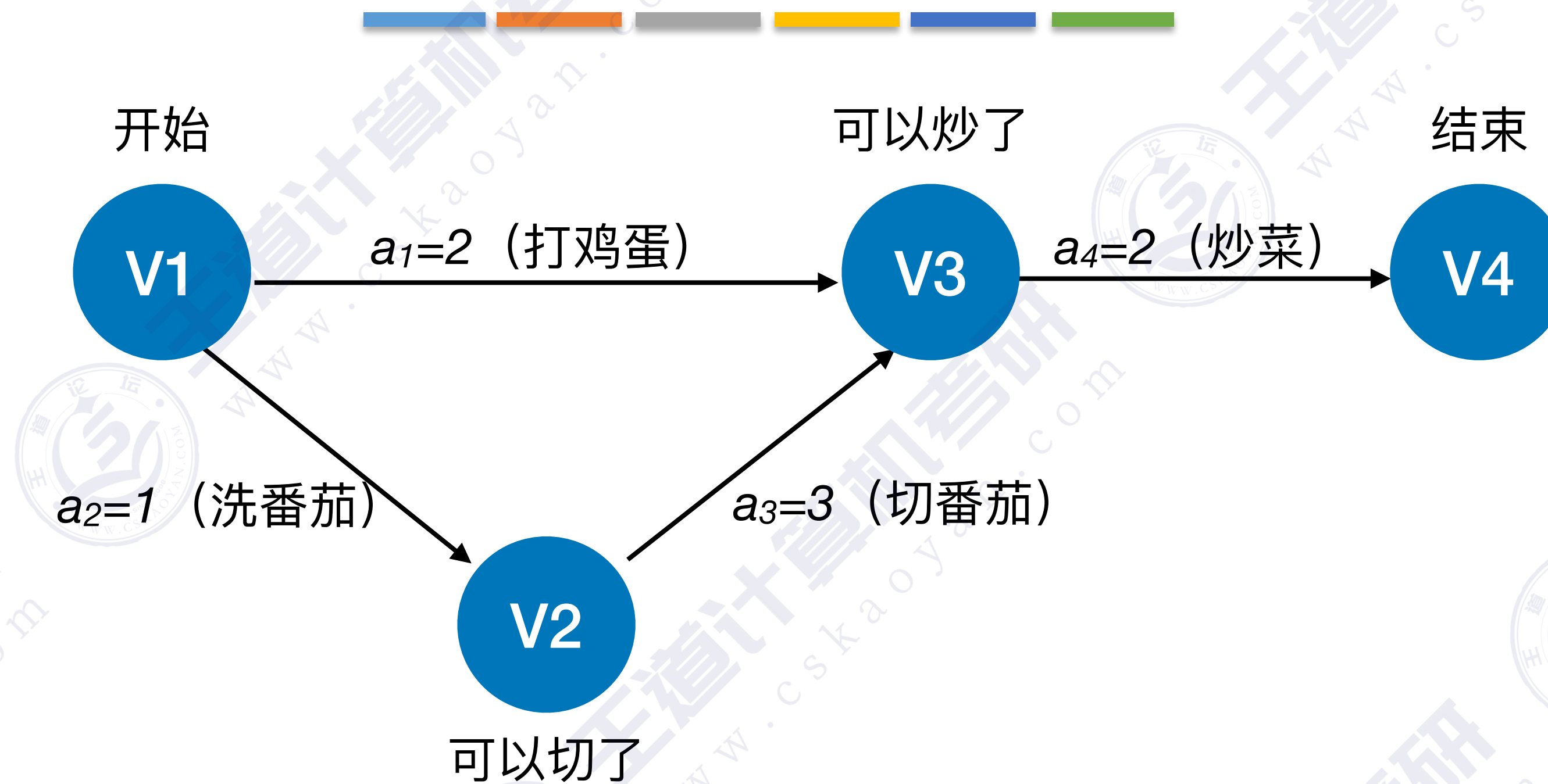
AOE网



在AOE网中仅有一个入度为0的顶点，称为开始顶点（源点），它表示整个工程的开始；

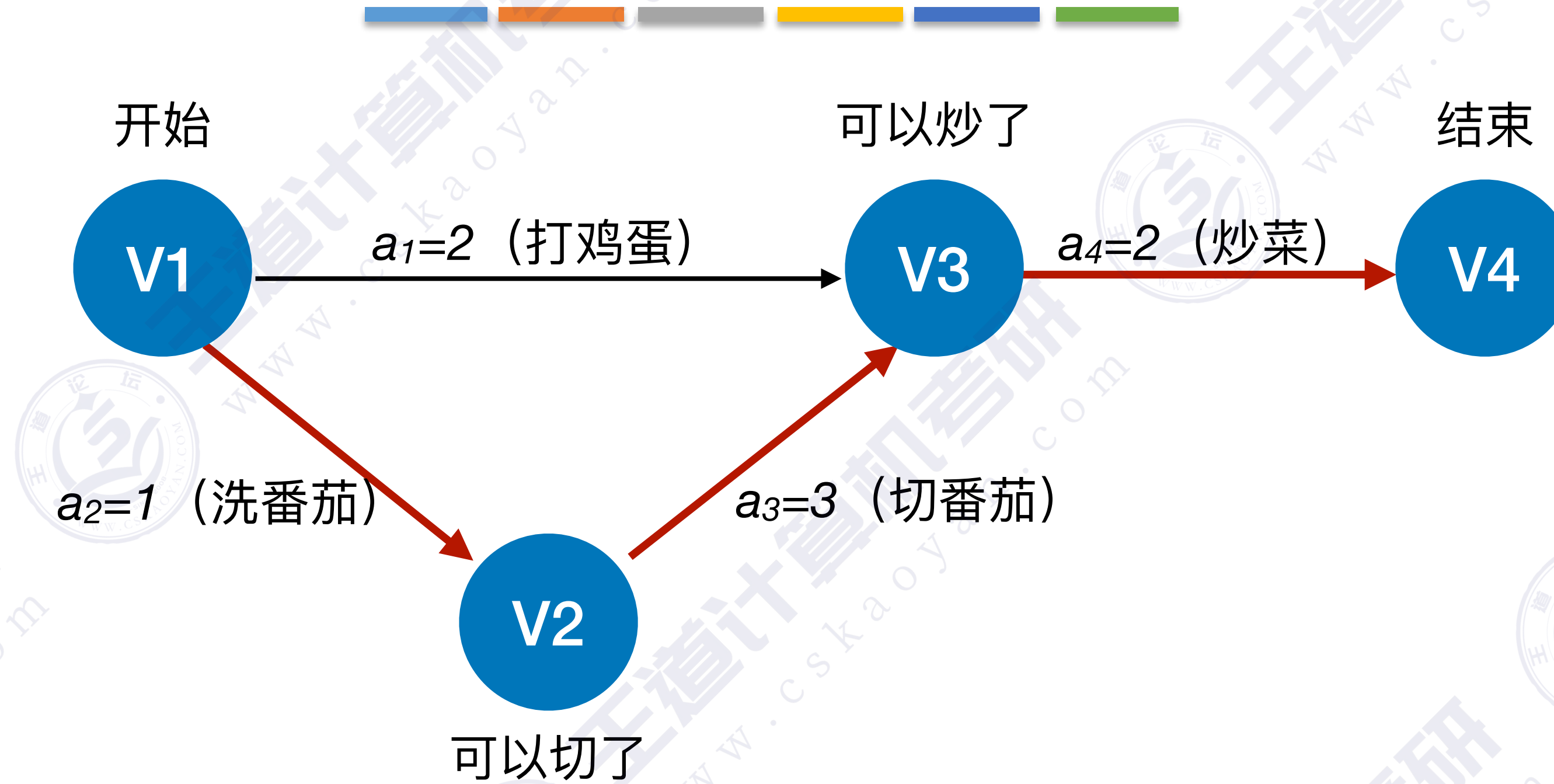
也仅有一个出度为0的顶点，称为结束顶点（汇点），它表示整个工程的结束。

关键路径



从源点到汇点的有向路径可能有多条，所有路径中，具有最大路径长度的路径称为**关键路径**，而把关键路径上的活动称为**关键活动**

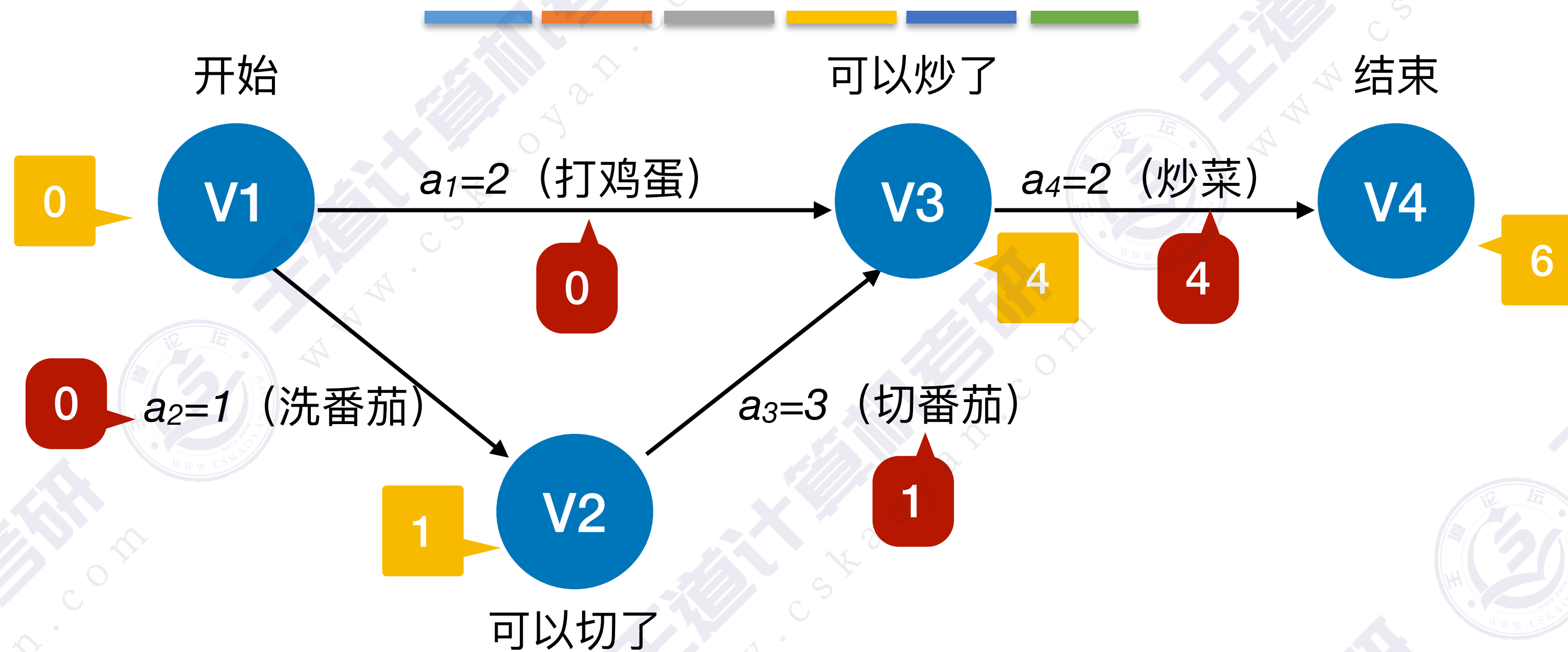
关键路径



从源点到汇点的有向路径可能有多条，所有路径中，具有最大路径长度的路径称为**关键路径**，而把关键路径上的活动称为**关键活动**

完成整个工程的最短时间就是关键路径的长度，若关键活动不能按时完成，则整个工程的完成时间就会延长

关键路径



最快多久可以开始做?

最快多久可以开切?

最快多久可以开炒?

最快多久可以吃?

现在, 马上!

1分钟

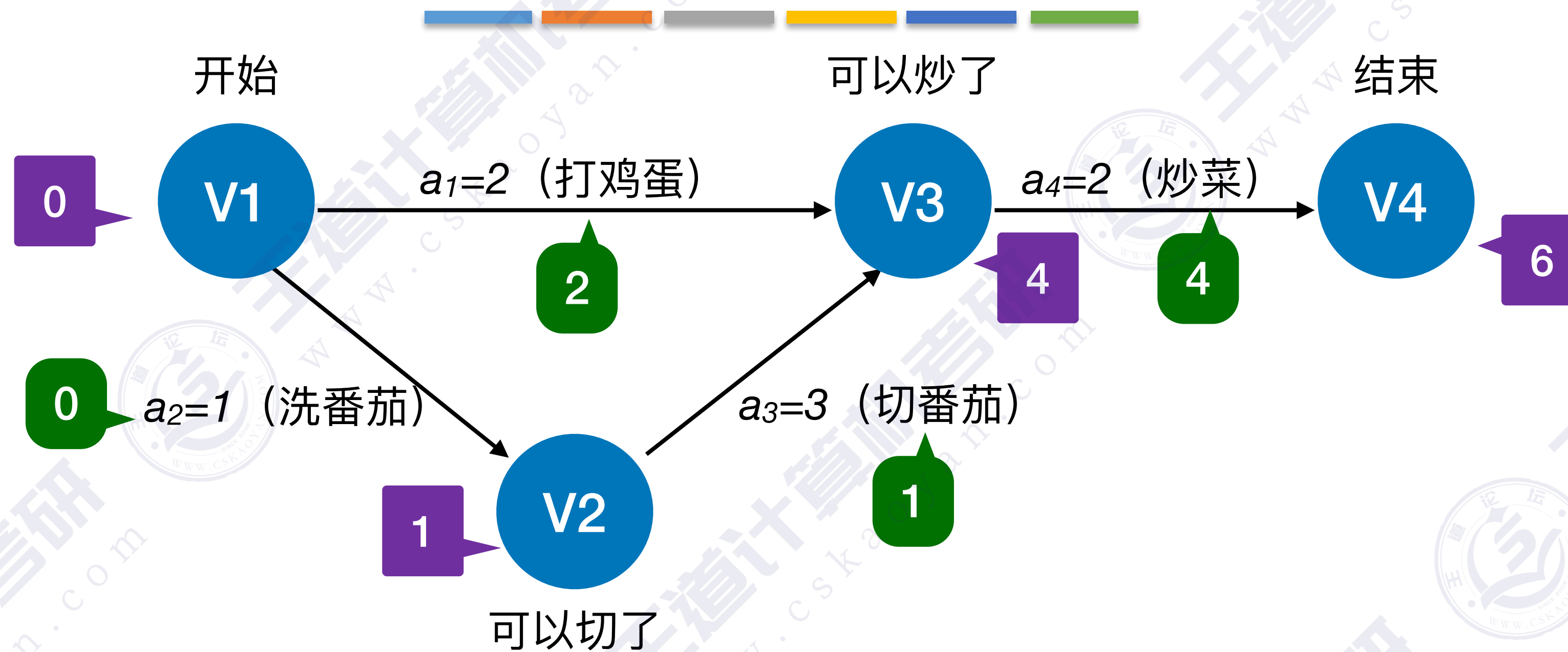
4分钟

6分钟



事件 v_k 的最早发生时间 $ve(k)$ ——决定了所有从 v_k 开始的活动能够开工的最早时间
活动 a_i 的最早开始时间 $e(i)$ ——指该活动弧的起点所表示的事件的最早发生时间

关键路径



我好饿~~~~~

那，要是6分钟后吃不上我就会饿死

6分钟后要结束

4分钟后一定要开炒

1分钟后一定要开切

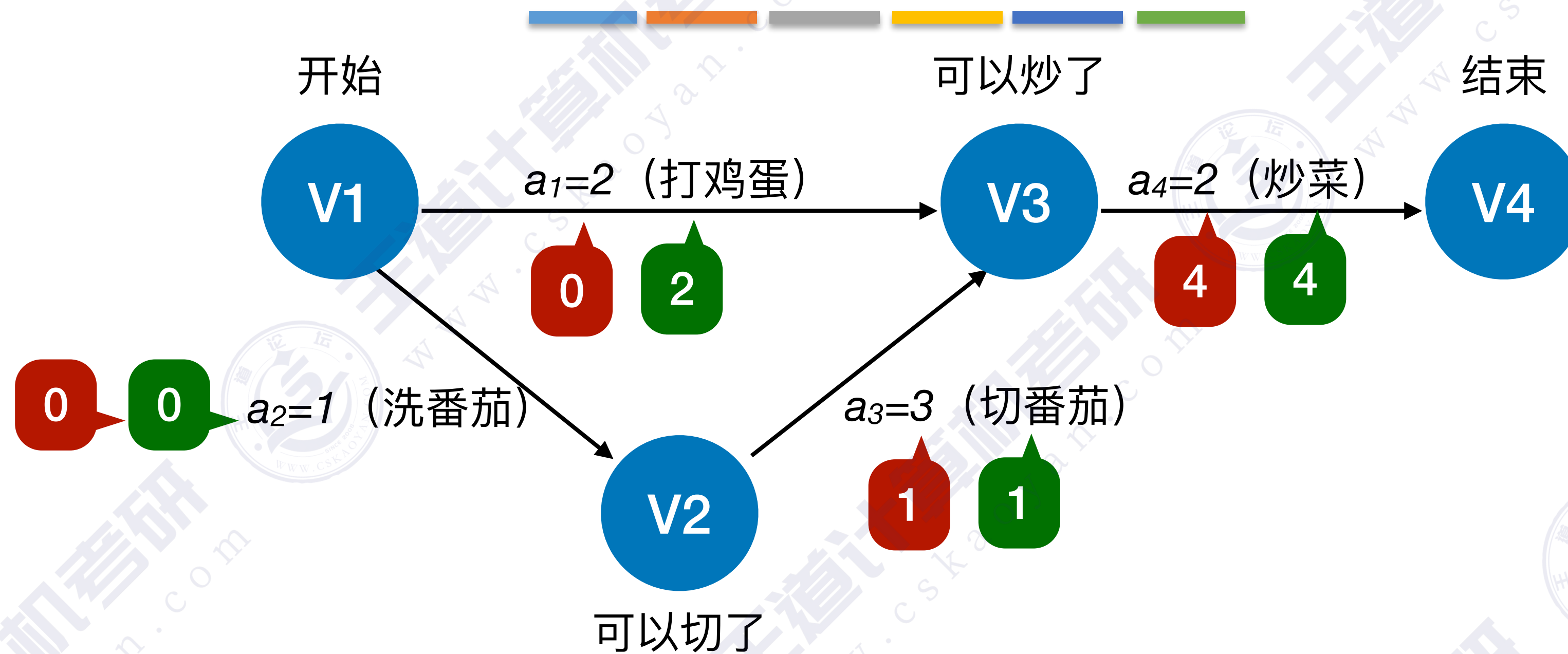
现在就要开始！



事件 v_k 的最迟发生时间 $vl(k)$ ——它是指在不推迟整个工程完成的前提下，该事件最迟必须发生的时间。

活动 a_i 的最迟开始时间 $l(i)$ ——它是指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。

关键路径



活动 a_i 的最早开始时间 $e(i)$ ——指该活动弧的起点所表示的事件的最早发生时间

活动 a_i 的最迟开始时间 $l(i)$ ——它是指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。

活动 a_i 的时间余量 $d(i)=l(i)-e(i)$ ，表示在不增加完成整个工程所需总时间的情况下，活动 a_i 可以拖延的时间

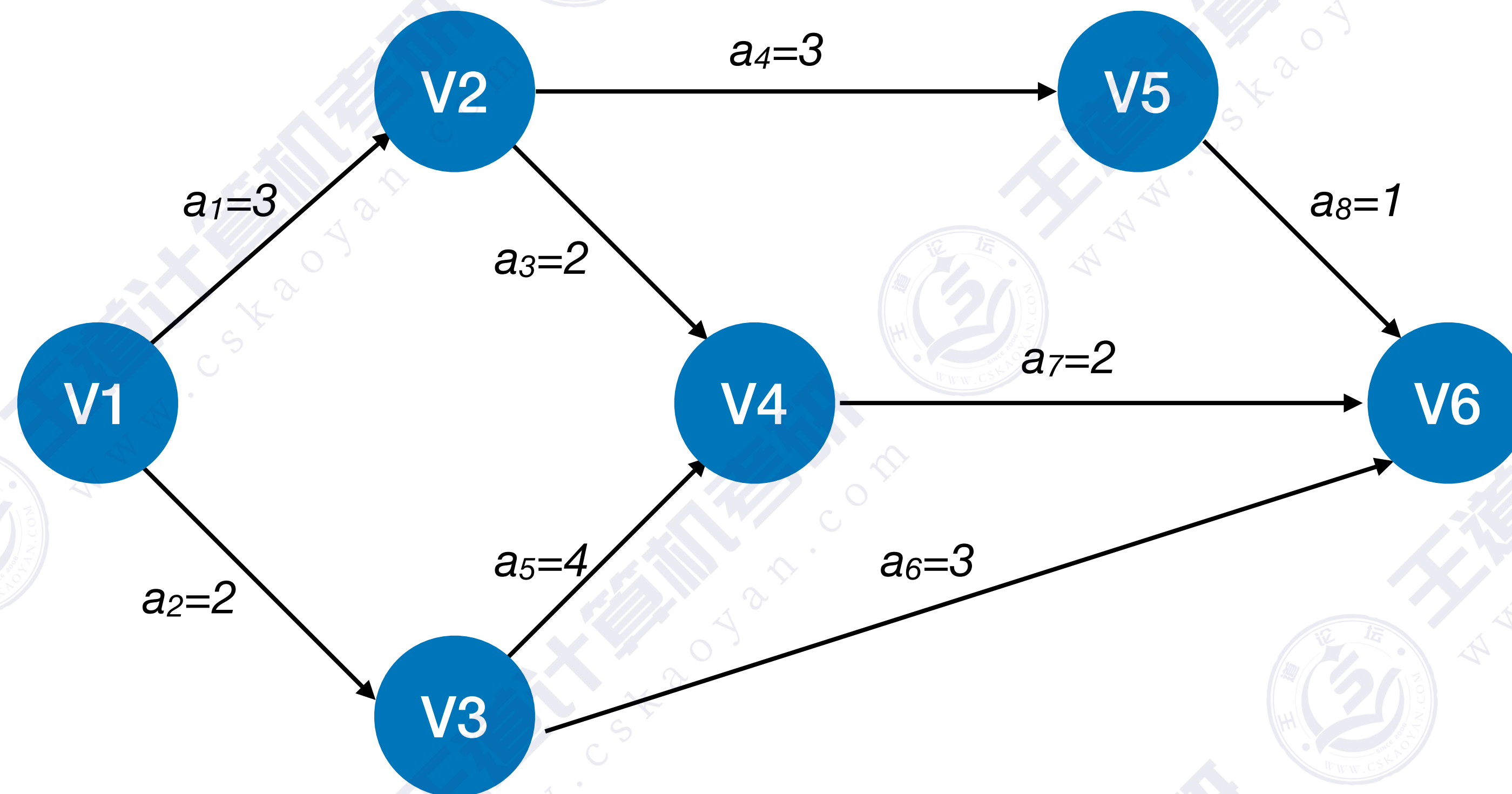
若一个活动的时间余量为零，则说明该活动必须要如期完成， $d(i)=0$ 即 $l(i)=e(i)$ 的活动 a_i 是**关键活动**

由**关键活动**组成的路径就是**关键路径**

求关键路径的步骤

- ① 求所有事件的最早发生时间 $ve()$
- ② 求所有事件的最迟发生时间 $vl()$
- ③ 求所有活动的最早发生时间 $e()$
- ④ 求所有活动的最迟发生时间 $l()$
- ⑤ 求所有活动的时间余量 $d()$

$d(i)=0$ 的活动就是关键活动, 由关键活动可得关键路径



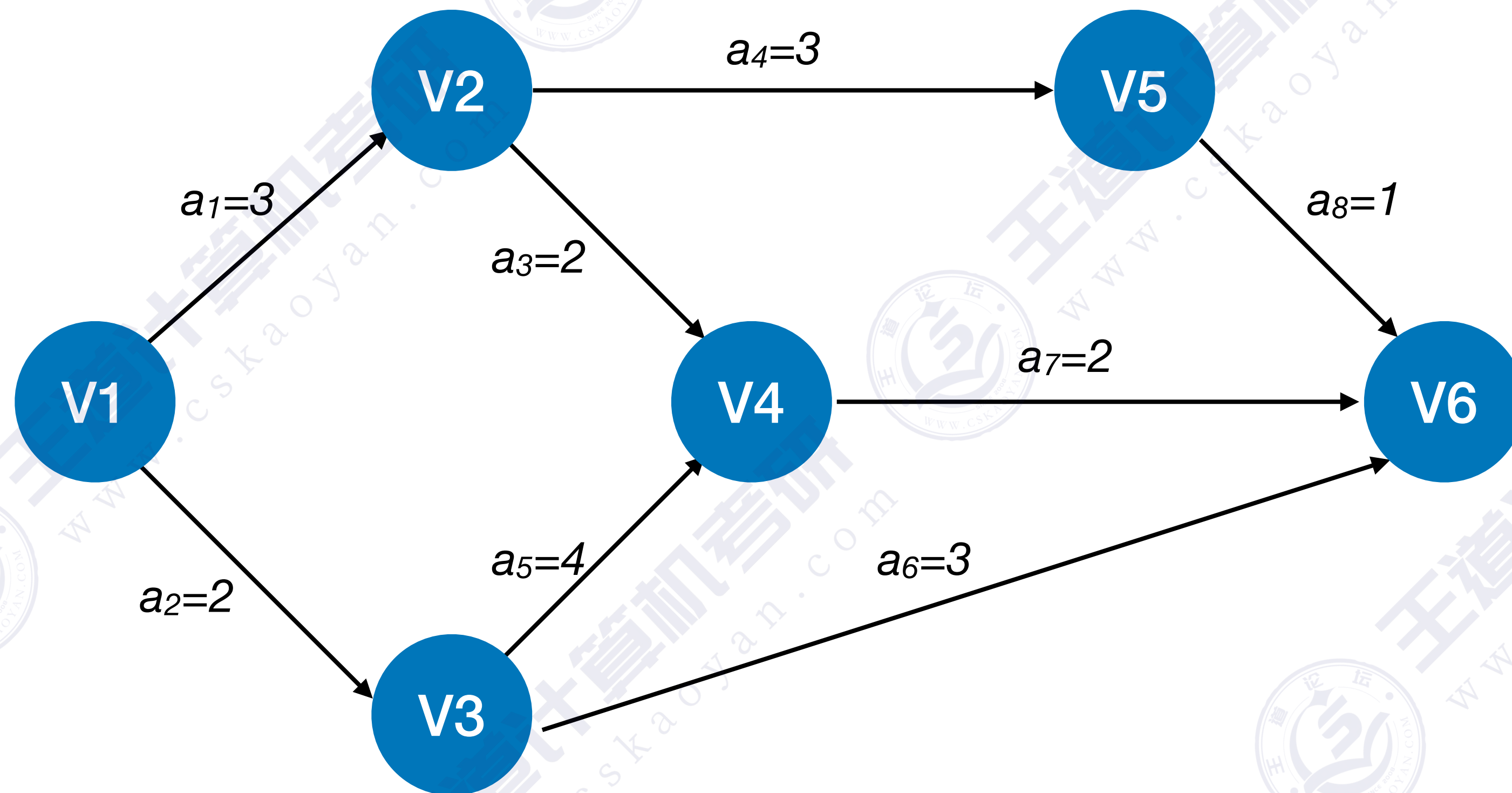
求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱



求所有事件的最早发生时间

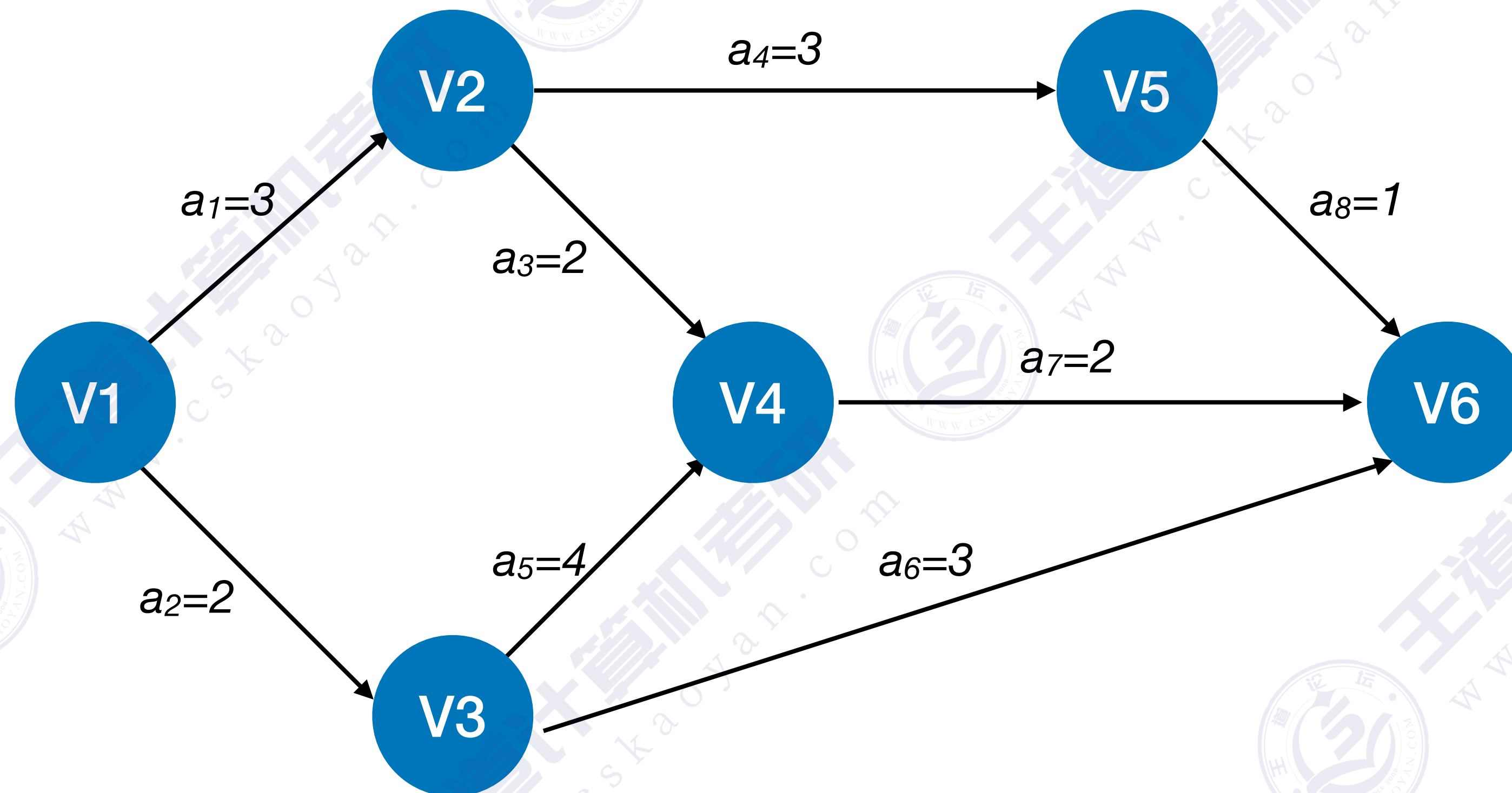
① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: V1、V3、V2、V5、V4、V6



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

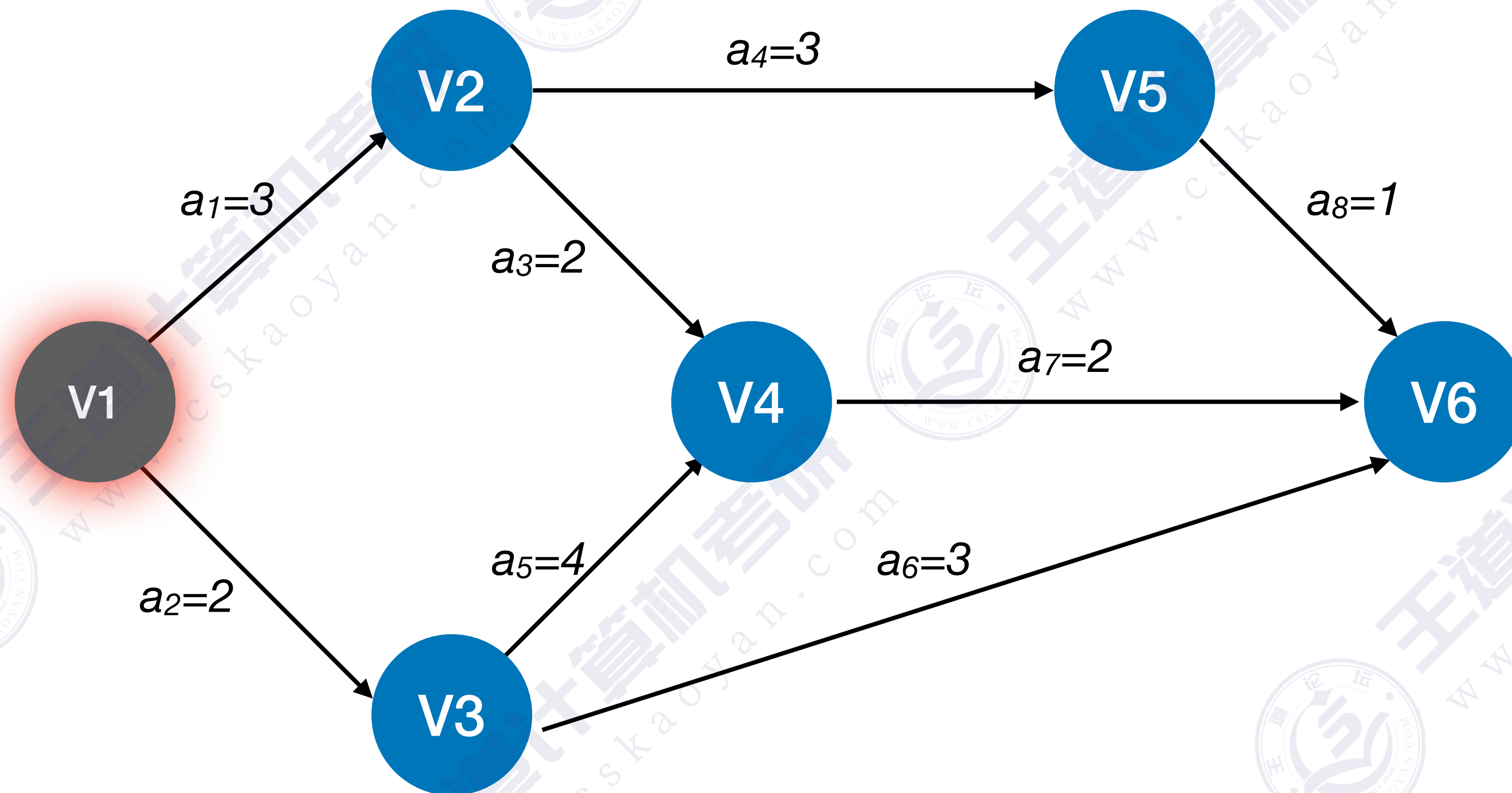
按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: V1、V3、V2、V5、V4、V6

$ve(1) = 0$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

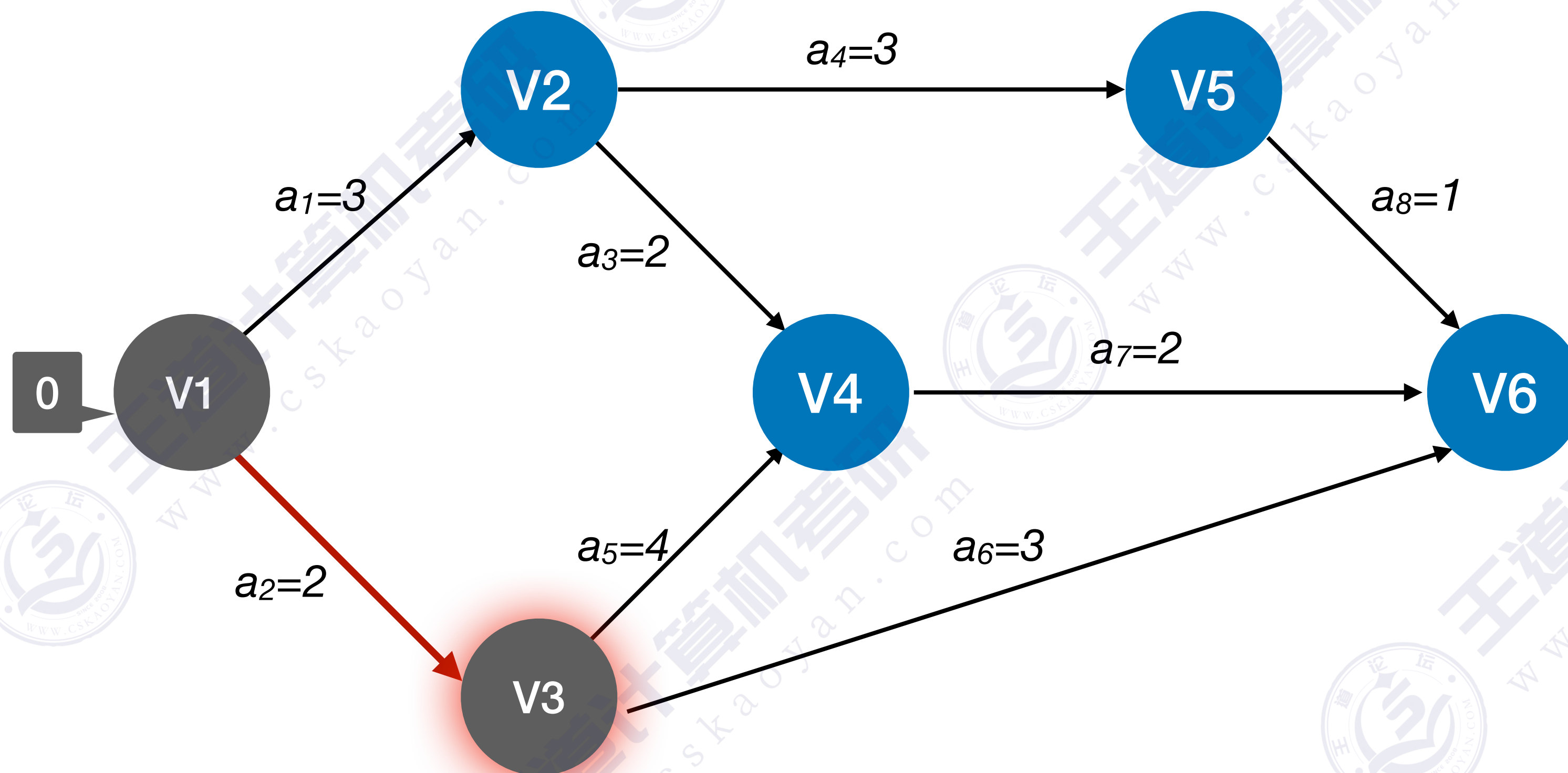
$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: **V1、V3、V2、V5、V4、V6**

$ve(1)=0$

$ve(3)=2$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

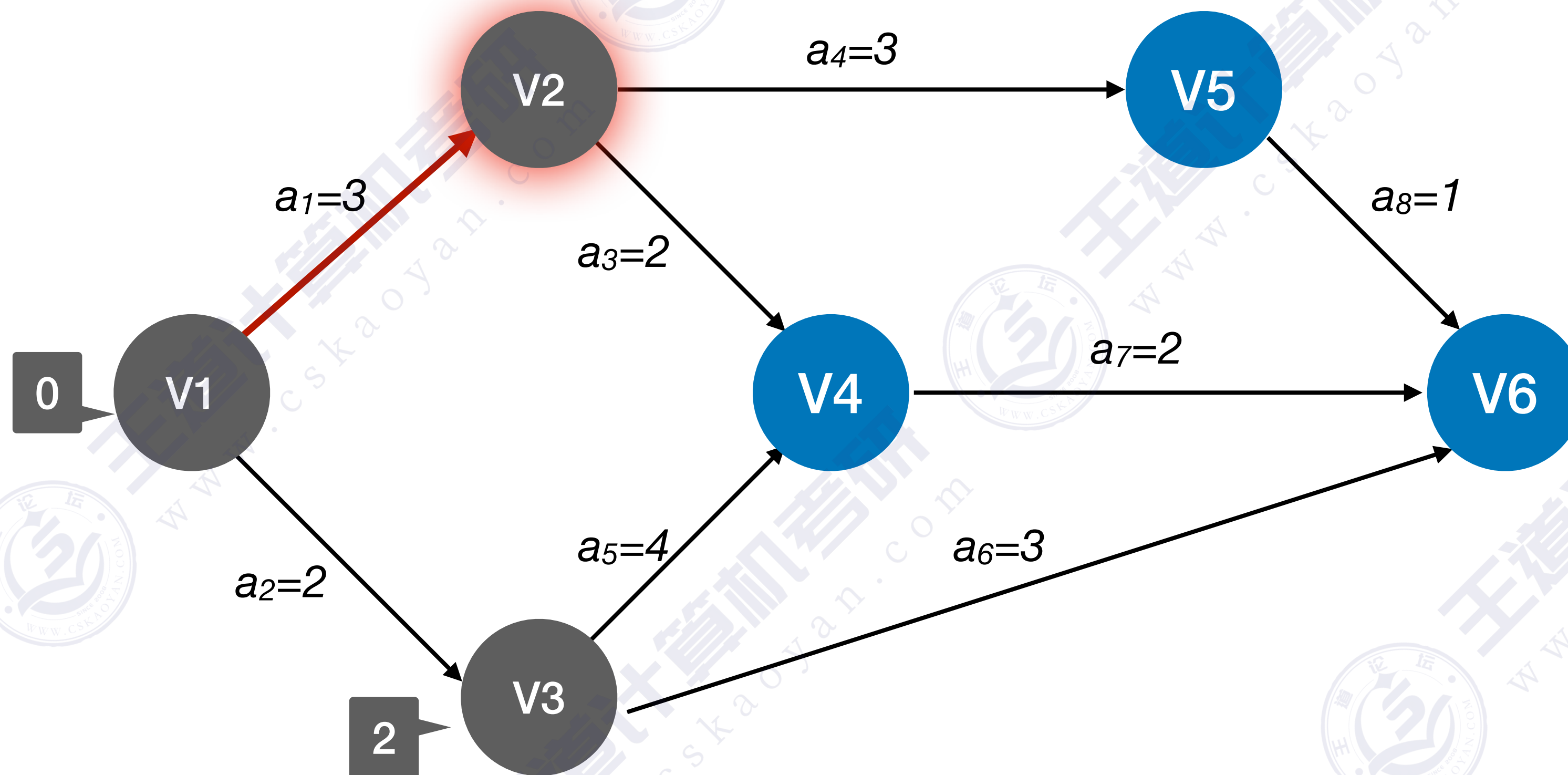
$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: **V1、V3、V2、V5、V4、V6**

$ve(1)=0$

$ve(3)=2$

$ve(2)=3$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

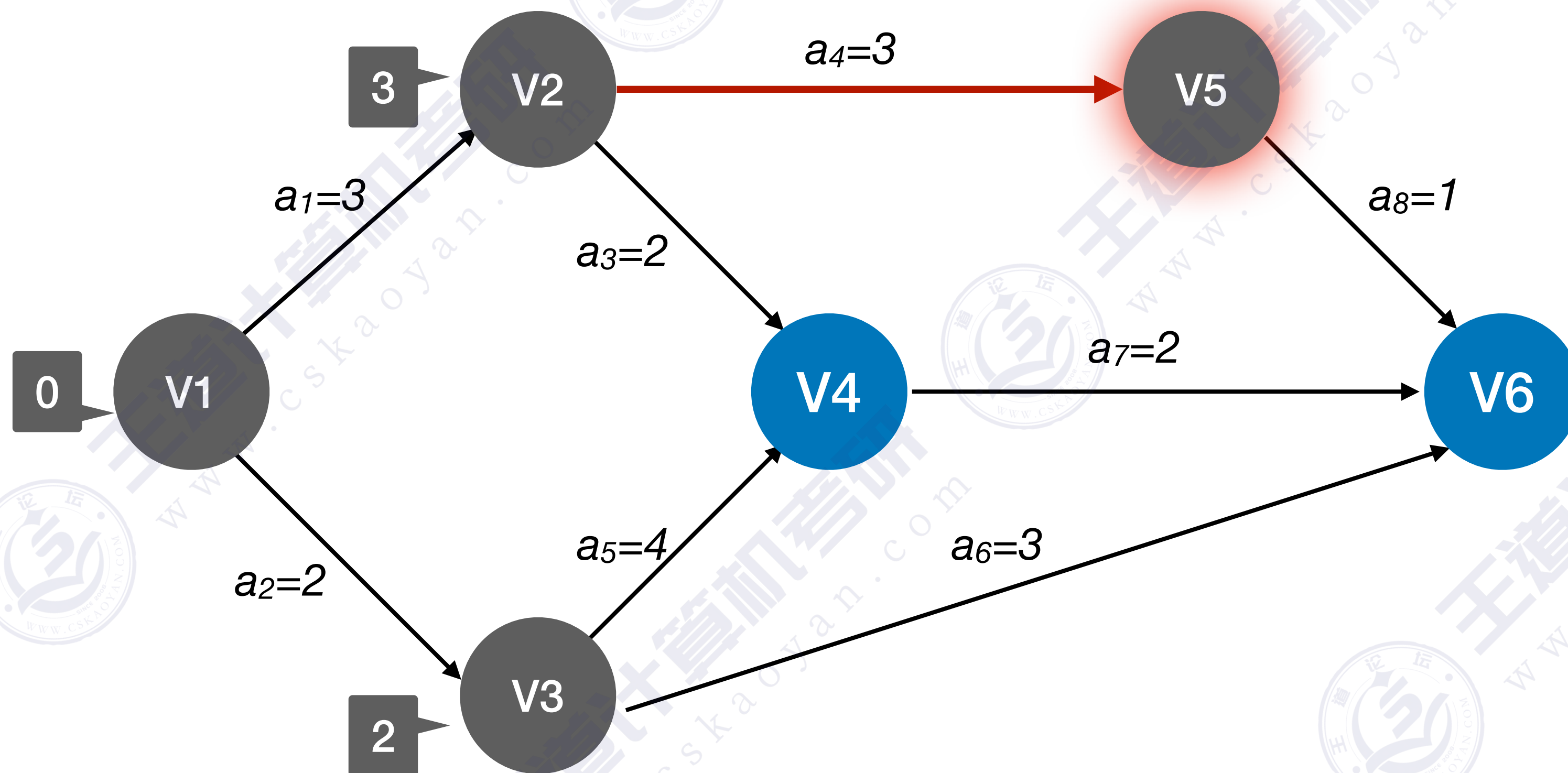
拓扑序列: **V1、V3、V2、V5、V4、V6**

$ve(1)=0$

$ve(3)=2$

$ve(2)=3$

$ve(5)=6$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: **V1、V3、V2、V5、V4、V6**

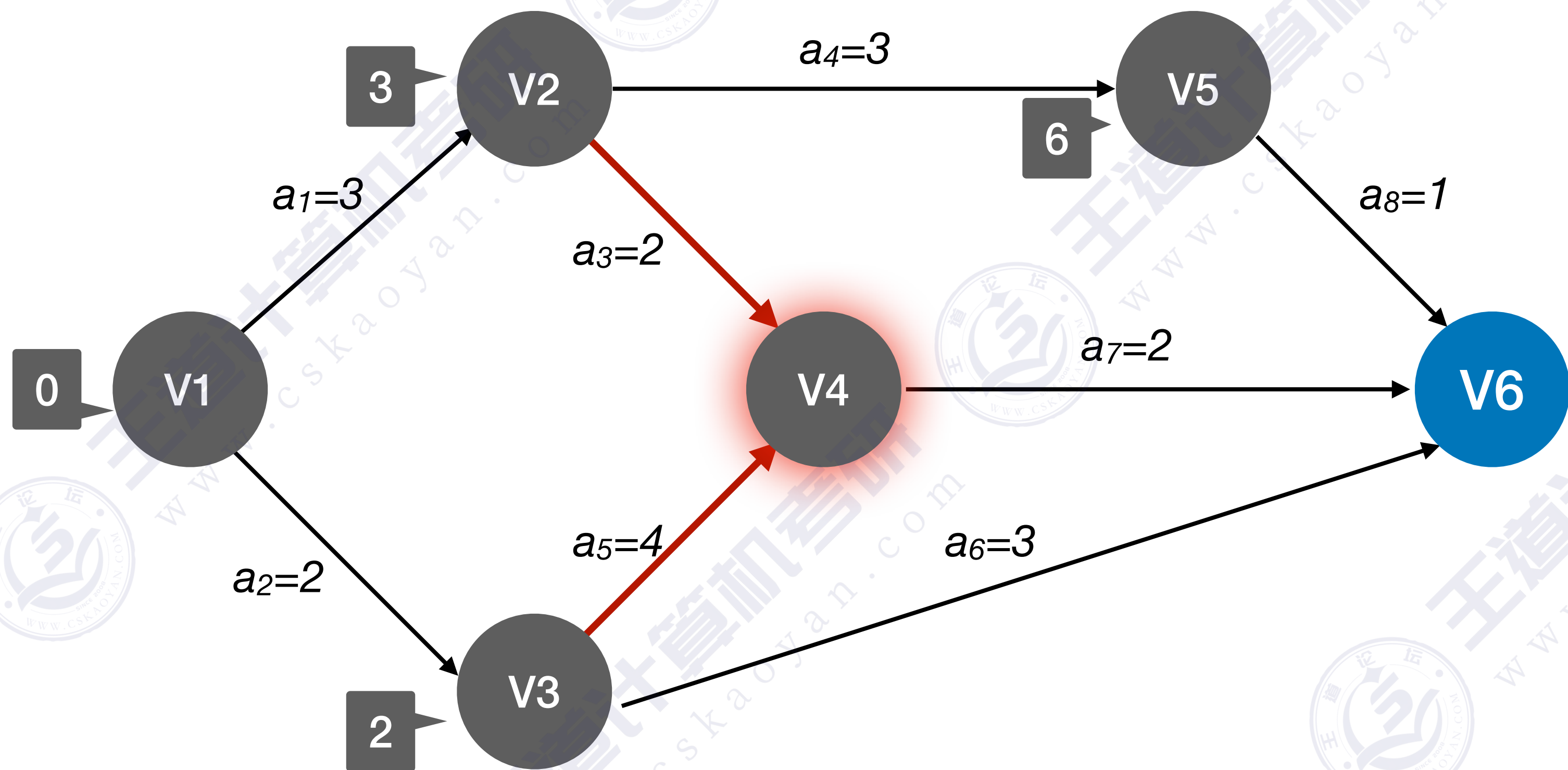
$ve(1) = 0$

$ve(3) = 2$

$ve(2) = 3$

$ve(5) = 6$

$ve(4) = \max\{ve(2) + 2, ve(3) + 4\} = 6$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: **V1、V3、V2、V5、V4、V6**

$ve(1) = 0$

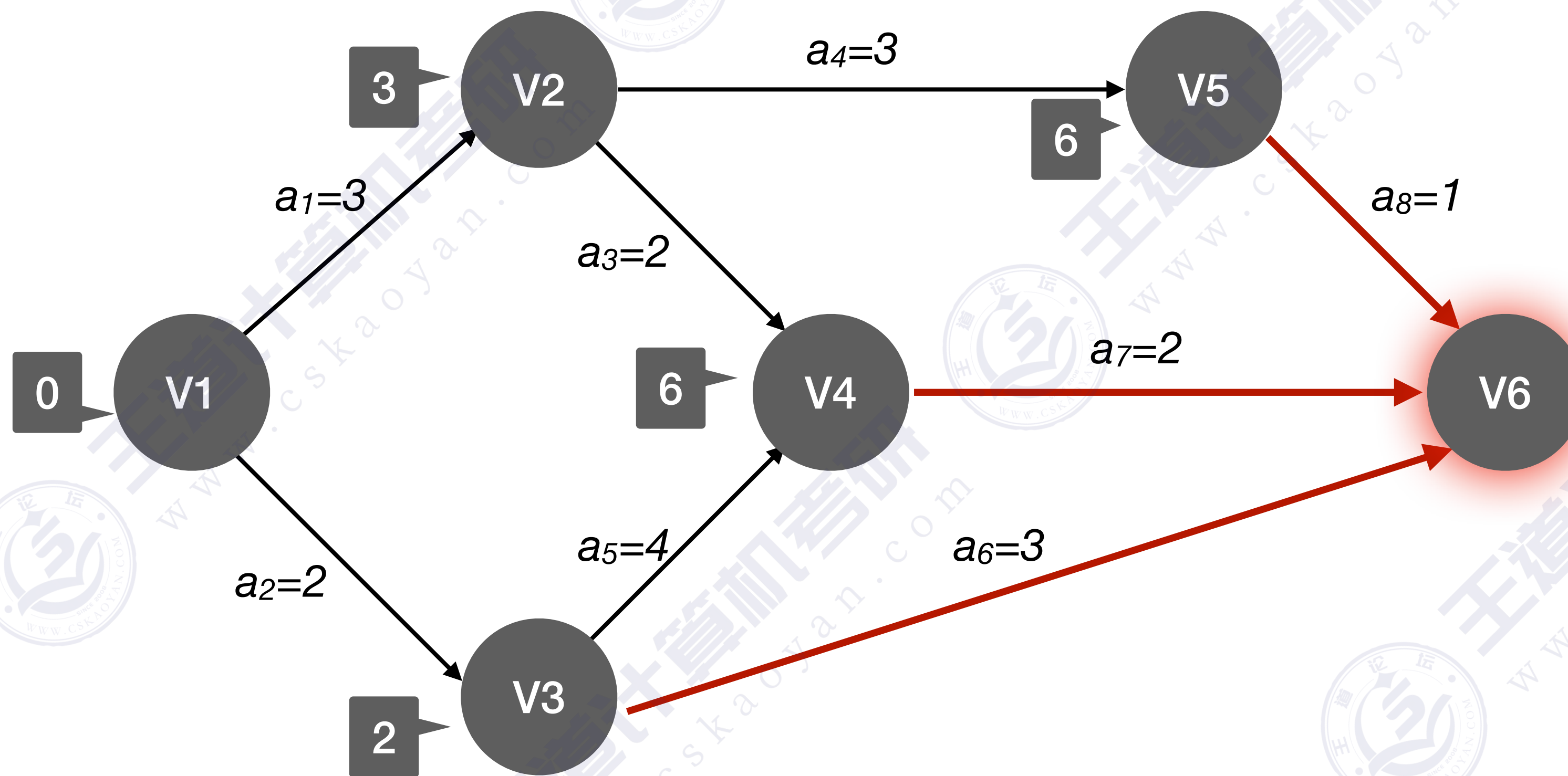
$ve(3) = 2$

$ve(2) = 3$

$ve(5) = 6$

$ve(4) = \max\{ve(2) + 2, ve(3) + 4\} = 6$

$ve(6) = \max\{ve(5) + 1, ve(4) + 2, ve(3) + 3\} = 8$



求所有事件的最早发生时间

① 求所有事件的最早发生时间 $ve()$

按拓扑排序序列，依次求各个顶点的 $ve(k)$:

$ve(\text{源点}) = 0$

$ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

拓扑序列: **V1、V3、V2、V5、V4、V6**

$ve(1)=0$

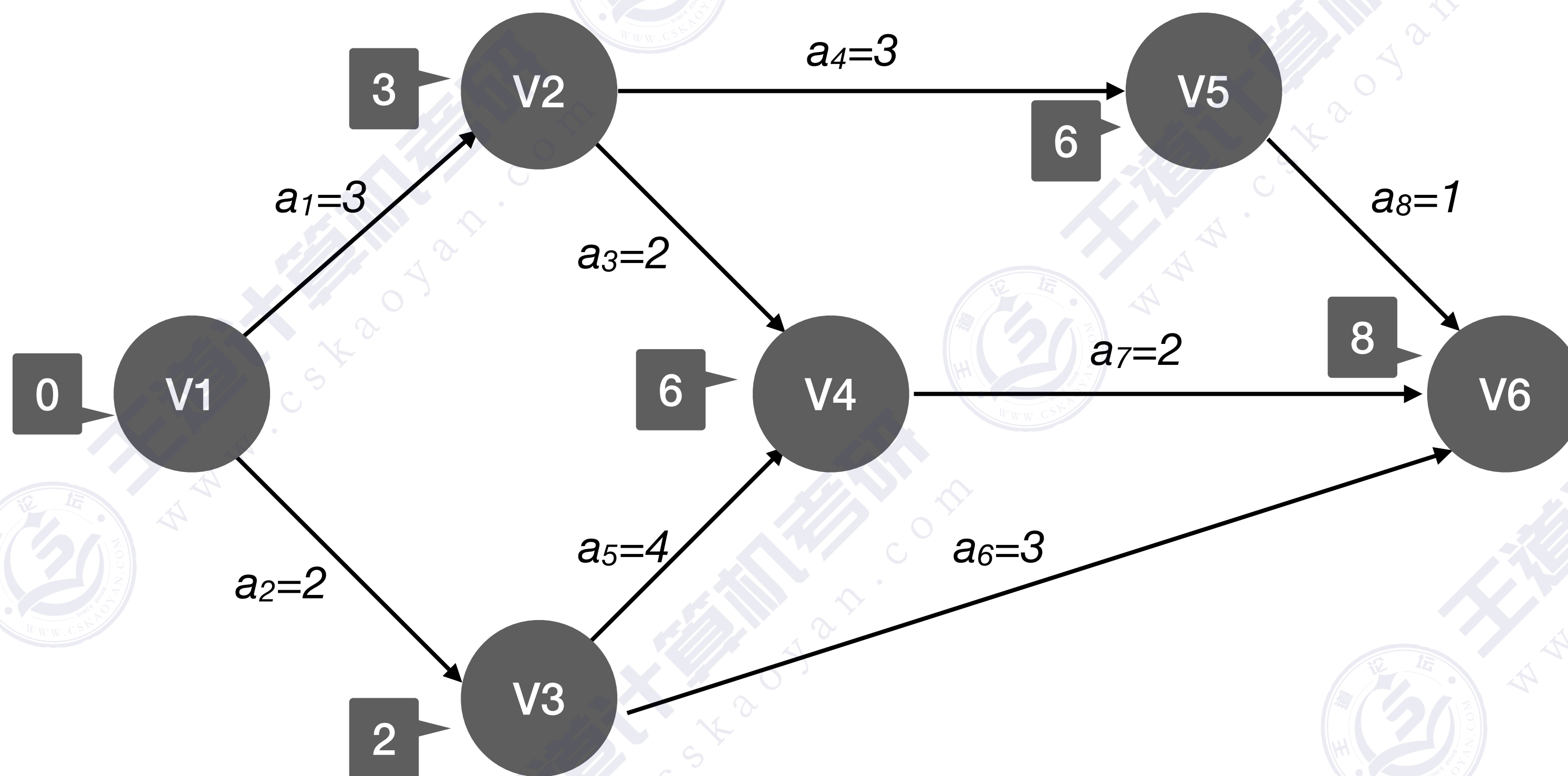
$ve(3)=2$

$ve(2)=3$

$ve(5)=6$

$ve(4)=\max\{ve(2)+2, ve(3)+4\}=6$

$ve(6)=\max\{ve(5)+1, ve(4)+2, ve(3)+3\}=8$



	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8

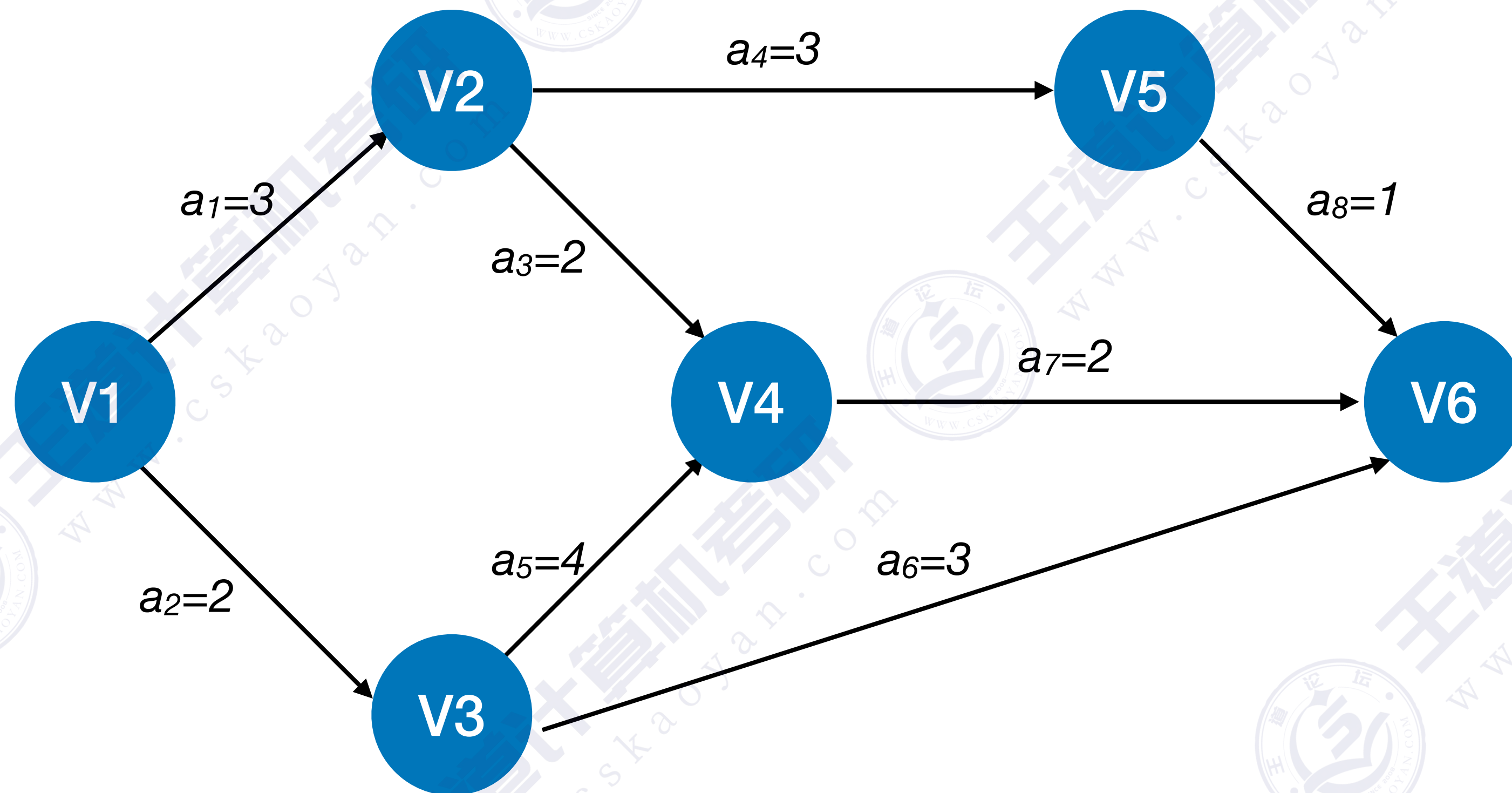
求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继



求所有事件的最迟发生时间

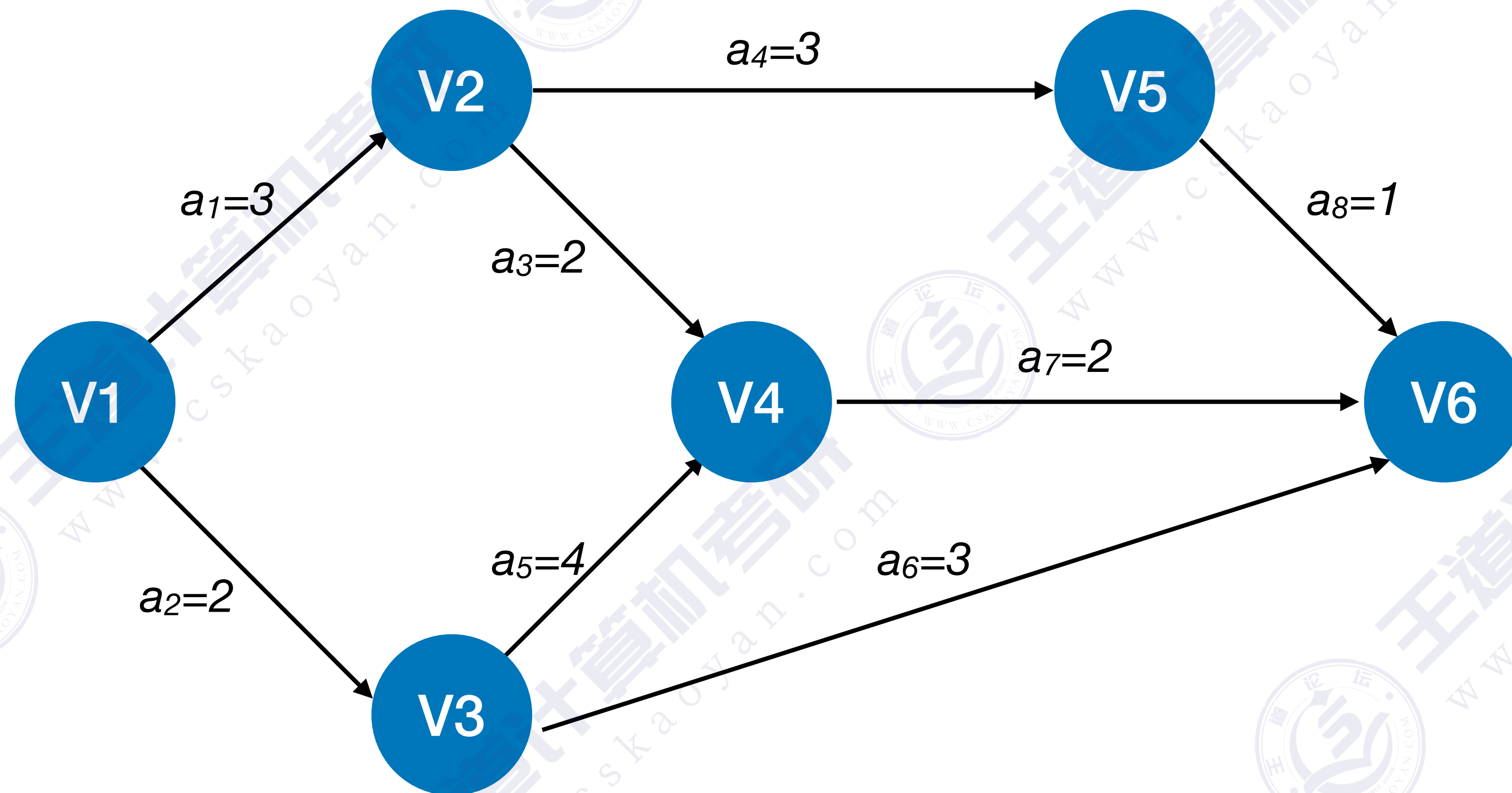
② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: V6、V5、V4、V2、V3、V1



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

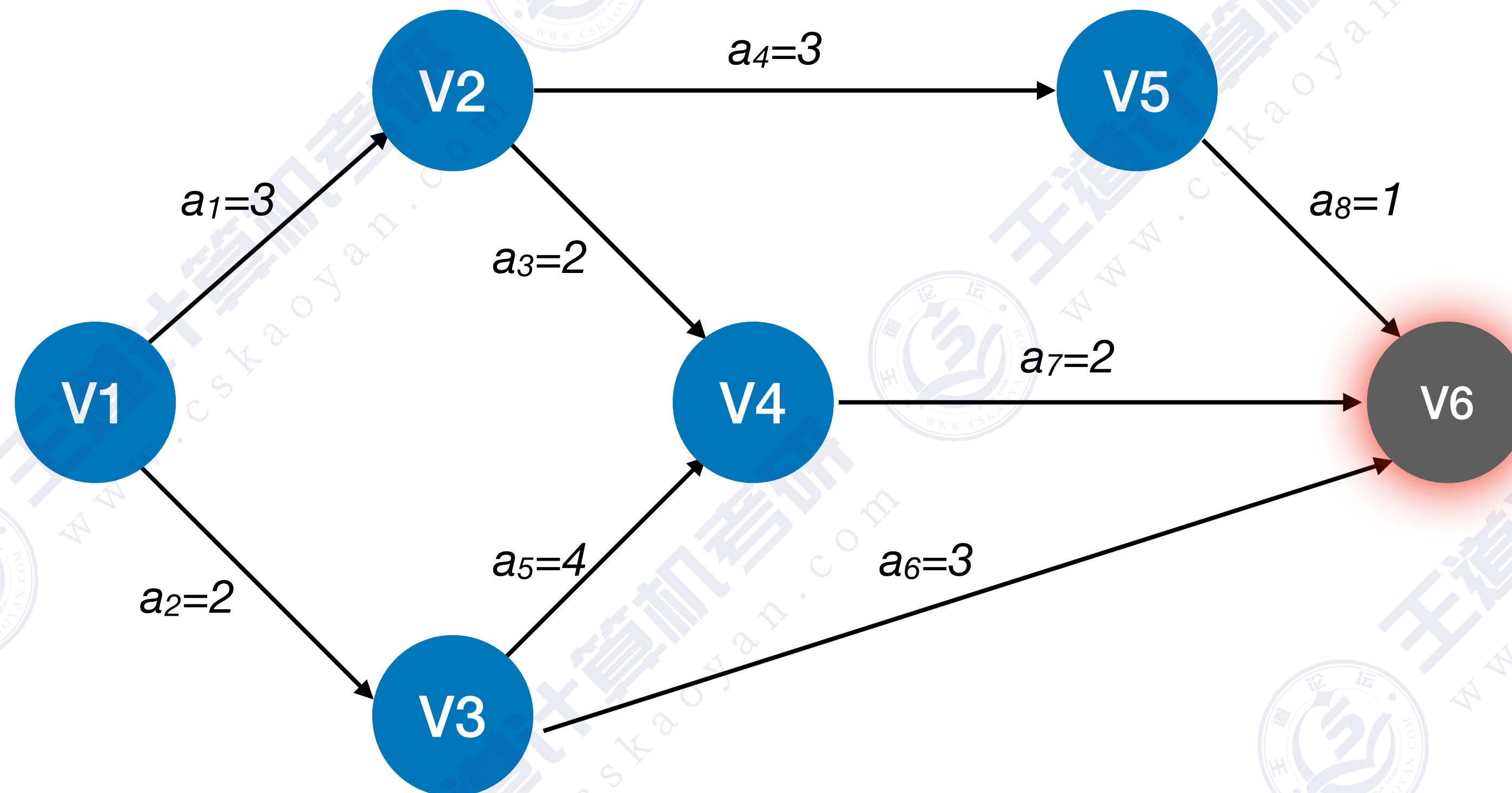
按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: V6、V5、V4、V2、V3、V1

$$vl(6)=8$$



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

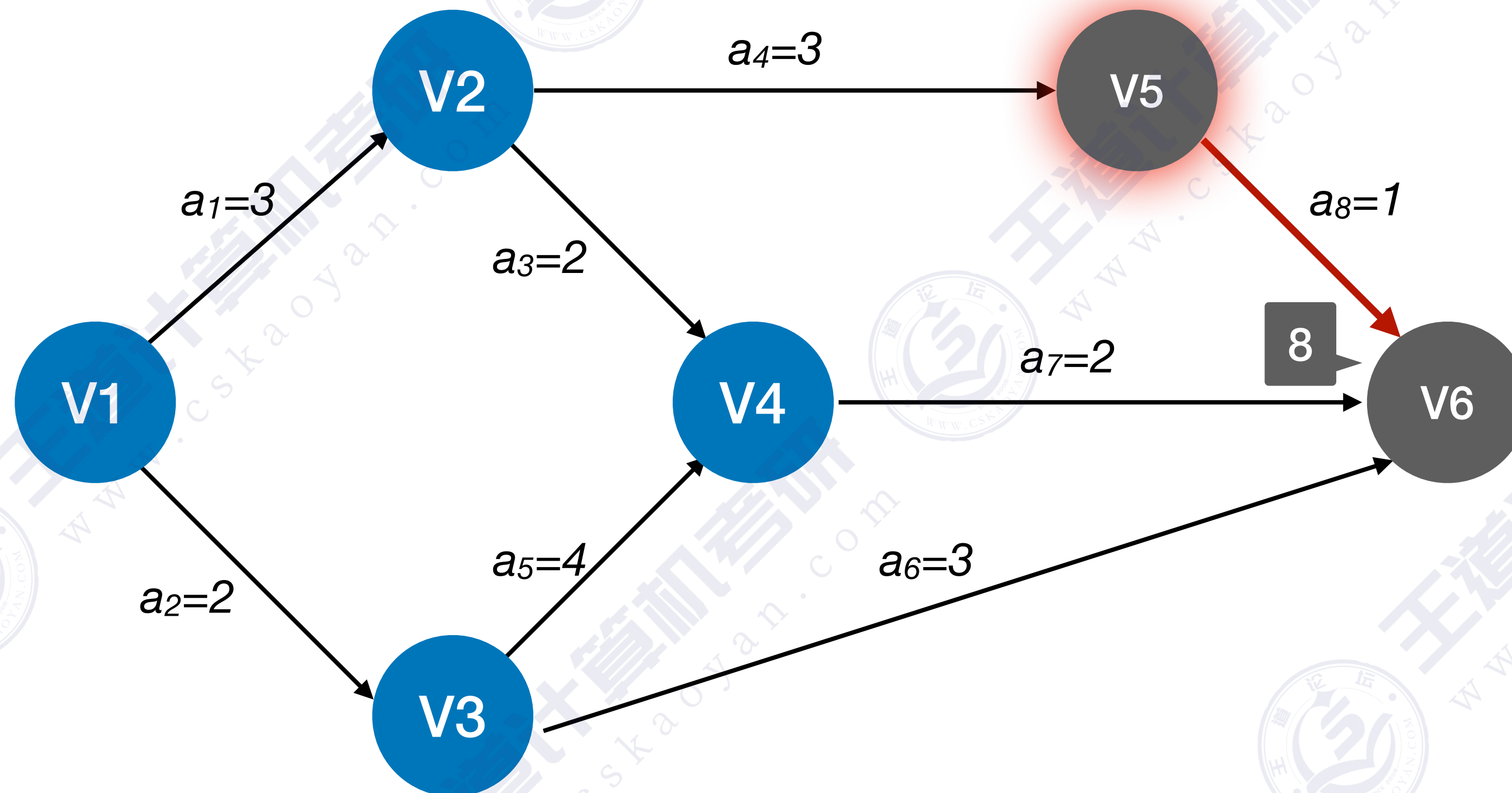
$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: V6、V5、V4、V2、V3、V1

$vl(6)=8$

$vl(5)=7$



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

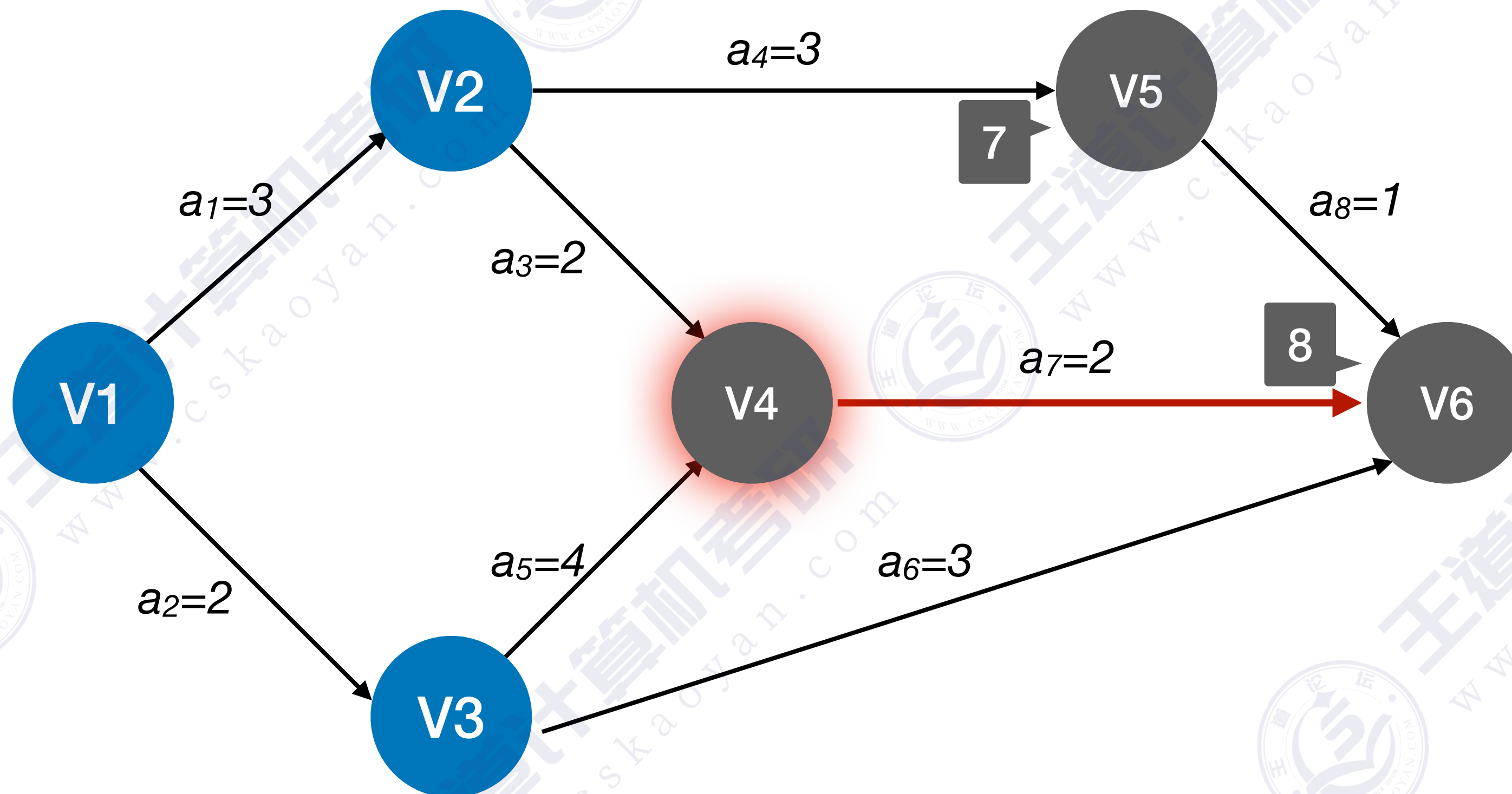
$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: V6、V5、V4、V2、V3、V1

$vl(6)=8$

$vl(5)=7$

$vl(4)=6$



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

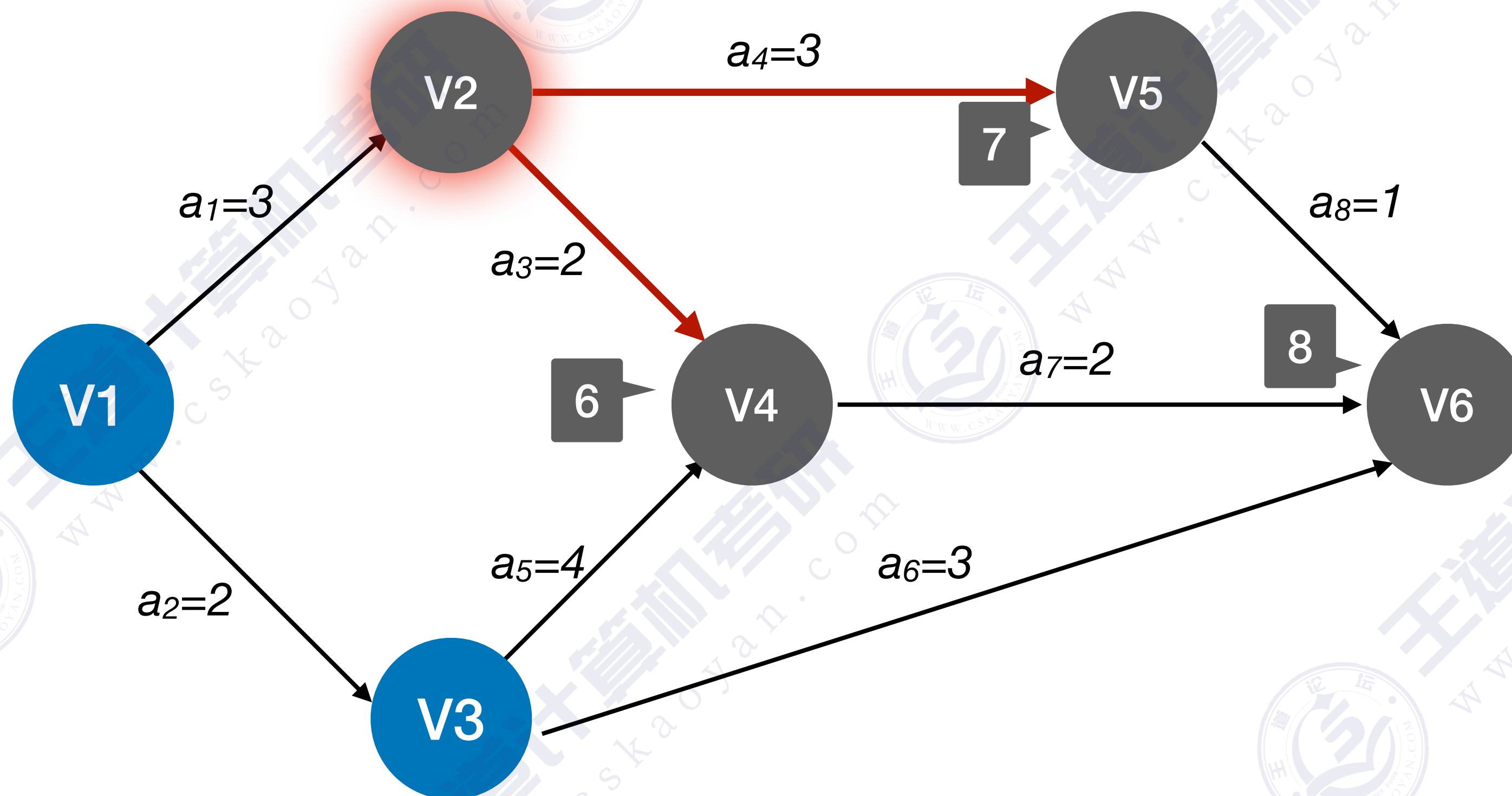
逆拓扑序列: V6、V5、V4、V2、V3、V1

$$vl(6)=8$$

$$vl(5)=7$$

$$vl(4)=6$$

$$vl(2)=\min\{vl(5)-1, vl(4)-2\}=4$$



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: **V6、V5、V4、V2、V3、V1**

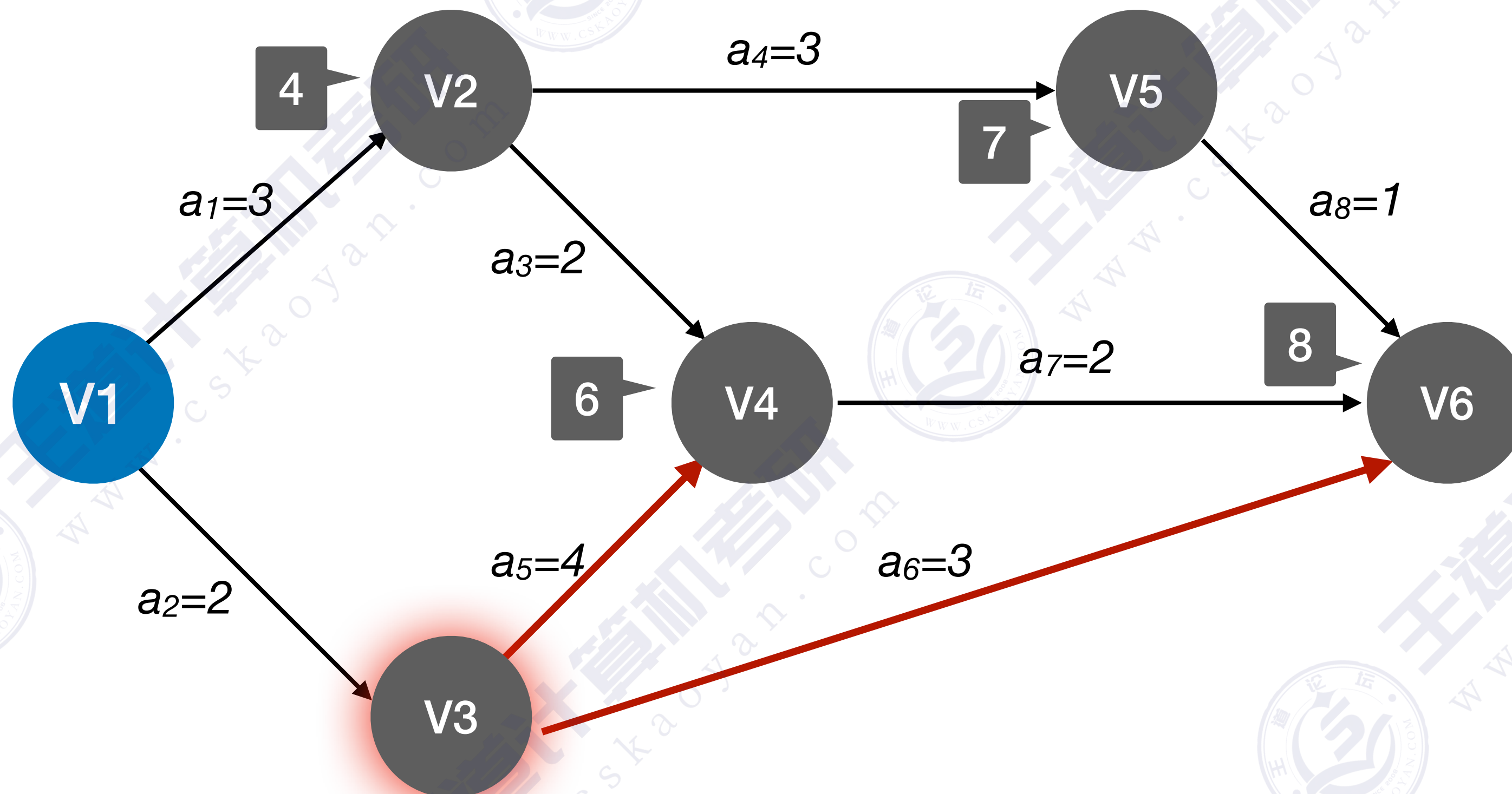
$$vl(6)=8$$

$$vl(5)=7$$

$$vl(4)=6$$

$$vl(2)=\min\{vl(5)-1, vl(4)-2\}=4$$

$$vl(3)=\min\{vl(4)-4, vl(6)-3\}=2$$



求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$:

$vl(\text{汇点}) = ve(\text{汇点})$

$vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

逆拓扑序列: **V6、V5、V4、V2、V3、V1**

$$vl(6)=8$$

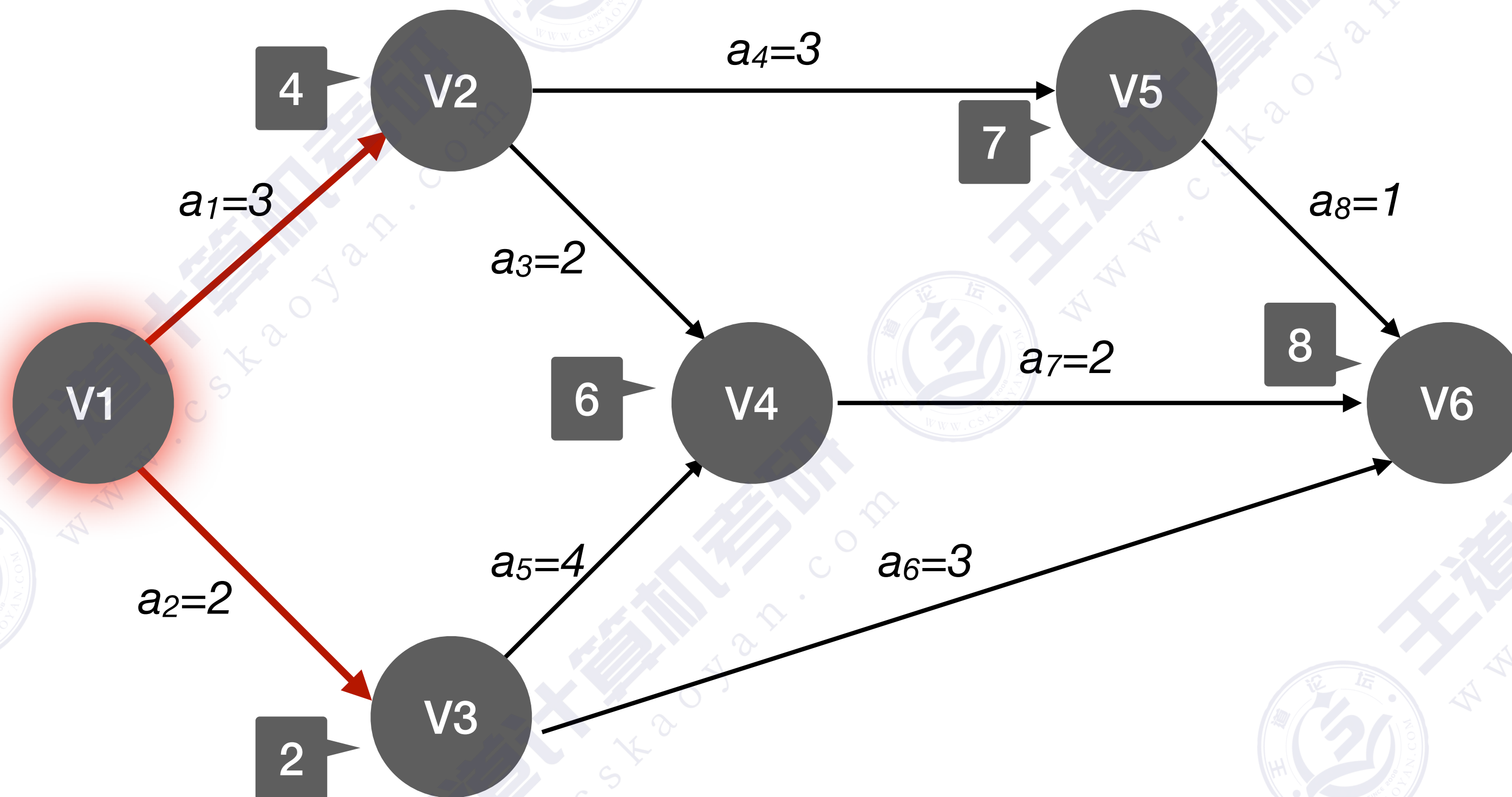
$$vl(5)=7$$

$$vl(4)=6$$

$$vl(2)=\min\{vl(5)-1, vl(4)-2\}=4$$

$$vl(3)=\min\{vl(4)-4, vl(6)-3\}=2$$

$$vl(1)=0$$



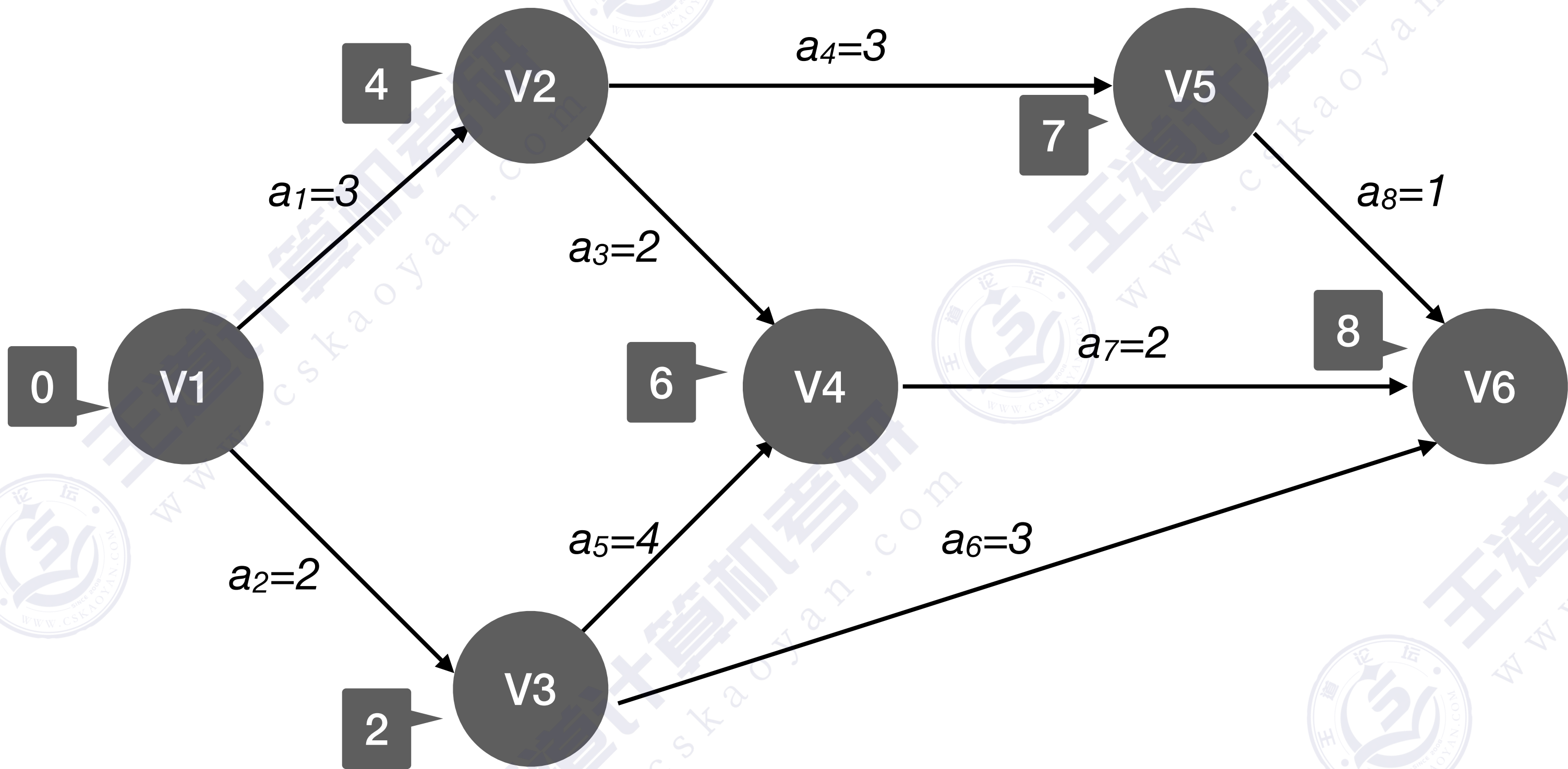
求所有事件的最迟发生时间

② 求所有事件的最迟发生时间 $vl()$

按逆拓扑排序序列，依次求各个顶点的 $vl(k)$ ：
 $vl(\text{汇点}) = ve(\text{汇点})$
 $vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$ ， v_j 为 v_k 的任意后继

逆拓扑序列：V6、V5、V4、V2、V3、V1

$vl(6)=8$
 $vl(5)=7$
 $vl(4)=6$
 $vl(2)=\min\{vl(5)-1, vl(4)-2\}=4$
 $vl(3)=\min\{vl(4)-4, vl(6)-3\}=2$
 $vl(1)=0$



	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

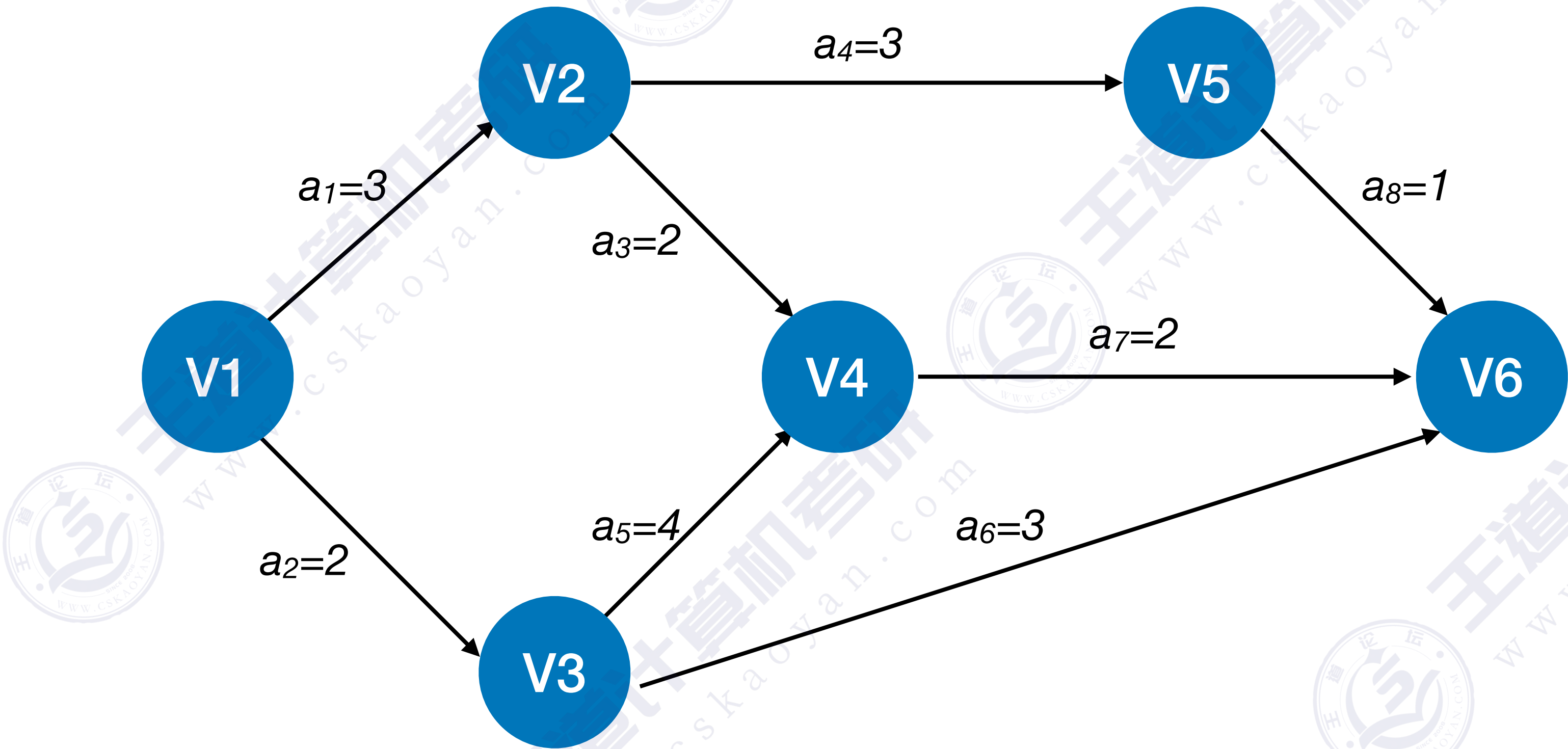
求所有活动的最早发生时间



③ 求所有活动的最早发生时间 $e()$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i ，则有 $e(i) = ve(k)$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8



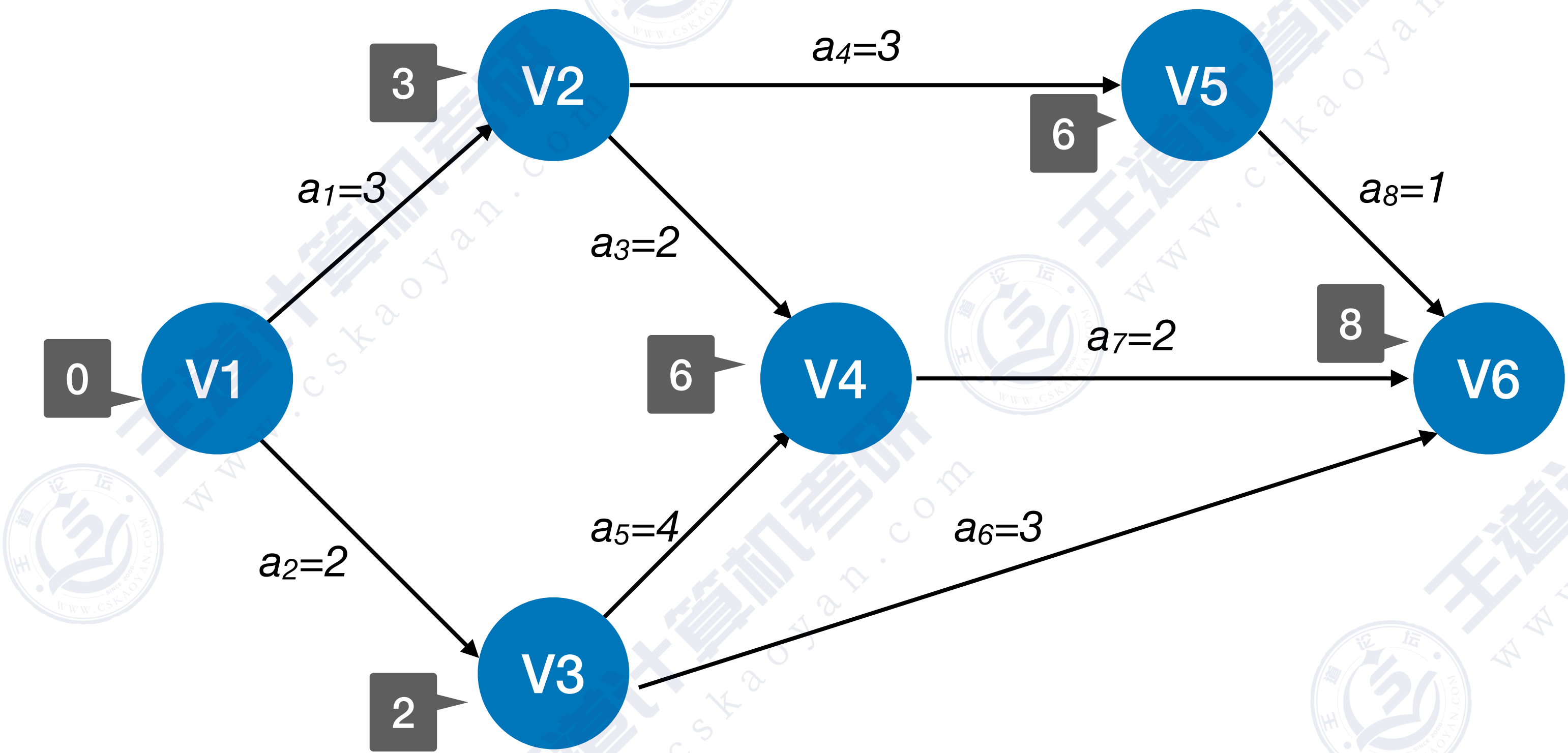
求所有活动的最早发生时间

③ 求所有活动的最早发生时间 $e()$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i ，则有 $e(i) = ve(k)$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

	a1	a2	a3	a4	a5	a6	a7	a8
$e(k)$	0	0	3	3	2	2	6	6

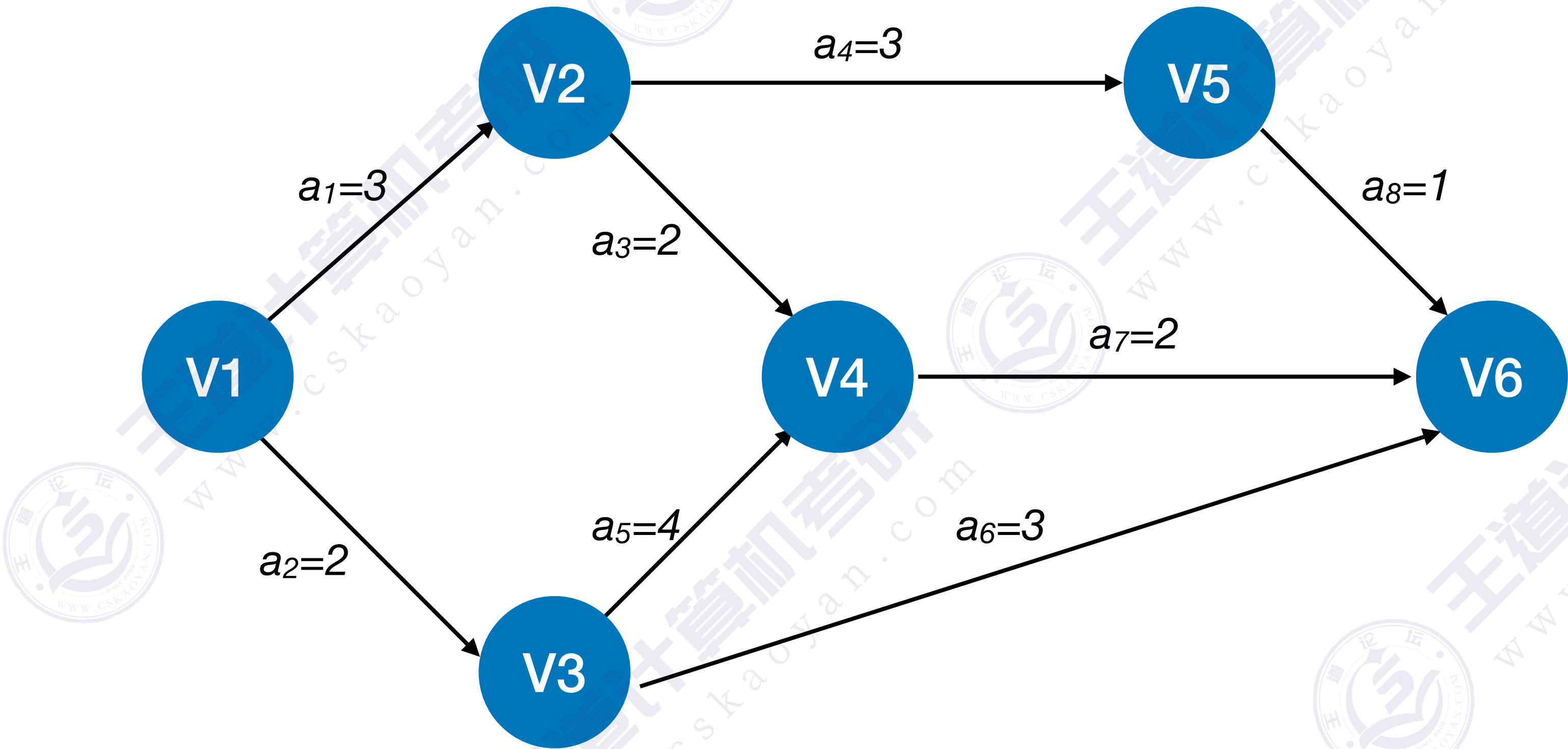


求所有活动的最迟发生时间

④ 求所有活动的最迟发生时间 $l(i)$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = vl(j) - \text{Weight}(v_k, v_j)$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

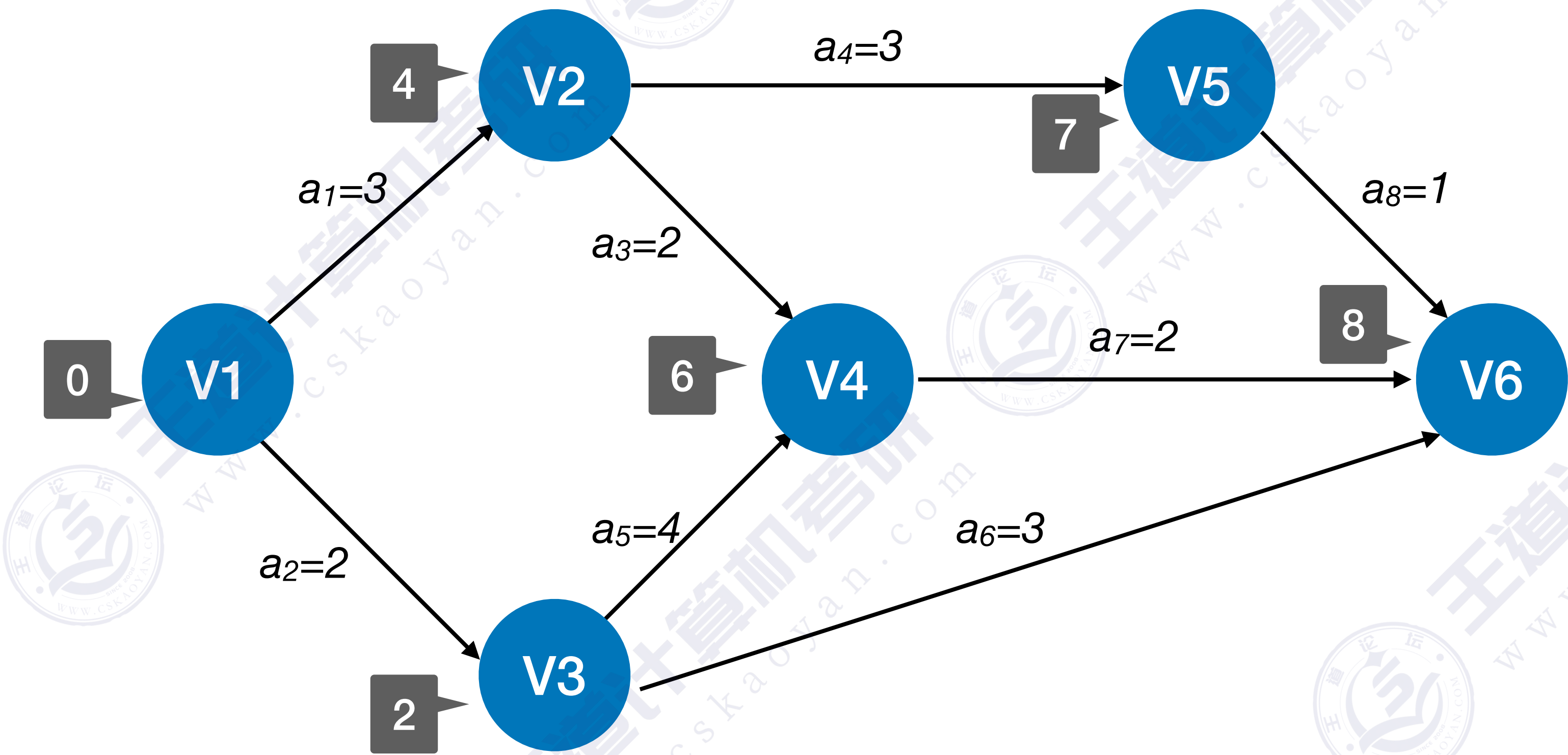


求所有活动的最迟发生时间

④ 求所有活动的最迟发生时间 $l(i)$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = vl(j) - \text{Weight}(v_k, v_j)$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8



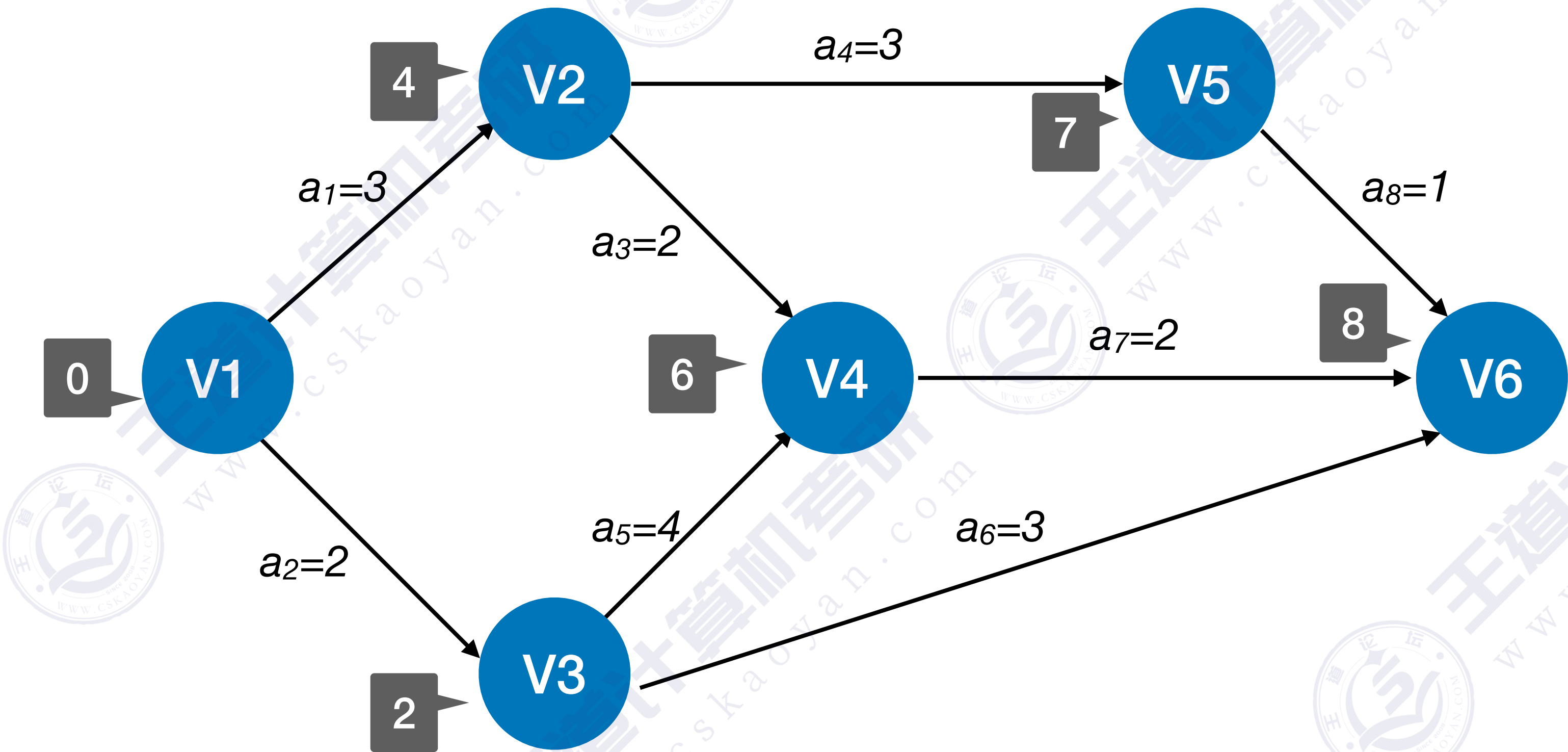
求所有活动的最迟发生时间

④ 求所有活动的最迟发生时间 $l(i)$

若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = vl(j) - \text{Weight}(v_k, v_j)$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(k)$	0	0	3	3	2	2	6	6
$l(k)$	1	0	4	4	2	5	6	7



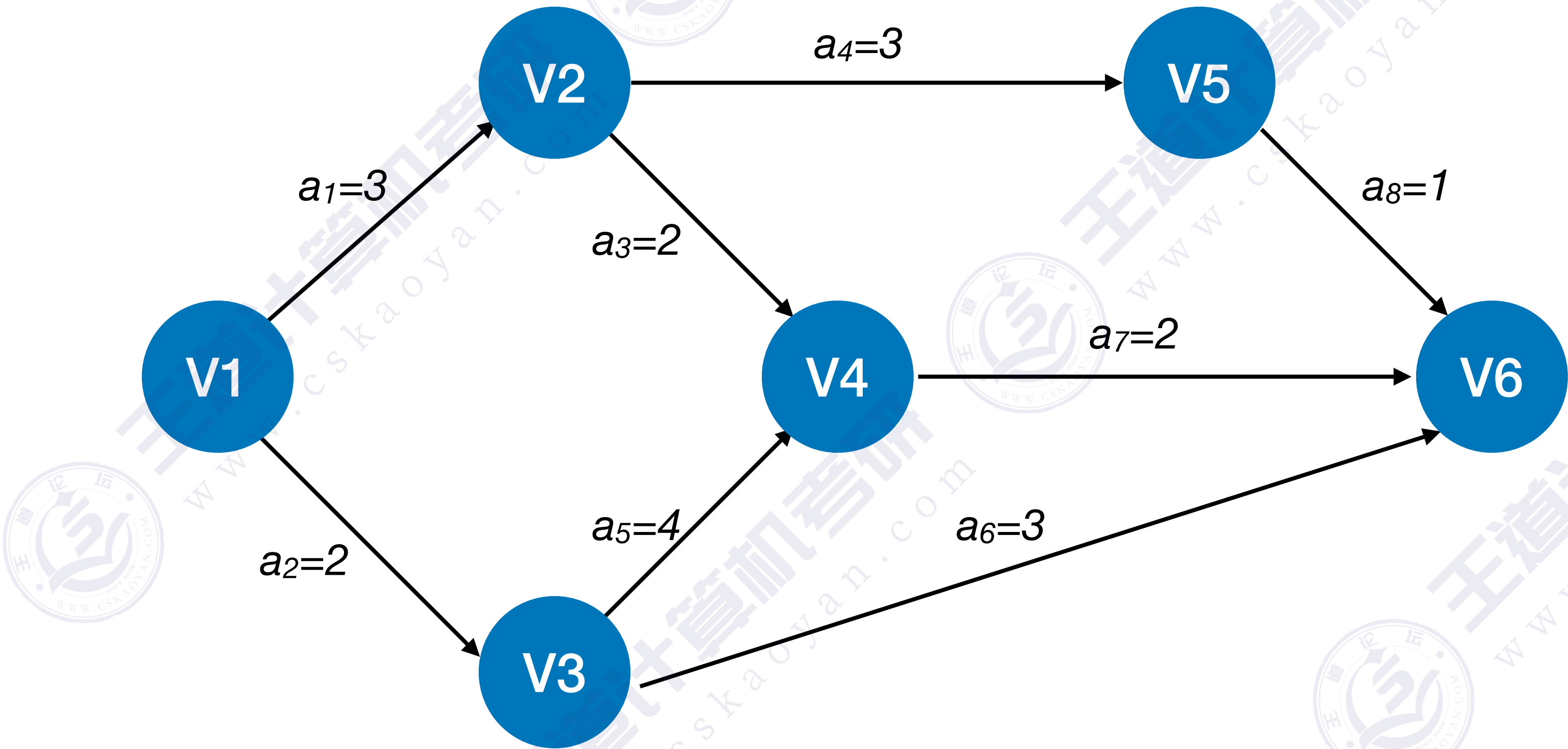
求所有活动的时间余量

⑤ 求所有活动的时间余量 $d()$

$$d(i) = l(i) - e(i)$$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

	a1	a2	a3	a4	a5	a6	a7	a8
$e(k)$	0	0	3	3	2	2	6	6
$l(k)$	1	0	4	4	2	5	6	7



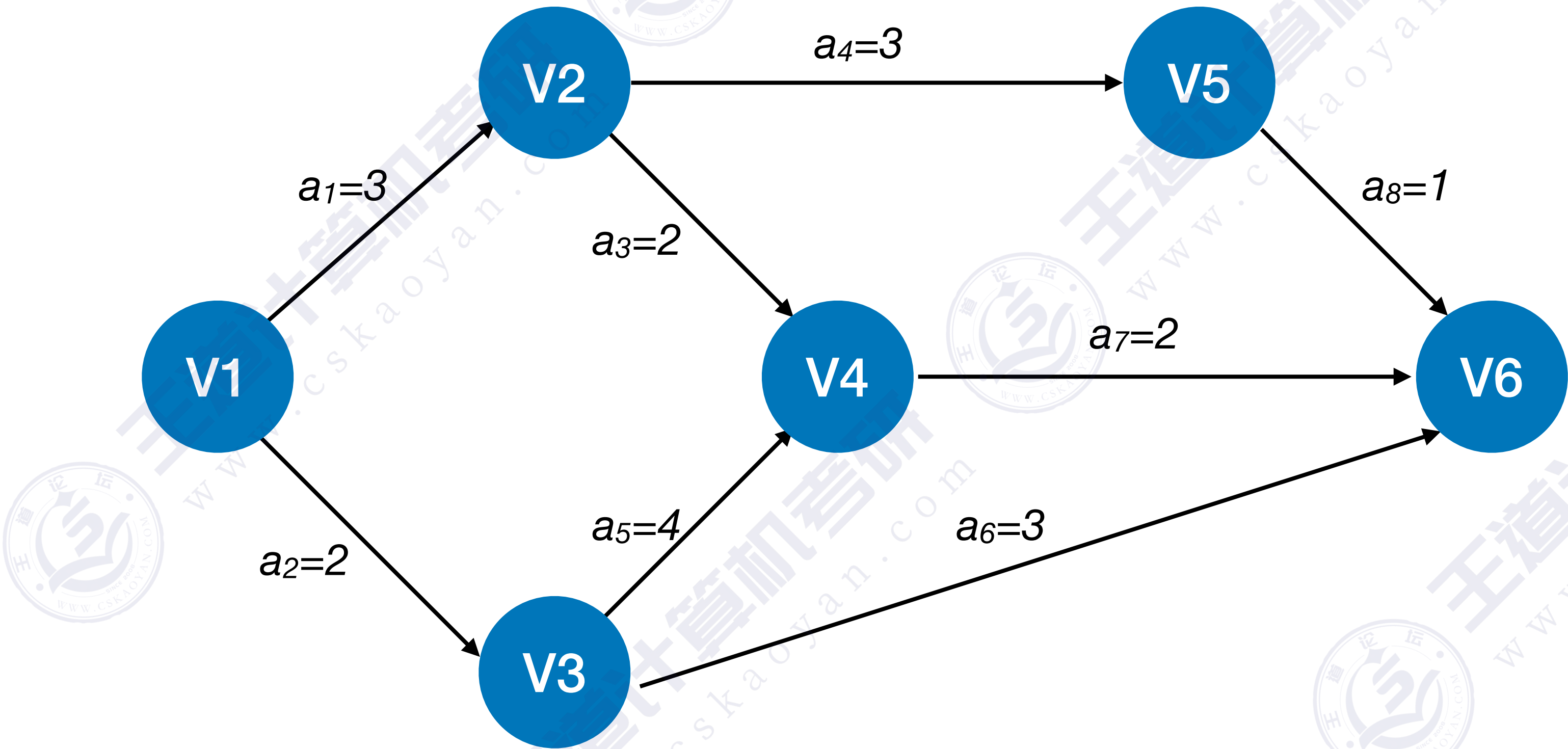
求所有活动的时间余量

⑤ 求所有活动的时间余量 $d()$

$$d(i) = l(i) - e(i)$$

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

	a1	a2	a3	a4	a5	a6	a7	a8
$e(k)$	0	0	3	3	2	2	6	6
$l(k)$	1	0	4	4	2	5	6	7
$d(k)$	1	0	1	1	0	3	0	1



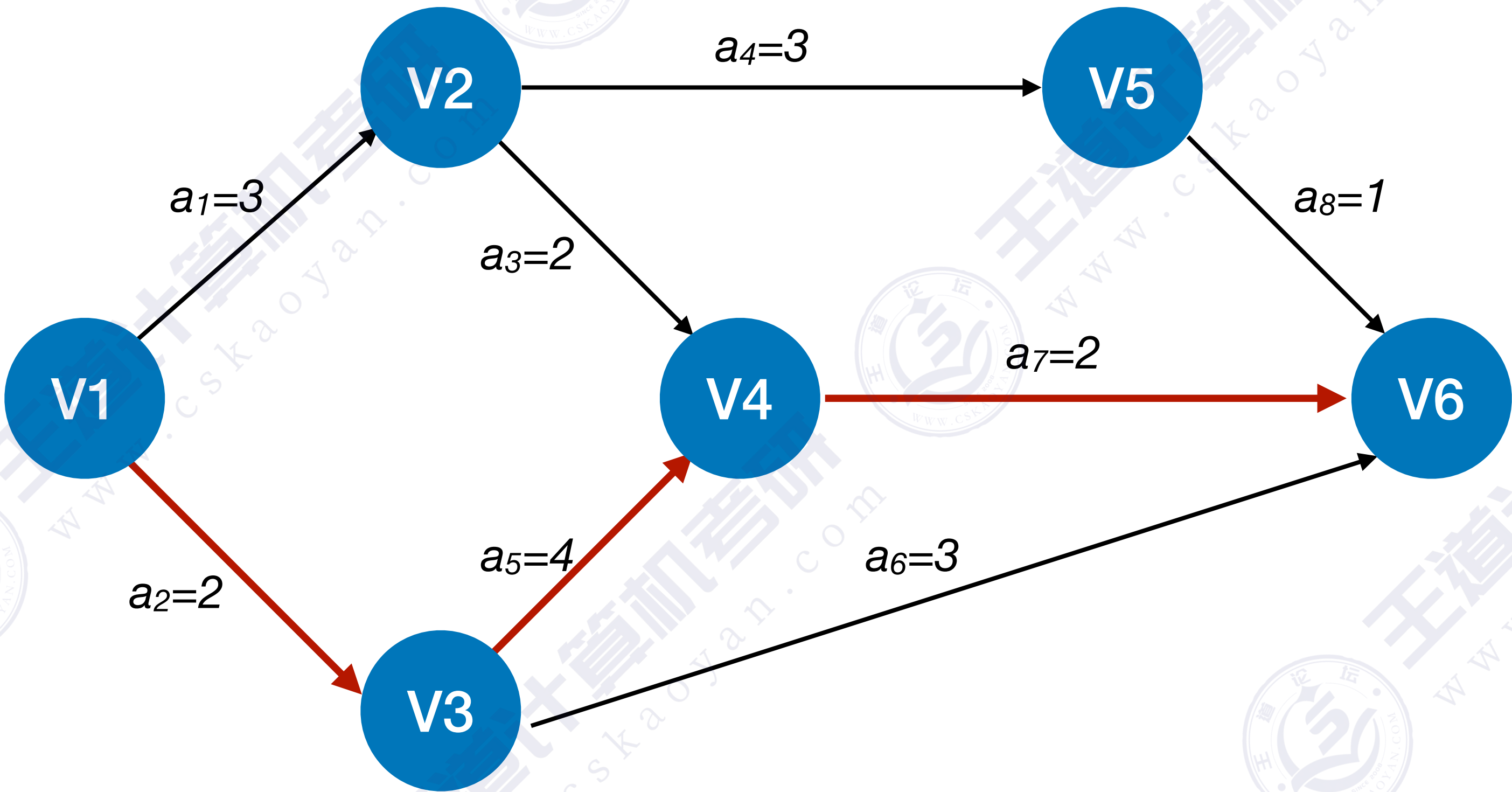
求得关键活动、关键路径

- ① 求所有事件的最早发生时间 $ve()$
- ② 求所有事件的最迟发生时间 $vl()$
- ③ 求所有活动的最早发生时间 $e()$
- ④ 求所有活动的最迟发生时间 $l()$
- ⑤ 求所有活动的时间余量 $d()$

$d(k)=0$ 的活动就是关键活动, 由关键活动可得关键路径

	V1	V2	V3	V4	V5	V6
$ve(k)$	0	3	2	6	6	8
$vl(k)$	0	4	2	6	7	8

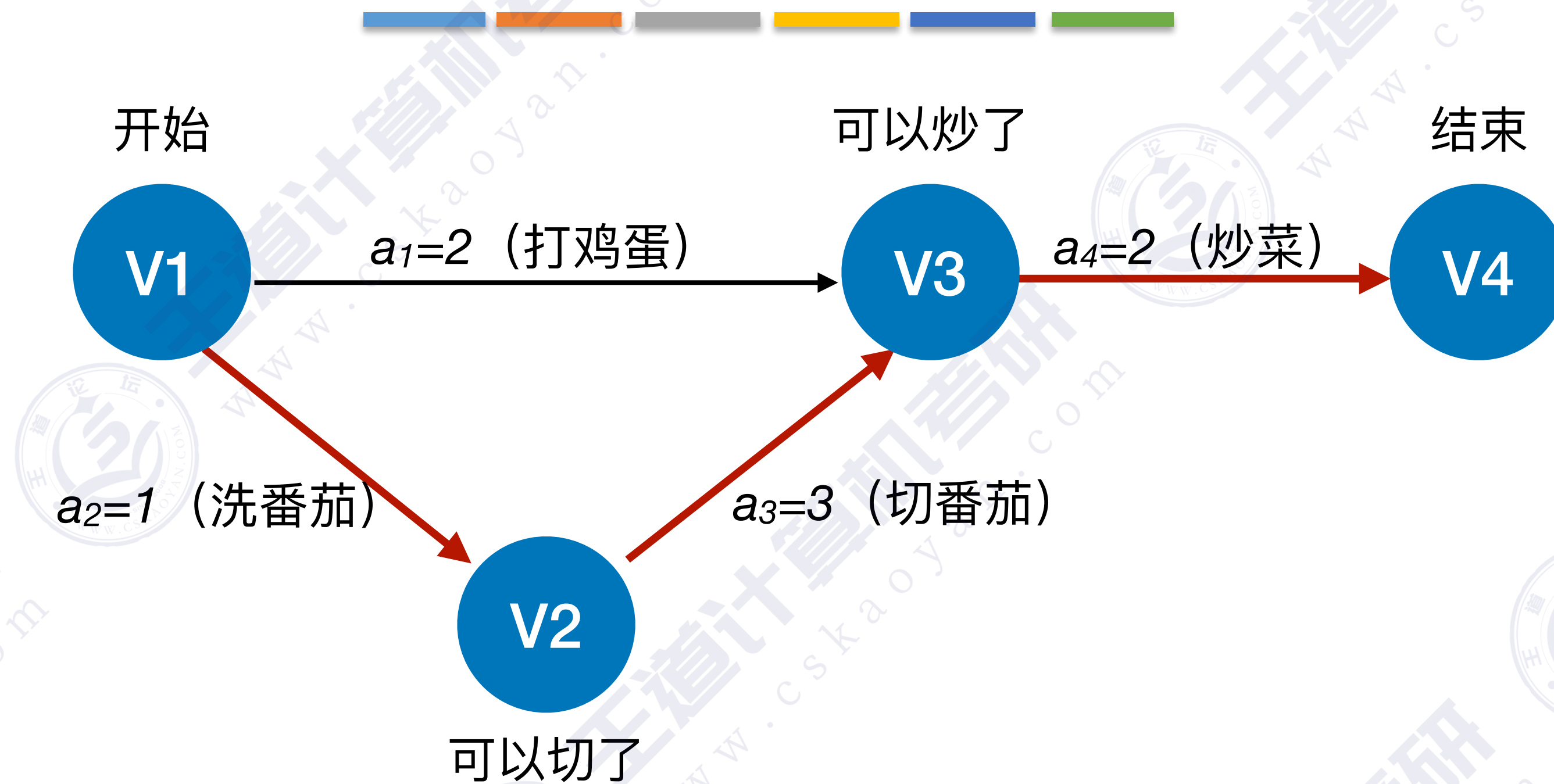
	a1	a2	a3	a4	a5	a6	a7	a8
$e(k)$	0	0	3	3	2	2	6	6
$l(k)$	1	0	4	4	2	5	6	7
$d(k)$	1	0	1	1	0	3	0	1



关键活动: a_2 、 a_5 、 a_7

关键路径: $V1 \rightarrow V3 \rightarrow V4 \rightarrow V6$

关键活动、关键路径的特性

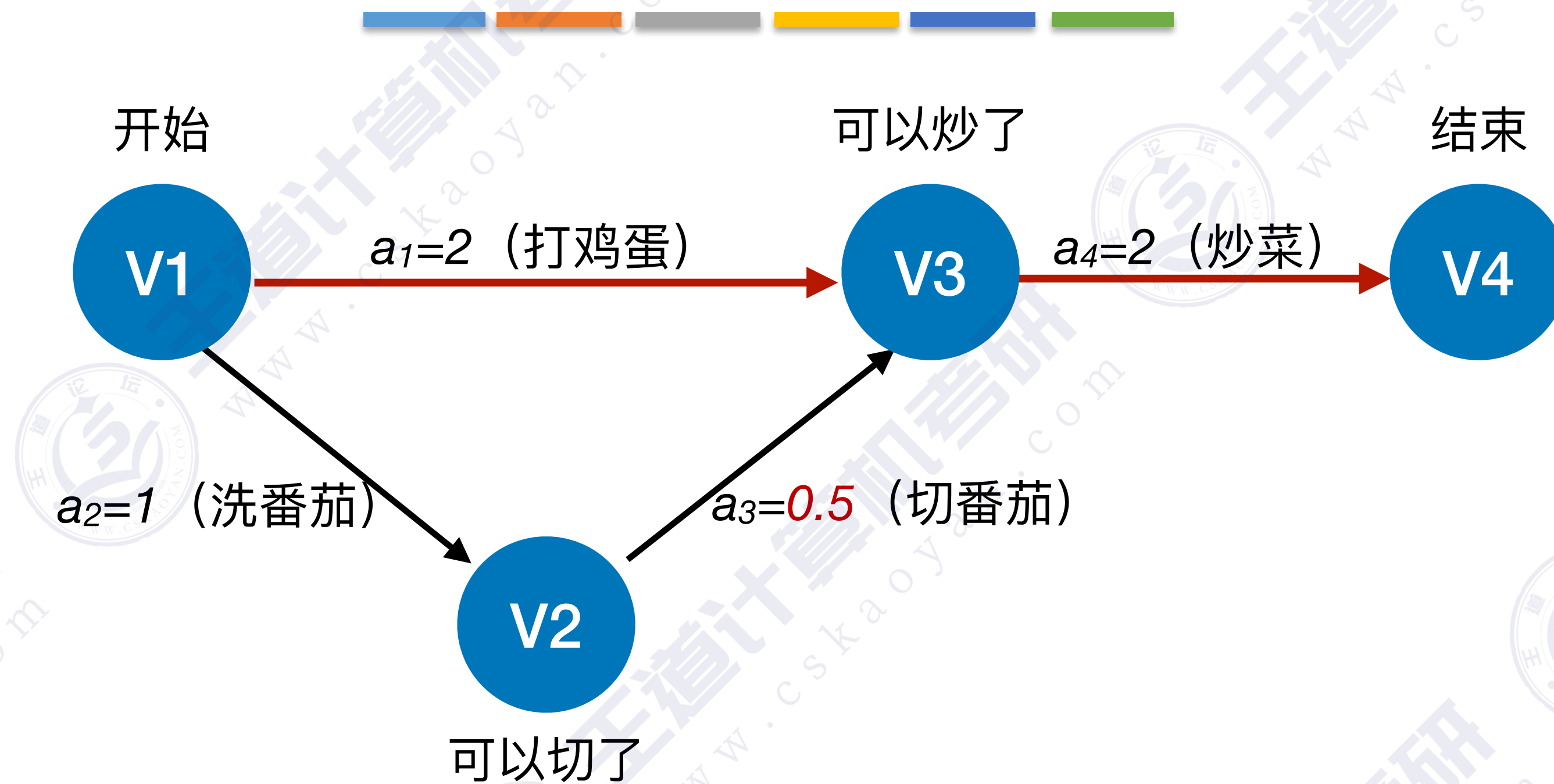


若关键活动耗时增加，则整个工程的工期将增长

缩短关键活动的时间，可以缩短整个工程的工期

当缩短到一定程度时，关键活动可能会变成非关键活动

关键活动、关键路径的特性

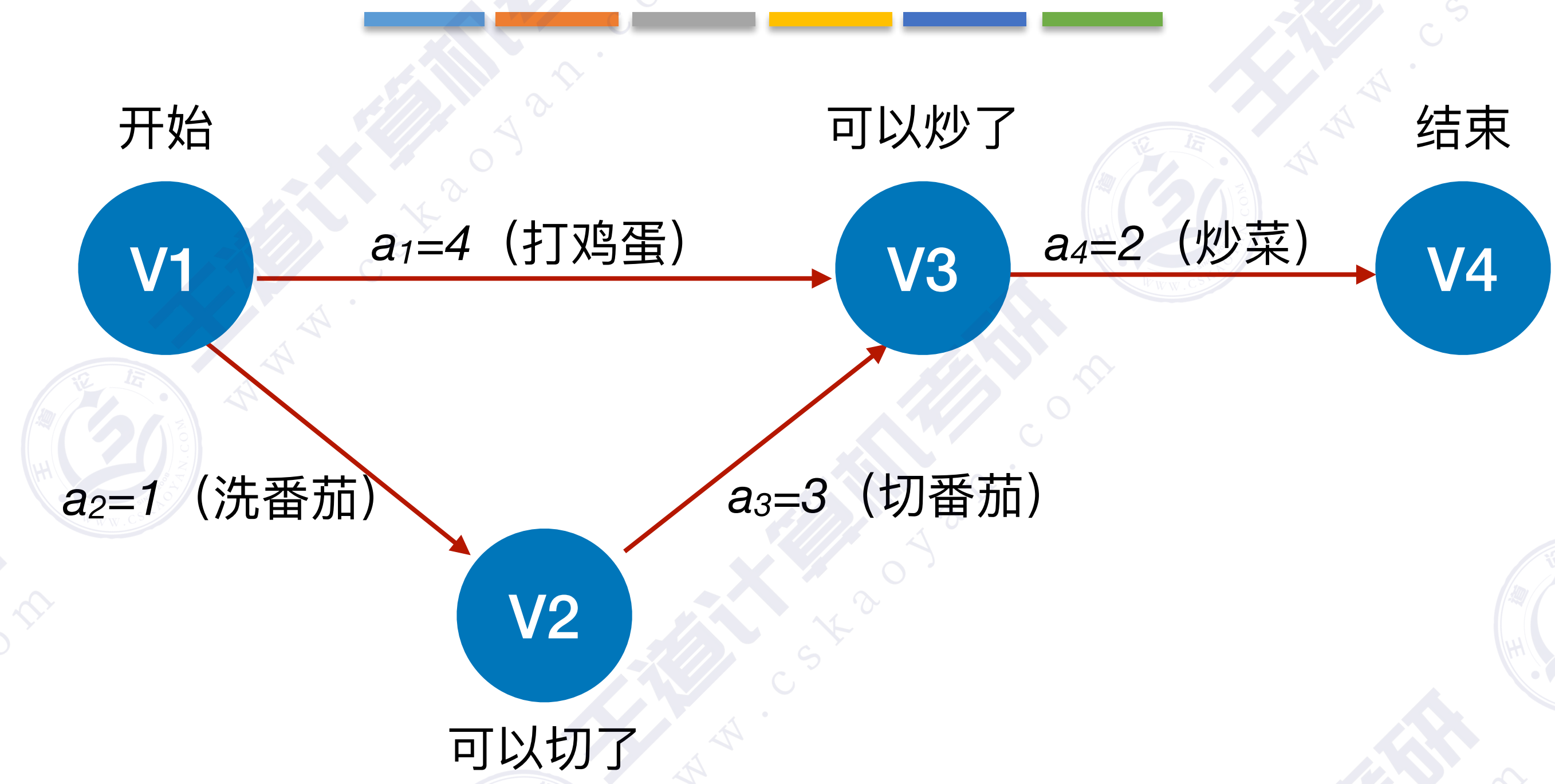


若关键活动耗时增加，则整个工程的工期将增长

缩短关键活动的时间，可以缩短整个工程的工期

当缩短到一定程度时，关键活动可能会变成非关键活动

关键活动、关键路径的特性



可能有多条关键路径，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的。

知识点回顾与重要考点

在带权有向图中，以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开销

AOE 网

相关概念

在 AOE 网中仅有一个入度为 0 的顶点，称为开始顶点（源点），它表示整个工程的开始；

也仅有一个出度为 0 的顶点，称为结束顶点（汇点），它表示整个工程的结束。

从源点到汇点的有向路径可能有多条，所有路径中，具有最大路径长度的路径称为关键路径，而把关键路径上的活动称为关键活动

关键路径

求解方法

① 求所有事件的最早发生时间 $ve()$

② 求所有事件的最迟发生时间 $vl()$

③ 求所有活动的最早发生时间 $e()$

④ 求所有活动的最迟发生时间 $l()$

⑤ 求所有活动的时间余量 $d()$ $d(i)=0$ 的活动就是关键活动，由关键活动可得关键路径

特性

若关键活动耗时增加，则整个工程的工期将增长

缩短关键活动的时间，可以缩短整个工程的工期

当缩短到一定程度时，关键活动可能会变成非关键活动

可能有多条关键路径，只提高一条关键路径上的关键活动速度并不能缩短整个工程的工期，只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的。

知识点回顾与重要考点

- ① 求所有事件的最早发生时间 $ve()$
- ② 求所有事件的最迟发生时间 $vl()$
- ③ 求所有活动的最早发生时间 $e()$
- ④ 求所有活动的最迟发生时间 $l()$
- ⑤ 求所有活动的时间余量 $d()$

$d(i)=0$ 的活动就是关键活动, 由关键活动可得关键路径

- ①按**拓扑排序**序列, 依次求各个顶点的 $ve(k)$:
 $ve(\text{源点}) = 0$
 $ve(k) = \text{Max}\{ve(j) + \text{Weight}(v_j, v_k)\}$, v_j 为 v_k 的任意前驱

- ②按**逆拓扑排序**序列, 依次求各个顶点的 $vl(k)$:
 $vl(\text{汇点}) = ve(\text{汇点})$
 $vl(k) = \text{Min}\{vl(j) - \text{Weight}(v_k, v_j)\}$, v_j 为 v_k 的任意后继

- ③若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $e(i) = ve(k)$

- ④若边 $\langle v_k, v_j \rangle$ 表示活动 a_i , 则有 $l(i) = vl(j) - \text{Weight}(v_k, v_j)$

- ⑤ $d(i) = l(i) - e(i)$